Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

C++ Programming

Complete notes for the Learn Being Forward C++ Programming syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

25	3	82	506
modules	levels	minimum days	C++ programs

How to use these notes

These notes follow the **Learn Being Forward C++ Programming syllabus** exactly: the same 25 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 9	20 days	Write procedural C++ programs with variables, streams, loops, arrays, vectors, strings and functions
2. Intermediate	10 to 18	25 days	Use pointers and references, build classes with inheritance and polymorphism, overload operators, handle exceptions, write templates and work with files
3. Advanced	19 to 25	37 days	Master the STL, modern C++ (C++11 to C++26), metaprogramming, concurrency, professional tooling, DSA and interview readiness
Total	1 to 25	82 days	About 14 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every C++ program in these notes was compiled with **GCC 13** (`g++ -std=c++20 -Wall -Wextra`) on Linux and run to produce the output shown. Examples that need C++23 are built with `-std=c++23`, and the deducing-this example with Clang 18. The few features that neither compiler implements yet (such as `std::print`, `std::generator` and C++26 additions), and the examples that need libraries that were not installed (nlohmann/json, GoogleTest), are marked in their boxes, with the output that a supporting setup prints. Values that depend on the machine, such as addresses and type sizes, are what a typical 64-bit Linux system prints.

HOW TO READ THE BOXES
Dark boxes with a gold top line are **C++ code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to C++	Beginner	1
02	Setup, first program and the build process	Beginner	1
03	Variables, data types and constants	Beginner	2
04	Operators and expressions	Beginner	2
05	Input and output streams	Beginner	1

Module	Topic	Level	Min days
06	Control flow	Beginner	4
07	Arrays and vectors	Beginner	3
08	Strings	Beginner	3
09	Functions and recursion	Beginner	3
10	Pointers and references	Intermediate	3
11	Dynamic memory management	Intermediate	2
12	Structures, enums, unions and namespaces	Intermediate	2
13	Object-oriented programming: classes and objects	Intermediate	4
14	Inheritance and polymorphism	Intermediate	5
15	Operator overloading	Intermediate	2
16	Exception handling	Intermediate	2
17	Templates and generic programming	Intermediate	3
18	File handling with streams	Intermediate	2
19	Standard Template Library (STL)	Advanced	8
20	Modern C++ (C++11, C++14, C++17)	Advanced	6
21	C++20, C++23 and C++26 features	Advanced	4
22	Advanced templates and metaprogramming	Advanced	3
23	Concurrency and multithreading	Advanced	4
24	Building, debugging, testing and best practices	Advanced	4
25	Data structures, competitive programming, projects and interviews	Advanced	8

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 9

Module 01 · Introduction to C++ 9

1.1 What is C++? 9
1.2 History of C++ 11
1.3 Why do we need C++? 13
1.4 Features of C++ 14
1.5 Where is C++ used? 17
1.6 C++ compared with other languages 18

Module 02 · Setup, first program and the build process 21

2.1 Setting up the environment 21
2.2 Structure of a C++ program 23
2.3 Hello World, line by line 25
2.4 Compiling and running 26
2.5 The build process 29
2.6 Basic syntax rules 32
2.7 Comments 34
2.8 Errors 34

Module 03 · Variables, data types and constants 39

3.1 What is a variable? 39
3.2 Creating and initializing variables 41
3.3 Scope of variables 43
3.4 Fundamental data types 44
3.5 Type inference 49
3.6 Literals 51
3.7 Constants 54
3.8 Type conversion 56
3.9 Type aliases 59

Module 04 · Operators and expressions 63

4.1 Basics 63
4.2 Arithmetic operators 64
4.3 Increment and decrement 65
4.4 Relational operators 67
4.5 Logical operators 68
4.6 Bitwise operators 70
4.7 Assignment operators 71
4.8 Other operators 72
4.9 Precedence and associativity 75
4.10 The cmath library 77

Module 05 · Input and output streams 80

5.1 Stream basics 80
5.2 Input 81
5.3 Output formatting with iomanip 85
5.4 Modern formatting 88
5.5 Faster I/O 90
5.6 C-style I/O in C++ 92

5.7 Command-line arguments 92

Module 06 · Control flow 95

6.1 Decision-making statements 95
6.2 switch statement 98
6.3 Loops 101
6.4 Nested loops 105
6.5 Jump statements 109
6.6 Classic loop problems 111

Module 07 · Arrays and vectors 118

7.1 C-style arrays 118
7.2 std::array (C++11) 122
7.3 std::vector (introduction) 124
7.4 Array algorithms 129

Module 08 · Strings 137

8.1 C-style strings 137
8.2 std::string 140
8.3 std::string methods 143
8.4 Characters 147
8.5 Conversions 148
8.6 std::string_view (C++17) 150
8.7 Classic string problems 152

Module 09 · Functions and recursion 157

9.1 Function basics 157
9.2 Parameter passing 160
9.3 Return values 162
9.4 Function features in C++ 165
9.5 Recursion 169

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 10 TO 18

Module 10 · Pointers and references 179

10.1 Pointers 179
10.2 Pointers and const 185
10.3 References 186
10.4 Function pointers 190
10.5 Value categories (introduction) 192

Module 11 · Dynamic memory management 195

11.1 Stack vs heap 195
11.2 new and delete 196
11.3 Dynamic arrays 199
11.4 Memory errors 201
11.5 RAII (introduction) 204

Module 12 · Structures, enums, unions and namespaces 208

12.1 Structures 208
12.2 Structured bindings (C++17) 214
12.3 Enumerations 215
12.4 Unions 218
12.5 Namespaces 219

Module 13 · Object-oriented programming: classes and objects 224
13.1 OOP concepts 224
13.2 Classes and objects 226
13.3 Access specifiers and encapsulation 229
13.4 Constructors 230
13.5 Destructors 234
13.6 Copying objects 237
13.7 The this pointer 239
13.8 Static members 240
13.9 Const correctness 241
13.10 Friends 243
13.11 Nested and local classes 244
13.12 Relationships between classes 245
Module 14 · Inheritance and polymorphism 250
14.1 Inheritance basics 250
14.2 Types of inheritance 252
14.3 Constructors and destructors in inheritance 254
14.4 The diamond problem 256
14.5 Compile-time polymorphism 258
14.6 Runtime polymorphism 259
14.7 Abstract classes and interfaces 265
14.8 Runtime type information (RTTI) 267
Module 15 · Operator overloading 272
15.1 Basics 272
15.2 Overloading common operators 274
15.3 Guidelines 281
Module 16 · Exception handling 285
16.1 Basics 285
16.2 Standard exceptions 288
16.3 Custom exceptions 289
16.4 Advanced exception topics 290
16.5 Alternatives to exceptions 296
Module 17 · Templates and generic programming 301
17.1 Why templates 301
17.2 Function templates 302
17.3 Class templates 307
17.4 Non-type template parameters 312
17.5 Template specialization 313
17.6 Templates in practice 316
17.7 Variadic templates (introduction) 318
Module 18 · File handling with streams 324
18.1 File stream classes 324
18.2 Text files 328
18.3 Binary files 332
18.4 Random access 334
18.5 Stream state and errors 337
18.6 std::filesystem (C++17) 339
18.7 Common formats 341

LEVEL 3 · ADVANCED LEVEL · MODULES 19 TO 25

Module 19 · Standard Template Library (STL) 348

19.1 STL overview 348
19.2 Iterators 350
19.3 Sequence containers 354
19.4 Container adaptors 362
19.5 Associative containers 365
19.6 Unordered containers 368
19.7 Utility types 373
19.8 Algorithms (<algorithm> and <numeric>)
19.9 Function objects and comparators 386
19.10 Time complexity of STL operations 389

Module 20 · Modern C++ (C++11, C++14, C++17) 393

20.1 Move semantics 393
20.2 Smart pointers 400
20.3 Lambda expressions 406
20.4 Language features 411
20.5 Vocabulary types (C++17) 419
20.6 Other library additions 423

Module 21 · C++20, C++23 and C++26 features 429

21.1 Concepts (C++20) 429
21.2 Ranges (C++20) 434
21.3 Modules (C++20) 440
21.4 Coroutines (C++20) 442
21.5 Other C++20 features 445
21.6 Other C++23 features 450
21.7 C++26 overview 454

Module 22 · Advanced templates and metaprogramming 460

22.1 Variadic templates in depth 460
22.2 Type traits (<type_traits>)
22.3 Compile-time selection 467
22.4 Compile-time computation 469
22.5 Template idioms 472
22.6 Templates and code organisation 477

Module 23 · Concurrency and multithreading 481

23.1 Basics 481
23.2 std::thread 483
23.3 Synchronisation 487
23.4 Atomics and the memory model 496
23.5 Asynchronous programming 498
23.6 C++20 synchronisation tools 501
23.7 Thread pools 504

Module 24 · Building, debugging, testing and best practices 509

24.1 Multi-file projects 509
24.2 Build systems 513
24.3 Debugging 517
24.4 Memory and correctness tools 520
24.5 Testing 526

24.6 Best practices 531
24.7 Design patterns in C++ 535
24.8 Performance 541

Module 25 · Data structures, competitive programming, projects and interviews 547

25.1 Data structures from scratch 547
25.2 Algorithms 559
25.3 Competitive programming with C++ 573
25.4 Capstone projects 576
25.5 Interview preparation 579
25.6 Next steps 584

LEVEL 1

Beginner level

Outcome: write procedural C++ programs with variables, streams, loops, arrays, vectors, strings and functions.

9 MODULES · 20 DAYS MINIMUM

Module 01 · Introduction to C++	1 day
Module 02 · Setup, first program and the build process	1 day
Module 03 · Variables, data types and constants	2 days
Module 04 · Operators and expressions	2 days
Module 05 · Input and output streams	1 day
Module 06 · Control flow	4 days
Module 07 · Arrays and vectors	3 days
Module 08 · Strings	3 days
Module 09 · Functions and recursion	3 days

MODULE 01 · BEGINNER

Introduction to C++

Minimum duration **1 day** | Level **Beginner**

This module explains what C++ is, why it was created, and where it is used, before any code is written. By the end you will know why C++ is still one of the most important languages in the world, more than forty years after its birth.

IN THIS MODULE

1.1 What is C++?
1.2 History of C++
1.3 Why do we need C++?
1.4 Features of C++
1.5 Where is C++ used?
1.6 C++ compared with other languages

1.1 What is C++?

C++ is a programming language for building software that must be **fast**, **large** and **reliable** at the same time: game engines, web browsers, databases, trading systems, operating-system components and the engines inside machine-learning libraries. It lets you write code at a high level of abstraction without paying a performance price for it.

1.1.1 Definition: a general-purpose, compiled, statically typed programming language

General-purpose

Not tied to one domain: the same language builds games, compilers, embedded firmware and financial systems.

Compiled

Source code is translated **ahead of time** by a compiler into machine code for one CPU and operating system.

Statically typed

Every variable has a type known **at compile time** (`int`, `double`, `std::string` …). Many mistakes are caught before the program ever runs.

	Static typing (C++, Java, Rust)	Dynamic typing (Python, JavaScript)
Type of a variable	fixed when declared	whatever value it currently holds
Type errors found	at compile time	at run time, when that line executes
Speed	compiler can generate optimal code	checks happen while running
Flexibility	less, but `auto` and templates help	more

1.1.2 Multi-paradigm: procedural, object-oriented, generic and functional programming

A **paradigm** is a style of organising programs. Many languages push you into one style; C++ supports several and lets you mix them in the same program, choosing the best tool for each part of the problem.

Paradigm	Idea	In C++
Procedural	a program is a sequence of steps grouped into functions	functions, loops, `if` (Modules 6 to 9)
Object-oriented	model the problem as objects that combine data and behaviour	classes, inheritance, virtual functions (Modules 13 to 15)
Generic	write an algorithm once, use it for any type	templates, the STL (Modules 17, 19, 22)
Functional	compose functions, avoid changing state	lambdas, `std::function`, ranges (Modules 20, 21)

FOUR PARADIGMS IN ONE SMALL PROGRAM

main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

// object-oriented: a class bundles data with behaviour
class Student {
public:
    Student(std::string n, int m) : name(std::move(n)), marks(m) {}
    std::string name;
    int marks;
};

// generic: one template works for any type that supports <
template <typename T>
T biggest(const std::vector<T>& v) {
    return *std::max_element(v.begin(), v.end());
}

int main() {
    // procedural: plain steps and a loop
    int total = 0;
    for (int i = 1; i <= 5; ++i) total += i;
    std::cout << "procedural  : sum 1..5 = " << total << '\n';

    std::vector<Student> cls = {{"Asha", 88}, {"Ravi", 72}, {"Meera", 95}};
    std::cout << "object      : " << cls[2].name << " has " << cls[2].marks << '\n';

    std::cout << "generic     : biggest int = " << biggest(std::vector<int>{4, 9, 2})
              << ", biggest string = " << biggest(std::vector<std::string>{"pear", "apple", "zoo"}) <<
'\n';

    // functional: pass a lambda to an algorithm
    auto toppers = std::count_if(cls.begin(), cls.end(), [](const Student& s) { return s.marks >= 85;
});
    std::cout << "functional  : " << toppers << " students scored 85 or more\n";
    return 0;
}
```

OUTPUT

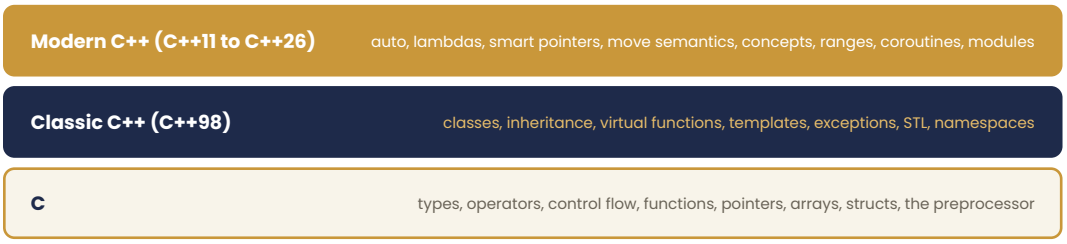
```
procedural  : sum 1..5 = 15
object      : Meera has 95
generic     : biggest int = 9, biggest string = zoo
functional  : 2 students scored 85 or more
```

NOTE

Do not worry about understanding every line yet. Each feature in this program is explained in detail in later modules; here the point is that C++ lets all four styles work together.

1.1.3 C++ as an extension of C ("C with Classes")

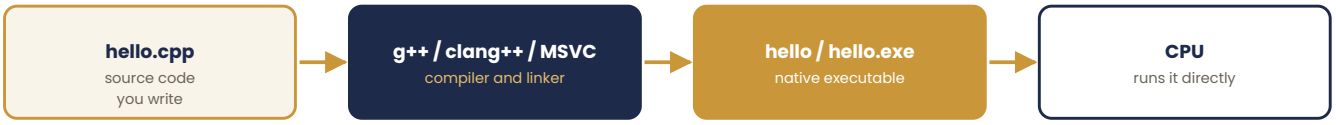
C++ started life as **"C with Classes"**: C plus a way to define your own types with data and functions together. It kept C's syntax, speed and ability to work close to the hardware. Almost every C program is also a valid C++ program (with small differences), and C++ code can call C libraries directly. Over the decades C++ grew far beyond C, adding references, templates, exceptions, a huge standard library, lambdas, smart pointers, concepts and modules.



C++ IS BUILT IN LAYERS ON TOP OF C

1.1.4 Compiled to native machine code

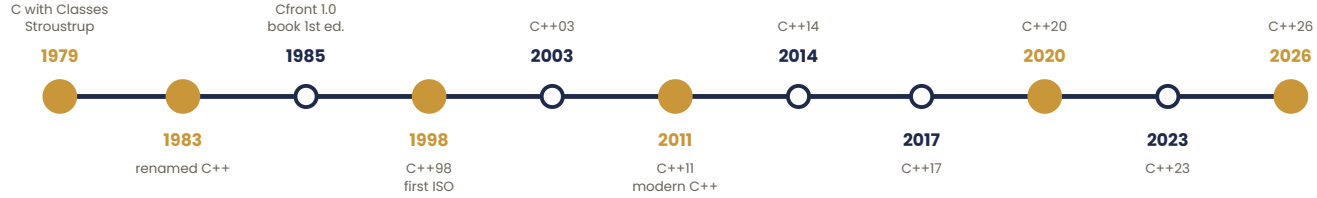
A C++ compiler (g++, clang++ or MSVC) translates the source into machine instructions for a specific processor. The result is a stand-alone executable that runs directly on the CPU, with no interpreter or virtual machine in between. This is a major reason for C++'s speed.



FROM SOURCE CODE TO A NATIVE PROGRAM

	Native compilation (C++)	Bytecode + VM (Java, C#)	Interpreter (Python)
When translated	once, before running	to bytecode first, then JIT at run time	while running
Needs to run	nothing extra	the JVM / .NET runtime	the Python interpreter
Start-up	instant	slower (VM start, warm-up)	fast
Peak speed	highest	high after JIT	much lower

1.2 History of C++



FOUR DECADES OF C++

1.2.1 Bjarne Stroustrup at Bell Labs (1979), "C with Classes"

In 1979 the Danish computer scientist **Bjarne Stroustrup** joined AT&T Bell Labs, the same lab where C and UNIX were born. For his PhD he had used **Simula**, the first object-oriented language, which made large programs easy to organise but ran far too slowly. He also knew C, which was fast but offered little help in structuring big programs. His goal was to get **Simula's organisation with C's efficiency**. The first version, "C with Classes", added classes, constructors, access control (public/private) and inheritance to C, implemented as a translator called **Cfront** that converted it into C.

1.2.2 Renamed C++ in 1983

In 1983 the language gained virtual functions, function and operator overloading, references and `const`, and was renamed **C++**. The name is a programmer's joke: `++` is C's increment operator, so C++ means "one more than C", the next step after C. In 1985 Stroustrup published *The C++ Programming Language* and Cfront 1.0 was released commercially.

1.2.3 First ISO standard: C++98

By the 1990s many companies shipped their own C++ compilers with slightly different behaviour. To make code portable, an international committee standardised the language. **ISO/IEC 14882:1998**, known as **C++98**, defined the language together with the **Standard Template Library** (STL) of containers and algorithms designed by Alexander Stepanov.

1.2.4 Standards timeline: C++03 → C++11 (modern C++) → C++14 → C++17 → C++20 → C++23 → C++26

Standard	Year	Highlights
C++98	1998	first ISO standard: classes, templates, exceptions, namespaces, STL
C++03	2003	bug-fix release
C++11	2011	the "modern C++" revolution: `auto`, range-based for, lambdas, move semantics, smart pointers, `nullptr`, threads, `constexpr`, variadic templates
C++14	2014	generic lambdas, `make_unique`, relaxed constexpr, binary literals, digit separators
C++17	2017	structured bindings, `if constexpr`, `std::optional`, `variant`, `string_view`, filesystem, parallel algorithms
C++20	2020	concepts, ranges, coroutines, modules, `std::format`, three-way comparison `<=>`, `std::span`
C++23	2023	`std::print`, `std::expected`, deducing `this`, `std::mdspan`, more ranges, `import std;`
C++26	2026	static reflection, contracts, `std::execution` (senders/receivers), pack indexing, more constexpr

> **NOTE**
> In these notes we use **C++20** as the default (`-std=c++20`), which every current major compiler supports well. Newer features are introduced in Modules 20 and 21 and are marked with the standard that added them.

1.2.5 Three-year release cycle and the ISO committee

C++ is not owned by any company. It is developed by the ISO committee **WG21**, made up of volunteers from compiler vendors (Microsoft, Google, Apple, Intel, NVIDIA), banks, game studios, universities and individuals. Since C++11 the committee ships a new standard **every three years** on a fixed schedule ("train model"): whatever features are ready board the train; unfinished ones wait for the next one. This predictable rhythm replaced the 13-year gap between C++98 and C++11.

HOW A FEATURE REACHES THE C++ STANDARD

1.3 Why do we need C++?

1.3.1 High performance with high-level abstractions ("zero-cost abstractions")

Stroustrup's guiding principle is the **zero-overhead principle**: *"What you don't use, you don't pay for. What you do use, you couldn't hand-code any better."* A `std::vector`, a range-based for loop or a templated `std::sort` compiles to machine code as efficient as a hand-written C loop, often more efficient because the compiler sees the whole type information. You write readable, high-level code and still get C-level speed.

EXPRESSIVE CODE WITH NO RUNTIME COST

main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> v = {42, 7, 19, 88, 3};
    std::sort(v.begin(), v.end());                      // high-level, yet as fast as hand-written C
    int sum = std::accumulate(v.begin(), v.end(), 0);
    for (int x : v) std::cout << x << ' ';              // range-based for loop
    std::cout << "\nsum = " << sum << '\n';
    return 0;
}
```

OUTPUT

```
3 7 19 42 88
sum = 159
```

1.3.2 Direct control over memory and hardware

Like C, C++ gives access to raw memory through pointers, lets you choose exactly where objects live (stack, heap, a specific address), control their layout and alignment, and talk to hardware registers. Unlike garbage-collected languages there are no unpredictable pauses, which is essential for games (a frame every 16 ms), audio, trading systems and real-time control.

1.3.3 Object-oriented design for large systems

Classes, encapsulation and inheritance let teams split a system of millions of lines into components with clear interfaces. Chrome, Unreal Engine, LLVM and Microsoft Office are each tens of millions of lines of

C++ maintained by hundreds of engineers.

1.3.4 Generic programming with templates

Templates let you write a function or class once and use it with any type, while the compiler generates specialised, fully optimised code for each type. A single `std::sort` sorts ints, strings or your own objects; a single `std::vector<T>` stores anything.

1.3.5 The Standard Template Library (STL)

The STL supplies ready-made, heavily tested and optimised **containers** (vector, list, map, unordered_map, set, deque …), **algorithms** (sort, find, binary_search, accumulate, more than 100 in total) and **iterators** that connect them. Problems that need pages of C code are a few lines in C++.

THE THREE PILLARS OF THE STL (MODULE 19)

1.3.6 Compatibility with C code and libraries

C++ can include C headers and call C functions directly, and with `extern "C"` it can expose functions to C. Decades of C libraries (OpenSSL, SQLite, zlib, the operating-system APIs) are immediately usable from C++.

1.3.7 Preferred language for competitive programming

On competitive-programming sites (Codeforces, CodeChef, AtCoder, ICPC) C++ is by far the most used language: it runs fast enough to pass strict time limits, and the STL provides sorting, priority queues, maps and sets in one line each. Module 25 covers competitive-programming techniques.

1.4 Features of C++

1.4.1 Object-oriented (classes, inheritance, polymorphism, encapsulation)

Encapsulation

Data and the functions that use it are bundled in a class; private members are hidden from the outside.

Abstraction

Users see a simple interface (what it does) and not the implementation (how).

Inheritance

A new class reuses and extends an existing one: a `SavingsAccount` is an `Account`.

Polymorphism

One call, many behaviours: `shape->area()` runs the right code for a circle or a square.

1.4.2 Fast and efficient

Native code, no garbage collector, inlining of small functions, compile-time computation with `constexpr` and templates, and control over memory layout make C++ one of the fastest languages available, consistently near the top of benchmarks together with C and Rust.

1.4.3 Portable

Standard C++ source compiles on Windows, Linux, macOS, Android, iOS, game consoles and microcontrollers. Three major compilers (GCC, Clang, MSVC) and many others implement the same ISO standard.

1.4.4 Rich standard library

Area	Examples (headers)
Containers	`<vector>` `<map>` `<unordered_map>` `<set>` `<deque>` `<array>`
Algorithms and numerics	`<algorithm>` `<numeric>` `<ranges>` `<random>` `<cmath>`
Strings and text	`<string>` `<string_view>` `<format>` `<regex>`
Input/output	`<iostream>` `<fstream>` `<sstream>` `<filesystem>`
Memory and utilities	`<memory>` (smart pointers) `<optional>` `<variant>` `<tuple>` `<functional>`
Concurrency and time	`<thread>` `<mutex>` `<atomic>` `<future>` `<chrono>`

1.4.5 Templates and generic programming

Templates are checked and instantiated at compile time. Since C++20, **concepts** let you state requirements on template parameters ("T must be sortable"), which gives clear error messages and self-documenting generic code.

1.4.6 Pointers, references and manual memory control

C++ keeps C's pointers and adds **references** (safe aliases for existing objects). You may allocate memory manually with `new` and `delete`, but modern C++ prefers **smart pointers** (`std::unique_ptr`, `std::shared_ptr`) that free memory automatically.

1.4.7 RAII and deterministic destruction

RAII (Resource Acquisition Is Initialisation) is C++'s most important idea. An object acquires a resource (memory, a file, a lock, a network connection) in its **constructor** and releases it in its **destructor**. Because C++ destroys local objects at a precisely known moment, at the closing brace of their scope, resources are always released, even when an error occurs. No garbage collector, no forgotten `close()`.

RAII: RESOURCES ARE RELEASED AT THE END OF THEIR SCOPE main.cpp

```cpp
#include <iostream>
#include <string>

class Resource {                        // a tiny RAII class
public:
    explicit Resource(std::string n) : name(std::move(n)) { std::cout << "acquire " << name << '\n'; }
    ~Resource() { std::cout << "release " << name << '\n'; }   // destructor
private:
    std::string name;
};

int main() {
    Resource a("file");
    {
        Resource b("lock");
        std::cout << "working with both\n";
    }                                   // b is destroyed here, automatically
    std::cout << "back in main\n";
    return 0;
}                                       // a is destroyed here
```

OUTPUT

```
acquire file
acquire lock
working with both
release lock
back in main
release file
```

1.4.8 Exception handling

Errors can be reported by **throwing** an exception, which travels up the call stack to the nearest matching `catch`. Thanks to RAII, every object on the way is destroyed properly. Module 16 covers exceptions in depth.

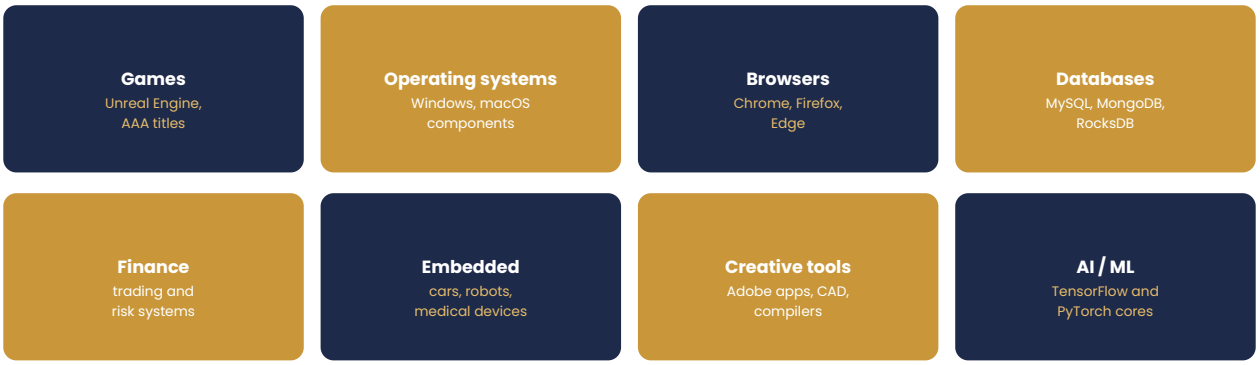
THROWING AND CATCHING AN EXCEPTION main.cpp

```cpp
#include <iostream>
#include <stdexcept>

double divide(double a, double b) {
    if (b == 0) throw std::invalid_argument("division by zero");
    return a / b;
}
int main() {
    try {
        std::cout << divide(10, 4) << '\n';
        std::cout << divide(1, 0) << '\n';          // throws
        std::cout << "never printed\n";
    } catch (const std::exception& e) {
        std::cout << "error: " << e.what() << '\n';
    }
    return 0;
}
```

OUTPUT

```
2.5
error: division by zero
```

1.5 Where is C++ used?



C++ POWERS PERFORMANCE-CRITICAL SOFTWARE IN EVERY INDUSTRY

1.5.1 Game engines and games (Unreal Engine, AAA titles)

Games must compute physics, AI and graphics for every frame within about 16 milliseconds (60 frames per second). Epic's **Unreal Engine** is written in C++ and game logic is scripted in C++ too; most big-budget ("AAA") console and PC games, and engines such as CryEngine and id Tech, are C++.

1.5.2 Operating systems and system software (parts of Windows, macOS)

Large parts of **Windows** (including much of its user interface layer and many system services), components of **macOS** and iOS (for example parts of the I/O Kit driver framework), and system tools such as compilers and virtual machines are written in C++.

1.5.3 Web browsers (Chrome, Firefox)

Google **Chrome** and the Chromium engine (also used by Edge, Opera and Brave), its V8 JavaScript engine, and much of Mozilla **Firefox** are C++. A browser is effectively an operating system for web pages and needs both speed and very large-scale structure.

1.5.4 Databases (MySQL, MongoDB)

MySQL, **MongoDB**, ClickHouse, RocksDB and ScyllaDB are written in C++, where precise control of memory and disk access gives high throughput.

1.5.5 Financial and high-frequency trading systems

Stock exchanges and trading firms use C++ where microseconds decide profit: order-matching engines, market-data handlers and pricing libraries. Predictable latency without garbage-collection pauses is the key requirement.

1.5.6 Embedded systems, robotics and automotive software

Car infotainment and driver-assistance systems (standards such as AUTOSAR use C++), drones, robots (the ROS framework), medical devices and telecom equipment use C++, often with coding rules such as MISRA C++ or AUTOSAR C++14 for safety.

1.5.7 Compilers, graphics, CAD and multimedia software (Adobe products)

The **LLVM**/Clang and GCC compilers, **Adobe** Photoshop, Illustrator and Premiere, Autodesk AutoCAD and Maya, Blender, and media tools such as FFmpeg-based players rely on C++ for heavy computation on images, video and 3D models.

1.5.8 Machine learning libraries (TensorFlow and PyTorch cores)

Python is the face of machine learning, but the heavy lifting in **TensorFlow**, **PyTorch** (its ATen/LibTorch core), ONNX Runtime and llama.cpp is done in C++ (with CUDA for GPUs). Python code calls these optimised C++ kernels.

1.6 C++ compared with other languages

1.6.1 C++ vs C

Aspect	C	C++
Paradigm	procedural	multi-paradigm
User-defined types	struct (data only)	class/struct with data, functions, access control
Memory management	malloc/free by hand	constructors/destructors, RAII, smart pointers
Generic code	macros, void *	templates and concepts (type-safe)
Error handling	return codes, errno	exceptions (plus return codes, std::expected)
Standard library	small	large: containers, algorithms, strings, threads
Strings	char arrays ending in \0	`std::string` that manages its own memory
Function overloading	no	yes
References	no	yes
Namespaces	no	yes

THE SAME JOB THAT WOULD NEED MALLOC, STRCPY, QSORT AND FREE IN C

main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    // C++: no manual sizes, no malloc/free, no strcpy
    std::vector<std::string> names = {"Ravi", "Asha", "Meera"};
    names.push_back("Kabir");                       // grows automatically
    std::sort(names.begin(), names.end());          // one line
    for (const auto& n : names) std::cout << n << " (" << n.size() << " letters)\n";
    return 0;
}                                                   // all memory released automatically
```

OUTPUT

```
Asha (4 letters)
Kabir (5 letters)
Meera (5 letters)
Ravi (4 letters)
```

1.6.2 C++ vs Java (native code vs JVM, manual vs garbage-collected memory, multiple inheritance)

Aspect	C++	Java
Execution	native machine code	bytecode on the Java Virtual Machine (JIT-compiled)
Memory	deterministic: RAII, smart pointers, or manual	garbage collector frees objects at some later time
Objects	can live on the stack or the heap; value semantics by default	always on the heap, accessed through references
Pointers	yes, with arithmetic	no (only references)
Multiple inheritance	of classes, yes	only of interfaces
Operator overloading	yes	no
Templates / generics	compile-time templates, generate code per type	generics with type erasure
Portability	recompile for each platform	"write once, run anywhere" bytecode

1.6.3 C++ vs Python (speed, typing, verbosity)

Aspect	C++	Python
Speed	often 10 to 100 times faster for CPU-heavy work	slower, but calls fast C/C++ libraries
Typing	static, checked at compile time	dynamic, checked at run time
Verbosity	more code, explicit types and braces	concise, indentation-based
Memory	controlled by you	automatic (reference counting + GC)
Learning curve	steep	gentle
Typical use	engines, systems, performance-critical code	scripting, data science, ML, automation, glue code

The two languages are partners more often than rivals: performance-critical parts are written in C++ and exposed to Python with tools such as pybind11.

1.6.4 C++ vs Rust (overview)

Rust (stable since 2015) targets the same space as C++: native speed with no garbage collector. Its **ownership and borrow checker** rules prove at compile time that a program has no use-after-free, no dangling references and no data races, whereas in C++ these remain the programmer's responsibility (helped by smart pointers, RAII, sanitizers and guidelines). C++ has a vastly larger existing codebase, ecosystem and job market, and its committee is working on safety profiles and contracts; Rust is growing fast in systems programming, and the two are increasingly used together.

	C++	Rust
Memory safety	by discipline, tools and guidelines	enforced by the compiler (outside `unsafe`)
Maturity	40+ years, huge ecosystem	~10 years, fast-growing
Learning curve	steep (large language)	steep (ownership rules)
Interoperability with C	direct	through FFI

C++ is a general-purpose, compiled, statically typed, multi-paradigm language created by Bjarne Stroustrup as "C with Classes". It combines C's speed and hardware control with high-level abstractions that cost nothing at run time, a rich standard library, templates and RAII, which is why it powers games, browsers, databases, trading systems and ML engines.

PRACTICE CHECKPOINT

Explain in your own words what **"multi-paradigm"** means and list **five well-known products built with C++**.

Sample answer. A paradigm is a style of organising a program. C++ is multi-paradigm because it supports several styles and lets you combine them: **procedural** code with plain functions and loops, **object-oriented** code with classes, inheritance and virtual functions, **generic** code with templates that work for any type, and **functional** code with lambdas and algorithms. A single program can use a class to model a student, a template function to find the best of anything, and a lambda to filter a list.

Product	Company / project	What C++ does there
Unreal Engine	Epic Games	the whole engine and game code
Google Chrome	Google	browser engine (Blink) and V8 JavaScript engine
MySQL	Oracle	the database server
Adobe Photoshop	Adobe	image processing core
TensorFlow / PyTorch	Google / Meta	tensor operations under the Python API
Microsoft Windows (parts)	Microsoft	system components and applications

MODULE 02 · BEGINNER

Setup, first program and the build process

Minimum duration **1 day** | Level **Beginner**

By the end of this module you will have a working compiler and editor, understand every line of Hello World, know how to compile with the right standard and warning flags, and see exactly what happens between your .cpp file and a running program.

IN THIS MODULE

2.1 Setting up the environment
2.2 Structure of a C++ program
2.3 Hello World, line by line
2.4 Compiling and running
2.5 The build process
2.6 Basic syntax rules
2.7 Comments
2.8 Errors

2.1 Setting up the environment

To write C++ you need two things: a **compiler** that turns source code into an executable, and an **editor or IDE** to write the code. Everything else (debugger, build tools) comes with them.

2.1.1 Compilers: GCC (g++), Clang (clang++), MSVC

Compiler	Command	Made by	Platforms	Notes
GCC	`g++`	GNU project	Linux, Windows (MinGW/MSYS2), macOS	the standard compiler on Linux; used in these notes
Clang	`clang++`	LLVM project (Apple, Google …)	all major systems	default on macOS; very clear error messages
MSVC	`cl`	Microsoft	Windows	part of Visual Studio; best Windows integration

All three implement the ISO C++ standard, so standard code compiles with any of them. Check that a compiler is installed and which version it is:

Terminal

```
$ g++ --version
g++ (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
$ clang++ --version
Ubuntu clang version 18.1.3 (1ubuntu1)
```

2.1.2 Installing on Windows (MSYS2 / Visual Studio), macOS (Xcode tools) and Linux

System	Recommended setup	Steps
Windows	Visual Studio Community (MSVC)	install, select the workload "Desktop development with C++"
Windows	MSYS2 (GCC)	install MSYS2, then in its terminal: `pacman -S mingw-w64-ucrt-x86_64-gcc`; add `C:\msys64\ucrt64\bin` to PATH
Windows	WSL (Linux inside Windows)	`wsl --install`, then follow the Linux steps
macOS	Xcode Command Line Tools (Clang)	`xcode-select --install` in Terminal; optionally `brew install gcc`
Linux (Ubuntu/Debian)	GCC	`sudo apt install build-essential gdb`
Linux (Fedora)	GCC	`sudo dnf install gcc-c++ gdb`

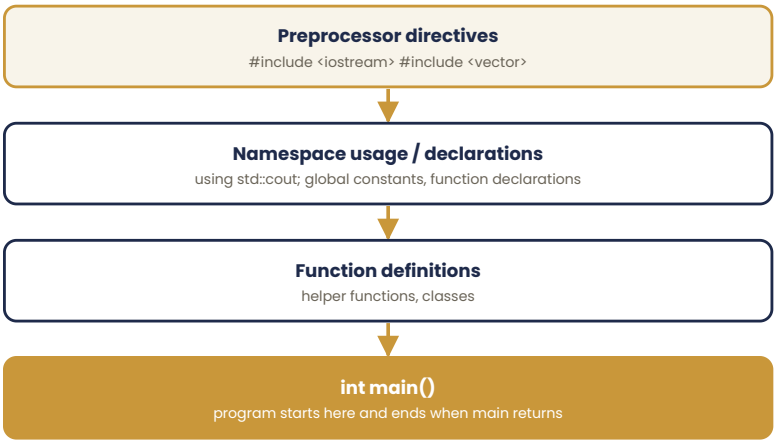
2.1.3 IDEs and editors: Visual Studio, VS Code, CLion, Code::Blocks

Tool	Type	Highlights
Visual Studio	full IDE (Windows)	excellent debugger, IntelliSense, MSVC built in, free Community edition
VS Code	editor + extensions (all OS)	install the C/C++ and CMake Tools extensions; works with g++, clang++ and MSVC
CLion	full IDE (JetBrains, all OS)	CMake-based, strong refactoring; free for students
Code::Blocks	lightweight IDE	simple, bundles MinGW on Windows; common in colleges
Qt Creator, Xcode, Vim/Neovim, Emacs	others	any editor works with a terminal and a compiler

2.1.4 Online compilers and Compiler Explorer (godbolt.org)

For quick experiments no installation is needed: online compilers such as **OnlineGDB**, **Coliru**, **Wandbox** and **cpp.sh** compile and run code in the browser. **Compiler Explorer** (godbolt.org, created by Matt Godbolt) shows the **assembly code** generated for your C++ by dozens of compilers and versions side by side, which is a wonderful way to see what your code really becomes and to test which compiler supports a feature.

2.2 Structure of a C++ program



THE USUAL LAYOUT OF A SINGLE-FILE C++ PROGRAM

ANATOMY OF A PROGRAM

main.cpp

```cpp
// greet.cpp: the parts of a C++ program
#include <iostream>          // 1. preprocessor directive: bring in the I/O library
#include <string>

const double PI = 3.14159;   // 2. global declarations

double circle_area(double r) {          // 3. function definition
    return PI * r * r;
}

int main() {                 // 4. the entry point
    std::string name = "Asha";           // statements end with ;
    std::cout << "Hello, " << name << "!\n";
    {                                    // a nested block
        double r = 2.0;
        std::cout << "Area of a circle of radius " << r << " is " << circle_area(r) << '\n';
    }
    return 0;                // 0 tells the operating system: success
}
```

OUTPUT

```
Hello, Asha!
Area of a circle of radius 2 is 12.5664
```

2.2.1 Preprocessor directives (#include)

Lines starting with `#` are handled by the **preprocessor** before compilation. `#include <iostream>` pastes in the declarations of the standard input/output library. Standard C++ headers have **no .h** extension (`<iostream>`, `<vector>`, `<string>`); C library headers are available with a **c** prefix (`<cmath>`, `<cstdio>`, `<cstdlib>`). Your own headers use quotes: `#include "geometry.h"`.

2.2.2 Namespaces and using namespace std; (and why to avoid it in headers)

A **namespace** groups names to avoid clashes. Everything in the standard library lives in the namespace `std`, so its full names are `std::cout`, `std::string`, `std::vector`. The `::` is the **scope resolution operator**. There are three ways to use these names:

THREE WAYS TO REFER TO STANDARD NAMES

main.cpp

```cpp
#include <iostream>
#include <string>

namespace school {                  // our own namespace
    int count = 42;
}

int count = 7;                      // a global with the same name: no clash

int main() {
    std::cout << "1. full name: std::cout\n";

    using std::string;              // 2. using-declaration: one name
    string s = "two";
    std::cout << "2. using-declaration for " << s << '\n';

    using namespace std;            // 3. using-directive: all of std (avoid in headers!)
    cout << "3. using namespace std\n";
    cout << "school::count = " << school::count << ", ::count = " << ::count << '\n';
    return 0;
}
```

OUTPUT

```
1. full name: std::cout
2. using-declaration for two
3. using namespace std
school::count = 42, ::count = 7
```

WATCH OUT

`using namespace std;` pulls hundreds of names (count, max, min, sort, distance, list …) into scope, which can silently clash with your own names. It is acceptable in small programs and exercises inside a .cpp file, but **never put it in a header file**, because every file that includes the header inherits the clash. These notes write `std::` explicitly, as professional code does.

2.2.3 The main() function

Every program has exactly one `main`. The operating system calls it to start the program, and its return value is the program's **exit status**: 0 (or `EXIT_SUCCESS`) means success, any other value signals an error to the shell or calling program. Two forms are standard: `int main()` and `int main(int argc, char* argv[])` for command-line arguments (Module 5.7). In `main` only, omitting `return` means `return 0;`.

2.2.4 Statements and blocks

A **statement** is one instruction and normally ends with a semicolon. A **block** (compound statement) is a sequence of statements in braces `{ }`; it can appear wherever one statement is allowed, and variables declared inside a block exist only until its closing brace.

2.3 Hello World, line by line

HELLO.CPP

hello.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

OUTPUT

```
Hello, World!
```

2.3.1 #include <iostream>

Includes the standard header that declares the stream objects `std::cout` (console output), `std::cin` (console input), `std::cerr` and `std::clog` (error and log output). Without it the compiler does not know what `std::cout` is.

2.3.2 int main()

Defines the function `main`, which takes no parameters and returns an `int`. The body between the braces runs when the program starts.

2.3.3 std::cout, << and std::endl vs '\n'

`std::cout` is the standard output stream, normally the terminal. The **insertion operator** `<<` sends the value on its right into the stream on its left and returns the stream again, which is why calls can be chained: `std::cout << a << b << c;`. `std::endl` writes a newline **and flushes** the stream's buffer to the device; `'\n'` writes only the newline. Flushing is slow when repeated thousands of times, so prefer `'\n'` and use `std::endl` (or `std::flush`) only when the output must appear immediately.

DATA FLOWS INTO THE OUTPUT STREAM AND IS FLUSHED TO THE SCREEN

2.3.4 return 0;

Ends `main` and returns 0 to the operating system, meaning "success". In a terminal you can see it with `echo $?` (Linux/macOS) or `echo %errorlevel%` (Windows).

RUN AS: ./PROG; ECHO "EXIT STATUS: $?"

main.cpp

```cpp
#include <iostream>
#include <cstdlib>

int main() {
    std::cout << "Something went wrong, exiting with status 3\n";
    return 3;                          // non-zero: failure
}
```

OUTPUT

```
Something went wrong, exiting with status 3
exit status: 3
```

2.4 Compiling and running

2.4.1 g++ hello.cpp -o hello

Terminal

```
$ g++ hello.cpp -o hello        # compile and link; -o names the output
$ ./hello                       # run it (on Windows: hello.exe)
Hello, World!
$ g++ hello.cpp                 # without -o the output is a.out (a.exe on Windows)
$ ./a.out
Hello, World!
```

Compiler	Compile	Run
GCC / MinGW	`g++ hello.cpp -o hello`	`./hello` or `hello.exe`
Clang	`clang++ hello.cpp -o hello`	`./hello`
MSVC (Developer Command Prompt)	`cl /EHsc hello.cpp`	`hello.exe`

2.4.2 Choosing a standard: -std=c++17, -std=c++20, -std=c++23

Each compiler version has a default standard (GCC 13 defaults to C++17 with GNU extensions). Always choose the standard explicitly so the code behaves the same everywhere. The predefined macro `__cplusplus` tells you which standard is active.

G++ -STD=C++17 VERSION.CPP

main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "__cplusplus = " << __cplusplus << '\n';
#if __cplusplus > 202002L
    std::cout << "compiled as C++23 (or newer)\n";
#elif __cplusplus >= 202002L
    std::cout << "compiled as C++20\n";
#elif __cplusplus >= 201703L
    std::cout << "compiled as C++17\n";
#endif
    return 0;
}
```

OUTPUT

```
__cplusplus = 201703
compiled as C++17
```

G++ -STD=C++20 VERSION.CPP

main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "__cplusplus = " << __cplusplus << '\n';
#if __cplusplus > 202002L
    std::cout << "compiled as C++23 (or newer)\n";
#elif __cplusplus >= 202002L
    std::cout << "compiled as C++20\n";
#elif __cplusplus >= 201703L
    std::cout << "compiled as C++17\n";
#endif
    return 0;
}
```

OUTPUT

```
__cplusplus = 202002
compiled as C++20
```

G++ -STD=C++23 VERSION.CPP

main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "__cplusplus = " << __cplusplus << '\n';
#if __cplusplus > 202002L
    std::cout << "compiled as C++23 (or newer)\n";
#elif __cplusplus >= 202002L
    std::cout << "compiled as C++20\n";
#elif __cplusplus >= 201703L
    std::cout << "compiled as C++17\n";
#endif
    return 0;
}
```

OUTPUT

```
__cplusplus = 202100
compiled as C++23 (or newer)
```

Code that uses a newer feature fails to compile with an older standard. Designated initializers for structures are a C++20 feature:

A C++20 FEATURE BUILT WITH -STD=C++17 -PEDANTIC-ERRORS

main.cpp

```cpp
#include <iostream>

struct Point { int x; int y; };

int main() {
    Point p{.x = 3, .y = 4};          // designated initializers: C++20
    std::cout << p.x + p.y << '\n';
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:6:13: error: C++ designated initializers only available with ‘-std=c++20’ or ‘-std=gnu++20’
[-Wc++20-extensions]
    6 |     Point p{.x = 3, .y = 4};          // designated initializers: C++20
      |             ^
main.cpp:6:21: error: C++ designated initializers only available with ‘-std=c++20’ or ‘-std=gnu++20’
[-Wc++20-extensions]
    6 |     Point p{.x = 3, .y = 4};          // designated initializers: C++20
      |                     ^
```

Flag	Standard	`__cplusplus`
`-std=c++11`	C++11	201103
`-std=c++14`	C++14	201402
`-std=c++17`	C++17	201703
`-std=c++20`	C++20	202002
`-std=c++23` (GCC 11+: also `c++2b`)	C++23	202302 (GCC 13 reports 202100 for its partial support)
`-std=c++26` (GCC 14+, `c++2c`)	C++26 (draft)	> 202302
`/std:c++20`, `/std:c++latest`	MSVC syntax	needs `/Zc:__cplusplus` to report correctly

2.4.3 Warning and debug flags: -Wall, -Wextra, -g, -O2

Flag	Meaning
`-Wall`	enable the most useful warnings
`-Wextra`	even more warnings
`-Wpedantic` / `-pedantic-errors`	warn about / reject non-standard extensions
`-Werror`	treat warnings as errors
`-g`	include debug information for gdb and IDE debuggers
`-O0` / `-O2` / `-O3`	no / standard / aggressive optimisation
`-fsanitize=address,undefined`	detect memory errors and undefined behaviour at run time

A good everyday command line is `g++ -std=c++20 -Wall -Wextra -g prog.cpp -o prog` while developing, and `-O2` instead of `-g` for the final build. Warnings point at real bugs:

WHAT -WALL -WEXTRA REPORTS

main.cpp

```
#include <iostream>

int main() {
    int count;                          // never initialized
    unsigned int limit = 3;
    for (int i = 0; i < limit; ++i)     // signed/unsigned comparison
        std::cout << i << ' ';
    if (count = 5)                      // = instead of ==
        std::cout << "five\n";
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:6:23: warning: comparison of integer expressions of different signedness: ‘int’ and ‘unsigned
int’ [-Wsign-compare]
    6 |     for (int i = 0; i < limit; ++i)     // signed/unsigned comparison
      |                     ~~^~~~~~~
main.cpp:8:15: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    8 |     if (count = 5)                      // = instead of ==
      |         ~~~~~~^~~
$ ./prog
0 1 2 five
```

2.5 The build process

"Compiling" is really four steps. The `g++` command runs them all, but you can stop after any step to look at the intermediate result.

PREPROCESS → COMPILE → ASSEMBLE → LINK

2.5.1 Preprocessing

The preprocessor removes comments, expands macros and replaces every `#include` with the full text of the header. `g++ -E` stops here. Our six-line program grows to **over 36,000 lines**, nearly all of them from `<iostream>` and the headers it includes; our own code is at the very end:

Terminal

```
$ g++ -E hello.cpp | wc -l
36588
$ g++ -E hello.cpp | tail -6
# 3 "hello.cpp"
int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

2.5.2 Compilation

The compiler proper checks syntax and types, optimises, and translates the preprocessed code into **assembly language** for the target CPU. `g++ -S` stops here and writes a `.s` file. Notice the odd names such as `_ZSt4cout`: C++ **mangles** names to encode namespaces and parameter types, so that overloaded functions get distinct symbols.

Assembly for Hello World

```
$ g++ -S hello.cpp             # creates hello.s (directives removed below)
main:
        endbr64
        pushq   %rbp
        movq    %rsp, %rbp
        leaq    .LC0(%rip), %rax
        movq    %rax, %rsi
        leaq    _ZSt4cout(%rip), %rax
        movq    %rax, %rdi
        call    _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@PLT
        movq    _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_@GOTPCREL(%rip), %rdx
        movq    %rdx, %rsi
        movq    %rax, %rdi
        call    _ZNSolsEPFRSoS_E@PLT
        movl    $0, %eax
        popq    %rbp
        ret
$ echo _ZSt4cout _Z3addii _Z3adddd | c++filt      # demangle
std::cout add(int, int) add(double, double)
```

2.5.3 Assembly

The assembler converts assembly into **machine code** stored in an **object file** (`.o`, or `.obj` with MSVC). `g++ -c` stops here. An object file contains our compiled `main` but still has **unresolved references** (U) to library code such as `std::cout`:

The object file: main is defined (T), library symbols are undefined (U)

```
$ g++ -c hello.cpp             # creates hello.o
$ nm -C hello.o                # -C demangles the names
                 U std::ostream::operator<<(std::ostream& (*)(std::ostream&))
                 U std::ios_base_library_init()
                 U std::cout
                 U std::basic_ostream<char, std::char_traits<char> >& std::endl<char,
std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&)
                 U std::basic_ostream<char, std::char_traits<char> >& std::operator<<
<std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, char const*)
0000000000000000 T main
```

2.5.4 Linking

The **linker** combines object files with libraries, resolves every undefined symbol to its definition, adds start-up code, and writes the executable. C++ programs link against the C++ standard library (`libstdc++` for GCC, `libc++` for Clang on macOS), which is why you should link with `g++`, not `gcc`.

Linking produces the executable

```
$ g++ hello.o -o hello
$ ls -l hello.cpp hello.o hello | awk '{print $5, $9}'
16504 hello
97 hello.cpp
2144 hello.o
$ ldd hello                      # shared libraries loaded at run time
        linux-vdso.so.1 (0x00007fd40620e000)
        libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6 (0x00007fd405e00000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007fd405a00000)
        libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x00007fd40610a000)
        /lib64/ld-linux-x86-64.so.2 (0x00007fd406210000)
        libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x00007fd4060dc000)
```

Step	Input → output	g++ option to stop here	Errors found
Preprocess	.cpp → .ii	`-E`	missing header, bad macro
Compile	.ii → .s	`-S`	syntax and type errors
Assemble	.s → .o	`-c`	(rare)
Link	.o + libraries → executable	(default)	undefined reference, multiple definition

2.5.5 Header files vs source files

Bigger programs are split into files. A **header** (`.h`, `.hpp`) holds **declarations** that other files need: function prototypes, class definitions, constants. A **source file** (`.cpp`) holds the **definitions** (function bodies). Each .cpp is compiled separately into a .o, and the linker joins them.

GEOMETRY.H C++

```cpp
// geometry.h : the interface
#pragma once                    // include this header only once
double area(double r);
```

GEOMETRY.CPP C++

```cpp
// geometry.cpp : the implementation
#include "geometry.h"
double area(double r) { return 3.14159 * r * r; }
```

MAIN.CPP (BUILT WITH: G++ MAIN.CPP GEOMETRY.CPP -O APP) main.cpp

```cpp
#include <iostream>
#include "geometry.h"

int main() {
    std::cout << "area(3) = " << area(3.0) << '\n';
    return 0;
}
```

OUTPUT

```
area(3) = 28.2743
```

A linker error and the fix

```
$ g++ main.cpp -o app           # forgot geometry.cpp
/usr/bin/ld: /tmp/cckQZswA.o: in function `main':
main.cpp:(.text+0x15): undefined reference to `area(double)'
collect2: error: ld returned 1 exit status
$ g++ -c geometry.cpp           # separate compilation
$ g++ -c main.cpp
$ g++ main.o geometry.o -o app
$ ./app
area(3) = 28.2743
```

Module 24 covers multi-file projects, libraries, CMake and build automation in depth.

2.6 Basic syntax rules

2.6.1 Case sensitivity and semicolons

- C++ is **case-sensitive**: `total`, `Total` and `TOTAL` are three different names; `Main` is not `main`, and `Int` is not a type.
- Every statement ends with a **semicolon**. Class and struct definitions also end with `};` (a very common beginner error). Function bodies and blocks do not need one after the closing brace.
- Whitespace (spaces, tabs, newlines) separates tokens but is otherwise ignored, so indent for readability.

2.6.2 Tokens, keywords and identifiers

The compiler reads source code as a sequence of **tokens**, the smallest meaningful units:

INT TOTAL = PRICE * 3; IS SEVEN TOKENS

Token kind	Examples
Keywords (reserved words)	`int` `class` `if` `return` `auto` `constexpr` `nullptr` … about 90 in C++20
Identifiers (names you choose)	`total`, `studentCount`, `_buffer`
Literals (fixed values)	`42` `3.14` `'A'` `"text"` `true` `nullptr`
Operators	`+ - * / = == << :: ->`
Punctuators	`; , { } ( ) [ ]`

Identifier rules: letters, digits and underscores only; must not start with a digit; must not be a keyword; case-sensitive. Names beginning with an underscore followed by a capital letter, or containing a double underscore, are reserved for the compiler and library: do not use them.

INVALID IDENTIFIERS main.cpp

```cpp
#include <iostream>

int main() {
    int 2fast = 1;          // starts with a digit
    int class = 5;          // keyword
    int total-marks = 90;   // hyphen is the minus operator
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:4:9: error: expected unqualified-id before numeric constant
    4 |     int 2fast = 1;          // starts with a digit
      |         ^~~~
main.cpp:5:5: error: expected primary-expression before ‘int’
    5 |     int class = 5;          // keyword
      |     ^~~
main.cpp:6:14: error: expected initializer before ‘-’ token
    6 |     int total-marks = 90;   // hyphen is the minus operator
      |              ^
```

2.6.3 Naming conventions

Style	Example	Commonly used for
snake_case	`student_count`	the standard library, many projects (variables, functions)
camelCase	`studentCount`	variables and functions in many codebases
PascalCase	`StudentRecord`	class and type names
UPPER_CASE	`MAX_SIZE`	macros (and sometimes constants)
m_ or trailing _	`m_name`, `name_`	private data members

Pick one convention per project and follow it. Above all, choose **descriptive** names: `average_marks` is far better than `a` or `x1`.

2.7 Comments

2.7.1 Single-line // and multi-line /* */

BOTH COMMENT STYLES

main.cpp

```
#include <iostream>

/* Multi-line comment:
   computes the simple interest for a loan.
   Comments are removed by the preprocessor. */
int main() {
    double principal = 50000;   // amount in rupees
    double rate = 7.5;          // percent per year
    int years = 3;
    double interest = principal * rate * years / 100;   // SI = P * R * T / 100
    std::cout << "Simple interest = " << interest << '\n';
    // std::cout << "this line is disabled\n";
    return 0;
}
```

OUTPUT

```
Simple interest = 11250
```

Good comments explain **why**, not what: the code already says what it does. `/* */` comments do not nest, so to disable a large block that contains comments use `#if 0 ... #endif`.

2.7.2 Documentation comments (Doxygen)

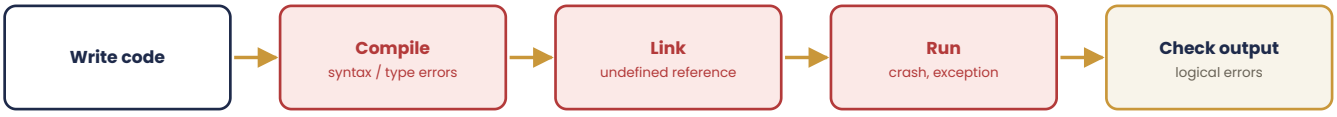
Doxygen is the standard tool for generating HTML or PDF documentation from specially formatted comments, `/** ... */` or `///`, placed before declarations. IDEs also show these comments as tooltips.

DOXYGEN-STYLE DOCUMENTATION

C++

```
/**
 * @brief  Computes the area of a rectangle.
 * @param  width   width in metres (must be >= 0)
 * @param  height  height in metres (must be >= 0)
 * @return the area in square metres
 */
double rectangle_area(double width, double height);

/// Number of students allowed in one batch.
constexpr int MAX_BATCH = 60;
```

2.8 Errors



WHERE EACH KIND OF ERROR APPEARS

2.8.1 Compile-time errors

Violations of the language rules: missing semicolons, undeclared names, type mismatches. The compiler names the file, line and column. Fix the **first** error first; later ones are often consequences of it.

A MISSING SEMICOLON AND A MISSPELT NAME

main.cpp

```
#include <iostream>

int main() {
    int marks = 90
    std::cout << "Marks: " << mark << '\n';
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:5:5: error: expected ‘,’ or ‘;’ before ‘std’
    5 |     std::cout << "Marks: " << mark << '\n';
      |     ^~~
main.cpp:4:9: warning: unused variable ‘marks’ [-Wunused-variable]
    4 |     int marks = 90
      |         ^~~~~
```

2.8.2 Linker errors

The code compiles but the linker cannot find a definition: a function was declared and called but never defined, a source file was not included in the build, or a library was not linked. They mention `undefined reference` (GCC/Clang) or `unresolved external symbol` (MSVC).

A FUNCTION THAT IS DECLARED BUT NOT DEFINED

main.cpp

```
#include <iostream>

int square(int x);                // declared, never defined

int main() {
    std::cout << square(5) << '\n';
    return 0;
}
```

OUTPUT

```
/usr/bin/ld: /tmp/ccDZVHgm.o: in function `main':
main.cpp:6:(.text+0xe): undefined reference to `square(int)'
collect2: error: ld returned 1 exit status
```

2.8.3 Runtime errors

The program builds but fails while running: dividing an integer by zero, accessing memory outside an array, running out of memory, or an uncaught exception. Some crash immediately; others silently corrupt data.

AN UNCAUGHT EXCEPTION TERMINATES THE PROGRAM main.cpp

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {10, 20, 30};
    std::cout << "v.at(1) = " << v.at(1) << '\n';
    std::cout << "v.at(5) = " << v.at(5) << '\n';    // out of range: throws
    return 0;
}
```

OUTPUT (crash)

```
v.at(1) = 20
terminate called after throwing an instance of 'std::out_of_range'
  what():  vector::_M_range_check: __n (which is 5) >= this->size() (which is 3)
Aborted (core dumped)
```

2.8.4 Logical errors

The program runs without complaint but gives the **wrong answer**, because the logic is wrong. Only testing with known inputs reveals them.

A LOGICAL ERROR: INTEGER DIVISION main.cpp

```cpp
#include <iostream>

int main() {
    int a = 7, b = 8;
    double average = (a + b) / 2;      // integer division: 15 / 2 = 7
    std::cout << "average = " << average << " (expected 7.5)\n";
    double fixed = (a + b) / 2.0;      // fix: make one operand a double
    std::cout << "fixed   = " << fixed << '\n';
    return 0;
}
```

OUTPUT

```
average = 7 (expected 7.5)
fixed   = 7.5
```

2.8.5 Reading long compiler error messages

Template-heavy C++ code can produce long error messages. Do not panic; read them systematically:

1. Find the **first** line containing `error:`. It gives your file and line.
2. Read what it says in plain words ("no matching function for call to …").
3. Lines starting with `note:` list the candidates the compiler considered and **why each failed** ("no known conversion for argument 1 from int to string").
4. Ignore the deep paths into library headers (`/usr/include/c++/13/bits/…`); the bug is almost always in your line.
5. Clang often gives shorter messages; trying another compiler on godbolt.org can help.

ONE MISTAKE, EIGHTEEN LINES OF MESSAGES

main.cpp

```
#include <string>
#include <vector>

int main() {
    std::vector<std::string> names = {"Asha", "Ravi"};
    names.push_back(42);              // an int is not a string
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:6:20: error: no matching function for call to ‘std::vector<std::__cxx11::basic_string<char>
>::push_back(int)’
    6 |     names.push_back(42);              // an int is not a string
      |     ~~~~~~~~~~~~~~~~^~~~
In file included from /usr/include/c++/13/vector:66,
                 from main.cpp:2:
/usr/include/c++/13/bits/stl_vector.h:1281:7: note: candidate: ‘constexpr void std::vector<_Tp,
_Alloc>::push_back(const value_type&) [with _Tp = std::__cxx11::basic_string<char>; _Alloc =
std::allocator<std::__cxx11::basic_string<char> >; value_type = std::__cxx11::basic_string<char>]’
 1281 |       push_back(const value_type& __x)
      |       ^~~~~~~~
/usr/include/c++/13/bits/stl_vector.h:1281:35: note:   no known conversion for argument 1 from ‘int’
to ‘const std::vector<std::__cxx11::basic_string<char> >::value_type&’ {aka ‘const
std::__cxx11::basic_string<char>&’}
 1281 |       push_back(const value_type& __x)
      |                 ~~~~~~~~~~~~~~~~~~^~~
/usr/include/c++/13/bits/stl_vector.h:1298:7: note: candidate: ‘constexpr void std::vector<_Tp,
_Alloc>::push_back(value_type&&) [with _Tp = std::__cxx11::basic_string<char>; _Alloc =
std::allocator<std::__cxx11::basic_string<char> >; value_type = std::__cxx11::basic_string<char>]’
 1298 |       push_back(value_type&& __x)
      |       ^~~~~~~~
/usr/include/c++/13/bits/stl_vector.h:1298:30: note:   no known conversion for argument 1 from ‘int’
to ‘std::vector<std::__cxx11::basic_string<char> >::value_type&&’ {aka
‘std::__cxx11::basic_string<char>&&’}
 1298 |       push_back(value_type&& __x)
      |                 ~~~~~~~~~~~~~^~~
```

The first line says it all: there is **no matching function** `push_back(int)` for a vector of strings. The two `note: candidate` entries are the two versions of push_back, each taking a string, and the "no known conversion from int" notes explain why neither fits.

Install a compiler (g++, clang++ or MSVC) and an editor, choose the standard with -std=c++20, and always compile with -Wall -Wextra. A build is preprocess → compile → assemble → link; errors from each step look different. Write std:: explicitly, prefer '\n' to std::endl, and read error messages from the first error line.

PRACTICE CHECKPOINT

Compile and run a program that prints your **name and course** from both the **terminal and an IDE**, once with **C++17** and once with **C++20**.

TERMINAL: G++ -STD=C++17 -WALL -WEXTRA ME.CPP -O ME && ./ME

me.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    const std::string name = "Lohit Kumar";
    const std::string course = "C++ Programming, Learn Being Forward";
    std::cout << "Name   : " << name << '\n';
    std::cout << "Course : " << course << '\n';
    std::cout << "Built with __cplusplus = " << __cplusplus;
    if (__cplusplus >= 202002L) std::cout << " (C++20)\n";
    else std::cout << " (C++17)\n";
    return 0;
}
```

OUTPUT

```
Name   : Lohit Kumar
Course : C++ Programming, Learn Being Forward
Built with __cplusplus = 201703 (C++17)
```

TERMINAL: G++ -STD=C++20 -WALL -WEXTRA ME.CPP -O ME && ./ME

me.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    const std::string name = "Lohit Kumar";
    const std::string course = "C++ Programming, Learn Being Forward";
    std::cout << "Name   : " << name << '\n';
    std::cout << "Course : " << course << '\n';
    std::cout << "Built with __cplusplus = " << __cplusplus;
    if (__cplusplus >= 202002L) std::cout << " (C++20)\n";
    else std::cout << " (C++17)\n";
    return 0;
}
```

OUTPUT

```
Name   : Lohit Kumar
Course : C++ Programming, Learn Being Forward
Built with __cplusplus = 202002 (C++20)
```

In an IDE: in VS Code, set `"-std=c++20"` in the `args` of `.vscode/tasks.json` (or `set(CMAKE_CXX_STANDARD 20)` in CMakeLists.txt); in Visual Studio, open Project Properties → C/C++ → Language → C++ Language Standard and pick ISO C++17 or C++20; in CLion, change `CMAKE_CXX_STANDARD`; in Code::Blocks, Settings → Compiler → tick the matching "-std" option. Build and run, then switch the standard and confirm that the printed value changes.

MODULE 03 · BEGINNER

Variables, data types and constants

Minimum duration **2 days** | Level **Beginner**

A variable is a named memory location with a fixed type. This module covers how to create and initialise variables, where they are visible, every fundamental type and its size and range, type inference with auto, literals, constants, conversions between types and type aliases.

IN THIS MODULE

3.1 What is a variable?
3.2 Creating and initializing variables
3.3 Scope of variables
3.4 Fundamental data types
3.5 Type inference
3.6 Literals
3.7 Constants
3.8 Type conversion
3.9 Type aliases

3.1 What is a variable?

3.1.1 Name, type, value and address

A **variable** is a named piece of memory that holds a value. Every variable has four properties:

Name
the identifier you use in code: `age`

Type
what kind of value it holds and how many bytes it uses: `int`, 4 bytes

Value
the data currently stored: `21`

Address
where it lives in memory: `&age`, e.g. 0x7ffd…4c

&age = 0x…4c (address of the first byte)

A VARIABLE OCCUPIES CONSECUTIVE BYTES; ITS ADDRESS IS THAT OF ITS FIRST BYTE

NAME, TYPE, VALUE AND ADDRESS main.cpp

```
#include <iostream>

int main() {
    int age = 21;
    double height = 1.68;
    char grade = 'A';
    std::cout << "age    = " << age << ", " << sizeof(age) << " bytes, at " << &age << '\n';
    std::cout << "height = " << height << ", " << sizeof(height) << " bytes, at " << &height << '\n';
    std::cout << "grade  = " << grade << ", " << sizeof(grade) << " byte,  at " << static_cast<void*>
(&grade) << '\n';
    age = age + 1;                        // the value can change; type and address cannot
    std::cout << "next year age = " << age << ", still at " << &age << '\n';
    return 0;
}
```

OUTPUT (addresses differ on every run)

```
age    = 21, 4 bytes, at 0x7ffe33d565ec
height = 1.68, 8 bytes, at 0x7ffe33d565f0
grade  = A, 1 byte,  at 0x7ffe33d565eb
next year age = 22, still at 0x7ffe33d565ec
```

NOTE

Printing `&grade` directly would treat it as a C string and print garbage characters, because `char*` has a special overload of `<<`. Casting to `void*` prints the address.

3.1.2 Static typing in C++

In C++ the type of a variable is fixed when it is declared and checked at compile time. The compiler uses the type to decide how many bytes to reserve, how to interpret the bits, and which operations are allowed. Trying to store an incompatible value is an error before the program ever runs.

THE COMPILER ENFORCES TYPES main.cpp

```
#include <string>

int main() {
    int count = 10;
    count = "ten";              // a string literal is not an int
    std::string name = 42;      // an int is not a string
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:5:13: error: invalid conversion from ‘const char*’ to ‘int’ [-fpermissive]
    5 |     count = "ten";              // a string literal is not an int
      |             ^~~~~
      |             |
      |             const char*
main.cpp:6:24: error: conversion from ‘int’ to non-scalar type ‘std::string’ {aka
‘std::__cxx11::basic_string<char>’} requested
    6 |     std::string name = 42;      // an int is not a string
      |                        ^~
```

3.2 Creating and initializing variables

3.2.1 Declaration and definition

A **declaration** introduces a name and its type to the compiler. A **definition** is a declaration that also creates the object (reserves memory). For ordinary local variables every declaration is a definition. The difference matters for globals shared between files: `extern int total;` only declares (the variable is defined in another file), while `int total = 0;` defines it. A program may contain many declarations of a name but exactly **one definition** (the One Definition Rule).

SNIPPET C++

```
int count;                 // definition (and declaration)
extern int shared_total;   // declaration only: defined in another .cpp file
int square(int x);         // function declaration (prototype)
int square(int x) { return x * x; }   // function definition
```

3.2.2 Copy initialization int a = 5;

3.2.3 Direct initialization int a(5);

3.2.4 Uniform (brace) initialization int a{5}; and narrowing prevention

C++ has several initialisation syntaxes for historical reasons:

Syntax	Name	Notes
`int a = 5;`	copy initialization	familiar from C; allows narrowing
`int a(5);`	direct initialization	looks like a constructor call; allows narrowing
`int a{5};`	brace (list / uniform) initialization, C++11	works for every type; **rejects narrowing**
`int a = {5};`	copy-list initialization	also rejects narrowing
`int a{};`	value initialization	zero for built-in types

A **narrowing conversion** loses information: double to int (drops the fraction), a large int to char, int to unsigned. Copy and direct initialisation silently accept it; brace initialisation makes it a compile-time error, which catches real bugs. Prefer braces for new code.

NOTE

The standard says narrowing inside braces is ill-formed. GCC turns it into a hard error for constants (`char c{300};`) but, for compatibility with old code, only **warns** when the value comes from a variable (`int r{price};`). Compile with `-pedantic-errors` (or `-Werror=narrowing`) to get the strict behaviour from every narrowing case, as in the examples below; Clang and MSVC reject all of them by default.

FOUR INITIALISATION STYLES main.cpp

```cpp
#include <iostream>

int main() {
    int a = 5;            // copy
    int b(7);             // direct
    int c{9};             // brace
    int d{};              // value-initialized: 0
    double price = 99.99;
    int rupees = price;   // silently becomes 99 (only a warning with -Wconversion)
    std::cout << a << ' ' << b << ' ' << c << ' ' << d << ' ' << rupees << '\n';
    return 0;
}
```

OUTPUT

```
5 7 9 0 99
```

BRACE INITIALIZATION REJECTS NARROWING (BUILT WITH -PEDANTIC-ERRORS) main.cpp

```cpp
int main() {
    double price = 99.99;
    int rupees{price};          // narrowing double -> int
    char small{300};            // 300 does not fit in a char
    unsigned u{-1};             // negative into unsigned
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:3:16: error: narrowing conversion of ‘price’ from ‘double’ to ‘int’ [-Wnarrowing]
    3 |     int rupees{price};          // narrowing double -> int
      |                ^~~~~
main.cpp:4:16: error: narrowing conversion of ‘300’ from ‘int’ to ‘char’ [-Wnarrowing]
    4 |     char small{300};            // 300 does not fit in a char
      |                ^~~
main.cpp:5:16: error: narrowing conversion of ‘-1’ from ‘int’ to ‘unsigned int’ [-Wnarrowing]
    5 |     unsigned u{-1};             // negative into unsigned
      |                ^~
main.cpp:3:9: warning: unused variable ‘rupees’ [-Wunused-variable]
    3 |     int rupees{price};          // narrowing double -> int
      |         ^~~~~~
main.cpp:4:10: warning: unused variable ‘small’ [-Wunused-variable]
    4 |     char small{300};            // 300 does not fit in a char
      |          ^~~~~
main.cpp:5:14: warning: unused variable ‘u’ [-Wunused-variable]
    5 |     unsigned u{-1};             // negative into unsigned
      |              ^
```

3.2.5 Uninitialized variables and garbage values

A local variable of a built-in type that is not initialised has an **indeterminate value**: whatever bytes happened to be in that memory. Reading it is undefined behaviour. Global and `static` variables, in contrast, are zero-initialised automatically. Always initialise variables when you declare them.

ALWAYS INITIALIZE LOCALS

main.cpp

```cpp
#include <iostream>

int global_count;                    // globals are zero-initialized

int main() {
    int safe{};                      // 0
    int risky;                       // indeterminate: never read this
    std::cout << "global_count = " << global_count << '\n';
    std::cout << "safe         = " << safe << '\n';
    (void)risky;
    return 0;
}
```

OUTPUT

```
global_count = 0
safe         = 0
```

3.3 Scope of variables

The **scope** of a name is the part of the program where it can be used. C++ has block scope, function scope (for parameters), namespace scope (globals), and class scope (members, Module 13).

3.3.1 Local variables

A variable declared inside a function or block is **local**: visible from its declaration to the end of that block, and destroyed there. Each call of a function gets fresh local variables.

3.3.2 Global variables

A variable declared outside every function is **global** (namespace scope): visible in the whole file from its declaration onwards, alive for the entire program. Globals are convenient but any function can change them, which makes programs hard to reason about and test. Prefer passing values as parameters; use globals mainly for constants.

3.3.3 Block scope and shadowing

An inner block may declare a variable with the same name as an outer one. The inner name **shadows** (hides) the outer one until the inner block ends. Shadowing is legal but confusing; `-Wshadow` warns about it.

3.3.4 Scope resolution operator :: for globals

`::name` with nothing before the colons refers to the **global** variable, even when a local of the same name shadows it.

LOCAL, GLOBAL, BLOCK SCOPE AND :: main.cpp

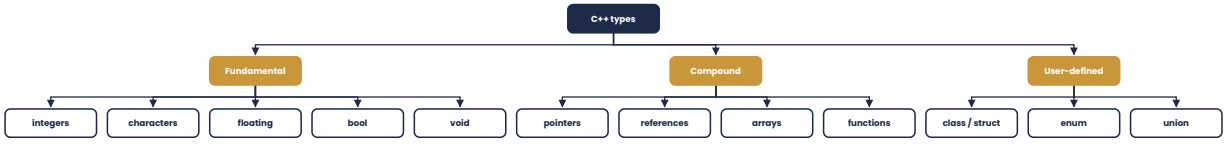
```cpp
#include <iostream>

int level = 1;                              // global

void show() { std::cout << "in show(): level = " << level << '\n'; }

int main() {
    std::cout << "global level = " << level << '\n';
    int level = 2;                          // local shadows the global
    std::cout << "local level  = " << level << ", ::level = " << ::level << '\n';
    {
        int level = 3;                      // shadows the local
        std::cout << "inner block  = " << level << '\n';
        ::level = 10;                       // modify the global
    }                                       // inner level destroyed here
    std::cout << "after block  = " << level << '\n';
    show();                                 // functions see the global
    for (int i = 0; i < 2; ++i) { int temp = i * 10; std::cout << "temp = " << temp << '\n'; }
    // std::cout << temp;  error: temp is out of scope here
    return 0;
}
```

OUTPUT

```
global level = 1
local level  = 2, ::level = 1
inner block  = 3
after block  = 2
in show(): level = 10
temp = 0
temp = 10
```

NESTED SCOPES: THE INNERMOST DECLARATION WINS

3.4 Fundamental data types



CATEGORIES OF C++ TYPES (THIS MODULE COVERS THE FUNDAMENTAL ONES)

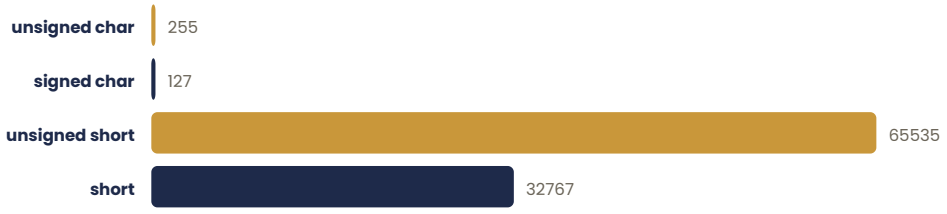
3.4.1 Integer types: short, int, long, long long

Integer types store whole numbers. The standard guarantees only **minimum** sizes and the order `short ≤ int ≤ long ≤ long long`; actual sizes depend on the platform:

Type	Minimum width	64-bit Linux/macOS	64-bit Windows	Typical range (signed)
`short`	16 bits	2 bytes	2 bytes	-32,768 to 32,767
`int`	16 bits	4 bytes	4 bytes	about ±2.1 billion
`long`	32 bits	**8 bytes**	**4 bytes**	differs between platforms!
`long long`	64 bits	8 bytes	8 bytes	about $\pm 9.2 \times 10^{18}$

3.4.2 Modifiers: signed, unsigned

Integer types are **signed** by default (they hold negative and positive values, using two's complement). `unsigned` versions hold only non-negative values but reach twice as high. Unsigned arithmetic wraps around modulo 2^n; signed overflow is undefined behaviour. Use unsigned for bit patterns and sizes, signed for ordinary arithmetic.



MAXIMUM VALUES: UNSIGNED TYPES REACH TWICE AS HIGH

3.4.3 Character types: char, wchar_t, char8_t, char16_t, char32_t

Type	Size	Holds	Literal
`char`	1 byte	one byte of text: ASCII or a piece of UTF-8	`'A'`, `"text"`
`signed char` / `unsigned char`	1 byte	small integers, raw bytes	
`wchar_t`	4 bytes Linux, 2 bytes Windows	wide characters (platform-specific)	`L'A'`
`char8_t` (C++20)	1 byte	a UTF-8 code unit	`u8'A'`, `u8"text"`
`char16_t` (C++11)	2 bytes	a UTF-16 code unit	`u'Ω'`
`char32_t` (C++11)	4 bytes	a full Unicode code point	`U'😀'`

A `char` is a small integer: `'A'` is stored as 65. Whether plain `char` is signed or unsigned is platform-dependent.

CHARACTERS ARE NUMBERS

main.cpp

```
#include <iostream>

int main() {
    char c = 'A';
    std::cout << "c = " << c << ", as int = " << static_cast<int>(c) << '\n';
    c = c + 2;
    std::cout << "c + 2 = " << c << '\n';
    char digit = '7';
    std::cout << "'7' - '0' = " << digit - '0' << " (digit to number)\n";
    char32_t omega = U'\u03A9';
    std::cout << "U'\\u03A9' has code point " << static_cast<unsigned>(omega)
              << ", sizeof(char32_t) = " << sizeof omega << '\n';
    std::cout << "\"\u0928\u092e\u0938\u094d\u0924\u0947\" in UTF-8 uses " <<
sizeof("\u0928\u092e\u0938\u094d\u0924\u0947") - 1 << " bytes\n";
    return 0;
}
```

OUTPUT

```
c = A, as int = 65
c + 2 = C
'7' - '0' = 7 (digit to number)
U'\u03A9' has code point 937, sizeof(char32_t) = 4
"नमस्ते" in UTF-8 uses 18 bytes
```

3.4.4 Floating-point types: float, double, long double

Type	Size (typical)	Significant decimal digits	Range (approx.)	Literal
`float`	4 bytes	6 to 7	$\pm 3.4 \times 10^{38}$	`3.14f`
`double`	8 bytes	15 to 16	$\pm 1.8 \times 10^{308}$	`3.14`
`long double`	16 bytes on x86-64 Linux (80-bit precision); 8 on MSVC	18 to 19	$\pm 1.2 \times 10^{4932}$	`3.14L`

Floating-point numbers use the IEEE 754 binary format, so most decimal fractions (0.1, 0.2) cannot be stored exactly. Use `double` by default, never compare floating-point values with `==`, and never use them for money.

PRECISION OF THE FLOATING-POINT TYPES main.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <cmath>

int main() {
    float f = 1.0f / 3;
    double d = 1.0 / 3;
    long double ld = 1.0L / 3;
    std::cout << std::setprecision(20);
    std::cout << "float       " << f << '\n';
    std::cout << "double      " << d << '\n';
    std::cout << "long double " << ld << '\n';
    double sum = 0.1 + 0.2;
    std::cout << "0.1 + 0.2 = " << sum << '\n';
    std::cout << std::boolalpha << "0.1 + 0.2 == 0.3 ? " << (sum == 0.3) << '\n';
    std::cout << "nearly equal? " << (std::fabs(sum - 0.3) < 1e-9) << '\n';
    return 0;
}
```

OUTPUT

```
float       0.3333333432674407959
double      0.33333333333333331483
long double 0.33333333333333333334
0.1 + 0.2 = 0.30000000000000004441
0.1 + 0.2 == 0.3 ? false
nearly equal? true
```

3.4.5 bool

`bool` holds `true` or `false` (1 byte). Conversions: any non-zero number becomes `true`, zero becomes `false`; `true` converts to 1 and `false` to 0. By default streams print 1 and 0; `std::boolalpha` prints the words.

BOOLEAN VALUES main.cpp

```cpp
#include <iostream>

int main() {
    bool passed = 72 >= 40;
    bool empty = false;
    std::cout << passed << ' ' << empty << '\n';
    std::cout << std::boolalpha << passed << ' ' << empty << '\n';
    bool from_int = 42;                    // non-zero -> true
    int from_bool = true + true;           // true -> 1
    std::cout << from_int << ' ' << from_bool << ' ' << sizeof(bool) << '\n';
    return 0;
}
```

OUTPUT

```
1 0
true false
true 2 1
```

3.4.6 void

`void` means "no type". It is used as the return type of functions that return nothing (`void print()`) and in `void*`, a pointer to memory of unknown type (Module 10). You cannot declare a variable of type void.

3.4.7 sizeof and platform-dependent sizes

`sizeof(type)` or `sizeof expression` gives the size in bytes as a `std::size_t`, computed at compile time. By definition `sizeof(char) == 1`. Because other sizes vary, never assume them; use sizeof or the fixed-width types (3.4.9).

3.4.8 <climits>, <limits> and std::numeric_limits

`<climits>` provides C-style macros (`INT_MAX`, `LLONG_MIN`, `CHAR_BIT`). The C++ way is the class template `std::numeric_limits<T>` in `<limits>`, which works for every arithmetic type and can be used in templates.

QUERYING LIMITS

main.cpp

```cpp
#include <climits>
#include <iostream>
#include <limits>

int main() {
    std::cout << "INT_MAX (climits)          = " << INT_MAX << '\n';
    std::cout << "numeric_limits<int>::max() = " << std::numeric_limits<int>::max() << '\n';
    std::cout << "numeric_limits<int>::min() = " << std::numeric_limits<int>::min() << '\n';
    std::cout << "unsigned max               = " << std::numeric_limits<unsigned>::max() << '\n';
    std::cout << "double max                 = " << std::numeric_limits<double>::max() << '\n';
    std::cout << "double lowest              = " << std::numeric_limits<double>::lowest() << '\n';
    std::cout << "double epsilon             = " << std::numeric_limits<double>::epsilon() << '\n';
    std::cout << "double digits10            = " << std::numeric_limits<double>::digits10 << '\n';
    std::cout << "is char signed?            = " << std::numeric_limits<char>::is_signed << '\n';
    std::cout << "bits in a byte (CHAR_BIT)  = " << CHAR_BIT << '\n';
    return 0;
}
```

OUTPUT

```
INT_MAX (climits)          = 2147483647
numeric_limits<int>::max() = 2147483647
numeric_limits<int>::min() = -2147483648
unsigned max               = 4294967295
double max                 = 1.79769e+308
double lowest              = -1.79769e+308
double epsilon             = 2.22045e-16
double digits10            = 15
is char signed?            = 1
bits in a byte (CHAR_BIT)  = 8
```

NOTE

For floating-point types, `min()` is the smallest **positive** normal value (about 2.2×10^{-308} for double), not the most negative one. Use `lowest()` for that.

3.4.9 Fixed-width integers in <cstdint>

When the exact width matters (file formats, network protocols, hardware, hashing), use the types from `<cstdint>`: `std::int8_t`, `std::int16_t`, `std::int32_t`, `std::int64_t` and their unsigned versions `std::uint8_t` … `std::uint64_t`. `std::size_t` (from `<cstddef>`) is the unsigned type of sizes and indexes.

EXACT-WIDTH INTEGER TYPES

main.cpp

```cpp
#include <cstdint>
#include <iostream>

int main() {
    std::int32_t  temperature = -40;
    std::uint64_t population  = 8'100'000'000;
    std::uint8_t  brightness  = 200;
    std::cout << "int32_t  " << temperature << " (" << sizeof temperature << " bytes)\n";
    std::cout << "uint64_t " << population  << " (" << sizeof population << " bytes)\n";
    std::cout << "uint8_t  " << +brightness << " (" << sizeof brightness << " byte; unary + prints it
as a number)\n";
    std::cout << "INT64_MAX = " << INT64_MAX << '\n';
    return 0;
}
```

OUTPUT

```
int32_t  -40 (4 bytes)
uint64_t 8100000000 (8 bytes)
uint8_t  200 (1 byte; unary + prints it as a number)
INT64_MAX = 9223372036854775807
```

WATCH OUT

`std::uint8_t` and `std::int8_t` are usually aliases of `unsigned char` and `signed char`, so `std::cout << brightness` prints a character, not a number. Promote with unary `+` or `static_cast<int>`.

3.5 Type inference

3.5.1 auto (C++11)

With `auto` the compiler deduces the variable's type from its initialiser. The variable is still statically typed: the type is fixed at compile time, you just do not have to write it. auto requires an initialiser, and it drops top-level `const` and references unless you write `const auto` or `auto&`.

AUTO DEDUCES THE TYPE

main.cpp

```
#include <iostream>
#include <map>
#include <string>
#include <typeinfo>
#include <vector>

int main() {
    auto count = 10;              // int
    auto price = 99.5;            // double
    auto ratio = 2.5f;            // float
    auto big   = 10'000'000'000;  // long (too big for int)
    auto name  = std::string{"Asha"};
    auto letter = 'x';            // char
    std::cout << typeid(count).name() << ' ' << typeid(price).name() << ' ' << typeid(ratio).name() <<
' '
              << typeid(big).name() << ' ' << typeid(letter).name() << "  (GCC's short type codes)\n";

    std::map<std::string, std::vector<int>> marks = {{"Asha", {88, 92}}, {"Ravi", {71}}};
    for (auto it = marks.begin(); it != marks.end(); ++it)          // instead of a very long type
        std::cout << it->first << " has " << it->second.size() << " mark(s)\n";
    auto literal = "hi";          // const char*, not std::string!
    std::cout << "sizeof literal = " << sizeof literal << " (a pointer)\n";
    return 0;
}
```

OUTPUT

```
i d f l c  (GCC's short type codes)
Asha has 2 mark(s)
Ravi has 1 mark(s)
sizeof literal = 8 (a pointer)
```

NOTE

GCC's `typeid(...).name()` prints mangled codes: i = int, d = double, f = float, l = long, c = char. Other compilers print full names.

3.5.2 decltype

`decltype(expression)` yields the **declared type** of an expression, without evaluating it. It is mainly used in templates and generic code to write "the same type as that".

DECLTYPE NAMES THE TYPE OF AN EXPRESSION

main.cpp

```cpp
#include <iostream>
#include <type_traits>

int main() {
    int x = 5;
    const double rate = 0.18;
    decltype(x) y = 10;               // int
    decltype(rate) r2 = 0.05;         // const double
    decltype(x + rate) mixed = 1.5;   // double
    std::cout << std::boolalpha;
    std::cout << "y is int?               " << std::is_same_v<decltype(y), int> << '\n';
    std::cout << "r2 is const double?     " << std::is_same_v<decltype(r2), const double> << '\n';
    std::cout << "x + rate is double?     " << std::is_same_v<decltype(mixed), double> << '\n';
    std::cout << "values: " << y << ' ' << r2 << ' ' << mixed << '\n';
    return 0;
}
```

OUTPUT

```
y is int?               true
r2 is const double?     true
x + rate is double?     true
values: 10 0.05 1.5
```

3.5.3 When to use auto and when not to

Use auto	Avoid auto
the type is obvious from the right-hand side: `auto p = std::make_unique<Car>();`	the type is not obvious and matters to the reader: `auto x = compute();`
long iterator and template types	when you need a specific type: `auto n = 5;` gives int when you wanted `std::size_t` or `long long`
range-based for loops: `for (const auto& s : students)`	string literals when you want a std::string (`auto s = "x";` is `const char*`)
lambdas (their type cannot be written)	with brace lists: `auto v = {1, 2};` is `std::initializer_list<int>`

3.6 Literals

A **literal** is a value written directly in the source code.

3.6.1 Integer literals: decimal, octal, hexadecimal, binary 0b

3.6.2 Digit separators 1'000'000 (C++14)

3.6.3 Suffixes: u, l, ll, f

NUMBER BASES, SEPARATORS AND SUFFIXES

main.cpp

```cpp
#include <bitset>
#include <iostream>

int main() {
    int dec = 173;
    int oct = 0255;                 // leading 0: octal
    int hex = 0xAD;                 // 0x: hexadecimal
    int bin = 0b1010'1101;          // 0b: binary (C++14), with a digit separator
    std::cout << dec << ' ' << oct << ' ' << hex << ' ' << bin << '\n';
    long long population = 1'428'627'663;    // separators make big numbers readable
    double avogadro = 6.022'140'76e23;
    std::cout << population << ' ' << avogadro << '\n';
    std::cout << "bits of 173: " << std::bitset<8>(173) << '\n';
    auto a = 42u;  auto b = 42l;  auto c = 42ll;  auto d = 42ull;  auto e = 4.2f;  auto f = 4.2L;
    std::cout << sizeof a << ' ' << sizeof b << ' ' << sizeof c << ' ' << sizeof d << ' '
              << sizeof e << ' ' << sizeof f << "  (bytes for u, l, ll, ull, f, L)\n";
    return 0;
}
```

OUTPUT

```
173 173 173 173
1428627663 6.02214e+23
bits of 173: 10101101
4 8 8 8 4 16  (bytes for u, l, ll, ull, f, L)
```

Suffix	Type	Example
(none)	int (or larger if needed)	`42`
`u` / `U`	unsigned	`42u`
`l` / `L`	long	`42L`
`ll` / `LL`	long long	`42LL`
`ull`	unsigned long long	`42ull`
`z` (C++23)	signed size type	`42z`, `42uz` = size_t
`f` / `F`	float	`3.14f`
(none) with a point	double	`3.14`, `1e-6`
`L` with a point	long double	`3.14L`

WATCH OUT

A leading zero makes an integer literal octal: `int code = 010;` is 8. Never pad decimal numbers with zeros.

3.6.4 Character and string literals, escape sequences

Escape	Meaning	Escape	Meaning
`\n`	newline	`\\`	backslash
`\t`	tab	`\'` `\"`	quotes
`\r`	carriage return	`\0`	null character
`\a`	alert (beep)	`\x41`	hex code ('A')
`\b`	backspace	`\u20B9`	Unicode code point (₹)

ESCAPE SEQUENCES AND STRING LITERALS main.cpp

```cpp
#include <iostream>
#include <string>

using namespace std::string_literals;    // enables the s suffix

int main() {
    std::cout << "Name\tMarks\nAsha\t88\n";
    std::cout << "She said \"C++ is fun\"\n";
    std::cout << "Path: C:\\Users\\asha\n";
    std::cout << "Price: \u20B9 499 (rupee sign as \\u20B9)\n";
    std::cout << "\x48\x69 from hex escapes\n";
    auto s1 = "plain";                    // const char*
    auto s2 = "std::string"s;             // std::string, thanks to the s suffix
    std::cout << sizeof(s1) << ' ' << s2.size() << '\n';
    std::cout << "adjacent " "literals " "are joined\n";
    return 0;
}
```

OUTPUT

```
Name    Marks
Asha    88
She said "C++ is fun"
Path: C:\Users\asha
Price: ₹ 499 (rupee sign as \u20B9)
Hi from hex escapes
8 11
adjacent literals are joined
```

3.6.5 Raw string literals R"(...)"

In a **raw string literal** `R"(...)"` backslashes and quotes are ordinary characters and the text may span lines. They are ideal for Windows paths, regular expressions, JSON and SQL. If the text itself contains `)"`, add a delimiter: `R"xyz(...)xyz"`.

RAW STRING LITERALS main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << R"(C:\Users\asha\notes.txt)" << '\n';
    std::cout << R"(regex: \d{3}-\d{4} and "quotes" need no escapes)" << '\n';
    std::cout << R"json({
  "name": "Asha",
  "marks": [88, 92]
})json" << '\n';
    return 0;
}
```

OUTPUT

```
C:\Users\asha\notes.txt
regex: \d{3}-\d{4} and "quotes" need no escapes
{
  "name": "Asha",
  "marks": [88, 92]
}
```

3.7 Constants

3.7.1 const

`const` makes a variable read-only after initialisation. It documents intent, prevents accidental changes, and lets the compiler optimise. A const variable must be initialised. Its value may be computed at run time.

CONST VALUES main.cpp

```cpp
#include <iostream>

int main() {
    const double GST_RATE = 0.18;
    int qty;
    std::cin >> qty;
    const double total = qty * 250.0 * (1 + GST_RATE);   // const, but known only at run time
    std::cout << "total for " << qty << " items = " << total << '\n';
    // GST_RATE = 0.12;   error: assignment of read-only variable
    return 0;
}
```

CONSOLE (user input in gold)

```
4
total for 4 items = 1180
```

3.7.2 constexpr (C++11)

`constexpr` means the value is known **at compile time**. It can be used where the language requires a constant: array sizes, template arguments, `case` labels, `static_assert`. Functions marked `constexpr` can be evaluated by the compiler when their arguments are constants, moving computation from run time to compile time.

COMPILE-TIME CONSTANTS

main.cpp

```cpp
#include <array>
#include <iostream>

constexpr int square(int x) { return x * x; }
constexpr double PI = 3.141592653589793;
constexpr int MAX_STUDENTS = 60;

int main() {
    std::array<int, MAX_STUDENTS> marks{};         // size must be a compile-time constant
    constexpr int area = square(12);                // computed by the compiler
    static_assert(area == 144, "square is wrong");  // checked at compile time
    int n = 7;
    int runtime = square(n);                        // same function, called at run time
    std::cout << marks.size() << ' ' << area << ' ' << runtime << ' ' << PI * 2 << '\n';
    return 0;
}
```

OUTPUT

```
60 144 49 6.28319
```

	const	constexpr
Meaning	read-only after initialisation	value known at compile time (and therefore read-only)
Initializer	any expression, even run-time input	a constant expression
Usable as array size / template argument	only if initialised with a constant	always
Applies to functions	no (const member functions are different)	yes: constexpr functions

3.7.3 #define and why to prefer const/constexpr

C programs define constants with the preprocessor: `#define PI 3.14159`. The preprocessor replaces the text before compilation, so the "constant" has no type, ignores scope and namespaces, does not appear in the debugger, and can produce surprising errors. In C++ prefer `constexpr` (or `const`), which are real typed variables. Macros are still used for conditional compilation and include guards.

A MACRO "CONSTANT" THAT BITES

main.cpp

```cpp
#include <iostream>

#define SIZE 10 + 5                 // text substitution, no parentheses
constexpr int SIZE2 = 10 + 5;       // a real typed constant

int main() {
    std::cout << "SIZE * 2  = " << SIZE * 2 << "   (10 + 5 * 2!)\n";
    std::cout << "SIZE2 * 2 = " << SIZE2 * 2 << '\n';
    return 0;
}
```

OUTPUT

```
SIZE * 2  = 20   (10 + 5 * 2!)
SIZE2 * 2 = 30
```

3.7.4 Enumerations (introduced; detailed in Module 12)

An **enumeration** defines a set of named integer constants, making code self-describing. C++11 **scoped enums** (`enum class`) keep their names inside the enum and do not convert silently to int, so they are preferred.

A FIRST LOOK AT SCOPED ENUMS main.cpp

```cpp
#include <iostream>

enum class Day { Mon, Tue, Wed, Thu, Fri, Sat, Sun };
enum class Grade : char { A = 'A', B = 'B', C = 'C' };

int main() {
    Day today = Day::Wed;
    if (today == Day::Wed) std::cout << "midweek\n";
    std::cout << "Wed has value " << static_cast<int>(today) << '\n';
    Grade g = Grade::B;
    std::cout << "grade " << static_cast<char>(g) << '\n';
    // int n = today;   error: no implicit conversion from a scoped enum
    return 0;
}
```

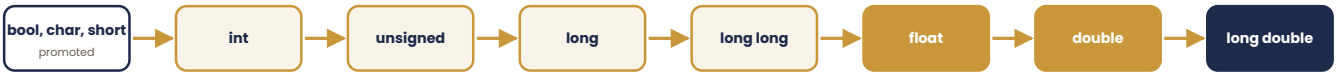
OUTPUT

```
midweek
Wed has value 2
grade B
```

3.8 Type conversion

3.8.1 Implicit conversions and promotions

When an expression mixes types, the compiler converts operands automatically. **Integral promotion**: `bool`, `char` and `short` become `int` before arithmetic. **Usual arithmetic conversions**: in a binary operation, the operand of the "smaller" type is converted to the "larger" one (int → long → long long → float → double → long double, with unsigned winning over signed of the same rank). Assignment converts the value to the target type, possibly losing information.



THE DIRECTION OF THE USUAL ARITHMETIC CONVERSIONS

IMPLICIT CONVERSIONS main.cpp

```cpp
#include <iostream>
#include <typeinfo>

int main() {
    char a = 'A';
    short s = 2;
    auto r1 = a + s;              // both promoted to int
    auto r2 = 5 + 2.5;            // int -> double
    auto r3 = 7 / 2;              // int / int = int
    auto r4 = 7 / 2.0;            // double
    int i = 3.99;                 // double -> int: truncates toward zero
    double d = 10;                // int -> double: exact
    std::cout << r1 << ' ' << typeid(r1).name() << " | " << r2 << ' ' << typeid(r2).name()
              << " | " << r3 << " | " << r4 << " | " << i << " | " << d << '\n';
    return 0;
}
```

OUTPUT

```
67 i | 7.5 d | 3 | 3.5 | 3 | 10
```

3.8.2 C-style casts

C writes explicit conversions as `(type)value` or, in C++, also `type(value)`. A C-style cast tries every kind of C++ cast in turn until one works, including dangerous ones that remove const or reinterpret bits. It is hard to search for and hides intent, so modern C++ avoids it.

3.8.3 C++ casts: static_cast, const_cast, reinterpret_cast, dynamic_cast (introduced)

Cast	Purpose	Example
`static_cast<T>(x)`	ordinary, checked conversions between related types; the one you use 95% of the time	`static_cast<double>(sum) / n`
`const_cast<T>(x)`	add or remove `const` (rarely needed; mainly for old C APIs)	`const_cast<char*>(cstr)`
`reinterpret_cast<T>(x)`	reinterpret the bits as an unrelated type (low-level, unsafe)	`reinterpret_cast<std::uintptr_t>(ptr)`
`dynamic_cast<T>(x)`	safe downcast in a class hierarchy, checked at run time (Module 14)	`dynamic_cast<Circle*>(shape)`

THE C++ CASTS IN ACTION

main.cpp

```cpp
#include <cstdint>
#include <iostream>

int main() {
    int total = 17, count = 4;
    std::cout << "C-style:     " << (double)total / count << '\n';
    std::cout << "static_cast: " << static_cast<double>(total) / count << '\n';
    double price = 249.75;
    int rupees = static_cast<int>(price);            // intent is explicit
    char letter = static_cast<char>(65 + 2);
    std::cout << rupees << ' ' << letter << '\n';

    const int locked = 5;
    const int* cp = &locked;
    int* p = const_cast<int*>(cp);                   // compiles, but writing *p would be undefined
    std::cout << "const_cast gives a non-const pointer: " << (p == cp) << '\n';

    std::uintptr_t addr = reinterpret_cast<std::uintptr_t>(&total);
    std::cout << "address as an integer is " << (addr != 0 ? "non-zero" : "zero") << '\n';
    return 0;
}
```

OUTPUT

```
C-style:     4.25
static_cast: 4.25
249 C
const_cast gives a non-const pointer: 1
address as an integer is non-zero
```

3.8.4 Overflow, truncation and signed/unsigned pitfalls

CLASSIC CONVERSION PITFALLS

main.cpp

```cpp
#include <iostream>
#include <limits>
#include <vector>

int main() {
    unsigned int u = 0;
    u = u - 1;                                      // unsigned wraps around: defined
    std::cout << "0u - 1 = " << u << '\n';

    int big = std::numeric_limits<int>::max();
    long long safe = static_cast<long long>(big) + 1; // widen before adding
    std::cout << "INT_MAX + 1 computed in long long = " << safe << '\n';

    double d = 3.99;
    std::cout << "int(3.99) = " << static_cast<int>(d) << " (truncated, not rounded)\n";

    short s = static_cast<short>(70000);            // does not fit in 16 bits
    std::cout << "short(70000) = " << s << '\n';

    int neg = -1;
    unsigned int one = 1;
    std::cout << "-1 < 1u ? " << (neg < one ? "true" : "false (-1 became 4294967295)") << '\n';

    std::vector<int> v;                             // empty vector
    std::cout << "v.size() - 1 = " << v.size() - 1 << " (size is unsigned!)\n";
    return 0;
}
```

OUTPUT

```
0u - 1 = 4294967295
INT_MAX + 1 computed in long long = 2147483648
int(3.99) = 3 (truncated, not rounded)
short(70000) = 4464
-1 < 1u ? false (-1 became 4294967295)
v.size() - 1 = 18446744073709551615 (size is unsigned!)
```

WATCH OUT

Signed integer overflow (`INT_MAX + 1` in int) is **undefined behaviour**; unsigned overflow wraps around. Loops such as `for (int i = 0; i < v.size() - 1; ++i)` break when the vector is empty because `v.size() - 1` becomes a huge unsigned number. Compile with `-Wall -Wextra -Wconversion` to see these problems.

3.9 Type aliases

3.9.1 typedef

3.9.2 using alias (C++11)

A **type alias** gives a new name to an existing type: it does not create a new type. C's `typedef` still works, but the C++11 `using` syntax reads left to right like an assignment, and it also supports **alias templates**, which typedef cannot.

TYPEDEF AND USING

main.cpp

```
#include <cstdint>
#include <iostream>
#include <map>
#include <string>
#include <vector>

typedef unsigned long ulong;                        // C style
using Rupees = std::int64_t;                        // C++11 style
using Gradebook = std::map<std::string, std::vector<int>>;
template <typename T>
using Matrix = std::vector<std::vector<T>>;         // alias template (typedef cannot do this)
using Callback = void (*)(int);                     // much clearer than typedef void (*Callback)
(int);

void announce(int n) { std::cout << "callback got " << n << '\n'; }

int main() {
    ulong big = 4'000'000'000;
    Rupees salary = 1'250'000;
    Gradebook book = {{"Asha", {88, 92}}};
    Matrix<double> m(2, std::vector<double>(3, 0.5));
    Callback cb = announce;
    std::cout << big << ' ' << salary << ' ' << book["Asha"][1] << ' ' << m[1][2] << '\n';
    cb(7);
    return 0;
}
```

OUTPUT

```
4000000000 1250000 92 0.5
callback got 7
```

Every variable has a name, type, value and address, and its type is fixed at compile time. Initialise every variable, preferably with braces to block narrowing. Know the fundamental types and check sizes with sizeof and std::numeric_limits; use <cstdint> when width matters. Use auto when the type is obvious, constexpr and const instead of #define, static_cast instead of C casts, and watch for overflow and signed/unsigned traps.

PRACTICE CHECKPOINT

Print the **size and range of every fundamental type** with sizeof and std::numeric_limits, and show **brace initialization rejecting a narrowing conversion**.

SAMPLE SOLUTION

main.cpp

```
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>

template <typename T>
void row(const std::string& name) {
    using L = std::numeric_limits<T>;
    std::cout << std::left << std::setw(20) << name << std::right << std::setw(3) << sizeof(T) << "
";
    if constexpr (std::is_floating_point_v<T>)
        std::cout << L::lowest() << " to " << L::max() << "  (" << L::digits10 << " digits)\n";
    else
```

```
        std::cout << +L::min() << " to " << +L::max() << '\n';   // + prints chars as numbers
}

int main() {
    std::cout << "type               bytes range\n";
    row<bool>("bool");
    row<char>("char");
    row<signed char>("signed char");
    row<unsigned char>("unsigned char");
    row<wchar_t>("wchar_t");
    row<char8_t>("char8_t");
    row<char16_t>("char16_t");
    row<char32_t>("char32_t");
    row<short>("short");
    row<unsigned short>("unsigned short");
    row<int>("int");
    row<unsigned int>("unsigned int");
    row<long>("long");
    row<unsigned long>("unsigned long");
    row<long long>("long long");
    row<unsigned long long>("unsigned long long");
    row<float>("float");
    row<double>("double");
    row<long double>("long double");

    double d = 7.9;
    int copy = d;                  // allowed: silently truncated to 7
    // int braced{d};              // uncomment: error, narrowing conversion
    std::cout << "copy-initialized from 7.9 gives " << copy << '\n';
    return 0;
}
```

OUTPUT

```
type               bytes range
bool                   1  0 to 1
char                   1  -128 to 127
signed char            1  -128 to 127
unsigned char          1  0 to 255
wchar_t                4  -2147483648 to 2147483647
char8_t                1  0 to 255
char16_t               2  0 to 65535
char32_t               4  0 to 4294967295
short                  2  -32768 to 32767
unsigned short         2  0 to 65535
int                    4  -2147483648 to 2147483647
unsigned int           4  0 to 4294967295
long                   8  -9223372036854775808 to 9223372036854775807
unsigned long          8  0 to 18446744073709551615
long long              8  -9223372036854775808 to 9223372036854775807
unsigned long long     8  0 to 18446744073709551615
float                  4  -3.40282e+38 to 3.40282e+38  (6 digits)
double                 8  -1.79769e+308 to 1.79769e+308  (15 digits)
long double           16  -1.18973e+4932 to 1.18973e+4932  (18 digits)
copy-initialized from 7.9 gives 7
```

Uncommenting the brace-initialised line makes the program fail to compile, which is exactly the protection we want:

THE NARROWING LINE ON ITS OWN (BUILT WITH -PEDANTIC-ERRORS)

main.cpp

```
int main() {
    double d = 7.9;
    int braced{d};
    return braced;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:3:16: error: narrowing conversion of ‘d’ from ‘double’ to ‘int’ [-Wnarrowing]
    3 |     int braced{d};
      |                ^
```

MODULE 04 · BEGINNER

Operators and expressions

Minimum duration **2 days** | Level **Beginner**

Operators act on values to produce new values. This module covers every group of C++ operators: arithmetic, increment, relational and the new three-way comparison, logical, bitwise, assignment and the special operators, then the rules of precedence and evaluation order, and the mathematical functions of <cmath>.

IN THIS MODULE

4.1 Basics
4.2 Arithmetic operators
4.3 Increment and decrement
4.4 Relational operators
4.5 Logical operators
4.6 Bitwise operators
4.7 Assignment operators
4.8 Other operators
4.9 Precedence and associativity
4.10 The cmath library

4.1 Basics

4.1.1 Operators, operands and expressions

An **operator** is a symbol that performs an operation; the values it works on are its **operands**; a combination of operands and operators that produces a value is an **expression**. Every expression has a **type** and a **value**, and some also have a **side effect** (they change something, like `x = 5` or `++i`).

OPERANDS, OPERATORS AND EXPRESSIONS

4.1.2 Unary, binary and ternary operators

Kind	Number of operands	Examples
Unary	one	`-x` `!flag` `++i` `~bits` `&var` `*ptr` `sizeof x`
Binary	two	`a + b` `a == b` `a && b` `a = b` `a << 2`
Ternary	three	`cond ? a : b` (the only one)

4.2 Arithmetic operators

4.2.1 + - * / %

Operator	Meaning	Example (a = 17, b = 5)	Result
`+`	addition	`a + b`	22
`-`	subtraction (and unary minus)	`a - b`	12
`*`	multiplication	`a * b`	85
`/`	division	`a / b`	3 (integer division)
`%`	remainder (modulo), integers only	`a % b`	2

ARITHMETIC AND A PRACTICAL USE OF / AND %

main.cpp

```
#include <iostream>

int main() {
    int a = 17, b = 5;
    std::cout << "a + b = " << a + b << '\n';
    std::cout << "a - b = " << a - b << '\n';
    std::cout << "a * b = " << a * b << '\n';
    std::cout << "a / b = " << a / b << '\n';
    std::cout << "a % b = " << a % b << '\n';
    std::cout << "-17 / 5 = " << -17 / 5 << ", -17 % 5 = " << -17 % 5 << " (truncate toward zero)\n";
    int seconds = 7384;
    std::cout << seconds << " s = " << seconds / 3600 << " h " << seconds % 3600 / 60 << " min "
              << seconds % 60 << " s\n";
    return 0;
}
```

OUTPUT

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3
a % b = 2
-17 / 5 = -3, -17 % 5 = -2 (truncate toward zero)
7384 s = 2 h 3 min 4 s
```

Since C++11, integer division truncates toward zero, and the remainder has the sign of the left operand, so that `(a / b) * b + a % b == a` always holds. Dividing an integer by zero is undefined behaviour (usually a crash); floating-point division by zero gives infinity or NaN.

4.2.2 Integer vs floating-point division

If **both** operands are integers, `/` performs integer division and discards the fraction. If **at least one** is floating-point, the result is floating-point. To get a fractional average from integers, convert one operand first.

WHERE THE CONVERSION HAPPENS MATTERS

main.cpp

```cpp
#include <iostream>

int main() {
    int total = 245, count = 4;
    std::cout << "total / count          = " << total / count << '\n';
    std::cout << "total / 4.0            = " << total / 4.0 << '\n';
    std::cout << "double(total) / count  = " << static_cast<double>(total) / count << '\n';
    std::cout << "double(total / count)  = " << static_cast<double>(total / count) << "  (too
late!)\n";
    std::cout << "7.5 / 0.0              = " << 7.5 / 0.0 << '\n';
    std::cout << "0.0 / 0.0              = " << 0.0 / 0.0 << '\n';
    return 0;
}
```

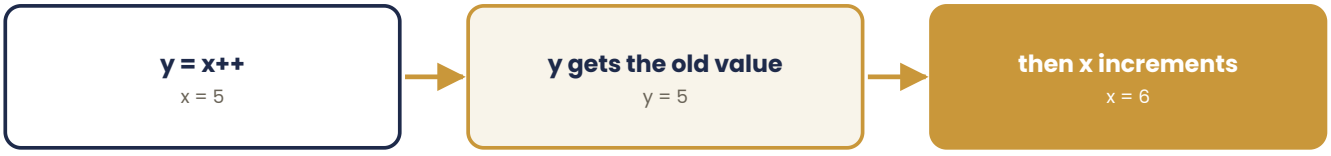
OUTPUT

```
total / count          = 61
total / 4.0            = 61.25
double(total) / count  = 61.25
double(total / count)  = 61  (too late!)
7.5 / 0.0              = inf
0.0 / 0.0              = -nan
```

4.3 Increment and decrement

4.3.1 Prefix vs postfix

`++x` and `x++` both add 1 to x (and `--` subtracts 1). The difference is the **value of the expression**: prefix `++x` increments first and yields the new value; postfix `x++` yields the old value and increments afterwards. Used as a statement on its own, both do the same; prefer prefix in C++, since for iterators and class types it avoids making a copy.



POSTFIX: USE, THEN CHANGE. PREFIX (Y = ++X): CHANGE, THEN USE, SO Y = 6

PREFIX AND POSTFIX

main.cpp

```cpp
#include <iostream>

int main() {
    int x = 5;
    int a = x++;            // a = 5, then x becomes 6
    std::cout << "a = " << a << ", x = " << x << '\n';
    int b = ++x;            // x becomes 7, then b = 7
    std::cout << "b = " << b << ", x = " << x << '\n';
    int c = x--;            // c = 7, x becomes 6
    int d = --x;            // x becomes 5, d = 5
    std::cout << "c = " << c << ", d = " << d << ", x = " << x << '\n';
    for (int i = 0; i < 3; ++i) std::cout << i << ' ';   // as a statement both forms are equal
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
a = 5, x = 6
b = 7, x = 7
c = 7, d = 5, x = 5
0 1 2
```

4.3.2 Undefined behaviour with multiple modifications

Modifying the same variable twice in one expression, or modifying it and reading it separately, without a sequencing rule between them, is **undefined behaviour**: the compiler may produce any result. C++17 added sequencing for some operators (`<<`, `>>`, `=`, `[]`), but expressions like `i++ + i++` remain undefined. Never write them.

DO NOT WRITE CODE LIKE THIS

main.cpp

```cpp
#include <iostream>

int main() {
    int i = 5;
    int r = i++ + ++i;          // undefined behaviour: two modifications of i
    std::cout << "r = " << r << " (could be anything)\n";
    int j = 5;
    j = j++;                    // well-defined since C++17 (=), but pointless and confusing
    std::cout << "j = " << j << '\n';
    return 0;
}
```

OUTPUT (compiler warnings, then the result GCC happened to produce)

```
main.cpp: In function ‘int main()’:
main.cpp:5:14: warning: operation on ‘i’ may be undefined [-Wsequence-point]
    5 |     int r = i++ + ++i;          // undefined behaviour: two modifications of i
      |             ~^~
main.cpp:8:7: warning: operation on ‘j’ may be undefined [-Wsequence-point]
    8 |     j = j++;                    // well-defined since C++17 (=), but pointless and confusing
      |     ~~^~~~
main.cpp:8:7: warning: operation on ‘j’ may be undefined [-Wsequence-point]
$ ./prog
r = 12 (could be anything)
j = 5
```

4.4 Relational operators

4.4.1 == != > < >= <=

Relational (comparison) operators compare two values and produce a `bool`. They work for numbers, characters (by code), `std::string` (alphabetically), and any type that defines them.

COMPARISONS PRODUCE BOOL

main.cpp

```
#include <iostream>
#include <string>

int main() {
    int a = 10, b = 20;
    std::cout << std::boolalpha;
    std::cout << "a == b: " << (a == b) << ", a != b: " << (a != b) << '\n';
    std::cout << "a <  b: " << (a < b)  << ", a >= b: " << (a >= b) << '\n';
    std::cout << "'a' < 'b': " << ('a' < 'b') << ", 'Z' < 'a': " << ('Z' < 'a') << '\n';
    std::string s1 = "apple", s2 = "banana";
    std::cout << "\"apple\" < \"banana\": " << (s1 < s2) << '\n';
    double x = 0.1 * 3;
    std::cout << "0.1 * 3 == 0.3: " << (x == 0.3) << "  (floating point!)\n";
    return 0;
}
```

OUTPUT

```
a == b: false, a != b: true
a <  b: true, a >= b: false
'a' < 'b': true, 'Z' < 'a': true
"apple" < "banana": true
0.1 * 3 == 0.3: false  (floating point!)
```

WATCH OUT

`=` assigns, `==` compares. `if (x = 5)` assigns 5 to x and is always true. `-Wall` warns about it. Also, chained comparisons do not work as in mathematics: `1 < x < 5` compares `(1 < x)`, which is 0 or 1, with 5, so it is always true; write `1 < x && x < 5`.

4.4.2 Three-way comparison <=> (C++20, introduced)

The **spaceship operator** `a <=> b` compares once and tells you whether a is less than, equal to, or greater than b. The result is an ordering object that you compare with 0. Its main use is in classes: defining (or defaulting) `<=>` generates all six relational operators automatically (Module 15).

THE SPACESHIP OPERATOR

main.cpp

```cpp
#include <compare>
#include <iostream>
#include <string>

struct Version {
    int major, minor;
    auto operator<=>(const Version&) const = default;   // generates < <= > >= == !=
};

int main() {
    int a = 3, b = 7;
    auto r = a <=> b;
    if (r < 0) std::cout << "a is less than b\n";
    if ((b <=> a) > 0) std::cout << "b is greater than a\n";
    if ((a <=> 3) == 0) std::cout << "a equals 3\n";
    std::cout << std::boolalpha << "\"cat\" <=> \"car\" > 0 : " << ((std::string("cat") <=> "car") >
0) << '\n';
    Version v1{2, 4}, v2{2, 10};
    std::cout << "v1 < v2: " << (v1 < v2) << ", v1 == v2: " << (v1 == v2) << '\n';
    return 0;
}
```

OUTPUT

```
a is less than b
b is greater than a
a equals 3
"cat" <=> "car" > 0 : true
v1 < v2: true, v1 == v2: false
```

4.5 Logical operators

4.5.1 &&, ||, ! and short-circuit evaluation

a	b	a && b	a || b	!a
false	false	false	false	true
false	true	false	true	true
true	false	false	true	false
true	true	true	true	false

`&&` and `||` evaluate left to right and **stop as soon as the result is known** (short-circuit evaluation): if the left side of `&&` is false, the right side is never evaluated; if the left side of `||` is true, the right side is skipped. This is used to guard dangerous operations.

SHORT-CIRCUIT EVALUATION main.cpp

```
#include <iostream>
#include <vector>

bool check(const char* name, bool value) {
    std::cout << "  evaluating " << name << '\n';
    return value;
}
int main() {
    std::cout << "false && ...:\n";
    bool r1 = check("left", false) && check("right", true);
    std::cout << "true || ...:\n";
    bool r2 = check("left", true) || check("right", false);
    std::cout << std::boolalpha << "results: " << r1 << ' ' << r2 << '\n';

    std::vector<int> v;                                    // empty
    std::size_t i = 0;
    if (i < v.size() && v[i] > 10)                          // v[i] never evaluated: safe
        std::cout << "big\n";
    else
        std::cout << "guarded access avoided v[0] on an empty vector\n";
    int age = 20; bool has_id = true;
    std::cout << "can enter: " << (age >= 18 && has_id) << ", minor: " << !(age >= 18) << '\n';
    return 0;
}
```

OUTPUT

```
false && ...:
  evaluating left
true || ...:
  evaluating left
results: false true
guarded access avoided v[0] on an empty vector
can enter: true, minor: false
```

4.5.2 Alternative tokens (and, or, not)

C++ also accepts the keywords `and`, `or`, `not`, `not_eq`, `bitand`, `bitor`, `xor`, `compl` and their `_eq` forms as alternative spellings. They exist for keyboards without some symbols; some programmers find `and` / `or` / `not` more readable. Most code uses the symbols.

ALTERNATIVE TOKENS main.cpp

```
#include <iostream>

int main() {
    int marks = 72;
    bool absent = false;
    if (marks >= 40 and not absent) std::cout << "pass\n";
    if (marks > 90 or marks == 72) std::cout << "special case\n";
    std::cout << (5 bitand 3) << ' ' << (5 bitor 3) << ' ' << (5 xor 3) << '\n';
    return 0;
}
```

OUTPUT

```
pass
special case
1 7 6
```

4.6 Bitwise operators

4.6.1 &, |, ^, ~, <<, >>

Operator	Name	Effect on each bit	0b1100 op 0b1010
`&`	AND	1 if both are 1	0b1000 (8)
`|`	OR	1 if either is 1	0b1110 (14)
`^`	XOR	1 if they differ	0b0110 (6)
`~`	NOT	flips every bit	~0b1100 = …11110011
`<<`	left shift	shift left, fill with 0: × 2^n	0b1100 << 1 = 0b11000 (24)
`>>`	right shift	shift right: ÷ 2^n for unsigned	0b1100 >> 2 = 0b11 (3)

BITWISE OPERATORS main.cpp

```cpp
#include <iostream>

int main() {
    unsigned a = 12, b = 10;                // 1100 and 1010
    std::cout << "a & b  = " << (a & b) << '\n';
    std::cout << "a | b  = " << (a | b) << '\n';
    std::cout << "a ^ b  = " << (a ^ b) << '\n';
    std::cout << "~a     = " << (~a) << " (32-bit unsigned)\n";
    std::cout << "a << 2 = " << (a << 2) << '\n';
    std::cout << "a >> 2 = " << (a >> 2) << '\n';
    unsigned flags = 0;
    flags |= 1u << 3;                       // set bit 3
    flags |= 1u << 0;                       // set bit 0
    std::cout << "flags = " << flags << ", bit 3 is " << ((flags >> 3) & 1u) << '\n';
    flags &= ~(1u << 3);                    // clear bit 3
    std::cout << "after clearing bit 3: " << flags << '\n';
    std::cout << "17 is " << ((17 & 1) ? "odd" : "even") << '\n';
    return 0;
}
```

OUTPUT

```
a & b  = 8
a | b  = 14
a ^ b  = 6
~a     = 4294967283 (32-bit unsigned)
a << 2 = 48
a >> 2 = 3
flags = 9, bit 3 is 1
after clearing bit 3: 1
17 is odd
```

NOTE

Note the parentheses around `(a & b)` inside the output statement: `<<` has higher precedence than `&`, `|` and `^`, so without them the expression would be parsed wrongly. Do bit manipulation on unsigned types.

4.6.2 std::bitset

`std::bitset<N>` from `<bitset>` is a fixed-size sequence of N bits with readable operations: print as 0s and 1s, set, reset, flip and test individual bits, count the set bits, and convert to and from integers and strings.

WORKING WITH BITS THROUGH STD::BITSET main.cpp

```cpp
#include <bitset>
#include <iostream>

int main() {
    std::bitset<8> b(173);                  // from an integer
    std::cout << "173 in binary : " << b << '\n';
    std::cout << "bits set      : " << b.count() << " of " << b.size() << '\n';
    std::cout << "bit 0, bit 1  : " << b[0] << ", " << b.test(1) << '\n';
    b.set(1);  b.reset(7);  b.flip(4);
    std::cout << "after changes : " << b << " = " << b.to_ulong() << '\n';
    std::bitset<8> mask("00001111");         // from a string
    std::cout << "b & mask      : " << (b & mask) << '\n';
    std::cout << "b << 2        : " << (b << 2) << '\n';
    std::cout << "-5 as int32   : " << std::bitset<32>(-5) << '\n';
    return 0;
}
```

OUTPUT

```
173 in binary : 10101101
bits set      : 5 of 8
bit 0, bit 1  : 1, 0
after changes : 00111111 = 63
b & mask      : 00001111
b << 2        : 11111100
-5 as int32   : 11111111111111111111111111111011
```

4.7 Assignment operators

4.7.1 = and compound assignment

`=` stores the value on the right in the object on the left (which must be modifiable, an **lvalue**). An assignment is itself an expression whose value is the assigned object, which allows chaining `a = b = 0;` (right to left). **Compound assignment** operators combine an operation with assignment: `x += 5` means `x = x + 5`, but x is evaluated only once.

Operator	Equivalent	Operator	Equivalent
`x += y`	`x = x + y`	`x &= y`	`x = x & y`
`x -= y`	`x = x - y`	`x |= y`	`x = x | y`
`x *= y`	`x = x * y`	`x ^= y`	`x = x ^ y`
`x /= y`	`x = x / y`	`x <<= y`	`x = x << y`
`x %= y`	`x = x % y`	`x >>= y`	`x = x >> y`

ASSIGNMENT AND COMPOUND ASSIGNMENT

main.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    int a, b, c;
    a = b = c = 10;                      // chained, right to left
    a += 5;  b -= 3;  c *= 2;
    std::cout << a << ' ' << b << ' ' << c << '\n';
    int balance = 1000;
    balance -= 250;                      // withdrawal
    balance += 75;                       // deposit
    balance /= 5;
    std::cout << "balance = " << balance << '\n';
    std::string msg = "Hello";
    msg += ", C++";                      // += works for strings too
    std::cout << msg << '\n';
    int x = 7;
    x %= 4;  std::cout << "7 % 4 = " << x << '\n';
    x <<= 3; std::cout << "3 << 3 = " << x << '\n';
    return 0;
}
```

OUTPUT

```
15 7 20
balance = 165
Hello, C++
7 % 4 = 3
3 << 3 = 24
```

4.8 Other operators

4.8.1 Ternary ?:

`condition ? value_if_true : value_if_false` is an expression version of if-else. Use it for short, simple choices; nested ternaries quickly become unreadable.

THE CONDITIONAL OPERATOR

main.cpp

```
#include <iostream>

int main() {
    int marks = 64;
    std::cout << "Result: " << (marks >= 40 ? "pass" : "fail") << '\n';
    int a = 12, b = 31;
    int bigger = a > b ? a : b;
    std::cout << "bigger = " << bigger << '\n';
    int n = 1;
    std::cout << n << (n == 1 ? " item\n" : " items\n");
    char grade = marks >= 90 ? 'A' : marks >= 75 ? 'B' : marks >= 60 ? 'C' : 'D';   // readable only
when short
    std::cout << "grade " << grade << '\n';
    return 0;
}
```

OUTPUT

```
Result: pass
bigger = 31
1 item
grade C
```

4.8.2 Comma operator

The comma operator evaluates its left operand, discards the result, then evaluates and yields its right operand. It appears mainly in `for` loops with two variables. Note that commas in function calls and declarations are separators, not this operator.

THE COMMA OPERATOR

main.cpp

```
#include <iostream>

int main() {
    for (int i = 0, j = 10; i < j; i += 3, j -= 3)          // two updates with the comma operator
        std::cout << "i = " << i << ", j = " << j << '\n';
    int x = (5, 7, 9);                                        // value of the last operand
    std::cout << "x = " << x << '\n';
    return 0;
}
```

OUTPUT

```
i = 0, j = 10
i = 3, j = 7
x = 9
```

4.8.3 sizeof, alignof

`sizeof` gives the size in bytes of a type or object; `alignof(T)` (C++11) gives the **alignment** requirement: the address of every object of type T must be a multiple of it. Both are evaluated at compile time.

SIZEOF AND ALIGNOF

main.cpp

```cpp
#include <iostream>
#include <string>

struct Student { char grade; int roll; double marks; };

int main() {
    int arr[10];
    std::cout << "sizeof(int) = " << sizeof(int) << ", alignof(int) = " << alignof(int) << '\n';
    std::cout << "sizeof(double) = " << sizeof(double) << ", alignof(double) = " << alignof(double) <<
'\n';
    std::cout << "sizeof arr = " << sizeof arr << ", elements = " << sizeof arr / sizeof arr[0] <<
'\n';
    std::cout << "sizeof(Student) = " << sizeof(Student) << " (1 + 4 + 8 plus padding)\n";
    std::cout << "sizeof(std::string) = " << sizeof(std::string) << " (the object, not the text)\n";
    return 0;
}
```

OUTPUT

```
sizeof(int) = 4, alignof(int) = 4
sizeof(double) = 8, alignof(double) = 8
sizeof arr = 40, elements = 10
sizeof(Student) = 16 (1 + 4 + 8 plus padding)
sizeof(std::string) = 32 (the object, not the text)
```

4.8.4 Scope resolution ::

`::` names something inside a namespace, class or enum: `std::cout`, `Day::Mon`, `Student::count`, and on its own (`::x`) the global namespace.

4.8.5 Member access . and ->

`object.member` accesses a member of an object; `pointer->member` accesses a member through a pointer and is shorthand for `(*pointer).member`. Both are covered fully with structures and classes (Modules 12 and 13).

. AND ->

main.cpp

```cpp
#include <iostream>
#include <string>

struct Point { int x; int y; };

int main() {
    Point p{3, 4};
    Point* pp = &p;
    std::cout << "p.x = " << p.x << ", pp->y = " << pp->y << ", (*pp).y = " << (*pp).y << '\n';
    std::string s = "hello";
    std::string* ps = &s;
    std::cout << "s.size() = " << s.size() << ", ps->size() = " << ps->size() << '\n';
    return 0;
}
```

OUTPUT

```
p.x = 3, pp->y = 4, (*pp).y = 4
s.size() = 5, ps->size() = 5
```

4.8.6 new and delete (introduced; detailed in Module 11)

`new` creates an object on the **heap** (free store) and returns a pointer to it; `delete` destroys it and frees the memory. Every `new` needs exactly one `delete` (and `new[]` needs `delete[]`). Modern C++ rarely uses them directly: containers and smart pointers manage heap memory for you.

A FIRST LOOK AT NEW AND DELETE

main.cpp

```cpp
#include <iostream>
#include <memory>

int main() {
    int* p = new int(42);              // allocate one int on the heap
    std::cout << "*p = " << *p << '\n';
    delete p;                          // free it
    int* arr = new int[5]{1, 2, 3, 4, 5};
    std::cout << "arr[4] = " << arr[4] << '\n';
    delete[] arr;                      // arrays need delete[]
    auto smart = std::make_unique<int>(7);   // modern C++: freed automatically
    std::cout << "*smart = " << *smart << '\n';
    return 0;
}
```

OUTPUT

```
*p = 42
arr[4] = 5
*smart = 7
```

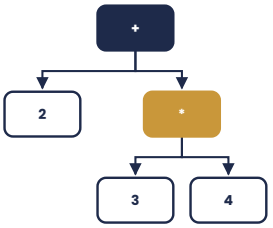
4.9 Precedence and associativity

4.9.1 Precedence table

Precedence decides which operator binds first when several appear in one expression (`2 + 3 * 4` is 14 because `*` binds tighter). **Associativity** decides the grouping of operators with the same precedence: most are left to right (`10 - 4 - 3` is `(10 - 4) - 3`), but assignment, unary operators and the conditional operator group right to left.

Level	Operators	Associativity
1 (highest)	`::`	left to right
2	`a++` `a--` `f()` `a[i]` `.` `->` casts	left to right
3	`++a` `--a` unary `+` `-` `!` `~` `*p` `&x` `sizeof` `new` `delete`	right to left
4	`.*` `->*`	left to right
5	`*` `/` `%`	left to right
6	`+` `-`	left to right
7	`<<` `>>`	left to right
8	`<=>`	left to right
9	`<` `<=` `>` `>=`	left to right
10	`==` `!=`	left to right

Level	Operators	Associativity
11	`&`	left to right
12	`^`	left to right
13	`|`	left to right
14	`&&`	left to right
15	`||`	left to right
16	`?:` `=` `+=` … `throw`	right to left
17 (lowest)	`,`	left to right



EXPRESSION TREE OF 2 + 3 * 4: THE HIGHER-PRECEDENCE * IS EVALUATED FIRST

4.9.2 Order of evaluation rules

Precedence decides **grouping**, not the **order** in which operands are evaluated. In `f() + g()` the standard does not say whether f or g runs first. The guaranteed orders are: `&&`, `||`, `?:` and the comma operator evaluate left to right; since C++17, also `<<`, `>>`, `a[b]`, `a.b`, `a->b` (left first) and assignments (right operand first). Function arguments may be evaluated in any order.

UNSPECIFIED VS GUARANTEED ORDER main.cpp

```cpp
#include <iostream>

int f() { std::cout << "f "; return 1; }
int g() { std::cout << "g "; return 2; }

int main() {
    int sum = f() + g();                    // order of f and g is unspecified
    std::cout << "-> " << sum << '\n';
    std::cout << f() << g() << '\n';        // C++17: << is sequenced left to right
    return 0;
}
```

OUTPUT (the first line could be g f with another compiler)

```
f g -> 3
f 1g 2
```

4.9.3 Using parentheses for clarity

When in doubt, add parentheses. They cost nothing and make intent obvious to readers. Classic traps:

PRECEDENCE TRAPS main.cpp

```cpp
#include <iostream>

int main() {
    int x = 6;
    std::cout << "x & 1 == 0      : " << (x & 1 == 0) << "  (parsed as x & (1 == 0))\n";
    std::cout << "(x & 1) == 0    : " << ((x & 1) == 0) << '\n';
    std::cout << "1 << 2 + 1      : " << (1 << 2 + 1) << "  (parsed as 1 << 3)\n";
    std::cout << "(1 << 2) + 1    : " << ((1 << 2) + 1) << '\n';
    int a = 10, b = 3;
    double avg = (a + b) / 2.0;
    std::cout << "a + b / 2.0     : " << a + b / 2.0 << '\n';
    std::cout << "(a + b) / 2.0   : " << avg << '\n';
    return 0;
}
```

OUTPUT

```
x & 1 == 0      : 0  (parsed as x & (1 == 0))
(x & 1) == 0    : 1
1 << 2 + 1      : 8  (parsed as 1 << 3)
(1 << 2) + 1    : 5
a + b / 2.0     : 11.5
(a + b) / 2.0   : 6.5
```

4.10 The cmath library

4.10.1 pow, sqrt, abs, ceil, floor, round, fmod, hypot

`<cmath>` provides the mathematical functions, overloaded for float, double and long double, all in namespace `std`.

Function	Returns	Example	Result
`std::pow(x, y)`	x^y	`pow(2, 10)`	1024
`std::sqrt(x)`	square root	`sqrt(144)`	12
`std::cbrt(x)`	cube root	`cbrt(27)`	3
`std::abs(x)`	absolute value	`abs(-7.5)`	7.5
`std::ceil(x)`	round up	`ceil(4.1)`	5
`std::floor(x)`	round down	`floor(4.9)`	4
`std::round(x)`	nearest, halves away from 0	`round(2.5)`	3
`std::trunc(x)`	drop fraction	`trunc(-4.7)`	-4
`std::fmod(x, y)`	floating remainder	`fmod(7.5, 2)`	1.5
`std::hypot(x, y)`	$\sqrt{(x^2 + y^2)}$	`hypot(3, 4)`	5
`std::sin cos tan`	trigonometry (radians)	`sin(π/2)`	1
`std::exp, log, log10, log2`	exponentials and logarithms	`log10(1000)`	3

MATHEMATICAL FUNCTIONS

main.cpp

```cpp
#include <cmath>
#include <iomanip>
#include <iostream>
#include <numbers>

int main() {
    std::cout << "pow(2, 10)   = " << std::pow(2, 10) << '\n';
    std::cout << "sqrt(144)    = " << std::sqrt(144) << '\n';
    std::cout << "abs(-7.5)    = " << std::abs(-7.5) << '\n';
    std::cout << "ceil(4.1)    = " << std::ceil(4.1) << ", floor(4.9) = " << std::floor(4.9) << '\n';
    std::cout << "round(2.5)   = " << std::round(2.5) << ", round(-2.5) = " << std::round(-2.5) <<
'\n';
    std::cout << "trunc(-4.7)  = " << std::trunc(-4.7) << '\n';
    std::cout << "fmod(7.5, 2) = " << std::fmod(7.5, 2) << '\n';
    std::cout << "hypot(3, 4)  = " << std::hypot(3, 4) << '\n';
    std::cout << std::fixed << std::setprecision(4);
    std::cout << "pi (C++20)   = " << std::numbers::pi << '\n';
    std::cout << "sin(pi/6)    = " << std::sin(std::numbers::pi / 6) << '\n';
    std::cout << "log10(1000)  = " << std::log10(1000.0) << ", exp(1) = " << std::exp(1.0) << '\n';
    double principal = 100000, rate = 0.08; int years = 5;
    std::cout << "compound interest amount = " << principal * std::pow(1 + rate, years) << '\n';
    return 0;
}
```

OUTPUT

```
pow(2, 10)   = 1024
sqrt(144)    = 12
abs(-7.5)    = 7.5
ceil(4.1)    = 5, floor(4.9) = 4
round(2.5)   = 3, round(-2.5) = -3
trunc(-4.7)  = -4
fmod(7.5, 2) = 1.5
hypot(3, 4)  = 5
pi (C++20)   = 3.1416
sin(pi/6)    = 0.5000
log10(1000)  = 3.0000, exp(1) = 2.7183
compound interest amount = 146932.8077
```

NOTE

C++20 added mathematical constants in `<numbers>`: `std::numbers::pi`, `e`, `sqrt2`, `ln2` and others, so you no longer need to define PI yourself.

Integer / integer truncates, % gives the remainder, and prefix ++ changes before use while postfix changes after. && and || short-circuit, = is not ==, and the bitwise operators work on individual bits (use unsigned and std::bitset). Precedence decides grouping, not evaluation order; never modify a variable twice in one expression, and use parentheses when in doubt.

PRACTICE CHECKPOINT

Write a **calculator**, an **even/odd check with bitwise AND**, and a program that **prints a number in binary using std::bitset**.

SAMPLE SOLUTION

main.cpp

```cpp
#include <bitset>
#include <cmath>
#include <iostream>

int main() {
    // 1. calculator
    double a, b;
    char op;
    std::cout << "Expression (e.g. 12.5 * 4): ";
    std::cin >> a >> op >> b;
    std::cout << a << ' ' << op << ' ' << b << " = ";
    if (op == '+') std::cout << a + b;
    else if (op == '-') std::cout << a - b;
    else if (op == '*') std::cout << a * b;
    else if (op == '/') { if (b != 0) std::cout << a / b; else std::cout << "error: division by zero";
}
    else if (op == '%') std::cout << std::fmod(a, b);
    else if (op == '^') std::cout << std::pow(a, b);
    else std::cout << "unknown operator";
    std::cout << '\n';

    // 2. even or odd with bitwise AND
    int n;
    std::cout << "Integer: ";
    std::cin >> n;
    std::cout << n << " is " << ((n & 1) ? "odd" : "even") << " (lowest bit = " << (n & 1) << ")\n";

    // 3. binary with std::bitset
    std::cout << "Binary (16 bits): " << std::bitset<16>(n) << '\n';
    std::cout << "Set bits: " << std::bitset<32>(n).count() << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
Expression (e.g. 12.5 * 4): 12.5 * 4
12.5 * 4 = 50
Integer: 2026
2026 is even (lowest bit = 0)
Binary (16 bits): 0000011111101010
Set bits: 8
```

MODULE 05 · BEGINNER

Input and output streams

Minimum duration **1 day** | Level **Beginner**

C++ performs input and output through stream objects. This module covers console input and output with cin and cout, reading whole lines, handling bad input, formatting with iomanip and the modern std::format and std::print, fast I/O for competitive programming, C-style printf/scanf, and command-line arguments.

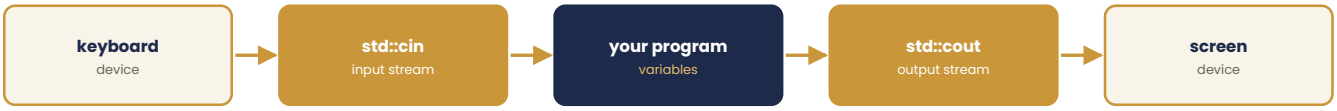
IN THIS MODULE

5.1 Stream basics
5.2 Input
5.3 Output formatting with iomanip
5.4 Modern formatting
5.5 Faster I/O
5.6 C-style I/O in C++
5.7 Command-line arguments

5.1 Stream basics

5.1.1 What a stream is

A **stream** is a sequence of characters flowing between a program and a device. The program does not need to know whether the device is a keyboard, a screen, a file or a network connection: it just inserts characters into an output stream or extracts them from an input stream. The same operators work for the console (this module), files (Module 18) and strings (`std::stringstream`, Module 8).



INPUT STREAMS BRING CHARACTERS IN; OUTPUT STREAMS CARRY THEM OUT

5.1.2 <iostream>: cin, cout, cerr, clog

Object	Type	Connected to	Buffered	Use for
`std::cin`	istream	standard input (keyboard)	yes	reading input
`std::cout`	ostream	standard output (screen)	yes	normal output
`std::cerr`	ostream	standard error (screen)	**no**: appears immediately	error messages
`std::clog`	ostream	standard error (screen)	yes	logging, diagnostics

Separating normal output from errors matters when output is redirected: `./prog > result.txt` sends cout to the file while cerr still appears on the screen.

THE FOUR STANDARD STREAMS

main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Report line written to standard output\n";
    std::cerr << "Warning: 2 records skipped (standard error)\n";
    std::clog << "Log: processing finished\n";
    return 0;
}
```

OUTPUT

```
Report line written to standard output
Warning: 2 records skipped (standard error)
Log: processing finished
```

5.1.3 Insertion << and extraction >> operators

`<<` (insertion, "put to") sends a value into an output stream; `>>` (extraction, "get from") reads a value from an input stream into a variable. Both return the stream itself, so they can be chained: `std::cout << a << b`, `std::cin >> x >> y`. The arrows point in the direction the data flows. They are overloaded for every built-in type and for `std::string`, and you can overload them for your own classes (Module 15).

5.2 Input

5.2.1 Reading numbers, characters and words with cin

`std::cin >> variable` skips leading whitespace (spaces, tabs, newlines), then reads characters that form a value of the variable's type and stops at the first character that does not fit. For a `std::string` it reads a single **word** (up to the next whitespace).

READING DIFFERENT TYPES

main.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    int age;
    double height;
    char initial;
    std::string city;
    std::cout << "Age: ";             std::cin >> age;
    std::cout << "Height in metres: "; std::cin >> height;
    std::cout << "Initial: ";         std::cin >> initial;
    std::cout << "City: ";            std::cin >> city;
    std::cout << initial << ", " << age << " years, " << height << " m, from " << city << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
Age: 21
Height in metres: 1.68
Initial: A
City: Bengaluru
A, 21 years, 1.68 m, from Bengaluru
```

5.2.2 Reading multiple values in one statement

CHAINED EXTRACTION AND READING IN A LOOP

main.cpp

```cpp
#include <iostream>

int main() {
    int a, b, c;
    std::cout << "Enter three marks: ";
    std::cin >> a >> b >> c;                 // separated by spaces or newlines
    std::cout << "total = " << a + b + c << ", average = " << (a + b + c) / 3.0 << '\n';
    int n, sum = 0, x;
    std::cout << "How many numbers? ";
    std::cin >> n;
    for (int i = 0; i < n; ++i) { std::cin >> x; sum += x; }
    std::cout << "sum = " << sum << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter three marks: 78 85 91
total = 254, average = 84.6667
How many numbers? 4
10 20
30 40
sum = 100
```

5.2.3 std::getline() for full lines

Because `>>` stops at whitespace, it cannot read a full name like "Asha Rao". `std::getline(std::cin, str)` reads everything up to the newline (which it consumes and discards) into a `std::string`. A third argument chooses another delimiter.

GETLINE READS WHOLE LINES

main.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string word, line;
    std::cout << "Full name with >> : ";
    std::cin >> word;                          // reads only "Asha"
    std::cout << "got \"" << word << "\"\n";
    std::getline(std::cin, line);              // the rest of that line: " Rao"
    std::cout << "rest of the line : \"" << line << "\"\n";
    std::cout << "Full address     : ";
    std::getline(std::cin, line);              // a whole line with spaces
    std::cout << "got \"" << line << "\"\n";
    std::string field;
    std::getline(std::cin, field, ',');        // read up to a comma
    std::cout << "first CSV field  : \"" << field << "\"\n";
    return 0;
}
```

CONSOLE (user input in gold)

```
Full name with >> : Asha Rao
got "Asha"
rest of the line : " Rao"
Full address     : 12 MG Road, Pune 411001
got "12 MG Road, Pune 411001"
Meera,92,BCA
first CSV field  : "Meera"
```

5.2.4 Mixing cin >> and getline (the leftover newline and cin.ignore())

The classic beginner bug: after `std::cin >> age;` the newline you typed is still waiting in the input buffer. A following `getline` reads up to that newline immediately and returns an **empty string**. Discard the leftover characters with `std::cin.ignore()` before calling getline, or use `std::ws` to skip whitespace.

WHY GETLINE RETURNS AN EMPTY STRING AFTER CIN >>

THE LEFTOVER NEWLINE AND TWO FIXES main.cpp

```cpp
#include <iostream>
#include <limits>
#include <string>

int main() {
    int age;
    std::string name;

    std::cout << "Age: ";
    std::cin >> age;
    std::cout << "Name: ";
    std::getline(std::cin, name);                    // BUG: reads the leftover newline
    std::cout << "[buggy] age " << age << ", name \"" << name << "\"\n";

    std::cout << "Age again: ";
    std::cin >> age;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');   // FIX: discard the rest of
the line
    std::cout << "Name: ";
    std::getline(std::cin, name);
    std::cout << "[fixed] age " << age << ", name \"" << name << "\"\n";

    std::cout << "Age once more: ";
    std::cin >> age;
    std::cout << "Name: ";
    std::getline(std::cin >> std::ws, name);         // alternative FIX: skip whitespace first
    std::cout << "[ws]    age " << age << ", name \"" << name << "\"\n";
    return 0;
}
```

CONSOLE (user input in gold)

```
Age: 21
Name: [buggy] age 21, name ""
Age again: 22
Name: Ravi Kumar
[fixed] age 22, name "Ravi Kumar"
Age once more: 23
Name: Ravi Kumar
[ws]    age 23, name "Ravi Kumar"
```

5.2.5 Checking input failure: cin.fail(), cin.clear()

If the input cannot be converted (letters when a number is expected), extraction **fails**: the variable is set to 0, the stream enters a **fail state**, and every further read is ignored until you call `std::cin.clear()` and remove the bad characters with `ignore`. The stream converts to `false` in a condition when it has failed, so `if (std::cin >> x)` tests success directly.

Member	Meaning
`cin.good()`	no error at all
`cin.fail()`	the last operation failed (bad format or end of input)
`cin.eof()`	end of input reached (Ctrl-D / Ctrl-Z)
`cin.bad()`	serious, unrecoverable stream error
`cin.clear()`	reset the error flags so reading can continue

Member	Meaning
`if (cin >> x)`	true if the read succeeded

ROBUST NUMERIC INPUT main.cpp

```cpp
#include <iostream>
#include <limits>

int read_int(const char* prompt, int lo, int hi) {
    int value;
    while (true) {
        std::cout << prompt;
        if (std::cin >> value && value >= lo && value <= hi) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            return value;
        }
        if (std::cin.eof()) return lo;                          // no more input at all
        if (std::cin.fail()) std::cout << "  not a number, try again\n";
        else std::cout << "  must be between " << lo << " and " << hi << '\n';
        std::cin.clear();                                       // 1. reset the fail state
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');   // 2. discard bad input
    }
}
int main() {
    int marks = read_int("Marks (0-100): ", 0, 100);
    std::cout << "accepted " << marks << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
Marks (0-100): ninety
  not a number, try again
Marks (0-100): 150
  must be between 0 and 100
Marks (0-100): 92
accepted 92
```

5.3 Output formatting with iomanip

Manipulators are objects you insert into a stream to change how the following values are printed. Those with arguments (setw, setprecision, setfill) need `<iomanip>`. All of them except `setw` are **sticky**: they stay in effect until changed.

5.3.1 setw, setprecision, fixed, scientific

PRECISION AND NOTATION

main.cpp

```
#include <iomanip>
#include <iostream>

int main() {
    double pi = 3.14159265358979, big = 1234567.891, tiny = 0.000123;
    std::cout << "default        : " << pi << ' ' << big << ' ' << tiny << '\n';
    std::cout << "setprecision(3): " << std::setprecision(3) << pi << ' ' << big << ' ' << tiny <<
'\n';
    std::cout << std::fixed << "fixed, 2 dp    : " << std::setprecision(2) << pi << ' ' << big << ' '
<< tiny << '\n';
    std::cout << std::scientific << "scientific     : " << pi << ' ' << big << ' ' << tiny << '\n';
    std::cout << std::defaultfloat << std::setprecision(6);    // back to the defaults
    std::cout << '[' << std::setw(8) << 42 << "] [" << 42 << "]  (setw affects only the next item)\n";
    return 0;
}
```

OUTPUT

```
default        : 3.14159 1.23457e+06 0.000123
setprecision(3): 3.14 1.23e+06 0.000123
fixed, 2 dp    : 3.14 1234567.89 0.00
scientific     : 3.14e+00 1.23e+06 1.23e-04
[      42] [42]  (setw affects only the next item)
```

Manipulator	Effect
`setw(n)`	minimum field width n for the **next** item only
`setprecision(n)`	n significant digits (default mode) or n digits after the point (fixed/scientific)
`fixed`	always use fixed-point notation 1234.57
`scientific`	always use 1.23e+03 notation
`defaultfloat`	back to the default choice

5.3.2 left, right, setfill

A NEATLY ALIGNED BILL

main.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <string>

int main() {
    struct Item { std::string name; int qty; double price; };
    Item items[] = {{"Notebook", 3, 45}, {"Pen", 10, 12.5}, {"Geometry box", 1, 189.99}};
    std::cout << std::left << std::setw(14) << "Item" << std::right << std::setw(5) << "Qty"
              << std::setw(10) << "Amount" << '\n';
    std::cout << std::setfill('-') << std::setw(29) << "" << std::setfill(' ') << '\n';
    double total = 0;
    std::cout << std::fixed << std::setprecision(2);
    for (const auto& it : items) {
        double amt = it.qty * it.price;
        total += amt;
        std::cout << std::left << std::setw(14) << it.name << std::right << std::setw(5) << it.qty
                  << std::setw(10) << amt << '\n';
    }
    std::cout << std::left << std::setw(19) << "TOTAL" << std::right << std::setw(10) << total <<
'\n';
    std::cout << "Invoice no. " << std::setfill('0') << std::setw(6) << 427 << '\n';
    return 0;
}
```

OUTPUT

```
Item            Qty    Amount
-----------------------------
Notebook          3    135.00
Pen              10    125.00
Geometry box      1    189.99
TOTAL                  449.99
Invoice no. 000427
```

5.3.3 boolalpha, hex, oct, dec, showbase

NUMBER BASES AND OTHER FLAGS

main.cpp

```
#include <iomanip>
#include <iostream>

int main() {
    bool ok = true;
    std::cout << ok << ' ' << std::boolalpha << ok << ' ' << std::noboolalpha << ok << '\n';
    int n = 255;
    std::cout << "dec " << std::dec << n << ", hex " << std::hex << n << ", oct " << std::oct << n <<
'\n';
    std::cout << std::showbase << "with showbase: " << std::hex << n << ' ' << std::oct << n <<
std::dec << '\n';
    std::cout << std::uppercase << std::hex << "uppercase hex: " << 48879 << std::nouppercase <<
std::dec << '\n';
    std::cout << std::showpos << "showpos: " << 5 << ' ' << -5 << std::noshowpos << '\n';
    std::cout << std::showpoint << "showpoint: " << 2.0 << std::noshowpoint << " vs " << 2.0 << '\n';
    return 0;
}
```

OUTPUT

```
1 true 1
dec 255, hex ff, oct 377
with showbase: 0xff 0377
uppercase hex: 0XBEEF
showpos: +5 -5
showpoint: 2.00000 vs 2
```

5.4 Modern formatting

5.4.1 std::format (C++20)

`std::format` from `<format>` builds a `std::string` from a format string with `{}` placeholders, in the style of Python. It is type-safe (unlike printf), compact (unlike long chains of manipulators), and the format specification after a colon controls width, alignment, precision and base: `{:>10.2f}` means right-aligned, width 10, fixed with 2 decimals.

STD::FORMAT

main.cpp

```cpp
#include <format>
#include <iostream>
#include <string>

int main() {
    std::string name = "Asha";
    int marks = 92;
    double avg = 87.4567;
    std::cout << std::format("{} scored {} marks\n", name, marks);
    std::cout << std::format("{1} was scored by {0}\n", name, marks);         // positional
    std::cout << std::format("average {:.2f}, rounded {:.0f}\n", avg, avg);
    std::cout << std::format("[{:<8}] [{:>8}] [{:^8}] [{:*^8}]\n", "left", "right", "mid", "fill");
    std::cout << std::format("[{:6d}] [{:06d}] [{:+d}]\n", 42, 42, 42);
    std::cout << std::format("255: dec {0} hex {0:x} {0:#X} oct {0:o} bin {0:b} {0:#010b}\n", 255);
    std::cout << std::format("{:e}  {:g}  {:.3}\n", 1234567.891, 0.0001234, 3.14159);
    std::cout << std::format("{} {}\n", true, 'x');
    std::string row = std::format("{:<10}|{:>6}|{:>8.2f}", "Pen", 10, 125.0);
    std::cout << row << "  (length " << row.size() << ")\n";
    return 0;
}
```

OUTPUT

```
Asha scored 92 marks
92 was scored by Asha
average 87.46, rounded 87
[left    ] [   right] [  mid   ] [**fill**]
[    42] [000042] [+42]
255: dec 255 hex ff 0XFF oct 377 bin 11111111 0b11111111
1.234568e+06  0.0001234  3.14
true x
Pen       |    10|  125.00  (length 26)
```

Spec	Meaning	Example → result
`{}`	next argument, default format	`format("{}", 42)` → 42
`{0}` `{1}`	argument by position	
`:<10` `:>10` `:^10`	align left / right / centre in width 10	
`:*^10`	fill with * while centring	`****mid***`
`:.2f`	fixed, 2 decimals	87.46
`:06d`	zero-padded width 6	000042
`:x` `:#X` `:b` `:o`	hex, prefixed upper hex, binary, octal	ff, 0XFF, 11111111, 377
`:+`	always show the sign	+42
`:e` `:g`	scientific, general (no % type, unlike Python)	

5.4.2 std::print and std::println (C++23)

C++23 adds `std::print` and `std::println` in `<print>`: they format exactly like std::format and write directly to the console (println adds a newline). They are faster than cout, handle Unicode correctly on Windows consoles, and read like Python's print. They are available in GCC 14+, Clang 18+ with libc++ and MSVC 19.37+; the GCC 13 used for these notes does not ship `<print>` yet, so this example shows the output a newer compiler produces.

STD::PRINT AND STD::PRINTLN (C++23) main.cpp

```cpp
#include <print>

int main() {
    std::println("Hello, {}!", "C++23");
    std::print("{} + {} = {}\n", 2, 3, 2 + 3);
    std::println("{:>8.2f}|{:<6}|{:#x}", 3.14159, "ok", 255);
    std::println(stderr, "errors can go to stderr too");
    return 0;
}
```

OUTPUT (GCC 14+, Clang 18+, MSVC 19.37+)

```
Hello, C++23!
2 + 3 = 5
    3.14|ok    |0xff
errors can go to stderr too
```

TIP

Until your compiler supports <print>, `std::cout << std::format(...)` gives the same formatting. The open-source {fmt} library, on which std::format was based, offers `fmt::print` for older compilers.

5.5 Faster I/O

5.5.1 ios::sync_with_stdio(false) and cin.tie(nullptr) for competitive programming

By default C++ streams are **synchronised** with C's stdio so that cout and printf can be mixed safely, and cin is **tied** to cout so that cout is flushed before every input operation (prompts appear before you type). Both cost time. When a program reads millions of numbers, as in competitive programming, two lines at the start of main make I/O several times faster:

THE COMPETITIVE-PROGRAMMING PREAMBLE C++

```cpp
int main() {
    std::ios::sync_with_stdio(false);   // stop syncing with C stdio: C++ streams get their own
buffers
    std::cin.tie(nullptr);              // do not flush cout before every cin read
    // ... fast reading and writing; do NOT mix printf/scanf with cin/cout after this
}
```

READING TWO MILLION NUMBERS main.cpp

```
#include <chrono>
#include <iostream>
#include <sstream>

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::ostringstream data;                          // build a large input in memory
    for (int i = 0; i < 2000000; ++i) data << i % 1000 << ' ';
    std::istringstream in(data.str());

    auto t0 = std::chrono::steady_clock::now();
    long long sum = 0;
    int x;
    while (in >> x) sum += x;
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::steady_clock::now() -
t0).count();
    std::cout << "read 2,000,000 numbers, sum = " << sum << ", in " << (ms < 1000 ? "under a second" :
"over a second") << '\n';
    return 0;
}
```

OUTPUT

```
read 2,000,000 numbers, sum = 999000000, in under a second
```

5.5.2 '\n' vs std::endl

`std::endl` writes a newline **and flushes** the buffer, forcing a system call. Printing a million lines with endl can be ten times slower than with `'\n'`. Use `'\n'` by default; flush explicitly (`std::flush`) only when output must appear immediately, for example before a long computation or a crash-prone step. Output is flushed automatically when the program ends normally.

THE COST OF FLUSHING main.cpp

```
#include <chrono>
#include <fstream>
#include <iostream>

int main() {
    using clk = std::chrono::steady_clock;
    std::ofstream out("lines.txt");
    auto t0 = clk::now();
    for (int i = 0; i < 200000; ++i) out << i << std::endl;    // flush every line
    auto t1 = clk::now();
    for (int i = 0; i < 200000; ++i) out << i << '\n';         // flush only when the buffer fills
    auto t2 = clk::now();
    double a = std::chrono::duration<double, std::milli>(t1 - t0).count();
    double b = std::chrono::duration<double, std::milli>(t2 - t1).count();
    std::cout << "std::endl was " << static_cast<int>(a / b + 0.5) << " times slower than '\\n'
here\n";
    return 0;
}
```

OUTPUT (the ratio varies by machine)

```
std::endl was 12 times slower than '\n' here
```

5.6 C-style I/O in C++

5.6.1 printf and scanf from <cstdio>

All of C's I/O is available through `<cstdio>`. `printf` uses format specifiers (`%d` int, `%f` double, `%s` C string, `%c` char, `%x` hex, `%zu` size_t) and is concise for formatted output; `scanf` needs the **address** of each variable. They are not type-safe (a wrong specifier is undefined behaviour), do not understand `std::string`, and cannot be extended to your own types, which is why C++ code prefers streams or std::format.

PRINTF AND SCANF

main.cpp

```
#include <cstdio>
#include <string>

int main() {
    int roll;
    double marks;
    char name[32];
    std::printf("Roll, name, marks: ");
    if (std::scanf("%d %31s %lf", &roll, name, &marks) != 3) return 1;
    std::printf("Roll %04d | %-8s | %6.2f | grade %c\n", roll, name, marks, marks >= 90 ? 'A' : 'B');
    std::string course = "C++";
    std::printf("Course: %s (%zu chars)\n", course.c_str(), course.size());   // .c_str() for %s
    std::printf("hex %x, octal %o, sci %e, percent sign %%\n", 255, 255, 12345.678);
    return 0;
}
```

CONSOLE (user input in gold)

```
Roll, name, marks: 17 Meera 91.5
Roll 0017 | Meera    |  91.50 | grade A
Course: C++ (3 chars)
hex ff, octal 377, sci 1.234568e+04, percent sign %
```

Specifier	Type	Specifier	Type
`%d` `%i`	int	`%s`	C string (char*)
`%u`	unsigned	`%c`	char
`%ld` `%lld`	long, long long	`%p`	pointer
`%f` `%e` `%g`	double (printf)	`%zu`	size_t
`%lf`	double (scanf)	`%x` `%o`	hex, octal

WATCH OUT

Do not mix `printf` / `scanf` with `cout` / `cin` after calling `sync_with_stdio(false)` : the two libraries then use separate buffers and output can appear out of order.

5.7 Command-line arguments

5.7.1 argc and argv

Programs can receive input from the command line that starts them: `./grade Asha 92`. Declare main as `int main(int argc, char* argv[])`. `argc` is the number of arguments including the program name; `argv`

is an array of C strings: `argv[0]` is the program name, `argv[1]` to `argv[argc - 1]` are the arguments, and `argv[argc]` is a null pointer. Arguments are always strings; convert numbers with `std::stoi` or `std::stod`.

HOW ./GRADE ASHA 92 ARRIVES IN MAIN

RUN AS: ./PROG ASHA 92

main.cpp

```cpp
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

int main(int argc, char* argv[]) {
    std::cout << "argc = " << argc << '\n';
    for (int i = 0; i < argc; ++i) std::cout << "argv[" << i << "] = " << argv[i] << '\n';
    if (argc != 3) {
        std::cerr << "usage: " << argv[0] << " NAME MARKS\n";
        return 1;
    }
    std::vector<std::string> args(argv + 1, argv + argc);        // convenient C++ container
    try {
        int marks = std::stoi(args[1]);
        std::cout << args[0] << " scored " << marks << " -> " << (marks >= 40 ? "pass" : "fail") <<
'\n';
    } catch (const std::exception&) {
        std::cerr << "MARKS must be a number\n";
        return 1;
    }
    return 0;
}
```

OUTPUT

```
argc = 3
argv[0] = ./prog
argv[1] = Asha
argv[2] = 92
Asha scored 92 -> pass
```

cin >> reads one whitespace-separated value; getline reads a whole line, so call cin.ignore() or use std::ws after >>. Check every read and recover with clear() and ignore(). Format with iomanip manipulators or, better, std::format (and std::print in C++23). Use '\n' instead of std::endl, sync_with_stdio(false) for heavy I/O, and argv for command-line input.

PRACTICE CHECKPOINT

Read a student's **full name with getline** and **three marks with cin**, then print a **neatly aligned report card** using iomanip.

SAMPLE SOLUTION

main.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <string>

int main() {
    std::string name, subjects[3] = {"Mathematics", "Physics", "Computer Science"};
    int marks[3];
    std::cout << "Student's full name: ";
    std::getline(std::cin, name);
    for (int i = 0; i < 3; ++i) {
        std::cout << subjects[i] << " marks: ";
        std::cin >> marks[i];
    }
    int total = marks[0] + marks[1] + marks[2];
    double pct = total / 3.0;
    char grade = pct >= 90 ? 'A' : pct >= 75 ? 'B' : pct >= 60 ? 'C' : pct >= 40 ? 'D' : 'F';

    const int W = 42;
    std::cout << '\n' << std::setfill('=') << std::setw(W) << "" << std::setfill(' ') << '\n';
    std::cout << std::setw((W + 11) / 2) << "REPORT CARD" << '\n';
    std::cout << std::setfill('=') << std::setw(W) << "" << std::setfill(' ') << '\n';
    std::cout << std::left << std::setw(12) << "Name" << ": " << name << '\n';
    std::cout << std::setfill('-') << std::setw(W) << "" << std::setfill(' ') << '\n';
    std::cout << std::left << std::setw(24) << "Subject" << std::right << std::setw(8) << "Marks" <<
std::setw(10) << "Out of" << '\n';
    for (int i = 0; i < 3; ++i)
        std::cout << std::left << std::setw(24) << subjects[i] << std::right << std::setw(8) <<
marks[i] << std::setw(10) << 100 << '\n';
    std::cout << std::setfill('-') << std::setw(W) << "" << std::setfill(' ') << '\n';
    std::cout << std::left << std::setw(24) << "Total" << std::right << std::setw(8) << total <<
std::setw(10) << 300 << '\n';
    std::cout << std::left << std::setw(24) << "Percentage" << std::right << std::setw(7) <<
std::fixed << std::setprecision(2) << pct << "%\n";
    std::cout << std::left << std::setw(24) << "Grade" << std::right << std::setw(8) << grade << '\n';
    std::cout << std::left << std::setw(24) << "Result" << std::right << std::setw(8) << (pct >= 40 ?
"PASS" : "FAIL") << '\n';
    std::cout << std::setfill('=') << std::setw(W) << "" << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
Student's full name: Asha Rao
Mathematics marks: 88
Physics marks: 92
Computer Science marks: 79

==========================================
               REPORT CARD
==========================================
Name        : Asha Rao
------------------------------------------
Subject                    Marks    Out of
Mathematics                   88       100
Physics                       92       100
Computer Science              79       100
------------------------------------------
Total                        259       300
Percentage                 86.33%
Grade                          B
Result                      PASS
==========================================
```

MODULE 06 · BEGINNER

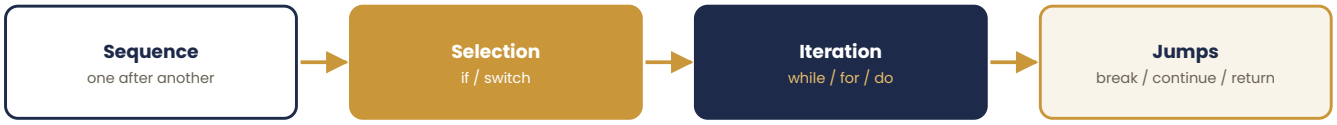
Control flow

Minimum duration **4 days** | Level **Beginner**

Control flow decides which statements run and how many times. This module covers decisions (if, switch, including the C++17 forms with initialisers), every kind of loop including range-based for, nested loops with a full set of pattern programs, jump statements, and the classic number problems that every programmer should be able to solve.

IN THIS MODULE

6.1 Decision-making statements
6.2 switch statement
6.3 Loops
6.4 Nested loops
6.5 Jump statements
6.6 Classic loop problems

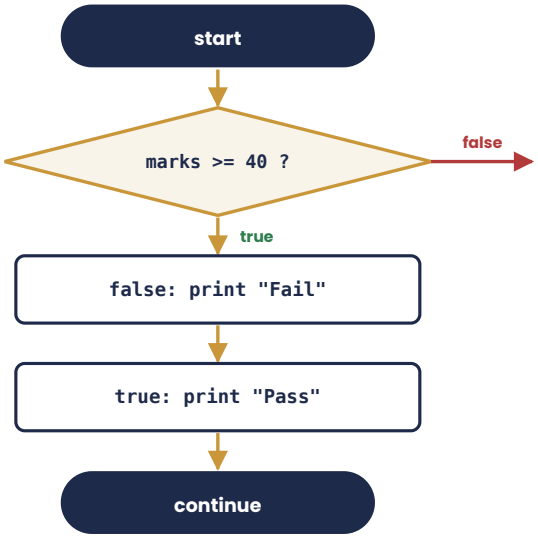


THE BUILDING BLOCKS OF CONTROL FLOW

6.1 Decision-making statements

6.1.1 if, if-else, else-if ladder, nested if

`if (condition) statement` runs the statement only when the condition is true. The condition is any expression that converts to `bool` (zero is false, non-zero true). Always use braces, even for one statement: it prevents bugs when a second line is added later.



IF-ELSE: EXACTLY ONE OF TWO BRANCHES RUNS

IF, IF-ELSE AND AN ELSE-IF LADDER

main.cpp

```cpp
#include <iostream>

int main() {
    int marks;
    std::cout << "Marks: ";
    std::cin >> marks;

    if (marks > 100 || marks < 0) {                // simple if
        std::cout << "invalid marks\n";
        return 1;
    }
    if (marks >= 40) {                             // if-else
        std::cout << "Result: pass\n";
    } else {
        std::cout << "Result: fail\n";
    }
    if (marks >= 90) {                             // else-if ladder: first true branch wins
        std::cout << "Grade: A+\n";
    } else if (marks >= 75) {
        std::cout << "Grade: A\n";
    } else if (marks >= 60) {
        std::cout << "Grade: B\n";
    } else if (marks >= 40) {
        std::cout << "Grade: C\n";
    } else {
        std::cout << "Grade: F\n";
    }
    return 0;
}
```

CONSOLE (user input in gold)

```
Marks: 78
Result: pass
Grade: A
```

A **nested if** places an if inside another. An `else` always belongs to the **nearest** unmatched `if` (the "dangling else"), which braces make explicit.

NESTED IF AND THE LEAP-YEAR RULE main.cpp

```cpp
#include <iostream>

int main() {
    int age = 20;
    bool has_licence = false;
    if (age >= 18) {
        if (has_licence) {
            std::cout << "You may drive\n";
        } else {
            std::cout << "Adult, but apply for a licence first\n";
        }
    } else {
        std::cout << "Too young to drive\n";
    }
    int year = 2024;                                    // leap year: nested logic
    bool leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    std::cout << year << (leap ? " is" : " is not") << " a leap year\n";
    return 0;
}
```

OUTPUT

```
Adult, but apply for a licence first
2024 is a leap year
```

6.1.2 if with initializer (C++17)

`if (init; condition)` declares a variable whose scope is limited to the if and its else branches. It keeps helper variables from leaking into the surrounding code.

SCOPED VARIABLES IN IF main.cpp

```cpp
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> stock = {{"pen", 40}, {"pencil", 0}};
    if (auto it = stock.find("pen"); it != stock.end() && it->second > 0) {
        std::cout << "pen in stock: " << it->second << '\n';
    } else {
        std::cout << "pen unavailable\n";
    }
    if (int qty = stock["pencil"]; qty == 0) {
        std::cout << "pencil out of stock\n";
    }
    // it and qty do not exist here
    return 0;
}
```

OUTPUT

```
pen in stock: 40
pencil out of stock
```

6.1.3 if constexpr (introduced; detailed in Module 22)

`if constexpr (condition)` is evaluated at **compile time**; the branch not taken is discarded entirely and need not even compile for the given type. It is used inside templates to choose code based on types.

COMPILE-TIME BRANCHING

main.cpp

```cpp
#include <iostream>
#include <string>
#include <type_traits>

template <typename T>
void describe(const T& value) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << value << " is an integer; doubled: " << value * 2 << '\n';
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << value << " is a floating-point number\n";
    } else {
        std::cout << '"' << value << "\" is something else (size " << value.size() << ")\n";
    }
}
int main() {
    describe(21);
    describe(3.5);
    describe(std::string("text"));    // value * 2 would not compile for a string, but that branch is
discarded
    return 0;
}
```

OUTPUT

```
21 is an integer; doubled: 42
3.5 is a floating-point number
"text" is something else (size 4)
```

6.2 switch statement

6.2.1 switch, case, break, default

`switch` selects among many fixed choices by comparing an **integral or enum** value with constant `case` labels. Execution jumps to the matching label and continues until a `break`; `default` handles every other value. Switch cannot test ranges, strings or floating-point values.

SWITCH ON AN INT main.cpp

```
#include <iostream>

int main() {
    int day;
    std::cout << "Day number (1-7): ";
    std::cin >> day;
    switch (day) {
    case 1: std::cout << "Monday\n";    break;
    case 2: std::cout << "Tuesday\n";   break;
    case 3: std::cout << "Wednesday\n"; break;
    case 4: std::cout << "Thursday\n";  break;
    case 5: std::cout << "Friday\n";    break;
    case 6:
    case 7: std::cout << "Weekend!\n";  break;      // two labels share code
    default: std::cout << "No such day\n";
    }
    return 0;
}
```

CONSOLE (user input in gold)

```
Day number (1-7): 6
Weekend!
```

6.2.2 Fall-through and [[fallthrough]] (C++17)

Without `break`, execution **falls through** into the next case. Occasionally intended, usually a bug, so `-Wimplicit-fallthrough` (part of -Wextra) warns about it. Mark intentional fall-through with the `[[fallthrough]];` attribute to document it and silence the warning.

INTENTIONAL FALL-THROUGH main.cpp

```
#include <iostream>

int main() {
    int level = 2;
    std::cout << "Access granted to:";
    switch (level) {                    // higher levels include the lower ones
    case 3:
        std::cout << " admin-panel";
        [[fallthrough]];
    case 2:
        std::cout << " reports";
        [[fallthrough]];
    case 1:
        std::cout << " dashboard";
        break;
    default:
        std::cout << " nothing";
    }
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
Access granted to: reports dashboard
```

6.2.3 switch with initializer (C++17)

A SWITCH WITH ITS OWN VARIABLE main.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string command = "save";
    switch (char c = command.empty() ? '?' : command[0]; c) {
    case 'o': std::cout << "open\n"; break;
    case 's': std::cout << "save (first letter '" << c << "')\n"; break;
    case 'q': std::cout << "quit\n"; break;
    default:  std::cout << "unknown\n";
    }
    return 0;
}
```

OUTPUT

```
save (first letter 's')
```

6.2.4 switch on enums

Switching over an `enum class` is clear and safe. With `-Wall`, GCC warns if a switch on an enum without a `default` does not handle every enumerator, catching cases you forgot when a new value is added.

SWITCH OVER A SCOPED ENUM main.cpp

```cpp
#include <iostream>

enum class Signal { Red, Yellow, Green };

const char* action(Signal s) {
    switch (s) {
    case Signal::Red:    return "stop";
    case Signal::Yellow: return "get ready";
    case Signal::Green:  return "go";
    }
    return "?";
}
int main() {
    for (Signal s : {Signal::Red, Signal::Yellow, Signal::Green})
        std::cout << static_cast<int>(s) << ": " << action(s) << '\n';
    return 0;
}
```

OUTPUT

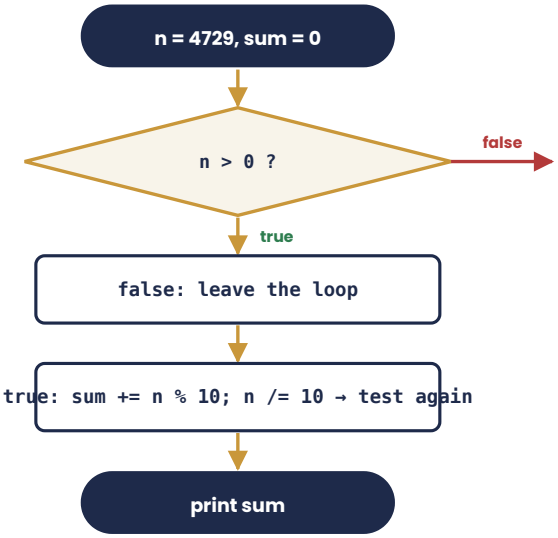
```
0: stop
1: get ready
2: go
```

	if / else-if	switch
Conditions	any boolean expression, ranges, strings	equality with integral/enum constants only
Readability for many fixed values	long chains	clear
Speed	tests in order	often a jump table: constant time

6.3 Loops

6.3.1 while

`while (condition) body` tests the condition **before** each iteration; if it is false initially, the body never runs. Use it when the number of iterations is not known in advance.



WHILE: TEST, BODY, TEST, BODY …

WHILE LOOPS main.cpp

```cpp
#include <iostream>

int main() {
    int n = 4729, sum = 0, digits = 0;
    while (n > 0) {
        sum += n % 10;              // last digit
        n /= 10;                    // drop it
        ++digits;
    }
    std::cout << digits << " digits, sum " << sum << '\n';
    int balance = 10000, years = 0;
    while (balance < 20000) {       // until the money doubles at 8%
        balance += balance * 8 / 100;
        ++years;
    }
    std::cout << "doubles after " << years << " years (" << balance << ")\n";
    return 0;
}
```

OUTPUT

```
4 digits, sum 22
doubles after 10 years (21584)
```

6.3.2 do-while

`do body while (condition);` tests **after** the body, so the body always runs at least once. Ideal for menus and input validation. Note the semicolon at the end.

DO-WHILE FOR INPUT VALIDATION main.cpp

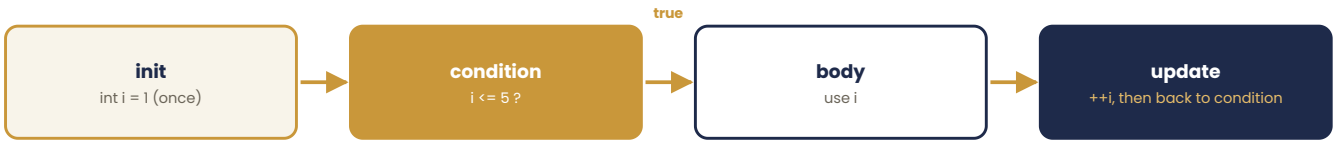
```cpp
#include <iostream>

int main() {
    int n;
    do {
        std::cout << "Enter a number from 1 to 10: ";
        std::cin >> n;
    } while (n < 1 || n > 10);               // ask again until valid
    std::cout << "accepted " << n << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter a number from 1 to 10: 0
Enter a number from 1 to 10: 15
Enter a number from 1 to 10: 7
accepted 7
```

6.3.3 for

`for (init; condition; update) body` gathers the three parts of a counting loop on one line. The init runs once; the condition is tested before every iteration; the update runs after each iteration. Any part may be empty.



EXECUTION ORDER OF A FOR LOOP

COUNTING LOOPS

main.cpp

```cpp
#include <iostream>

int main() {
    for (int i = 1; i <= 5; ++i) std::cout << i << ' ';
    std::cout << '\n';
    for (int i = 10; i > 0; i -= 3) std::cout << i << ' ';          // counting down in steps
    std::cout << '\n';
    int n = 6;
    std::cout << "table of " << n << ":\n";
    for (int i = 1; i <= 10; ++i) std::cout << n << " x " << i << " = " << n * i << '\n';
    long long fact = 1;
    for (int i = 2; i <= 15; ++i) fact *= i;
    std::cout << "15! = " << fact << '\n';
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
10 7 4 1
table of 6:
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60
15! = 1307674368000
```

6.3.4 Range-based for (C++11)

`for (declaration : range) body` visits every element of an array, a container, a string or a brace list, without indexes. Use `const auto&` to read elements without copying, `auto&` to modify them in place, and plain `auto` to work on copies.

RANGE-BASED FOR LOOPS

main.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

int main() {
    int marks[] = {72, 85, 91, 64};
    int total = 0;
    for (int m : marks) total += m;                       // a copy of each element
    std::cout << "total " << total << '\n';

    std::vector<std::string> names = {"asha", "ravi", "meera"};
    for (auto& name : names) name[0] = static_cast<char>(name[0] - 32);   // modify in place
    for (const auto& name : names) std::cout << name << ' ';           // read without copying
    std::cout << '\n';

    for (char c : std::string("C++")) std::cout << '[' << c << ']';
    for (int x : {2, 3, 5, 7}) std::cout << ' ' << x * x;
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
total 312
Asha Ravi Meera
[C][+][+] 4 9 25 49
```

6.3.5 Range-based for with initializer (C++20)

A COUNTER ALONGSIDE A RANGE

main.cpp

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> scores = {40, 75, 90};
    for (int rank = 1; int s : scores)                    // C++20: init statement before the range
        std::cout << "#" << rank++ << ": " << s << '\n';
    for (auto v = std::vector{3, 1, 2}; int x : v)        // keeps a temporary alive for the loop
        std::cout << x << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
#1: 40
#2: 75
#3: 90
3 1 2
```

6.3.6 Infinite loops and choosing the right loop

A loop whose condition never becomes false runs forever. Intentional infinite loops (`while (true)` or `for (;;)`) are common in servers, games and menus, and are left with `break` or `return`. Unintentional ones are bugs: forgetting the update, or updating the wrong variable.

AN INTENTIONAL INFINITE LOOP WITH BREAK

main.cpp

```cpp
#include <iostream>

int main() {
    int total = 0, x;
    while (true) {                      // intentional: stop on a sentinel value
        std::cin >> x;
        if (x == 0) break;
        total += x;
    }
    std::cout << "total = " << total << '\n';
    return 0;
}
```

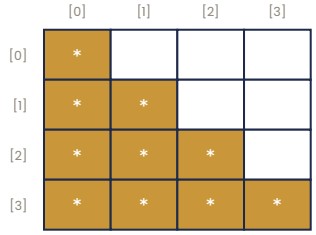
CONSOLE (user input in gold)

```
12
30
8
0
total = 50
```

Loop	Use when
`for`	counting a known number of times, or stepping through indexes
range-based `for`	visiting every element of a container or array
`while`	repeating while a condition holds, unknown number of iterations
`do-while`	the body must run at least once (menus, input validation)

6.4 Nested loops

A loop inside another loop: for each iteration of the outer loop, the inner loop runs completely. With an outer loop of n and an inner loop of m iterations, the body runs n × m times. Pattern programs are the classic exercise: the outer loop controls **rows**, inner loops print **spaces and symbols** in each row.



ROW I PRINTS I STARS: THE INNER LOOP LIMIT DEPENDS ON THE OUTER VARIABLE

6.4.1 Pattern printing

6.4.1.1 Star patterns (square, triangles, pyramid, diamond)

STAR PATTERNS

main.cpp

```cpp
#include <iostream>

int main() {
    const int n = 4;
    std::cout << "Square:\n";
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) std::cout << "* ";
        std::cout << '\n';
    }
    std::cout << "Right triangle:\n";
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= i; ++j) std::cout << "* ";
        std::cout << '\n';
    }
    std::cout << "Inverted triangle:\n";
    for (int i = n; i >= 1; --i) {
        for (int j = 1; j <= i; ++j) std::cout << "* ";
        std::cout << '\n';
    }
    std::cout << "Pyramid:\n";
    for (int i = 1; i <= n; ++i) {
        for (int s = 1; s <= n - i; ++s) std::cout << ' ';      // leading spaces
        for (int j = 1; j <= 2 * i - 1; ++j) std::cout << '*';  // odd number of stars
        std::cout << '\n';
    }
    std::cout << "Diamond:\n";
    for (int i = 1; i <= 2 * n - 1; ++i) {
        int row = i <= n ? i : 2 * n - i;                       // grows, then shrinks
        for (int s = 0; s < n - row; ++s) std::cout << ' ';
        for (int j = 0; j < 2 * row - 1; ++j) std::cout << '*';
        std::cout << '\n';
    }
    std::cout << "Hollow square:\n";
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j)
            std::cout << (i == 0 || i == n - 1 || j == 0 || j == n - 1 ? "* " : "  ");
        std::cout << '\n';
    }
    return 0;
}
```

OUTPUT

```
Square:
* * * *
* * * *
* * * *
* * * *
Right triangle:
*
* *
* * *
* * * *
Inverted triangle:
* * * *
* * *
* *
*
Pyramid:
   *
  ***
 *****
*******
Diamond:
   *
  ***
 *****
*******
 *****
  ***
   *
Hollow square:
* * * *
*     *
*     *
* * * *
```

Pattern	Row i (1-based) prints
Right triangle	i stars
Inverted triangle	n - i + 1 stars
Pyramid	n - i spaces, then 2i - 1 stars
Diamond	pyramid rows 1..n, then n-1..1
Hollow square	a star only on the border: first/last row or column

6.4.1.2 Number patterns (Floyd's triangle, Pascal's triangle)

Floyd's triangle prints consecutive numbers, one more per row. **Pascal's triangle** contains the binomial coefficients: each number is the sum of the two above it, and the k-th entry of row n is C(n, k), which can be computed from the previous one as `C(n, k) = C(n, k-1) × (n - k + 1) / k`.

NUMBER PATTERNS

main.cpp

```cpp
#include <iomanip>
#include <iostream>

int main() {
    const int rows = 5;
    std::cout << "Floyd's triangle:\n";
    int num = 1;
    for (int i = 1; i <= rows; ++i) {
        for (int j = 1; j <= i; ++j) std::cout << std::setw(3) << num++;
        std::cout << '\n';
    }
    std::cout << "Pascal's triangle:\n";
    for (int n = 0; n < rows + 1; ++n) {
        std::cout << std::string(2 * (rows - n), ' ');
        long long c = 1;                                    // C(n, 0)
        for (int k = 0; k <= n; ++k) {
            std::cout << std::setw(4) << c;
            c = c * (n - k) / (k + 1);                      // next coefficient
        }
        std::cout << '\n';
    }
    std::cout << "Number pyramid:\n";
    for (int i = 1; i <= 4; ++i) {
        std::cout << std::string(4 - i, ' ');
        for (int j = 1; j <= i; ++j) std::cout << j;
        for (int j = i - 1; j >= 1; --j) std::cout << j;
        std::cout << '\n';
    }
    return 0;
}
```

OUTPUT

```
Floyd's triangle:
  1
  2  3
  4  5  6
  7  8  9 10
 11 12 13 14 15
Pascal's triangle:
             1
          1   1
        1   2   1
      1   3   3   1
    1   4   6   4   1
  1   5  10  10   5   1
Number pyramid:
   1
  121
 12321
1234321
```

6.4.1.3 Character patterns

CHARACTER PATTERNS

main.cpp

```cpp
#include <iostream>

int main() {
    const int n = 5;
    for (int i = 0; i < n; ++i) {                          // A, AB, ABC ...
        for (char c = 'A'; c <= 'A' + i; ++c) std::cout << c;
        std::cout << '\n';
    }
    for (int i = 0; i < n; ++i) {                          // each row one letter
        for (int j = 0; j <= i; ++j) std::cout << static_cast<char>('A' + i) << ' ';
        std::cout << '\n';
    }
    char ch = 'A';
    for (int i = 1; i <= 4; ++i) {                         // continuous letters
        for (int j = 1; j <= i; ++j) std::cout << ch++ << ' ';
        std::cout << '\n';
    }
    return 0;
}
```

OUTPUT

```
A
AB
ABC
ABCD
ABCDE
A
B B
C C C
D D D D
E E E E E
A
B C
D E F
G H I J
```

6.5 Jump statements

6.5.1 break, continue, return

Statement	Effect
`break`	leave the innermost loop or switch immediately
`continue`	skip the rest of this iteration; go to the next one (the update in a for loop still runs)
`return`	leave the whole function (from main: end the program)

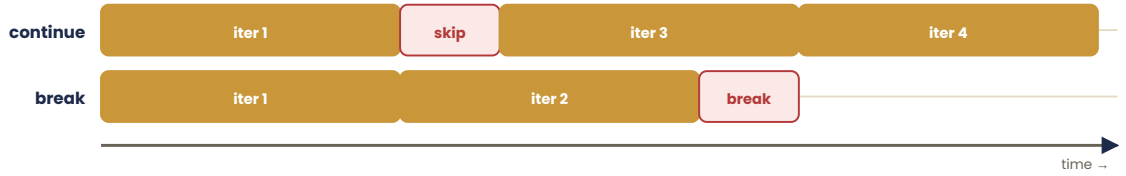
BREAK AND CONTINUE

main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "continue skips multiples of 3:";
    for (int i = 1; i <= 10; ++i) {
        if (i % 3 == 0) continue;
        std::cout << ' ' << i;
    }
    std::cout << "\nbreak at the first number divisible by 7 and 5: ";
    for (int i = 1; ; ++i) {
        if (i % 7 == 0 && i % 5 == 0) { std::cout << i << '\n'; break; }
    }
    std::cout << "break leaves only the inner loop:\n";
    for (int i = 1; i <= 3; ++i) {
        for (int j = 1; j <= 3; ++j) {
            if (j == 2) break;
            std::cout << "  i=" << i << " j=" << j << '\n';
        }
    }
    return 0;
}
```

OUTPUT

```
continue skips multiples of 3: 1 2 4 5 7 8 10
break at the first number divisible by 7 and 5: 35
break leaves only the inner loop:
  i=1 j=1
  i=2 j=1
  i=3 j=1
```



CONTINUE SKIPS ONE ITERATION; BREAK ENDS THE LOOP

6.5.2 goto and why to avoid it

`goto label;` jumps to `label:` anywhere in the same function. Unrestricted jumps produce tangled "spaghetti code" that is hard to follow, and in C++ goto may not jump over the initialisation of a variable. The one situation where people still argue for it, breaking out of several nested loops, is better solved with a flag, a function that `return`s, or a lambda.

GOTO VS A FUNCTION WITH RETURN main.cpp

```cpp
#include <iostream>

bool find_pair(int target, int& a, int& b) {          // return replaces a multi-level break
    for (a = 1; a < 10; ++a)
        for (b = a; b < 10; ++b)
            if (a * b == target) return true;
    return false;
}
int main() {
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j)
            if (i * j == 2) { std::cout << "goto version found i=" << i << " j=" << j << '\n'; goto
done; }
done:
    int a, b;
    if (find_pair(42, a, b)) std::cout << "function version found " << a << " x " << b << " = 42\n";
    return 0;
}
```

OUTPUT

```
goto version found i=1 j=2
function version found 6 x 7 = 42
```

6.6 Classic loop problems

6.6.1 Sum of digits, reverse a number, palindrome number

DIGITS, REVERSAL AND PALINDROMES main.cpp

```cpp
#include <iostream>

int main() {
    long long n;
    std::cout << "Number: ";
    std::cin >> n;
    long long original = n, reversed = 0;
    int sum = 0;
    while (n > 0) {
        int d = n % 10;
        sum += d;
        reversed = reversed * 10 + d;      // append the digit
        n /= 10;
    }
    std::cout << "sum of digits = " << sum << '\n';
    std::cout << "reversed      = " << reversed << '\n';
    std::cout << original << (original == reversed ? " is" : " is not") << " a palindrome\n";
    return 0;
}
```

CONSOLE (user input in gold)

```
Number: 12321
sum of digits = 9
reversed      = 12321
12321 is a palindrome
```

Step	n	digit	reversed
start	12321		0
1	1232	1	1
2	123	2	12
3	12	3	123
4	1	2	1232
5	0	1	12321

6.6.2 Factorial and Fibonacci

FACTORIAL AND FIBONACCI

main.cpp

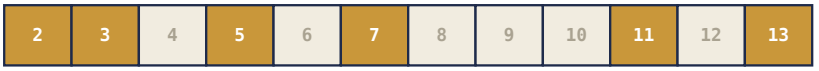
```cpp
#include <iostream>

int main() {
    int n = 20;
    unsigned long long fact = 1;
    for (int i = 2; i <= n; ++i) fact *= i;
    std::cout << n << "! = " << fact << "  (21! would overflow 64 bits)\n";

    std::cout << "First 15 Fibonacci numbers:";
    long long a = 0, b = 1;
    for (int i = 0; i < 15; ++i) {
        std::cout << ' ' << a;
        long long next = a + b;
        a = b;
        b = next;
    }
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
20! = 2432902008176640000  (21! would overflow 64 bits)
First 15 Fibonacci numbers: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

6.6.3 Prime check and Sieve of Eratosthenes

A number n > 1 is prime if no number from 2 to √n divides it; testing up to √n suffices because any factor larger than √n pairs with one smaller than it. To find **all** primes up to N, the **Sieve of Eratosthenes** is much faster: start with every number marked prime, and for each prime p cross out its multiples from p² onwards. It runs in O(N log log N).



SIEVE UP TO 13: MULTIPLES OF 2 AND 3 CROSSED OUT (GREY), PRIMES REMAIN (GOLD)

PRIME TEST AND SIEVE

main.cpp

```
#include <iostream>
#include <vector>

bool is_prime(int n) {
    if (n < 2) return false;
    for (int d = 2; d * d <= n; ++d)
        if (n % d == 0) return false;
    return true;
}
int main() {
    for (int n : {1, 2, 17, 21, 97, 7919})
        std::cout << n << (is_prime(n) ? " is prime\n" : " is not prime\n");

    const int N = 100;
    std::vector<bool> prime(N + 1, true);
    prime[0] = prime[1] = false;
    for (int p = 2; p * p <= N; ++p)
        if (prime[p])
            for (int m = p * p; m <= N; m += p) prime[m] = false;
    std::cout << "primes up to " << N << ":";
    int count = 0;
    for (int i = 2; i <= N; ++i) if (prime[i]) { std::cout << ' ' << i; ++count; }
    std::cout << "\n(" << count << " primes)\n";
    return 0;
}
```

OUTPUT

```
1 is not prime
2 is prime
17 is prime
21 is not prime
97 is prime
7919 is prime
primes up to 100: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
(25 primes)
```

6.6.4 Armstrong, perfect and strong numbers

Kind	Definition	Example
Armstrong (narcissistic)	sum of its digits each raised to the number of digits equals the number	153 = $1^3 + 5^3 + 3^3$
Perfect	sum of its proper divisors equals the number	28 = 1 + 2 + 4 + 7 + 14
Strong	sum of the factorials of its digits equals the number	145 = 1! + 4! + 5!

SPECIAL NUMBERS

main.cpp

```cpp
#include <iostream>

int digit_count(int n) { int c = 0; do { ++c; n /= 10; } while (n); return c; }
bool armstrong(int n) {
    int k = digit_count(n), sum = 0;
    for (int t = n; t > 0; t /= 10) {
        int p = 1, d = t % 10;
        for (int i = 0; i < k; ++i) p *= d;
        sum += p;
    }
    return sum == n;
}
bool perfect(int n) {
    int sum = 1;
    for (int d = 2; d * d <= n; ++d)
        if (n % d == 0) { sum += d; if (d != n / d) sum += n / d; }
    return n > 1 && sum == n;
}
bool strong(int n) {
    const int fact[] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};
    int sum = 0;
    for (int t = n; t > 0; t /= 10) sum += fact[t % 10];
    return n > 0 && sum == n;
}
int main() {
    std::cout << "Armstrong below 10000:";
    for (int i = 1; i < 10000; ++i) if (armstrong(i)) std::cout << ' ' << i;
    std::cout << "\nPerfect below 10000  :";
    for (int i = 1; i < 10000; ++i) if (perfect(i)) std::cout << ' ' << i;
    std::cout << "\nStrong below 100000  :";
    for (int i = 1; i < 100000; ++i) if (strong(i)) std::cout << ' ' << i;
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
Armstrong below 10000: 1 2 3 4 5 6 7 8 9 153 370 371 407 1634 8208 9474
Perfect below 10000  : 6 28 496 8128
Strong below 100000  : 1 2 145 40585
```

6.6.5 GCD and LCM

The **greatest common divisor** is found with Euclid's algorithm: `gcd(a, b) = gcd(b, a % b)`, until b is 0. The **least common multiple** follows from `lcm(a, b) = a / gcd(a, b) * b` (divide first to avoid overflow). C++17 provides `std::gcd` and `std::lcm` in `<numeric>`.

a	b	a % b
252	105	42
105	42	21
42	21	0
21	0	→ gcd = 21

EUCLID'S ALGORITHM AND THE STANDARD FUNCTIONS

main.cpp

```
#include <iostream>
#include <numeric>

int main() {
    int a = 252, b = 105;
    int x = a, y = b;
    while (y != 0) {              // Euclid
        int r = x % y;
        x = y;
        y = r;
    }
    std::cout << "gcd(" << a << ", " << b << ") = " << x << ", lcm = " << a / x * b << '\n';
    std::cout << "std::gcd = " << std::gcd(a, b) << ", std::lcm = " << std::lcm(a, b) << '\n';
    return 0;
}
```

OUTPUT

```
gcd(252, 105) = 21, lcm = 1260
std::gcd = 21, std::lcm = 1260
```

Use if/else for conditions and ranges, switch for many fixed integral or enum values (with break, and [[fallthrough]] when falling through on purpose). Choose for for counting, range-based for for containers, while for unknown repetitions and do-while for at-least-once. Nested loops build patterns, break/continue/return control them, and goto is best avoided.

PRACTICE CHECKPOINT

Write a **menu-driven program using switch inside a do-while loop**, plus **five pattern programs**.

SAMPLE SOLUTION

main.cpp

```
#include <iostream>
#include <numeric>

void patterns(int n) {
    std::cout << "1) Right triangle\n";
    for (int i = 1; i <= n; ++i) { for (int j = 0; j < i; ++j) std::cout << '*'; std::cout << '\n'; }
    std::cout << "2) Pyramid\n";
    for (int i = 1; i <= n; ++i) std::cout << std::string(n - i, ' ') << std::string(2 * i - 1, '*')
<< '\n';
    std::cout << "3) Inverted number triangle\n";
    for (int i = n; i >= 1; --i) { for (int j = 1; j <= i; ++j) std::cout << j; std::cout << '\n'; }
    std::cout << "4) Floyd's triangle\n";
    for (int i = 1, k = 1; i <= n; ++i) { for (int j = 0; j < i; ++j) std::cout << k++ << ' ';
std::cout << '\n'; }
    std::cout << "5) Hollow diamond\n";
    for (int i = 1; i <= 2 * n - 1; ++i) {
        int r = i <= n ? i : 2 * n - i;
        std::cout << std::string(n - r, ' ');
        for (int j = 1; j <= 2 * r - 1; ++j) std::cout << (j == 1 || j == 2 * r - 1 ? '*' : ' ');
        std::cout << '\n';
    }
}
int main() {
    int choice;
    do {
        std::cout << "\n1 Even/odd  2 Factorial  3 GCD  4 Patterns  0 Exit\nChoice: ";
```

```
        if (!(std::cin >> choice)) break;
        switch (choice) {
        case 1: {
            int n; std::cout << "n: "; std::cin >> n;
            std::cout << n << (n % 2 == 0 ? " is even\n" : " is odd\n");
            break;
        }
        case 2: {
            int n; std::cout << "n: "; std::cin >> n;
            unsigned long long f = 1;
            for (int i = 2; i <= n; ++i) f *= i;
            std::cout << n << "! = " << f << '\n';
            break;
        }
        case 3: {
            int a, b; std::cout << "a b: "; std::cin >> a >> b;
            std::cout << "gcd = " << std::gcd(a, b) << '\n';
            break;
        }
        case 4: {
            int n; std::cout << "size: "; std::cin >> n;
            patterns(n);
            break;
        }
        case 0:
            std::cout << "Goodbye!\n";
            break;
        default:
            std::cout << "Invalid choice\n";
        }
    } while (choice != 0);
    return 0;
}
```

CONSOLE (user input in gold)

```
1 Even/odd  2 Factorial  3 GCD  4 Patterns  0 Exit
Choice: 1
n: 17
17 is odd

1 Even/odd  2 Factorial  3 GCD  4 Patterns  0 Exit
Choice: 2
n: 10
10! = 3628800

1 Even/odd  2 Factorial  3 GCD  4 Patterns  0 Exit
Choice: 3
a b: 84 36
gcd = 12

1 Even/odd  2 Factorial  3 GCD  4 Patterns  0 Exit
Choice: 9
Invalid choice

1 Even/odd  2 Factorial  3 GCD  4 Patterns  0 Exit
Choice: 4
size: 4
1) Right triangle
*
**
***
****
2) Pyramid
   *
  ***
 *****
*******
3) Inverted number triangle
1234
123
12
1
4) Floyd's triangle
1
2 3
4 5 6
7 8 9 10
5) Hollow diamond
   *
  * *
 *   *
*     *
 *   *
  * *
   *

1 Even/odd  2 Factorial  3 GCD  4 Patterns  0 Exit
Choice: 0
Goodbye!
```

MODULE 07 · BEGINNER

Arrays and vectors

Minimum duration **3 days** | Level **Beginner**

Arrays hold a fixed number of elements of one type in consecutive memory. C++ inherited C-style arrays, added the safer fixed-size std::array, and provides std::vector, the resizable array that modern C++ uses by default. This module covers all three and the classic searching, sorting and matrix algorithms.

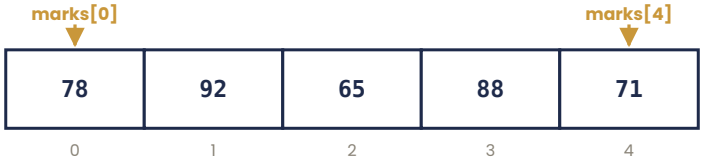
IN THIS MODULE

7.1 C-style arrays
7.2 std::array (C++11)
7.3 std::vector (introduction)
7.4 Array algorithms

7.1 C-style arrays

7.1.1 Declaration and initialization

`type name[size];` creates `size` elements of `type` side by side in memory. The size must be a compile-time constant. Elements are numbered from **0** to **size − 1**.



INT MARKS[5]: FIVE INTS IN A ROW, INDEXES 0 TO 4 (20 BYTES)

WAYS TO INITIALISE AN ARRAY

main.cpp

```cpp
#include <iostream>

int main() {
    int marks[5] = {78, 92, 65, 88, 71};     // full initializer
    int partial[5] = {1, 2};                  // rest are zero: 1 2 0 0 0
    int zeros[5] = {};                        // all zero
    int primes[] = {2, 3, 5, 7, 11, 13};      // size deduced: 6
    double prices[3]{9.5, 12.0, 4.25};        // brace initialization (C++11)
    char word[] = "C++";                      // 4 chars including '\0'
    int garbage[3];                           // uninitialized local: indeterminate values

    for (int x : partial) std::cout << x << ' ';
    std::cout << "| primes has " << sizeof(primes) / sizeof(primes[0]) << " elements"
              << " | prices[2] = " << prices[2] << " | sizeof word = " << sizeof word << '\n';
    (void)marks; (void)zeros; (void)garbage;
    return 0;
}
```

OUTPUT

```
1 2 0 0 0 | primes has 6 elements | prices[2] = 4.25 | sizeof word = 4
```

7.1.2 Indexing, traversal and size with sizeof / std::size

`arr[i]` accesses element i. The element count of an array is `sizeof(arr) / sizeof(arr[0])` in C style, or `std::size(arr)` (C++17, in `<iterator>` and every container header) which is clearer and refuses to compile for pointers.

TRAVERSING AN ARRAY

main.cpp

```
#include <iostream>
#include <iterator>

int main() {
    int marks[] = {78, 92, 65, 88, 71};
    std::size_t n = std::size(marks);                    // 5
    int total = 0, best = marks[0];
    for (std::size_t i = 0; i < n; ++i) {                // index-based traversal
        total += marks[i];
        if (marks[i] > best) best = marks[i];
    }
    std::cout << "n = " << n << ", total = " << total << ", average = " << static_cast<double>(total)
/ n
              << ", best = " << best << '\n';
    marks[2] += 10;                                       // modify one element
    for (int m : marks) std::cout << m << ' ';            // range-based traversal
    std::cout << "\nlast element: " << marks[n - 1] << '\n';
    return 0;
}
```

OUTPUT

```
n = 5, total = 394, average = 78.8, best = 92
78 92 75 88 71
last element: 71
```

7.1.3 No bounds checking

C-style arrays do not check indexes. `marks[5]` or `marks[-1]` compiles and runs, reading or overwriting whatever memory is next to the array. This is undefined behaviour: wrong results, crashes, or security holes. std::array and std::vector offer `at()`, which checks.

READING PAST THE END

main.cpp

```cpp
#include <iostream>

int main() {
    int a[3] = {10, 20, 30};
    int secret = 42;
    std::cout << "a[2] = " << a[2] << '\n';
    std::cout << "a[3] = " << a[3] << "   <- out of bounds: undefined behaviour\n";
    (void)secret;
    return 0;
}
```

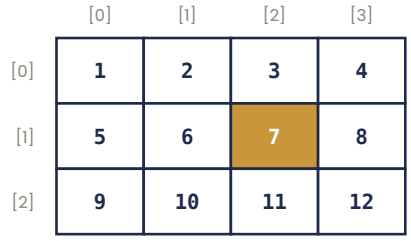
OUTPUT (with AddressSanitizer; report trimmed)

```
$ g++ -g -fsanitize=address main.cpp -o prog && ./prog
a[2] = 30
=================================================================
==2036==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7fee5fd0002c
READ of size 4 at 0x7fee5fd0002c thread T0
    #0 0x565491062498 in main main.cpp:7
Address 0x7fee5fd0002c is located in stack of thread T0 at offset 44 in frame
    [32, 44) 'a' (line 4) <== Memory access at offset 44 overflows this variable
SUMMARY: AddressSanitizer: stack-buffer-overflow main.cpp:7 in main
```

Built normally, the same program usually prints some garbage number for `a[3]` and carries on, which is far more dangerous than a crash. Compile with `-fsanitize=address` while developing.

7.1.4 Multi-dimensional arrays

`int m[3][4];` is an array of 3 rows, each an array of 4 ints, stored row after row (**row-major order**). Use nested loops: the outer one for rows, the inner one for columns.



INT M[3][4]: M[1][2] IS 7

A 2D ARRAY WITH ROW AND COLUMN SUMS

main.cpp

```cpp
#include <iomanip>
#include <iostream>

int main() {
    int m[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
    int row_sum[3] = {}, col_sum[4] = {};
    for (int r = 0; r < 3; ++r) {
        for (int c = 0; c < 4; ++c) {
            std::cout << std::setw(4) << m[r][c];
            row_sum[r] += m[r][c];
            col_sum[c] += m[r][c];
        }
        std::cout << "  | " << row_sum[r] << '\n';
    }
    std::cout << "--------------------\n";
    for (int c = 0; c < 4; ++c) std::cout << std::setw(4) << col_sum[c];
    std::cout << "\nm[1][2] = " << m[1][2] << ", sizeof m = " << sizeof m << " bytes\n";
    for (const auto& row : m) {                    // range-for works too
        for (int x : row) std::cout << x % 2;
        std::cout << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
   1   2   3   4  | 10
   5   6   7   8  | 26
   9  10  11  12  | 42
--------------------
  15  18  21  24
m[1][2] = 7, sizeof m = 48 bytes
1010 1010 1010
```

7.1.5 Passing arrays to functions (decay to pointer)

When an array is passed to a function, it **decays** to a pointer to its first element: the function receives only an address and **loses the size**. `void f(int a[])` is exactly `void f(int* a)`, so sizeof inside the function gives the pointer's size. Pass the size separately, use a reference to an array, or better, use std::array, std::vector or std::span (C++20).

ARRAY DECAY AND STD::SPAN

main.cpp

```cpp
#include <iostream>
#include <span>

int sum_c(const int a[], int n) {               // really const int* a
    std::cout << "  inside sum_c: sizeof a = " << sizeof(a) << " (a pointer!)\n";
    int s = 0;
    for (int i = 0; i < n; ++i) s += a[i];
    return s;
}
void double_all(int a[], int n) {               // changes the caller's array
    for (int i = 0; i < n; ++i) a[i] *= 2;
}
int sum_span(std::span<const int> s) {           // C++20: pointer + size together
    int t = 0;
    for (int x : s) t += x;
    return t;
}
int main() {
    int v[] = {1, 2, 3, 4, 5};
    std::cout << "in main: sizeof v = " << sizeof v << '\n';
    int total = sum_c(v, 5);
    std::cout << "sum_c = " << total << '\n';
    double_all(v, 5);
    std::cout << "after double_all, v[4] = " << v[4] << '\n';
    std::cout << "sum_span = " << sum_span(v) << " (size known: no separate n)\n";
    return 0;
}
```

OUTPUT

```
in main: sizeof v = 20
  inside sum_c: sizeof a = 8 (a pointer!)
sum_c = 15
after double_all, v[4] = 10
sum_span = 30 (size known: no separate n)
```

7.2 std::array (C++11)

7.2.1 Declaring std::array<int, 5>

`std::array<T, N>` from `<array>` wraps a fixed-size C array in a class. It has exactly the same memory layout and speed, but knows its size, can be copied and compared, does not decay to a pointer, and works with all the standard algorithms.

7.2.2 size(), at(), front(), back(), fill()

USING STD::ARRAY

main.cpp

```
#include <algorithm>
#include <array>
#include <iostream>

int main() {
    std::array<int, 5> marks = {78, 92, 65, 88, 71};
    std::cout << "size " << marks.size() << ", front " << marks.front() << ", back " << marks.back()
<< '\n';
    std::cout << "marks[1] = " << marks[1] << ", marks.at(1) = " << marks.at(1) << '\n';
    std::sort(marks.begin(), marks.end());
    for (int m : marks) std::cout << m << ' ';
    std::array<int, 5> copy = marks;                // arrays can be copied with =
    copy.fill(0);
    std::cout << "\ncopy after fill(0): " << copy[0] << copy[4] << ", original still " << marks[0] <<
'\n';
    std::cout << std::boolalpha << "marks == copy ? " << (marks == copy) << '\n';
    try {
        std::cout << marks.at(7) << '\n';           // checked access
    } catch (const std::out_of_range& e) {
        std::cout << "at(7) threw out_of_range\n";
    }
    std::array<std::array<int, 3>, 2> grid{{{1, 2, 3}, {4, 5, 6}}};
    std::cout << "grid[1][2] = " << grid[1][2] << '\n';
    return 0;
}
```

OUTPUT

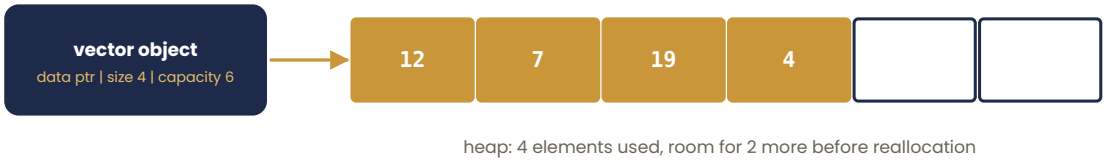
```
size 5, front 78, back 71
marks[1] = 92, marks.at(1) = 92
65 71 78 88 92
copy after fill(0): 00, original still 65
marks == copy ? false
at(7) threw out_of_range
grid[1][2] = 6
```

7.2.3 Advantages over C arrays

	C array `int a[5]`	`std::array<int, 5>`
Knows its size	no (sizeof trick only in the same scope)	`.size()` always
Passed to a function	decays to a pointer, size lost	passed as a real object (by value or reference)
Copy with =, compare with ==	no	yes
Bounds-checked access	no	`.at(i)`
Return from a function	not possible	yes
Works with STL algorithms	with std::begin/std::end	directly
Overhead	none	none

7.3 std::vector (introduction)

`std::vector<T>` from `<vector>` is a **dynamic array**: its elements are contiguous like an array, but it grows and shrinks at run time and manages its own heap memory. It is the default container in C++: "when in doubt, use vector".



A VECTOR HOLDS A POINTER TO A HEAP BLOCK PLUS ITS SIZE AND CAPACITY

7.3.1 Creating and initializing vectors

CONSTRUCTING VECTORS

main.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

void show(const std::string& label, const std::vector<int>& v) {
    std::cout << label << " (size " << v.size() << "):";
    for (int x : v) std::cout << ' ' << x;
    std::cout << '\n';
}
int main() {
    std::vector<int> a;                     // empty
    std::vector<int> b(4);                  // 4 zeros
    std::vector<int> c(4, 7);               // 4 sevens
    std::vector<int> d{4, 7};               // TWO elements: 4 and 7 (braces = list of values)
    std::vector<int> e = {10, 20, 30};
    std::vector<int> f(e);                  // copy
    std::vector<int> g(e.begin(), e.begin() + 2);   // from a range
    std::vector<std::string> names(3, "?");
    show("a", a); show("b", b); show("c", c); show("d", d); show("e", e); show("f", f); show("g", g);
    std::cout << "names: " << names[0] << names[1] << names[2] << '\n';
    return 0;
}
```

OUTPUT

```
a (size 0):
b (size 4): 0 0 0 0
c (size 4): 7 7 7 7
d (size 2): 4 7
e (size 3): 10 20 30
f (size 3): 10 20 30
g (size 2): 10 20
names: ???
```

WATCH OUT

`std::vector<int> v(4, 7);` (parentheses) means "4 copies of 7", while `std::vector<int> v{4, 7};` (braces) means "the two values 4 and 7". This difference trips up many programmers.

7.3.2 push_back(), pop_back(), size(), empty(), clear()

GROWING AND SHRINKING A VECTOR

main.cpp

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    std::cout << std::boolalpha << "empty? " << v.empty() << '\n';
    for (int i = 1; i <= 6; ++i) {
        v.push_back(i * 10);                    // add at the end
        std::cout << "push " << i * 10 << " -> size " << v.size() << ", capacity " << v.capacity() <<
'\n';
    }
    v.pop_back();                               // remove the last element
    std::cout << "after pop_back, back() = " << v.back() << ", size " << v.size() << '\n';
    v.insert(v.begin() + 1, 15);                // insert at position 1
    v.erase(v.begin());                         // remove the first
    for (int x : v) std::cout << x << ' ';
    v.clear();                                  // remove everything
    std::cout << "\nafter clear: size " << v.size() << ", empty? " << v.empty() << '\n';
    return 0;
}
```

OUTPUT

```
empty? true
push 10 -> size 1, capacity 1
push 20 -> size 2, capacity 2
push 30 -> size 3, capacity 4
push 40 -> size 4, capacity 4
push 50 -> size 5, capacity 8
push 60 -> size 6, capacity 8
after pop_back, back() = 50, size 5
15 20 30 40 50
after clear: size 0, empty? true
```

When size reaches capacity, push_back allocates a **bigger block** (GCC doubles it), moves the elements and frees the old block. This makes push_back fast on average (amortised O(1)). If you know the final size, `v.reserve(n)` allocates once up front.

7.3.3 Access with [] and at()

[] VS AT()

main.cpp

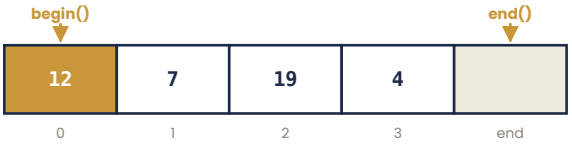
```cpp
#include <iostream>
#include <stdexcept>
#include <vector>

int main() {
    std::vector<int> v = {5, 10, 15};
    v[1] = 11;                                  // unchecked, fastest
    v.at(2) = 16;                               // checked
    std::cout << v[0] << ' ' << v.at(1) << ' ' << v.back() << '\n';
    try {
        std::cout << v.at(3) << '\n';
    } catch (const std::out_of_range& e) {
        std::cout << "at(3) -> out_of_range: " << e.what() << '\n';
    }
    return 0;
}
```

OUTPUT

```
5 11 16
at(3) -> out_of_range: vector::_M_range_check: __n (which is 3) >= this->size() (which is 3)
```

7.3.4 Iterating with index, range-for and iterators

An **iterator** is an object that points to an element of a container and can be moved to the next one. `v.begin()` points to the first element and `v.end()` one **past** the last; `*it` gives the element. Iterators are how the standard algorithms work with every container (Module 19).



BEGIN() POINTS AT THE FIRST ELEMENT; END() IS ONE PAST THE LAST

THREE WAYS TO ITERATE

main.cpp

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {12, 7, 19, 4};
    for (std::size_t i = 0; i < v.size(); ++i) std::cout << v[i] << ' ';            // 1. index
    std::cout << "| ";
    for (int x : v) std::cout << x << ' ';                                          // 2. range-for
    std::cout << "| ";
    for (auto it = v.begin(); it != v.end(); ++it) std::cout << *it << ' ';         // 3. iterators
    std::cout << "| reverse: ";
    for (auto it = v.rbegin(); it != v.rend(); ++it) std::cout << *it << ' ';       // reverse
iterators
    std::cout << '\n';
    for (auto& x : v) x *= 10;                                                      // modify via
reference
    std::cout << "after *10: " << v.front() << " .. " << v.back() << '\n';
    return 0;
}
```

OUTPUT

```
12 7 19 4 | 12 7 19 4 | 12 7 19 4 | reverse: 4 19 7 12
after *10: 120 .. 40
```

7.3.5 2D vectors vector<vector<int>>

A vector of vectors makes a matrix whose dimensions are chosen at run time. Each row is a separate vector, so rows can even have different lengths (a jagged array).

RECTANGULAR AND JAGGED 2D VECTORS

main.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <vector>

int main() {
    int rows = 3, cols = 4;
    std::vector<std::vector<int>> grid(rows, std::vector<int>(cols, 0));   // 3 x 4 zeros
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c) grid[r][c] = r * cols + c;
    for (const auto& row : grid) {
        for (int x : row) std::cout << std::setw(3) << x;
        std::cout << '\n';
    }
    std::vector<std::vector<int>> triangle;                                // jagged
    for (int i = 1; i <= 4; ++i) triangle.push_back(std::vector<int>(i, i));
    for (const auto& row : triangle) {
        for (int x : row) std::cout << x;
        std::cout << " (" << row.size() << ")\n";
    }
    return 0;
}
```

OUTPUT

```
  0  1  2  3
  4  5  6  7
  8  9 10 11
1 (1)
22 (2)
333 (3)
4444 (4)
```

7.3.6 Passing vectors to functions by reference

Passing a vector **by value** copies every element, which is slow for large vectors. Pass by `const std::vector<T>&` to read without copying, and by `std::vector<T>&` to modify the caller's vector. Returning a vector by value is cheap thanks to move semantics (Module 20).

PASSING AND RETURNING VECTORS main.cpp

```
#include <iostream>
#include <vector>

double average(const std::vector<int>& v) {       // read-only, no copy
    long long s = 0;
    for (int x : v) s += x;
    return v.empty() ? 0.0 : static_cast<double>(s) / v.size();
}
void add_bonus(std::vector<int>& v, int bonus) {  // modifies the caller's vector
    for (int& x : v) x += bonus;
}
void try_change(std::vector<int> v) {             // a copy: caller unaffected
    v[0] = -1;
}
std::vector<int> evens_up_to(int n) {             // returning by value is efficient
    std::vector<int> r;
    for (int i = 2; i <= n; i += 2) r.push_back(i);
    return r;
}
int main() {
    std::vector<int> marks = {70, 80, 90};
    add_bonus(marks, 5);
    try_change(marks);
    std::cout << "marks[0] = " << marks[0] << ", average = " << average(marks) << '\n';
    std::cout << "evens up to 10 has " << evens_up_to(10).size() << " elements\n";
    return 0;
}
```

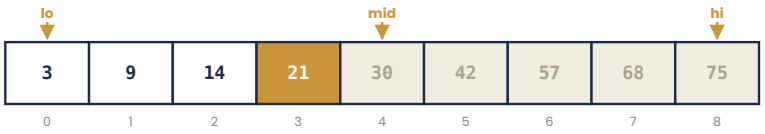
OUTPUT

```
marks[0] = 75, average = 85
evens up to 10 has 5 elements
```

7.4 Array algorithms

7.4.1 Linear and binary search

Linear search checks elements one by one: O(n), works on any data. **Binary search** works only on **sorted** data: compare with the middle element and discard the half that cannot contain the key, so each step halves the range: O(log n). A million elements need at most 20 comparisons.



BINARY SEARCH FOR 21: 21 < A[MID] = 30, SO THE RIGHT HALF (GREY) IS DISCARDED

LINEAR AND BINARY SEARCH main.cpp

```
#include <iostream>
#include <vector>

int linear_search(const std::vector<int>& v, int key) {
    for (std::size_t i = 0; i < v.size(); ++i)
        if (v[i] == key) return static_cast<int>(i);
    return -1;
}
int binary_search(const std::vector<int>& v, int key, int& steps) {
    int lo = 0, hi = static_cast<int>(v.size()) - 1;
    steps = 0;
    while (lo <= hi) {
        ++steps;
        int mid = lo + (hi - lo) / 2;           // avoids overflow of (lo + hi)
        if (v[mid] == key) return mid;
        if (v[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
int main() {
    std::vector<int> v = {3, 9, 14, 21, 30, 42, 57, 68, 75};
    std::cout << "linear 57 -> " << linear_search(v, 57) << '\n';
    for (int key : {21, 75, 50}) {
        int steps;
        int pos = binary_search(v, key, steps);
        std::cout << "binary " << key << " -> " << pos << " (" << steps << " steps)\n";
    }
    return 0;
}
```

OUTPUT

```
linear 57 -> 6
binary 21 -> 3 (4 steps)
binary 75 -> 8 (4 steps)
binary 50 -> -1 (3 steps)
```

7.4.2 Bubble, selection and insertion sort

Algorithm	Idea	Time (worst)	Stable
Bubble sort	repeatedly swap adjacent out-of-order pairs; the largest "bubbles" to the end each pass	$O(n^2)$	yes
Selection sort	find the minimum of the unsorted part and swap it to the front	$O(n^2)$	no
Insertion sort	take the next element and insert it into its place in the sorted prefix	$O(n^2)$, $O(n)$ if nearly sorted	yes

THREE ELEMENTARY SORTS main.cpp

```
#include <iostream>
#include <utility>
#include <vector>

void print(const char* name, const std::vector<int>& v) {
    std::cout << name;
    for (int x : v) std::cout << ' ' << x;
```

```
    std::cout << '\n';
}
void bubble_sort(std::vector<int>& a) {
    for (std::size_t pass = 0; pass + 1 < a.size(); ++pass) {
        bool swapped = false;
        for (std::size_t j = 0; j + 1 < a.size() - pass; ++j)
            if (a[j] > a[j + 1]) { std::swap(a[j], a[j + 1]); swapped = true; }
        if (!swapped) break;                        // already sorted: stop early
    }
}
void selection_sort(std::vector<int>& a) {
    for (std::size_t i = 0; i + 1 < a.size(); ++i) {
        std::size_t min_i = i;
        for (std::size_t j = i + 1; j < a.size(); ++j)
            if (a[j] < a[min_i]) min_i = j;
        std::swap(a[i], a[min_i]);
    }
}
void insertion_sort(std::vector<int>& a) {
    for (std::size_t i = 1; i < a.size(); ++i) {
        int key = a[i];
        std::size_t j = i;
        while (j > 0 && a[j - 1] > key) { a[j] = a[j - 1]; --j; }   // shift bigger ones right
        a[j] = key;
    }
}
int main() {
    const std::vector<int> data = {29, 10, 14, 37, 13, 5, 41};
    auto a = data, b = data, c = data;
    bubble_sort(a); selection_sort(b); insertion_sort(c);
    print("bubble   :", a);
    print("selection:", b);
    print("insertion:", c);
    return 0;
}
```

OUTPUT

```
bubble   : 5 10 13 14 29 37 41
selection: 5 10 13 14 29 37 41
insertion: 5 10 13 14 29 37 41
```

Insertion sort pass	Array (sorted prefix | rest)
start	29 | 10 14 37 13 5 41
insert 10	10 29 | 14 37 13 5 41
insert 14	10 14 29 | 37 13 5 41
insert 37	10 14 29 37 | 13 5 41
insert 13	10 13 14 29 37 | 5 41
insert 5	5 10 13 14 29 37 | 41
insert 41	5 10 13 14 29 37 41

7.4.3 Common problems: max/min, second largest, reverse, rotate, remove duplicates, frequency count, prefix sums

SEVEN CLASSIC ARRAY PROBLEMS

main.cpp

```
#include <iostream>
#include <limits>
#include <unordered_map>
#include <vector>

void print(const char* t, const std::vector<int>& v) { std::cout << t; for (int x : v) std::cout << '
' << x; std::cout << '\n'; }

int main() {
    std::vector<int> v = {4, 9, 2, 9, 7, 4, 1};

    int mx = v[0], mn = v[0];                                   // max and min in one pass
    for (int x : v) { if (x > mx) mx = x; if (x < mn) mn = x; }
    std::cout << "max " << mx << ", min " << mn << '\n';

    int first = std::numeric_limits<int>::min(), second = first;  // second largest distinct
    for (int x : v) {
        if (x > first) { second = first; first = x; }
        else if (x > second && x != first) second = x;
    }
    std::cout << "second largest " << second << '\n';

    std::vector<int> r = v;                                     // reverse in place
    for (std::size_t i = 0, j = r.size() - 1; i < j; ++i, --j) std::swap(r[i], r[j]);
    print("reversed:", r);

    std::vector<int> rot(v.size());                             // rotate left by k
    std::size_t k = 2;
    for (std::size_t i = 0; i < v.size(); ++i) rot[i] = v[(i + k) % v.size()];
    print("rotated left by 2:", rot);

    std::vector<int> uniq;                                      // remove duplicates, keep order
    std::unordered_map<int, int> freq;
    for (int x : v) if (freq[x]++ == 0) uniq.push_back(x);
    print("without duplicates:", uniq);
    std::cout << "frequency:";
    for (int x : uniq) std::cout << ' ' << x << 'x' << freq[x];
    std::cout << '\n';

    std::vector<int> prefix(v.size() + 1, 0);                   // prefix sums
    for (std::size_t i = 0; i < v.size(); ++i) prefix[i + 1] = prefix[i] + v[i];
    print("prefix sums:", prefix);
    std::cout << "sum of v[2..5] = prefix[6] - prefix[2] = " << prefix[6] - prefix[2] << '\n';
    return 0;
}
```

OUTPUT

```
max 9, min 1
second largest 7
reversed: 1 4 7 9 2 9 4
rotated left by 2: 2 9 7 4 1 4 9
without duplicates: 4 9 2 7 1
frequency: 4x2 9x2 2x1 7x1 1x1
prefix sums: 0 4 13 15 24 31 35 36
sum of v[2..5] = prefix[6] - prefix[2] = 22
```

NOTE

Prefix sums answer "sum of a range" questions in O(1) each after one O(n) pass: the sum of elements i..j is `prefix[j+1] - prefix[i]`. This trick appears constantly in competitive programming.

7.4.4 Matrix problems: addition, multiplication, transpose, spiral print

Matrix multiplication: if A is n × m and B is m × p, then C = A × B is n × p and `C[i][j] = Σ A[i][k] × B[k][j]` over k, three nested loops, O(n·m·p). The **transpose** swaps rows and columns. A **spiral print** walks the outer ring (top row, right column, bottom row, left column) and then moves inwards.

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	12	13	14	5
[2]	11	16	15	6
[3]	10	9	8	7

VISITING ORDER OF A SPIRAL TRAVERSAL: 1 → 16

MATRIX OPERATIONS main.cpp

```
#include <iomanip>
#include <iostream>
#include <vector>

using Matrix = std::vector<std::vector<int>>;

void print(const char* title, const Matrix& m) {
    std::cout << title << '\n';
    for (const auto& row : m) { for (int x : row) std::cout << std::setw(5) << x; std::cout << '\n'; }
}
Matrix add(const Matrix& a, const Matrix& b) {
    Matrix c = a;
    for (std::size_t i = 0; i < a.size(); ++i)
        for (std::size_t j = 0; j < a[0].size(); ++j) c[i][j] += b[i][j];
    return c;
}
Matrix multiply(const Matrix& a, const Matrix& b) {
    std::size_t n = a.size(), m = b.size(), p = b[0].size();
    Matrix c(n, std::vector<int>(p, 0));
    for (std::size_t i = 0; i < n; ++i)
        for (std::size_t j = 0; j < p; ++j)
            for (std::size_t k = 0; k < m; ++k) c[i][j] += a[i][k] * b[k][j];
    return c;
}
Matrix transpose(const Matrix& a) {
    Matrix t(a[0].size(), std::vector<int>(a.size()));
    for (std::size_t i = 0; i < a.size(); ++i)
        for (std::size_t j = 0; j < a[0].size(); ++j) t[j][i] = a[i][j];
    return t;
}
void spiral(const Matrix& m) {
    int top = 0, bottom = static_cast<int>(m.size()) - 1, left = 0, right = static_cast<int>
(m[0].size()) - 1;
    std::cout << "spiral:";
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; ++j) std::cout << ' ' << m[top][j];
        ++top;
        for (int i = top; i <= bottom; ++i) std::cout << ' ' << m[i][right];
```

```
        --right;
        if (top <= bottom) { for (int j = right; j >= left; --j) std::cout << ' ' << m[bottom][j]; --
bottom; }
        if (left <= right) { for (int i = bottom; i >= top; --i) std::cout << ' ' << m[i][left];
++left; }
    }
    std::cout << '\n';
}
int main() {
    Matrix a = {{1, 2, 3}, {4, 5, 6}};
    Matrix b = {{7, 8}, {9, 10}, {11, 12}};
    print("A + A:", add(a, a));
    print("A x B (2x3 times 3x2):", multiply(a, b));
    print("transpose(A):", transpose(a));
    spiral({{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15, 16}});
    return 0;
}
```

OUTPUT

```
A + A:
    2    4    6
    8   10   12
A x B (2x3 times 3x2):
   58   64
  139  154
transpose(A):
    1    4
    2    5
    3    6
spiral: 1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10
```

7.4.5 std::sort and std::reverse (introduced; detailed in Module 19)

In real code you rarely write sorts yourself. `<algorithm>` provides `std::sort` (O(n log n), typically introsort), `std::reverse`, `std::binary_search`, `std::lower_bound`, `std::max_element` and many more, all working on iterator ranges of any container.

A TASTE OF THE ALGORITHM LIBRARY

main.cpp

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> v = {29, 10, 14, 37, 13, 5, 41};
    std::sort(v.begin(), v.end());                          // ascending
    for (int x : v) std::cout << x << ' ';
    std::sort(v.begin(), v.end(), std::greater<int>());     // descending
    std::cout << "| ";
    for (int x : v) std::cout << x << ' ';
    std::reverse(v.begin(), v.end());                       // back to ascending
    std::cout << "\nbinary_search 37: " << std::boolalpha << std::binary_search(v.begin(), v.end(),
37);
    auto it = std::lower_bound(v.begin(), v.end(), 20);     // first element >= 20
    std::cout << ", first >= 20 is " << *it << " at index " << (it - v.begin());
    std::cout << "\nmax " << *std::max_element(v.begin(), v.end())
              << ", sum " << std::accumulate(v.begin(), v.end(), 0) << '\n';
    return 0;
}
```

OUTPUT

```
5 10 13 14 29 37 41 | 41 37 29 14 13 10 5
binary_search 37: true, first >= 20 is 29 at index 4
max 41, sum 149
```

C arrays are fixed-size, unchecked and decay to pointers; std::array adds size(), at(), copying and comparison at no cost; std::vector is the resizable default: push_back, size, at, iterators, and pass it by (const) reference. Know linear vs binary search and the O(n²) sorts, then use std::sort and the other algorithms in real code.

PRACTICE CHECKPOINT

Read **n numbers into a vector**, sort them with **insertion sort**, **binary-search** for a value, then **compare your result with std::sort**.

SAMPLE SOLUTION

main.cpp

```
#include <algorithm>
#include <iostream>
#include <vector>

void insertion_sort(std::vector<int>& a) {
    for (std::size_t i = 1; i < a.size(); ++i) {
        int key = a[i];
        std::size_t j = i;
        while (j > 0 && a[j - 1] > key) { a[j] = a[j - 1]; --j; }
        a[j] = key;
    }
}
int binary_search(const std::vector<int>& a, int key) {
    int lo = 0, hi = static_cast<int>(a.size()) - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == key) return mid;
```

```
        if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
int main() {
    int n;
    std::cout << "How many numbers? ";
    std::cin >> n;
    std::vector<int> nums(n);
    std::cout << "Enter " << n << " numbers: ";
    for (int& x : nums) std::cin >> x;

    std::vector<int> mine = nums, library = nums;
    insertion_sort(mine);
    std::sort(library.begin(), library.end());

    std::cout << "insertion sort: ";
    for (int x : mine) std::cout << x << ' ';
    std::cout << "\nstd::sort     : ";
    for (int x : library) std::cout << x << ' ';
    std::cout << "\nresults " << (mine == library ? "match" : "DIFFER") << '\n';

    int key;
    std::cout << "Value to find: ";
    std::cin >> key;
    int pos = binary_search(mine, key);
    if (pos >= 0) std::cout << key << " found at index " << pos << " of the sorted vector\n";
    else std::cout << key << " not found\n";
    bool lib = std::binary_search(library.begin(), library.end(), key);
    std::cout << "std::binary_search agrees: " << std::boolalpha << ((pos >= 0) == lib) << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
How many numbers? 8
Enter 8 numbers: 42 17 8 99 23 4 61 15
insertion sort: 4 8 15 17 23 42 61 99
std::sort     : 4 8 15 17 23 42 61 99
results match
Value to find: 23
23 found at index 4 of the sorted vector
std::binary_search agrees: true
```

MODULE 08 · BEGINNER

Strings

Minimum duration **3 days** | Level **Beginner**

Text is everywhere: names, messages, file contents, commands. C++ supports C-style character arrays inherited from C and the much safer std::string class, plus the lightweight std::string_view. This module covers all three, character handling, conversions between strings and numbers, and the classic string problems.

IN THIS MODULE

8.1 C-style strings
8.2 std::string
8.3 std::string methods
8.4 Characters
8.5 Conversions
8.6 std::string_view (C++17)
8.7 Classic string problems

8.1 C-style strings

8.1.1 Character arrays and the null terminator

In C, a string is an array of `char` that ends with the **null character** `'\0'` (value 0). Functions find the end of the text by searching for that terminator, so the array must always have room for it: "Asha" needs 5 bytes. String literals such as `"Asha"` are such arrays, stored in read-only memory.

CHAR NAME[8] = "ASHA"; THE TERMINATOR MARKS THE END; THE REST IS UNUSED

C-STYLE STRINGS

main.cpp

```cpp
#include <iostream>

int main() {
    char name[8] = "Asha";              // 4 letters + '\0', 3 bytes spare
    char greeting[] = "Hi";             // size 3 deduced
    const char* literal = "C++";        // pointer to a read-only literal
    std::cout << name << ' ' << greeting << ' ' << literal << '\n';
    std::cout << "sizeof name = " << sizeof name << ", sizeof greeting = " << sizeof greeting << '\n';
    for (int i = 0; name[i] != '\0'; ++i)             // walk until the terminator
        std::cout << name[i] << '(' << static_cast<int>(name[i]) << ") ";
    std::cout << '\n';
    name[0] = 'U';                      // arrays are modifiable; literals are not
    std::cout << name << '\n';
    return 0;
}
```

OUTPUT

```
Asha Hi C++
sizeof name = 8, sizeof greeting = 3
A(65) s(115) h(104) a(97)
Usha
```

8.1.2 <cstring> functions: strlen, strcpy, strcat, strcmp

Function	Purpose	Danger
`std::strlen(s)`	number of characters before \0	undefined if there is no terminator
`std::strcpy(dst, src)`	copy src into dst	overflows dst if it is too small
`std::strcat(dst, src)`	append src to dst	overflows dst if it is too small
`std::strcmp(a, b)`	<0, 0 or >0 like alphabetical comparison	none (if both terminated)
`std::strncpy`, `std::strncat`	bounded versions	strncpy may leave no terminator
`std::strchr`, `std::strstr`	find a char / substring	

THE CLASSIC C STRING FUNCTIONS

main.cpp

```cpp
#include <cstring>
#include <iostream>

int main() {
    char first[20] = "Learn";
    char full[40];
    std::strcpy(full, first);                    // full = "Learn"
    std::strcat(full, " Being Forward");         // full = "Learn Being Forward"
    std::cout << full << " (" << std::strlen(full) << " chars)\n";
    std::cout << "strcmp(\"apple\", \"apply\") = " << std::strcmp("apple", "apply") << '\n';
    std::cout << "strcmp(\"same\", \"same\")  = " << std::strcmp("same", "same") << '\n';
    if (const char* p = std::strstr(full, "Being")) std::cout << "\"Being\" found at index " << p -
full << '\n';
    return 0;
}
```

OUTPUT

```
Learn Being Forward (19 chars)
strcmp("apple", "apply") = -1
strcmp("same", "same")  = 0
"Being" found at index 6
```

8.1.3 Risks of C-strings

- **Buffer overflow**: strcpy and strcat do not know the destination size; copying a long name into a small array writes past its end (a classic security hole).
- **Missing terminator**: functions run past the end of the array looking for \0.
- **No assignment or comparison**: `a = b` does not compile for arrays and `a == b` compares addresses, not text.
- **Manual memory management** for strings of unknown length.
- **Slow length**: strlen walks the whole string every time.

WHY C STRINGS ARE ERROR-PRONE

main.cpp

```cpp
#include <cstring>
#include <iostream>

int main() {
    const char* a = "hello";
    char b[] = "hello";
    std::cout << std::boolalpha << "a == b ? " << (a == b) << "   (compares addresses!)\n";
    std::cout << "strcmp(a, b) == 0 ? " << (std::strcmp(a, b) == 0) << '\n';
    char small[6];
    const char* input = "Bartholomew";
    if (std::strlen(input) < sizeof small) std::strcpy(small, input);   // must check by hand
    else std::cout << "input too long for the buffer: refused\n";
    return 0;
}
```

OUTPUT

```
a == b ? false   (compares addresses!)
strcmp(a, b) == 0 ? true
input too long for the buffer: refused
```

std::string solves all of these. Use C-strings only when talking to C APIs (and then convert with `c_str()`).

8.2 std::string

`std::string` from `<string>` is a class that owns and manages a sequence of characters: it grows automatically, knows its length, copies with `=`, compares with `==` and `<`, concatenates with `+`, and frees its memory automatically. Short strings (up to 15 characters in GCC) are stored inside the object itself without any heap allocation (**small string optimisation**).

8.2.1 Creating and initializing strings

WAYS TO CREATE A STRING

main.cpp

```cpp
#include <iostream>
#include <string>

using namespace std::string_literals;

int main() {
    std::string a;                      // empty
    std::string b = "Asha";             // from a literal
    std::string c("Ravi");
    std::string d(5, '*');              // "*****"
    std::string e = b;                  // copy
    std::string f(b, 1, 2);             // substring of b from index 1, length 2: "sh"
    std::string g = "Hello"s + ", C++"; // "s" suffix makes a std::string literal
    std::string h{'L', 'B', 'F'};       // from characters
    std::cout << '[' << a << "] " << b << ' ' << c << ' ' << d << ' ' << e << ' ' << f << ' ' << g <<
' ' << h << '\n';
    std::cout << std::boolalpha << "a.empty() = " << a.empty() << ", sizeof(std::string) = " <<
sizeof(std::string) << '\n';
    return 0;
}
```

OUTPUT

```
[] Asha Ravi ***** Asha sh Hello, C++ LBF
a.empty() = true, sizeof(std::string) = 32
```

8.2.2 Input with cin and getline

`std::cin >> s` reads one word; `std::getline(std::cin, s)` reads a whole line (Module 5.2). Both resize the string automatically: no buffer sizes to worry about.

READING STRINGS

main.cpp

```
#include <iostream>
#include <string>

int main() {
    std::string first, full;
    std::cout << "First name: ";
    std::cin >> first;
    std::cout << "Full address: ";
    std::getline(std::cin >> std::ws, full);
    std::cout << "Hello " << first << ", you live at \"" << full << "\" (" << full.size() << "
chars)\n";
    return 0;
}
```

CONSOLE (user input in gold)

```
First name: Meera
Full address: Flat 4B, Lake View Apartments, Kochi
Hello Meera, you live at "Flat 4B, Lake View Apartments, Kochi" (36 chars)
```

8.2.3 Length, indexing, at(), front(), back()

ACCESSING CHARACTERS

main.cpp

```
#include <iostream>
#include <string>

int main() {
    std::string s = "Bengaluru";
    std::cout << "size " << s.size() << ", length " << s.length() << '\n';       // same thing
    std::cout << "s[0] = " << s[0] << ", s.at(3) = " << s.at(3) << '\n';
    std::cout << "front " << s.front() << ", back " << s.back() << '\n';
    s[0] = 'b';                                  // strings are mutable
    s.back() = 'U';
    std::cout << s << '\n';
    for (std::size_t i = 0; i < s.size(); i += 2) std::cout << s[i];   // every second letter
    std::cout << '\n';
    try { s.at(20); } catch (const std::out_of_range&) { std::cout << "at(20) -> out_of_range\n"; }
    return 0;
}
```

OUTPUT

```
size 9, length 9
s[0] = B, s.at(3) = g
front B, back u
bengalurU
bnauU
at(20) -> out_of_range
```

8.2.4 Concatenation with + and append()

JOINING STRINGS

main.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string first = "Lohit", last = "Kumar";
    std::string full = first + " " + last;          // + creates a new string
    full += ", LBF";                                  // += appends in place
    full.append(" (founder)");
    full.push_back('!');
    std::cout << full << '\n';
    std::string line;
    for (int i = 1; i <= 5; ++i) line += std::to_string(i) + (i < 5 ? "-" : "");
    std::cout << line << '\n';
    // std::string bad = "Hello" + " World";   error: cannot add two const char* pointers
    std::string ok = std::string("Hello") + " World";
    std::cout << ok << '\n';
    return 0;
}
```

OUTPUT

```
Lohit Kumar, LBF (founder)!
1-2-3-4-5
Hello World
```

8.2.5 Comparison operators and compare()

`==`, `!=`, `<`, `>` (and `<=>`) compare strings **lexicographically**: character by character by their codes, like a dictionary, but uppercase letters (65-90) come before lowercase (97-122). `a.compare(b)` returns a negative number, 0 or a positive number.

COMPARING STRINGS

main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::string a = "apple", b = "apply", c = "Apple";
    std::cout << std::boolalpha;
    std::cout << "apple == apple : " << (a == "apple") << '\n';
    std::cout << "apple <  apply : " << (a < b) << '\n';
    std::cout << "Apple <  apple : " << (c < a) << "  (uppercase sorts first)\n";
    std::cout << "compare        : " << a.compare(b) << ' ' << a.compare(a) << ' ' << b.compare(a) <<
'\n';
    std::vector<std::string> names = {"ravi", "Asha", "meera", "Kabir"};
    std::sort(names.begin(), names.end());
    for (const auto& n : names) std::cout << n << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
apple == apple : true
apple <  apply : true
Apple <  apple : true  (uppercase sorts first)
compare        : -20 0 20
Asha Kabir meera ravi
```

8.3 std::string methods

Positions are indexes starting at 0. Search functions return the position found, or the special value `std::string::npos` (the largest `size_t`) when nothing is found.

8.3.1 substr()

8.3.2 find(), rfind(), find_first_of()

SUBSTR AND THE FIND FAMILY

main.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string email = "lohit.kumar@learnbeingforward.in";
    std::size_t at = email.find('@');
    std::string user = email.substr(0, at);             // from 0, length at
    std::string domain = email.substr(at + 1);          // from at+1 to the end
    std::cout << "user = " << user << ", domain = " << domain << '\n';

    std::string s = "the cat sat on the mat";
    std::cout << "find(\"the\")       = " << s.find("the") << '\n';
    std::cout << "rfind(\"the\")      = " << s.rfind("the") << '\n';
    std::cout << "find(\"at\", 6)     = " << s.find("at", 6) << "  (search from index 6)\n";
    std::cout << "find_first_of(\"aeiou\") = " << s.find_first_of("aeiou") << '\n';
    std::cout << "find_last_not_of(\"tam \") = " << s.find_last_not_of("tam ") << '\n';
    if (s.find("dog") == std::string::npos) std::cout << "\"dog\" not found (npos)\n";

    std::size_t pos = 0; int count = 0;                  // count occurrences
    while ((pos = s.find("at", pos)) != std::string::npos) { ++count; pos += 2; }
    std::cout << "\"at\" occurs " << count << " times\n";
    return 0;
}
```

OUTPUT

```
user = lohit.kumar, domain = learnbeingforward.in
find("the")       = 0
rfind("the")      = 15
find("at", 6)     = 9  (search from index 6)
find_first_of("aeiou") = 2
find_last_not_of("tam ") = 17
"dog" not found (npos)
"at" occurs 3 times
```

8.3.3 insert(), erase(), replace()

8.3.4 push_back(), pop_back()

EDITING STRINGS IN PLACE

main.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string s = "C Programming";
    s.insert(1, "++");                      // "C++ Programming"
    std::cout << s << '\n';
    s.erase(3, 4);                          // remove 4 chars at index 3: "C++gramming"
    std::cout << s << '\n';
    s.replace(0, 3, "Pro");                 // replace 3 chars at 0: "Programming"
    std::cout << s << '\n';
    s.push_back('!');
    s.pop_back();
    s.pop_back();                           // remove the last 'g'
    std::cout << s << '\n';

    std::string text = "red, green, red, blue";          // replace every occurrence
    for (std::size_t p = text.find("red"); p != std::string::npos; p = text.find("red", p + 6))
        text.replace(p, 3, "orange");
    std::cout << text << '\n';
    std::string padded = "   trim me   ";
    padded.erase(0, padded.find_first_not_of(' '));
    padded.erase(padded.find_last_not_of(' ') + 1);
    std::cout << '[' << padded << "]\n";
    return 0;
}
```

OUTPUT

```
C++ Programming
C++gramming
Programming
Programmin
orange, green, orange, blue
[trim me]
```

8.3.5 starts_with(), ends_with() (C++20), contains() (C++23)

PREFIX, SUFFIX AND SUBSTRING TESTS

main.cpp

```
#include <iostream>
#include <string>

int main() {
    std::string file = "report_2026.pdf";
    std::cout << std::boolalpha;
    std::cout << "starts_with(\"report\") : " << file.starts_with("report") << '\n';
    std::cout << "ends_with(\".pdf\")     : " << file.ends_with(".pdf") << '\n';
    std::cout << "contains(\"2026\")      : " << file.contains("2026") << "  (C++23)\n";
    std::cout << "contains('_')          : " << file.contains('_') << '\n';
    return 0;
}
```

OUTPUT

```
starts_with("report") : true
ends_with(".pdf")     : true
contains("2026")      : true  (C++23)
contains('_')          : true
```

8.3.6 c_str() and data()

`s.c_str()` returns a `const char*` to a null-terminated copy of the text, for passing to C functions (printf, fopen, OS APIs). `s.data()` returns the same pointer (non-const since C++17). The pointer is valid only until the string is modified or destroyed.

INTERFACING WITH C

main.cpp

```
#include <cstdio>
#include <cstring>
#include <string>

int main() {
    std::string name = "Asha";
    std::printf("printf needs a C string: %s (strlen %zu)\n", name.c_str(),
std::strlen(name.c_str()));
    char* p = name.data();
    p[0] = 'U';                                  // allowed: data() is writable since C++17
    std::printf("%s\n", name.c_str());
    return 0;
}
```

OUTPUT

```
printf needs a C string: Asha (strlen 4)
Usha
```

Method	Does	Method	Does
`size()`, `length()`	number of characters	`find(x, pos)`	first position of x at or after pos
`empty()`	true if size is 0	`rfind(x)`	last position of x
`substr(pos, n)`	copy of n chars from pos	`insert(pos, s)`	insert s at pos
`append(s)`, `+=`	add at the end	`erase(pos, n)`	remove n chars
`clear()`	make empty	`replace(pos, n, s)`	replace n chars with s

Method	Does	Method	Does
`compare(s)`	<0, 0, >0	`resize(n, c)`	shorten or pad with c

8.4 Characters

8.4.1 <cctype>: isalpha, isdigit, isspace, toupper, tolower

Function	True for	Function	Returns
`std::isalpha(c)`	letters	`std::toupper(c)`	upper-case version
`std::isdigit(c)`	0–9	`std::tolower(c)`	lower-case version
`std::isalnum(c)`	letters or digits	`std::isupper(c)`	true for A–Z
`std::isspace(c)`	space, tab, newline …	`std::islower(c)`	true for a–z
`std::ispunct(c)`	punctuation	`std::isxdigit(c)`	0–9, a–f, A–F

WATCH OUT

Pass `unsigned char` values to the <cctype> functions: calling them with a negative `char` (for example a byte of UTF-8 text) is undefined behaviour. The safe idiom is `std::isalpha(static_cast<unsigned char>(c))`. They also return int, so cast toupper's result back to char.

CLASSIFYING AND CONVERTING CHARACTERS main.cpp

```
#include <cctype>
#include <iostream>
#include <string>

int main() {
    std::string s = "Pass@word 2026!";
    int letters = 0, digits = 0, spaces = 0, others = 0;
    for (unsigned char c : s) {
        if (std::isalpha(c)) ++letters;
        else if (std::isdigit(c)) ++digits;
        else if (std::isspace(c)) ++spaces;
        else ++others;
    }
    std::cout << letters << " letters, " << digits << " digits, " << spaces << " space, " << others <<
" others\n";
    std::string up = s, low = s;
    for (char& c : up) c = static_cast<char>(std::toupper(static_cast<unsigned char>(c)));
    for (char& c : low) c = static_cast<char>(std::tolower(static_cast<unsigned char>(c)));
    std::cout << up << " | " << low << '\n';
    return 0;
}
```

OUTPUT

```
8 letters, 4 digits, 1 space, 2 others
PASS@WORD 2026! | pass@word 2026!
```

8.4.2 ASCII arithmetic

Characters are small integers (ASCII codes for English text): `'A'` = 65 … `'Z'` = 90, `'a'` = 97 … `'z'` = 122, `'0'` = 48 … `'9'` = 57. Letters and digits are consecutive, which allows arithmetic: `c - '0'` converts a

digit character to its value, `c - 'a'` gives a letter's position in the alphabet, and lower and upper case differ by 32.

ARITHMETIC ON CHARACTERS main.cpp

```
#include <iostream>
#include <string>

std::string caesar(const std::string& text, int shift) {
    std::string out = text;
    for (char& c : out) {
        if (c >= 'a' && c <= 'z') c = static_cast<char>('a' + (c - 'a' + shift) % 26);
        else if (c >= 'A' && c <= 'Z') c = static_cast<char>('A' + (c - 'A' + shift) % 26);
    }
    return out;
}
int main() {
    std::cout << "'A' = " << int('A') << ", 'a' = " << int('a') << ", difference " << 'a' - 'A' <<
'\n';
    std::string digits = "4096";
    int value = 0;
    for (char c : digits) value = value * 10 + (c - '0');      // manual string-to-int
    std::cout << "\"4096\" as a number + 1 = " << value + 1 << '\n';
    std::cout << "position of 'k' in the alphabet: " << 'k' - 'a' + 1 << '\n';
    std::string secret = caesar("Hello, World", 3);
    std::cout << "Caesar +3: " << secret << ", decoded: " << caesar(secret, 23) << '\n';
    return 0;
}
```

OUTPUT

```
'A' = 65, 'a' = 97, difference 32
"4096" as a number + 1 = 4097
position of 'k' in the alphabet: 11
Caesar +3: Khoor, Zruog, decoded: Hello, World
```

8.5 Conversions

8.5.1 std::to_string()

8.5.2 stoi(), stol(), stod()

`std::to_string(x)` converts any number to a string. In the other direction, `std::stoi` (int), `stol`, `stoll`, `stoul`, `stof`, `stod` parse a number from the start of a string, skipping leading spaces. They throw `std::invalid_argument` if no number is found and `std::out_of_range` if it does not fit; an optional second argument receives how many characters were used, and an optional third gives the base.

NUMBERS TO STRINGS AND BACK

main.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>

int main() {
    std::string s = "Total: " + std::to_string(42) + ", avg " + std::to_string(87.5);
    std::cout << s << '\n';
    std::cout << std::stoi("123") + 1 << ' ' << std::stod("3.75") * 2 << ' ' <<
std::stoll("9000000000") << '\n';
    std::size_t used;
    int n = std::stoi("  250kg", &used);
    std::cout << "stoi(\"  250kg\") = " << n << ", used " << used << " characters\n";
    std::cout << "stoi(\"ff\", nullptr, 16) = " << std::stoi("ff", nullptr, 16)
              << ", stoi(\"1011\", nullptr, 2) = " << std::stoi("1011", nullptr, 2) << '\n';
    for (const char* bad : {"hello", "99999999999"}) {
        try {
            std::stoi(bad);
        } catch (const std::invalid_argument&) {
            std::cout << '"' << bad << "\" -> invalid_argument\n";
        } catch (const std::out_of_range&) {
            std::cout << '"' << bad << "\" -> out_of_range\n";
        }
    }
    return 0;
}
```

OUTPUT

```
Total: 42, avg 87.500000
124 7.5 9000000000
stoi("  250kg") = 250, used 5 characters
stoi("ff", nullptr, 16) = 255, stoi("1011", nullptr, 2) = 11
"hello" -> invalid_argument
"99999999999" -> out_of_range
```

8.5.3 std::stringstream for parsing and building strings

A string stream (`<sstream>`) is a stream whose device is a string: `std::istringstream` reads from a string with `>>` and getline exactly as cin reads the keyboard, and `std::ostringstream` builds a string with `<<` and all the iomanip formatting. They are the standard way to split text into words or fields.

PARSING AND BUILDING WITH STRING STREAMS main.cpp

```
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

int main() {
    std::istringstream in("Asha 21 88.5");            // parse fields of different types
    std::string name; int age; double marks;
    in >> name >> age >> marks;
    std::cout << name << " is " << age << " and scored " << marks << '\n';

    std::istringstream words("split   this sentence into words");
    std::vector<std::string> list;
    for (std::string w; words >> w; ) list.push_back(w);   // >> skips any amount of whitespace
    std::cout << list.size() << " words, last = " << list.back() << '\n';

    std::istringstream csv("Pune,411001,Maharashtra");
    for (std::string field; std::getline(csv, field, ','); ) std::cout << '[' << field << ']';
    std::cout << '\n';

    std::ostringstream out;                                // build a formatted string
    out << "Invoice #" << std::setw(5) << std::setfill('0') << 42 << " total " << std::fixed <<
std::setprecision(2) << 1299.5;
    std::string invoice = out.str();
    std::cout << invoice << " (" << invoice.size() << " chars)\n";
    return 0;
}
```

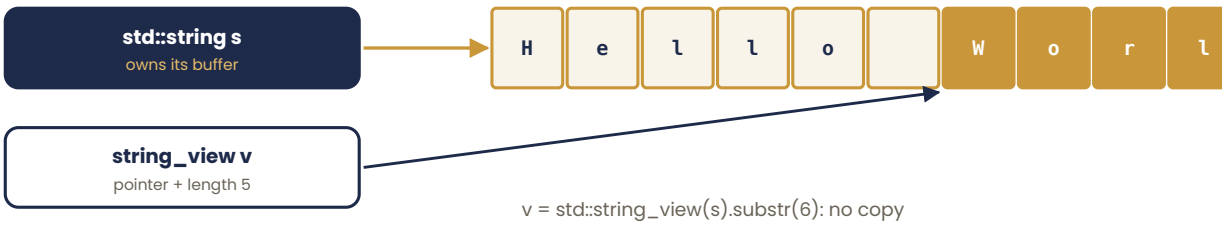
OUTPUT

```
Asha is 21 and scored 88.5
5 words, last = words
[Pune][411001][Maharashtra]
Invoice #00042 total 1299.50 (28 chars)
```

8.6 std::string_view (C++17)

8.6.1 Non-owning views and when to use them

`std::string_view` (header `<string_view>`) is a lightweight **view** of characters owned by someone else: just a pointer and a length (16 bytes). Creating one never copies or allocates, and `substr` on a view is O(1). It is ideal for **read-only function parameters**, because a single `string_view` parameter accepts a std::string, a string literal or a char array without copying any of them.



A STRING_VIEW POINTS INTO CHARACTERS IT DOES NOT OWN

USING STRING_VIEW

main.cpp

```cpp
#include <iostream>
#include <string>
#include <string_view>

std::size_t count_vowels(std::string_view text) {      // accepts std::string, literals, char arrays
    std::size_t n = 0;
    for (char c : text) n += (std::string_view("aeiouAEIOU").find(c) != std::string_view::npos);
    return n;
}
int main() {
    std::string s = "Learn Being Forward";
    char arr[] = "character array";
    std::cout << count_vowels(s) << ' ' << count_vowels("a literal") << ' ' << count_vowels(arr) <<
'\n';

    std::string_view v = s;
    std::string_view word = v.substr(6, 5);              // no allocation
    std::cout << "word = " << word << ", size " << word.size() << ", sizeof(view) = " << sizeof v <<
'\n';
    v.remove_prefix(6);                                  // shrink the view, the string is untouched
    std::cout << "view now: " << v << " | string still: " << s << '\n';
    std::string owned{word};                             // make an owning copy when needed
    std::cout << std::boolalpha << "starts_with: " << v.starts_with("Being") << '\n';
    return 0;
}
```

OUTPUT

```
6 4 5
word = Being, size 5, sizeof(view) = 16
view now: Being Forward | string still: Learn Being Forward
starts_with: true
```

8.6.2 Dangling string_view pitfalls

A view does not keep the text alive. If the string it refers to is destroyed or modified (and reallocated), the view **dangles**: reading it is undefined behaviour. Never return a string_view to a local string, never store a view of a temporary, and remember that views are not null-terminated (do not pass `v.data()` to C functions).

DANGLING VIEWS: THE PROGRAM COMPILES SILENTLY, SO THE BUG IS EASY TO MISS

main.cpp

```
#include <iostream>
#include <string>
#include <string_view>

std::string make_name() { return "temporary text that lives in a heap buffer"; }

std::string_view bad_view() {
    std::string local = "a local string that is far too long for SSO";
    return local;                       // local is destroyed: the view dangles
}
int main() {
    std::string_view v1 = make_name();    // view of a temporary destroyed at the end of this line
    std::string_view v2 = bad_view();
    std::cout << "sizes look fine: " << v1.size() << ' ' << v2.size() << '\n';
    std::cout << "but reading them is undefined behaviour\n";
    std::string safe = make_name();       // fix: keep an owning std::string alive
    std::string_view ok = safe;
    std::cout << ok.substr(0, 9) << '\n';
    return 0;
}
```

OUTPUT

```
sizes look fine: 42 43
but reading them is undefined behaviour
temporary
```

Use	Parameter type
read-only text, any source	`std::string_view`
need a null-terminated string for a C API	`const std::string&`
the function stores or modifies its own copy	`std::string` (by value) then move
modify the caller's string	`std::string&`

8.7 Classic string problems

8.7.1 Reverse a string and reverse words

8.7.2 Palindrome and anagram check

REVERSAL, PALINDROMES AND ANAGRAMS

main.cpp

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

std::string reverse_manual(std::string s) {
    for (std::size_t i = 0, j = s.size(); i + 1 < j; ++i, --j) std::swap(s[i], s[j - 1]);
    return s;
}
std::string reverse_words(const std::string& sentence) {
    std::istringstream in(sentence);
```

```
    std::vector<std::string> words;
    for (std::string w; in >> w; ) words.push_back(w);
    std::string out;
    for (auto it = words.rbegin(); it != words.rend(); ++it) out += *it + (it + 1 != words.rend() ? "
" : "");
    return out;
}
bool is_palindrome(const std::string& s) {           // ignore case and non-letters
    std::string clean;
    for (unsigned char c : s) if (std::isalnum(c)) clean += static_cast<char>(std::tolower(c));
    return std::equal(clean.begin(), clean.begin() + clean.size() / 2, clean.rbegin());
}
bool is_anagram(std::string a, std::string b) {
    auto prep = [](std::string& s) {
        std::string t;
        for (unsigned char c : s) if (std::isalpha(c)) t += static_cast<char>(std::tolower(c));
        std::sort(t.begin(), t.end());
        s = t;
    };
    prep(a); prep(b);
    return a == b;
}
int main() {
    std::string s = "Forward";
    std::string r = s;
    std::reverse(r.begin(), r.end());
    std::cout << reverse_manual(s) << " | " << r << " | " << std::string(s.rbegin(), s.rend()) <<
'\n';
    std::cout << reverse_words("practice makes a programmer perfect") << '\n';
    std::cout << std::boolalpha;
    for (const char* t : {"Madam", "A man, a plan, a canal: Panama", "Learn"})
        std::cout << '"' << t << "\" palindrome? " << is_palindrome(t) << '\n';
    std::cout << "listen/silent anagram? " << is_anagram("Listen", "Silent")
              << ", dormitory/dirty room? " << is_anagram("Dormitory", "Dirty room")
              << ", apple/paper? " << is_anagram("apple", "paper") << '\n';
    return 0;
}
```

OUTPUT

```
drawroF | drawroF | drawroF
perfect programmer a makes practice
"Madam" palindrome? true
"A man, a plan, a canal: Panama" palindrome? true
"Learn" palindrome? false
listen/silent anagram? true, dormitory/dirty room? true, apple/paper? false
```

8.7.3 Count vowels, consonants, digits

8.7.4 Character frequency, first non-repeating character

COUNTING CHARACTERS

main.cpp

```cpp
#include <array>
#include <cctype>
#include <iostream>
#include <map>
#include <string>

int main() {
    std::string s = "Programming in C++ since 2026";
    int vowels = 0, consonants = 0, digits = 0;
    for (unsigned char c : s) {
        if (std::isdigit(c)) ++digits;
        else if (std::isalpha(c)) {
            char l = static_cast<char>(std::tolower(c));
            (l == 'a' || l == 'e' || l == 'i' || l == 'o' || l == 'u') ? ++vowels : ++consonants;
        }
    }
    std::cout << vowels << " vowels, " << consonants << " consonants, " << digits << " digits\n";

    std::array<int, 256> freq{};                        // counting array indexed by character
    for (unsigned char c : s) ++freq[c];
    std::cout << "letters appearing more than once:";
    for (int c = 'a'; c <= 'z'; ++c) if (freq[c] > 1) std::cout << ' ' << static_cast<char>(c) << '='
<< freq[c];
    std::cout << '\n';

    std::string w = "swiss engineering";
    std::map<char, int> count;                           // ordered map alternative
    for (char c : w) ++count[c];
    for (char c : w)
        if (count[c] == 1) { std::cout << "first non-repeating in \"" << w << "\": '" << c << "'\n";
break; }
    return 0;
}
```

OUTPUT

```
6 vowels, 13 consonants, 4 digits
letters appearing more than once: g=2 i=3 m=2 n=3 r=2
first non-repeating in "swiss engineering": 'w'
```

8.7.5 Split a sentence into words, remove duplicates, string compression

Run-length compression replaces each run of a repeated character with the character and the count: "aaabccdddd" becomes "a3b1c2d4". It is only worth using when the result is shorter.

SPLIT, DE-DUPLICATE AND COMPRESS

main.cpp

```cpp
#include <iostream>
#include <sstream>
#include <string>
#include <unordered_set>
#include <vector>

std::vector<std::string> split(const std::string& s, char delim) {
    std::vector<std::string> parts;
    std::string cur;
    for (char c : s) {
        if (c == delim) { if (!cur.empty()) parts.push_back(cur); cur.clear(); }
        else cur += c;
    }
    if (!cur.empty()) parts.push_back(cur);
    return parts;
}
std::string remove_duplicate_chars(const std::string& s) {
    std::string out;
    std::unordered_set<char> seen;
    for (char c : s) if (seen.insert(c).second) out += c;     // insert succeeds only the first time
    return out;
}
std::string compress(const std::string& s) {
    std::string out;
    for (std::size_t i = 0; i < s.size(); ) {
        std::size_t j = i;
        while (j < s.size() && s[j] == s[i]) ++j;
        out += s[i] + std::to_string(j - i);
        i = j;
    }
    return out.size() < s.size() ? out : s;
}
int main() {
    for (const auto& part : split("2026-10-06", '-')) std::cout << '[' << part << ']';
    std::cout << '\n';
    std::cout << remove_duplicate_chars("programming") << '\n';
    std::cout << compress("aaabccdddd") << ' ' << compress("abc") << " (not compressible)\n";
    return 0;
}
```

OUTPUT

```
[2026][10][06]
progamin
a3b1c2d4 abc (not compressible)
```

Prefer std::string: it manages memory, knows its size, and supports +, ==, <, find, substr, insert, erase and replace. Use std::string_view for read-only parameters but never let it outlive the text. Convert with to_string and stoi/stod, parse and build text with string streams, cast to unsigned char before <cctype> calls, and keep C strings for C APIs only (via c_str()).

PRACTICE CHECKPOINT

Split a sentence into words with stringstream, **count word frequencies**, and **check whether two strings are anagrams**.

SAMPLE SOLUTION

main.cpp

```
#include <algorithm>
#include <cctype>
#include <iomanip>
#include <iostream>
#include <map>
#include <sstream>
#include <string>

std::string normalise(const std::string& w) {             // lower case, letters only
    std::string out;
    for (unsigned char c : w) if (std::isalpha(c)) out += static_cast<char>(std::tolower(c));
    return out;
}
bool anagrams(const std::string& a, const std::string& b) {
    std::string x = normalise(a), y = normalise(b);
    std::sort(x.begin(), x.end());
    std::sort(y.begin(), y.end());
    return x == y;
}
int main() {
    std::string sentence;
    std::cout << "Sentence: ";
    std::getline(std::cin, sentence);

    std::istringstream in(sentence);
    std::map<std::string, int> freq;
    int total = 0;
    for (std::string word; in >> word; ) {
        std::string w = normalise(word);
        if (!w.empty()) { ++freq[w]; ++total; }
    }
    std::cout << total << " words, " << freq.size() << " distinct\n";
    for (const auto& [word, n] : freq)
        std::cout << "  " << std::left << std::setw(10) << word << std::string(n, '#') << ' ' << n <<
'\n';

    std::string a, b;
    std::cout << "Two words or phrases (one per line):\n";
    std::getline(std::cin, a);
    std::getline(std::cin, b);
    std::cout << '"' << a << "\" and \"" << b << "\" are " << (anagrams(a, b) ? "" : "NOT ") <<
"anagrams\n";
    return 0;
}
```

CONSOLE (user input in gold)

```
Sentence: The cat and the dog and THE bird, the end.
10 words, 6 distinct
  and       ## 2
  bird      # 1
  cat       # 1
  dog       # 1
  end       # 1
  the       #### 4
Two words or phrases (one per line):
Debit card
Bad credit
"Debit card" and "Bad credit" are anagrams
```

MODULE 09 · BEGINNER

Functions and recursion

Minimum duration **3 days** | Level **Beginner**

Functions package a piece of logic under a name so it can be reused, tested and reasoned about on its own. C++ builds on C functions with references, overloading, default arguments, inline and constexpr functions, and attributes such as [[nodiscard]]. The second half of the module covers recursion thoroughly, from the call stack to backtracking and memoization.

IN THIS MODULE

9.1 Function basics
9.2 Parameter passing
9.3 Return values
9.4 Function features in C++
9.5 Recursion

9.1 Function basics

9.1.1 Declaration (prototype), definition and call

A **declaration** (prototype) tells the compiler a function's name, return type and parameter types. A **definition** adds the body. A **call** runs it with actual **arguments**, which initialise the **parameters**. A function must be declared before it is called; declarations usually go in headers, definitions in .cpp files.

```
double area ( double width, double height ) { return width * height; }
```

return type | name | parameters | body

```
double a = area(4.5, 2.0); // call: 4.5 and 2.0 are the arguments
```

the call is replaced by the returned value 9.0

PARTS OF A FUNCTION

DECLARE, CALL, DEFINE main.cpp

```cpp
#include <iostream>

double area(double width, double height);       // declaration (prototype)
void print_line(int n);

int main() {
    double a = area(4.5, 2.0);                   // call
    print_line(20);
    std::cout << "area = " << a << '\n';
    std::cout << "bigger room: " << area(6, 4) << '\n';
    print_line(20);
    return 0;
}

double area(double width, double height) {      // definition
    return width * height;
}
void print_line(int n) {
    for (int i = 0; i < n; ++i) std::cout << '-';
    std::cout << '\n';
}
```

```
OUTPUT
--------------------
area = 9
bigger room: 24
--------------------
```

9.1.2 Return types and void

A function returns one value of its declared type with `return expression;`, which also ends the function immediately. A `void` function returns nothing; it may use a bare `return;` to exit early. Falling off the end of a non-void function (other than main) is undefined behaviour, and `-Wall` warns about it.

RETURNING VALUES

main.cpp

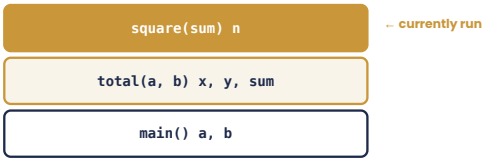
```cpp
#include <iostream>
#include <string>

std::string grade(int marks) {
    if (marks >= 90) return "A";           // return ends the function at once
    if (marks >= 75) return "B";
    if (marks >= 40) return "C";
    return "F";
}
void greet(const std::string& name) {
    if (name.empty()) return;              // early exit from a void function
    std::cout << "Hello, " << name << "!\n";
}
bool is_even(int n) { return n % 2 == 0; }

int main() {
    std::cout << grade(93) << grade(80) << grade(55) << grade(12) << '\n';
    greet("Meera");
    greet("");
    std::cout << std::boolalpha << is_even(10) << ' ' << is_even(7) << '\n';
    return 0;
}
```

OUTPUT

```
ABCF
Hello, Meera!
true false
```

9.1.3 The call stack and stack frames

Every call creates a **stack frame** on the **call stack**, holding the function's parameters, local variables and the return address (where to continue in the caller). When the function returns, its frame is removed. This is why local variables disappear when a function ends, and why each call (including each recursive call) has its own copies of them.



MAIN CALLS TOTAL, WHICH CALLS SQUARE: THREE FRAMES ON THE STACK

FRAMES ARE PUSHED ON CALL AND POPPED ON RETURN

main.cpp

```cpp
#include <iostream>

int square(int n) {
    std::cout << "    square(" << n << ") frame: n at " << &n << '\n';
    return n * n;
}
int total(int x, int y) {
    int sum = x + y;
    std::cout << "  total(" << x << ", " << y << ") frame: sum at " << &sum << '\n';
    return square(sum);
}
int main() {
    int a = 3, b = 4;
    std::cout << "main frame: a at " << &a << '\n';
    int r = total(a, b);
    std::cout << "result " << r << " (frames of total and square are gone)\n";
    return 0;
}
```

OUTPUT (addresses differ; deeper calls usually have lower addresses)

```
main frame: a at 0x7ffea6ce684c
  total(3, 4) frame: sum at 0x7ffea6ce6824
    square(7) frame: n at 0x7ffea6ce67fc
result 49 (frames of total and square are gone)
```

9.2 Parameter passing

9.2.1 Pass by value

By default the parameter is a **copy** of the argument. Changes inside the function affect only the copy. Fine for small types (int, double, char, small structs); expensive for large objects such as long strings or vectors.

9.2.2 Pass by reference &

A **reference parameter** `T& p` is another name (alias) for the caller's variable. No copy is made, and changes inside the function change the caller's variable. The argument must be a modifiable variable (an lvalue), not a literal or temporary.

9.2.3 Pass by const reference for large objects

`const T& p` avoids the copy **and** promises not to modify the argument. It is the standard way to pass strings, vectors and class objects that the function only reads, and it also accepts temporaries and literals.

9.2.4 Pass by pointer

Passing an address (`T* p`, called with `&x`) also lets the function modify the caller's variable, as in C. Pointers can be `nullptr`, so they are useful for **optional** arguments, but the function must check for null and the call site must take the address. In C++ references are preferred when the argument is required.

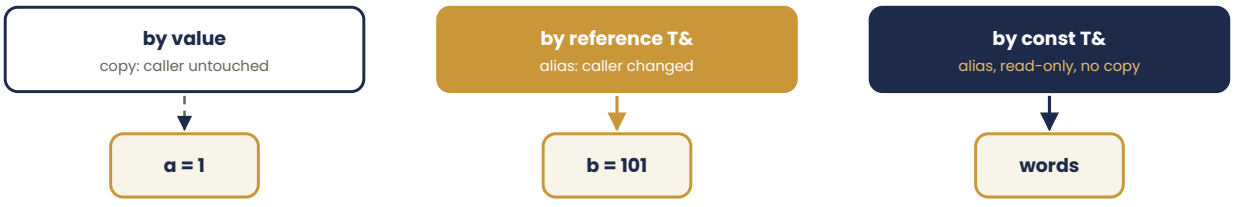
VALUE, REFERENCE, POINTER

main.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

void by_value(int x)        { x += 100; }
void by_reference(int& x)   { x += 100; }
void by_pointer(int* x)     { if (x) *x += 100; }
void swap_ref(int& a, int& b) { int t = a; a = b; b = t; }

std::size_t total_length(const std::vector<std::string>& words) {   // no copy, read-only
    std::size_t n = 0;
    for (const auto& w : words) n += w.size();
    return n;
}
int main() {
    int a = 1, b = 1, c = 1;
    by_value(a);
    by_reference(b);
    by_pointer(&c);
    by_pointer(nullptr);                    // allowed and safe: the function checks
    std::cout << "by value " << a << ", by reference " << b << ", by pointer " << c << '\n';
    int p = 5, q = 9;
    swap_ref(p, q);
    std::cout << "after swap: p = " << p << ", q = " << q << '\n';
    std::vector<std::string> words = {"pass", "by", "const", "reference"};
    std::cout << "total length " << total_length(words) << '\n';
    // by_reference(5);   error: cannot bind a non-const reference to a literal
    return 0;
}
```

OUTPUT

```
by value 1, by reference 101, by pointer 101
after swap: p = 9, q = 5
total length 20
```



WHAT THE FUNCTION RECEIVES IN EACH CASE

9.2.5 Choosing the right passing method

Situation	Use	Example
small type (int, double, char, pointer, string_view), read only	by value	`int square(int x)`
large object, read only	by const reference	`void print(const std::vector<int>& v)`
the function must modify the caller's object	by reference	`void sort_in_place(std::vector<int>& v)`

Situation	Use	Example
the argument is optional (may be absent)	by pointer (check nullptr) or std::optional	`void log(const Config* cfg = nullptr)`
the function keeps its own copy anyway	by value, then std::move (Module 20)	`void set_name(std::string n)`

9.3 Return values

9.3.1 Returning by value

Returning by value is the normal, safe choice even for large objects: the compiler constructs the result directly in the caller's variable (**return value optimisation**, guaranteed since C++17 for returned temporaries) or moves it (Module 20), so returning a big vector does not copy its elements.

9.3.2 Returning by reference (and the dangling reference trap)

A function may return a reference to an object that **outlives** the call, such as an element of a container passed in, or a static variable. This allows `get(v, i) = 5;`. Returning a reference (or pointer) to a **local** variable is a serious bug: the local is destroyed when the function returns, leaving a **dangling reference**.

RETURNING REFERENCES SAFELY

main.cpp

```
#include <iostream>
#include <vector>

int& element(std::vector<int>& v, std::size_t i) { return v[i]; }   // OK: v outlives the call

int& counter() {
    static int count = 0;            // static: lives for the whole program
    return ++count;
}
// int& bad() { int local = 42; return local; }   // dangling: GCC warns "reference to local variable returned"

int main() {
    std::vector<int> v = {1, 2, 3};
    element(v, 1) = 20;              // assigning through the returned reference
    std::cout << v[0] << ' ' << v[1] << ' ' << v[2] << '\n';
    counter(); counter();
    std::cout << "counter() = " << counter() << '\n';
    return 0;
}
```

OUTPUT

```
1 20 3
counter() = 3
```

THE DANGLING REFERENCE TRAP

main.cpp

```
int& bad() {
    int local = 42;
    return local;          // reference to a destroyed variable
}
int main() { return bad(); }
```

OUTPUT

```
main.cpp: In function ‘int& bad()’:
main.cpp:3:12: error: reference to local variable ‘local’ returned [-Werror=return-local-addr]
    3 |     return local;          // reference to a destroyed variable
      |            ^~~~~
main.cpp:2:9: note: declared here
    2 |     int local = 42;
      |         ^~~~~
cc1plus: some warnings being treated as errors
```

9.3.3 Returning multiple values with std::pair, std::tuple and structs

A function returns one value, but that value can bundle several: `std::pair` (two values, `.first` and `.second`), `std::tuple` (any number, accessed with `std::get`), or best of all a small **struct** with meaningful member names. C++17 **structured bindings** unpack the result into named variables.

RETURNING SEVERAL VALUES main.cpp

```cpp
#include <iostream>
#include <string>
#include <tuple>
#include <utility>
#include <vector>

std::pair<int, int> min_max(const std::vector<int>& v) {
    int lo = v[0], hi = v[0];
    for (int x : v) { if (x < lo) lo = x; if (x > hi) hi = x; }
    return {lo, hi};
}
std::tuple<int, int, double> stats(const std::vector<int>& v) {
    int sum = 0;
    for (int x : v) sum += x;
    return {static_cast<int>(v.size()), sum, static_cast<double>(sum) / v.size()};
}
struct Division { int quotient; int remainder; };          // clearest: named fields
Division divide(int a, int b) { return {a / b, a % b}; }

int main() {
    std::vector<int> marks = {72, 95, 61, 88};
    auto mm = min_max(marks);
    std::cout << "min " << mm.first << ", max " << mm.second << '\n';
    auto [count, sum, avg] = stats(marks);                  // structured binding (C++17)
    std::cout << count << " marks, sum " << sum << ", average " << avg << '\n';
    auto [q, r] = divide(47, 5);
    std::cout << "47 / 5 = " << q << " remainder " << r << '\n';
    std::tuple<std::string, int> t{"Asha", 21};
    std::cout << std::get<0>(t) << " is " << std::get<1>(t) << '\n';
    return 0;
}
```

OUTPUT

```
min 61, max 95
4 marks, sum 316, average 79
47 / 5 = 9 remainder 2
Asha is 21
```

9.3.4 Trailing return type auto f() -> int

C++11 allows the return type to be written **after** the parameters: `auto f(params) -> type`. It is required when the return type depends on the parameters (in templates with decltype) and some teams use it everywhere for alignment. Lambdas also use this syntax.

9.3.5 auto return type deduction (C++14)

With just `auto` as the return type, the compiler deduces it from the return statements, which must all deduce the same type. Handy for short functions and templates; for public interfaces an explicit type is easier to read.

TRAILING AND DEDUCED RETURN TYPES

main.cpp

```cpp
#include <iostream>
#include <string>

auto add(int a, int b) -> int { return a + b; }              // trailing return type

template <typename A, typename B>
auto multiply(A a, B b) -> decltype(a * b) { return a * b; }  // type depends on the parameters

auto half(double x) { return x / 2; }                         // deduced: double
auto greeting(const std::string& n) { return "Hi " + n; }     // deduced: std::string

int main() {
    std::cout << add(2, 3) << ' ' << multiply(3, 2.5) << ' ' << half(9) << ' ' << greeting("Ravi") <<
'\n';
    return 0;
}
```

OUTPUT

```
5 7.5 4.5 Hi Ravi
```

9.4 Function features in C++

9.4.1 Default arguments

Parameters may have **default values**, used when the caller omits the argument. Defaults must be at the **end** of the parameter list (you cannot skip a middle argument), and are written in the declaration only, not repeated in the definition.

DEFAULT ARGUMENTS

main.cpp

```cpp
#include <iostream>
#include <string>

void print_bill(const std::string& item, double price, int qty = 1, double gst = 0.18) {
    double total = price * qty * (1 + gst);
    std::cout << item << " x" << qty << " @ " << price << " +" << gst * 100 << "% GST = " << total <<
'\n';
}
int main() {
    print_bill("Notebook", 50);              // qty = 1, gst = 0.18
    print_bill("Pen", 10, 5);                // gst = 0.18
    print_bill("Rice (5 kg)", 400, 2, 0.05); // all given
    return 0;
}
```

OUTPUT

```
Notebook x1 @ 50 +18% GST = 59
Pen x5 @ 10 +18% GST = 59
Rice (5 kg) x2 @ 400 +5% GST = 840
```

9.4.2 Function overloading and overload resolution

Several functions may share a name if their **parameter lists differ** (number or types of parameters). The compiler picks the best match for each call: **overload resolution** prefers an exact match, then promotions (char → int, float → double), then standard conversions (int → double), then user-defined

conversions. If two candidates are equally good, the call is **ambiguous** and does not compile. The return type alone cannot distinguish overloads.

OVERLOADED FUNCTIONS main.cpp

```
#include <iostream>
#include <string>

int    area(int side)                { return side * side; }           // square
double area(double radius)           { return 3.14159 * radius * radius; }  // circle
int    area(int length, int width)   { return length * width; }        // rectangle
void   show(int x)                   { std::cout << "int " << x << '\n'; }
void   show(double x)                { std::cout << "double " << x << '\n'; }
void   show(const std::string& s)    { std::cout << "string " << s << '\n'; }

int main() {
    std::cout << area(4) << ' ' << area(2.0) << ' ' << area(3, 5) << '\n';
    show(42);           // exact: int
    show(4.2);          // exact: double
    show('A');          // char -> int promotion
    show(3.5f);         // float -> double promotion
    show("text");       // const char* -> std::string conversion
    return 0;
}
```

OUTPUT

```
16 12.5664 15
int 42
double 4.2
int 65
double 3.5
string text
```

AN AMBIGUOUS CALL main.cpp

```
void show(int x) {}
void show(double x) {}

int main() {
    show(10L);          // long -> int and long -> double are equally good
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:5:9: error: call of overloaded ‘show(long int)’ is ambiguous
    5 |     show(10L);          // long -> int and long -> double are equally good
      |     ~~~~^~~~
main.cpp:1:6: note: candidate: ‘void show(int)’
    1 | void show(int x) {}
      |      ^~~~
main.cpp:2:6: note: candidate: ‘void show(double)’
    2 | void show(double x) {}
      |      ^~~~
```

9.4.3 Inline functions

`inline` originally asked the compiler to paste the function body at each call site to avoid call overhead. Modern compilers decide inlining by themselves (at -O2), so today the keyword mainly means

something else: an inline function (or variable, C++17) may be **defined in a header** included by several .cpp files without "multiple definition" link errors. Functions defined inside a class body are implicitly inline.

MATH_UTILS.H

C++

```cpp
// math_utils.h  (included by many .cpp files)
#pragma once
inline int cube(int x) { return x * x * x; }   // OK in a header: one definition rule relaxed
inline constexpr double GOLDEN_RATIO = 1.6180339887;   // inline variable (C++17)
```

9.4.4 constexpr functions

A `constexpr` function can be evaluated **at compile time** when called with constant arguments, and at run time otherwise. Results used where a constant is required (array sizes, static_assert, template arguments) must be computed at compile time. C++20 `consteval` functions must always run at compile time.

COMPILE-TIME EVALUATION

main.cpp

```cpp
#include <array>
#include <iostream>

constexpr long long factorial(int n) {
    long long r = 1;
    for (int i = 2; i <= n; ++i) r *= i;
    return r;
}
consteval int kib(int n) { return n * 1024; }          // C++20: compile time only

int main() {
    static_assert(factorial(5) == 120);                // checked by the compiler
    std::array<int, factorial(4)> slots{};             // array of 24 elements
    constexpr int buffer = kib(4);
    int n;
    std::cin >> n;
    std::cout << "slots " << slots.size() << ", buffer " << buffer << ", " << n << "! = " <<
factorial(n) << " (computed at run time)\n";
    return 0;
}
```

CONSOLE (user input in gold)

```
12
slots 24, buffer 4096, 12! = 479001600 (computed at run time)
```

9.4.5 [[nodiscard]] attribute (C++17)

`[[nodiscard]]` on a function makes the compiler warn when a caller ignores its return value, which catches bugs like calling a function that returns an error code, or `v.empty()` when `v.clear()` was meant. C++20 allows a reason: `[[nodiscard("check for errors")]]`.

WARNINGS FOR IGNORED RESULTS main.cpp

```cpp
#include <string>

[[nodiscard]] bool save_file(const std::string& name) { return !name.empty(); }
[[nodiscard("the result is the only output")]] int add(int a, int b) { return a + b; }

int main() {
    save_file("report.txt");      // result ignored: was the save successful?
    add(2, 3);                    // pointless call
    bool ok = save_file("x");     // fine
    return ok ? 0 : 1;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:7:14: warning: ignoring return value of ‘bool save_file(const std::string&)’, declared with
attribute ‘nodiscard’ [-Wunused-result]
    7 |     save_file("report.txt");      // result ignored: was the save successful?
      |     ~~~~~~~~~^~~~~~~~~~~~~~
main.cpp:3:20: note: declared here
    3 | [[nodiscard]] bool save_file(const std::string& name) { return !name.empty(); }
      |                    ^~~~~~~~~
main.cpp:8:8: warning: ignoring return value of ‘int add(int, int)’, declared with attribute
‘nodiscard’: ‘the result is the only output’ [-Wunused-result]
    8 |     add(2, 3);                    // pointless call
      |     ~~~^~~~~
main.cpp:4:52: note: declared here
    4 | [[nodiscard("the result is the only output")]] int add(int a, int b) { return a + b; }
      |                                                    ^~~
$ ./prog
```

9.4.6 noexcept (introduced)

`noexcept` declares that a function never throws an exception. The compiler and library can then generate faster code; for example std::vector moves elements during reallocation only if their move constructor is noexcept. If a noexcept function does throw, the program terminates. Module 16 covers exceptions.

DECLARING AND QUERYING NOEXCEPT main.cpp

```cpp
#include <iostream>

int add(int a, int b) noexcept { return a + b; }
void may_throw() {}

int main() {
    std::cout << std::boolalpha << "add is noexcept: " << noexcept(add(1, 2))
              << ", may_throw is noexcept: " << noexcept(may_throw()) << '\n';
    return 0;
}
```

OUTPUT

```
add is noexcept: true, may_throw is noexcept: false
```

9.5 Recursion

A function is **recursive** when it calls itself, directly or indirectly. Recursion solves a problem by reducing it to a **smaller instance of the same problem**. It is the natural tool for problems defined recursively: factorials, trees, divide-and-conquer algorithms, and exploring all possibilities (backtracking).

9.5.1 Base case and recursive case

Every recursive function needs: a **base case** that is answered directly without recursion, and a **recursive case** that calls the function on a smaller input that moves toward the base case. Without a reachable base case, recursion never stops.

RECURSIVE FACTORIAL — main.cpp

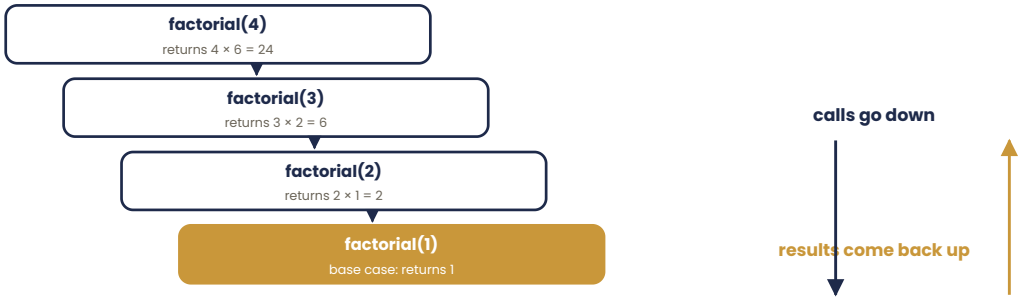
```cpp
#include <iostream>

long long factorial(int n) {
    if (n <= 1) return 1;                // base case
    return n * factorial(n - 1);         // recursive case: smaller problem
}
int main() {
    for (int n : {0, 1, 5, 10, 20}) std::cout << n << "! = " << factorial(n) << '\n';
    return 0;
}
```

OUTPUT

```
0! = 1
1! = 1
5! = 120
10! = 3628800
20! = 2432902008176640000
```

9.5.2 How the call stack unwinds

Each recursive call pushes a new frame with its own `n`. When the base case is reached, the calls return one by one in reverse order (the stack **unwinds**), each multiplying its n by the value returned from below.



FACTORIAL(4): FOUR FRAMES, THEN UNWINDING 1 → 2 → 6 → 24

TRACING THE CALLS AND RETURNS main.cpp

```cpp
#include <iostream>
#include <string>

int depth = 0;
long long factorial(int n) {
    std::string indent(depth * 2, ' ');
    std::cout << indent << "call factorial(" << n << ")\n";
    ++depth;
    long long r = (n <= 1) ? 1 : n * factorial(n - 1);
    --depth;
    std::cout << indent << "return " << r << " from factorial(" << n << ")\n";
    return r;
}
int main() {
    factorial(4);
    return 0;
}
```

OUTPUT

```
call factorial(4)
  call factorial(3)
    call factorial(2)
      call factorial(1)
      return 1 from factorial(1)
    return 2 from factorial(2)
  return 6 from factorial(3)
return 24 from factorial(4)
```

9.5.3 Stack overflow

Each call uses stack memory (typically 8 MB in total on Linux, 1 MB on Windows). Too deep a recursion, most often a missing or unreachable base case, exhausts the stack and the program crashes with a **stack overflow** (on Linux a segmentation fault).

RECURSION TOO DEEP (BUILT WITH -O0) main.cpp

```cpp
#include <iostream>

long long count_down(long long n) {
    if (n == 0) return 0;
    return 1 + count_down(n - 1);          // one frame per level
}
int main() {
    std::cout << "depth 10,000: " << count_down(10000) << '\n';
    std::cout.flush();
    std::cout << "depth 100,000,000: " << count_down(100000000) << '\n';   // far too deep
    return 0;
}
```

OUTPUT (crash)

```
depth 10,000: 10000
Segmentation fault (core dumped)
```

9.5.4 Types: direct, indirect, head, tail, tree recursion

Type	Meaning	Example
Direct	the function calls itself	`f() { … f(); }`

Type	Meaning	Example
Indirect (mutual)	f calls g, which calls f	`is_even` / `is_odd`
Head	the recursive call comes first; work happens while returning	print 1..n
Tail	the recursive call is the very last action; compilers can turn it into a loop	factorial with an accumulator
Tree	more than one recursive call per invocation; calls form a tree	Fibonacci, subsets, merge sort

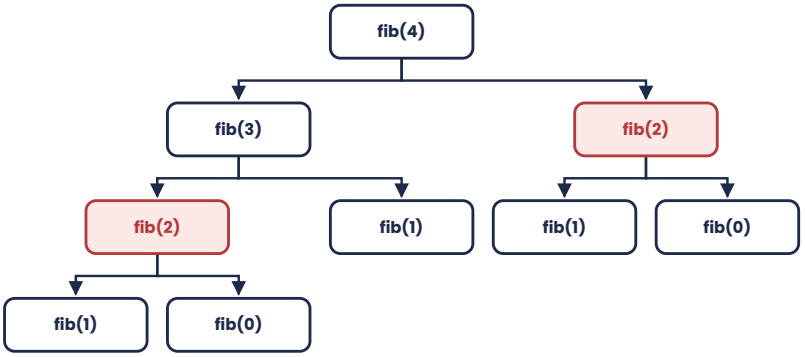
KINDS OF RECURSION

main.cpp

```
#include <iostream>

bool is_odd(int n);
bool is_even(int n) { return n == 0 ? true : is_odd(n - 1); }     // indirect recursion
bool is_odd(int n)  { return n == 0 ? false : is_even(n - 1); }

void head(int n) { if (n == 0) return; head(n - 1); std::cout << n << ' '; }   // work after the call
void tail(int n) { if (n == 0) return; std::cout << n << ' '; tail(n - 1); }   // call is the last
action

long long fact_tail(int n, long long acc = 1) {                    // tail recursive with
accumulator
    return n <= 1 ? acc : fact_tail(n - 1, acc * n);
}
int calls = 0;
int fib(int n) { ++calls; return n < 2 ? n : fib(n - 1) + fib(n - 2); }    // tree recursion

int main() {
    std::cout << std::boolalpha << "is_even(10) " << is_even(10) << ", is_odd(7) " << is_odd(7) <<
'\n';
    std::cout << "head: "; head(5); std::cout << "| tail: "; tail(5); std::cout << '\n';
    std::cout << "fact_tail(10) = " << fact_tail(10) << '\n';
    int f = fib(20);
    std::cout << "fib(20) = " << f << " needed " << calls << " calls\n";
    return 0;
}
```

OUTPUT

```
is_even(10) true, is_odd(7) true
head: 1 2 3 4 5 | tail: 5 4 3 2 1
fact_tail(10) = 3628800
fib(20) = 6765 needed 21891 calls
```



TREE RECURSION: FIB(2) (RED) IS COMPUTED TWICE; THE WASTE GROWS EXPONENTIALLY

9.5.5 Recursion vs iteration

	Recursion	Iteration
Code	often shorter and closer to the mathematical definition	often longer but explicit
Memory	one stack frame per level	constant
Risk	stack overflow for deep inputs	none from depth
Speed	call overhead; naive tree recursion can be exponential	usually faster
Best for	trees, divide and conquer, backtracking	simple counting and accumulation

Any recursion can be rewritten as a loop (with an explicit stack if needed) and vice versa. Use recursion when it makes the solution clearer, and make sure the depth stays modest.

9.5.6 Classic problems

9.5.6.1 Factorial, Fibonacci, power, sum of digits

NUMBER RECURSION

main.cpp

```cpp
#include <iostream>

long long power(long long base, int exp) {        // fast power: O(log exp)
    if (exp == 0) return 1;
    long long half = power(base, exp / 2);
    return exp % 2 == 0 ? half * half : half * half * base;
}
int sum_digits(int n) { return n < 10 ? n : n % 10 + sum_digits(n / 10); }
int fib(int n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }
int count_digits(long long n) { return n < 10 ? 1 : 1 + count_digits(n / 10); }

int main() {
    std::cout << "2^10 = " << power(2, 10) << ", 3^20 = " << power(3, 20) << '\n';
    std::cout << "sum_digits(98765) = " << sum_digits(98765) << '\n';
    std::cout << "fib(0..10):";
    for (int i = 0; i <= 10; ++i) std::cout << ' ' << fib(i);
    std::cout << "\ndigits in 9876543210 = " << count_digits(9876543210LL) << '\n';
    return 0;
}
```

OUTPUT

```
2^10 = 1024, 3^20 = 3486784401
sum_digits(98765) = 35
fib(0..10): 0 1 1 2 3 5 8 13 21 34 55
digits in 9876543210 = 10
```

9.5.6.2 Reverse a string, palindrome

STRING RECURSION

main.cpp

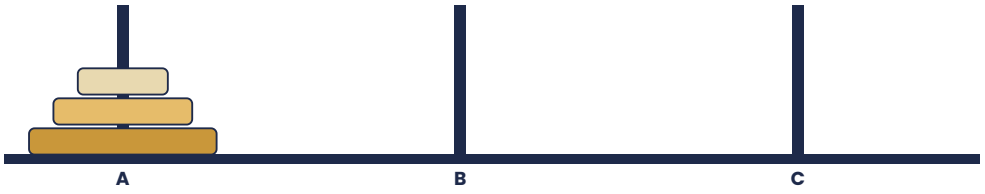
```cpp
#include <iostream>
#include <string>

std::string reverse(const std::string& s) {
    if (s.size() <= 1) return s;
    return reverse(s.substr(1)) + s[0];                // reverse the rest, then append the first
}
bool palindrome(const std::string& s, std::size_t lo, std::size_t hi) {
    if (lo >= hi) return true;                         // 0 or 1 characters left
    if (s[lo] != s[hi]) return false;
    return palindrome(s, lo + 1, hi - 1);
}
int main() {
    std::cout << reverse("recursion") << '\n';
    for (std::string w : {"level", "rotor", "rotate"})
        std::cout << w << (palindrome(w, 0, w.size() - 1) ? " is" : " is not") << " a palindrome\n";
    return 0;
}
```

OUTPUT

```
noisrucer
level is a palindrome
rotor is a palindrome
rotate is not a palindrome
```

9.5.6.3 Tower of Hanoi

Move n disks from peg A to peg C using B, never placing a larger disk on a smaller one. Recursive insight: move the top n-1 disks to B, move the largest disk to C, then move the n-1 disks from B to C. It takes exactly $2^n - 1$ moves.



THREE DISKS ON PEG A; GOAL: MOVE THEM ALL TO C

TOWER OF HANOI main.cpp

```cpp
#include <iostream>

int moves = 0;
void hanoi(int n, char from, char to, char via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);                        // clear the way
    std::cout << "move disk " << n << ' ' << from << " -> " << to << '\n';
    ++moves;
    hanoi(n - 1, via, to, from);                        // put the smaller ones back on top
}
int main() {
    hanoi(3, 'A', 'C', 'B');
    std::cout << moves << " moves (2^3 - 1)\n";
    return 0;
}
```

OUTPUT

```
move disk 1 A -> C
move disk 2 A -> B
move disk 1 C -> B
move disk 3 A -> C
move disk 1 B -> A
move disk 2 B -> C
move disk 1 A -> C
7 moves (2^3 - 1)
```

9.5.6.4 Recursive binary search

BINARY SEARCH, RECURSIVELY main.cpp

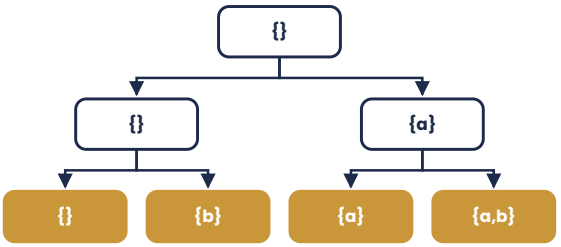
```cpp
#include <iostream>
#include <vector>

int bsearch(const std::vector<int>& v, int key, int lo, int hi) {
    if (lo > hi) return -1;                             // empty range: not found
    int mid = lo + (hi - lo) / 2;
    if (v[mid] == key) return mid;
    return v[mid] < key ? bsearch(v, key, mid + 1, hi)  // search the right half
                        : bsearch(v, key, lo, mid - 1); // or the left half
}
int main() {
    std::vector<int> v = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    for (int key : {23, 2, 91, 40})
        std::cout << key << " -> index " << bsearch(v, key, 0, static_cast<int>(v.size()) - 1) <<
'\n';
    return 0;
}
```

OUTPUT

```
23 -> index 5
2 -> index 0
91 -> index 9
40 -> index -1
```

9.5.6.5 Subsets and permutations (intro to backtracking)

Backtracking builds candidate solutions step by step and undoes ("backtracks") each choice after exploring it. For **subsets**, each element is either excluded or included (2^n results). For **permutations**, each position is filled with every unused element in turn (n! results).



SUBSETS OF {A, B}: EXCLUDE (LEFT) OR INCLUDE (RIGHT) EACH ELEMENT; THE LEAVES ARE THE 4 SUBSETS

GENERATING SUBSETS AND PERMUTATIONS main.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

void subsets(const std::string& s, std::size_t i, std::string& cur) {
    if (i == s.size()) { std::cout << '{' << cur << "} "; return; }
    subsets(s, i + 1, cur);              // exclude s[i]
    cur.push_back(s[i]);                 // choose
    subsets(s, i + 1, cur);              // include s[i]
    cur.pop_back();                      // un-choose (backtrack)
}
void permutations(std::string& s, std::size_t k) {
    if (k == s.size()) { std::cout << s << ' '; return; }
    for (std::size_t i = k; i < s.size(); ++i) {
        std::swap(s[k], s[i]);           // choose s[i] for position k
        permutations(s, k + 1);
        std::swap(s[k], s[i]);           // backtrack
    }
}
int main() {
    std::string cur;
    std::cout << "subsets of abc: ";
    subsets("abc", 0, cur);
    std::string p = "abc";
    std::cout << "\npermutations of abc: ";
    permutations(p, 0);
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
subsets of abc: {} {c} {b} {bc} {a} {ac} {ab} {abc}
permutations of abc: abc acb bac bca cba cab
```

9.5.7 Memoization (intro)

Tree recursion often recomputes the same sub-problems (fib(20) makes 21,891 calls). **Memoization** stores each result the first time it is computed and returns the stored value afterwards, turning exponential time into linear time. It is the first step towards **dynamic programming** (Module 25).

MEMOIZED FIBONACCI main.cpp

```cpp
#include <chrono>
#include <iostream>
#include <unordered_map>

long long calls_plain = 0, calls_memo = 0;

long long fib_plain(int n) { ++calls_plain; return n < 2 ? n : fib_plain(n - 1) + fib_plain(n - 2); }

std::unordered_map<int, long long> memo;
long long fib_memo(int n) {
    ++calls_memo;
    if (n < 2) return n;
    if (auto it = memo.find(n); it != memo.end()) return it->second;   // already known
    return memo[n] = fib_memo(n - 1) + fib_memo(n - 2);                 // compute once, store
}
int main() {
    long long a = fib_plain(32);
    long long b = fib_memo(32);
    std::cout << "plain: fib(32) = " << a << " with " << calls_plain << " calls\n";
    std::cout << "memo : fib(32) = " << b << " with " << calls_memo << " calls\n";
    std::cout << "memo : fib(90) = " << fib_memo(90) << " (plain recursion would take years)\n";
    return 0;
}
```

OUTPUT

```
plain: fib(32) = 2178309 with 7049155 calls
memo : fib(32) = 2178309 with 63 calls
memo : fib(90) = 2880067194370816120 (plain recursion would take years)
```

Declare before use, pass small values by value, large read-only objects by const reference, and use references (or pointers for optional arguments) to modify the caller's data. Return by value, bundle multiple results in a struct or with structured bindings, and never return a reference to a local. Overloading, default arguments, constexpr and [[nodiscard]] make functions safer and clearer. Recursion needs a base case, uses one stack frame per level, and becomes efficient with memoization.

PRACTICE CHECKPOINT

Write **overloaded area() functions with default arguments**, a **swap using references**, and **recursive Fibonacci with memoization**.

SAMPLE SOLUTION main.cpp

```cpp
#include <cmath>
#include <iostream>
#include <numbers>
#include <vector>

// overloaded area() with default arguments
double area(double radius) { return std::numbers::pi * radius * radius; }            // circle
double area(double length, double width) { return length * width; }                   // rectangle
double area(double a, double b, double c, bool triangle) {                            // triangle
(Heron)
    (void)triangle;
    double s = (a + b + c) / 2;
    return std::sqrt(s * (s - a) * (s - b) * (s - c));
}
int area(int side = 1, int copies = 1) { return side * side * copies; }              // squares
```

```cpp
// swap with references
void swap_values(int& a, int& b) { int t = a; a = b; b = t; }

// Fibonacci with memoization
long long fib(int n, std::vector<long long>& memo) {
    if (n < 2) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
}
int main() {
    std::cout << "circle r=2      : " << area(2.0) << '\n';
    std::cout << "rectangle 3x4.5 : " << area(3.0, 4.5) << '\n';
    std::cout << "triangle 3,4,5  : " << area(3.0, 4.0, 5.0, true) << '\n';
    std::cout << "square 5        : " << area(5) << '\n';
    std::cout << "3 squares of 5  : " << area(5, 3) << '\n';
    std::cout << "default square  : " << area() << '\n';

    int x = 10, y = 25;
    swap_values(x, y);
    std::cout << "after swap: x = " << x << ", y = " << y << '\n';

    int n = 50;
    std::vector<long long> memo(n + 1, -1);
    std::cout << "Fibonacci:";
    for (int i = 0; i <= 15; ++i) std::cout << ' ' << fib(i, memo);
    std::cout << "\nfib(" << n << ") = " << fib(n, memo) << '\n';
    return 0;
}
```

OUTPUT

```
circle r=2      : 12.5664
rectangle 3x4.5 : 13.5
triangle 3,4,5  : 6
square 5        : 25
3 squares of 5  : 75
default square  : 1
after swap: x = 25, y = 10
Fibonacci: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610
fib(50) = 12586269025
```

LEVEL 2

Intermediate level

Outcome: use pointers and references, build classes with inheritance and polymorphism, overload operators, handle exceptions, write templates and work with files.

9 MODULES · 25 DAYS MINIMUM

Module 10 · Pointers and references	**3 days**
Module 11 · Dynamic memory management	**2 days**
Module 12 · Structures, enums, unions and namespaces	**2 days**
Module 13 · Object-oriented programming: classes and objects	**4 days**
Module 14 · Inheritance and polymorphism	**5 days**
Module 15 · Operator overloading	**2 days**
Module 16 · Exception handling	**2 days**
Module 17 · Templates and generic programming	**3 days**
Module 18 · File handling with streams	**2 days**

MODULE 10 · INTERMEDIATE

Pointers and references

Minimum duration **3 days** | Level **Intermediate**

Pointers store memory addresses; references are aliases for existing objects. Together they let functions share and modify data without copying it, and they are the foundation of dynamic memory, data structures and polymorphism. This module covers both in depth, plus function pointers and a first look at value categories.

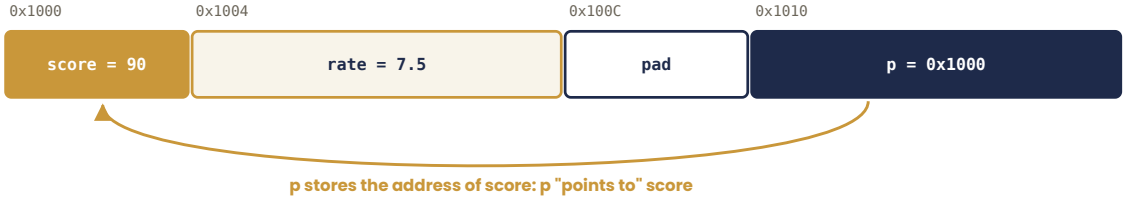
IN THIS MODULE

10.1 Pointers
10.2 Pointers and const
10.3 References
10.4 Function pointers
10.5 Value categories (introduction)

10.1 Pointers

10.1.1 Memory addresses and the & operator

Memory is a long sequence of bytes, each with a numeric **address**. Every variable occupies one or more consecutive bytes, and its address is the address of its first byte. The **address-of operator** `&` gives that address.



VARIABLES OCCUPY BYTE RANGES; A POINTER HOLDS ANOTHER VARIABLE'S ADDRESS

TAKING ADDRESSES

main.cpp

```cpp
#include <iostream>

int main() {
    int score = 90;
    double rate = 7.5;
    std::cout << "score = " << score << " at " << &score << " (" << sizeof score << " bytes)\n";
    std::cout << "rate  = " << rate << " at " << &rate << " (" << sizeof rate << " bytes)\n";
    return 0;
}
```

OUTPUT (addresses differ on every run)

```
score = 90 at 0x7ffc2e4bb09c (4 bytes)
rate  = 7.5 at 0x7ffc2e4bb0a0 (8 bytes)
```

10.1.2 Declaring, initializing and dereferencing pointers

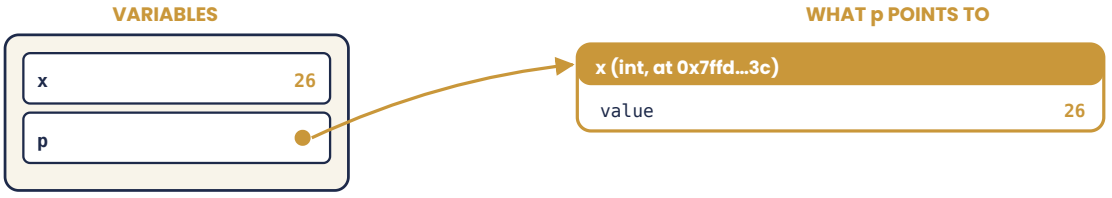
`int* p;` declares `p` as a **pointer to int**, a variable that holds the address of an int. `p = &x;` makes it point to x. The **dereference operator** `*p` means "the object p points to": you can read it and assign to it. Note the two meanings of `*`: in a declaration it says "is a pointer"; in an expression it means "follow the address".

USING A POINTER main.cpp

```
#include <iostream>

int main() {
    int x = 10;
    int* p = &x;                        // p holds x's address
    std::cout << "x = " << x << ", p = " << p << ", *p = " << *p << '\n';
    *p = 25;                            // writes to x through the pointer
    std::cout << "after *p = 25, x = " << x << '\n';
    ++*p;                               // increments x
    std::cout << "after ++*p, x = " << x << '\n';
    int y = 100;
    p = &y;                             // a pointer can be re-pointed
    std::cout << "now *p = " << *p << '\n';
    int* a, b;                          // careful: a is int*, b is a plain int!
    a = &b; b = 7;
    std::cout << "*a = " << *a << ", sizeof(int*) = " << sizeof(int*) << ", sizeof(double*) = " <<
sizeof(double*) << '\n';
    return 0;
}
```

OUTPUT (addresses differ per run)

```
x = 10, p = 0x7ffd49e7f4bc, *p = 10
after *p = 25, x = 25
after ++*p, x = 26
now *p = 100
*a = 7, sizeof(int*) = 8, sizeof(double*) = 8
```



P STORES X'S ADDRESS; *P IS X ITSELF

Expression	Meaning	Value (x = 26, p = &x)
`x`	value of x	26
`&x`	address of x	0x7ffd…3c
`p`	value of p: an address	0x7ffd…3c
`*p`	object at that address: x	26
`&p`	address of the pointer itself	0x7ffd…30
`*&x`	* and & cancel out	26

10.1.3 nullptr (C++11) vs NULL and 0

A pointer that points nowhere should hold the **null pointer** value. C++11 introduced the keyword `nullptr` for it, which has its own type (`std::nullptr_t`) and converts only to pointer types. The older `NULL` is a macro for 0 (or 0L), which is an **integer**, so it can silently pick the wrong overload. Always use `nullptr`, and check before dereferencing: dereferencing a null pointer is undefined behaviour (usually a crash).

NULLPTR

main.cpp

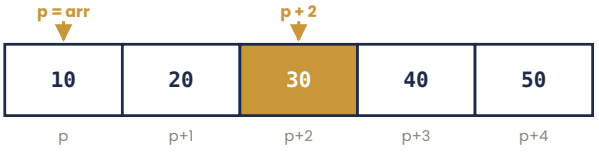
```
#include <iostream>

void f(int)   { std::cout << "f(int) called\n"; }
void f(int*)  { std::cout << "f(int*) called\n"; }

int main() {
    int* p = nullptr;
    if (p == nullptr) std::cout << "p is null\n";
    if (!p) std::cout << "a null pointer converts to false\n";
    f(0);            // 0 is an int
    f(nullptr);      // unambiguous: the pointer overload
    // f(NULL);      // with GCC: ambiguous or picks f(int), never what you meant
    int value = 5;
    p = &value;
    if (p) std::cout << "p now points to " << *p << '\n';
    return 0;
}
```

OUTPUT

```
p is null
a null pointer converts to false
f(int) called
f(int*) called
p now points to 5
```

10.1.4 Pointer arithmetic

Adding an integer n to a pointer moves it by **n elements**, not n bytes: `p + 1` points to the next int (4 bytes further for int, 8 for double). Subtracting two pointers into the same array gives the number of elements between them (type `std::ptrdiff_t`). Pointers into the same array can be compared. Arithmetic is only defined within an array and up to one past its end.



P + 2 MOVES TWO ELEMENTS (8 BYTES FOR INT)

MOVING A POINTER THROUGH AN ARRAY main.cpp

```cpp
#include <iostream>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int* p = arr;                                   // points to arr[0]
    std::cout << "*p = " << *p << ", *(p + 2) = " << *(p + 2) << '\n';
    std::cout << "p + 1 is " << reinterpret_cast<char*>(p + 1) - reinterpret_cast<char*>(p) << " bytes
after p\n";
    ++p;                                            // now arr[1]
    std::cout << "after ++p: *p = " << *p << ", p[1] = " << p[1] << '\n';
    int* last = &arr[4];
    std::cout << "last - p = " << last - p << " elements\n";
    for (int* q = arr; q != arr + 5; ++q) std::cout << *q << ' ';      // walk with a pointer
    std::cout << "\nsum via pointer: ";
    int sum = 0;
    for (const int* q = std::begin(arr); q < std::end(arr); ++q) sum += *q;
    std::cout << sum << '\n';
    return 0;
}
```

OUTPUT

```
*p = 10, *(p + 2) = 30
p + 1 is 4 bytes after p
after ++p: *p = 20, p[1] = 30
last - p = 3 elements
10 20 30 40 50
sum via pointer: 150
```

10.1.5 Pointers and arrays

In most expressions an array name **decays** to a pointer to its first element, so `arr` means `&arr[0]`. Indexing is defined through pointers: `arr[i]` is exactly `*(arr + i)`. An array is **not** a pointer, though: `sizeof arr` is the whole array, and an array name cannot be re-pointed.

ARRAY NAMES AND POINTERS main.cpp

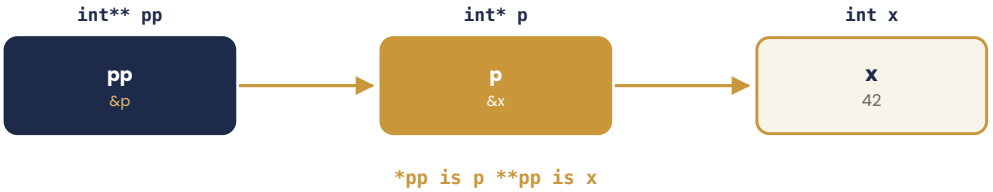
```cpp
#include <iostream>

int main() {
    int arr[4] = {3, 6, 9, 12};
    int* p = arr;
    std::cout << arr[2] << ' ' << *(arr + 2) << ' ' << p[2] << ' ' << *(p + 2) << " (all the same)\n";
    std::cout << "arr == &arr[0] ? " << std::boolalpha << (arr == &arr[0]) << '\n';
    std::cout << "sizeof arr = " << sizeof arr << ", sizeof p = " << sizeof p << '\n';
    // arr = p;        error: an array name cannot be assigned
    const char* s = "pointer";
    int len = 0;
    while (*(s + len) != '\0') ++len;
    std::cout << "length of \"" << s << "\" by pointer walk: " << len << '\n';
    return 0;
}
```

OUTPUT

```
9 9 9 9 (all the same)
arr == &arr[0] ? true
sizeof arr = 16, sizeof p = 8
length of "pointer" by pointer walk: 7
```

10.1.6 Pointer to pointer

A pointer is itself a variable with an address, so another pointer can point to it: `int** pp = &p;`. `*pp` is p and `**pp` is the int. Pointers to pointers appear with arrays of strings (`char** argv`), 2D dynamic arrays, and C APIs that return a pointer through a parameter.



*pp is p **pp is x

TWO LEVELS OF INDIRECTION

DOUBLE POINTERS

main.cpp

```cpp
#include <iostream>

void allocate(int** out) {          // C-style: change the caller's pointer through int**
    *out = new int(77);
}
void allocate_ref(int*& out) {      // C++ style: a reference to a pointer
    out = new int(88);
}
int main() {
    int x = 42;
    int* p = &x;
    int** pp = &p;
    std::cout << "x = " << x << ", *p = " << *p << ", **pp = " << **pp << '\n';
    **pp = 50;
    std::cout << "after **pp = 50, x = " << x << '\n';
    int* a = nullptr;
    int* b = nullptr;
    allocate(&a);
    allocate_ref(b);
    std::cout << "*a = " << *a << ", *b = " << *b << '\n';
    delete a;
    delete b;
    return 0;
}
```

OUTPUT

```
x = 42, *p = 42, **pp = 42
after **pp = 50, x = 50
*a = 77, *b = 88
```

10.1.7 void* pointers

`void*` is a pointer to "something of unknown type". Any object pointer converts to void* implicitly, but to use the object you must cast back to the correct type with `static_cast`; void* cannot be dereferenced or used in arithmetic. It is common in C APIs (malloc, memcpy, thread arguments) and low-level code. In C++, templates and std::any are type-safe alternatives.

VOID* AND BYTE VIEWS

main.cpp

```
#include <cstring>
#include <iostream>

void print_bytes(const void* data, std::size_t n) {
    const unsigned char* bytes = static_cast<const unsigned char*>(data);
    for (std::size_t i = 0; i < n; ++i) std::cout << std::hex << static_cast<int>(bytes[i]) << ' ';
    std::cout << std::dec << '\n';
}
int main() {
    int n = 258;                      // 0x00000102
    double d = 1.0;
    void* vp = &n;                    // any object pointer converts to void*
    int* ip = static_cast<int*>(vp);  // cast back to use it
    std::cout << "*ip = " << *ip << '\n';
    std::cout << "bytes of 258: ";  print_bytes(&n, sizeof n);
    std::cout << "bytes of 1.0: ";  print_bytes(&d, sizeof d);
    int copy;
    std::memcpy(&copy, &n, sizeof n);  // memcpy takes void* parameters
    std::cout << "copy = " << copy << '\n';
    return 0;
}
```

OUTPUT

```
*ip = 258
bytes of 258: 2 1 0 0
bytes of 1.0: 0 0 0 0 0 0 f0 3f
copy = 258
```

10.1.8 Wild and dangling pointers

A **wild pointer** is an uninitialised pointer: it holds garbage and may point anywhere. A **dangling pointer** points to an object that no longer exists: a local variable whose function has returned, or memory that has been deleted. Using either is undefined behaviour, often silently corrupting data. Prevention: always initialise pointers (to an address or nullptr), set them to nullptr after delete, never return the address of a local, and prefer references, containers and smart pointers.

HOW POINTERS GO BAD main.cpp

```cpp
#include <iostream>

int* make_local() {
    int local = 123;
    return &local;              // BUG: local dies when the function returns
}
int main() {
    int* wild;                  // BUG: uninitialized: never dereference
    int* dangling = make_local();
    int* heap = new int(5);
    delete heap;                // heap now dangles
    heap = nullptr;             // fix: make it a clearly null pointer
    std::cout << "wild and dangling pointers compiled; using them would be undefined behaviour\n";
    std::cout << "heap is " << (heap ? "set" : "nullptr after delete") << '\n';
    (void)wild; (void)dangling;
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int* make_local()’:
main.cpp:5:12: warning: address of local variable ‘local’ returned [-Wreturn-local-addr]
    5 |     return &local;              // BUG: local dies when the function returns
      |            ^~~~~~
main.cpp:4:9: note: declared here
    4 |     int local = 123;
      |         ^~~~~
$ ./prog
wild and dangling pointers compiled; using them would be undefined behaviour
heap is nullptr after delete
```

10.2 Pointers and const

Two different things can be constant: the **data** pointed to, and the **pointer** itself. Read the declaration from right to left: `const int* p` is "p is a pointer to an int that is const"; `int* const p` is "p is a const pointer to an int".

10.2.1 Pointer to const

`const int* p` (also written `int const* p`): the int cannot be changed **through p**, but p can be re-pointed. Use it for read-only access, for example function parameters that only inspect data.

10.2.2 Const pointer

`int* const p = &x;`: p must be initialised and always points to x, but x can be modified through it.

10.2.3 Const pointer to const

`const int* const p`: neither the pointer nor the data can change through it.

Declaration	Read as	`*p = 5`	`p = &y`
`int* p`	pointer to int	✔	✔
`const int* p`	pointer to const int	✘	✔
`int* const p`	const pointer to int	✔	✘

Declaration	Read as	`*p = 5`	`p = &y`
`const int* const p`	const pointer to const int	✘	✘

CONST AND POINTERS

main.cpp

```
#include <iostream>

int sum(const int* a, int n) {            // promises not to modify the array
    int s = 0;
    for (const int* q = a; q != a + n; ++q) s += *q;
    return s;
}
int main() {
    int x = 10, y = 20;
    const int* p1 = &x;   p1 = &y;        // re-point: fine
    int* const p2 = &x;   *p2 = 11;       // modify x: fine
    const int* const p3 = &y;
    std::cout << *p1 << ' ' << x << ' ' << *p3 << '\n';
    int v[] = {1, 2, 3, 4};
    std::cout << "sum = " << sum(v, 4) << '\n';
    return 0;
}
```

OUTPUT

```
20 11 20
sum = 10
```

WHAT THE COMPILER REJECTS

main.cpp

```
int main() {
    int x = 10, y = 20;
    const int* p1 = &x;
    int* const p2 = &x;
    *p1 = 5;          // data is const through p1
    p2 = &y;          // p2 itself is const
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:5:9: error: assignment of read-only location ‘* p1’
    5 |     *p1 = 5;          // data is const through p1
      |     ~~~~^~~
main.cpp:6:8: error: assignment of read-only variable ‘p2’
    6 |     p2 = &y;          // p2 itself is const
      |     ~~~^~~~
```

10.3 References

10.3.1 Lvalue references int& r = x;

A **reference** is an alias: another name for an existing object. `int& r = x;` makes r refer to x, and from then on every use of r is a use of x. There is no separate "reference object" to manipulate, no dereferencing syntax, and no null reference. Under the hood the compiler usually implements references with addresses, but the language treats them as the object itself.

A REFERENCE IS AN ALIAS main.cpp

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    int x = 10;
    int& r = x;                     // r is another name for x
    r = 20;                         // assigns to x
    std::cout << "x = " << x << ", r = " << r << ", same address? " << std::boolalpha << (&r == &x) <<
'\n';
    std::vector<std::string> names = {"asha", "ravi"};
    std::string& first = names[0];    // alias to an element
    first[0] = 'A';
    std::cout << names[0] << '\n';
    for (auto& n : names) n += "!";   // reference loop variable modifies the elements
    std::cout << names[0] << ' ' << names[1] << '\n';
    return 0;
}
```

OUTPUT

```
x = 20, r = 20, same address? true
Asha
Asha! ravi!
```

10.3.2 References must be initialized and cannot be reseated

A reference must be bound to an object when it is created, and it stays bound to that same object for its whole life. Assigning to a reference later does **not** rebind it; it assigns the value to the referred object.

ASSIGNMENT THROUGH A REFERENCE main.cpp

```
#include <iostream>

int main() {
    int a = 1, b = 2;
    int& r = a;
    r = b;                      // NOT rebinding: copies b's value into a
    std::cout << "a = " << a << ", b = " << b << " (r still refers to a)\n";
    b = 99;
    std::cout << "r = " << r << " (unchanged by b)\n";
    return 0;
}
```

OUTPUT

```
a = 2, b = 2 (r still refers to a)
r = 2 (unchanged by b)
```

INVALID REFERENCES main.cpp

```
int main() {
    int& r;           // a reference must be initialized
    int& s = 5;       // a non-const reference cannot bind to a literal
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:2:10: error: ‘r’ declared as reference but not initialized
    2 |     int& r;           // a reference must be initialized
      |          ^
main.cpp:3:14: error: cannot bind non-const lvalue reference of type ‘int&’ to an rvalue of type ‘int’
    3 |     int& s = 5;       // a non-const reference cannot bind to a literal
      |              ^
main.cpp:2:10: warning: unused variable ‘r’ [-Wunused-variable]
    2 |     int& r;           // a reference must be initialized
      |          ^
main.cpp:3:10: warning: unused variable ‘s’ [-Wunused-variable]
    3 |     int& s = 5;       // a non-const reference cannot bind to a literal
      |          ^
```

10.3.3 Const references and binding to temporaries

A `const T&` can bind to anything of type T, including **temporaries** and literals. The temporary's lifetime is then extended to match the reference. This is why `const std::string&` parameters accept `"literals"`.

CONST REFERENCES ACCEPT TEMPORARIES main.cpp

```
#include <iostream>
#include <string>

std::string make_title() { return "Learn Being Forward"; }
void print(const std::string& s) { std::cout << "[" << s << "]\n"; }

int main() {
    const int& five = 5;                    // binds to a temporary int
    const std::string& t = make_title();    // temporary's lifetime extended
    std::cout << five << ' ' << t << '\n';
    print("a literal");                     // converted to a temporary std::string
    print(t + "!");
    return 0;
}
```

OUTPUT

```
5 Learn Being Forward
[a literal]
[Learn Being Forward!]
```

10.3.4 References as function parameters and return values

Reference parameters avoid copies and let functions modify arguments (Module 9.2). Functions may return references to objects that outlive the call, for example an element of a container or a member of an object, allowing chained or assignable calls. Never return a reference to a local variable.

REFERENCES IN AND OUT OF FUNCTIONS main.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

void to_upper(std::string& s) { for (char& c : s) if (c >= 'a' && c <= 'z') c -= 32; }

int& largest(std::vector<int>& v) {          // returns a reference to an element
    int* best = &v[0];
    for (int& x : v) if (x > *best) best = &x;
    return *best;
}
int main() {
    std::string s = "reference";
    to_upper(s);
    std::cout << s << '\n';
    std::vector<int> v = {4, 17, 9};
    largest(v) = 0;                           // modifies the element itself
    std::cout << v[0] << ' ' << v[1] << ' ' << v[2] << '\n';
    return 0;
}
```

OUTPUT

```
REFERENCE
4 0 9
```

10.3.5 Pointers vs references comparison

	Pointer `T*`	Reference `T&`
Can be null	yes (nullptr)	no
Must be initialised	no (but should be)	yes
Can be re-pointed	yes	no, bound for life
Syntax to access the object	`*p`, `p->m`	plain name, `r.m`
Arithmetic	yes	no
Has its own address	yes (`&p`)	no (`&r` is the object's address)
Typical use	optional objects, arrays, dynamic memory, data structures	parameters, return values, aliases, range-for

TIP

Guideline: use a reference when an object must exist, and a pointer when "no object" is a valid possibility or the target must change. For owning heap memory, use smart pointers (Module 20) rather than raw pointers.

10.4 Function pointers

10.4.1 Declaring and calling function pointers

Functions have addresses too. A **function pointer** stores the address of a function with a given signature: `int (*op)(int, int);` declares op as a pointer to a function taking two ints and returning int. The parentheses around `*op` are essential. A type alias makes this readable: `using BinOp = int(*)(int, int);`.

FUNCTION POINTERS

main.cpp

```
#include <iostream>

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int main() {
    int (*op)(int, int) = add;            // a function name decays to its address
    std::cout << "op(6, 7) = " << op(6, 7) << '\n';
    op = &mul;                            // & is optional
    std::cout << "(*op)(6, 7) = " << (*op)(6, 7) << '\n';
    using BinOp = int (*)(int, int);
    BinOp table[] = {add, mul};           // an array of function pointers
    const char* names[] = {"add", "mul"};
    for (int i = 0; i < 2; ++i) std::cout << names[i] << "(3, 4) = " << table[i](3, 4) << '\n';
    return 0;
}
```

OUTPUT

```
op(6, 7) = 13
(*op)(6, 7) = 42
add(3, 4) = 7
mul(3, 4) = 12
```

10.4.2 Callbacks

A **callback** is a function passed to another function, which calls it back at the right moment. Callbacks let one generic function do many jobs: sorting with a custom order, applying any transformation, reacting to events.

PASSING FUNCTIONS AS ARGUMENTS main.cpp

```
#include <algorithm>
#include <iostream>
#include <vector>

void apply(std::vector<int>& v, int (*f)(int)) {      // accepts any int -> int function
    for (int& x : v) x = f(x);
}
int square(int x) { return x * x; }
int negate(int x) { return -x; }
bool descending(int a, int b) { return a > b; }

int main() {
    std::vector<int> v = {3, 1, 4, 1, 5};
    apply(v, square);
    for (int x : v) std::cout << x << ' ';
    apply(v, negate);
    std::cout << "| ";
    for (int x : v) std::cout << x << ' ';
    std::sort(v.begin(), v.end(), descending);         // the standard library uses callbacks too
    std::cout << "| sorted descending: ";
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
9 1 16 1 25 | -9 -1 -16 -1 -25 | sorted descending: -1 -1 -9 -16 -25
```

10.4.3 std::function (introduced; detailed in Module 20)

A function pointer can only hold plain functions. `std::function<int(int)>` from `<functional>` can hold **anything callable** with that signature: functions, lambdas (including ones that capture variables), and function objects. It is the flexible choice for stored callbacks.

STD::FUNCTION HOLDS ANY CALLABLE main.cpp

```
#include <functional>
#include <iostream>
#include <vector>

int twice(int x) { return 2 * x; }

int main() {
    int offset = 10;
    std::vector<std::function<int(int)>> steps = {
        twice,                                  // a plain function
        [](int x) { return x + 1; },            // a lambda
        [offset](int x) { return x - offset; }  // a lambda that captures a variable
    };
    int value = 7;
    for (const auto& f : steps) value = f(value);
    std::cout << "((7 * 2) + 1) - 10 = " << value << '\n';
    return 0;
}
```

OUTPUT

```
((7 * 2) + 1) - 10 = 5
```

10.5 Value categories (introduction)

10.5.1 Lvalues and rvalues

Every expression is either an **lvalue** or an **rvalue** (C++11 refines this further). Roughly: an **lvalue** names an object with an identity and a lasting address, something that can appear on the left of `=`: a variable, `v[2]`, `*p`. An **rvalue** is a temporary value without a lasting identity: `42`, `x + 1`, the result of a function returning by value. You can take the address of an lvalue but not of an rvalue.

Expression	Category	Why
`x`	lvalue	a named variable
`v[2]`, `*p`	lvalue	refers to an existing object
`42`	rvalue	a literal value
`x + 1`	rvalue	a computed temporary
`std::string("hi")`	rvalue	a temporary object
`make_name()` returning by value	rvalue	a temporary result
`++x`	lvalue	returns x itself
`x++`	rvalue	returns a copy of the old value

10.5.2 Rvalue references && (introduced; detailed in Module 20)

C++11 added **rvalue references**, written `T&&`, which bind only to rvalues (temporaries). Because a temporary is about to disappear anyway, a function receiving it by `T&&` may safely "steal" its resources instead of copying them. This is the basis of **move semantics**, which makes returning and passing large objects cheap.

OVERLOADING ON LVALUES AND RVALUES

main.cpp

```
#include <iostream>
#include <string>
#include <utility>

void take(std::string& s)        { std::cout << "lvalue reference: " << s << '\n'; }
void take(std::string&& s)       { std::cout << "rvalue reference: " << s << " (may steal its
buffer)\n"; }

int main() {
    std::string name = "Asha";
    take(name);                          // name is an lvalue
    take(std::string("temporary"));      // a temporary is an rvalue
    take(name + " Rao");                 // the result of + is an rvalue
    take(std::move(name));               // std::move turns an lvalue into an rvalue
    std::string target = std::move(name);   // move construction: name's contents are stolen
    std::cout << "target = \"" << target << "\", name is now \"" << name << "\" (valid but
unspecified, here empty)\n";
    return 0;
}
```

OUTPUT

```
lvalue reference: Asha
rvalue reference: temporary (may steal its buffer)
rvalue reference: Asha Rao (may steal its buffer)
rvalue reference: Asha (may steal its buffer)
target = "Asha", name is now "" (valid but unspecified, here empty)
```

A pointer holds an address: & takes it, * follows it, arithmetic moves by elements, nullptr means "nowhere", and const can protect the data, the pointer, or both. A reference is a permanent alias that must be initialised and cannot be null. Prefer references for required parameters, pointers for optional or re-pointable ones, and never keep pointers or references to dead objects. Function pointers and std::function pass behaviour around; rvalue references enable moves.

PRACTICE CHECKPOINT

Swap two values three ways (by value, by pointer, by reference) and **explain why only two of them work**.

SAMPLE SOLUTION main.cpp

```cpp
#include <iostream>

void swap_by_value(int a, int b) {           // receives COPIES
    int t = a; a = b; b = t;                 // swaps the copies only
    std::cout << "  inside swap_by_value: a = " << a << ", b = " << b << '\n';
}
void swap_by_pointer(int* a, int* b) {        // receives ADDRESSES
    if (!a || !b) return;
    int t = *a; *a = *b; *b = t;             // writes through the addresses
}
void swap_by_reference(int& a, int& b) {      // receives ALIASES
    int t = a; a = b; b = t;                 // a and b ARE the caller's variables
}
int main() {
    int x = 1, y = 2;
    swap_by_value(x, y);
    std::cout << "after swap_by_value    : x = " << x << ", y = " << y << "   (unchanged)\n";
    swap_by_pointer(&x, &y);
    std::cout << "after swap_by_pointer  : x = " << x << ", y = " << y << "   (swapped)\n";
    swap_by_reference(x, y);
    std::cout << "after swap_by_reference: x = " << x << ", y = " << y << "   (swapped back)\n";
    return 0;
}
```

OUTPUT

```
  inside swap_by_value: a = 2, b = 1
after swap_by_value    : x = 1, y = 2   (unchanged)
after swap_by_pointer  : x = 2, y = 1   (swapped)
after swap_by_reference: x = 1, y = 2   (swapped back)
```

Explanation. Pass by value gives the function its own copies of x and y; swapping the copies does not touch the originals, which vanish when the function returns. Pass by pointer gives the function the **addresses** of x and y, so writing to `*a` and `*b` changes the caller's variables. Pass by reference makes `a` and `b` other names for x and y, so the swap acts on the originals directly, with cleaner syntax and no possibility of null. In real code use `std::swap(x, y)`.

MODULE 11 · INTERMEDIATE

Dynamic memory management

Minimum duration **2 days** | Level **Intermediate**

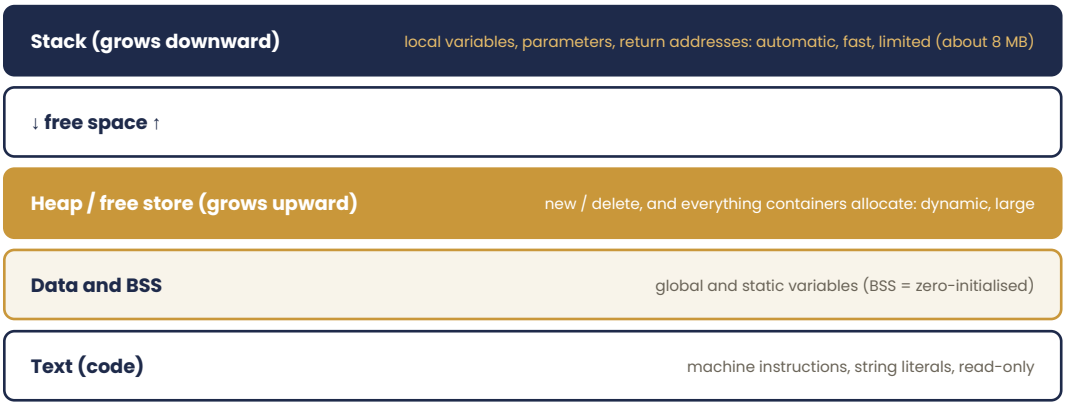
Dynamic memory lets a program create objects on the heap at run time, in any number and of any size, and keep them alive beyond the function that created them. This module covers manual management with new and delete, the memory errors it invites, and why modern C++ wraps every resource in RAII objects instead.

IN THIS MODULE

11.1 Stack vs heap
11.2 new and delete
11.3 Dynamic arrays
11.4 Memory errors
11.5 RAII (introduction)

11.1 Stack vs heap

11.1.1 Memory layout of a C++ program



TYPICAL MEMORY LAYOUT OF A RUNNING C++ PROGRAM

WHERE THINGS LIVE main.cpp

```cpp
#include <iostream>
#include <vector>

int global_counter = 0;                        // data / BSS segment
static const char* motto = "Learn Being Forward";   // the text lives in read-only data

int main() {
    int local = 5;                             // stack
    int* heap_int = new int(7);                // heap
    std::vector<int> v(3);                     // the vector object is on the stack, its elements on
the heap
    std::cout << "code  (main)     : " << reinterpret_cast<void*>(&main) << '\n';
    std::cout << "literal          : " << static_cast<const void*>(motto) << '\n';
    std::cout << "global           : " << &global_counter << '\n';
    std::cout << "heap int         : " << heap_int << '\n';
    std::cout << "vector elements  : " << v.data() << '\n';
    std::cout << "stack local      : " << &local << '\n';
    std::cout << "vector object    : " << &v << '\n';
    delete heap_int;
    return 0;
}
```

OUTPUT (addresses differ per run; note the ordering)

```
code  (main)     : 0x5582cc566289
literal          : 0x5582cc567008
global           : 0x5582cc569154
heap int         : 0x5582e56ee2b0
vector elements  : 0x5582e56ee2d0
stack local      : 0x7ffcdc78043c
vector object    : 0x7ffcdc780450
```

11.1.2 Automatic vs dynamic lifetime

	Automatic (stack)	Dynamic (heap)
Created by	declaring a local variable	`new` (or a container / smart pointer)
Destroyed	automatically at the end of its scope	when you call `delete` (or the owner does)
Size	fixed at compile time	chosen at run time
Lifetime	tied to the block	until deleted: can outlive the function that created it
Speed	very fast	slower: the allocator must find space
Limit	small stack (a few MB)	available RAM

Use the heap when you do not know how many objects you need until run time, when an object must outlive the scope that creates it, or when it is too large for the stack.

11.2 new and delete

11.2.1 Allocating single objects with new

`new T` allocates memory for one T on the heap, **constructs** the object there, and returns a `T*`. `delete p` runs the object's **destructor** and frees the memory. Every `new` must be matched by exactly one `delete`.

NEW AND DELETE FOR SINGLE OBJECTS

main.cpp

```
#include <iostream>
#include <string>

struct Student {
    std::string name;
    int marks;
    Student(std::string n, int m) : name(std::move(n)), marks(m) { std::cout << "  constructed " <<
name << '\n'; }
    ~Student() { std::cout << "  destroyed " << name << '\n'; }
};
Student* make_topper() {
    return new Student("Meera", 95);          // outlives this function
}
int main() {
    int* p = new int(42);
    std::cout << "*p = " << *p << '\n';
    delete p;

    Student* s = make_topper();
    std::cout << s->name << " scored " << s->marks << '\n';
    delete s;                                 // runs the destructor, frees the memory
    std::cout << "end of main\n";
    return 0;
}
```

OUTPUT

```
*p = 42
  constructed Meera
Meera scored 95
  destroyed Meera
end of main
```

11.2.2 Allocating arrays with new[]

`new T[n]` allocates an array of n objects, where n may be a run-time value, and returns a pointer to the first element. The array must be freed with `delete[]`.

11.2.3 delete vs delete[]

`delete` is for single objects created with `new`; `delete[]` is for arrays created with `new[]`. Mixing them is undefined behaviour: `delete` on an array destroys only the first element (and may corrupt the heap), because only `delete[]` knows to call every element's destructor.

DYNAMIC ARRAYS AND DELETE[] main.cpp

```cpp
#include <iostream>

struct Tracker {
    int id;
    Tracker() : id(++count) { std::cout << "+" << id << ' '; }
    ~Tracker() { std::cout << "-" << id << ' '; }
    static inline int count = 0;
};
int main() {
    int n;
    std::cout << "How many scores? ";
    std::cin >> n;
    int* scores = new int[n];                  // size known only at run time
    for (int i = 0; i < n; ++i) scores[i] = (i + 1) * 10;
    for (int i = 0; i < n; ++i) std::cout << scores[i] << ' ';
    std::cout << '\n';
    delete[] scores;                           // arrays need delete[]

    Tracker* t = new Tracker[3];
    std::cout << '\n';
    delete[] t;                                // destructors run for all three
    std::cout << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
How many scores? 4
10 20 30 40
+1 +2 +3
-3 -2 -1
```

11.2.4 Initializing dynamically allocated memory

INITIALISATION FORMS main.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    int* a = new int;            // default-initialized: indeterminate value for int
    int* b = new int();          // value-initialized: 0
    int* c = new int{7};         // 7
    int* arr1 = new int[5];      // indeterminate values
    int* arr2 = new int[5]();    // all zero
    int* arr3 = new int[5]{1, 2, 3};          // 1 2 3 0 0
    std::string* s = new std::string(3, 'x'); // class types are always constructed: "xxx"
    std::cout << *b << ' ' << *c << " | ";
    for (int i = 0; i < 5; ++i) std::cout << arr2[i] << arr3[i] << ' ';
    std::cout << "| " << *s << '\n';
    delete a; delete b; delete c; delete[] arr1; delete[] arr2; delete[] arr3; delete s;
    return 0;
}
```

OUTPUT

```
0 7 | 01 02 03 00 00 | xxx
```

11.2.5 new vs malloc

	`new` / `delete`	`malloc` / `free` (C)
Returns	a correctly typed pointer	`void*` that must be cast
Size	computed from the type	bytes, computed by you
Constructors / destructors	called	**not** called: unusable for std::string, std::vector, classes
Failure	throws `std::bad_alloc`	returns NULL
Can be customised	operator new can be overloaded per class	no

WATCH OUT

Never mix the families: memory from `new` must be freed with `delete`, memory from `malloc` with `free`. In C++ code use new (or, better, containers and smart pointers); malloc is only for interfacing with C.

11.2.6 std::bad_alloc and nothrow

When the heap cannot satisfy a request, `new` throws `std::bad_alloc` (from `<new>`). With `new (std::nothrow) T` it returns `nullptr` instead, C-style.

ALLOCATION FAILURE main.cpp

```cpp
#include <iostream>
#include <new>

int main() {
    try {
        std::size_t huge = static_cast<std::size_t>(1) << 40;   // 4 terabytes of ints
        int* p = new int[huge];
        delete[] p;
    } catch (const std::bad_alloc& e) {
        std::cout << "caught std::bad_alloc: " << e.what() << '\n';
    }
    std::size_t huge = static_cast<std::size_t>(1) << 40;
    int* q = new (std::nothrow) int[huge];
    std::cout << "nothrow new returned " << (q ? "memory" : "nullptr") << '\n';
    delete[] q;                          // deleting nullptr is harmless
    return 0;
}
```

OUTPUT

```
caught std::bad_alloc: std::bad_alloc
nothrow new returned nullptr
```

11.3 Dynamic arrays

11.3.1 Dynamic 1D arrays

A raw dynamic array is a pointer plus a size you must track yourself. Growing it means allocating a bigger array, copying, and deleting the old one, exactly what std::vector does for you. The manual version is shown here to understand vector, not as a recommendation.

A HAND-MADE GROWABLE ARRAY (WHAT VECTOR DOES INTERNALLY)

main.cpp

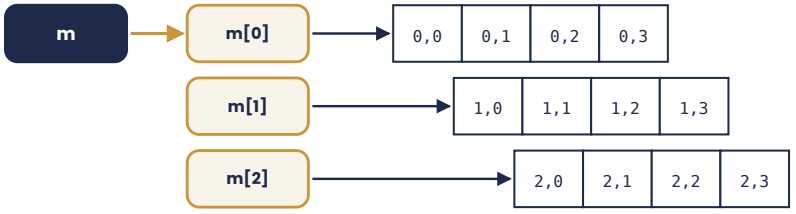
```cpp
#include <iostream>

int main() {
    int capacity = 2, size = 0;
    int* data = new int[capacity];
    int x;
    while (std::cin >> x) {
        if (size == capacity) {                       // full: grow by doubling
            int* bigger = new int[capacity * 2];
            for (int i = 0; i < size; ++i) bigger[i] = data[i];
            delete[] data;
            data = bigger;
            capacity *= 2;
            std::cout << "  grew to capacity " << capacity << '\n';
        }
        data[size++] = x;
    }
    std::cout << size << " numbers:";
    for (int i = 0; i < size; ++i) std::cout << ' ' << data[i];
    std::cout << '\n';
    delete[] data;
    return 0;
}
```

CONSOLE (user input in gold)

```
5 8 13 21 34 55
  grew to capacity 4
  grew to capacity 8
6 numbers: 5 8 13 21 34 55
```

11.3.2 Dynamic 2D arrays

Two common layouts: an array of row pointers, each pointing to its own row (supports `m[i][j]`, rows can differ in length, many allocations), or a single contiguous block indexed with `i * cols + j` (one allocation, cache-friendly). In modern C++ use `std::vector<std::vector<T>>` or a single `std::vector<T>` with index arithmetic.



ARRAY OF ROW POINTERS: ROWS+1 ALLOCATIONS, ROWS ANYWHERE ON THE HEAP

TWO WAYS TO BUILD A DYNAMIC MATRIX main.cpp

```
#include <iomanip>
#include <iostream>

int main() {
    int rows = 3, cols = 4;
    // method 1: array of row pointers
    int** m = new int*[rows];
    for (int r = 0; r < rows; ++r) m[r] = new int[cols];
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c) m[r][c] = r * cols + c;
    for (int r = 0; r < rows; ++r) {
        for (int c = 0; c < cols; ++c) std::cout << std::setw(3) << m[r][c];
        std::cout << '\n';
    }
    for (int r = 0; r < rows; ++r) delete[] m[r];     // free the rows first
    delete[] m;                                        // then the array of pointers

    // method 2: one contiguous block
    int* block = new int[rows * cols];
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c) block[r * cols + c] = (r + 1) * (c + 1);
    std::cout << "block[2][3] = " << block[2 * cols + 3] << '\n';
    delete[] block;
    return 0;
}
```

OUTPUT

```
  0  1  2  3
  4  5  6  7
  8  9 10 11
block[2][3] = 12
```

11.4 Memory errors

Manual memory management invites four classic bugs. None of them is caught by the compiler, and they often do not crash at once. Tools such as AddressSanitizer (`-fsanitize=address`) and Valgrind find them (Module 24).

Memory leak
new without delete: memory is never returned

Dangling pointer
using memory after it was deleted

Double delete
deleting the same memory twice

Mismatched forms
new[] with delete, malloc with delete ...

11.4.1 Memory leaks

A **leak** occurs when the last pointer to a heap object is lost before the object is deleted: an early return, an exception, or reassigning the pointer. The memory stays reserved until the program ends. In long-running programs (servers, games) leaks accumulate until memory runs out.

A LEAK ON AN EARLY-RETURN PATH, DETECTED BY COUNTING

main.cpp

```
#include <iostream>
#include <stdexcept>

int allocations = 0, deallocations = 0;      // simple counting to detect leaks

int* make_buffer(int n) { ++allocations; return new int[n]; }
void free_buffer(int* p) { ++deallocations; delete[] p; }

bool process(int n) {
    int* buf = make_buffer(n);
    if (n > 100) return false;               // BUG: early return leaks buf
    buf[0] = n;
    free_buffer(buf);
    return true;
}
int main() {
    for (int n : {10, 500, 20, 1000}) process(n);
    std::cout << "allocations " << allocations << ", deallocations " << deallocations
              << " -> " << allocations - deallocations << " leaked buffers\n";
    return 0;
}
```

OUTPUT

```
allocations 4, deallocations 2 -> 2 leaked buffers
```

11.4.2 Dangling pointers

SETTING ONE POINTER TO NULLPTR DOES NOT FIX ITS COPIES

main.cpp

```
#include <iostream>
#include <string>

int main() {
    std::string* p = new std::string("original data");
    std::string* alias = p;            // two pointers to the same object
    delete p;                          // the object is gone ...
    p = nullptr;                       // ... p is safe now, but alias still dangles
    std::cout << "p is " << (p ? "valid" : "nullptr") << "; alias still holds an address, using it is
undefined\n";
    // std::cout << *alias;   use after free: may print garbage, crash, or seem to work
    (void)alias;
    return 0;
}
```

OUTPUT

```
p is nullptr; alias still holds an address, using it is undefined
```

11.4.3 Double delete

11.4.4 Mismatched new/delete forms

DOUBLE DELETE main.cpp

```cpp
#include <iostream>

int main() {
    int* p = new int(5);
    delete p;
    std::cout << "deleted once\n" << std::flush;
    delete p;                          // BUG: double delete
    return 0;
}
```

OUTPUT (with AddressSanitizer; report trimmed)

```
$ g++ -g -fsanitize=address main.cpp -o prog && ./prog
deleted once
=================================================================
==3990==ERROR: AddressSanitizer: attempting double-free on 0x502000000010 in thread T0:
    #0 0x7f12bf8ff5e8 in operator delete(void*, unsigned long)
    #1 0x55b2017e72f6 in main main.cpp:7
0x502000000010 is located 0 bytes inside of 4-byte region [0x502000000010,0x502000000014)
freed by thread T0 here:
    #0 0x7f12bf8ff5e8 in operator delete(void*, unsigned long)
    #1 0x55b2017e72b5 in main main.cpp:5
SUMMARY: AddressSanitizer: double-free
```

NEW[] FREED WITH DELETE main.cpp

```cpp
int main() {
    int* arr = new int[10];
    delete arr;                        // BUG: should be delete[]
    return 0;
}
```

OUTPUT (with AddressSanitizer; report trimmed)

```
$ g++ -g -fsanitize=address main.cpp -o prog && ./prog
=================================================================
==3968==ERROR: AddressSanitizer: alloc-dealloc-mismatch (operator new [] vs operator delete) on
0x504000000010
    #0 0x7f935faff5e8 in operator delete(void*, unsigned long)
    #1 0x5573782e41d8 in main main.cpp:3
0x504000000010 is located 0 bytes inside of 40-byte region [0x504000000010,0x504000000038)
allocated by thread T0 here:
    #0 0x7f935fafe6c8 in operator new[](unsigned long)
    #1 0x5573782e41be in main main.cpp:2
SUMMARY: AddressSanitizer: alloc-dealloc-mismatch
```

GCC also warns at compile time for this simple case (`-Wmismatched-new-delete`), but in real code the new and the delete are usually far apart, in different functions or files.

11.5 RAII (introduction)

11.5.1 Resource Acquisition Is Initialization

RAII ties a resource to the lifetime of an object: the **constructor** acquires it (allocates memory, opens a file, locks a mutex) and the **destructor** releases it. Because C++ destroys automatic objects at the end of their scope, **no matter how** the scope is left (normal end, return, break, or an exception), the resource is always released, exactly once. RAII turns manual cleanup into automatic cleanup.

RAII RELEASES MEMORY ON EVERY PATH

main.cpp

```
#include <iostream>
#include <stdexcept>

class IntBuffer {                              // a minimal RAII wrapper around new[] / delete[]
public:
    explicit IntBuffer(std::size_t n) : data_(new int[n]()), size_(n) { std::cout << "  acquire " << n
<< " ints\n"; }
    ~IntBuffer() { delete[] data_; std::cout << "  release " << size_ << " ints\n"; }
    IntBuffer(const IntBuffer&) = delete;      // copying is forbidden (Module 13 explains why)
    IntBuffer& operator=(const IntBuffer&) = delete;
    int& operator[](std::size_t i) { return data_[i]; }
    std::size_t size() const { return size_; }
private:
    int* data_;
    std::size_t size_;
};
bool process(std::size_t n) {
    IntBuffer buf(n);
    if (n > 100) return false;                 // early return: destructor still runs
    if (n == 13) throw std::runtime_error("unlucky size");   // exception: destructor still runs
    buf[0] = 1;
    return true;
}
int main() {
    for (std::size_t n : {10, 500, 13}) {
        try {
            bool ok = process(n);
            std::cout << "process(" << n << ") -> " << std::boolalpha << ok << '\n';
        } catch (const std::exception& e) {
            std::cout << "exception: " << e.what() << '\n';
        }
    }
    return 0;
}
```

OUTPUT

```
  acquire 10 ints
  release 10 ints
process(10) -> true
  acquire 500 ints
  release 500 ints
process(500) -> false
  acquire 13 ints
  release 13 ints
exception: unlucky size
```

11.5.2 Why modern C++ avoids raw new/delete (smart pointers in Module 20)

The C++ Core Guidelines say: **no naked new and delete**. Every heap resource should be owned by an RAII object:

Need	Use instead of new/delete
a dynamic array	`std::vector<T>`
a dynamic string	`std::string`
one object with a single owner	`std::unique_ptr<T>` via `std::make_unique<T>(args)`
one object shared by several owners	`std::shared_ptr<T>` via `std::make_shared<T>(args)`
an optional value	`std::optional<T>`
a file / lock / socket	std::fstream, std::lock_guard, a small RAII class

THE MODERN WAY: NO NEW, NO DELETE

main.cpp

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Report {
    std::string title;
    explicit Report(std::string t) : title(std::move(t)) {}
    ~Report() { std::cout << "  report \"" << title << "\" freed automatically\n"; }
};
int main() {
    std::vector<int> numbers(1000, 7);                          // instead of new int[1000]
    auto report = std::make_unique<Report>("Q3 results");       // instead of new Report(...)
    auto shared = std::make_shared<Report>("shared summary");
    auto another_owner = shared;                                // two owners, one object
    std::cout << numbers.size() << " numbers, " << report->title << ", owners of summary: " <<
shared.use_count() << '\n';
    std::cout << "leaving main: no delete anywhere\n";
    return 0;
}
```

OUTPUT

```
1000 numbers, Q3 results, owners of summary: 2
leaving main: no delete anywhere
  report "shared summary" freed automatically
  report "Q3 results" freed automatically
```

Stack objects are automatic; heap objects live until deleted. new/delete and new[]/delete[] must be paired exactly, and mistakes cause leaks, dangling pointers, double deletes and mismatches that the compiler cannot see. RAII solves this: owning objects release resources in their destructors on every path. In modern C++ use vector, string, unique_ptr and shared_ptr instead of raw new and delete.

PRACTICE CHECKPOINT

Allocate and free a dynamic 2D matrix correctly, then **deliberately leak memory and detect it** with a tool or by counting allocations.

SAMPLE SOLUTION

main.cpp

```
#include <iomanip>
#include <iostream>

static long live_blocks = 0;                    // allocation counter

template <typename T> T* counted_new_array(std::size_t n) { ++live_blocks; return new T[n](); }
template <typename T> void counted_delete_array(T* p) { if (p) { --live_blocks; delete[] p; } }

double** create_matrix(int rows, int cols) {
    double** m = counted_new_array<double*>(rows);
    for (int r = 0; r < rows; ++r) m[r] = counted_new_array<double>(cols);
    return m;
}
void destroy_matrix(double** m, int rows) {
    for (int r = 0; r < rows; ++r) counted_delete_array(m[r]);
    counted_delete_array(m);
}
int main() {
    int rows = 3, cols = 3;
    double** m = create_matrix(rows, cols);
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c) m[r][c] = (r == c) ? 1.0 : 0.5 * (r + c);
    for (int r = 0; r < rows; ++r) {
        for (int c = 0; c < cols; ++c) std::cout << std::setw(6) << m[r][c];
        std::cout << '\n';
    }
    std::cout << "live blocks while in use : " << live_blocks << '\n';
    destroy_matrix(m, rows);
    std::cout << "after destroy_matrix     : " << live_blocks << " (correct: nothing leaked)\n";

    double** leaky = create_matrix(rows, cols);
    counted_delete_array(leaky);                // BUG on purpose: rows never freed
    std::cout << "after the buggy cleanup  : " << live_blocks << " blocks still allocated -> LEAK of "
              << live_blocks * cols * sizeof(double) << " bytes\n";
    return 0;
}
```

OUTPUT

```
     1   0.5     1
   0.5     1   1.5
     1   1.5     1
live blocks while in use : 4
after destroy_matrix     : 0 (correct: nothing leaked)
after the buggy cleanup  : 3 blocks still allocated -> LEAK of 72 bytes
```

A tool gives the same verdict without any counting code. Built with `g++ -g -fsanitize=address`, LeakSanitizer reports the three row arrays allocated in create_matrix at exit:

LeakSanitizer confirms the leak

```
$ g++ -std=c++20 -g -fsanitize=address matrix.cpp -o matrix && ./matrix
   ... (program output as above) ...
=================================================================
==4220==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 72 byte(s) in 3 object(s) allocated from:
    #0 0x7fabc6cfe6c8 in operator new[](unsigned long)
../../../../src/libsanitizer/asan/asan_new_delete.cpp:98
    #1 0x5568c2a53866 in double* counted_new_array<double>(unsigned long) matrix.cpp:6
    #2 0x5568c2a53378 in create_matrix(int, int) matrix.cpp:11
    #3 0x5568c2a5369e in main matrix.cpp:31

SUMMARY: AddressSanitizer: 72 byte(s) leaked in 3 allocation(s).
```

MODULE 12 · INTERMEDIATE

Structures, enums, unions and namespaces

Minimum duration **2 days** | Level **Intermediate**

These features group related data and organise names. Structures bundle values into records and, in C++, can carry their own functions, forming the bridge from procedural code to classes. Enumerations name sets of constants, unions overlay alternatives in memory, and namespaces keep large programs free of name clashes.

IN THIS MODULE

12.1 Structures
12.2 Structured bindings (C++17)
12.3 Enumerations
12.4 Unions
12.5 Namespaces

12.1 Structures

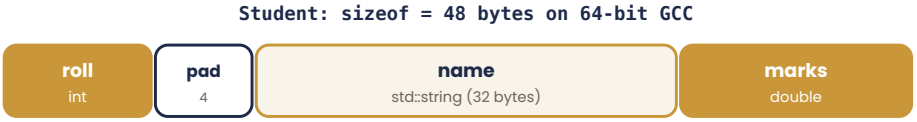
12.1.1 Defining a struct and creating variables

A **structure** groups related values of different types into one new type. In C++, after `struct Student { ... };` the name `Student` is a type on its own (no need to write `struct Student` as in C). The definition creates no memory; variables of the type do.

DEFINING A STRUCTURE

C++

```
struct Student {           // a new type
    int roll;               // data members
    std::string name;
    double marks;
};                          // the semicolon is required

Student s1;                 // a variable (object) of type Student
Student s2{102, "Ravi", 71.5};
```



MEMBERS ARE LAID OUT IN DECLARATION ORDER (PADDING EXPLAINED IN 12.1.8)

12.1.2 Member access with . and ->

ACCESSING MEMBERS

main.cpp

```
#include <iostream>
#include <string>

struct Student {
    int roll;
    std::string name;
    double marks;
};

int main() {
    Student s{101, "Asha", 88.5};
    std::cout << s.roll << ' ' << s.name << ' ' << s.marks << '\n';      // . on an object
    s.marks += 2;
    Student* p = &s;
    p->name = "Asha Rao";                                                 // -> through a pointer
    std::cout << p->name << " now has " << (*p).marks << '\n';            // (*p).m is the same as p->m
    Student copy = s;                                                     // memberwise copy
    copy.roll = 999;
    std::cout << "original roll " << s.roll << ", copy roll " << copy.roll << '\n';
    return 0;
}
```

OUTPUT

```
101 Asha 88.5
Asha Rao now has 90.5
original roll 101, copy roll 999
```

12.1.3 Arrays and vectors of structs

A CART AS A VECTOR OF STRUCTS, SORTED BY AMOUNT

main.cpp

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

struct Item { std::string name; int qty; double price; };

int main() {
    std::vector<Item> cart = {{"Notebook", 3, 45.0}, {"Pen", 10, 12.5}, {"Bag", 1, 899.0}};
    cart.push_back({"Eraser", 4, 5.0});
    std::sort(cart.begin(), cart.end(), [](const Item& a, const Item& b) { return a.price * a.qty >
b.price * b.qty; });
    double total = 0;
    for (const auto& it : cart) {
        double amt = it.qty * it.price;
        total += amt;
        std::cout << std::left << std::setw(10) << it.name << std::right << std::setw(4) << it.qty
                  << std::setw(9) << std::fixed << std::setprecision(2) << amt << '\n';
    }
    std::cout << "TOTAL " << std::setw(17) << total << '\n';
    return 0;
}
```

OUTPUT

```
Bag          1   899.00
Notebook     3   135.00
Pen         10   125.00
Eraser       4    20.00
TOTAL          1179.00
```

12.1.4 Nested structs

STRUCTURES INSIDE STRUCTURES

main.cpp

```
#include <iostream>
#include <string>

struct Date { int day, month, year; };
struct Address { std::string city; int pin; };
struct Employee {
    int id;
    std::string name;
    Date joined;              // a struct inside a struct
    Address addr;
};

int main() {
    Employee e{7, "Farah", {12, 6, 2023}, {"Bengaluru", 560001}};
    std::cout << e.name << " joined " << e.joined.day << '/' << e.joined.month << '/' << e.joined.year
              << " and lives in " << e.addr.city << " - " << e.addr.pin << '\n';
    e.addr.pin = 560034;
    std::cout << "new pin " << e.addr.pin << '\n';
    return 0;
}
```

OUTPUT

```
Farah joined 12/6/2023 and lives in Bengaluru - 560001
new pin 560034
```

12.1.5 Structs with member functions

Unlike C, a C++ struct can contain **member functions** that operate on its data. Inside a member function, members are used directly by name. Functions that do not modify the object are marked `const`. This is already a small class.

A STRUCT WITH BEHAVIOUR

main.cpp

```cpp
#include <cmath>
#include <iostream>

struct Point {
    double x = 0, y = 0;                              // default member initializers
    double distance_to(const Point& o) const {        // const: does not modify *this
        return std::hypot(x - o.x, y - o.y);
    }
    void move_by(double dx, double dy) { x += dx; y += dy; }
    void print() const { std::cout << '(' << x << ", " << y << ")"; }
};

int main() {
    Point a{1, 2}, b{4, 6};
    std::cout << "distance = " << a.distance_to(b) << '\n';
    a.move_by(3, 4);
    a.print();
    std::cout << '\n';
    Point origin;                                      // uses the defaults (0, 0)
    origin.print();
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
distance = 5
(4, 6)
(0, 0)
```

12.1.6 Aggregate and designated initialization (C++20)

A struct with only public data members and no user-declared constructors is an **aggregate**, initialised with a brace list in member order. C++20 adds **designated initializers** `{.name = …}`: members must appear in declaration order, but may be skipped (they get their default value), making long initialisations self-documenting.

AGGREGATE AND DESIGNATED INITIALISATION

main.cpp

```cpp
#include <iostream>
#include <string>

struct Config {
    std::string host = "localhost";
    int port = 8080;
    bool tls = false;
    int timeout_s = 30;
};

int main() {
    Config a;                                       // all defaults
    Config b{"example.com", 443, true};             // positional: timeout_s keeps its default
    Config c{.port = 9000, .timeout_s = 5};         // C++20 designated: host and tls default
    for (const Config& cfg : {a, b, c})
        std::cout << cfg.host << ':' << cfg.port << " tls=" << cfg.tls << " timeout=" << cfg.timeout_s
<< '\n';
    return 0;
}
```

OUTPUT

```
localhost:8080 tls=0 timeout=30
example.com:443 tls=1 timeout=30
localhost:9000 tls=0 timeout=5
```

12.1.7 struct vs class (default access)

In C++ `struct` and `class` are the **same thing** with one difference: members and base classes of a struct are **public** by default, while those of a class are **private** by default. By convention, `struct` is used for simple passive data bundles whose members are all public, and `class` for types with invariants, private data and behaviour (Module 13).

THE ONLY DIFFERENCE

main.cpp

```cpp
struct S { int value = 1; };       // public by default
class  C { int value = 1; };       // private by default

int main() {
    S s; s.value = 5;              // OK
    C c; c.value = 5;              // error: value is private
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:6:12: error: ‘int C::value’ is private within this context
    6 |     C c; c.value = 5;              // error: value is private
      |            ^~~~~
main.cpp:2:16: note: declared private here
    2 | class  C { int value = 1; };       // private by default
      |                ^~~~~
```

12.1.8 Padding and alignment (overview)

Each type has an **alignment**: an int usually must start at an address divisible by 4, a double by 8. The compiler inserts unused **padding** bytes between members (and at the end) to satisfy alignment, so a

struct can be larger than the sum of its members. Ordering members from largest to smallest alignment minimises padding.

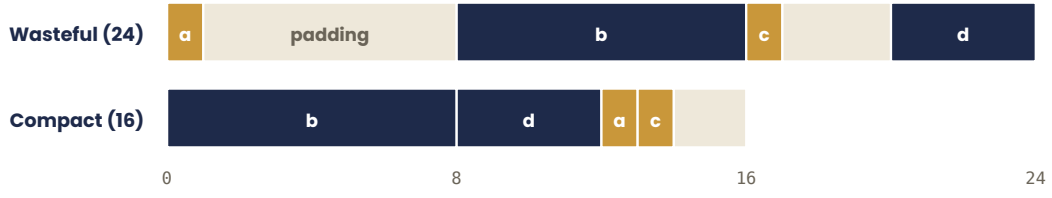
THE SAME MEMBERS IN TWO ORDERS

main.cpp

```
#include <cstddef>
#include <iostream>

struct Wasteful { char a; double b; char c; int d; };
struct Compact  { double b; int d; char a; char c; };

int main() {
    std::cout << "Wasteful: " << sizeof(Wasteful) << " bytes (a@" << offsetof(Wasteful, a) << " b@" <<
offsetof(Wasteful, b)
              << " c@" << offsetof(Wasteful, c) << " d@" << offsetof(Wasteful, d) << ")\n";
    std::cout << "Compact : " << sizeof(Compact) << " bytes (b@" << offsetof(Compact, b) << " d@" <<
offsetof(Compact, d)
              << " a@" << offsetof(Compact, a) << " c@" << offsetof(Compact, c) << ")\n";
    std::cout << "alignof(double) = " << alignof(double) << ", alignof(Wasteful) = " <<
alignof(Wasteful) << '\n';
    return 0;
}
```

OUTPUT

```
Wasteful: 24 bytes (a@0 b@8 c@16 d@20)
Compact : 16 bytes (b@0 d@8 a@12 c@13)
alignof(double) = 8, alignof(Wasteful) = 8
```



BYTE MAPS: GREY CELLS ARE PADDING

12.2 Structured bindings (C++17)

12.2.1 auto [x, y] = point;

A **structured binding** declares several names at once, bound to the members of a struct, the elements of an array or std::array, or the parts of a std::pair or std::tuple. It makes code that works with records, maps and multiple return values much clearer. Add `&` to bind by reference.

UNPACKING WITH STRUCTURED BINDINGS

main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <map>
#include <string>
#include <tuple>

struct Point { int x, y; };
struct MinMax { int min, max; };
MinMax min_max(int a, int b, int c) { return {std::min({a, b, c}), std::max({a, b, c})}; }

int main() {
    Point p{3, 4};
    auto [x, y] = p;                            // copies of the members
    std::cout << "x = " << x << ", y = " << y << '\n';
    auto& [rx, ry] = p;                         // references to the members
    rx = 10;
    std::cout << "p.x is now " << p.x << '\n';
    auto [lo, hi] = min_max(7, 2, 9);
    std::cout << "min " << lo << ", max " << hi << '\n';
    int arr[3] = {1, 2, 3};
    auto [a, b, c] = arr;
    std::cout << a + b + c << '\n';
    std::map<std::string, int> stock = {{"pen", 40}, {"book", 12}};
    for (const auto& [item, qty] : stock) std::cout << item << ": " << qty << '\n';   // best use
    auto [name, age, city] = std::tuple{"Asha", 21, "Pune"};
    std::cout << name << ' ' << age << ' ' << city << '\n';
    return 0;
}
```

OUTPUT

```
x = 3, y = 4
p.x is now 10
min 2, max 9
6
book: 12
pen: 40
Asha 21 Pune
```

12.3 Enumerations

12.3.1 Unscoped enums

A plain (C-style) `enum` defines named integer constants. Its enumerators leak into the surrounding scope (so two enums cannot both have a `Red`) and convert silently to int, which allows meaningless comparisons and arithmetic.

UNSCOPED ENUMS

main.cpp

```
#include <iostream>

enum Color { Red, Green, Blue };            // Red = 0, Green = 1, Blue = 2
enum Http { Ok = 200, NotFound = 404 };      // custom values

int main() {
    Color c = Green;
    int n = c;                               // implicit conversion to int
    std::cout << "Green = " << n << ", NotFound = " << NotFound << '\n';
    if (c == 1) std::cout << "comparing a color with 1 compiles: easy to misuse\n";
    // enum Fruit { Apple, Red };   error: Red is already declared
    return 0;
}
```

OUTPUT

```
Green = 1, NotFound = 404
comparing a color with 1 compiles: easy to misuse
```

12.3.2 Scoped enums enum class (C++11)

`enum class` (or `enum struct`) keeps enumerators inside its scope (`Color::Red`) and does **not** convert implicitly to int. Use `static_cast` when you really need the number. Prefer scoped enums in new code.

SCOPED ENUMS

main.cpp

```
#include <iostream>

enum class Color { Red, Green, Blue };
enum class Fruit { Apple, Orange, Red };     // no clash with Color::Red

int main() {
    Color c = Color::Green;
    Fruit f = Fruit::Red;
    std::cout << "Color::Green = " << static_cast<int>(c) << ", Fruit::Red = " << static_cast<int>(f)
<< '\n';
    // int n = c;          error: no implicit conversion
    // if (c == 1) ...     error: cannot compare with int
    // if (c == f) ...     error: different enum types
    return 0;
}
```

OUTPUT

```
Color::Green = 1, Fruit::Red = 2
```

12.3.3 Underlying type

Every enum is stored as some integer type, its **underlying type** (int by default for enum class). You can choose it explicitly, for example `std::uint8_t` to save memory in large arrays or to match a file format. `std::underlying_type_t<E>` names it, and C++23 adds `std::to_underlying(e)`.

CHOOSING THE UNDERLYING TYPE

main.cpp

```cpp
#include <cstdint>
#include <iostream>
#include <type_traits>
#include <utility>

enum class Level : std::uint8_t { Low = 1, Medium = 5, High = 10 };
enum class Big : long long { Huge = 9'000'000'000 };

int main() {
    std::cout << "sizeof(Level) = " << sizeof(Level) << ", sizeof(Big) = " << sizeof(Big) << '\n';
    Level l = Level::High;
    auto raw = static_cast<std::underlying_type_t<Level>>(l);
    std::cout << "High = " << +raw << ", via to_underlying: " << +std::to_underlying(l) << '\n';
    return 0;
}
```

OUTPUT

```
sizeof(Level) = 1, sizeof(Big) = 8
High = 10, via to_underlying: 10
```

12.3.4 Enums in switch statements

CONVERTING AN ENUM TO TEXT WITH SWITCH

main.cpp

```cpp
#include <iostream>
#include <string_view>

enum class Status { Pending, Shipped, Delivered, Cancelled };

std::string_view to_string(Status s) {
    switch (s) {
    case Status::Pending:   return "pending";
    case Status::Shipped:   return "shipped";
    case Status::Delivered: return "delivered";
    case Status::Cancelled: return "cancelled";
    }
    return "unknown";
}
int main() {
    for (Status s : {Status::Pending, Status::Shipped, Status::Delivered, Status::Cancelled})
        std::cout << static_cast<int>(s) << " -> " << to_string(s) << '\n';
    return 0;
}
```

OUTPUT

```
0 -> pending
1 -> shipped
2 -> delivered
3 -> cancelled
```

TIP

With `-Wall`, GCC warns when a switch over an enum misses an enumerator (and has no default), so adding `Status::Returned` later immediately shows every switch that must be updated.

12.4 Unions

12.4.1 Unions and shared memory

A **union** looks like a struct, but all its members share the **same memory**; its size is that of the largest member. Only one member holds a value at a time: the one last written. Reading a different member is undefined behaviour in C++ (unlike C). Unions save memory in low-level code, but nothing records which member is active, and members with constructors (such as std::string) need manual care.

ALL MEMBERS SHARE ONE ADDRESS

main.cpp

```
#include <cstdint>
#include <iostream>

union Number {
    std::int32_t i;
    float f;
    unsigned char bytes[4];
};

int main() {
    Number n;
    n.i = 42;
    std::cout << "sizeof(Number) = " << sizeof(Number) << ", n.i = " << n.i << '\n';
    n.f = 3.5f;                              // now f is the active member
    std::cout << "n.f = " << n.f << '\n';
    std::cout << "&n.i == &n.f ? " << std::boolalpha << (static_cast<void*>(&n.i) ==
static_cast<void*>(&n.f)) << '\n';
    return 0;
}
```

OUTPUT

```
sizeof(Number) = 4, n.i = 42
n.f = 3.5
&n.i == &n.f ? true
```

A UNION OVERLAYS ITS MEMBERS

12.4.2 std::variant as the type-safe alternative (detailed in Module 20)

`std::variant<int, double, std::string>` (C++17, header `<variant>`) is a **tagged union**: it holds one of the listed types and **remembers which**. Accessing the wrong alternative throws `std::bad_variant_access` instead of silently reading garbage, it constructs and destroys strings correctly, and `std::visit` runs the right code for the current type.

STD::VARIANT KNOWS WHAT IT HOLDS

main.cpp

```cpp
#include <iostream>
#include <string>
#include <variant>
#include <vector>

using Value = std::variant<int, double, std::string>;

int main() {
    std::vector<Value> cells = {42, 3.14, std::string("hello")};
    for (const Value& v : cells) {
        std::cout << "index " << v.index() << ": ";
        std::visit([](const auto& x) { std::cout << x << '\n'; }, v);
    }
    Value v = 10;
    if (std::holds_alternative<int>(v)) std::cout << "holds int " << std::get<int>(v) << '\n';
    try {
        std::cout << std::get<std::string>(v) << '\n';     // wrong alternative
    } catch (const std::bad_variant_access&) {
        std::cout << "bad_variant_access: v does not hold a string\n";
    }
    return 0;
}
```

OUTPUT

```
index 0: 42
index 1: 3.14
index 2: hello
holds int 10
bad_variant_access: v does not hold a string
```

12.5 Namespaces

12.5.1 Why namespaces

Large programs combine code from many people and libraries. Without namespaces, two libraries that both define a function `log()` or a class `Matrix` could not be used together. A **namespace** is a named scope that qualifies its names: `graphics::Matrix` and `math::Matrix` can coexist. The entire standard library lives in `std`.

12.5.2 Defining and using namespaces

DEFINING NAMESPACES

main.cpp

```cpp
#include <iostream>

namespace geometry {
    constexpr double PI = 3.14159265358979;
    double circle_area(double r) { return PI * r * r; }
}
namespace finance {
    double area(double length, double width) { return length * width; }   // unrelated meaning
    double simple_interest(double p, double r, int years) { return p * r * years / 100; }
}
namespace geometry {                        // namespaces are open: add more later
    double square_area(double s) { return s * s; }
}

int main() {
    std::cout << geometry::circle_area(2) << ' ' << geometry::square_area(3) << '\n';
    std::cout << finance::area(10, 20) << ' ' << finance::simple_interest(10000, 8, 2) << '\n';
    return 0;
}
```

OUTPUT

```
12.5664 9
200 1600
```

12.5.3 using declarations vs using namespace directives

Form	Effect	Use
`geometry::circle_area(2)`	fully qualified name	always safe, clearest
`using geometry::circle_area;`	using-declaration: brings **one** name into scope	fine in .cpp files and inside functions
`using namespace geometry;`	using-directive: makes **all** names visible	only in small scopes of .cpp files; never in headers
`namespace geo = geometry;`	namespace alias: a short name	for long nested names

BRINGING NAMES INTO SCOPE

main.cpp

```
#include <iostream>

namespace lbf::reports::monthly { int count() { return 12; } }

int main() {
    using std::cout;                        // just one name
    namespace rep = lbf::reports::monthly; // alias
    cout << "reports: " << rep::count() << '\n';
    {
        using namespace lbf::reports;      // limited to this block
        cout << "monthly::count() = " << monthly::count() << '\n';
    }
    return 0;
}
```

OUTPUT

```
reports: 12
monthly::count() = 12
```

12.5.4 Nested namespaces namespace a::b (C++17)

Namespaces can nest. Since C++17, `namespace a::b::c { }` replaces the verbose `namespace a { namespace b { namespace c { } } }`. Companies typically use `company::product::module`.

12.5.5 Anonymous namespaces

A namespace without a name, `namespace { ... }`, makes its contents visible **only in the current .cpp file** (internal linkage). It is the C++ replacement for `static` on file-level functions and variables: helper functions with common names cannot clash with other files at link time.

HELPERS HIDDEN IN AN ANONYMOUS NAMESPACE

C++

```
// parser.cpp
namespace {                                 // private to this file
    bool is_separator(char c) { return c == ',' || c == ';'; }
    int calls = 0;
}
int count_fields(const std::string& line) {   // visible to other files
    ++calls;
    int n = 1;
    for (char c : line) n += is_separator(c);
    return n;
}
```

12.5.6 Inline namespaces (overview)

Members of an `inline namespace` are also visible as members of the enclosing namespace. Libraries use this for **versioning**: `namespace lib { inline namespace v2 { ... } namespace v1 { ... } }` makes `lib::parse` mean the v2 version while old code can still ask for `lib::v1::parse`. The standard library uses inline namespaces for its literal suffixes (`std::literals::string_literals`).

VERSIONING WITH AN INLINE NAMESPACE

main.cpp

```
#include <iostream>

namespace mylib {
    namespace v1 { int parse() { return 1; } }
    inline namespace v2 { int parse() { return 2; } }    // the default version
}
int main() {
    std::cout << "mylib::parse() = " << mylib::parse() << " (v2 by default), mylib::v1::parse() = " <<
mylib::v1::parse() << '\n';
    return 0;
}
```

OUTPUT

```
mylib::parse() = 2 (v2 by default), mylib::v1::parse() = 1
```

A struct groups data (and can have member functions); access members with . or ->, initialise with braces or C++20 designators, and remember padding. Structured bindings unpack structs, pairs, tuples and maps. Prefer enum class to plain enums, std::variant to unions, and organise code in namespaces, never writing using namespace in headers.

PRACTICE CHECKPOINT

Build a **Student Record program** using a **vector of structs** with an **enum class for grade**, **sorted by marks**, and **organised inside a namespace**.

SAMPLE SOLUTION

main.cpp

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <string>
#include <string_view>
#include <vector>

namespace lbf::records {

enum class Grade { A, B, C, D, F };

struct Student {
    int roll;
    std::string name;
    double marks;
    Grade grade() const {
        if (marks >= 90) return Grade::A;
        if (marks >= 75) return Grade::B;
        if (marks >= 60) return Grade::C;
        if (marks >= 40) return Grade::D;
        return Grade::F;
    }
};

std::string_view to_string(Grade g) {
    switch (g) {
    case Grade::A: return "A";
    case Grade::B: return "B";
    case Grade::C: return "C";
    case Grade::D: return "D";
```

```
    case Grade::F: return "F";
    }
    return "?";
}
void sort_by_marks(std::vector<Student>& list) {
    std::sort(list.begin(), list.end(), [](const Student& a, const Student& b) { return a.marks >
b.marks; });
}
void print(const std::vector<Student>& list) {
    std::cout << "Rank Roll  Name        Marks  Grade\n";
    int rank = 1;
    for (const auto& [roll, name, marks] : list) {
        Student s{roll, name, marks};
        std::cout << std::setw(4) << rank++ << std::setw(5) << roll << "  " << std::left <<
std::setw(10) << name
                  << std::right << std::setw(7) << std::fixed << std::setprecision(1) << marks << "  "
<< to_string(s.grade()) << '\n';
    }
}
}  // namespace lbf::records

int main() {
    using namespace lbf::records;
    std::vector<Student> students;
    int n;
    std::cout << "Number of students: ";
    std::cin >> n;
    for (int i = 0; i < n; ++i) {
        Student s;
        std::cout << "roll name marks: ";
        std::cin >> s.roll >> s.name >> s.marks;
        students.push_back(s);
    }
    sort_by_marks(students);
    print(students);
    auto failed = std::count_if(students.begin(), students.end(), [](const Student& s) { return
s.grade() == Grade::F; });
    std::cout << "students with grade F: " << failed << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
Number of students: 5
roll name marks: 101 Asha 88.5
roll name marks: 102 Ravi 71
roll name marks: 103 Meera 93
roll name marks: 104 Kabir 38
roll name marks: 105 Zoya 64
Rank Roll  Name        Marks  Grade
   1  103  Meera        93.0  A
   2  101  Asha         88.5  B
   3  102  Ravi         71.0  C
   4  105  Zoya         64.0  C
   5  104  Kabir        38.0  F
students with grade F: 1
```

MODULE 13 · INTERMEDIATE

Object-oriented programming: classes and objects

Minimum duration **4 days** | Level **Intermediate**

A class bundles data with the functions that operate on it, and controls who may touch that data. This module builds classes correctly from the ground up: access control and invariants, every kind of constructor, destructors and RAII, copying (and the Rule of Three), the this pointer, static members, const correctness, friends, nested classes and the relationships between classes.

IN THIS MODULE

13.1 OOP concepts
13.2 Classes and objects
13.3 Access specifiers and encapsulation
13.4 Constructors
13.5 Destructors
13.6 Copying objects
13.7 The this pointer
13.8 Static members
13.9 Const correctness
13.10 Friends
13.11 Nested and local classes
13.12 Relationships between classes

13.1 OOP concepts

13.1.1 Procedural vs object-oriented programming

In **procedural** programming, data and the functions that work on it are separate: a `struct Account` and free functions `deposit(Account&, double)`. Any code anywhere can change the balance directly, so nothing guarantees it stays valid. In **object-oriented** programming, the data and its operations form one unit, an **object**, and the data is reachable only through the operations, which enforce the rules.

	Procedural	Object-oriented
Organised around	functions (actions)	objects (things with state and behaviour)
Data	passed between functions, often global	hidden inside objects
Who enforces the rules	every caller, by discipline	the class, in one place
Change the representation	update all code using the data	update the class; users unaffected
Good for	small programs, algorithms	large systems, GUIs, simulations, games

THE SAME IDEA, PROCEDURAL AND OBJECT-ORIENTED

main.cpp

```cpp
#include <iostream>

// procedural: nothing stops invalid states
struct RawAccount { double balance; };
void withdraw(RawAccount& a, double amt) { a.balance -= amt; }

// object-oriented: the class guards its data
class Account {
public:
    bool withdraw(double amt) {
        if (amt <= 0 || amt > balance_) return false;   // rule enforced in one place
        balance_ -= amt;
        return true;
    }
    void deposit(double amt) { if (amt > 0) balance_ += amt; }
    double balance() const { return balance_; }
private:
    double balance_ = 0;                                // unreachable from outside
};

int main() {
    RawAccount r{100};
    withdraw(r, 500);
    r.balance = -1e9;                                   // anyone can do this
    std::cout << "raw balance: " << r.balance << '\n';

    Account a;
    a.deposit(100);
    std::cout << std::boolalpha << "withdraw 500 allowed? " << a.withdraw(500) << ", balance " <<
a.balance() << '\n';
    // a.balance_ = -1e9;   error: private
    return 0;
}
```

OUTPUT

```
raw balance: -1e+09
withdraw 500 allowed? false, balance 100
```

13.1.2 The four pillars: encapsulation, abstraction, inheritance, polymorphism

Encapsulation

Bundle data with the methods that use it and **hide** the data (private), exposing a controlled interface.

Abstraction

Show **what** an object does, hide **how**: you drive a car with a wheel and pedals, not by controlling fuel injection.

Inheritance

Create a new class from an existing one, reusing and extending it: a `SavingsAccount` **is an** `Account` (Module 14).

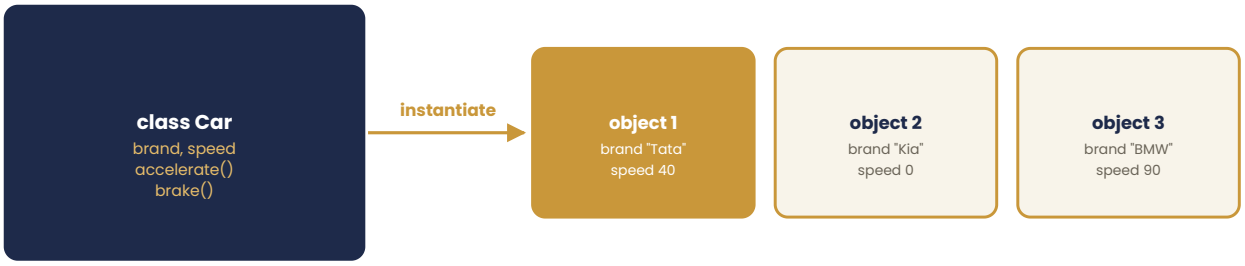
Polymorphism

"Many forms": one interface, different behaviour. `shape.area()` computes correctly for a circle or a rectangle (Module 14).

13.2 Classes and objects

13.2.1 Defining a class

A **class** is a blueprint: it describes what data each object holds and what it can do. An **object** (instance) is a concrete thing built from that blueprint, with its own copy of the data. One `Car` class, many car objects.



ONE CLASS, MANY OBJECTS, EACH WITH ITS OWN DATA

13.2.2 Data members and member functions

MEMBERS OF A CLASS

main.cpp

```cpp
#include <iostream>
#include <string>

class Car {
public:
    std::string brand;            // data members (attributes)
    int speed = 0;

    void accelerate(int by) {     // member functions (methods)
        speed += by;
        if (speed > 180) speed = 180;
    }
    void brake() { speed = 0; }
    void show() const { std::cout << brand << " is at " << speed << " km/h\n"; }
};

int main() {
    Car a, b;                     // two objects, independent data
    a.brand = "Tata";
    b.brand = "Kia";
    a.accelerate(40);
    b.accelerate(250);            // capped at 180
    a.show();
    b.show();
    b.brake();
    b.show();
    std::cout << "sizeof(Car) = " << sizeof(Car) << " (functions take no space in objects)\n";
    return 0;
}
```

OUTPUT

```
Tata is at 40 km/h
Kia is at 180 km/h
Kia is at 0 km/h
sizeof(Car) = 40 (functions take no space in objects)
```

13.2.3 Creating objects on the stack and the heap

OBJECT LIFETIMES

main.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>

class Car {
public:
    std::string brand;
    explicit Car(std::string b) : brand(std::move(b)) { std::cout << "  built " << brand << '\n'; }
    ~Car() { std::cout << "  scrapped " << brand << '\n'; }
};

int main() {
    Car local("Stack Car");                          // automatic: destroyed at the end of main
    Car* raw = new Car("Heap Car");                  // dynamic: lives until delete
    auto smart = std::make_unique<Car>("Smart Car");// dynamic but owned: deleted automatically
    std::cout << raw->brand << ", " << smart->brand << ", " << local.brand << '\n';
    delete raw;
    std::cout << "end of main\n";
    return 0;
}
```

OUTPUT

```
  built Stack Car
  built Heap Car
  built Smart Car
Heap Car, Smart Car, Stack Car
  scrapped Heap Car
end of main
  scrapped Smart Car
  scrapped Stack Car
```

13.2.4 Defining member functions inside and outside the class

Member functions can be defined inside the class body (implicitly inline, good for short ones) or declared in the class and defined outside it with the scope resolution operator: `ReturnType ClassName::function(params) { ... }`.

INSIDE AND OUTSIDE DEFINITIONS main.cpp

```
#include <iostream>

class Rectangle {
public:
    double width = 0, height = 0;
    double area() const { return width * height; }   // defined inside
    double perimeter() const;                         // only declared
    void scale(double factor);
};

double Rectangle::perimeter() const {                 // defined outside: ClassName::
    return 2 * (width + height);
}
void Rectangle::scale(double factor) {
    width *= factor;
    height *= factor;
}
int main() {
    Rectangle r;
    r.width = 3; r.height = 4;
    r.scale(2);
    std::cout << "area " << r.area() << ", perimeter " << r.perimeter() << '\n';
    return 0;
}
```

OUTPUT

```
area 48, perimeter 28
```

13.2.5 Separating a class into .h and .cpp files

Real projects put the **class definition** (the interface) in a header and the **member function definitions** in a source file. Users include the header; only the .cpp needs recompiling when the implementation changes.

RECTANGLE.H C++

```
#pragma once
#include <string>

class Rectangle {
public:
    Rectangle(double w, double h);
    double area() const;
    std::string describe() const;
private:
    double width_, height_;
};
```

RECTANGLE.CPP C++

```
#include "rectangle.h"

Rectangle::Rectangle(double w, double h) : width_(w), height_(h) {}

double Rectangle::area() const { return width_ * height_; }

std::string Rectangle::describe() const {
    return std::to_string(width_) + " x " + std::to_string(height_);
}
```

MAIN.CPP (BUILT WITH: G++ MAIN.CPP RECTANGLE.CPP) main.cpp

```
#include <iostream>
#include "rectangle.h"

int main() {
    Rectangle r(2.5, 4);
    std::cout << r.describe() << " has area " << r.area() << '\n';
    return 0;
}
```

OUTPUT

```
2.500000 x 4.000000 has area 10
```

13.3 Access specifiers and encapsulation

13.3.1 public, private, protected

Specifier	Accessible from	Typical content
`public`	anywhere	the interface: constructors and member functions users call
`private`	only member functions (and friends) of the same class	data members and internal helpers
`protected`	the class and classes derived from it (Module 14)	things meant for subclasses

A class's members are **private by default**. Each specifier applies until the next one. Access is checked per class, not per object: a member function may access private members of **another** object of the same class.

13.3.2 Getters and setters

Getters (accessors) return the value of a private member; **setters** (mutators) change it after validating the new value. Do not add a getter and setter for every member automatically: expose operations that make sense ("deposit", "withdraw") rather than raw data ("setBalance").

13.3.3 Data hiding and class invariants

An **invariant** is a condition that must always be true for a valid object: a balance is never negative, a date's day is 1 to 31, a vector's size ≤ capacity. Constructors establish the invariant and every public member function preserves it. Because the data is private, no outside code can break it, so the class can rely on it.

A CLASS THAT PROTECTS ITS INVARIANT main.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>

class Student {
public:
    Student(std::string name, int marks) : name_(std::move(name)) { set_marks(marks); }
    const std::string& name() const { return name_; }           // getter
    int marks() const { return marks_; }                         // getter
    void set_marks(int m) {                                      // setter with validation
        if (m < 0 || m > 100) throw std::invalid_argument("marks must be 0-100, got " +
std::to_string(m));
        marks_ = m;
    }
    char grade() const { return marks_ >= 90 ? 'A' : marks_ >= 75 ? 'B' : marks_ >= 40 ? 'C' : 'F'; }
private:
    std::string name_;
    int marks_ = 0;                                              // invariant: 0 <= marks_ <= 100
};

int main() {
    Student s("Asha", 82);
    std::cout << s.name() << ": " << s.marks() << " (" << s.grade() << ")\n";
    s.set_marks(95);
    std::cout << "updated: " << s.marks() << " (" << s.grade() << ")\n";
    try {
        s.set_marks(140);
    } catch (const std::invalid_argument& e) {
        std::cout << "rejected: " << e.what() << '\n';
    }
    std::cout << "still valid: " << s.marks() << '\n';
    return 0;
}
```

OUTPUT

```
Asha: 82 (B)
updated: 95 (A)
rejected: marks must be 0-100, got 140
still valid: 95
```

13.4 Constructors

A **constructor** is a special member function with the class's name and no return type, run automatically when an object is created. Its job is to put the object into a valid state.

13.4.1 Default constructor

13.4.2 Parameterized constructor

13.4.3 Member initializer lists

A **default constructor** takes no arguments; a **parameterised** one takes the values needed. Members should be initialised in the **member initializer list** (after the colon), not assigned in the body: assignment in the body first default-constructs each member, then overwrites it, and const members, references and members without default constructors **can only** be initialised in the list. Members are always initialised in the order they are **declared** in the class, regardless of the order in the list.

DEFAULT AND PARAMETERISED CONSTRUCTORS WITH INITIALIZER LISTS

main.cpp

```
#include <iostream>
#include <string>

class Time {
public:
    Time() : h_(0), m_(0) { std::cout << "  default constructor\n"; }              // default
    Time(int h, int m) : h_(h % 24), m_(m % 60) { std::cout << "  (h, m) constructor\n"; }   //
parameterized
    void show() const { std::cout << (h_ < 10 ? "0" : "") << h_ << ':' << (m_ < 10 ? "0" : "") << m_
<< '\n'; }
private:
    int h_, m_;
};

class Badge {
public:
    Badge(std::string owner, int id) : owner_(owner), id_(id), issued_by_(issuer) {}   // must use the
list
    void show() const { std::cout << "badge " << id_ << " for " << owner_ << " issued by " <<
issued_by_ << '\n'; }
private:
    const std::string owner_;      // const member: cannot be assigned later
    const int id_;
    const std::string& issued_by_; // reference member: must be bound at construction
    static inline const std::string issuer = "LBF Admin";
};

int main() {
    Time t1;          t1.show();
    Time t2(9, 5);    t2.show();
    Time t3{25, 70};  t3.show();     // normalised by the constructor
    Badge b("Ravi", 1042);
    b.show();
    return 0;
}
```

OUTPUT

```
  default constructor
00:00
  (h, m) constructor
09:05
  (h, m) constructor
01:10
badge 1042 for Ravi issued by LBF Admin
```

13.4.4 Default member initializers (C++11)

Members can be given default values directly in the class: `int speed = 0;`. Any constructor that does not mention that member in its initializer list uses the default. This removes repetition across several constructors.

13.4.5 Constructor overloading

13.4.6 Delegating constructors (C++11)

A class can have several constructors with different parameters (overloading). A **delegating constructor** calls another constructor of the same class in its initializer list, so the real initialisation logic lives in one place.

OVERLOADED AND DELEGATING CONSTRUCTORS

main.cpp

```
#include <iostream>
#include <string>

class Product {
public:
    Product(std::string name, double price, int stock)            // the "main" constructor
        : name_(std::move(name)), price_(price), stock_(stock) {
        if (price_ < 0) price_ = 0;
    }
    Product(std::string name, double price) : Product(std::move(name), price, 0) {}  // delegates
    explicit Product(std::string name) : Product(std::move(name), 0.0) {}           // delegates again
    Product() : Product("unnamed") {}
    void show() const { std::cout << name_ << ": Rs " << price_ << ", stock " << stock_ << ", category
" << category_ << '\n'; }
private:
    std::string name_;
    double price_;
    int stock_;
    std::string category_ = "general";                            // default member initializer
};

int main() {
    Product a("Notebook", 45, 120);
    Product b("Pen", 10);
    Product c("Sample");
    Product d;
    for (const auto& p : {a, b, c, d}) p.show();
    return 0;
}
```

OUTPUT

```
Notebook: Rs 45, stock 120, category general
Pen: Rs 10, stock 0, category general
Sample: Rs 0, stock 0, category general
unnamed: Rs 0, stock 0, category general
```

13.4.7 explicit constructors

A constructor callable with one argument also acts as an **implicit conversion**: with `Meters(double)`, the call `walk(5.0)` would silently create a Meters from a plain double. Mark single-argument constructors `explicit` to forbid such accidental conversions; the caller must then write `Meters(5.0)` or `Meters{5.0}`.

EXPLICIT PREVENTS SURPRISE CONVERSIONS

main.cpp

```cpp
#include <iostream>

class Meters {
public:
    explicit Meters(double v) : value(v) {}
    double value;
};
class Celsius {
public:
    Celsius(double v) : value(v) {}          // NOT explicit: converts silently
    double value;
};
void walk(Meters m) { std::cout << "walking " << m.value << " m\n"; }
void heat(Celsius c) { std::cout << "heating to " << c.value << " C\n"; }

int main() {
    walk(Meters{5.0});       // explicit construction required
    // walk(5.0);            error: no implicit conversion from double to Meters
    heat(5.0);               // compiles: 5.0 silently becomes Celsius (maybe a bug)
    return 0;
}
```

OUTPUT

```
walking 5 m
heating to 5 C
```

13.4.8 = default and = delete

`= default` asks the compiler to generate the standard version of a special member function (default constructor, copy, move, destructor), for example to keep a default constructor alongside other constructors. `= delete` forbids a function: any call is a compile-time error. It is used to make classes non-copyable, or to block unwanted overloads.

DEFAULTED AND DELETED FUNCTIONS

main.cpp

```cpp
#include <iostream>
#include <string>

class Config {
public:
    Config() = default;                                  // keep the default constructor
    explicit Config(std::string file) : file_(std::move(file)) {}
    const std::string& file() const { return file_; }
private:
    std::string file_ = "app.ini";
};

class Connection {
public:
    explicit Connection(std::string host) : host_(std::move(host)) {}
    Connection(const Connection&) = delete;              // non-copyable
    Connection& operator=(const Connection&) = delete;
    const std::string& host() const { return host_; }
private:
    std::string host_;
};

void set_level(int level) { std::cout << "level " << level << '\n'; }
void set_level(double) = delete;                         // forbid set_level(2.5)

int main() {
    Config a, b("custom.ini");
    std::cout << a.file() << ' ' << b.file() << '\n';
    Connection c("db.example.com");
    // Connection copy = c;   error: use of deleted function
    std::cout << c.host() << '\n';
    set_level(3);
    // set_level(2.5);        error: use of deleted function
    return 0;
}
```

OUTPUT

```
app.ini custom.ini
db.example.com
level 3
```

13.5 Destructors

13.5.1 When destructors run

A **destructor** is named `~ClassName()`, takes no parameters and returns nothing. It runs automatically when an object's lifetime ends: at the closing brace of the scope for local objects, at `delete` for heap objects, when the program ends for globals and statics, and when the owning object is destroyed for members and container elements.

13.5.2 Releasing resources in destructors (RAII)

Destructors are where RAII happens (Module 11.5): a class that acquires a resource in its constructor releases it in its destructor, so the resource can never be forgotten. Destructors should not throw exceptions (they are implicitly noexcept).

A DESTRUCTOR THAT FINISHES AND CLOSES A LOG FILE

main.cpp

```
#include <fstream>
#include <iostream>
#include <string>

class Logger {                                  // RAII: file opened in constructor, closed in
destructor
public:
    explicit Logger(const std::string& path) : out_(path) {
        std::cout << "  opened " << path << '\n';
        out_ << "log started\n";
    }
    ~Logger() {
        out_ << "log finished\n";
        std::cout << "  closing log (destructor)\n";   // ofstream closes itself after this
    }
    void write(const std::string& msg) { out_ << msg << '\n'; }
private:
    std::ofstream out_;
};

int main() {
    {
        Logger log("app.log");
        log.write("user logged in");
        std::cout << "  leaving the block...\n";
    }                                           // destructor runs here
    std::ifstream in("app.log");
    std::cout << "file contents:\n";
    for (std::string line; std::getline(in, line); ) std::cout << "  | " << line << '\n';
    return 0;
}
```

OUTPUT

```
  opened app.log
  leaving the block...
  closing log (destructor)
file contents:
  | log started
  | user logged in
  | log finished
```

13.5.3 Order of construction and destruction

Construction goes from the inside out: members are constructed first (in declaration order), then the class's own constructor body runs. Destruction is the exact reverse: the destructor body runs first, then members are destroyed in reverse declaration order. Local objects in a scope are destroyed in the reverse order of their creation (last in, first out).

CONSTRUCTION INSIDE-OUT, DESTRUCTION OUTSIDE-IN

main.cpp

```cpp
#include <iostream>
#include <string>

class Part {
public:
    explicit Part(std::string n) : name_(std::move(n)) { std::cout << "  construct " << name_ << '\n';
}
    ~Part() { std::cout << "  destroy   " << name_ << '\n'; }
private:
    std::string name_;
};
class Car {
public:
    Car() : engine_("engine"), wheels_("wheels") { std::cout << "  construct car (body)\n"; }
    ~Car() { std::cout << "  destroy   car (body)\n"; }
private:
    Part engine_;          // declared first: constructed first, destroyed last
    Part wheels_;
};
int main() {
    std::cout << "creating a and b:\n";
    Part a("a");
    Part b("b");
    std::cout << "creating car:\n";
    {
        Car c;
        std::cout << "car in use\n";
    }
    std::cout << "end of main:\n";
    return 0;
}
```

OUTPUT

```
creating a and b:
  construct a
  construct b
creating car:
  construct engine
  construct wheels
  construct car (body)
car in use
  destroy   car (body)
  destroy   wheels
  destroy   engine
end of main:
  destroy   b
  destroy   a
```

LIFETIME OF AN OBJECT WITH MEMBERS

13.6 Copying objects

13.6.1 Copy constructor

13.6.2 Copy assignment operator

The **copy constructor** `T(const T& other)` creates a new object as a copy of an existing one: `T b = a;`, `T b(a);`, passing by value, returning by value. The **copy assignment operator** `T& operator=(const T& other)` overwrites an **existing** object: `b = a;`. If you do not write them, the compiler generates versions that copy every member.

WHEN COPIES HAPPEN

main.cpp

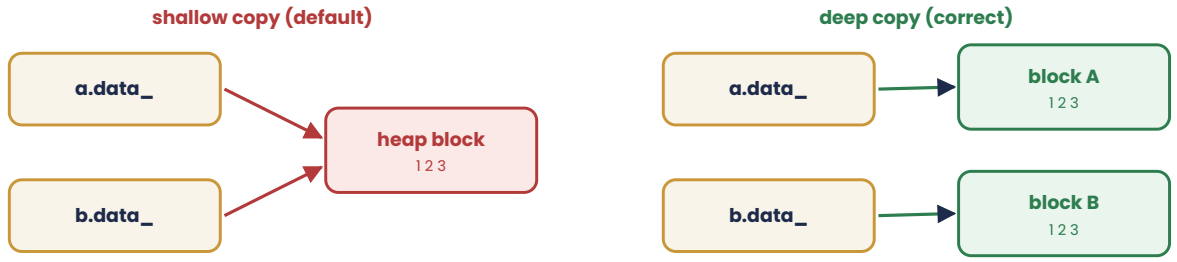
```
#include <iostream>
#include <string>

class Note {
public:
    explicit Note(std::string t) : text_(std::move(t)) { std::cout << "  construct \"" << text_ <<
"\"\n"; }
    Note(const Note& other) : text_(other.text_) { std::cout << "  copy-construct \"" << text_ <<
"\"\n"; }
    Note& operator=(const Note& other) {
        std::cout << "  copy-assign \"" << other.text_ << "\" over \"" << text_ << "\"\n";
        text_ = other.text_;
        return *this;
    }
private:
    std::string text_;
};
void by_value(Note n) { (void)n; }

int main() {
    Note a("first");
    Note b = a;              // copy constructor (initialisation, not assignment!)
    Note c("second");
    c = a;                   // copy assignment: c already exists
    by_value(a);             // pass by value: copy constructor
    return 0;
}
```

OUTPUT

```
  construct "first"
  copy-construct "first"
  construct "second"
  copy-assign "first" over "second"
  copy-construct "first"
```

13.6.3 Shallow copy vs deep copy

The compiler-generated copy copies each member. For a **pointer** member it copies the **address**, not the data pointed to: a **shallow copy**. Both objects then share one heap block; when the first is destroyed it deletes the block, and the second is left with a dangling pointer and later deletes the same block again (double delete). A class that owns a resource needs a **deep copy**: allocate a new block and copy the contents.



SHALLOW COPIES SHARE ONE BLOCK; DEEP COPIES OWN SEPARATE BLOCKS

A SHALLOW COPY LEADS TO A DOUBLE DELETE

main.cpp

```cpp
#include <iostream>

class Shallow {                             // relies on the compiler-generated copy: BUG
public:
    explicit Shallow(int n) : size_(n), data_(new int[n]{}) {}
    ~Shallow() { delete[] data_; }
    void set(int i, int v) { data_[i] = v; }
    int get(int i) const { return data_[i]; }
private:
    int size_;
    int* data_;
};
int main() {
    Shallow a(3);
    a.set(0, 10);
    {
        Shallow b = a;                      // copies the POINTER only
        b.set(0, 99);                       // also changes a!
        std::cout << "a.get(0) after changing b: " << a.get(0) << '\n';
    }                                       // b's destructor frees the shared block
    std::cout << "b is gone; a now holds a dangling pointer\n" << std::flush;
    return 0;                               // a's destructor frees it AGAIN
}
```

OUTPUT (crash at the end)

```
a.get(0) after changing b: 99
b is gone; a now holds a dangling pointer
free(): double free detected in tcache 2
Aborted (core dumped)
```

13.6.4 Rule of Three

Rule of Three: if a class needs a user-written **destructor**, **copy constructor** or **copy assignment operator**, it almost certainly needs all three, because it manages a resource. (C++11 extends this to the Rule of Five with move operations, Module 20; and the best option, the **Rule of Zero**, is to hold resources in members such as std::vector so that no special functions are needed at all.)

A CORRECT RULE-OF-THREE CLASS main.cpp

```
#include <algorithm>
#include <iostream>

class IntArray {
public:
    explicit IntArray(std::size_t n) : size_(n), data_(new int[n]{}) {}
    ~IntArray() { delete[] data_; }                                    // 1. destructor

    IntArray(const IntArray& other)                                    // 2. copy constructor: deep
copy
        : size_(other.size_), data_(new int[other.size_]) {
        std::copy(other.data_, other.data_ + size_, data_);
    }
    IntArray& operator=(const IntArray& other) {                       // 3. copy assignment
        if (this == &other) return *this;                              // self-assignment check
        int* fresh = new int[other.size_];                             // allocate first (exception
safety)
        std::copy(other.data_, other.data_ + other.size_, fresh);
        delete[] data_;
        data_ = fresh;
        size_ = other.size_;
        return *this;
    }
    int& operator[](std::size_t i) { return data_[i]; }
    std::size_t size() const { return size_; }
private:
    std::size_t size_;
    int* data_;
};

int main() {
    IntArray a(3);
    a[0] = 10;
    IntArray b = a;          // deep copy
    b[0] = 99;
    IntArray c(1);
    c = a;                   // deep copy assignment
    c[0] = 55;
    a = a;                   // self-assignment is safe
    std::cout << "a[0] = " << a[0] << ", b[0] = " << b[0] << ", c[0] = " << c[0] << ", c.size() = " <<
c.size() << '\n';
    return 0;
}
```

OUTPUT

```
a[0] = 10, b[0] = 99, c[0] = 55, c.size() = 3
```

13.7 The this pointer

13.7.1 Using this to resolve name conflicts

Inside a non-static member function, `this` is a pointer to the object the function was called on: in `a.show()`, this is `&a`. Members are implicitly accessed through it (`x` means `this->x`). Writing `this->` explicitly resolves conflicts when a parameter has the same name as a member.

13.7.2 Returning *this for method chaining

A member function that returns `*this` by reference lets calls be chained: `builder.set_a(1).set_b(2).build()`. The stream operators use the same idea.

THIS AND METHOD CHAINING

main.cpp

```
#include <iostream>
#include <string>

class Pizza {
public:
    Pizza& size(const std::string& size) { this->size_ = size; return *this; }   // this-> resolves the clash
    Pizza& add(const std::string& topping) { toppings_ += (toppings_.empty() ? "" : ", ") + topping; return *this; }
    Pizza& extra_cheese() { cheese_ = true; return *this; }
    void order() const {
        std::cout << size_ << " pizza with " << toppings_ << (cheese_ ? " + extra cheese" : "") << '\n';
    }
    const Pizza* address() const { return this; }
private:
    std::string size_ = "medium", toppings_;
    bool cheese_ = false;
};

int main() {
    Pizza p;
    p.size("large").add("paneer").add("onion").add("capsicum").extra_cheese().order();   // chaining
    std::cout << std::boolalpha << "this == &p ? " << (p.address() == &p) << '\n';
    return 0;
}
```

OUTPUT

```
large pizza with paneer, onion, capsicum + extra cheese
this == &p ? true
```

13.8 Static members

13.8.1 Static data members

A **static data member** belongs to the **class**, not to any object: there is exactly one copy, shared by all objects, existing even if no objects exist. Use it for counters, shared configuration and constants. Before C++17 it had to be declared in the class and defined once in a .cpp file.

13.8.2 Static member functions

A **static member function** also belongs to the class and is called as `ClassName::function()`. It has no `this` pointer, so it can access only static members (or members of objects passed to it).

13.8.3 Inline static members (C++17)

C++17 `static inline` data members are defined directly in the class, so no separate .cpp definition is needed.

A SHARED COUNTER AND SHARED CONSTANTS main.cpp

```
#include <iostream>
#include <string>

class Account {
public:
    explicit Account(std::string owner) : owner_(std::move(owner)), number_(next_number_++) {
++open_count_; }
    ~Account() { --open_count_; }
    static int open_count() { return open_count_; }        // static function: no this
    static double interest_rate() { return rate_; }
    int number() const { return number_; }
private:
    std::string owner_;
    int number_;
    static inline int next_number_ = 1001;                  // C++17: defined right here
    static inline int open_count_ = 0;
    static constexpr double rate_ = 0.045;                  // constexpr statics are implicitly inline
};

int main() {
    std::cout << "open accounts: " << Account::open_count() << " (no object needed)\n";
    Account a("Asha"), b("Ravi");
    {
        Account c("Meera");
        std::cout << "numbers " << a.number() << ' ' << b.number() << ' ' << c.number()
                  << ", open: " << Account::open_count() << '\n';
    }
    std::cout << "after c closes: " << Account::open_count() << ", rate " << Account::interest_rate()
* 100 << "%\n";
    return 0;
}
```

OUTPUT

```
open accounts: 0 (no object needed)
numbers 1001 1002 1003, open: 3
after c closes: 2, rate 4.5%
```

13.9 Const correctness

13.9.1 Const objects

13.9.2 Const member functions

A member function marked `const` (after the parameter list) promises not to modify the object; inside it, `this` is a pointer to const. Only const member functions can be called on **const objects** and through **const references**, which is how objects are usually passed. So every member function that does not change the object must be const, or the class becomes unusable through const references.

CONST MEMBER FUNCTIONS

main.cpp

```
#include <iostream>
#include <string>

class Circle {
public:
    explicit Circle(double r) : r_(r) {}
    double area() const { return 3.14159 * r_ * r_; }   // const: read-only
    void resize(double r) { r_ = r; }                    // not const: modifies
    double radius() const { return r_; }
private:
    double r_;
};
void report(const Circle& c) {                           // const reference parameter
    std::cout << "radius " << c.radius() << ", area " << c.area() << '\n';
    // c.resize(5);   error: resize is not const
}
int main() {
    Circle c(2);
    report(c);
    const Circle unit(1);                                 // a const object
    std::cout << "unit area " << unit.area() << '\n';
    // unit.resize(3);  error: passing const Circle as this discards qualifiers
    return 0;
}
```

OUTPUT

```
radius 2, area 12.5664
unit area 3.14159
```

FORGETTING CONST BREAKS CONST REFERENCES

main.cpp

```
class Circle {
public:
    explicit Circle(double r) : r_(r) {}
    double area() { return 3.14159 * r_ * r_; }   // forgot const
private:
    double r_;
};
double total(const Circle& c) { return c.area(); }
int main() { return total(Circle(1)) > 0 ? 0 : 1; }
```

OUTPUT

```
main.cpp: In function ‘double total(const Circle&)’:
main.cpp:8:46: error: passing ‘const Circle’ as ‘this’ argument discards qualifiers [-fpermissive]
    8 | double total(const Circle& c) { return c.area(); }
      |                                       ~~~~~~^~
main.cpp:4:12: note:   in call to ‘double Circle::area()’
    4 |     double area() { return 3.14159 * r_ * r_; }   // forgot const
      |            ^~~~
```

13.9.3 mutable members

A member declared `mutable` may be changed even inside const member functions. It is meant for internal details that do not affect the object's observable value: caches, access counters, mutexes.

A CACHE INSIDE A CONST OBJECT

main.cpp

```
#include <cmath>
#include <iostream>

class Polygon {
public:
    explicit Polygon(int sides, double len) : sides_(sides), len_(len) {}
    double area() const {                       // const, yet it fills a cache
        ++calls_;
        if (!cached_) {
            area_ = sides_ * len_ * len_ / (4 * std::tan(3.14159265358979 / sides_));
            cached_ = true;
        }
        return area_;
    }
    int calls() const { return calls_; }
private:
    int sides_;
    double len_;
    mutable bool cached_ = false;               // mutable: not part of the logical value
    mutable double area_ = 0;
    mutable int calls_ = 0;
};
int main() {
    const Polygon hex(6, 2.0);
    std::cout << hex.area() << ' ' << hex.area() << " (computed once, asked " << hex.calls() << "
times)\n";
    return 0;
}
```

OUTPUT

```
10.3923 10.3923 (computed once, asked 2 times)
```

13.10 Friends

13.10.1 Friend functions

13.10.2 Friend classes

A class can grant a non-member function, or another class, access to its private members by declaring it a `friend`. Friendship is granted, not taken; it is not inherited and not transitive. The classic use is `operator<<` for printing (Module 15), and tightly coupled helper classes such as an iterator for a container. Use friends sparingly: they widen access to private data.

FRIEND FUNCTION AND FRIEND CLASS

main.cpp

```
#include <iostream>
#include <string>

class Wallet;

class Auditor {                                  // a friend class
public:
    static void inspect(const Wallet& w);
};

class Wallet {
public:
    explicit Wallet(double cash) : cash_(cash) {}
    friend void print(const Wallet& w);          // a friend function
    friend class Auditor;
private:
    double cash_;
    std::string pin_ = "4321";
};

void print(const Wallet& w) { std::cout << "wallet holds Rs " << w.cash_ << '\n'; }
void Auditor::inspect(const Wallet& w) {
    std::cout << "auditor sees cash " << w.cash_ << " and pin length " << w.pin_.size() << '\n';
}
int main() {
    Wallet w(2500);
    print(w);
    Auditor::inspect(w);
    return 0;
}
```

OUTPUT

```
wallet holds Rs 2500
auditor sees cash 2500 and pin length 4
```

13.11 Nested and local classes

A **nested class** is declared inside another class. It is a member of the enclosing class's scope (`Outer::Inner`), follows its access rules (a private nested class is usable only inside), and can access the enclosing class's private members. Containers use nested classes for their iterators and nodes. A **local class** is declared inside a function and is visible only there.

A PRIVATE NESTED NODE AND A LOCAL CLASS

main.cpp

```cpp
#include <iostream>
#include <string>

class LinkedList {
public:
    void push_front(int v) { head_ = new Node{v, head_}; ++size_; }
    void print() const {
        for (Node* n = head_; n; n = n->next) std::cout << n->value << " -> ";
        std::cout << "null (" << size_ << " nodes)\n";
    }
    ~LinkedList() { while (head_) { Node* n = head_->next; delete head_; head_ = n; } }
private:
    struct Node {                       // nested, private: an implementation detail
        int value;
        Node* next;
    };
    Node* head_ = nullptr;
    int size_ = 0;
};

int main() {
    LinkedList list;
    for (int v : {30, 20, 10}) list.push_front(v);
    list.print();

    struct Pair { std::string key; int value; };   // local class: only inside main
    Pair p{"answer", 42};
    std::cout << p.key << " = " << p.value << '\n';
    return 0;
}
```

OUTPUT

```
10 -> 20 -> 30 -> null (3 nodes)
answer = 42
```

13.12 Relationships between classes

Objects collaborate. The relationships are distinguished by **ownership and lifetime**:

13.12.1 Association

Association ("uses" / "knows about"): two independent objects are linked, often for a while; neither owns the other. A Teacher teaches Students; a Doctor treats a Patient. Typically implemented with references or (non-owning) pointers.

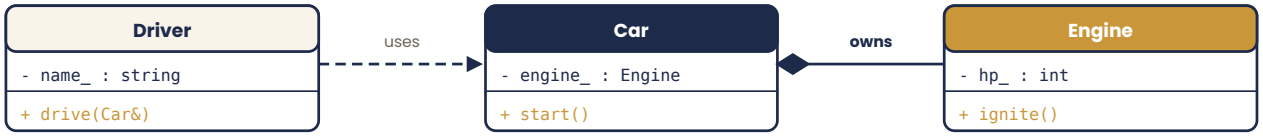
13.12.2 Aggregation

Aggregation ("has a", weak whole/part): the whole holds the parts but does not control their lifetime; parts can exist alone or belong to several wholes. A Department has Professors; if the department closes, the professors still exist.

13.12.3 Composition

Composition ("owns a" / "is made of", strong whole/part): the whole owns its parts, which are created and destroyed with it. A House has Rooms; a Car has an Engine. Implemented with member objects (or unique_ptr members).

Relationship	Phrase	Lifetime	Typical C++	UML
Association	uses / knows	independent	reference or pointer parameter / member	plain line
Aggregation	has a (shared)	parts outlive the whole	pointer / reference / shared_ptr member	hollow diamond
Composition	owns / is made of	parts die with the whole	member object, vector of objects, unique_ptr	filled diamond



ASSOCIATION (A DRIVER USES A CAR) AND COMPOSITION (A CAR OWNS ITS ENGINE: FILLED DIAMOND)

COMPOSITION, AGGREGATION AND ASSOCIATION IN CODE

main.cpp

```
#include <iostream>
#include <string>
#include <vector>

class Engine {                                          // part (composition)
public:
    explicit Engine(int hp) : hp_(hp) { std::cout << "  engine built\n"; }
    ~Engine() { std::cout << "  engine destroyed with its car\n"; }
    int hp() const { return hp_; }
private:
    int hp_;
};
class Car {
public:
    Car(std::string model, int hp) : model_(std::move(model)), engine_(hp) {}
    const std::string& model() const { return model_; }
    int power() const { return engine_.hp(); }
private:
    std::string model_;
    Engine engine_;                                     // composition: owned member
};
class Professor { public: explicit Professor(std::string n) : name(std::move(n)) {} std::string name;
};
class Department {                                      // aggregation: refers to professors it
does not own
public:
    void add(Professor& p) { staff_.push_back(&p); }
    void list() const { for (auto* p : staff_) std::cout << "  " << p->name << '\n'; }
private:
    std::vector<Professor*> staff_;
};
class Driver {                                          // association: uses a car temporarily
public:
    explicit Driver(std::string n) : name_(std::move(n)) {}
    void drive(const Car& c) const { std::cout << name_ << " drives the " << c.model() << " (" <<
c.power() << " hp)\n"; }
private:
    std::string name_;
};
```

```
int main() {
    Professor p1("Dr. Rao"), p2("Dr. Iyer");
    {
        Department cs;
        cs.add(p1);
        cs.add(p2);
        std::cout << "CS department staff:\n";
        cs.list();
    }
    std::cout << "department closed, professors still exist: " << p1.name << ", " << p2.name << '\n';
    {
        Car car("Nexon", 118);
        Driver d("Asha");
        d.drive(car);
    }
    std::cout << "done\n";
    return 0;
}
```

OUTPUT

```
CS department staff:
  Dr. Rao
  Dr. Iyer
department closed, professors still exist: Dr. Rao, Dr. Iyer
  engine built
Asha drives the Nexon (118 hp)
  engine destroyed with its car
done
```

A class bundles private data with public operations and protects its invariants. Initialise members in constructor initializer lists (or with default member initializers), use explicit for single-argument constructors, and release resources in destructors (RAII). A class that owns a resource needs a deep copy: the Rule of Three. Mark non-modifying members const, use static members for class-wide data, friends sparingly, and choose composition, aggregation or association according to ownership.

PRACTICE CHECKPOINT

Build a **BankAccount** class with a private balance, constructors, a **destructor that logs**, a **static account counter** and **const getters**, and a **deep-copying DynamicArray** class following the **Rule of Three**.

SAMPLE SOLUTION

main.cpp

```
#include <algorithm>
#include <iostream>
#include <stdexcept>
#include <string>

class BankAccount {
public:
    BankAccount(std::string owner, double opening)
        : owner_(std::move(owner)), number_(next_number_++) {
        if (opening < 0) throw std::invalid_argument("opening balance cannot be negative");
        balance_ = opening;
        ++count_;
        std::cout << "[open]  #" << number_ << " " << owner_ << " with " << balance_ << '\n';
    }
    explicit BankAccount(std::string owner) : BankAccount(std::move(owner), 0) {}
```

```
    ~BankAccount() { --count_; std::cout << "[close] #" << number_ << " " << owner_ << " (balance " <<
balance_ << ")\n"; }

    void deposit(double amt) { if (amt <= 0) throw std::invalid_argument("deposit must be positive");
balance_ += amt; }
    bool withdraw(double amt) { if (amt <= 0 || amt > balance_) return false; balance_ -= amt; return
true; }

    double balance() const { return balance_; }             // const getters
    const std::string& owner() const { return owner_; }
    int number() const { return number_; }
    static int count() { return count_; }
private:
    std::string owner_;
    int number_;
    double balance_ = 0;
    static inline int next_number_ = 5001;
    static inline int count_ = 0;
};

class DynamicArray {
public:
    explicit DynamicArray(std::size_t n = 0) : size_(n), data_(n ? new int[n]{} : nullptr) {}
    ~DynamicArray() { delete[] data_; }
    DynamicArray(const DynamicArray& o) : size_(o.size_), data_(o.size_ ? new int[o.size_] : nullptr)
{
        std::copy(o.data_, o.data_ + size_, data_);
    }
    DynamicArray& operator=(const DynamicArray& o) {
        if (this != &o) {
            int* fresh = o.size_ ? new int[o.size_] : nullptr;
            std::copy(o.data_, o.data_ + o.size_, fresh);
            delete[] data_;
            data_ = fresh;
            size_ = o.size_;
        }
        return *this;
    }
    int& operator[](std::size_t i) { if (i >= size_) throw std::out_of_range("index"); return
data_[i]; }
    std::size_t size() const { return size_; }
    void print(const char* label) const {
        std::cout << label << ':';
        for (std::size_t i = 0; i < size_; ++i) std::cout << ' ' << data_[i];
        std::cout << '\n';
    }
private:
    std::size_t size_;
    int* data_;
};

int main() {
    {
        BankAccount a("Asha", 10000), b("Ravi");
        a.deposit(2500);
        std::cout << std::boolalpha << "Ravi withdraws 100: " << b.withdraw(100) << '\n';
        a.withdraw(3000);
        std::cout << a.owner() << " has " << a.balance() << "; open accounts: " <<
BankAccount::count() << '\n';
    }
    std::cout << "open accounts after the block: " << BankAccount::count() << "\n\n";

    DynamicArray x(4);
    for (std::size_t i = 0; i < x.size(); ++i) x[i] = static_cast<int>((i + 1) * 10);
```

```
    DynamicArray y = x;          // copy constructor: deep copy
    DynamicArray z;
    z = x;                       // copy assignment: deep copy
    y[0] = -1;
    z[3] = 999;
    x.print("x");
    y.print("y");
    z.print("z");
    return 0;
}
```

OUTPUT

```
[open]  #5001 Asha with 10000
[open]  #5002 Ravi with 0
Ravi withdraws 100: false
Asha has 9500; open accounts: 2
[close] #5002 Ravi (balance 0)
[close] #5001 Asha (balance 9500)
open accounts after the block: 0

x: 10 20 30 40
y: -1 20 30 40
z: 10 20 30 999
```

MODULE 14 · INTERMEDIATE

Inheritance and polymorphism

Minimum duration **5 days** | Level **Intermediate**

Inheritance builds new classes from existing ones, reusing their code and modelling "is-a" relationships. Polymorphism lets one interface call different behaviour: at compile time through overloading and templates, and at run time through virtual functions. This module covers every form of inheritance, the diamond problem, virtual functions and how they work underneath, abstract classes and interfaces, and run-time type information.

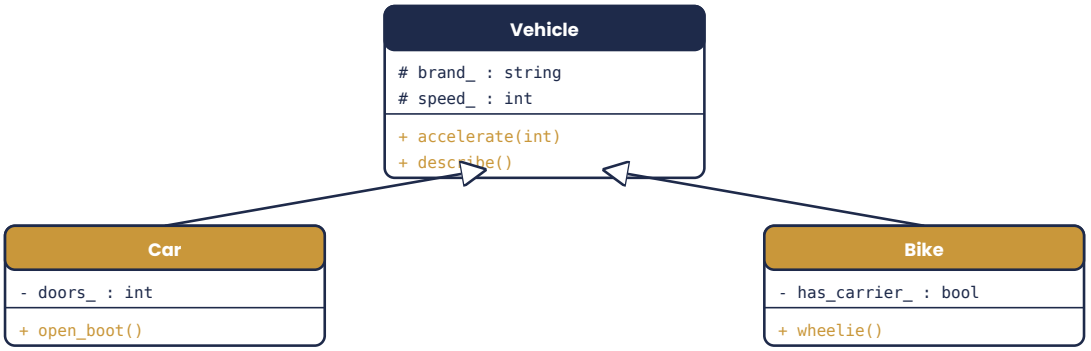
IN THIS MODULE

14.1 Inheritance basics
14.2 Types of inheritance
14.3 Constructors and destructors in inheritance
14.4 The diamond problem
14.5 Compile-time polymorphism
14.6 Runtime polymorphism
14.7 Abstract classes and interfaces
14.8 Runtime type information (RTTI)

14.1 Inheritance basics

14.1.1 Base and derived classes (IS-A relationship)

Inheritance creates a new class (the **derived** class, subclass or child) from an existing one (the **base** class, superclass or parent). The derived class automatically contains all the base class's members and adds or changes its own. Use it when the relationship is **IS-A**: a Car **is a** Vehicle, a SavingsAccount **is an** Account. If the relationship is HAS-A (a Car has an Engine), use composition instead.



CAR AND BIKE INHERIT FROM VEHICLE (HOLLOW TRIANGLES POINT TO THE BASE)

14.1.2 Syntax class Derived : public Base

A FIRST DERIVED CLASS

main.cpp

```cpp
#include <iostream>
#include <string>

class Vehicle {
public:
    explicit Vehicle(std::string brand) : brand_(std::move(brand)) {}
    void accelerate(int by) { speed_ += by; }
    void describe() const { std::cout << brand_ << " moving at " << speed_ << " km/h\n"; }
protected:                              // visible to derived classes
    std::string brand_;
    int speed_ = 0;
};

class Car : public Vehicle {            // Car IS-A Vehicle
public:
    Car(std::string brand, int doors) : Vehicle(std::move(brand)), doors_(doors) {}
    void open_boot() const { std::cout << brand_ << " (" << doors_ << " doors) opens its boot\n"; }
private:
    int doors_;
};

int main() {
    Car c("Tata Nexon", 5);
    c.accelerate(60);                   // inherited from Vehicle
    c.describe();                       // inherited
    c.open_boot();                      // Car's own
    Vehicle& v = c;                     // a Car can be used wherever a Vehicle is expected
    v.accelerate(20);
    v.describe();
    return 0;
}
```

OUTPUT

```
Tata Nexon moving at 60 km/h
Tata Nexon (5 doors) opens its boot
Tata Nexon moving at 80 km/h
```

14.1.3 Public, protected and private inheritance

The access keyword before the base class decides how inherited members are seen **from outside** the derived class. **Public inheritance** (by far the most common) models IS-A: public stays public. Protected and private inheritance hide the base's interface and model "implemented in terms of"; composition is usually a clearer way to express that.

Base member is	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

14.1.4 Access of base members in derived classes

A derived class can use the base's **public** and **protected** members, but **never** its private members (they exist inside the object but are inaccessible). Protected members are a middle ground: hidden from the

outside world, visible to subclasses. Many designers prefer keeping data private even from subclasses and giving them protected functions instead.

WHAT A DERIVED CLASS CAN SEE

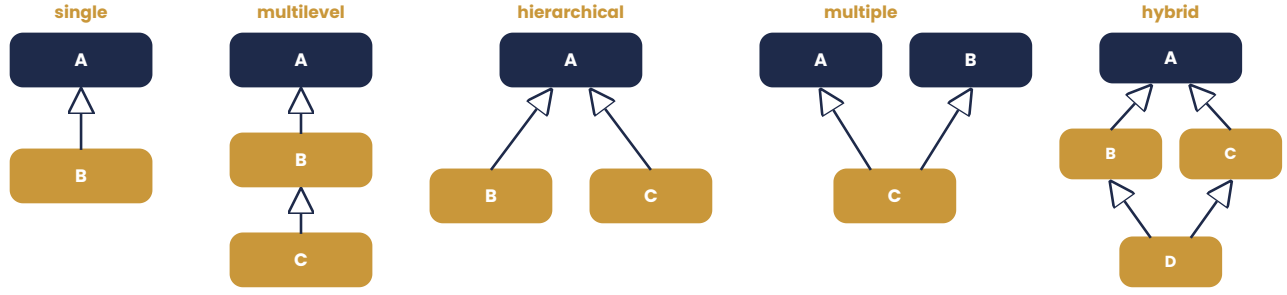
main.cpp

```
#include <iostream>

class Base {
public:
    int pub = 1;
protected:
    int prot = 2;
private:
    int priv = 3;
};
class PublicChild : public Base {
public:
    void show() const {
        std::cout << "pub " << pub << ", prot " << prot << '\n';   // OK
        // std::cout << priv;   error: private in Base
    }
};
class PrivateChild : private Base {
public:
    int get_pub() const { return pub; }      // usable inside ...
};
int main() {
    PublicChild a;
    a.show();
    std::cout << "a.pub from outside: " << a.pub << '\n';
    PrivateChild b;
    std::cout << "b.get_pub(): " << b.get_pub() << '\n';
    // std::cout << b.pub;   error: ... but hidden outside with private inheritance
    // std::cout << a.prot;  error: protected
    return 0;
}
```

OUTPUT

```
pub 1, prot 2
a.pub from outside: 1
b.get_pub(): 1
```

14.2 Types of inheritance



THE FIVE SHAPES OF INHERITANCE (ARROWS POINT TO THE BASE)

14.2.1 Single

14.2.2 Multilevel

14.2.3 Hierarchical

SINGLE, MULTILEVEL AND HIERARCHICAL INHERITANCE

main.cpp

```cpp
#include <iostream>
#include <string>

class Person {                                      // base of everything below
public:
    explicit Person(std::string n) : name_(std::move(n)) {}
    std::string name() const { return name_; }
private:
    std::string name_;
};
class Employee : public Person {                    // single: Person -> Employee
public:
    Employee(std::string n, double salary) : Person(std::move(n)), salary_(salary) {}
    double salary() const { return salary_; }
private:
    double salary_;
};
class Manager : public Employee {                   // multilevel: Person -> Employee -> Manager
public:
    Manager(std::string n, double salary, int team) : Employee(std::move(n), salary), team_(team) {}
    int team() const { return team_; }
private:
    int team_;
};
class Student : public Person {                     // hierarchical: Employee and Student share
Person
public:
    Student(std::string n, int roll) : Person(std::move(n)), roll_(roll) {}
    int roll() const { return roll_; }
private:
    int roll_;
};

int main() {
    Manager m("Farah", 180000, 8);
    Student s("Kabir", 42);
    std::cout << m.name() << " earns " << m.salary() << " and leads " << m.team() << " people\n";
    std::cout << s.name() << " has roll number " << s.roll() << '\n';
    return 0;
}
```

OUTPUT

```
Farah earns 180000 and leads 8 people
Kabir has roll number 42
```

14.2.4 Multiple

14.2.5 Hybrid

In **multiple inheritance** a class has several direct bases and inherits from all of them. It is powerful for combining independent capabilities (often interfaces), but if two bases have members with the same

name, the derived class must qualify which one it means. **Hybrid** inheritance combines several of the patterns above; when two paths lead back to the same base it forms the diamond of 14.4.

MULTIPLE INHERITANCE AND A NAME CLASH

main.cpp

```
#include <iostream>
#include <string>

class Camera {
public:
    void take_photo() const { std::cout << "  click: photo saved\n"; }
    std::string id() const { return "CAM-01"; }
};
class Phone {
public:
    void call(const std::string& who) const { std::cout << "  calling " << who << '\n'; }
    std::string id() const { return "IMEI-3579"; }
};
class SmartPhone : public Camera, public Phone {        // multiple inheritance
public:
    void selfie_and_share(const std::string& who) const { take_photo(); call(who); }
};
int main() {
    SmartPhone sp;
    sp.selfie_and_share("Mom");
    // sp.id();                 error: ambiguous (Camera::id or Phone::id?)
    std::cout << "camera id " << sp.Camera::id() << ", phone id " << sp.Phone::id() << '\n';
    return 0;
}
```

OUTPUT

```
  click: photo saved
  calling Mom
camera id CAM-01, phone id IMEI-3579
```

14.3 Constructors and destructors in inheritance

14.3.1 Order of constructor and destructor calls

A derived object contains a base subobject. Construction goes from the **base down**: the base constructor runs first, then the derived class's members, then the derived constructor body. Destruction runs in exactly the **reverse** order: derived destructor, derived members, base destructor.

14.3.2 Calling base class constructors

The derived constructor chooses which base constructor to run by naming it first in its member initializer list. If it does not, the base's default constructor is used, which is an error if the base has none.

BASE FIRST, DERIVED LAST; THEN REVERSED

main.cpp

```
#include <iostream>
#include <string>

class Base {
public:
    explicit Base(int v) : value_(v) { std::cout << "  Base(" << v << ")\n"; }
    ~Base() { std::cout << "  ~Base\n"; }
protected:
    int value_;
};
class Member {
public:
    Member() { std::cout << "  Member\n"; }
    ~Member() { std::cout << "  ~Member\n"; }
};
class Derived : public Base {
public:
    Derived(int v, std::string tag) : Base(v * 10), tag_(std::move(tag)) { std::cout << "  Derived
body (" << tag_ << ")\n"; }
    ~Derived() { std::cout << "  ~Derived\n"; }
private:
    Member m_;
    std::string tag_;
};
int main() {
    std::cout << "constructing:\n";
    {
        Derived d(4, "demo");
        std::cout << "using d\n";
    }
    std::cout << "done\n";
    return 0;
}
```

OUTPUT

```
constructing:
  Base(40)
  Member
  Derived body (demo)
using d
  ~Derived
  ~Member
  ~Base
done
```

CONSTRUCTION AND DESTRUCTION ORDER IN A HIERARCHY

14.3.3 Inheriting constructors with using (C++11)

Constructors are not inherited automatically. If a derived class adds no data needing initialisation, `using Base::Base;` makes all base constructors available for the derived class, avoiding boilerplate forwarding constructors.

INHERITING CONSTRUCTORS

main.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>

class AppError : public std::runtime_error {
public:
    using std::runtime_error::runtime_error;     // inherit runtime_error(const std::string&) and
(const char*)
};
class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
    void show() const { std::cout << '(' << x_ << ", " << y_ << ")\n"; }
private:
    int x_, y_;
};
class LabeledPoint : public Point {
public:
    using Point::Point;                          // LabeledPoint(int, int) now exists
    std::string label = "unnamed";
};
int main() {
    LabeledPoint p(3, 4);
    p.show();
    std::cout << p.label << '\n';
    try { throw AppError("config file missing"); }
    catch (const std::exception& e) { std::cout << "caught: " << e.what() << '\n'; }
    return 0;
}
```

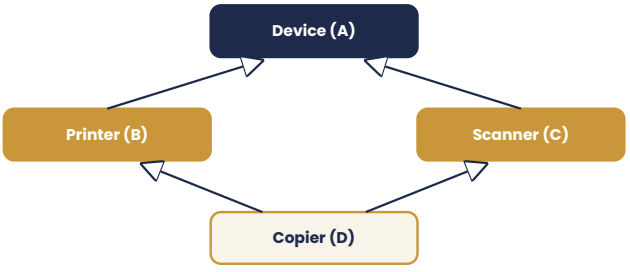
OUTPUT

```
(3, 4)
unnamed
caught: config file missing
```

14.4 The diamond problem

14.4.1 Ambiguity in multiple inheritance

If B and C both inherit from A, and D inherits from both B and C, then a D object contains **two** separate A subobjects. Every use of an A member through D is ambiguous (which copy?), and the data is duplicated.



THE DIAMOND: WITHOUT VIRTUAL INHERITANCE A COPIER CONTAINS TWO DEVICES

TWO COPIES OF THE BASE

main.cpp

```
#include <iostream>

class Device {
public:
    int power_watts = 0;
};
class Printer : public Device {};
class Scanner : public Device {};
class Copier : public Printer, public Scanner {};

int main() {
    Copier c;
    c.Printer::power_watts = 30;          // two separate copies of Device!
    c.Scanner::power_watts = 20;
    std::cout << "two Device subobjects: " << c.Printer::power_watts << " and " <<
c.Scanner::power_watts << '\n';
    std::cout << "sizeof(Copier) = " << sizeof(Copier) << " (two ints)\n";
    // c.power_watts = 50;   error: ambiguous
    return 0;
}
```

OUTPUT

```
two Device subobjects: 30 and 20
sizeof(Copier) = 8 (two ints)
```

14.4.2 Virtual inheritance

Declaring the intermediate bases with `virtual` inheritance (`class Printer : virtual public Device`) makes all paths share **one** Device subobject. The most-derived class (Copier) is then responsible for constructing the virtual base. Virtual inheritance has a small cost in size and speed; it is used mostly for diamonds of interfaces.

ONE SHARED BASE WITH VIRTUAL INHERITANCE main.cpp

```cpp
#include <iostream>

class Device {
public:
    explicit Device(int w = 0) : power_watts(w) { std::cout << "  Device(" << w << ") constructed
once\n"; }
    int power_watts;
};
class Printer : virtual public Device { public: Printer() : Device(30) {} };
class Scanner : virtual public Device { public: Scanner() : Device(20) {} };
class Copier : public Printer, public Scanner {
public:
    Copier() : Device(50) {}                    // the most-derived class builds the virtual base
};

int main() {
    Copier c;
    c.power_watts = 55;                         // no ambiguity: only one Device
    std::cout << "power " << c.power_watts << ", via Printer " << c.Printer::power_watts
              << ", via Scanner " << c.Scanner::power_watts << '\n';
    return 0;
}
```

OUTPUT

```
  Device(50) constructed once
power 55, via Printer 55, via Scanner 55
```

14.5 Compile-time polymorphism

Polymorphism means "many forms": the same name or interface performs different actions. In **compile-time** (static) polymorphism the compiler decides which code to call, so there is no run-time cost.

14.5.1 Function overloading

Several functions with the same name and different parameter lists: the compiler selects one from the argument types (Module 9.4.2). `print(int)`, `print(double)` and `print(const std::string&)` are one name with three forms.

14.5.2 Operator overloading (Module 15)

Operators are functions too: `a + b` calls `operator+` for the operand types, so `+` means integer addition, floating addition, string concatenation or matrix addition depending on the types.

14.5.3 Templates (Module 17)

A template is one definition that the compiler instantiates for each type it is used with; `std::max(3, 7)` and `std::max(2.5, 1.5)` run different generated code.

THREE KINDS OF COMPILE-TIME POLYMORPHISM

main.cpp

```cpp
#include <iostream>
#include <string>

void print(int x) { std::cout << "int: " << x << '\n'; }                    // overloading
void print(const std::string& s) { std::cout << "string: " << s << '\n'; }

struct Money { long paise; };
Money operator+(Money a, Money b) { return {a.paise + b.paise}; }              // operator
overloading

template <typename T>                                                          // template
T twice(T v) { return v + v; }

int main() {
    print(42);
    print("hello");
    Money total = Money{150} + Money{275};
    std::cout << "money: " << total.paise / 100 << '.' << total.paise % 100 << '\n';
    std::cout << twice(21) << ' ' << twice(1.25) << ' ' << twice(std::string("ab")) << '\n';
    return 0;
}
```

OUTPUT

```
int: 42
string: hello
money: 4.25
42 2.5 abab
```

14.6 Runtime polymorphism

14.6.1 Function overriding and name hiding

A derived class can provide its own version of a base member function with the same signature. Without `virtual` this merely **hides** the base version: which function runs depends on the **static type** of the pointer or reference, decided at compile time. Also, declaring **any** function named f in the derived class hides **all** base overloads of f (name hiding), unless brought back with `using Base::f;`.

HIDING IS NOT OVERRIDING main.cpp

```
#include <iostream>

class Animal {
public:
    void speak() const { std::cout << "...\n"; }           // NOT virtual
    void eat(int grams) const { std::cout << "eats " << grams << " g\n"; }
    void eat(const char* food) const { std::cout << "eats " << food << '\n'; }
};
class Dog : public Animal {
public:
    void speak() const { std::cout << "Woof!\n"; }         // hides Animal::speak
    void eat(double kg) const { std::cout << "eats " << kg << " kg\n"; }   // hides BOTH Animal::eat
overloads
};
int main() {
    Dog d;
    Animal& a = d;
    d.speak();                // Woof!  (static type Dog)
    a.speak();                // ...    (static type Animal: no virtual, no override)
    d.eat(1.5);
    // d.eat("bones");         error: Animal::eat(const char*) is hidden by Dog::eat
    d.Animal::eat("bones");    // explicit qualification still works
    return 0;
}
```

OUTPUT

```
Woof!
...
eats 1.5 kg
eats bones
```

14.6.2 Virtual functions

Declaring a member function `virtual` in the base makes calls through base pointers and references **dynamically dispatched**: the function of the object's actual (**dynamic**) type runs, decided at run time. This is runtime polymorphism, the heart of object-oriented design: code written against the base interface works with every present and future derived class.

14.6.3 override and final specifiers (C++11)

Write `override` on every overriding function: the compiler then checks that it really overrides a virtual base function, catching typos and signature mismatches that would otherwise silently create a new, unrelated function. `final` forbids further overriding of a function, or further derivation from a class.

14.6.4 Base class pointers and references to derived objects

VIRTUAL FUNCTIONS IN ACTION main.cpp

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Animal {
public:
    explicit Animal(std::string name) : name_(std::move(name)) {}
    virtual ~Animal() = default;                              // see 14.6.6
    virtual std::string sound() const { return "..."; }       // virtual
```

```
    void introduce() const { std::cout << name_ << " says " << sound() << '\n'; }   // calls the
virtual
protected:
    std::string name_;
};
class Dog : public Animal {
public:
    using Animal::Animal;
    std::string sound() const override { return "Woof"; }
};
class Cat : public Animal {
public:
    using Animal::Animal;
    std::string sound() const override { return "Meow"; }
};
class Kitten final : public Cat {                              // no class may derive from Kitten
public:
    using Cat::Cat;
    std::string sound() const override { return "mew"; }
};

int main() {
    std::vector<std::unique_ptr<Animal>> zoo;
    zoo.push_back(std::make_unique<Dog>("Bruno"));
    zoo.push_back(std::make_unique<Cat>("Misty"));
    zoo.push_back(std::make_unique<Kitten>("Tiny"));
    zoo.push_back(std::make_unique<Animal>("Generic"));
    for (const auto& a : zoo) a->introduce();                  // one loop, four behaviours
    Dog d("Rex");
    Animal& ref = d;                                           // reference to base, object is a Dog
    std::cout << "through a reference: " << ref.sound() << '\n';
    return 0;
}
```

OUTPUT

```
Bruno says Woof
Misty says Meow
Tiny says mew
Generic says ...
through a reference: Woof
```

OVERRIDE CATCHES A SIGNATURE MISTAKE main.cpp

```
class Shape {
public:
    virtual double area() const { return 0; }
    virtual ~Shape() = default;
};
class Square : public Shape {
public:
    double area() override { return side * side; }     // forgot const: does NOT override
    double side = 2;
};
```

OUTPUT

```
main.cpp:8:12: error: ‘double Square::area()’ marked ‘override’, but does not override
    8 |     double area() override { return side * side; }     // forgot const: does NOT override
      |            ^~~~
main.cpp:3:20: warning: ‘virtual double Shape::area() const’ was hidden [-Woverloaded-virtual=]
    3 |     virtual double area() const { return 0; }
      |                    ^~~~
main.cpp:8:12: note:   by ‘double Square::area()’
    8 |     double area() override { return side * side; }     // forgot const: does NOT override
      |            ^~~~
```

14.6.5 Object slicing

Assigning or passing a derived object **by value** as a base object copies only the base part: the derived members and the derived behaviour are "sliced off". Virtual dispatch works only through **pointers and references**. Pass polymorphic objects by reference or (smart) pointer, never by value.

SLICING DESTROYS POLYMORPHISM main.cpp

```
#include <iostream>
#include <string>

class Animal {
public:
    virtual ~Animal() = default;
    virtual std::string sound() const { return "..."; }
};
class Dog : public Animal {
public:
    std::string sound() const override { return "Woof"; }
    std::string toy = "ball";
};
void by_value(Animal a)      { std::cout << "by value     : " << a.sound() << '\n'; }
void by_ref(const Animal& a) { std::cout << "by reference : " << a.sound() << '\n'; }

int main() {
    Dog d;
    by_value(d);              // sliced: only the Animal part is copied
    by_ref(d);                // dynamic dispatch works
    Animal copy = d;          // slicing in assignment too
    std::cout << "sliced copy  : " << copy.sound() << '\n';
    return 0;
}
```

OUTPUT

```
by value     : ...
by reference : Woof
sliced copy  : ...
```

COPYING A DOG INTO AN ANIMAL KEEPS ONLY THE ANIMAL PART

14.6.6 Virtual destructors and why they matter

If a derived object is deleted through a **base pointer** and the base destructor is **not virtual**, only the base destructor runs: the derived part is never cleaned up (undefined behaviour, typically a leak). Rule: **any class with virtual functions, or meant to be a base class used polymorphically, must have a public virtual destructor** (or a protected non-virtual one).

MISSING VIRTUAL DESTRUCTOR main.cpp

```
#include <iostream>
#include <memory>

class BadBase {
public:
    ~BadBase() { std::cout << "  ~BadBase\n"; }              // NOT virtual
};
class BadDerived : public BadBase {
public:
    BadDerived() : buffer_(new int[1000]) {}
    ~BadDerived() { delete[] buffer_; std::cout << "  ~BadDerived (buffer freed)\n"; }
private:
    int* buffer_;
};
class GoodBase {
public:
    virtual ~GoodBase() { std::cout << "  ~GoodBase\n"; }    // virtual
};
class GoodDerived : public GoodBase {
public:
    ~GoodDerived() override { std::cout << "  ~GoodDerived (buffer freed)\n"; }
};
int main() {
    std::cout << "delete through a base pointer, non-virtual destructor:\n";
    BadBase* b = new BadDerived;
    delete b;                                                  // ~BadDerived never runs: leak
    std::cout << "delete through a base pointer, virtual destructor:\n";
    std::unique_ptr<GoodBase> g = std::make_unique<GoodDerived>();
    g.reset();
    return 0;
}
```

OUTPUT

```
delete through a base pointer, non-virtual destructor:
  ~BadBase
delete through a base pointer, virtual destructor:
  ~GoodDerived (buffer freed)
  ~GoodBase
```

14.6.7 How virtual functions work: vtable and vptr

Compilers implement virtual functions with a table. Each class that has virtual functions gets one **vtable**: an array of pointers to its versions of the virtual functions. Each **object** of such a class contains a hidden

pointer, the **vptr**, set by the constructor to its class's vtable. A virtual call `p->sound()` becomes: follow p to the object, read its vptr, look up the slot for sound, call that function. The cost is one extra pointer per object and an indirect call, typically a few nanoseconds.

EACH OBJECT'S VPTR POINTS TO ITS CLASS'S VTABLE OF FUNCTION ADDRESSES

THE COST OF A VPTR

main.cpp

```
#include <iostream>

class Plain { int x = 0; void f() {} };
class WithVirtual { int x = 0; virtual void f() {} };
class Base { public: virtual ~Base() = default; virtual void a() {} virtual void b() {} virtual void
c() {} };

int main() {
    std::cout << "sizeof(Plain)       = " << sizeof(Plain) << '\n';
    std::cout << "sizeof(WithVirtual) = " << sizeof(WithVirtual) << "  (int + hidden vptr +
padding)\n";
    std::cout << "sizeof(Base)        = " << sizeof(Base) << "  (one vptr, however many virtual
functions)\n";
    return 0;
}
```

OUTPUT

```
sizeof(Plain)       = 4
sizeof(WithVirtual) = 16  (int + hidden vptr + padding)
sizeof(Base)        = 8  (one vptr, however many virtual functions)
```

14.6.8 Calling virtual functions in constructors

During a base class constructor, the derived part of the object does not exist yet, so the vptr points to the **base** vtable: a virtual call inside a constructor (or destructor) runs the base version, not the override. This surprises programmers coming from Java or C#. Do not rely on virtual calls during construction; use a separate initialisation step or pass the needed data to the base constructor.

VIRTUAL CALLS DURING CONSTRUCTION USE THE CURRENT CLASS

main.cpp

```
#include <iostream>
#include <string>

class Base {
public:
    Base() { std::cout << "Base constructor calls describe(): " << describe() << '\n'; }
    virtual ~Base() = default;
    virtual std::string describe() const { return "Base"; }
};
class Derived : public Base {
public:
    Derived() { std::cout << "Derived constructor calls describe(): " << describe() << '\n'; }
    std::string describe() const override { return "Derived (" + extra_ + ")"; }
private:
    std::string extra_ = "ready";
};
int main() {
    Derived d;
    std::cout << "after construction: " << d.describe() << '\n';
    return 0;
}
```

OUTPUT

```
Base constructor calls describe(): Base
Derived constructor calls describe(): Derived (ready)
after construction: Derived (ready)
```

14.7 Abstract classes and interfaces

14.7.1 Pure virtual functions = 0

14.7.2 Abstract base classes

A **pure virtual function**, declared with `= 0`, has no implementation in the base (or only an optional default one) and **must** be overridden by concrete derived classes. A class with at least one pure virtual function is **abstract**: it cannot be instantiated, only used as a base and through pointers or references. Abstract classes define what every derived class must be able to do.

AN ABSTRACT SHAPE

main.cpp

```cpp
#include <iostream>
#include <memory>
#include <numbers>
#include <vector>

class Shape {                                       // abstract base class
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;                // pure virtual: no general formula
    virtual const char* name() const = 0;
    void report() const { std::cout << name() << " with area " << area() << '\n'; }
};
class Circle : public Shape {
public:
    explicit Circle(double r) : r_(r) {}
    double area() const override { return std::numbers::pi * r_ * r_; }
    const char* name() const override { return "Circle"; }
private:
    double r_;
};
class Square : public Shape {
public:
    explicit Square(double s) : s_(s) {}
    double area() const override { return s_ * s_; }
    const char* name() const override { return "Square"; }
private:
    double s_;
};
int main() {
    // Shape s;          error: cannot declare a variable of abstract type
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(1.5));
    shapes.push_back(std::make_unique<Square>(3));
    double total = 0;
    for (const auto& s : shapes) { s->report(); total += s->area(); }
    std::cout << "total area " << total << '\n';
    return 0;
}
```

OUTPUT

```
Circle with area 7.06858
Square with area 9
total area 16.0686
```

14.7.3 Interfaces in C++ (classes with only pure virtual functions)

C++ has no `interface` keyword. An **interface** is simply an abstract class containing only pure virtual functions (and a virtual destructor) and no data. Classes "implement" interfaces by inheriting from them publicly, and a class can implement several interfaces through multiple inheritance, which is safe because interfaces carry no state.

INTERFACES

main.cpp

```
#include <iostream>
#include <string>
#include <vector>

class Printable {                                   // interface
public:
    virtual ~Printable() = default;
    virtual std::string to_text() const = 0;
};
class Payable {                                     // interface
public:
    virtual ~Payable() = default;
    virtual double amount_due() const = 0;
};
class Invoice : public Printable, public Payable {  // implements two interfaces
public:
    Invoice(int no, double amt) : no_(no), amt_(amt) {}
    std::string to_text() const override { return "Invoice #" + std::to_string(no_); }
    double amount_due() const override { return amt_; }
private:
    int no_;
    double amt_;
};
class Salary : public Payable {
public:
    explicit Salary(double s) : s_(s) {}
    double amount_due() const override { return s_; }
private:
    double s_;
};
void print_all(const std::vector<const Printable*>& items) { for (auto* p : items) std::cout << p-
>to_text() << '\n'; }
double total_due(const std::vector<const Payable*>& items) { double t = 0; for (auto* p : items) t +=
p->amount_due(); return t; }

int main() {
    Invoice i1(101, 2500), i2(102, 4000);
    Salary s(30000);
    print_all({&i1, &i2});
    std::cout << "total payable: " << total_due({&i1, &i2, &s}) << '\n';
    return 0;
}
```

OUTPUT

```
Invoice #101
Invoice #102
total payable: 36500
```

14.8 Runtime type information (RTTI)

14.8.1 dynamic_cast

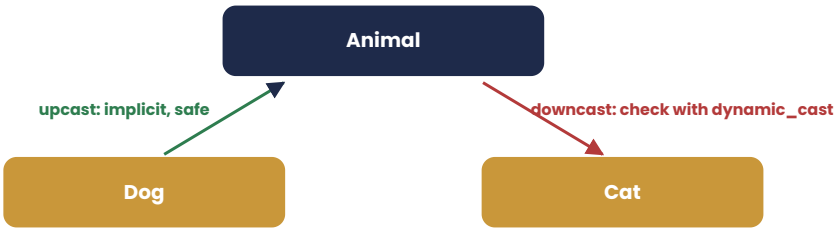
`dynamic_cast<Derived*>(base_ptr)` converts a base pointer to a derived pointer **only if** the object really is (or derives from) that type, checking at run time; otherwise it returns `nullptr`. For references it throws `std::bad_cast` on failure. It requires the base to be polymorphic (have at least one virtual function). Needing many dynamic_casts usually means a virtual function is missing from the base.

14.8.2 typeid

`typeid(expr)` (header `<typeinfo>`) returns a `std::type_info` describing the type. For a reference to a polymorphic object it gives the **dynamic** type. `name()` is implementation-specific (GCC prints mangled names such as `3Dog`); comparing `typeid(a) == typeid(b)` is portable.

14.8.3 Upcasting and downcasting

Upcasting (derived to base) is always safe and implicit: every Dog is an Animal. **Downcasting** (base to derived) is not always valid: an Animal may not be a Dog. Use `dynamic_cast` when you are not sure; `static_cast` is faster but unchecked, and wrong if you are mistaken.



UPCASTING IS ALWAYS SAFE; DOWNCASTING MUST BE CHECKED

DYNAMIC_CAST AND TYPEID

main.cpp

```
#include <iostream>
#include <memory>
#include <typeinfo>
#include <vector>

class Animal { public: virtual ~Animal() = default; virtual const char* kind() const { return
"animal"; } };
class Dog : public Animal {
public:
    const char* kind() const override { return "dog"; }
    void fetch() const { std::cout << "  the dog fetches the ball\n"; }
};
class Cat : public Animal { public: const char* kind() const override { return "cat"; } };

int main() {
    std::vector<std::unique_ptr<Animal>> pets;
    pets.push_back(std::make_unique<Dog>());
    pets.push_back(std::make_unique<Cat>());
    for (const auto& p : pets) {
        std::cout << p->kind() << ", typeid name " << typeid(*p).name() << '\n';
        if (auto* d = dynamic_cast<Dog*>(p.get()))      // downcast succeeds only for the Dog
            d->fetch();
        else
            std::cout << "  not a dog: dynamic_cast returned nullptr\n";
    }
    Cat c;
    Animal& ref = c;                                    // upcast: implicit
    std::cout << std::boolalpha << "typeid(ref) == typeid(Cat)? " << (typeid(ref) == typeid(Cat)) <<
'\n';
    try {
        Dog& wrong = dynamic_cast<Dog&>(ref);           // reference form throws on failure
        (void)wrong;
    } catch (const std::bad_cast& e) {
        std::cout << "bad_cast: " << e.what() << '\n';
    }
    return 0;
}
```

OUTPUT

```
dog, typeid name 3Dog
  the dog fetches the ball
cat, typeid name 3Cat
  not a dog: dynamic_cast returned nullptr
typeid(ref) == typeid(Cat)? true
bad_cast: std::bad_cast
```

Public inheritance models IS-A; constructors run base-first and destructors in reverse. Virtual inheritance solves the diamond. Polymorphism is static (overloading, templates) or dynamic (virtual functions through pointers and references, implemented with vtables). Always write override, give polymorphic bases a virtual destructor, never pass them by value (slicing), use pure virtual functions for abstract classes and interfaces, and dynamic_cast only when a downcast is unavoidable.

PRACTICE CHECKPOINT

Build a **Shape hierarchy** with an **abstract Shape** class, **Circle, Rectangle and Triangle** subclasses, a **vector<Shape*>** (later **unique_ptr**), and virtual **area()** and **destructor**.

SAMPLE SOLUTION

main.cpp

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <memory>
#include <numbers>
#include <string>
#include <vector>

class Shape {
public:
    explicit Shape(std::string name) : name_(std::move(name)) {}
    virtual ~Shape() { std::cout << "  ~Shape(" << name_ << ")\n"; }        // virtual destructor
    virtual double area() const = 0;
    virtual double perimeter() const = 0;
    const std::string& name() const { return name_; }
private:
    std::string name_;
};
class Circle : public Shape {
public:
    explicit Circle(double r) : Shape("circle"), r_(r) {}
    ~Circle() override { std::cout << "  ~Circle\n"; }
    double area() const override { return std::numbers::pi * r_ * r_; }
    double perimeter() const override { return 2 * std::numbers::pi * r_; }
private:
    double r_;
};
class Rectangle : public Shape {
public:
    Rectangle(double w, double h) : Shape("rectangle"), w_(w), h_(h) {}
    ~Rectangle() override { std::cout << "  ~Rectangle\n"; }
    double area() const override { return w_ * h_; }
    double perimeter() const override { return 2 * (w_ + h_); }
private:
    double w_, h_;
};
class Triangle : public Shape {
public:
    Triangle(double a, double b, double c) : Shape("triangle"), a_(a), b_(b), c_(c) {}
    ~Triangle() override { std::cout << "  ~Triangle\n"; }
    double area() const override { double s = perimeter() / 2; return std::sqrt(s * (s - a_) * (s -
b_) * (s - c_)); }
    double perimeter() const override { return a_ + b_ + c_; }
private:
    double a_, b_, c_;
};
void report(const std::vector<Shape*>& shapes) {
    for (const Shape* s : shapes)
        std::cout << std::left << std::setw(10) << s->name() << std::right << std::fixed <<
std::setprecision(2)
                  << " area " << std::setw(7) << s->area() << "  perimeter " << std::setw(6) << s-
>perimeter() << '\n';
}
int main() {
    std::cout << "Version 1: vector<Shape*> with manual delete\n";
    std::vector<Shape*> raw = {new Circle(2), new Rectangle(3, 4.5), new Triangle(3, 4, 5)};
    report(raw);
    for (Shape* s : raw) delete s;                       // virtual destructor: full cleanup
    raw.clear();

    std::cout << "Version 2: vector<unique_ptr<Shape>>, no delete needed\n";
    std::vector<std::unique_ptr<Shape>> shapes;
```

```
    shapes.push_back(std::make_unique<Circle>(1));
    shapes.push_back(std::make_unique<Rectangle>(2, 2));
    shapes.push_back(std::make_unique<Triangle>(5, 5, 6));
    double total = 0;
    const Shape* largest = shapes.front().get();
    for (const auto& s : shapes) {
        total += s->area();
        if (s->area() > largest->area()) largest = s.get();
    }
    std::cout << "total area " << total << ", largest is the " << largest->name() << '\n';
    std::cout << "end of main: unique_ptrs destroy the shapes\n";
    return 0;
}
```

OUTPUT

```
Version 1: vector<Shape*> with manual delete
circle     area   12.57  perimeter  12.57
rectangle  area   13.50  perimeter  15.00
triangle   area    6.00  perimeter  12.00
  ~Circle
  ~Shape(circle)
  ~Rectangle
  ~Shape(rectangle)
  ~Triangle
  ~Shape(triangle)
Version 2: vector<unique_ptr<Shape>>, no delete needed
total area 19.14, largest is the triangle
end of main: unique_ptrs destroy the shapes
  ~Circle
  ~Shape(circle)
  ~Rectangle
  ~Shape(rectangle)
  ~Triangle
  ~Shape(triangle)
```

MODULE 15 · INTERMEDIATE

Operator overloading

Minimum duration **2 days** | Level **Intermediate**

Operator overloading lets your own types use the natural syntax of built-in types: a + b for fractions and matrices, a == b for comparisons, std::cout << obj for printing, m[i][j] for element access, f(x) for function objects. Used well, it makes code read like the mathematics it implements.

IN THIS MODULE

15.1 Basics
15.2 Overloading common operators
15.3 Guidelines

15.1 Basics

15.1.1 What operator overloading is and when to use it

An operator applied to an object of class type is a call to a function named `operator@`, where @ is the symbol: `a + b` becomes `a.operator+(b)` or `operator+(a, b)`. Defining such functions gives your types the same notation as built-in ones. Overload operators when the meaning is **obvious and conventional**: arithmetic for numeric types (fractions, complex numbers, vectors, matrices, money), comparisons for ordered values, `<<` for printing, `[]` for containers. Do not invent surprising meanings.

AN OPERATOR IS A FUNCTION WITH SPECIAL SYNTAX

main.cpp

```
#include <iostream>

struct Vec2 {
    double x, y;
};
Vec2 add(Vec2 a, Vec2 b) { return {a.x + b.x, a.y + b.y}; }          // without overloading
Vec2 operator+(Vec2 a, Vec2 b) { return {a.x + b.x, a.y + b.y}; }    // with overloading
Vec2 operator*(double k, Vec2 v) { return {k * v.x, k * v.y}; }

int main() {
    Vec2 pos{1, 2}, vel{3, 4};
    Vec2 a = add(pos, add(vel, vel));          // hard to read
    Vec2 b = pos + 2 * vel;                    // reads like the formula
    Vec2 c = operator+(pos, vel);              // the operator is just a function
    std::cout << a.x << ',' << a.y << "  " << b.x << ',' << b.y << "  " << c.x << ',' << c.y << '\n';
    return 0;
}
```

OUTPUT

```
7,10  7,10  4,6
```

15.1.2 Operators that can and cannot be overloaded

Can be overloaded	Cannot be overloaded
`+ - * / % ^ & | ~ ! = < > += -= *= /= %=`	`::` scope resolution
`^= &= |= << >> >>= <<= == != <= >= <=>`	`.` member access
`&& || ++ -- , ->* -> () []`	`.*` pointer-to-member access
`new new[] delete delete[]`, conversion operators, `co_await`	`?:` conditional, `sizeof`, `alignof`, `typeid`

Rules: at least one operand must be a user-defined type (you cannot change `int + int`); you cannot create new operators, change precedence, associativity or the number of operands; and overloaded `&&`, `||` and `,` lose their short-circuit and ordering behaviour, so do not overload them.

15.1.3 Member vs non-member (friend) overloads

An operator can be a **member function** (the left operand is `*this`) or a **non-member** function (both operands are parameters, often a friend to access private data). Guidelines:

Operator	Usually	Why
`=`, `[]`, `()`, `->`, conversions	member (required)	the language demands it
`+=`, `-=`, `++`, `--`, unary `-`	member	they modify or belong to the object
`+ - * /` and comparisons	non-member (often friend or hidden friend)	symmetric: allows conversions on the left operand too (`2 * v`)
`<<` and `>>` for streams	non-member	the left operand is the stream, not your class

MEMBER VS NON-MEMBER SYMMETRY

main.cpp

```cpp
#include <iostream>

class Meters {
public:
    Meters(double v) : v_(v) {}                                  // implicit on purpose, for the demo
    Meters operator+(const Meters& o) const { return v_ + o.v_; }       // member
    friend Meters operator-(const Meters& a, const Meters& b) { return a.v_ - b.v_; }   // non-member
friend
    double value() const { return v_; }
private:
    double v_;
};
int main() {
    Meters a(5);
    std::cout << (a + 2).value() << '\n';      // a.operator+(2): 2 converts to Meters
    // std::cout << (2 + a).value();            error: member + needs a Meters on the LEFT
    std::cout << (10 - a).value() << '\n';     // non-member: conversion allowed on either side
    return 0;
}
```

OUTPUT

```
7
5
```

15.2 Overloading common operators

The examples in this section build up one class, `Fraction`, step by step. The complete class appears in the checkpoint.

15.2.1 Arithmetic operators + - * /

15.2.2 Compound assignment +=

The standard pattern: implement the compound assignment `+=` as a member that modifies `*this` and returns `*this` by reference, then implement `+` as a non-member in terms of `+=` (take the left operand by value, add, return). This avoids duplicating the arithmetic.

ARITHMETIC AND COMPOUND ASSIGNMENT main.cpp

```
#include <iostream>
#include <numeric>

class Fraction {
public:
    Fraction(long n = 0, long d = 1) : num_(n), den_(d) { normalise(); }
    Fraction& operator+=(const Fraction& o) { num_ = num_ * o.den_ + o.num_ * den_; den_ *= o.den_;
normalise(); return *this; }
    Fraction& operator-=(const Fraction& o) { return *this += Fraction(-o.num_, o.den_); }
    Fraction& operator*=(const Fraction& o) { num_ *= o.num_; den_ *= o.den_; normalise(); return
*this; }
    Fraction& operator/=(const Fraction& o) { num_ *= o.den_; den_ *= o.num_; normalise(); return
*this; }
    friend Fraction operator+(Fraction a, const Fraction& b) { return a += b; }   // built from +=
    friend Fraction operator-(Fraction a, const Fraction& b) { return a -= b; }
    friend Fraction operator*(Fraction a, const Fraction& b) { return a *= b; }
    friend Fraction operator/(Fraction a, const Fraction& b) { return a /= b; }
    void print() const { std::cout << num_ << '/' << den_; }
private:
    void normalise() {
        if (den_ < 0) { num_ = -num_; den_ = -den_; }
        long g = std::gcd(num_, den_);
        if (g) { num_ /= g; den_ /= g; }
    }
    long num_, den_;
};
int main() {
    Fraction a(1, 2), b(1, 3);
    (a + b).print(); std::cout << "  ";
    (a - b).print(); std::cout << "  ";
    (a * b).print(); std::cout << "  ";
    (a / b).print(); std::cout << "  ";
    (a + 1).print(); std::cout << "  ";           // 1 converts to Fraction(1, 1)
    (2 * b).print(); std::cout << '\n';           // works because operator* is a non-member
    Fraction sum;
    for (int i = 1; i <= 4; ++i) sum += Fraction(1, i);   // 1 + 1/2 + 1/3 + 1/4
    sum.print(); std::cout << '\n';
    return 0;
}
```

OUTPUT

```
5/6  1/6  1/6  3/2  3/2  2/3
25/12
```

15.2.3 Comparison operators == != < >

Before C++20 you wrote all six comparison operators by hand, usually defining `==` and `<` and expressing the others through them. Since C++20 the compiler rewrites `a != b` as `!(a == b)` automatically, so defining `==` is enough for both.

HAND-WRITTEN COMPARISONS main.cpp

```
#include <iostream>
#include <string>

class Version {
public:
    Version(int major, int minor) : major_(major), minor_(minor) {}
    friend bool operator==(const Version& a, const Version& b) { return a.major_ == b.major_ &&
a.minor_ == b.minor_; }
    friend bool operator<(const Version& a, const Version& b) {
        return a.major_ != b.major_ ? a.major_ < b.major_ : a.minor_ < b.minor_;
    }
    friend bool operator>(const Version& a, const Version& b)  { return b < a; }
    friend bool operator<=(const Version& a, const Version& b) { return !(b < a); }
    friend bool operator>=(const Version& a, const Version& b) { return !(a < b); }
    // operator!= not written: C++20 derives it from ==
private:
    int major_, minor_;
};
int main() {
    Version v1(2, 4), v2(2, 10), v3(2, 4);
    std::cout << std::boolalpha << (v1 == v3) << ' ' << (v1 != v2) << ' ' << (v1 < v2) << ' '
              << (v2 > v1) << ' ' << (v1 <= v3) << ' ' << (v2 >= v1) << '\n';
    return 0;
}
```

OUTPUT

```
true true true true true true
```

15.2.4 Three-way comparison <=> and defaulted comparisons (C++20)

With `auto operator<=>(const T&) const = default;` the compiler generates a three-way comparison that compares members in declaration order, and from it derives `<`, `<=`, `>`, `>=`; a defaulted `==` (implied by the defaulted `<=>`) gives `==` and `!=`. One line replaces six functions. For custom ordering, write `<=>` yourself and return `std::strong_ordering`, `weak_ordering` or `partial_ordering`.

THE SPACESHIP OPERATOR

main.cpp

```cpp
#include <algorithm>
#include <compare>
#include <iostream>
#include <string>
#include <vector>

struct Date {
    int year, month, day;                                  // compared in this order
    auto operator<=>(const Date&) const = default;         // all six operators
};
struct Student {
    std::string name;
    double marks;
    std::weak_ordering operator<=>(const Student& o) const { // custom: by marks, descending
        if (marks > o.marks) return std::weak_ordering::less;       // higher marks sort first
        if (marks < o.marks) return std::weak_ordering::greater;
        return std::weak_ordering::equivalent;
    }
    bool operator==(const Student& o) const { return marks == o.marks; }
};
int main() {
    Date a{2026, 3, 15}, b{2026, 10, 6};
    std::cout << std::boolalpha << "a < b: " << (a < b) << ", a == b: " << (a == b) << ", a >= b: " <<
(a >= b) << '\n';
    auto r = a <=> b;
    std::cout << "a <=> b is " << (r < 0 ? "less" : r > 0 ? "greater" : "equal") << '\n';
    std::vector<Student> cls = {{"Asha", 82}, {"Ravi", 91}, {"Meera", 77}};
    std::sort(cls.begin(), cls.end());                     // uses the custom ordering
    for (const auto& s : cls) std::cout << s.name << ' ' << s.marks << "  ";
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
a < b: true, a == b: false, a >= b: false
a <=> b is less
Ravi 91  Asha 82  Meera 77
```

15.2.5 Unary operators and prefix/postfix ++ / --

Unary operators (`-x`, `!x`, `~x`) take no parameter as members. Prefix and postfix increment are distinguished by a dummy `int` parameter: `T& operator++()` is prefix (increment, return the object), `T operator++(int)` is postfix (save a copy, increment, return the old copy). This is why prefix is never slower than postfix.

PREFIX AND POSTFIX INCREMENT

main.cpp

```
#include <iostream>

class Counter {
public:
    explicit Counter(int v = 0) : v_(v) {}
    Counter operator-() const { return Counter(-v_); }       // unary minus
    bool operator!() const { return v_ == 0; }               // logical not
    Counter& operator++() { ++v_; return *this; }            // prefix: ++c
    Counter operator++(int) { Counter old = *this; ++v_; return old; }   // postfix: c++ (dummy int)
    Counter& operator--() { --v_; return *this; }
    Counter operator--(int) { Counter old = *this; --v_; return old; }
    int value() const { return v_; }
private:
    int v_;
};
int main() {
    Counter c(5);
    Counter a = ++c;          // c = 6, a = 6
    Counter b = c++;          // b = 6, c = 7
    std::cout << "a " << a.value() << ", b " << b.value() << ", c " << c.value() << '\n';
    std::cout << "-c = " << (-c).value() << ", !c = " << std::boolalpha << !c << ", !Counter(0) = " <<
!Counter(0) << '\n';
    --c; c--;
    std::cout << "after two decrements c = " << c.value() << '\n';
    return 0;
}
```

OUTPUT

```
a 6, b 6, c 7
-c = -7, !c = false, !Counter(0) = true
after two decrements c = 5
```

15.2.6 Stream operators << and >>

To print with `std::cout << obj`, define a **non-member** `std::ostream& operator<<(std::ostream& os, const T& obj)` that writes to `os` and returns it, so calls can be chained. For input, `std::istream& operator>>(std::istream& is, T& obj)` reads into the object and sets the stream's fail state on bad input. Both are usually friends. They automatically work with files and string streams too.

PRINTING AND READING A POINT main.cpp

```
#include <iostream>
#include <sstream>

class Point {
public:
    Point(int x = 0, int y = 0) : x_(x), y_(y) {}
    friend std::ostream& operator<<(std::ostream& os, const Point& p) {
        return os << '(' << p.x_ << ", " << p.y_ << ')';
    }
    friend std::istream& operator>>(std::istream& is, Point& p) {       // expects "(x, y)" or "x y"
        char c;
        if (is >> c && c == '(') {
            int x, y; char comma, close;
            if (is >> x >> comma >> y >> close && comma == ',' && close == ')') { p = Point(x, y); }
            else is.setstate(std::ios::failbit);
        } else {
            is.putback(c);
            int x, y;
            if (is >> x >> y) p = Point(x, y);
        }
        return is;
    }
private:
    int x_, y_;
};
int main() {
    Point a(3, 4);
    std::cout << "a = " << a << ", origin = " << Point() << '\n';      // chaining works
    std::istringstream in("(7, -2) 10 20 (bad");
    Point p;
    while (in >> p) std::cout << "read " << p << '\n';
    std::cout << "stopped: stream failed on the malformed point\n";
    return 0;
}
```

OUTPUT

```
a = (3, 4), origin = (0, 0)
read (7, -2)
read (10, 20)
stopped: stream failed on the malformed point
```

15.2.7 Subscript []

`operator[]` must be a member. Provide two versions: a non-const one returning a reference (so `m[i] = x` works) and a const one returning a const reference or value (for const objects). C++23 allows multiple arguments: `m[i, j]`.

15.2.8 Function call () (functors)

An object of a class with `operator()` can be called like a function: a **function object** or **functor**. Unlike a plain function it can carry state (configuration, counters). Functors are what the standard algorithms accept as comparators and predicates; lambdas (Module 20) are a short way to write them.

OPERATOR() AND OPERATOR[]

main.cpp

```
#include <algorithm>
#include <iostream>
#include <stdexcept>
#include <vector>

class Polynomial {                                  // a functor: evaluate the polynomial at x
public:
    explicit Polynomial(std::vector<double> c) : coef_(std::move(c)) {}
    double operator()(double x) const {             // Horner's method
        double r = 0;
        for (auto it = coef_.rbegin(); it != coef_.rend(); ++it) r = r * x + *it;
        return r;
    }
    double& operator[](std::size_t i) { if (i >= coef_.size()) throw std::out_of_range("degree");
return coef_[i]; }
    const double& operator[](std::size_t i) const { return coef_.at(i); }
private:
    std::vector<double> coef_;                      // coef_[i] multiplies x^i
};
class InRange {                                     // a stateful predicate
public:
    InRange(int lo, int hi) : lo_(lo), hi_(hi) {}
    bool operator()(int v) { ++checks; return v >= lo_ && v <= hi_; }
    int checks = 0;
private:
    int lo_, hi_;
};
int main() {
    Polynomial p({1, -3, 2});                       // 1 - 3x + 2x^2
    std::cout << "p(0) = " << p(0) << ", p(2) = " << p(2) << ", p(0.5) = " << p(0.5) << '\n';
    p[0] = 5;                                       // subscript returns a reference
    const Polynomial& cp = p;
    std::cout << "after p[0] = 5: p(2) = " << p(2) << ", cp[2] = " << cp[2] << '\n';
    std::vector<int> marks = {35, 72, 91, 48, 60, 99};
    InRange pass(40, 100);
    auto n = std::count_if(marks.begin(), marks.end(), std::ref(pass));
    std::cout << n << " of " << marks.size() << " passed (" << pass.checks << " checks made)\n";
    return 0;
}
```

OUTPUT

```
p(0) = 1, p(2) = 3, p(0.5) = 0
after p[0] = 5: p(2) = 7, cp[2] = 2
5 of 6 passed (6 checks made)
```

15.2.9 Assignment =

The copy assignment operator must be a member returning `*this` by reference (Module 13.6). It must handle self-assignment and should give the **strong guarantee**: if it fails, the object is unchanged. The elegant way is the **copy-and-swap idiom**: take the parameter **by value** (the copy is made by the copy constructor), then swap its contents with *this; the old contents are destroyed with the parameter.

COPY-AND-SWAP ASSIGNMENT

main.cpp

```
#include <algorithm>
#include <iostream>
#include <utility>

class Buffer {
public:
    explicit Buffer(std::size_t n = 0) : n_(n), p_(n ? new int[n]() : nullptr) {}
    Buffer(const Buffer& o) : n_(o.n_), p_(o.n_ ? new int[o.n_] : nullptr) { std::copy(o.p_, o.p_ +
n_, p_); }
    ~Buffer() { delete[] p_; }
    Buffer& operator=(Buffer other) {          // by value: the copy is already made
        swap(*this, other);                    // take its contents; give it ours
        return *this;                          // other (with our old data) is destroyed here
    }
    friend void swap(Buffer& a, Buffer& b) noexcept { std::swap(a.n_, b.n_); std::swap(a.p_, b.p_); }
    int& operator[](std::size_t i) { return p_[i]; }
    std::size_t size() const { return n_; }
private:
    std::size_t n_;
    int* p_;
};
int main() {
    Buffer a(3), b(5);
    a[0] = 7;
    b = a;                                     // copy-and-swap
    b[0] = 9;
    a = a;                                     // self-assignment is safe automatically
    std::cout << "a[0] = " << a[0] << ", b[0] = " << b[0] << ", b.size() = " << b.size() << '\n';
    return 0;
}
```

OUTPUT

```
a[0] = 7, b[0] = 9, b.size() = 3
```

15.2.10 Conversion operators and explicit

`operator T() const` converts an object to type T. An **implicit** conversion operator can fire in unexpected places (for example a class convertible to bool or int taking part in arithmetic), so mark conversions `explicit`: they then work only with an explicit cast, or in boolean contexts such as `if (obj)` for `explicit operator bool()`.

EXPLICIT AND IMPLICIT CONVERSION OPERATORS main.cpp

```cpp
#include <iostream>
#include <string>

class Percentage {
public:
    explicit Percentage(double v) : v_(v) {}
    explicit operator double() const { return v_ / 100; }        // explicit conversion to double
    explicit operator bool() const { return v_ > 0; }            // usable in if/while/!/&&
    operator std::string() const { return std::to_string(static_cast<int>(v_)) + "%"; }   // implicit
(for the demo)
private:
    double v_;
};
int main() {
    Percentage gst(18);
    double fraction = static_cast<double>(gst);                  // must be explicit
    std::cout << "fraction " << fraction << '\n';
    if (gst) std::cout << "gst applies\n";                        // explicit bool works in conditions
    std::string label = gst;                                      // implicit conversion
    std::cout << "label " << label << '\n';
    // double d = gst;        error: conversion is explicit
    // int n = gst + 1;       error: no implicit conversion to bool/double
    return 0;
}
```

OUTPUT

```
fraction 0.18
gst applies
label 18%
```

15.3 Guidelines

15.3.1 Keep the natural meaning of operators

- An operator should do what a reader expects from the built-in version: `+` adds and does not modify its operands; `==` is symmetric and consistent with `!=`; `<` is a strict ordering usable by std::sort.
- If you define `+`, also define `+=` (and implement one through the other); if you define `==`, the meaning of `!=` must match.
- Do not overload `&&`, `||`, the comma operator or unary `&`.
- If the meaning is not obvious, use a named function: `v.dot(w)` is clearer than an invented `v ^ w`.

15.3.2 Return types and const correctness

Operator	Recommended signature	Returns
`a + b`	`T operator+(T a, const T& b)` (non-member)	a new value
`a += b`	`T& operator+=(const T& b)` (member)	`*this` by reference
`a == b`	`bool operator==(const T&) const`	bool
`a <=> b`	`auto operator<=>(const T&) const = default`	an ordering
`++a`	`T& operator++()`	`*this`

Operator	Recommended signature	Returns
`a++`	`T operator++(int)`	old value by value
`a[i]`	`T& operator[](size_t)` and a const version	reference
`os << a`	`std::ostream& operator<<(std::ostream&, const T&)`	the stream
`a = b`	`T& operator=(const T&)` or by value (copy-and-swap)	`*this`

Operators that do not modify the object must be `const` members (or take `const T&`), so they work on const objects and temporaries.

Overload operators only where their meaning is natural. Implement compound assignment as members returning *this and binary arithmetic as non-members built on them; default <=> (C++20) for comparisons; write << and >> as non-member friends returning the stream; provide const and non-const operator[]; use operator() for function objects; make conversion operators explicit; and keep every operator const-correct.

PRACTICE CHECKPOINT

Write a **Fraction** (or Complex) class with **arithmetic, comparison and stream operators**, and a **Matrix** class with **[]** and ***.

SAMPLE SOLUTION

main.cpp

```
#include <compare>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <sstream>
#include <stdexcept>
#include <vector>

class Fraction {
public:
    Fraction(long n = 0, long d = 1) : n_(n), d_(d) {
        if (d_ == 0) throw std::invalid_argument("zero denominator");
        reduce();
    }
    Fraction& operator+=(const Fraction& o) { n_ = n_ * o.d_ + o.n_ * d_; d_ *= o.d_; reduce(); return
*this; }
    Fraction& operator-=(const Fraction& o) { n_ = n_ * o.d_ - o.n_ * d_; d_ *= o.d_; reduce(); return
*this; }
    Fraction& operator*=(const Fraction& o) { n_ *= o.n_; d_ *= o.d_; reduce(); return *this; }
    Fraction& operator/=(const Fraction& o) { if (o.n_ == 0) throw std::domain_error("divide by
zero"); n_ *= o.d_; d_ *= o.n_; reduce(); return *this; }
    friend Fraction operator+(Fraction a, const Fraction& b) { return a += b; }
    friend Fraction operator-(Fraction a, const Fraction& b) { return a -= b; }
    friend Fraction operator*(Fraction a, const Fraction& b) { return a *= b; }
    friend Fraction operator/(Fraction a, const Fraction& b) { return a /= b; }
    Fraction operator-() const { return Fraction(-n_, d_); }
    friend bool operator==(const Fraction& a, const Fraction& b) { return a.n_ == b.n_ && a.d_ ==
b.d_; }
    friend std::strong_ordering operator<=>(const Fraction& a, const Fraction& b) { return a.n_ * b.d_
<=> b.n_ * a.d_; }
    friend std::ostream& operator<<(std::ostream& os, const Fraction& f) {
        return f.d_ == 1 ? os << f.n_ : os << f.n_ << '/' << f.d_;
    }
```

```
    friend std::istream& operator>>(std::istream& is, Fraction& f) {      // reads "a/b" or "a"
        long n, d = 1;
        if (is >> n) { if (!is.eof() && is.peek() == '/') { is.get(); is >> d; } if (!is.fail()) f =
Fraction(n, d); }
        return is;
    }
    explicit operator double() const { return static_cast<double>(n_) / d_; }
private:
    void reduce() { if (d_ < 0) { n_ = -n_; d_ = -d_; } long g = std::gcd(n_, d_); n_ /= g; d_ /= g; }
    long n_, d_;
};

class Matrix {
public:
    Matrix(std::size_t r, std::size_t c, double v = 0) : r_(r), c_(c), a_(r * c, v) {}
    Matrix(std::initializer_list<std::initializer_list<double>> rows) : r_(rows.size()),
c_(rows.begin()->size()) {
        for (const auto& row : rows) a_.insert(a_.end(), row.begin(), row.end());
    }
    double* operator[](std::size_t i) { return &a_[i * c_]; }               // m[i][j]
    const double* operator[](std::size_t i) const { return &a_[i * c_]; }
    friend Matrix operator*(const Matrix& x, const Matrix& y) {
        if (x.c_ != y.r_) throw std::invalid_argument("dimension mismatch");
        Matrix m(x.r_, y.c_);
        for (std::size_t i = 0; i < x.r_; ++i)
            for (std::size_t j = 0; j < y.c_; ++j)
                for (std::size_t k = 0; k < x.c_; ++k) m[i][j] += x[i][k] * y[k][j];
        return m;
    }
    friend std::ostream& operator<<(std::ostream& os, const Matrix& m) {
        for (std::size_t i = 0; i < m.r_; ++i) {
            for (std::size_t j = 0; j < m.c_; ++j) os << std::setw(6) << m[i][j];
            os << '\n';
        }
        return os;
    }
private:
    std::size_t r_, c_;
    std::vector<double> a_;
};

int main() {
    Fraction a(3, 4), b(5, 6);
    std::cout << a << " + " << b << " = " << a + b << '\n';
    std::cout << a << " - " << b << " = " << a - b << '\n';
    std::cout << a << " * " << b << " = " << a * b << '\n';
    std::cout << a << " / " << b << " = " << a / b << '\n';
    std::cout << std::boolalpha << a << " < " << b << " ? " << (a < b) << ", 2/4 == 1/2 ? " <<
(Fraction(2, 4) == Fraction(1, 2)) << '\n';
    std::istringstream in("7/3 -2/8 5");
    Fraction f, total;
    while (in >> f) total += f;
    std::cout << "sum of 7/3, -2/8 and 5 = " << total << " = " << static_cast<double>(total) << '\n';

    Matrix m1{{1, 2, 3}, {4, 5, 6}};
    Matrix m2{{7, 8}, {9, 10}, {11, 12}};
    m1[0][0] = 1;
    std::cout << "m1 * m2 =\n" << m1 * m2;
    std::cout << "m2 * m1 =\n" << m2 * m1;
    return 0;
}
```

OUTPUT

```
3/4 + 5/6 = 19/12
3/4 - 5/6 = -1/12
3/4 * 5/6 = 5/8
3/4 / 5/6 = 9/10
3/4 < 5/6 ? true, 2/4 == 1/2 ? true
sum of 7/3, -2/8 and 5 = 85/12 = 7.08333
m1 * m2 =
    58    64
   139   154
m2 * m1 =
    39    54    69
    49    68    87
    59    82   105
```

MODULE 16 · INTERMEDIATE

Exception handling

Minimum duration **2 days** | Level **Intermediate**

Exceptions separate error-handling code from the normal logic and let an error travel up the call stack, through any number of functions, to the place that can handle it, while RAII cleans up everything on the way. This module covers try, throw and catch, the standard exception hierarchy, custom exceptions, stack unwinding, noexcept and the exception-safety guarantees, and the modern alternatives std::optional and std::expected.

IN THIS MODULE

16.1 Basics
16.2 Standard exceptions
16.3 Custom exceptions
16.4 Advanced exception topics
16.5 Alternatives to exceptions

16.1 Basics

16.1.1 Errors vs exceptions

Some problems are **bugs** in the program (an out-of-range index we computed wrongly, a null pointer); the fix is to correct the code, and assertions help find them. Others are **exceptional conditions** that a correct program must still expect: a missing file, invalid user input, a full disk, a lost network connection, insufficient funds. Exceptions are the C++ mechanism for reporting the second kind when the function that detects the problem cannot deal with it itself.

Approach	How errors are reported	Drawbacks
Return codes (C style)	special return value or bool + errno	easy to ignore; every caller must check and pass errors up; mixes error code with logic
Exceptions	`throw` an object; the nearest matching `catch` handles it	cost when thrown; control flow less visible
`std::optional` / `std::expected`	the return value holds either a result or nothing / an error	caller must unwrap; no automatic propagation

16.1.2 try, throw, catch

`throw expression;` creates an exception object and abandons the current function. The runtime looks for an enclosing `try` block whose `catch` clause matches the object's type; on the way it leaves every function in between, destroying their local objects. If a matching handler is found, it runs and execution continues after the try/catch. If none is found anywhere, `std::terminate` ends the program.

AN EXCEPTION TRAVELS UP THE CALL STACK TO THE NEAREST MATCHING HANDLER

THROWING AND CATCHING

main.cpp

```cpp
#include <iostream>
#include <stdexcept>
#include <string>

int parse_age(const std::string& text) {
    for (char c : text)
        if (c < '0' || c > '9') throw std::invalid_argument("not a number: \"" + text + "\"");
    int age = std::stoi(text);
    if (age > 150) throw std::out_of_range("age too large: " + text);
    return age;
}
int years_to_retirement(const std::string& text) {
    int age = parse_age(text);                  // may throw: this function does not handle it
    return age >= 60 ? 0 : 60 - age;
}
int main() {
    for (std::string input : {"35", "abc", "200", "61"}) {
        try {
            int years = years_to_retirement(input);   // if this throws, the next line is skipped
            std::cout << input << " -> " << years << " years to retirement\n";
        } catch (const std::invalid_argument& e) {
            std::cout << input << " -> invalid input: " << e.what() << '\n';
        } catch (const std::out_of_range& e) {
            std::cout << input << " -> out of range: " << e.what() << '\n';
        }
    }
    std::cout << "program continues normally\n";
    return 0;
}
```

OUTPUT

```
35 -> 25 years to retirement
abc -> invalid input: not a number: "abc"
200 -> out of range: age too large: 200
61 -> 0 years to retirement
program continues normally
```

16.1.3 Multiple catch blocks and catch-all catch (...)

Several catch clauses may follow one try; they are tried **in order** and the **first** match wins, so put the most specific types first and more general base classes later. `catch (...)` catches anything, including non-standard objects; it is a last resort, typically used to log and rethrow, or at the outermost level of a program or thread.

ORDERED CATCH CLAUSES

main.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

void risky(int kind) {
    switch (kind) {
    case 0: throw std::out_of_range("index 9 >= size 3");
    case 1: throw std::runtime_error("network unreachable");
    case 2: throw std::string("a plain string was thrown");
    case 3: throw 42;
    }
}
int main() {
    for (int k = 0; k < 4; ++k) {
        try {
            risky(k);
        } catch (const std::out_of_range& e) {           // most specific first
            std::cout << "out_of_range: " << e.what() << '\n';
        } catch (const std::exception& e) {              // any standard exception
            std::cout << "std::exception: " << e.what() << '\n';
        } catch (const std::string& s) {
            std::cout << "string: " << s << '\n';
        } catch (...) {                                   // everything else
            std::cout << "unknown exception type\n";
        }
    }
    return 0;
}
```

OUTPUT

```
out_of_range: index 9 >= size 3
std::exception: network unreachable
string: a plain string was thrown
unknown exception type
```

16.1.4 Catching by reference

Always throw **by value** and catch **by (const) reference**: `catch (const std::exception& e)`. Catching by value copies the exception and **slices** it: a derived exception caught as a base object loses its derived part, and `what()` then returns the base message. Catching by pointer forces someone to delete the object.

CATCHING BY VALUE SLICES THE EXCEPTION main.cpp

```cpp
#include <iostream>
#include <stdexcept>

class DiskFull : public std::runtime_error {
public:
    DiskFull() : std::runtime_error("disk full") {}
    const char* what() const noexcept override { return "disk full: free some space on drive D:"; }
};
int main() {
    try { throw DiskFull(); }
    catch (std::runtime_error e) { std::cout << "by value    : " << e.what() << "   (sliced!)\n"; }
    try { throw DiskFull(); }
    catch (const std::runtime_error& e) { std::cout << "by reference: " << e.what() << '\n'; }
    return 0;
}
```

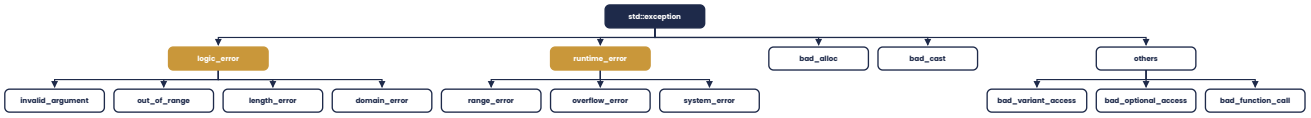
OUTPUT

```
by value    : disk full   (sliced!)
by reference: disk full: free some space on drive D:
```

16.2 Standard exceptions

16.2.1 std::exception hierarchy

The standard library throws objects of classes derived from `std::exception` (header `<exception>`; most derived classes in `<stdexcept>`). Catching `const std::exception&` therefore catches every standard exception.



THE STANDARD EXCEPTION HIERARCHY (MAIN MEMBERS)

16.2.2 logic_error, invalid_argument, out_of_range, runtime_error, bad_alloc

Exception	Meaning	Thrown by (examples)
`std::logic_error`	a mistake that could have been detected before running (a precondition was violated)	base for the next four
`std::invalid_argument`	an argument has an unacceptable value	`std::stoi("abc")`, `std::bitset("12")`
`std::out_of_range`	an index or value outside the allowed range	`vector::at`, `string::substr`, `stoi("99999999999")`
`std::length_error`	an object would exceed its maximum size	`vector::reserve(huge)`
`std::runtime_error`	an error only detectable while running	base for your own run-time errors; `std::system_error` for OS errors
`std::bad_alloc`	memory allocation failed	`new`, container growth

Exception	Meaning	Thrown by (examples)
`std::bad_cast`	a dynamic_cast to a reference failed	`dynamic_cast<T&>`
`std::bad_optional_access`	value() on an empty optional	`std::optional::value`

16.2.3 what()

`virtual const char* what() const noexcept` returns a description of the error. The standard library's messages are implementation-specific (GCC's often name the internal function); your own exceptions should give messages useful to the person reading the log.

STANDARD EXCEPTIONS AND THEIR MESSAGES main.cpp

```cpp
#include <bitset>
#include <iostream>
#include <map>
#include <optional>
#include <stdexcept>
#include <string>
#include <vector>

int main() {
    std::vector<int> v(3);
    std::map<std::string, int> m;
    std::optional<int> empty;
    auto attempt = [](const char* label, auto action) {
        try { action(); std::cout << label << ": no exception\n"; }
        catch (const std::exception& e) { std::cout << label << ": " << e.what() << '\n'; }
    };
    attempt("vector::at(10)  ", [&] { (void)v.at(10); });
    attempt("stoi(\"x9\")       ", [] { (void)std::stoi("x9"); });
    attempt("string::substr  ", [] { (void)std::string("abc").substr(5); });
    attempt("map::at(\"none\") ", [&] { (void)m.at("none"); });
    attempt("bitset(\"102\")   ", [] { std::bitset<4> b(std::string("102")); });
    attempt("optional::value ", [&] { (void)empty.value(); });
    attempt("vector::reserve ", [&] { v.reserve(v.max_size() + 1); });
    return 0;
}
```

OUTPUT

```
vector::at(10)  : vector::_M_range_check: __n (which is 10) >= this->size() (which is 3)
stoi("x9")       : stoi
string::substr  : basic_string::substr: __pos (which is 5) > this->size() (which is 3)
map::at("none") : map::at
bitset("102")   : bitset::_M_copy_from_ptr
optional::value : bad optional access
vector::reserve : vector::reserve
```

16.3 Custom exceptions

16.3.1 Deriving from std::exception

Define your own exception classes for your domain's errors, deriving from `std::runtime_error` or `std::logic_error` (which store the message for you) or directly from `std::exception`. A custom class lets callers catch exactly the errors they care about and can carry extra data (an account number, a file name, an error code). Build a small hierarchy when errors fall into groups.

A SMALL EXCEPTION HIERARCHY main.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>

class BankError : public std::runtime_error {                  // base of all banking errors
public:
    using std::runtime_error::runtime_error;
};
class InsufficientFunds : public BankError {
public:
    InsufficientFunds(double balance, double requested)
        : BankError("insufficient funds: balance " + std::to_string(static_cast<long>(balance)) + ",
requested " + std::to_string(static_cast<long>(requested))),
          shortfall_(requested - balance) {}
    double shortfall() const { return shortfall_; }            // extra data for the handler
private:
    double shortfall_;
};
class AccountLocked : public BankError {
public:
    explicit AccountLocked(int no) : BankError("account " + std::to_string(no) + " is locked") {}
};

void withdraw(int account, double balance, double amount) {
    if (account == 13) throw AccountLocked(account);
    if (amount > balance) throw InsufficientFunds(balance, amount);
    std::cout << "account " << account << ": withdrew " << amount << '\n';
}
int main() {
    struct { int acc; double bal, amt; } tests[] = {{7, 5000, 1200}, {8, 300, 1000}, {13, 9999, 10}};
    for (const auto& t : tests) {
        try {
            withdraw(t.acc, t.bal, t.amt);
        } catch (const InsufficientFunds& e) {
            std::cout << "declined (" << e.what() << "), short by " << e.shortfall() << '\n';
        } catch (const BankError& e) {                          // any other banking error
            std::cout << "bank error: " << e.what() << '\n';
        }
    }
    return 0;
}
```

OUTPUT

```
account 7: withdrew 1200
declined (insufficient funds: balance 300, requested 1000), short by 700
bank error: account 13 is locked
```

16.4 Advanced exception topics

16.4.1 Rethrowing exceptions

Inside a catch block, a bare `throw;` rethrows the **same** exception object (with its full dynamic type) to outer handlers. Use it when a layer wants to log, clean up or add context but cannot fully handle the error. `throw e;` would instead throw a **copy** of the static type, slicing it. To add context while keeping the original, C++11 offers nested exceptions (`std::throw_with_nested`).

LOG AND RETHROW

main.cpp

```cpp
#include <iostream>
#include <stdexcept>

void read_config() { throw std::runtime_error("config.ini: line 12: missing '='"); }

void start_server() {
    try {
        read_config();
    } catch (const std::exception& e) {
        std::cout << "[start_server] logging: " << e.what() << '\n';
        throw;                                  // pass the same exception upward
    }
}
int main() {
    try {
        start_server();
    } catch (const std::runtime_error& e) {
        std::cout << "[main] server failed to start: " << e.what() << '\n';
    }
    return 0;
}
```

OUTPUT

```
[start_server] logging: config.ini: line 12: missing '='
[main] server failed to start: config.ini: line 12: missing '='
```

16.4.2 Stack unwinding and destructors

While an exception propagates, every function it leaves is exited as if by return: all fully constructed local objects are **destroyed**, in reverse order, so their destructors release their resources. This is **stack unwinding**, and it is why RAII makes C++ code exception-safe automatically. Raw resources (a raw new without delete) are **not** cleaned up.

DESTRUCTORS RUN DURING UNWINDING

main.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>

struct Guard {
    std::string name;
    explicit Guard(std::string n) : name(std::move(n)) { std::cout << "  acquire " << name << '\n'; }
    ~Guard() { std::cout << "  release " << name << " (unwinding)\n"; }
};
void level3() { Guard g("lock C"); throw std::runtime_error("failure deep in level3"); }
void level2() { Guard g("file B"); level3(); std::cout << "never printed\n"; }
void level1() { Guard g("connection A"); level2(); }

int main() {
    try {
        level1();
    } catch (const std::exception& e) {
        std::cout << "caught in main: " << e.what() << '\n';
    }
    return 0;
}
```

OUTPUT

```
  acquire connection A
  acquire file B
  acquire lock C
  release lock C (unwinding)
  release file B (unwinding)
  release connection A (unwinding)
caught in main: failure deep in level3
```

16.4.3 Exceptions in constructors and destructors

Constructors should throw when they cannot establish the object's invariant: there is no other way to report failure. If a constructor throws, the object never existed, its destructor does **not** run, but members and bases already constructed **are** destroyed. So hold resources in RAII members, never in raw pointers acquired in the constructor body. **Destructors** must not throw: they are noexcept by default, and a destructor throwing during unwinding would call std::terminate.

A THROWING CONSTRUCTOR

main.cpp

```
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>

struct Member {
    std::string name;
    explicit Member(std::string n) : name(std::move(n)) { std::cout << "  member " << name << "
constructed\n"; }
    ~Member() { std::cout << "  member " << name << " destroyed\n"; }
};
class Widget {
public:
    explicit Widget(int size) : a_("a"), buffer_(std::make_unique<int[]>(size)), b_("b") {
        std::cout << "  constructor body running\n";
        if (size > 100) throw std::length_error("size too large");     // object is never created
    }
    ~Widget() { std::cout << "  ~Widget (only for fully constructed objects)\n"; }
private:
    Member a_;
    std::unique_ptr<int[]> buffer_;          // RAII member: freed even if the constructor throws
    Member b_;
};
int main() {
    try {
        std::cout << "Widget(10):\n";
        Widget ok(10);
        std::cout << "Widget(500):\n";
        Widget bad(500);
    } catch (const std::exception& e) {
        std::cout << "caught: " << e.what() << '\n';
    }
    return 0;
}
```

OUTPUT

```
Widget(10):
  member a constructed
  member b constructed
  constructor body running
Widget(500):
  member a constructed
  member b constructed
  constructor body running
  member b destroyed
  member a destroyed
  ~Widget (only for fully constructed objects)
  member b destroyed
  member a destroyed
caught: size too large
```

16.4.4 noexcept specifier and operator

The `noexcept` **specifier** declares that a function does not throw. If it does throw anyway, `std::terminate` is called immediately. The compiler can generate smaller, faster code, and the library relies on it: `std::vector` moves elements when growing only if their move constructor is noexcept, otherwise it copies them to keep the strong guarantee. The `noexcept(expr)` **operator** asks at compile time whether an expression can throw. Destructors, move operations and swap functions should be noexcept.

NOEXCEPT CHANGES WHAT THE LIBRARY DOES

main.cpp

```cpp
#include <iostream>
#include <stdexcept>
#include <vector>

struct FastMove {
    FastMove() = default;
    FastMove(const FastMove&) { std::cout << "copy "; }
    FastMove(FastMove&&) noexcept { std::cout << "move "; }      // noexcept move
};
struct SlowMove {
    SlowMove() = default;
    SlowMove(const SlowMove&) { std::cout << "copy "; }
    SlowMove(SlowMove&&) { std::cout << "move "; }               // may throw (not declared noexcept)
};
int safe_add(int a, int b) noexcept { return a + b; }
void broken() noexcept { throw std::runtime_error("oops"); }   // will terminate if called

int main() {
    std::cout << std::boolalpha << "safe_add noexcept? " << noexcept(safe_add(1, 2))
              << ", std::vector<int>::size noexcept? " << noexcept(std::vector<int>{}.size()) << '\n';
    std::vector<FastMove> a(2);
    std::cout << "growing vector<FastMove>: ";
    a.emplace_back();                                // reallocation: moves the old elements
    std::cout << "\ngrowing vector<SlowMove>: ";
    std::vector<SlowMove> b(2);
    b.emplace_back();                                // reallocation: copies (move might throw)
    std::cout << '\n';
    (void)broken;
    return 0;
}
```

OUTPUT

```
safe_add noexcept? true, std::vector<int>::size noexcept? true
growing vector<FastMove>: move move
growing vector<SlowMove>: copy copy
```

16.4.5 Exception safety guarantees: no-throw, strong, basic

Guarantee	Promise if an exception is thrown	Example
No-throw (nofail)	the operation never throws	destructors, swap, move constructors, `vector::size`
Strong (commit or rollback)	the state is exactly as before the call: all or nothing	`vector::push_back` (with noexcept moves), copy-and-swap assignment
Basic	no resources leak and all objects remain valid (usable and destructible), but their values may have changed	most operations on most containers
None	anything can happen: leaks, corrupt objects	code with raw resources and no RAII: avoid

THE STRONG GUARANTEE WITH "COPY, MODIFY, SWAP"

main.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

class Roster {
public:
    // strong guarantee: build the new state on the side, then commit with a non-throwing swap
    void add_all(const std::vector<std::string>& names) {
        std::vector<std::string> copy = names_;                    // may throw: names_ untouched
        for (const auto& n : names) {
            if (n.empty()) throw std::invalid_argument("empty name in batch");
            copy.push_back(n);
        }
        names_.swap(copy);                                          // noexcept commit
    }
    void print() const { for (const auto& n : names_) std::cout << n << ' '; std::cout << "(" <<
names_.size() << ")\n"; }
private:
    std::vector<std::string> names_ = {"Asha"};
};
int main() {
    Roster r;
    r.add_all({"Ravi", "Meera"});
    r.print();
    try {
        r.add_all({"Kabir", "", "Zoya"});                           // fails half-way
    } catch (const std::exception& e) {
        std::cout << "batch rejected: " << e.what() << " -> roster unchanged: ";
        r.print();
    }
    return 0;
}
```

OUTPUT

```
Asha Ravi Meera (3)
batch rejected: empty name in batch -> roster unchanged: Asha Ravi Meera (3)
```

16.4.6 RAII as the key to exception safety

Exceptions and manual cleanup do not mix: any line may throw, so cleanup code at the end of a function is easily skipped. With RAII every resource is owned by an object whose destructor runs during unwinding, so writing exception-safe code mostly means: own every resource with an RAII type (containers, smart pointers, std::fstream, std::lock_guard), and order operations so that state changes are committed only after everything that may throw has succeeded.

THE SAME FUNCTION WITHOUT AND WITH RAII

main.cpp

```cpp
#include <iostream>
#include <memory>
#include <mutex>
#include <stdexcept>

std::mutex m;
int counter = 0;

void unsafe_update(bool fail) {
    int* scratch = new int[100];          // raw resource
    m.lock();                              // raw lock
    if (fail) throw std::runtime_error("failure");   // leaks scratch, leaves m locked forever!
    ++counter;
    m.unlock();
    delete[] scratch;
}
void safe_update(bool fail) {
    auto scratch = std::make_unique<int[]>(100);     // freed automatically
    std::lock_guard<std::mutex> lock(m);              // unlocked automatically
    if (fail) throw std::runtime_error("failure");
    ++counter;
}
int main() {
    try { safe_update(true); } catch (const std::exception&) {}
    safe_update(false);
    std::cout << "safe version: counter = " << counter << ", mutex usable again: "
              << std::boolalpha << m.try_lock() << '\n';
    m.unlock();
    (void)unsafe_update;
    return 0;
}
```

OUTPUT

```
safe version: counter = 1, mutex usable again: true
```

16.5 Alternatives to exceptions

Exceptions are best for errors that are rare and handled far from where they occur. For "expected failures" that the immediate caller will handle (parse this number, find this key), and in code that disables exceptions (some games, embedded and real-time systems), C++ offers value-based alternatives.

16.5.1 Error codes

The classic C approach: return a status (bool, enum or `std::error_code`) and deliver the result through an output parameter. Cheap and explicit, but easy to ignore (mark such functions `[[nodiscard]]`) and verbose to propagate.

16.5.2 std::optional (C++17)

`std::optional<T>` (header `<optional>`) holds either a `T` or nothing (`std::nullopt`). Return it from functions that may legitimately have no result. Test it with `if (opt)` or `has_value()`, access the value with `*opt` or `value()` (which throws if empty), or use `value_or(default)`. It says that something failed, but not why.

16.5.3 std::expected (C++23)

`std::expected<T, E>` (header `<expected>`) holds either a value of type T or an **error** of type E, so the caller learns why the operation failed, without exceptions. Return `std::unexpected(error)` for the failure case. C++23 also adds monadic helpers (`and_then`, `transform`, `or_else`) to chain operations that may fail.

ERROR CODES, STD::OPTIONAL AND STD::EXPECTED main.cpp

```
#include <charconv>
#include <expected>
#include <iostream>
#include <optional>
#include <string>
#include <string_view>

enum class ParseError { Empty, NotANumber, OutOfRange };
const char* describe(ParseError e) {
    switch (e) {
    case ParseError::Empty: return "empty input";
    case ParseError::NotANumber: return "not a number";
    case ParseError::OutOfRange: return "out of range 0-150";
    }
    return "?";
}
[[nodiscard]] bool parse_code(std::string_view s, int& out) {               // 1. error code style
    auto [p, ec] = std::from_chars(s.data(), s.data() + s.size(), out);
    return ec == std::errc{} && p == s.data() + s.size();
}
std::optional<int> parse_opt(std::string_view s) {                          // 2. optional: value or
nothing
    int v;
    if (!parse_code(s, v)) return std::nullopt;
    return v;
}
std::expected<int, ParseError> parse_age(std::string_view s) {             // 3. expected: value or
reason
    if (s.empty()) return std::unexpected(ParseError::Empty);
    int v;
    if (!parse_code(s, v)) return std::unexpected(ParseError::NotANumber);
    if (v < 0 || v > 150) return std::unexpected(ParseError::OutOfRange);
    return v;
}
int main() {
    for (std::string_view in : {"42", "4x2", "", "999"}) {
        int v;
        std::cout << '"' << in << "\": code " << (parse_code(in, v) ? "ok" : "fail")
                  << ", optional " << parse_opt(in).value_or(-1);
        auto r = parse_age(in);
        if (r) std::cout << ", expected " << *r;
        else   std::cout << ", expected error: " << describe(r.error());
        auto next_year = parse_age(in).transform([](int a) { return a + 1; });   // C++23 monadic op
        std::cout << " | next year: " << (next_year ? std::to_string(*next_year) : std::string("n/a"))
<< '\n';
    }
    return 0;
}
```

```
OUTPUT
"42": code ok, optional 42, expected 42 | next year: 43
"4x2": code fail, optional -1, expected error: not a number | next year: n/a
"": code fail, optional -1, expected error: empty input | next year: n/a
"999": code ok, optional 999, expected error: out of range 0-150 | next year: n/a
```

Notice the last line: `parse_code` and `parse_opt` only check that the text is a number, so "999" passes them, while `parse_age` also checks the range and its `std::expected` reports *which* rule failed. `transform` applies a function only when a value is present and passes the error through untouched, so failures flow through a chain of steps without any if statements.

Mechanism	Use when
Exceptions	the error is rare and must be handled several levels up; constructors; you want errors impossible to ignore
`std::expected<T, E>`	failure is a normal outcome and the caller needs the reason
`std::optional<T>`	"no result" is a normal outcome and the reason does not matter (lookup, search)
Error codes / bool	C interop, very hot paths, code built with exceptions disabled
`assert`	bugs: conditions that can never happen in a correct program

Throw by value, catch by const reference, most specific first. Derive custom exceptions from std::runtime_error (or logic_error) and use a bare throw; to rethrow. Stack unwinding destroys locals, so RAII makes code exception-safe; constructors may throw, destructors must not; mark moves and swaps noexcept; aim for the strong guarantee with copy-modify-swap. For expected failures prefer std::optional or std::expected.

PRACTICE CHECKPOINT

Write an **ATM withdrawal** program with a custom **InsufficientFunds** exception, and the same function rewritten to return **std::optional** or **std::expected**.

SAMPLE SOLUTION main.cpp

```cpp
#include <expected>
#include <iomanip>
#include <iostream>
#include <optional>
#include <stdexcept>
#include <string>

class InsufficientFunds : public std::runtime_error {
public:
    InsufficientFunds(double balance, double amount)
        : std::runtime_error("insufficient funds"), balance_(balance), amount_(amount) {}
    double shortfall() const { return amount_ - balance_; }
private:
    double balance_, amount_;
};
class InvalidAmount : public std::invalid_argument {
public:
    explicit InvalidAmount(double a) : std::invalid_argument("amount must be a positive multiple of
100, got " + std::to_string(static_cast<int>(a))) {}
};
```

```
// Version 1: exceptions
double withdraw(double& balance, double amount) {
    if (amount <= 0 || static_cast<long>(amount) % 100 != 0) throw InvalidAmount(amount);
    if (amount > balance) throw InsufficientFunds(balance, amount);
    balance -= amount;
    return balance;
}
// Version 2: std::optional (success or nothing)
std::optional<double> try_withdraw(double& balance, double amount) {
    if (amount <= 0 || static_cast<long>(amount) % 100 != 0 || amount > balance) return std::nullopt;
    balance -= amount;
    return balance;
}
// Version 3: std::expected (success or a reason)
enum class AtmError { InvalidAmount, InsufficientFunds };
std::expected<double, AtmError> withdraw_expected(double& balance, double amount) {
    if (amount <= 0 || static_cast<long>(amount) % 100 != 0) return
std::unexpected(AtmError::InvalidAmount);
    if (amount > balance) return std::unexpected(AtmError::InsufficientFunds);
    balance -= amount;
    return balance;
}

int main() {
    std::cout << std::fixed << std::setprecision(2);
    double balance = 5000;
    double requests[] = {1500, 250, 8000, 3500};

    std::cout << "== exceptions ==\n";
    for (double r : requests) {
        try {
            double left = withdraw(balance, r);
            std::cout << "dispensed " << r << ", balance " << left << '\n';
        } catch (const InsufficientFunds& e) {
            std::cout << "declined " << r << ": " << e.what() << " (short by " << e.shortfall() <<
")\n";
        } catch (const std::invalid_argument& e) {
            std::cout << "declined " << r << ": " << e.what() << '\n';
        }
    }
    std::cout << "== std::optional ==\n";
    balance = 5000;
    for (double r : requests) {
        if (auto left = try_withdraw(balance, r)) std::cout << "dispensed " << r << ", balance " <<
*left << '\n';
        else std::cout << "declined " << r << " (no reason available)\n";
    }
    std::cout << "== std::expected ==\n";
    balance = 5000;
    for (double r : requests) {
        auto res = withdraw_expected(balance, r);
        if (res) std::cout << "dispensed " << r << ", balance " << *res << '\n';
        else std::cout << "declined " << r << ": "
                       << (res.error() == AtmError::InsufficientFunds ? "insufficient funds" :
"invalid amount") << '\n';
    }
    return 0;
}
```

OUTPUT

```
== exceptions ==
dispensed 1500.00, balance 3500.00
declined 250.00: amount must be a positive multiple of 100, got 250
declined 8000.00: insufficient funds (short by 4500.00)
dispensed 3500.00, balance 0.00
== std::optional ==
dispensed 1500.00, balance 3500.00
declined 250.00 (no reason available)
declined 8000.00 (no reason available)
dispensed 3500.00, balance 0.00
== std::expected ==
dispensed 1500.00, balance 3500.00
declined 250.00: invalid amount
declined 8000.00: insufficient funds
dispensed 3500.00, balance 0.00
```

MODULE 17 · INTERMEDIATE

Templates and generic programming

Minimum duration **3 days** | Level **Intermediate**

Templates let you write a function or a class once and use it with many types: the compiler generates a separate, fully type-checked version for every type you use. This is generic programming, and it is how the whole Standard Template Library (vector, map, sort, find) is built. This module covers function templates, class templates, non-type parameters, specialization, how templates are organised in headers, how to read their error messages, and variadic templates with fold expressions.

IN THIS MODULE

17.1 Why templates
17.2 Function templates
17.3 Class templates
17.4 Non-type template parameters
17.5 Template specialization
17.6 Templates in practice
17.7 Variadic templates (introduction)

17.1 Why templates

17.1.1 Code duplication without templates

Suppose we need a function that returns the larger of two values. Without templates we must write one overload per type, with identical bodies. Every bug fix has to be repeated in every copy, and a new type (say `long long` or a `Date` class) needs yet another copy.

THREE COPIES OF THE SAME LOGIC

main.cpp

```
#include <iostream>
#include <string>

int         larger(int a, int b)                                { return a > b ? a : b; }
double      larger(double a, double b)                          { return a > b ? a : b; }
std::string larger(const std::string& a, const std::string& b) { return a > b ? a : b; }
// ... and again for long, char, float, Date, Money ...

int main() {
    std::cout << larger(3, 9) << ' ' << larger(2.5, 1.5) << ' ' << larger(std::string("pear"),
std::string("apple")) << '\n';
    return 0;
}
```

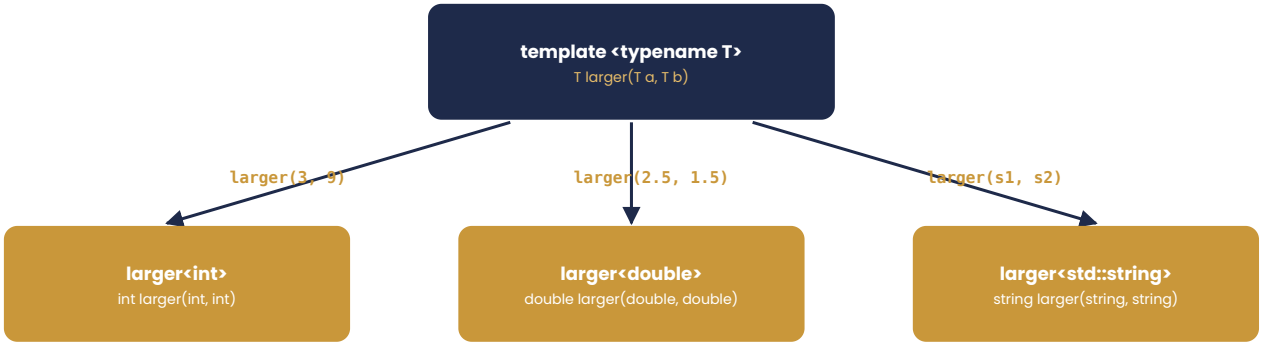
OUTPUT

```
9 2.5 pear
```

The old C answer was a macro, `#define LARGER(a,b) ((a)>(b)?(a):(b))`, which has no type checking and evaluates its arguments twice. Another answer is to work with `void*` and sizes, as C's `qsort` does, which throws type safety away. Templates give us one definition **and** full type safety.

17.1.2 Compile-time code generation

A template is a **pattern** for code, not code itself. When you use it with a particular type, the compiler **instantiates** it: it substitutes the type for the parameter, produces an ordinary function or class, and compiles that. The instantiated code is just as fast as code written by hand for that type; there is no run-time cost, no boxing and no virtual calls. The price is paid at compile time: longer builds and, when something is wrong, longer error messages.



ONE TEMPLATE, THREE INSTANTIATIONS GENERATED BY THE COMPILER

	Overloads	Macros	`void*` (C style)	Templates
Written once	no	yes	yes	yes
Type checked	yes	no	no	yes
Speed	full	full	indirect calls, casts	full
Works with new types	add a copy	yes, unsafely	yes, unsafely	yes

17.2 Function templates

17.2.1 Syntax template <typename T>

A function template begins with a **template parameter list**: `template <typename T>`. `T` is a type parameter, a placeholder for any type; inside the function it is used like an ordinary type name. The keyword `class` means exactly the same as `typename` here (`template <class T>`); modern code prefers `typename`. The body must make sense for every type you use: `larger` needs `>` and copying.

A FIRST FUNCTION TEMPLATE

main.cpp

```cpp
#include <iostream>
#include <string>

template <typename T>                // T: a placeholder for a type
T larger(const T& a, const T& b) {   // const T&: avoids copying big objects
    return a > b ? a : b;
}

template <typename T>
void show_twice(const T& value) {
    std::cout << value << ' ' << value << '\n';
}

int main() {
    std::cout << larger(3, 9) << '\n';                                  // T = int
    std::cout << larger(2.5, 1.5) << '\n';                              // T = double
    std::cout << larger(std::string("pear"), std::string("apple")) << '\n'; // T = std::string
    std::cout << larger('a', 'z') << '\n';                              // T = char
    show_twice("hi");
    show_twice(42);
    return 0;
}
```

OUTPUT

```
9
2.5
pear
z
hi hi
42 42
```

NOTE

A template is compiled in two phases. When the definition is read, everything that does not depend on `T` is checked (syntax, names not involving T). Only when it is instantiated with a real type are the dependent parts checked: `a > b` is verified for `int` when you call `larger(3, 9)`. A template that is never used generates no code at all.

17.2.2 Template argument deduction

Usually you do not name `T`: the compiler **deduces** it from the function arguments. With a by-value parameter `T`, top-level `const` and references are dropped and arrays decay to pointers; with `const T&` the exact type is kept. Each argument is deduced independently and all must agree: `larger(3, 2.5)` fails, because the first argument says `T = int` and the second `T = double`. No implicit conversions are applied during deduction.

WHAT DOES T BECOME?

main.cpp

```
#include <iostream>
#include <typeinfo>

template <typename T>
void what(T) { std::cout << typeid(T).name() << '\n'; }   // mangled name; i = int, d = double...

template <typename T>
T larger(const T& a, const T& b) { return a > b ? a : b; }

int main() {
    const int ci = 5;
    int arr[3] = {1, 2, 3};
    what(10);        // T = int
    what(3.14);      // T = double
    what(ci);        // T = int (top-level const dropped for by-value T)
    what(arr);       // T = int* (array decays to pointer)
    what("text");    // T = const char*
    // larger(3, 2.5);   error: deduced conflicting types 'int' and 'double'
    return 0;
}
```

OUTPUT

```
i
d
i
Pi
PKc
```

NOTE

`typeid(T).name()` returns an implementation-defined name; GCC prints mangled names: `i` is int, `d` double, `Pi` pointer to int, `PKc` pointer to const char. The command `c++filt -t PKc` turns them back into `char const*`.

CONFLICTING DEDUCTION IS AN ERROR

main.cpp

```
template <typename T>
T larger(const T& a, const T& b) { return a > b ? a : b; }

int main() {
    return larger(3, 2.5) > 0 ? 0 : 1;
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:5:18: error: no matching function for call to ‘larger(int, double)’
    5 |     return larger(3, 2.5) > 0 ? 0 : 1;
      |            ~~~~~~^~~~~~~~
main.cpp:2:3: note: candidate: ‘template<class T> T larger(const T&, const T&)’
    2 | T larger(const T& a, const T& b) { return a > b ? a : b; }
      |   ^~~~~~
main.cpp:2:3: note:   template argument deduction/substitution failed:
main.cpp:5:18: note:   deduced conflicting types for parameter ‘const T’ (‘int’ and ‘double’)
    5 |     return larger(3, 2.5) > 0 ? 0 : 1;
      |            ~~~~~~^~~~~~~~
```

17.2.3 Explicit template arguments

You can name the template argument yourself in angle brackets: `larger<double>(3, 2.5)`. Then nothing is deduced for that parameter, and ordinary conversions apply to the arguments (3 becomes 3.0). Explicit arguments are required when a template parameter does not appear in the function parameters at all, for example a parameter that only describes the return type.

NAMING T EXPLICITLY main.cpp

```
#include <iostream>
#include <string>
#include <sstream>

template <typename T>
T larger(const T& a, const T& b) { return a > b ? a : b; }

template <typename T>                    // T appears only in the return type
T parse(const std::string& text) {
    std::istringstream in(text);
    T value{};
    in >> value;
    return value;
}

int main() {
    std::cout << larger<double>(3, 2.5) << '\n';          // 3 converted to 3.0
    std::cout << larger<std::string>("kiwi", "fig") << '\n';  // literals converted to std::string
    int n = parse<int>("42");
    double d = parse<double>("2.75");
    std::cout << n + 1 << ' ' << d * 2 << '\n';
    return 0;
}
```

OUTPUT

```
3
kiwi
43 5.5
```

WATCH OUT

Without `<std::string>`, `larger("kiwi", "fig")` would not even compile (the two literals are arrays of different lengths, `const char[5]` and `const char[4]`), and with equal lengths it would compare the **addresses** of the two literals, not their text. Be careful when a template meets C strings.

17.2.4 Multiple template parameters

A template may have several type parameters. A function that adds two different types needs a return type that depends on both; since C++14 we simply write `auto` and let the compiler deduce it from the `return` statement. (In C++11 this needed a trailing `-> decltype(a + b)`.) The standard `std::common_type_t<A, B>` names the type both convert to.

TWO TYPE PARAMETERS

main.cpp

```
#include <iostream>
#include <string>
#include <type_traits>

template <typename A, typename B>
auto add(const A& a, const B& b) {            // return type deduced from a + b
    return a + b;
}

template <typename A, typename B>
std::common_type_t<A, B> larger(const A& a, const B& b) {
    return a > b ? a : b;
}

template <typename K, typename V>
void print_pair(const K& key, const V& value) {
    std::cout << key << " = " << value << '\n';
}

int main() {
    std::cout << add(3, 2.5) << '\n';                          // int + double -> double
    std::cout << add(std::string("C"), "++") << '\n';          // string + const char* -> string
    std::cout << larger(3, 2.5) << ' ' << larger(10, 7.5) << '\n';
    print_pair("pi", 3.14159);
    print_pair(7, 'x');
    return 0;
}
```

OUTPUT

```
5.5
C++
3 10
pi = 3.14159
7 = x
```

17.2.5 Overloading function templates

Function templates can be overloaded with other templates and with ordinary functions. Overload resolution picks the best match; when a template and a non-template match equally well, the **non-template wins**. A more specialised template (for example `T*` rather than `T`) is preferred over a more general one. This lets you provide special behaviour for particular kinds of arguments.

THREE OVERLOADS OF LARGER main.cpp

```
#include <cstring>
#include <iostream>

template <typename T>
const T& larger(const T& a, const T& b) {              // #1 general
    std::cout << "[general] ";
    return a > b ? a : b;
}
template <typename T>
T* larger(T* a, T* b) {                                 // #2 for pointers: compare what they point
to
    std::cout << "[pointer] ";
    return *a > *b ? a : b;
}
const char* larger(const char* a, const char* b) {      // #3 ordinary function for C strings
    std::cout << "[c-string] ";
    return std::strcmp(a, b) > 0 ? a : b;
}

int main() {
    int x = 4, y = 11;
    std::cout << larger(x, y) << '\n';
    std::cout << *larger(&x, &y) << '\n';
    std::cout << larger("apple", "pear") << '\n';
    return 0;
}
```

OUTPUT

```
[general] 11
[pointer] 11
[c-string] pear
```

`larger(x, y)` matches only #1. `larger(&x, &y)` matches #1 (T = int*) and #2 (T = int); #2 is more specialised, so it wins and compares the values 4 and 11 rather than the addresses. `larger("apple", "pear")` picks the ordinary function #3, which compares text.

17.3 Class templates

17.3.1 Defining a class template

A **class template** describes a family of classes. `std::vector<int>` and `std::vector<std::string>` are two different classes generated from the same template. Inside the template, the class can refer to itself simply as `Box` (the injected class name) instead of `Box<T>`.

A SIMPLE CLASS TEMPLATE

main.cpp

```
#include <iostream>
#include <string>

template <typename T>
class Box {
public:
    explicit Box(T value) : value_(std::move(value)) {}
    const T& get() const { return value_; }
    void set(T value) { value_ = std::move(value); }
    bool same_as(const Box& other) const { return value_ == other.value_; }   // Box means Box<T>
private:
    T value_;
};

int main() {
    Box<int> a(42);
    Box<std::string> b("hello");
    Box<int> c(42);
    b.set(b.get() + " world");
    std::cout << a.get() << " | " << b.get() << '\n';
    std::cout << std::boolalpha << "a same as c? " << a.same_as(c) << '\n';
    // a.same_as(b);   error: Box<int> and Box<std::string> are unrelated types
    std::cout << "sizeof(Box<char>) = " << sizeof(Box<char>) << ", sizeof(Box<double>) = " <<
sizeof(Box<double>) << '\n';
    return 0;
}
```

OUTPUT

```
42 | hello world
a same as c? true
sizeof(Box<char>) = 1, sizeof(Box<double>) = 8
```

A bigger example: a fixed-capacity stack backed by a `std::vector`. The same code gives us a stack of ints, of strings or of any class.

STACK<T>

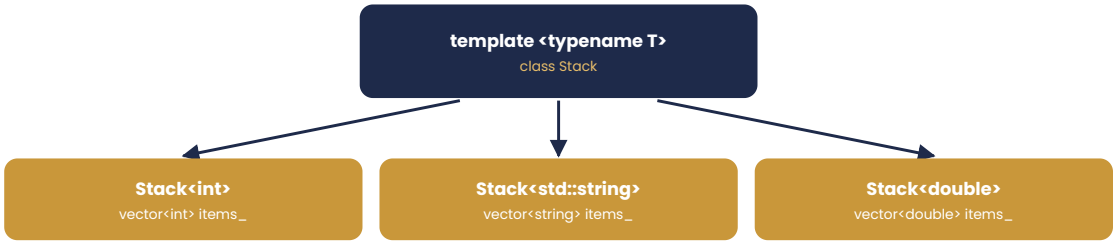
main.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

template <typename T>
class Stack {
public:
    void push(const T& item) { items_.push_back(item); }
    void pop() {
        if (items_.empty()) throw std::out_of_range("pop on empty stack");
        items_.pop_back();
    }
    const T& top() const {
        if (items_.empty()) throw std::out_of_range("top on empty stack");
        return items_.back();
    }
    bool empty() const { return items_.empty(); }
    std::size_t size() const { return items_.size(); }
private:
    std::vector<T> items_;
};

int main() {
```

```
    Stack<int> numbers;
    for (int i = 1; i <= 4; ++i) numbers.push(i * i);
    while (!numbers.empty()) {
        std::cout << numbers.top() << ' ';
        numbers.pop();
    }
    std::cout << '\n';

    Stack<std::string> undo;
    undo.push("type 'Hello'");
    undo.push("make bold");
    undo.push("delete line 3");
    std::cout << "undo: " << undo.top() << " (" << undo.size() << " actions)\n";
    try {
        Stack<double> empty;
        empty.pop();
    } catch (const std::out_of_range& e) {
        std::cout << "error: " << e.what() << '\n';
    }
    return 0;
}
```

OUTPUT

```
16 9 4 1
undo: delete line 3 (3 actions)
error: pop on empty stack
```



EACH USE WITH A NEW TYPE GENERATES A SEPARATE CLASS

NOTE

Member functions of a class template are instantiated only if they are used. A `Stack<T>` with a `print()` member that uses `<<` still works for a type without `<<`, as long as nobody calls `print()` on it.

17.3.2 Member functions outside the class

A member function defined outside the class body must repeat the template header and qualify the name with the full class name `Stack<T>::`. Inside the function, after the qualified name, the short name `Stack` can be used again.

DEFINING MEMBERS OUTSIDE THE CLASS TEMPLATE main.cpp

```
#include <iostream>
#include <vector>

template <typename T>
class Stack {
public:
    void push(const T& item);
    T pop();
    bool empty() const;
    Stack& operator+=(const T& item);        // short name inside the class
private:
```

```
    std::vector<T> items_;
};

template <typename T>                          // repeat the template header ...
void Stack<T>::push(const T& item) {          // ... and qualify with Stack<T>::
    items_.push_back(item);
}
template <typename T>
T Stack<T>::pop() {
    T top = items_.back();
    items_.pop_back();
    return top;
}
template <typename T>
bool Stack<T>::empty() const { return items_.empty(); }

template <typename T>
Stack<T>& Stack<T>::operator+=(const T& item) {   // return type is outside the scope: Stack<T>&
    push(item);
    return *this;
}

int main() {
    Stack<char> s;
    s += 'a';
    s += 'b';
    s.push('c');
    while (!s.empty()) std::cout << s.pop();
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
cba
```

17.3.3 Class template argument deduction (CTAD, C++17)

Since C++17 the compiler can deduce class template arguments from the constructor arguments, just as for function templates: `std::pair p(1, 2.5);` is a `std::pair<int, double>`, `std::vector v{1, 2, 3};` a `std::vector<int>`. For your own classes this works automatically when the constructor parameters mention the template parameters. When they do not, you can add a **deduction guide**.

CTAD AND DEDUCTION GUIDES

main.cpp

```cpp
#include <iostream>
#include <string>
#include <utility>
#include <vector>

template <typename T>
class Box {
public:
    explicit Box(T value) : value_(value) {}
    const T& get() const { return value_; }
private:
    T value_;
};

template <typename A, typename B>
struct Pair {
    A first;
    B second;
};
template <typename A, typename B>
Pair(A, B) -> Pair<A, B>;                       // deduction guide for the aggregate

Box(const char*) -> Box<std::string>;            // guide: string literals make a Box<std::string>

int main() {
    Box b1(10);                                  // Box<int>
    Box b2(2.5);                                 // Box<double>
    Box b3("text");                              // Box<std::string> thanks to the guide
    std::pair p(1, 'x');                         // std::pair<int, char>
    std::vector v{1.5, 2.5};                     // std::vector<double>
    Pair q{"id", 7};                             // Pair<const char*, int>
    std::cout << b1.get() + 1 << ' ' << b2.get() * 2 << ' ' << b3.get().size() << '\n';
    std::cout << p.first << p.second << ' ' << v.size() << ' ' << q.first << q.second << '\n';
    return 0;
}
```

OUTPUT

```
11 5 4
1x 2 id7
```

WATCH OUT

A classic CTAD surprise: `std::vector v{5, 1};` is a vector holding 5 and 1, but `std::vector v(5, 1);` is five 1s. And `std::vector v2{v};` is a copy of `v` (a `vector<int>`), not a vector of vectors. When in doubt, spell out the type.

17.3.4 Default template arguments

Template parameters can have defaults, used when the argument is not given. The standard library uses them everywhere: `std::vector<T, Allocator = std::allocator<T>>`, `std::map<K, V, Compare = std::less<K>>`. That is why `std::map<std::string, int, std::greater<>>` sorts in reverse: we replaced the default comparison.

A SORTED LIST WITH A DEFAULT COMPARISON main.cpp

```
#include <functional>
#include <iostream>
#include <map>
#include <string>
#include <vector>

template <typename T, typename Compare = std::less<T>>       // default: ascending order
class SortedList {
public:
    void insert(const T& value) {
        auto it = items_.begin();
        while (it != items_.end() && cmp_(*it, value)) ++it;   // find the first position not before
value
        items_.insert(it, value);
    }
    void print() const {
        for (const T& x : items_) std::cout << x << ' ';
        std::cout << '\n';
    }
private:
    std::vector<T> items_;
    Compare cmp_;
};

int main() {
    SortedList<int> up;                          // Compare = std::less<int>
    SortedList<int, std::greater<int>> down;       // override the default
    for (int x : {5, 1, 4, 2, 3}) { up.insert(x); down.insert(x); }
    up.print();
    down.print();
    SortedList<std::string> words;
    for (const char* w : {"mango", "apple", "kiwi"}) words.insert(w);
    words.print();
    std::map<std::string, int, std::greater<>> scores{{"Asha", 90}, {"Ravi", 75}, {"Meera", 88}};
    for (const auto& [name, s] : scores) std::cout << name << ':' << s << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
5 4 3 2 1
apple kiwi mango
Ravi:75 Meera:88 Asha:90
```

17.4 Non-type template parameters

17.4.1 template <int N>

Template parameters do not have to be types. A **non-type template parameter** is a compile-time constant value: an integer, an enum, a pointer, and since C++20 also a floating-point value or a simple class object. The best-known example is `std::array<int, 5>`, whose size is part of the type. Because the value is known at compile time it can size arrays, unroll loops and be checked with `static_assert`.

VALUES AS TEMPLATE PARAMETERS main.cpp

```
#include <cstddef>
#include <iostream>
```

```cpp
template <typename T, std::size_t N>
class FixedArray {
public:
    T& operator[](std::size_t i) { return data_[i]; }
    const T& operator[](std::size_t i) const { return data_[i]; }
    constexpr std::size_t size() const { return N; }
    T sum() const {
        T total{};
        for (std::size_t i = 0; i < N; ++i) total += data_[i];
        return total;
    }
private:
    T data_[N]{};                           // N is a compile-time constant
};

template <int Times>
void repeat(char c) {
    static_assert(Times > 0, "Times must be positive");
    for (int i = 0; i < Times; ++i) std::cout << c;
    std::cout << '\n';
}

template <typename T, std::size_t N>
std::size_t length(const T (&)[N]) { return N; }   // N deduced from a built-in array

int main() {
    FixedArray<int, 4> a;
    for (std::size_t i = 0; i < a.size(); ++i) a[i] = static_cast<int>(i + 1) * 10;
    std::cout << "size " << a.size() << ", sum " << a.sum() << ", bytes " << sizeof(a) << '\n';
    FixedArray<double, 2> b;
    b[0] = 1.25; b[1] = 2.5;
    std::cout << "sum " << b.sum() << '\n';
    repeat<5>('*');
    repeat<3>('-');
    int primes[] = {2, 3, 5, 7, 11, 13};
    std::cout << "primes has " << length(primes) << " elements\n";
    return 0;
}
```

OUTPUT

```
size 4, sum 100, bytes 16
sum 3.75
*****
---
primes has 6 elements
```

NOTE

`FixedArray<int, 4>` and `FixedArray<int, 5>` are different types: you cannot assign one to the other. The argument must be a constant expression; `repeat<n>` with a runtime variable `n` does not compile, while `constexpr int n = 5;` does.

17.5 Template specialization

17.5.1 Full specialization

Sometimes the general template is wrong or inefficient for one particular type. A **full (explicit) specialization** provides a completely separate implementation for a specific set of template arguments.

It starts with `template <>` and names the arguments after the template name. The specialised class can have entirely different members.

FULL SPECIALIZATIONS main.cpp

```
#include <iostream>
#include <string>

template <typename T>
struct TypeInfo {                                   // primary template
    static std::string name() { return "unknown type"; }
};
template <> struct TypeInfo<int>    { static std::string name() { return "int"; } };
template <> struct TypeInfo<double> { static std::string name() { return "double"; } };
template <> struct TypeInfo<bool>   { static std::string name() { return "bool"; } };

template <typename T>
class Printer {                                     // general version
public:
    void print(const T& v) const { std::cout << v << '\n'; }
};
template <>
class Printer<bool> {                               // full specialization for bool
public:
    void print(bool v) const { std::cout << (v ? "yes" : "no") << '\n'; }
};

template <typename T>
bool equal(const T& a, const T& b) { return a == b; }
template <>
bool equal<double>(const double& a, const double& b) {   // compare doubles with a tolerance
    double diff = a > b ? a - b : b - a;
    return diff < 1e-9;
}

int main() {
    std::cout << TypeInfo<int>::name() << ", " << TypeInfo<double>::name() << ", " <<
TypeInfo<char>::name() << '\n';
    Printer<int>().print(7);
    Printer<bool>().print(true);
    std::cout << std::boolalpha << equal(1, 1) << ' ' << equal(0.1 + 0.2, 0.3) << ' ' << (0.1 + 0.2 ==
0.3) << '\n';
    return 0;
}
```

OUTPUT

```
int, double, unknown type
7
yes
true true false
```

TIP

For functions, overloading is usually simpler than full specialization: an ordinary function `bool equal(double, double)` would have done the same job. Specialization is mainly used for class templates; `std::vector<bool>` (packed into bits) and `std::hash<std::string>` are standard library examples.

17.5.2 Partial specialization

A **partial specialization** specialises a class template for a *family* of arguments, keeping some parameters open: all pointer types `T*`, all pairs with the same type twice, all arrays of any size. It is

available for class templates only (not functions). The compiler picks the most specialised version that matches.

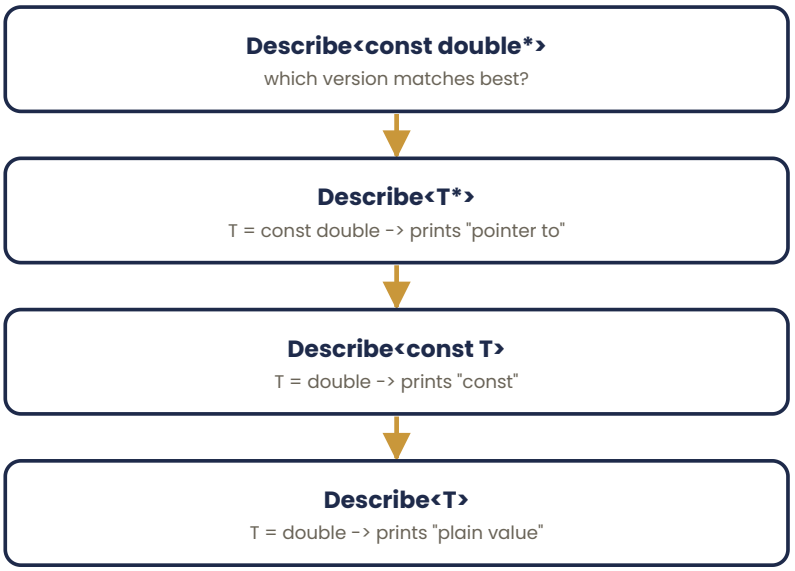
PARTIAL SPECIALIZATIONS

main.cpp

```
#include <iostream>
#include <string>

template <typename T>
struct Describe {                                       // primary
    static void show() { std::cout << "plain value\n"; }
};
template <typename T>
struct Describe<T*> {                                   // any pointer type
    static void show() { std::cout << "pointer to "; Describe<T>::show(); }
};
template <typename T>
struct Describe<const T> {                              // any const type
    static void show() { std::cout << "const "; Describe<T>::show(); }
};
template <typename T, std::size_t N>
struct Describe<T[N]> {                                 // any array type
    static void show() { std::cout << "array of " << N << " x "; Describe<T>::show(); }
};

template <typename A, typename B>
struct Pair { static constexpr const char* kind = "pair of two types"; };
template <typename A>
struct Pair<A, A> { static constexpr const char* kind = "pair of one type twice"; };

int main() {
    Describe<int>::show();
    Describe<int*>::show();
    Describe<const double*>::show();
    Describe<char**>::show();
    Describe<int[8]>::show();
    std::cout << Pair<int, double>::kind << '\n' << Pair<std::string, std::string>::kind << '\n';
    return 0;
}
```

OUTPUT

```
plain value
pointer to plain value
pointer to const plain value
pointer to pointer to plain value
array of 8 x plain value
pair of two types
pair of one type twice
```



HOW THE COMPILER PEELS THE TYPE ONE LAYER AT A TIME

17.6 Templates in practice

17.6.1 Why templates live in header files

For ordinary functions we put declarations in a header and definitions in a `.cpp` file. That does not work for templates. The compiler can instantiate `Stack<int>::push` only where it can see the **definition** of `push` and knows that `T = int`. Each `.cpp` file is compiled separately: `stack.cpp` sees the definition but not the use with int; `main.cpp` sees the use but only the declaration. Nobody generates the code, and the linker reports an undefined reference.

TEMPLATE DEFINITIONS HIDDEN IN A .CPP FILE

main.cpp

```
#include "stack.h"
#include <iostream>

int main() {
    Stack<int> s;
    s.push(5);
    std::cout << s.top() << '\n';
    return 0;
}
```

OUTPUT

```
/usr/bin/ld: /tmp/ccPj92qz.o: in function `main':
main.cpp:6:(.text+0x3f): undefined reference to `Stack<int>::push(int const&)'
/usr/bin/ld: main.cpp:7:(.text+0x4b): undefined reference to `Stack<int>::top() const'
collect2: error: ld returned 1 exit status
```

The standard solution: put the **whole** template, declarations and definitions, in the header. Every file that includes it can instantiate what it needs; the linker merges the duplicate instantiations (template functions are implicitly allowed to be defined in several translation units). Large libraries sometimes keep the bodies in a separate file named `stack.tpp` or `stack_impl.h` that the header includes at its end, purely for readability.

EVERYTHING IN THE HEADER: WORKS main.cpp

```
#include "stack.h"
#include <iostream>
#include <string>

int main() {
    Stack<int> s;
    s.push(5);
    s.push(8);
    Stack<std::string> names;
    names.push("Asha");
    std::cout << s.top() << ' ' << names.top() << '\n';
    return 0;
}
```

OUTPUT
```
8 Asha
```

An alternative, when only a few known types will ever be used, is **explicit instantiation**: keep the definitions in `stack.cpp` and add `template class Stack<int>;` there, which forces the compiler to generate all members for int in that file. Using `Stack<double>` elsewhere would then fail to link.

EXPLICIT INSTANTIATION IN THE .CPP FILE main.cpp

```
#include "stack.h"
#include <iostream>

int main() {
    Stack<int> s;
    s.push(5);
    std::cout << s.top() << '\n';
    return 0;
}
```

OUTPUT
```
5
```

17.6.2 Reading template error messages

Template errors are notorious for their length, because the compiler reports the whole chain of instantiations that led to the problem. Read them with a plan:

1. Find the **first** line containing `error:`. That is the actual problem; later errors are often consequences.
2. Read the **"required from"** lines bottom-up: the last one is in *your* code, showing which call started the instantiation, with the types that were used (`[with T = Point]`).
3. Ask what operation the template needs from your type (here: `>`) and whether your type provides it.
4. Ignore deep library internals (`__gnu_cxx`, `_M_...`) until you have checked your own call.

A TEMPLATE ERROR, AND HOW TO READ IT

main.cpp

```cpp
#include <string>

struct Point { int x, y; };

template <typename T>
const T& larger(const T& a, const T& b) {
    return a > b ? a : b;                       // needs operator> for T
}

int main() {
    Point p{1, 2}, q{3, 4};
    larger(p, q);                               // Point has no operator>
    return 0;
}
```

OUTPUT

```
main.cpp: In instantiation of ‘const T& larger(const T&, const T&) [with T = Point]’:
main.cpp:12:11:   required from here
main.cpp:7:14: error: no match for ‘operator>’ (operand types are ‘const Point’ and ‘const Point’)
    7 |     return a > b ? a : b;                       // needs operator> for T
      |            ~~^~~
```

The message says it all once you know where to look: `no match for operator>` with operand types `const Point`, in `larger<Point>`, required from `main` at the line of our call. The fix is to give `Point` an `operator>`, or to call a different function. Errors from standard algorithms work the same way: `std::sort` on a type without `operator<` produces a long report whose first `error:` line says `no match for operator<`.

TIP

C++20 **concepts** (Module 22) let a template state its requirements up front, for example `template <std::totally_ordered T>`. The error then says, at the call site, that Point does not satisfy totally_ordered, which is far shorter and clearer.

17.7 Variadic templates (introduction)

17.7.1 Parameter packs

A **variadic template** accepts any number of template arguments. `typename... Ts` declares a **template parameter pack**; `Ts... args` a **function parameter pack**. `sizeof...(args)` gives the number of elements, and `args...` **expands** the pack into a comma-separated list. Before C++17 the usual way to process a pack was recursion: handle the first argument, then call yourself with the rest, with a non-variadic overload ending the recursion.

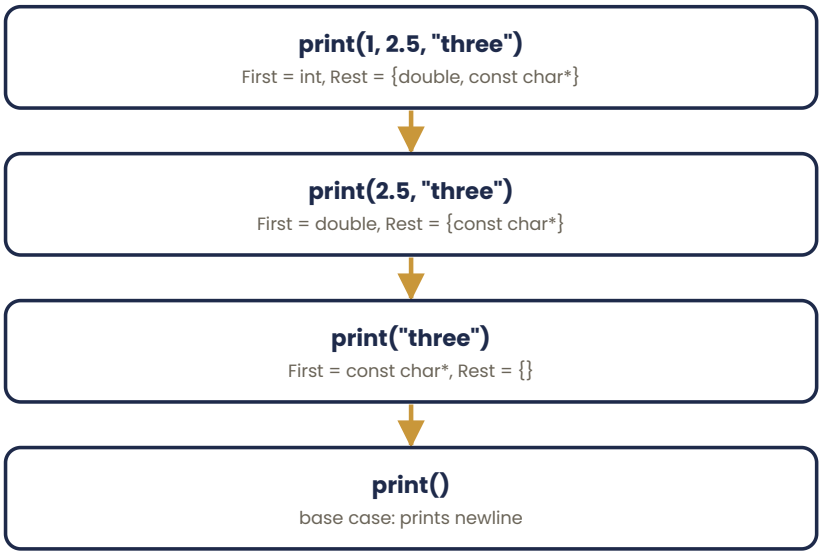
PARAMETER PACKS AND RECURSION

main.cpp

```cpp
#include <iostream>
#include <string>

template <typename... Ts>
void count(const Ts&... args) {
    std::cout << sizeof...(Ts) << " argument(s)\n";
}

void print() { std::cout << '\n'; }                    // base case: nothing left

template <typename First, typename... Rest>
void print(const First& first, const Rest&... rest) {  // peel off the first argument
    std::cout << first;
    if (sizeof...(rest) > 0) std::cout << ", ";
    print(rest...);                                    // recurse with the remaining pack
}

int main() {
    count();
    count(1, 2.5, "three");
    print(1, 2.5, "three", 'c', std::string("five"));
    print("only one");
    return 0;
}
```

OUTPUT

```
0 argument(s)
3 argument(s)
1, 2.5, three, c, five
only one
```



EACH CALL PEELS ONE ARGUMENT OFF THE PACK

A pack can also be expanded inside other expressions: `f(args)...` calls f on each element; `std::vector<int>{args...}` builds a vector; `std::make_unique<T>(std::forward<Args>(args)...)` passes every argument on unchanged. This is how `emplace_back` and `make_unique` accept any constructor arguments.

EXPANDING PACKS INTO OTHER CODE

main.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Student {
    std::string name;
    int age;
    double gpa;
    Student(std::string n, int a, double g) : name(std::move(n)), age(a), gpa(g) {}
};

template <typename T, typename... Args>
std::unique_ptr<T> create(Args&&... args) {            // like std::make_unique
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}

template <typename... Ts>
std::vector<int> lengths(const Ts&... words) {
    return {static_cast<int>(std::string(words).size())...};   // pattern expanded per element
}

int main() {
    auto s = create<Student>("Asha", 20, 8.7);
    std::cout << s->name << ", " << s->age << ", " << s->gpa << '\n';
    for (int n : lengths("a", "tree", "banana")) std::cout << n << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
Asha, 20, 8.7
1 4 6
```

17.7.2 Fold expressions (C++17)

C++17 **fold expressions** apply a binary operator across a whole pack without recursion. The four forms are: unary right fold `(args op ...)`, unary left fold `(... op args)`, and the binary folds `(args op ... op init)` and `(init op ... op args)` with a starting value. The parentheses are part of the syntax.

Fold	Expands to (for args a, b, c)
`(... + args)`	`((a + b) + c)`
`(args + ...)`	`(a + (b + c))`
`(0 + ... + args)`	`(((0 + a) + b) + c)`, and just `0` for an empty pack
`(std::cout << ... << args)`	`((std::cout << a) << b) << c`
`(f(args), ...)`	calls `f(a), f(b), f(c)` in order (comma operator)

FOLD EXPRESSIONS

main.cpp

```
#include <iostream>
#include <string>

template <typename... Ts>
auto sum(const Ts&... args) {
    return (0 + ... + args);                    // binary left fold: works for an empty pack too
}
template <typename... Ts>
bool all_positive(const Ts&... args) {
    return (... && (args > 0));                 // empty pack: true
}
template <typename... Ts>
void print_all(const Ts&... args) {
    ((std::cout << args << ' '), ...);          // comma fold: one statement per argument
    std::cout << '\n';
}
template <typename T, typename... Ts>
bool is_one_of(const T& value, const Ts&... options) {
    return ((value == options) || ...);
}

int main() {
    std::cout << sum(1, 2, 3, 4) << ' ' << sum(1.5, 2) << ' ' << sum() << '\n';
    std::cout << std::boolalpha << all_positive(3, 7, 1) << ' ' << all_positive(3, -7, 1) << '\n';
    print_all("id", 42, 3.5, 'x');
    std::string day = "sat";
    std::cout << "weekend? " << is_one_of(day, "sat", "sun") << '\n';
    return 0;
}
```

OUTPUT

```
10 3.5 0
true false
id 42 3.5 x
weekend? true
```

NOTE

With an empty pack, a unary fold over `&&` gives `true`, over `||` gives `false`, over the comma operator gives `void()`; for any other operator (such as `+`) an empty unary fold is an error. That is why `sum` uses the binary form with the starting value 0.

Templates are patterns the compiler instantiates for each type or value you use, giving reusable code with full type safety and no run-time cost. Function template arguments are deduced from the call; class template arguments can be deduced from constructors (CTAD). Use non-type parameters for compile-time sizes, specialization for type-specific behaviour, keep templates in headers, read errors from the first "error:" and the "required from" lines, and process parameter packs with fold expressions.

PRACTICE CHECKPOINT

Write a generic **max function template**, a **Stack<T> class template**, and a **variadic sum()** using a fold expression.

SAMPLE SOLUTION

main.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

// 1. Generic max: works for any type with operator<
template <typename T>
const T& my_max(const T& a, const T& b) {
    return a < b ? b : a;
}
template <typename T, typename... Rest>
const T& my_max(const T& first, const T& second, const Rest&... rest) {   // any number of arguments
    return my_max(my_max(first, second), rest...);
}

// 2. Stack<T> class template
template <typename T>
class Stack {
public:
    void push(const T& item) { items_.push_back(item); }
    T pop() {
        if (items_.empty()) throw std::out_of_range("pop from empty stack");
        T top = items_.back();
        items_.pop_back();
        return top;
    }
    const T& peek() const {
        if (items_.empty()) throw std::out_of_range("peek at empty stack");
        return items_.back();
    }
    bool empty() const { return items_.empty(); }
    std::size_t size() const { return items_.size(); }
private:
    std::vector<T> items_;
};

// 3. Variadic sum with a fold expression
template <typename... Ts>
auto sum(const Ts&... values) {
    return (0 + ... + values);
}

int main() {
    std::cout << "max(3, 9) = " << my_max(3, 9) << '\n';
    std::cout << "max(2.5, -1.0) = " << my_max(2.5, -1.0) << '\n';
    std::cout << "max(\"kiwi\", \"apple\") = " << my_max(std::string("kiwi"), std::string("apple")) <<
'\n';
    std::cout << "max(4, 17, 8, 12) = " << my_max(4, 17, 8, 12) << '\n';

    Stack<int> s;
    for (int x : {10, 20, 30}) s.push(x);
    std::cout << "stack size " << s.size() << ", top " << s.peek() << ", popped:";
    while (!s.empty()) std::cout << ' ' << s.pop();
    std::cout << '\n';
    Stack<std::string> words;
    words.push("generic");
    words.push("programming");
    std::cout << "string stack top: " << words.peek() << '\n';
    try {
        Stack<double>().pop();
    } catch (const std::out_of_range& e) {
        std::cout << "error: " << e.what() << '\n';
```

```
    }

    std::cout << "sum(1, 2, 3, 4, 5) = " << sum(1, 2, 3, 4, 5) << '\n';
    std::cout << "sum(1.5, 2, 0.25) = " << sum(1.5, 2, 0.25) << '\n';
    std::cout << "sum() = " << sum() << '\n';
    return 0;
}
```

OUTPUT

```
max(3, 9) = 9
max(2.5, -1.0) = 2.5
max("kiwi", "apple") = kiwi
max(4, 17, 8, 12) = 17
stack size 3, top 30, popped: 30 20 10
string stack top: programming
error: pop from empty stack
sum(1, 2, 3, 4, 5) = 15
sum(1.5, 2, 0.25) = 3.75
sum() = 0
```

MODULE 18 · INTERMEDIATE

File handling with streams

Minimum duration **2 days** | Level **Intermediate**

Programs need to keep data after they exit: settings, records, logs, reports. C++ file streams read and write files with exactly the same << and >> syntax used for cin and cout, and std::filesystem (C++17) manages the files and folders themselves. This module covers text and binary files, random access, stream states, std::filesystem, and the CSV and JSON formats.

IN THIS MODULE

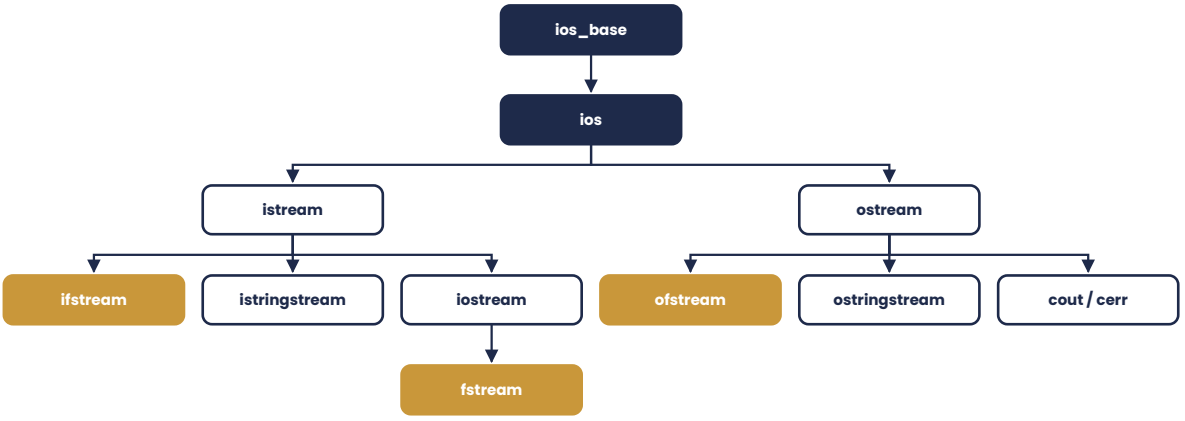
18.1 File stream classes
18.2 Text files
18.3 Binary files
18.4 Random access
18.5 Stream state and errors
18.6 std::filesystem (C++17)
18.7 Common formats

18.1 File stream classes

18.1.1 ifstream, ofstream, fstream

The header `<fstream>` provides three classes. They derive from the same base classes as `cin` and `cout`, so everything you know about console streams (`<<`, `>>`, `getline`, manipulators, stream states) works with files too. A function that takes a `std::ostream&` can write to the console, a file or a string without changes.

Class	Direction	Derives from	Typical use
`std::ifstream`	input (read)	`std::istream`	read a configuration or data file
`std::ofstream`	output (write)	`std::ostream`	write a report or log
`std::fstream`	both	`std::iostream`	update records in place



THE STREAM CLASS FAMILY: FILE STREAMS ARE ORDINARY STREAMS

SAME CODE FOR CONSOLE AND FILE

main.cpp

```cpp
#include <fstream>
#include <iostream>
#include <string>

void report(std::ostream& out) {                // works for cout, files, string streams
    out << "Daily report\n";
    out << "Orders: " << 42 << ", revenue: " << 18350.5 << '\n';
}

int main() {
    report(std::cout);                          // to the screen

    std::ofstream file("report.txt");           // create (or overwrite) a file
    report(file);                               // same function, now to the file
    file.close();

    std::ifstream in("report.txt");             // read it back
    std::string line;
    while (std::getline(in, line)) std::cout << "file says: " << line << '\n';
    return 0;
}
```

OUTPUT

```
Daily report
Orders: 42, revenue: 18350.5
file says: Daily report
file says: Orders: 42, revenue: 18350.5
```

18.1.2 Opening and closing files

A file can be opened by passing its name to the constructor, or later with `open()`. The file is closed automatically when the stream object is destroyed (RAII), so explicit `close()` is needed only when you want to close it early: to reopen it in another mode, to make the data visible to another part of the program, or to check that the final write succeeded. Output is **buffered**: data reaches the disk when the buffer fills, on `flush()` / `std::endl`, or on close.

CONSTRUCTOR, OPEN() AND CLOSE()

main.cpp

```cpp
#include <fstream>
#include <iostream>
#include <string>

int main() {
    {
        std::ofstream log("app.log");          // opened by the constructor
        log << "started\n";
    }                                          // closed here by the destructor (RAII)

    std::ofstream log;
    log.open("app.log", std::ios::app);        // opened later, in append mode
    std::cout << "is_open: " << std::boolalpha << log.is_open() << '\n';
    log << "processed 3 items\n";
    log.close();                               // explicit close: flushes the buffer now
    std::cout << "after close, is_open: " << log.is_open() << '\n';

    std::ifstream in("app.log");
    for (std::string line; std::getline(in, line); ) std::cout << "  " << line << '\n';
    return 0;
}
```

OUTPUT

```
is_open: true
after close, is_open: false
  started
  processed 3 items
```

18.1.3 File modes: ios::in, ios::out, ios::app, ios::trunc, ios::binary, ios::ate

The second argument to the constructor or `open()` is a combination of **open mode** flags joined with `|`. Each stream class has a default: ifstream uses `in`, ofstream uses `out` (which implies `trunc`), fstream uses `in | out`.

Flag	Meaning
`std::ios::in`	open for reading; the file must exist
`std::ios::out`	open for writing; creates the file; alone it truncates an existing file
`std::ios::app`	append: every write goes to the end of the file; existing content is kept
`std::ios::trunc`	discard existing content when opening
`std::ios::binary`	no newline translation (matters on Windows: `\n` vs `\r\n`); use for non-text data
`std::ios::ate`	"at end": open and move to the end immediately, but allow seeking back (unlike app)

THE EFFECT OF DIFFERENT MODES

main.cpp

```cpp
#include <fstream>
#include <iostream>
#include <string>

void show(const char* title) {
    std::ifstream in("notes.txt");
    std::cout << title << ": ";
    for (std::string w; in >> w; ) std::cout << w << ' ';
    std::cout << '\n';
}

int main() {
    std::ofstream("notes.txt") << "one two";                          // out: create
    show("after out");
    std::ofstream("notes.txt", std::ios::app) << " three";            // app: keep and add
    show("after app");
    std::ofstream("notes.txt", std::ios::out) << "fresh";              // out alone: truncates!
    show("after out again");
    std::fstream f("notes.txt", std::ios::in | std::ios::out | std::ios::ate);
    std::cout << "ate: opened at position " << f.tellg() << '\n';    // at the end, size of the file
    std::fstream nf("missing.txt", std::ios::in | std::ios::out);       // in|out does not create
    std::cout << "in|out on a missing file opened? " << std::boolalpha << nf.is_open() << '\n';
    return 0;
}
```

OUTPUT

```
after out: one two
after app: one two three
after out again: fresh
ate: opened at position 5
in|out on a missing file opened? false
```

WATCH OUT

Opening an existing file with a plain `std::ofstream` erases its content immediately, before you write anything. To add to a log or data file, use `std::ios::app`.

18.1.4 Checking if a file opened successfully

Opening can fail: the file does not exist, the path is wrong, permission is denied, the disk is full. A stream that failed to open is in a fail state and silently ignores every operation, so always test it. The stream converts to `bool` (`if (!in)`), or call `is_open()`. To find out why, `errno` / `std::strerror` usually tells you on POSIX systems.

ALWAYS CHECK THE STREAM

main.cpp

```cpp
#include <cerrno>
#include <cstring>
#include <fstream>
#include <iostream>

int main() {
    std::ifstream in("does_not_exist.txt");
    if (!in) {                                          // same as !in.is_open() here
        std::cerr << "cannot open does_not_exist.txt: " << std::strerror(errno) << '\n';
    }
    std::ofstream out("/no_such_folder/out.txt");
    if (!out.is_open()) {
        std::cerr << "cannot create /no_such_folder/out.txt: " << std::strerror(errno) << '\n';
    }
    int x = 99;
    in >> x;                                            // ignored: the stream has failed
    std::cout << "x is still " << x << '\n';
    return 0;
}
```

OUTPUT

```
cannot open does_not_exist.txt: No such file or directory
cannot create /no_such_folder/out.txt: No such file or directory
x is still 99
```

18.2 Text files

18.2.1 Writing with <<

Writing a text file looks exactly like printing. Remember to write separators (spaces, commas, newlines) yourself so the data can be read back, and use the same manipulators as for `cout` (`std::fixed`, `std::setprecision`, `std::setw`) for tidy output.

WRITING A FORMATTED BILL main.cpp

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

struct Item { std::string name; int qty; double price; };

int main() {
    std::vector<Item> cart = {{"Notebook", 3, 45.0}, {"Pen", 10, 12.5}, {"Backpack", 1, 1299.0}};
    std::ofstream bill("bill.txt");
    if (!bill) { std::cerr << "cannot create bill.txt\n"; return 1; }
    bill << std::fixed << std::setprecision(2);
    bill << std::left << std::setw(10) << "Item" << std::right << std::setw(5) << "Qty" <<
std::setw(10) << "Amount" << '\n';
    double total = 0;
    for (const Item& it : cart) {
        double amount = it.qty * it.price;
        total += amount;
        bill << std::left << std::setw(10) << it.name << std::right << std::setw(5) << it.qty <<
std::setw(10) << amount << '\n';
    }
    bill << std::left << std::setw(15) << "TOTAL" << std::right << std::setw(10) << total << '\n';
    bill.close();

    std::ifstream in("bill.txt");
    std::cout << in.rdbuf();                    // print the whole file
    return 0;
}
```

OUTPUT

```
Item        Qty    Amount
Notebook      3    135.00
Pen          10    125.00
Backpack      1   1299.00
TOTAL             1559.00
```

18.2.2 Reading word by word with >>

`>>` skips whitespace (spaces, tabs, newlines) and reads one token, converting it to the variable's type. Using the read itself as the loop condition, `while (in >> word)`, stops cleanly at the end of the file or at the first token that cannot be converted.

STORY.TXT text

```
the cat sat on the mat
the dog sat too
```

MARKS.TXT text

```
78 91
64
88 70
```

COUNTING WORDS AND READING NUMBERS main.cpp

```cpp
#include <fstream>
#include <iostream>
#include <map>
#include <string>

int main() {
    std::ifstream in("story.txt");
    if (!in) return 1;
    std::string word;
    int words = 0;
    std::map<std::string, int> freq;
    while (in >> word) {                       // stops at end of file
        ++words;
        ++freq[word];
    }
    std::cout << "words: " << words << '\n';
    for (const auto& [w, n] : freq)
        if (n > 1) std::cout << "  '" << w << "' appears " << n << " times\n";

    std::ifstream nums("marks.txt");
    int mark, sum = 0, count = 0;
    while (nums >> mark) { sum += mark; ++count; }
    std::cout << count << " marks, average " << static_cast<double>(sum) / count << '\n';
    return 0;
}
```

OUTPUT

```
words: 10
  'sat' appears 2 times
  'the' appears 3 times
5 marks, average 78.2
```

18.2.3 Reading line by line with getline

`std::getline(in, line)` reads a whole line, including spaces, and discards the newline. It is the right tool for line-oriented files: logs, configuration files, CSV. A line can then be split further with a `std::istringstream`.

SETTINGS.INI text

```
# app settings
theme=dark
font size=14

language=en-IN
broken line
```

SCORES.TXT text

```
Asha 78 85 90
Ravi 66 72 81
```

LINE-BY-LINE PROCESSING

main.cpp

```
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>

int main() {
    std::ifstream cfg("settings.ini");
    if (!cfg) return 1;
    std::string line;
    int line_no = 0;
    while (std::getline(cfg, line)) {
        ++line_no;
        if (line.empty() || line[0] == '#') continue;         // skip blanks and comments
        auto eq = line.find('=');
        if (eq == std::string::npos) {
            std::cout << "line " << line_no << ": ignored (no '=')\n";
            continue;
        }
        std::string key = line.substr(0, eq), value = line.substr(eq + 1);
        std::cout << "line " << line_no << ": [" << key << "] = [" << value << "]\n";
    }

    std::ifstream data("scores.txt");
    for (std::string row; std::getline(data, row); ) {
        std::istringstream fields(row);                       // split one line
        std::string name;
        int a, b, c;
        fields >> name >> a >> b >> c;
        std::cout << name << " total " << a + b + c << '\n';
    }
    return 0;
}
```

OUTPUT

```
line 2: [theme] = [dark]
line 3: [font size] = [14]
line 5: [language] = [en-IN]
line 6: ignored (no '=')
Asha total 253
Ravi total 219
```

18.2.4 Detecting end of file correctly

A very common bug is `while (!in.eof())`. `eof()` becomes true only **after** a read has tried to go past the end. So the loop runs one extra time: the last read fails, the variable keeps its old value, and the last item is processed twice. The correct pattern puts the read operation in the condition.

WHY WHILE (!EOF()) IS WRONG

main.cpp

```cpp
#include <fstream>
#include <iostream>

int main() {
    std::ofstream("nums.txt") << "10\n20\n30\n";          // file ends with a newline

    std::ifstream a("nums.txt");
    int x = 0, total = 0;
    while (!a.eof()) {          // WRONG
        a >> x;                 // the 4th read fails, x keeps 30
        total += x;
    }
    std::cout << "wrong loop total: " << total << '\n';

    std::ifstream b("nums.txt");
    total = 0;
    while (b >> x) total += x;  // RIGHT: test the read itself
    std::cout << "right loop total: " << total << '\n';
    return 0;
}
```

OUTPUT

```
wrong loop total: 90
right loop total: 60
```

EOF() ONLY BECOMES TRUE AFTER A FAILED ATTEMPT TO READ PAST THE END

18.3 Binary files

18.3.1 write() and read()

Text files store numbers as characters ("1299.5" is six bytes). **Binary** files store the raw bytes of values as they are in memory: an int is always 4 bytes, a double 8 bytes. Binary I/O is faster, files are compact and every record has a fixed size (which makes random access easy), but the files are not human-readable and not portable between systems with different type sizes or byte order. Open with `std::ios::binary` and use `write(const char*, count)` and `read(char*, count)`; `reinterpret_cast<char*>` views an object as bytes, and `gcount()` tells how many bytes the last read actually got.

RAW BYTES WITH WRITE() AND READ()

main.cpp

```cpp
#include <fstream>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values = {7, 300, 65536, -1};
    {
        std::ofstream out("values.bin", std::ios::binary);
        out.write(reinterpret_cast<const char*>(values.data()), values.size() * sizeof(int));
        std::ofstream txt("values.txt");
        for (int v : values) txt << v << '\n';
    }
    std::ifstream bin("values.bin", std::ios::binary | std::ios::ate);
    std::ifstream txt("values.txt", std::ios::ate);
    std::cout << "binary file: " << bin.tellg() << " bytes, text file: " << txt.tellg() << " bytes\n";

    bin.seekg(0);
    std::vector<int> back(4);
    bin.read(reinterpret_cast<char*>(back.data()), back.size() * sizeof(int));
    std::cout << "read " << bin.gcount() << " bytes: ";
    for (int v : back) std::cout << v << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
binary file: 16 bytes, text file: 15 bytes
read 16 bytes: 7 300 65536 -1
```

Looking at the bytes with od (little-endian ints)

```
$ od -A d -t x1 values.bin
0000000 07 00 00 00 2c 01 00 00 00 00 01 00 ff ff ff ff
0000016
```

Each int takes 4 bytes, least significant byte first on x86 (little-endian): 7 is `07 00 00 00`, 300 = 0x12C is `2c 01 00 00`, 65536 = 0x10000 is `00 00 01 00`, and -1 is `ff ff ff ff`.

18.3.2 Saving and loading objects

An object can be written as raw bytes **only if** it is trivially copyable: built-in types and fixed-size arrays, no pointers, no `std::string` or `std::vector` members (those contain pointers to heap memory, and writing the pointer saves an address that is meaningless in the next run). For records, use fixed-size char arrays, or write each field yourself (**serialization**): for a string, write its length first and then its characters.

RAW RECORDS AND FIELD-BY-FIELD SERIALIZATION

main.cpp

```cpp
#include <cstring>
#include <fstream>
#include <iostream>
#include <string>
#include <type_traits>

struct Record {                    // fixed size, trivially copyable: safe to write as bytes
    int roll;
    char name[20];
    double gpa;
};
```

```cpp
static_assert(std::is_trivially_copyable_v<Record>);

struct Student {                      // contains std::string: must be serialized field by field
    int roll;
    std::string name;
};
void save(std::ostream& out, const Student& s) {
    out.write(reinterpret_cast<const char*>(&s.roll), sizeof s.roll);
    std::size_t len = s.name.size();
    out.write(reinterpret_cast<const char*>(&len), sizeof len);      // length first
    out.write(s.name.data(), static_cast<std::streamsize>(len));     // then the characters
}
bool load(std::istream& in, Student& s) {
    std::size_t len = 0;
    if (!in.read(reinterpret_cast<char*>(&s.roll), sizeof s.roll)) return false;
    in.read(reinterpret_cast<char*>(&len), sizeof len);
    s.name.resize(len);
    in.read(s.name.data(), static_cast<std::streamsize>(len));
    return static_cast<bool>(in);
}

int main() {
    Record r{101, "", 8.75};
    std::strncpy(r.name, "Asha Rao", sizeof r.name - 1);
    std::ofstream("rec.bin", std::ios::binary).write(reinterpret_cast<const char*>(&r), sizeof r);
    Record r2{};
    std::ifstream("rec.bin", std::ios::binary).read(reinterpret_cast<char*>(&r2), sizeof r2);
    std::cout << "Record (" << sizeof(Record) << " bytes): " << r2.roll << ' ' << r2.name << ' ' <<
r2.gpa << '\n';

    {
        std::ofstream out("students.bin", std::ios::binary);
        save(out, {1, "Ravi"});
        save(out, {2, "Meera Krishnan, a much longer name than twenty characters"});
    }
    std::ifstream in("students.bin", std::ios::binary);
    for (Student s; load(in, s); )
        std::cout << "Student " << s.roll << ": " << s.name << '\n';
    return 0;
}
```

OUTPUT

```
Record (32 bytes): 101 Asha Rao 8.75
Student 1: Ravi
Student 2: Meera Krishnan, a much longer name than twenty characters
```

WATCH OUT

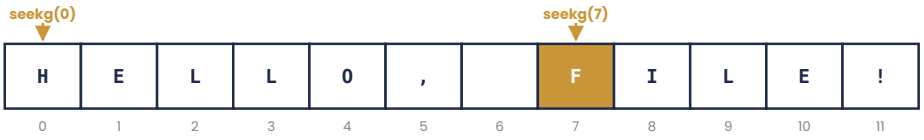
Binary files written this way depend on the compiler and platform: type sizes, padding and byte order. Use them for local caches and fixed-record databases; for files exchanged between programs or machines, prefer a text format (CSV, JSON) or a serialization library.

18.4 Random access

18.4.1 seekg(), seekp(), tellg(), tellp()

Every file stream has a **position**. For input it is the "get" position, read with `tellg()` and moved with `seekg()`; for output the "put" position, `tellp()` and `seekp()` (for file streams the two move together).

`seekg(pos)` jumps to an absolute byte offset; `seekg(offset, dir)` moves relative to `std::ios::beg`, `std::ios::cur` or `std::ios::end`.



BYTE POSITIONS IN A 12-BYTE FILE

MOVING AROUND IN A FILE main.cpp

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ofstream("greet.txt") << "HELLO, FILE!";
    std::fstream f("greet.txt", std::ios::in | std::ios::out);

    f.seekg(0, std::ios::end);
    std::cout << "file size: " << f.tellg() << " bytes\n";

    f.seekg(7);                                  // absolute position
    std::string word;
    f >> word;
    std::cout << "from byte 7: " << word << '\n';

    f.clear();                                   // reading hit the end: clear eof before seeking
    f.seekg(-5, std::ios::end);                  // 5 bytes before the end
    char c;
    f.get(c);
    std::cout << "5 bytes before end: '" << c << "', now at " << f.tellg() << '\n';

    f.seekp(0);                                  // overwrite the first 5 bytes in place
    f << "JELLO";
    f.seekg(0);
    std::getline(f, word);
    std::cout << "after overwrite: " << word << '\n';
    return 0;
}
```

OUTPUT

```
file size: 12 bytes
from byte 7: FILE!
5 bytes before end: 'F', now at 8
after overwrite: JELLO, FILE!
```

18.4.2 Updating a record in place

With fixed-size binary records, record *n* starts at byte `n * sizeof(Record)`. We can read or rewrite any record directly, without reading or copying the rest of the file. This is how simple databases and index files work.

EDITING RECORD 2 WITHOUT REWRITING THE FILE main.cpp

```
#include <cstring>
#include <fstream>
#include <iostream>

struct Account { int id; char owner[16]; double balance; };
```

```cpp
void print_all(std::fstream& f) {
    f.clear();
    f.seekg(0);
    Account a;
    while (f.read(reinterpret_cast<char*>(&a), sizeof a))
        std::cout << "  #" << a.id << ' ' << a.owner << ' ' << a.balance << '\n';
}

int main() {
    {
        std::ofstream out("accounts.dat", std::ios::binary);
        const char* owners[] = {"Asha", "Ravi", "Meera", "Kiran"};
        for (int i = 0; i < 4; ++i) {
            Account a{100 + i, "", 1000.0 * (i + 1)};
            std::strncpy(a.owner, owners[i], sizeof a.owner - 1);
            out.write(reinterpret_cast<const char*>(&a), sizeof a);
        }
    }
    std::fstream f("accounts.dat", std::ios::in | std::ios::out | std::ios::binary);
    std::cout << "record size " << sizeof(Account) << " bytes; before:\n";
    print_all(f);

    const int n = 2;                                    // update the third record (index 2)
    Account a;
    f.clear();
    f.seekg(n * sizeof(Account));
    f.read(reinterpret_cast<char*>(&a), sizeof a);
    a.balance += 750.25;                                // deposit
    f.seekp(n * sizeof(Account));                       // go back to the start of that record
    f.write(reinterpret_cast<const char*>(&a), sizeof a);    // overwrite only those bytes

    std::cout << "after depositing 750.25 into record " << n << ":\n";
    print_all(f);
    return 0;
}
```

OUTPUT

```
record size 32 bytes; before:
  #100 Asha 1000
  #101 Ravi 2000
  #102 Meera 3000
  #103 Kiran 4000
after depositing 750.25 into record 2:
  #100 Asha 1000
  #101 Ravi 2000
  #102 Meera 3750.25
  #103 Kiran 4000
```

FIXED-SIZE RECORDS: RECORD N STARTS AT N * SIZEOF(ACCOUNT)

18.5 Stream state and errors

18.5.1 good(), eof(), fail(), bad()

Every stream keeps a set of state flags. After a failure the stream stops working until you call `clear()`; that is why a failed read must be handled, not ignored.

Function	True when	Example
`good()`	no flag is set: everything fine so far	normal reading
`eof()`	a read tried to go past the end (eofbit)	after the last item was consumed and another read was tried
`fail()`	the last operation failed (failbit or badbit)	reading "abc" into an int; opening a missing file
`bad()`	a serious, usually unrecoverable error (badbit)	disk read error, stream buffer failure
`operator bool`	`!fail()`	`if (in)`, `while (in >> x)`
`clear()`	resets the flags so the stream can be used again	after skipping bad input, before seekg at EOF

VALUES.TXT (NOTE THE TYPO: LETTER O IN 40)

text

```
10 20 x 30
40 40
```

WATCHING THE STATE FLAGS

main.cpp

```cpp
#include <fstream>
#include <iostream>
#include <limits>
#include <string>

void state(const char* when, const std::istream& s) {
    std::cout << std::boolalpha << when << ": good=" << s.good() << " eof=" << s.eof()
              << " fail=" << s.fail() << " bad=" << s.bad() << '\n';
}

int main() {
    std::ofstream("mixed.txt") << "12 34 abc 56";
    std::ifstream in("mixed.txt");
    int x;
    in >> x;  state("read 12       ", in);
    in >> x;  state("read 34       ", in);
    in >> x;  state("read 'abc'    ", in);                 // conversion fails: failbit
    in.clear();                                            // recover ...
    in.ignore(std::numeric_limits<std::streamsize>::max(), ' ');   // ... and skip the bad token
    in >> x;  state("read 56       ", in);                 // reaches end while reading: eofbit too
    std::cout << "x = " << x << '\n';
    in >> x;  state("read past end ", in);

    std::ifstream sum_in("values.txt");
    int total = 0, skipped = 0;
    while (true) {
        int v;
        if (sum_in >> v) { total += v; continue; }
        if (sum_in.eof()) break;                           // normal end
        sum_in.clear();                                    // a bad token: skip it and go on
        std::string junk;
        sum_in >> junk;
        ++skipped;
    }
    std::cout << "total " << total << ", skipped " << skipped << " bad token(s)\n";
    return 0;
}
```

OUTPUT

```
read 12       : good=true eof=false fail=false bad=false
read 34       : good=true eof=false fail=false bad=false
read 'abc'    : good=false eof=false fail=true bad=false
read 56       : good=false eof=true fail=false bad=false
x = 56
read past end : good=false eof=true fail=true bad=false
total 104, skipped 2 bad token(s)
```

In `values.txt`, `x` cannot start a number, so that read fails and the token is skipped. `40` is subtler: `>>` happily reads the digit 4 and stops at the letter O, which then fails as the next token. Stream extraction reads as much as it can; it does not validate a whole token. To reject `40` completely, read tokens as strings and convert them with `std::stoi` plus a check that every character was used.

TIP

A stream can also be told to throw exceptions instead of setting flags quietly: `in.exceptions(std::ios::failbit | std::ios::badbit);` makes any failure throw `std::ios_base::failure`. Note that reaching the end with `while (in >> x)` also sets failbit, so this suits code that reads a known amount of data.

18.6 std::filesystem (C++17)

18.6.1 Paths

The header `<filesystem>` (namespace `std::filesystem`, usually aliased as `fs`) works with files and folders themselves. `fs::path` represents a path portably: it joins parts with `/`, and splits a path into its components. File streams accept a `path` directly.

TAKING A PATH APART main.cpp

```
#include <filesystem>
#include <iostream>
namespace fs = std::filesystem;

int main() {
    fs::path base = "/home/asha/projects";
    fs::path file = base / "report" / "summary.final.txt";      // operator/ joins parts
    std::cout << "full path     : " << file << '\n';
    std::cout << "root_path     : " << file.root_path() << '\n';
    std::cout << "parent_path   : " << file.parent_path() << '\n';
    std::cout << "filename      : " << file.filename() << '\n';
    std::cout << "stem          : " << file.stem() << '\n';
    std::cout << "extension     : " << file.extension() << '\n';
    std::cout << "is_absolute   : " << std::boolalpha << file.is_absolute() << '\n';
    fs::path copy = file;
    copy.replace_extension(".pdf");
    std::cout << "replaced ext  : " << copy.string() << '\n';      // .string(): without quotes
    std::cout << "parts         :";
    for (const auto& part : file) std::cout << " [" << part.string() << ']';
    std::cout << '\n';
    fs::path rel = "../data/./input.csv";
    std::cout << "normal form   : " << rel.lexically_normal() << '\n';
    return 0;
}
```

OUTPUT

```
full path     : "/home/asha/projects/report/summary.final.txt"
root_path     : "/"
parent_path   : "/home/asha/projects/report"
filename      : "summary.final.txt"
stem          : "summary.final"
extension     : ".txt"
is_absolute   : true
replaced ext  : /home/asha/projects/report/summary.final.pdf
parts         : [/] [home] [asha] [projects] [report] [summary.final.txt]
normal form   : "../data/input.csv"
```

NOTE

Printing a `path` with `<<` puts it in quotes (so paths with spaces are unambiguous); use `.string()` for the plain text. On Windows, paths use `\\` as the separator and `fs::path` handles both.

18.6.2 Checking existence, file size, creating and removing directories

The free functions in `std::filesystem` query and change the file system. Most throw `fs::filesystem_error` on failure; each also has an overload taking a `std::error_code&` that reports the error without throwing.

Function	Purpose
`exists(p)`, `is_regular_file(p)`, `is_directory(p)`	test what is at a path
`file_size(p)`	size in bytes of a regular file
`create_directory(p)` / `create_directories(p)`	make one folder / a whole chain of folders
`copy_file(a, b)`, `rename(a, b)`	copy, move or rename
`remove(p)` / `remove_all(p)`	delete a file or empty folder / a folder and everything inside (returns the count)
`current_path()`, `temp_directory_path()`	the working folder and the system temp folder
`last_write_time(p)`, `space(p)`	modification time and disk space

WORKING WITH FILES AND FOLDERS

main.cpp

```cpp
#include <filesystem>
#include <fstream>
#include <iostream>
#include <system_error>
namespace fs = std::filesystem;

int main() {
    fs::create_directories("backup/2026/oct");                // creates all three levels
    std::ofstream("backup/2026/oct/data.txt") << "important data\n";
    fs::path p = "backup/2026/oct/data.txt";
    std::cout << std::boolalpha;
    std::cout << "exists: " << fs::exists(p) << ", regular file: " << fs::is_regular_file(p)
              << ", directory: " << fs::is_directory(p.parent_path()) << '\n';
    std::cout << "size: " << fs::file_size(p) << " bytes\n";

    fs::copy_file(p, "backup/2026/oct/data_copy.txt");
    fs::rename("backup/2026/oct/data_copy.txt", "backup/2026/copy.txt");
    std::cout << "copy moved up: " << fs::exists("backup/2026/copy.txt") << '\n';

    try {
        fs::file_size("no_such_file.txt");                    // throwing version
    } catch (const fs::filesystem_error& e) {
        std::cout << "error: " << e.code().message() << " [" << e.path1().string() << "]\n";
    }
    std::error_code ec;
    fs::remove("backup", ec);                                 // non-throwing version: folder not
empty
    std::cout << "remove(\"backup\"): " << (ec ? ec.message() : "ok") << '\n';
    auto n = fs::remove_all("backup");
    std::cout << "remove_all deleted " << n << " entries; exists now: " << fs::exists("backup") <<
'\n';
    return 0;
}
```

OUTPUT

```
exists: true, regular file: true, directory: true
size: 15 bytes
copy moved up: true
error: No such file or directory [no_such_file.txt]
remove("backup"): Directory not empty
remove_all deleted 5 entries; exists now: false
```

18.6.3 Iterating over a directory

`fs::directory_iterator` lists the entries of one folder; `fs::recursive_directory_iterator` also walks into subfolders (its `depth()` tells how deep it is). Each element is a `fs::directory_entry` with `path()`, `is_regular_file()`, `is_directory()` and `file_size()`. The order of entries is **unspecified** (it is whatever the operating system returns), so sort them when the order matters.

LISTING A FOLDER AND ITS SUBFOLDERS main.cpp

```
#include <algorithm>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <vector>
namespace fs = std::filesystem;

int main() {
    fs::create_directories("project/src");
    fs::create_directories("project/docs");
    std::ofstream("project/README.md") << "# Project\n";
    std::ofstream("project/src/main.cpp") << "int main() { return 0; }\n";
    std::ofstream("project/src/util.cpp") << "// helpers\n";
    std::ofstream("project/docs/guide.txt") << "How to use the project\n";

    std::vector<fs::path> top;
    for (const fs::directory_entry& e : fs::directory_iterator("project"))
        top.push_back(e.path());
    std::sort(top.begin(), top.end());
    std::cout << "project/ contains:\n";
    for (const auto& p : top) std::cout << "  " << p.filename().string() << (fs::is_directory(p) ? "/"
: "") << '\n';

    std::vector<std::pair<fs::path, std::uintmax_t>> files;
    for (const auto& e : fs::recursive_directory_iterator("project"))
        if (e.is_regular_file()) files.emplace_back(e.path(), e.file_size());
    std::sort(files.begin(), files.end());
    std::cout << "all files, recursively:\n";
    for (const auto& [p, size] : files) std::cout << "  " << p.string() << " (" << size << "
bytes)\n";
    fs::remove_all("project");
    return 0;
}
```

OUTPUT

```
project/ contains:
  README.md
  docs/
  src/
all files, recursively:
  project/README.md (10 bytes)
  project/docs/guide.txt (23 bytes)
  project/src/main.cpp (25 bytes)
  project/src/util.cpp (11 bytes)
```

18.7 Common formats

18.7.1 Reading and writing CSV files

CSV (comma-separated values) is the simplest way to exchange tables with spreadsheets and other programs: one record per line, fields separated by commas, often with a header line. Write it with `<<` and

commas; read each line with `getline`, then split the line on commas with `getline(stream, field, ',')`. Real CSV may quote fields that contain commas (`"Rao, Asha"`); a small parser that understands quotes handles that.

WRITING AND PARSING CSV

main.cpp

```
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

std::vector<std::string> split_csv(const std::string& line) {      // handles "quoted, fields"
    std::vector<std::string> fields;
    std::string cur;
    bool quoted = false;
    for (char c : line) {
        if (c == '"') quoted = !quoted;
        else if (c == ',' && !quoted) { fields.push_back(cur); cur.clear(); }
        else cur += c;
    }
    fields.push_back(cur);
    return fields;
}

int main() {
    {
        std::ofstream out("products.csv");
        out << "id,name,price,stock\n";
        out << "1,USB cable,199.00,40\n";
        out << "2,\"Mouse, wireless\",649.50,12\n";
        out << "3,Keyboard,1249.00,0\n";
    }
    std::ifstream in("products.csv");
    std::string line;
    std::getline(in, line);                                  // skip the header
    double stock_value = 0;
    while (std::getline(in, line)) {
        auto f = split_csv(line);
        int id = std::stoi(f[0]);
        double price = std::stod(f[2]);
        int stock = std::stoi(f[3]);
        stock_value += price * stock;
        std::cout << id << ": " << f[1] << (stock == 0 ? "  (out of stock)" : "") << '\n';
    }
    std::cout << "total stock value: " << stock_value << '\n';

    std::istringstream simple("red,green,blue");             // the simple way, without quotes
    for (std::string field; std::getline(simple, field, ','); ) std::cout << '<' << field << '>';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
1: USB cable
2: Mouse, wireless
3: Keyboard  (out of stock)
total stock value: 15754
<red><green><blue>
```

18.7.2 JSON with a library (nlohmann/json, overview)

JSON is the standard text format for configuration files and web APIs: objects `{"key": value}`, arrays `[...]`, strings, numbers, booleans and null, nested freely. The standard library has no JSON support, so we use a library. The most popular is **nlohmann/json**: a single header `json.hpp`, installable with a package manager (`vcpkg install nlohmann-json`, `apt install nlohmann-json3-dev`) or by copying the file. A `json` object behaves like a mix of map, vector and value.

READING AND WRITING JSON WITH NLOHMANN/JSON main.cpp

```
#include <fstream>
#include <iostream>
#include <nlohmann/json.hpp>
using json = nlohmann::json;

int main() {
    json config = {
        {"app", "Library Manager"},
        {"version", 2},
        {"dark_mode", true},
        {"users", {{{"name", "Asha"}, {"role", "admin"}}, {{"name", "Ravi"}, {"role", "member"}}}}
    };
    config["max_books"] = 5;                                  // add a key like a map
    std::ofstream("config.json") << config.dump(2);           // pretty-print with 2-space indent

    std::ifstream in("config.json");
    json loaded = json::parse(in);                            // throws json::parse_error on bad input
    std::cout << loaded["app"].get<std::string>() << " v" << loaded["version"] << '\n';
    for (const auto& u : loaded["users"])
        std::cout << "  " << u["name"].get<std::string>() << " is " << u["role"].get<std::string>() <<
'\n';
    std::cout << "max_books = " << loaded.value("max_books", 3) << ", missing key with default: " <<
loaded.value("theme", "light") << '\n';
    std::cout << config.dump(2) << '\n';
    return 0;
}
```

OUTPUT

```
Library Manager v2
  Asha is admin
  Ravi is member
max_books = 5, missing key with default: light
{
  "app": "Library Manager",
  "dark_mode": true,
  "max_books": 5,
  "users": [
    {
      "name": "Asha",
      "role": "admin"
    },
    {
      "name": "Ravi",
      "role": "member"
    }
  ],
  "version": 2
}
```

Needs the nlohmann/json header (version 3.11): build with g++ -std=c++20 main.cpp after installing the library. The default json type stores object keys in a std::map, so dump() prints them in alphabetical order, not insertion order (use nlohmann::ordered_json to keep insertion order).

The library can also convert your own structs automatically: define `to_json` and `from_json` functions, or use the macro `NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE(Student, roll, name, gpa)`, after which `json j = student;` and `auto s = j.get<Student>();` just work.

Format	Readable	Structure	Best for
Plain text	yes	free-form	logs, simple notes
CSV	yes	flat table	spreadsheets, data export
JSON	yes	nested objects and arrays	configuration, web APIs, structured records
Binary	no	fixed records	speed, random access, local caches

Include <fstream> and use ifstream/ofstream/fstream exactly like cin and cout; always check that a file opened, choose the mode deliberately (app to add, out to replace), and loop with while (in >> x) or while (getline(in, line)), never while (!eof()). Binary files use write()/read() and allow random access with seekg/seekp, but only for trivially copyable data. std::filesystem handles paths, folders and directory listings; CSV and JSON are the everyday exchange formats.

PRACTICE CHECKPOINT

Extend the **Student Record program** to save records to a CSV file and load them on startup, and list all **.txt files in a folder** using std::filesystem.

SAMPLE SOLUTION

main.cpp

```
#include <algorithm>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
namespace fs = std::filesystem;

struct Student {
    int roll;
    std::string name;
    double gpa;
};

const fs::path DATA_FILE = "students.csv";

std::vector<Student> load_students() {
    std::vector<Student> list;
    std::ifstream in(DATA_FILE);
    if (!in) return list;                                   // first run: no file yet
    std::string line;
    std::getline(in, line);                                 // header
    while (std::getline(in, line)) {
        std::istringstream row(line);
        std::string roll, name, gpa;
        if (std::getline(row, roll, ',') && std::getline(row, name, ',') && std::getline(row, gpa))
            list.push_back({std::stoi(roll), name, std::stod(gpa)});
    }
    return list;
```

```
}

void save_students(const std::vector<Student>& list) {
    std::ofstream out(DATA_FILE);                          // replace the whole file
    out << "roll,name,gpa\n" << std::fixed << std::setprecision(2);
    for (const Student& s : list) out << s.roll << ',' << s.name << ',' << s.gpa << '\n';
}

void show(const std::vector<Student>& list) {
    for (const Student& s : list)
        std::cout << "  " << std::setw(3) << s.roll << "  " << std::left << std::setw(14) << s.name
                  << std::right << std::fixed << std::setprecision(2) << s.gpa << '\n';
}

void list_txt_files(const fs::path& folder) {
    std::vector<fs::path> found;
    for (const auto& entry : fs::directory_iterator(folder))
        if (entry.is_regular_file() && entry.path().extension() == ".txt")
            found.push_back(entry.path().filename());
    std::sort(found.begin(), found.end());
    std::cout << ".txt files in " << folder.string() << "/: " << found.size() << '\n';
    for (const auto& f : found) std::cout << "  " << f.string() << " (" << fs::file_size(folder / f)
<< " bytes)\n";
}

int main() {
    // --- first run of the program: no CSV yet ---
    auto students = load_students();
    std::cout << "run 1: loaded " << students.size() << " records\n";
    students.push_back({1, "Asha Rao", 8.7});
    students.push_back({2, "Ravi Kumar", 7.95});
    save_students(students);

    // --- second run: records are loaded on startup ---
    students = load_students();
    std::cout << "run 2: loaded " << students.size() << " records\n";
    show(students);
    students.push_back({3, "Meera Iyer", 9.1});
    save_students(students);
    std::cout << "students.csv now contains:\n";
    std::ifstream csv(DATA_FILE);
    for (std::string line; std::getline(csv, line); ) std::cout << "  | " << line << '\n';

    // --- list .txt files in a folder ---
    fs::create_directory("notes");
    std::ofstream("notes/todo.txt") << "submit assignment\n";
    std::ofstream("notes/ideas.txt") << "student portal\n";
    std::ofstream("notes/photo.jpg") << "not text";
    fs::create_directory("notes/old.txt");                 // a folder named like a file: skipped
    list_txt_files("notes");
    return 0;
}
```

OUTPUT

```
run 1: loaded 0 records
run 2: loaded 2 records
    1  Asha Rao      8.70
    2  Ravi Kumar    7.95
students.csv now contains:
  | roll,name,gpa
  | 1,Asha Rao,8.70
  | 2,Ravi Kumar,7.95
  | 3,Meera Iyer,9.10
.txt files in notes/: 2
  ideas.txt (15 bytes)
  todo.txt (18 bytes)
```

LEVEL 3

Advanced level

Outcome: master the STL, modern C++ (C++11 to C++26), metaprogramming, concurrency, professional tooling, data structures and algorithms, and be ready for interviews.

7 MODULES · 37 DAYS MINIMUM

Module 19 · Standard Template Library (STL)	**8 days**
Module 20 · Modern C++ (C++11, C++14, C++17)	**6 days**
Module 21 · C++20, C++23 and C++26 features	**4 days**
Module 22 · Advanced templates and metaprogramming	**3 days**
Module 23 · Concurrency and multithreading	**4 days**
Module 24 · Building, debugging, testing and best practices	**4 days**
Module 25 · Data structures, competitive programming, projects and interviews	**8 days**

MODULE 19 · ADVANCED

Standard Template Library (STL)

Minimum duration **8 days** Level **Advanced**

The Standard Template Library is the part of the C++ standard library built from templates: containers that hold data, iterators that walk through them, algorithms that work on any range of elements, and function objects that customise the algorithms. Knowing which container to choose and which algorithm to call replaces hundreds of lines of hand-written loops, and it is essential both for real projects and for competitive programming.

IN THIS MODULE

19.1 STL overview
19.2 Iterators
19.3 Sequence containers
19.4 Container adaptors
19.5 Associative containers
19.6 Unordered containers
19.7 Utility types
19.8 Algorithms
19.9 Function objects and comparators
19.10 Time complexity of STL operations

19.1 STL overview

19.1.1 Containers, iterators, algorithms and function objects

The STL is designed around a simple idea: **separate the data structures from the algorithms**. Containers store elements; algorithms process elements; **iterators** connect the two. An algorithm such as `std::sort` or `std::find` never knows which container it works on: it only receives a pair of iterators marking a range. So *M* containers and *N* algorithms need *M* + *N* pieces of code instead of *M* × *N*.

Containers

Hold the data: `vector`, `deque`, `list`, `array`, `set`, `map`, `unordered_map` … plus the adaptors `stack`, `queue`, `priority_queue`.

Iterators

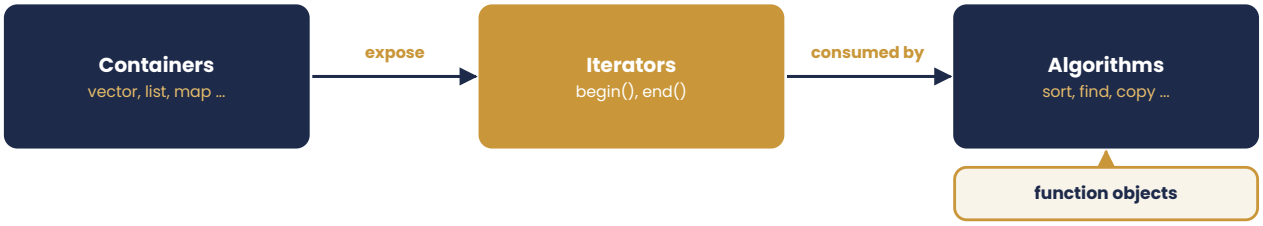
Generalised pointers: `*it` reads the element, `++it` moves to the next. `begin()` and `end()` give the range.

Algorithms

Over 100 functions in `<algorithm>` and `<numeric>`: sort, find, count, copy, transform, accumulate …

Function objects

Callables that customise algorithms: comparators for sort order, predicates for find_if, operations for transform. Lambdas are the usual form.



ALGORITHMS REACH CONTAINER ELEMENTS ONLY THROUGH ITERATORS

THE FOUR PARTS WORKING TOGETHER

main.cpp

```
#include <algorithm>
#include <iostream>
#include <list>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> v = {40, 10, 30, 20};        // container
    std::list<int>   l = {5, 15, 25};             // a different container

    std::sort(v.begin(), v.end());                // algorithm on an iterator range
    auto it = std::find(l.begin(), l.end(), 15);  // the same find works on vector, list, array ...
    int total = std::accumulate(v.begin(), v.end(), 0);
    int big = std::count_if(v.begin(), v.end(),
                            [](int x) { return x > 15; });   // function object (a lambda)

    for (int x : v) std::cout << x << ' ';
    std::cout << "| found 15? " << std::boolalpha << (it != l.end()) << " | total " << total << " |
>15: " << big << '\n';
    return 0;
}
```

OUTPUT

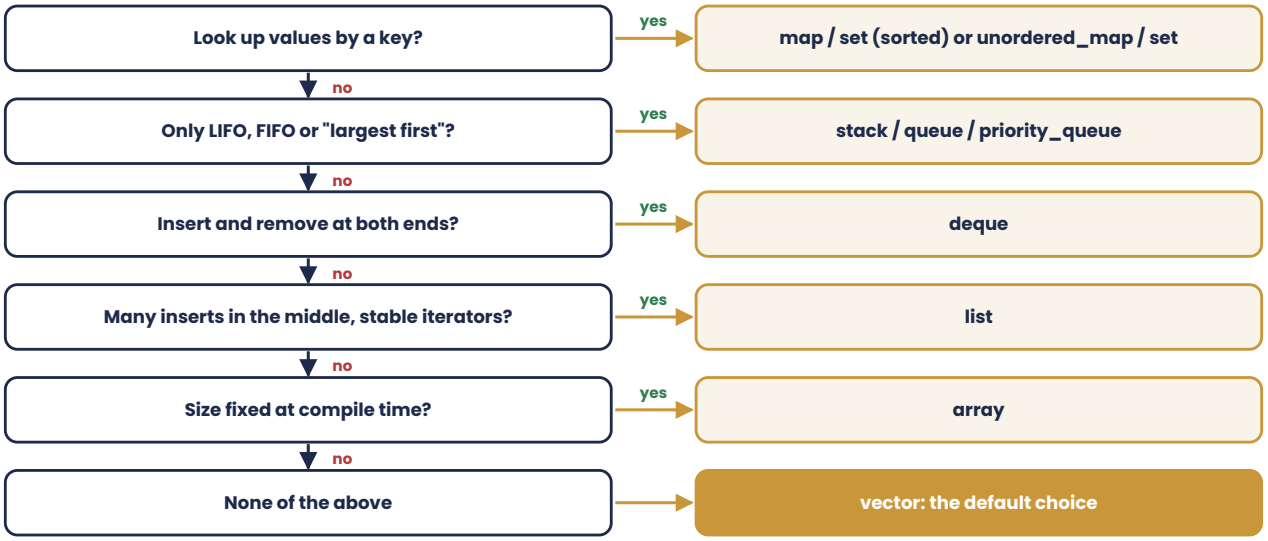
```
10 20 30 40 | found 15? true | total 100 | >15: 3
```

19.1.2 Choosing a container by complexity

Each container is a different data structure with different costs. The question to ask is: **which operations does my program do most often?** Access by position, search by key, insertion in the middle, insertion at the ends, keeping things sorted? The table gives the big-O cost of the common operations; section 19.10 has the complete reference.

Container	Structure	Access by index	Search	Insert/erase at end	Insert/erase in middle
`vector`	dynamic array	O(1)	O(n)	O(1) amortised	O(n)
`deque`	blocks of arrays	O(1)	O(n)	O(1) both ends	O(n)
`list`	doubly linked list	-	O(n)	O(1)	O(1) with an iterator
`set` / `map`	balanced tree (red-black)	-	O(log n)	O(log n)	O(log n)

Container	Structure	Access by index	Search	Insert/erase at end	Insert/erase in middle
`unordered_set` / `_map`	hash table	-	O(1) average	O(1) average	O(1) average



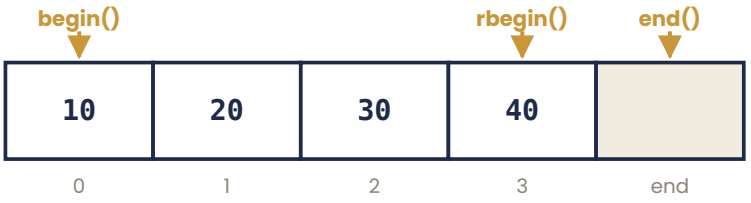
CHOOSING A CONTAINER: ASK THE QUESTIONS FROM THE TOP

When in doubt, use `std::vector`. Its elements sit next to each other in memory, which makes it cache-friendly and usually the fastest container in practice, even where big-O suggests otherwise.

19.2 Iterators

19.2.1 begin(), end(), rbegin(), rend()

Every container provides `begin()`, an iterator to the first element, and `end()`, an iterator **one past** the last element. The range is **half-open**, [begin, end): `end()` must never be dereferenced, and an empty container has `begin() == end()`. `rbegin()` / `rend()` walk backwards; `cbegin()` / `cend()` give const iterators that cannot modify elements. The free functions `std::begin(x)` / `std::end(x)` also work on built-in arrays.



END() POINTS ONE PAST THE LAST ELEMENT

WALKING A CONTAINER WITH ITERATORS main.cpp

```
#include <iostream>
#include <iterator>
#include <vector>

int main() {
    std::vector<int> v = {10, 20, 30, 40};
    for (auto it = v.begin(); it != v.end(); ++it) {   // classic iterator loop
        *it += 1;                                       // modify through the iterator
        std::cout << *it << ' ';
    }
    std::cout << '\n';
    for (auto it = v.rbegin(); it != v.rend(); ++it) std::cout << *it << ' ';   // backwards
    std::cout << '\n';
    for (auto it = v.cbegin(); it != v.cend(); ++it) std::cout << *it * 2 << ' '; // read-only
    std::cout << '\n';

    auto mid = v.begin() + 2;                           // random-access arithmetic
    std::cout << "element at begin()+2: " << *mid << ", distance from begin: " <<
std::distance(v.begin(), mid) << '\n';
    std::cout << "next(begin): " << *std::next(v.begin()) << ", prev(end): " << *std::prev(v.end()) <<
'\n';

    int raw[] = {3, 1, 2};
    std::cout << "array size via iterators: " << std::end(raw) - std::begin(raw) << '\n';
    return 0;
}
```

OUTPUT

```
11 21 31 41
41 31 21 11
22 42 62 82
element at begin()+2: 31, distance from begin: 2
next(begin): 21, prev(end): 41
array size via iterators: 3
```

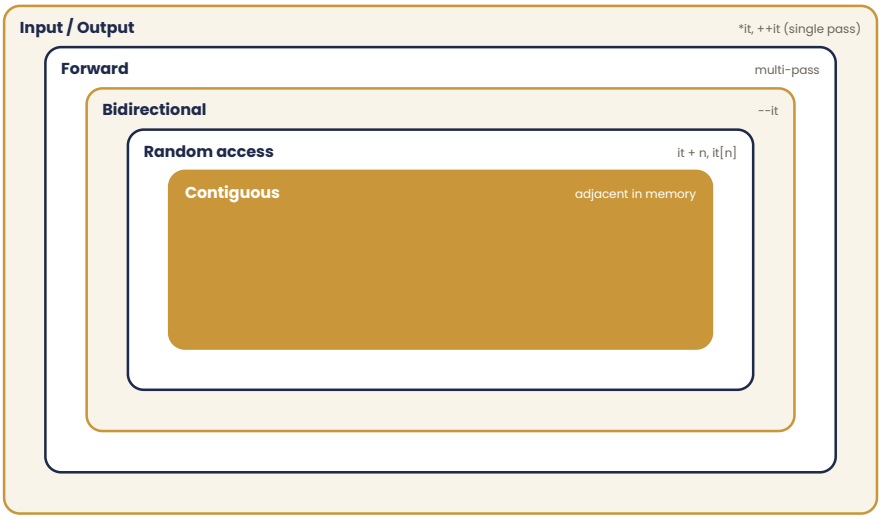
The range-based `for` loop is just shorthand for an iterator loop from `begin()` to `end()`. Use explicit iterators when you need the position itself: to erase, to insert, or to pass part of a range to an algorithm.

19.2.2 Iterator categories: input, output, forward, bidirectional, random access, contiguous

Not every iterator supports every operation. A list iterator can move forwards and backwards but cannot jump by 5; a vector iterator can do everything a pointer can. The standard groups iterators into **categories**, each adding abilities to the previous one. An algorithm states the category it needs: `std::sort` needs random access, so it does not compile with a `std::list` (which has its own `list::sort()` member).

Category	Adds	Example
Input	read once, move forward (`*it` , `++it`), single pass	`std::istream_iterator`
Output	write once, move forward	`std::ostream_iterator` , `std::back_inserter`
Forward	multi-pass reading and writing	`forward_list` , `unordered_map`
Bidirectional	`--it`	`list` , `set` , `map`

Category	Adds	Example
Random access	`it + n`, `it[n]`, `it2 - it1`, `<`	`deque`
Contiguous (C++20)	elements adjacent in memory	`vector`, `array`, `string`, raw pointers



EACH CATEGORY INCLUDES ALL THE ABILITIES OF THE OUTER ONES

ITERATORS OF DIFFERENT CATEGORIES main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <iterator>
#include <list>
#include <sstream>
#include <vector>

int main() {
    std::istringstream input("5 3 9 1");
    std::vector<int> v(std::istream_iterator<int>{input}, std::istream_iterator<int>{});  // input iterators
    std::sort(v.begin(), v.end());                                  // needs random access: ok for vector
    std::copy(v.begin(), v.end(), std::ostream_iterator<int>(std::cout, " "));   // output iterator
    std::cout << '\n';

    std::list<int> l = {5, 3, 9, 1};
    // std::sort(l.begin(), l.end());     error: list iterators are only bidirectional
    l.sort();                                                       // list has its own sort
    auto it = l.end();
    --it;                                                           // bidirectional: can go back
    std::cout << "list sorted, last = " << *it << '\n';
    auto lit = l.begin();
    std::advance(lit, 2);                                           // steps one by one: O(n) for lists
    std::cout << "third list element: " << *lit << '\n';

    std::vector<int> evens;
    std::copy_if(v.begin(), v.end(), std::back_inserter(evens), [](int x) { return x > 2; });  // inserter grows the vector
    std::cout << "copied " << evens.size() << " elements > 2\n";
    return 0;
}
```

OUTPUT

```
1 3 5 9
list sorted, last = 9
third list element: 5
copied 3 elements > 2
```

19.2.3 Iterator invalidation

An iterator (or pointer or reference) into a container becomes **invalid** when the container changes in a way that moves or destroys the element. Using an invalid iterator is undefined behaviour: it may print garbage, crash, or appear to work. The rules differ per container:

Container	Insert invalidates	Erase invalidates
`vector`, `string`	all, if the capacity grows (reallocation); otherwise those at and after the insert point	those at and after the erased element
`deque`	all iterators (references stay valid when inserting at the ends)	all, unless erasing at an end
`list`, `forward_list`	none	only the erased element
`set`, `map`	none	only the erased element

Container	Insert invalidates	Erase invalidates
`unordered_*`	all iterators if a rehash happens; references stay valid	only the erased element

INVALIDATION IN PRACTICE

main.cpp

```cpp
#include <iostream>
#include <list>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3};
    std::cout << "capacity " << v.capacity() << '\n';
    int* first = &v[0];
    v.push_back(4);                                  // capacity was 3: reallocation, elements move
    std::cout << "first element moved? " << std::boolalpha << (first != &v[0]) << '\n';
    // *first is now a dangling pointer: reading it would be undefined behaviour

    // Correct way to erase while iterating: use the iterator returned by erase()
    std::vector<int> nums = {1, 2, 3, 4, 5, 6};
    for (auto it = nums.begin(); it != nums.end(); ) {
        if (*it % 2 == 0) it = nums.erase(it);       // erase returns the next valid iterator
        else ++it;
    }
    for (int x : nums) std::cout << x << ' ';
    std::cout << '\n';

    std::list<int> l = {10, 20, 30};
    auto keep = std::next(l.begin());                // points to 20
    l.push_front(5);
    l.erase(l.begin());
    l.push_back(40);
    std::cout << "list iterator still valid: " << *keep << '\n';
    return 0;
}
```

OUTPUT

```
capacity 3
first element moved? true
1 3 5
list iterator still valid: 20
```

WATCH OUT

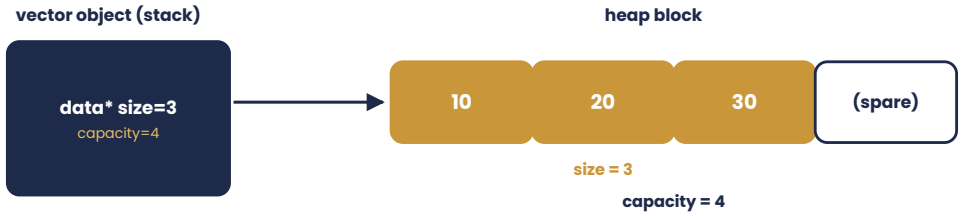
The classic bug: `for (auto it = v.begin(); it != v.end(); ++it) if (bad(*it)) v.erase(it);`. After `erase`, `it` is invalid and `++it` is undefined behaviour. Use `it = v.erase(it)` as above, or better, the erase-remove idiom / `std::erase_if` (19.3.1.3).

19.3 Sequence containers

Sequence containers keep elements in the order you put them in. They differ in how they store elements and which operations are cheap.

19.3.1 vector in depth

`std::vector<T>` is a dynamic array: the elements live in one contiguous heap block. It tracks two numbers: **size**, the number of elements, and **capacity**, the number of elements that fit before it has to allocate a bigger block.



A VECTOR: A SMALL OBJECT POINTING TO A HEAP ARRAY WITH SPARE ROOM

19.3.1.1 Capacity, reserve(), shrink_to_fit()

When `push_back` finds no spare room, the vector allocates a larger block (GCC and Clang double the capacity; MSVC grows it by 1.5), moves the elements across and frees the old block. Because growth is geometric, the total copying cost is spread out and `push_back` is **O(1) amortised**. If you know the final size, `reserve(n)` allocates once up front; `shrink_to_fit()` asks the vector to release unused capacity (a non-binding request); `clear()` removes the elements but keeps the capacity.

WATCHING THE CAPACITY GROW

main.cpp

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    std::size_t last_cap = v.capacity();
    std::cout << "size 0, capacity " << last_cap << '\n';
    for (int i = 1; i <= 33; ++i) {
        v.push_back(i);
        if (v.capacity() != last_cap) {                // report each reallocation
            last_cap = v.capacity();
            std::cout << "size " << v.size() << " -> capacity grows to " << last_cap << '\n';
        }
    }
    std::vector<int> r;
    r.reserve(1000);                                   // one allocation up front
    for (int i = 0; i < 1000; ++i) r.push_back(i);
    std::cout << "with reserve: size " << r.size() << ", capacity " << r.capacity() << '\n';

    r.resize(10);                                      // size shrinks, capacity does not
    std::cout << "after resize(10): size " << r.size() << ", capacity " << r.capacity() << '\n';
    r.shrink_to_fit();
    std::cout << "after shrink_to_fit: capacity " << r.capacity() << '\n';
    r.clear();
    std::cout << "after clear: size " << r.size() << ", capacity " << r.capacity() << ", empty " <<
r.empty() << '\n';
    return 0;
}
```

OUTPUT

```
size 0, capacity 0
size 1 -> capacity grows to 1
size 2 -> capacity grows to 2
size 3 -> capacity grows to 4
size 5 -> capacity grows to 8
size 9 -> capacity grows to 16
size 17 -> capacity grows to 32
size 33 -> capacity grows to 64
with reserve: size 1000, capacity 1000
after resize(10): size 10, capacity 1000
after shrink_to_fit: capacity 10
after clear: size 0, capacity 10, empty 1
```

Function	Effect
`size()` / `capacity()`	elements stored / room before reallocating
`reserve(n)`	make capacity at least n; size unchanged
`resize(n)` / `resize(n, value)`	change the size: add value-initialised (or given) elements, or destroy extras
`shrink_to_fit()`	request capacity == size
`clear()`	size 0, capacity kept
`front()`, `back()`, `data()`	first, last element, pointer to the array (for C APIs)
`at(i)` vs `[i]`	`at` checks the index and throws `std::out_of_range`; `[]` does not check

19.3.1.2 insert(), erase(), emplace_back()

`insert(pos, value)` puts an element before the iterator `pos`, shifting everything after it one place right: O(n) unless at the end. `erase(pos)` or `erase(first, last)` removes elements and shifts the rest left. `emplace_back(args...)` constructs the element **in place** from constructor arguments, instead of constructing a temporary and moving it in as `push_back(T(args...))` does.

INSERT, ERASE AND EMPLACE_BACK

main.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

struct Book {
    std::string title;
    int year;
    Book(std::string t, int y) : title(std::move(t)), year(y) { std::cout << "  construct " << title
<< '\n'; }
    Book(const Book& b) : title(b.title), year(b.year) { std::cout << "  copy " << title << '\n'; }
    Book(Book&& b) noexcept : title(std::move(b.title)), year(b.year) { std::cout << "  move " <<
title << '\n'; }
};

void show(const std::vector<int>& v) {
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';
}

int main() {
    std::vector<int> v = {10, 20, 30, 40, 50};
    v.insert(v.begin() + 1, 15);                    // before index 1
    show(v);
    v.insert(v.end(), {60, 70});                    // several at once
    show(v);
    v.insert(v.begin(), 2, 0);                      // two zeros at the front
    show(v);
    v.erase(v.begin() + 3);                         // remove one element (20)
    show(v);
    v.erase(v.begin(), v.begin() + 2);              // remove a range [first, last)
    show(v);

    std::vector<Book> shelf;
    shelf.reserve(4);                               // avoid reallocation noise
    std::cout << "push_back(Book(...)):\n";
    shelf.push_back(Book("Dune", 1965));            // temporary + move
    std::cout << "emplace_back(...):\n";
    shelf.emplace_back("Emma", 1815);               // built directly inside the vector
    return 0;
}
```

OUTPUT

```
10 15 20 30 40 50
10 15 20 30 40 50 60 70
0 0 10 15 20 30 40 50 60 70
0 0 10 20 30 40 50 60 70
10 20 30 40 50 60 70
push_back(Book(...)):
  construct Dune
  move Dune
emplace_back(...):
  construct Emma
```

19.3.1.3 Erase-remove idiom and std::erase (C++20)

To remove all elements that match a value or a condition, calling `erase` in a loop is $O(n^2)$. The algorithm `std::remove` (or `remove_if`) instead moves the elements to keep to the front in one pass and returns an iterator to the new logical end; it cannot change the container's size, because algorithms only see iterators. A following `erase` chops off the leftover tail. This two-step pattern is the **erase-remove idiom**.

C++20 wraps it in `std::erase(container, value)` and `std::erase_if(container, pred)`, which work for every standard container.

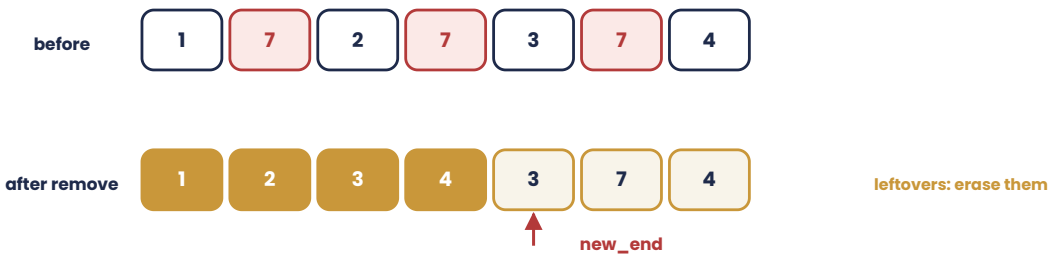
REMOVING ELEMENTS EFFICIENTLY

main.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

void show(const char* label, const std::vector<int>& v) {
    std::cout << label;
    for (int x : v) std::cout << x << ' ';
    std::cout << "(size " << v.size() << ")\n";
}

int main() {
    std::vector<int> v = {1, 7, 2, 7, 3, 7, 4};
    auto new_end = std::remove(v.begin(), v.end(), 7);         // step 1: compact
    std::cout << "after remove, logical size " << (new_end - v.begin()) << ", physical size " <<
v.size() << '\n';
    v.erase(new_end, v.end());                                 // step 2: erase the tail
    show("erase-remove: ", v);

    std::vector<int> w = {5, 12, 8, 21, 3, 14};
    w.erase(std::remove_if(w.begin(), w.end(), [](int x) { return x % 2 == 0; }), w.end());
    show("odd only:     ", w);

    std::vector<int> c20 = {1, 7, 2, 7, 3};
    auto removed = std::erase(c20, 7);                         // C++20: one call
    std::cout << "std::erase removed " << removed << " element(s): ";
    show("", c20);
    std::string s = "b-a-n-a-n-a";
    std::erase_if(s, [](char ch) { return ch == '-'; });
    std::cout << "std::erase_if on a string: " << s << '\n';
    return 0;
}
```

OUTPUT

```
after remove, logical size 4, physical size 7
erase-remove: 1 2 3 4 (size 4)
odd only:     5 21 3 (size 3)
std::erase removed 2 element(s): 1 2 3 (size 3)
std::erase_if on a string: banana
```

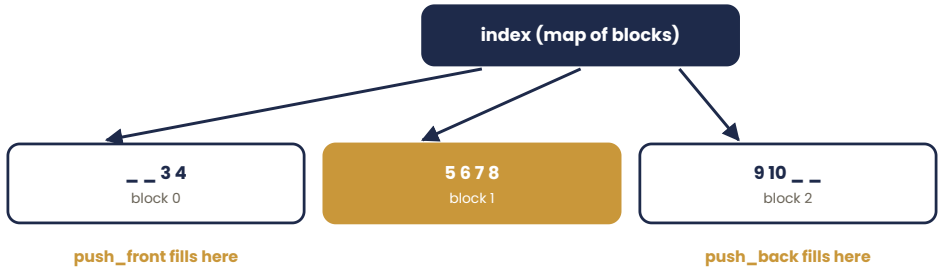


STD::REMOVE COMPACTS THE KEPT ELEMENTS; THE TAIL STILL HOLDS LEFTOVER VALUES

(The values left in the tail are unspecified; GCC leaves whatever was there. Only the part before `new_end` is meaningful.)

19.3.2 deque

`std::deque` (double-ended queue, pronounced "deck") supports fast insertion and removal at **both** ends, plus indexed access. Internally it is a sequence of fixed-size blocks with an index of block pointers, so growing at the front does not move existing elements. Elements are not contiguous, so `data()` does not exist and it is a little slower than vector for plain iteration.



A DEQUE STORES ELEMENTS IN BLOCKS; BOTH ENDS HAVE ROOM TO GROW

DEQUE AT BOTH ENDS main.cpp

```cpp
#include <deque>
#include <iostream>

int main() {
    std::deque<int> d = {3, 4, 5};
    d.push_front(2);
    d.push_front(1);
    d.push_back(6);
    std::cout << "deque: ";
    for (int x : d) std::cout << x << ' ';
    std::cout << "| d[2] = " << d[2] << ", front " << d.front() << ", back " << d.back() << '\n';
    d.pop_front();
    d.pop_back();
    std::cout << "after pop_front and pop_back: ";
    for (int x : d) std::cout << x << ' ';
    std::cout << '\n';

    std::deque<int> window;                          // sliding window of the last 3 readings
    for (int reading : {12, 15, 11, 18, 20, 9}) {
        window.push_back(reading);
        if (window.size() > 3) window.pop_front();
        int sum = 0;
        for (int x : window) sum += x;
        std::cout << "reading " << reading << " -> moving average " << sum / static_cast<double>
(window.size()) << '\n';
    }
    return 0;
}
```

OUTPUT

```
deque: 1 2 3 4 5 6 | d[2] = 3, front 1, back 6
after pop_front and pop_back: 2 3 4 5
reading 12 -> moving average 12
reading 15 -> moving average 13.5
reading 11 -> moving average 12.6667
reading 18 -> moving average 14.6667
reading 20 -> moving average 16.3333
reading 9 -> moving average 15.6667
```

19.3.3 list and forward_list

`std::list` is a **doubly linked list**: every element lives in its own node with pointers to the previous and next node. Inserting or erasing anywhere is O(1) once you have an iterator, and iterators to other elements are never invalidated. In exchange there is no indexing, every node is a separate allocation, and walking the list is slow because nodes are scattered in memory. `std::forward_list` is a **singly linked list**: one pointer per node, forward-only iteration, and operations named `insert_after` / `erase_after`. Lists have special member operations that relink nodes without copying: `splice`, `merge`, `sort`, `unique`, `reverse`, `remove_if`.

EACH LIST ELEMENT SITS IN ITS OWN NODE (LIST NODES ALSO POINT BACK TO THE PREVIOUS NODE)

LIST AND FORWARD_LIST OPERATIONS main.cpp

```
#include <forward_list>
#include <iostream>
#include <iterator>
#include <list>
#include <string>

template <typename C>
void show(const char* label, const C& c) {
    std::cout << label;
    for (const auto& x : c) std::cout << x << ' ';
    std::cout << '\n';
}

int main() {
    std::list<std::string> playlist = {"Intro", "Song A", "Song C"};
    auto pos = std::next(playlist.begin(), 2);                // points to "Song C"
    playlist.insert(pos, "Song B");                           // O(1): just relinks pointers
    playlist.push_front("Jingle");
    show("playlist: ", playlist);
    std::cout << "pos still refers to: " << *pos << '\n';

    std::list<int> a = {1, 4, 9}, b = {2, 3, 10};
    a.merge(b);                                               // both sorted: merges by relinking
    show("merged: ", a);
    std::cout << "b is now empty: " << std::boolalpha << b.empty() << '\n';
    std::list<int> extra = {100, 200};
    a.splice(a.begin(), extra);                               // move all nodes of extra to the front
    show("after splice: ", a);
    a.remove_if([](int x) { return x > 50; });
    a.reverse();
    show("filtered and reversed: ", a);

    std::forward_list<int> f = {3, 1, 2};
    f.push_front(0);
    f.insert_after(f.begin(), 9);                            // after the first element
    f.sort();
    show("forward_list: ", f);
    std::cout << "sizeof(list<int>::iterator) = " << sizeof(std::list<int>::iterator)
              << ", nodes per element: list 2 pointers, forward_list 1 pointer\n";
    return 0;
}
```

OUTPUT

```
playlist: Jingle Intro Song A Song B Song C
pos still refers to: Song C
merged: 1 2 3 4 9 10
b is now empty: true
after splice: 100 200 1 2 3 4 9 10
filtered and reversed: 10 9 4 3 2 1
forward_list: 0 1 2 3 9
sizeof(list<int>::iterator) = 8, nodes per element: list 2 pointers, forward_list 1 pointer
```

19.3.4 array

`std::array<T, N>` is a fixed-size array with a container interface: `size()`, iterators, `at()`, `fill()`, comparison operators and assignment. The elements live inside the object itself (on the stack for a local variable), with no heap allocation, and unlike a built-in array it does not decay to a pointer, can be returned from functions and copied by assignment.

STD::ARRAY

main.cpp

```cpp
#include <algorithm>
#include <array>
#include <iostream>

std::array<int, 3> rgb(int hex) {                          // arrays can be returned by value
    return {(hex >> 16) & 0xFF, (hex >> 8) & 0xFF, hex & 0xFF};
}

int main() {
    std::array<int, 5> marks = {72, 95, 60, 88, 79};
    std::sort(marks.begin(), marks.end());
    std::cout << "sorted: ";
    for (int m : marks) std::cout << m << ' ';
    std::cout << "| size " << marks.size() << ", sizeof " << sizeof(marks) << " bytes\n";

    std::array<int, 5> copy = marks;                       // copy by assignment
    copy.fill(0);
    std::cout << "copy after fill(0): " << copy[0] << ' ' << copy[4] << ", original unchanged: " <<
marks[0] << '\n';
    std::cout << std::boolalpha << "marks == copy? " << (marks == copy) << '\n';

    auto [r, g, b] = rgb(0xC9973A);                        // the LBF gold colour
    std::cout << "gold = rgb(" << r << ", " << g << ", " << b << ")\n";
    try {
        marks.at(5) = 1;
    } catch (const std::out_of_range& e) {
        std::cout << "at(5): " << e.what() << '\n';
    }
    return 0;
}
```

OUTPUT

```
sorted: 60 72 79 88 95 | size 5, sizeof 20 bytes
copy after fill(0): 0 0, original unchanged: 60
marks == copy? false
gold = rgb(201, 151, 58)
at(5): array::at: __n (which is 5) >= _Nm (which is 5)
```

19.4 Container adaptors

An **adaptor** wraps another container and restricts its interface to one access pattern. There are no iterators: you can only reach the element at the "top" or "front". Each adaptor has a default underlying container that you can change with a second template argument.

19.4.1 stack

`std::stack<T>` is last-in, first-out (LIFO): `push` adds on top, `top` reads it, `pop` removes it (and returns nothing). Default container: `deque`. Typical uses: undo history, matching brackets, depth-first search, evaluating expressions.

STACK: BROWSER HISTORY AND BRACKET MATCHING main.cpp

```
#include <iostream>
#include <stack>
#include <string>

bool balanced(const std::string& s) {
    std::stack<char> open;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') open.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (open.empty()) return false;
            char o = open.top();
            open.pop();
            if ((c == ')' && o != '(') || (c == ']' && o != '[') || (c == '}' && o != '{')) return
false;
        }
    }
    return open.empty();
}

int main() {
    std::stack<std::string> history;
    for (const char* page : {"home", "courses", "cpp", "module-19"}) history.push(page);
    std::cout << "current: " << history.top() << " (" << history.size() << " pages)\n";
    history.pop();
    history.pop();
    std::cout << "after going back twice: " << history.top() << '\n';
    for (const char* expr : {"{a[i] + (b * c)}", "(a + b]", "((x)"})
        std::cout << expr << " -> " << (balanced(expr) ? "balanced" : "NOT balanced") << '\n';
    return 0;
}
```

OUTPUT

```
current: module-19 (4 pages)
after going back twice: courses
{a[i] + (b * c)} -> balanced
(a + b] -> NOT balanced
((x) -> NOT balanced
```

19.4.2 queue

`std::queue<T>` is first-in, first-out (FIFO): `push` at the back, `front` and `pop` at the front, plus `back()`. Default container: `deque`. Typical uses: task queues, breadth-first search, simulations of a line of customers.

QUEUE: PRINT JOBS AND ROUND-ROBIN main.cpp

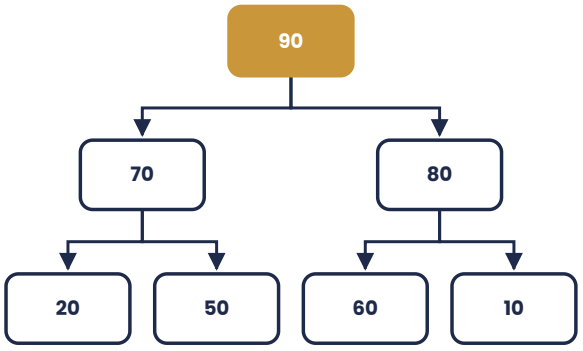
```cpp
#include <iostream>
#include <queue>
#include <string>

int main() {
    std::queue<std::string> printer;
    printer.push("report.pdf");
    printer.push("photo.png");
    printer.push("notes.txt");
    std::cout << printer.size() << " jobs, first " << printer.front() << ", last " << printer.back()
<< '\n';
    while (!printer.empty()) {
        std::cout << "printing " << printer.front() << '\n';
        printer.pop();
    }

    // Round-robin scheduling: each task gets 2 units of time, then goes to the back
    std::queue<std::pair<char, int>> ready;                // (task, time left)
    ready.push({'A', 5});
    ready.push({'B', 2});
    ready.push({'C', 3});
    int clock = 0;
    while (!ready.empty()) {
        auto [task, left] = ready.front();
        ready.pop();
        int run = left < 2 ? left : 2;
        clock += run;
        if (left - run > 0) ready.push({task, left - run});
        else std::cout << "task " << task << " finished at t=" << clock << '\n';
    }
    return 0;
}
```

OUTPUT

```
3 jobs, first report.pdf, last notes.txt
printing report.pdf
printing photo.png
printing notes.txt
task B finished at t=4
task C finished at t=9
task A finished at t=10
```

19.4.3 priority_queue (max-heap, min-heap with greater, custom comparators)

`std::priority_queue<T>` always gives access to the **largest** element: `top()` is O(1), `push` and `pop` are O(log n). It is a **binary heap** stored in a `vector`. For a **min-heap**, pass `std::greater<T>` as the comparator (the third template argument, so the container must be named too). For your own types, pass a comparator that returns true when the first argument has **lower** priority.



A MAX-HEAP: EVERY PARENT IS AT LEAST AS LARGE AS ITS CHILDREN; THE TOP IS THE MAXIMUM

MAX-HEAP, MIN-HEAP AND CUSTOM PRIORITIES

main.cpp

```cpp
#include <functional>
#include <iostream>
#include <queue>
#include <string>
#include <vector>

struct Patient {
    std::string name;
    int severity;          // higher = more urgent
    int arrival;           // earlier = first among equals
};
struct LowerPriority {     // returns true if a should be served AFTER b
    bool operator()(const Patient& a, const Patient& b) const {
        if (a.severity != b.severity) return a.severity < b.severity;
        return a.arrival > b.arrival;
    }
};

int main() {
    std::priority_queue<int> maxh;
    for (int x : {30, 90, 10, 70, 50}) maxh.push(x);
    std::cout << "max-heap pops: ";
    while (!maxh.empty()) { std::cout << maxh.top() << ' '; maxh.pop(); }
    std::cout << '\n';

    std::priority_queue<int, std::vector<int>, std::greater<int>> minh;   // min-heap
    for (int x : {30, 90, 10, 70, 50}) minh.push(x);
    std::cout << "min-heap pops: ";
    while (!minh.empty()) { std::cout << minh.top() << ' '; minh.pop(); }
    std::cout << '\n';

    std::priority_queue<Patient, std::vector<Patient>, LowerPriority> er;
    er.push({"Ravi", 2, 1});
    er.push({"Asha", 5, 2});
    er.push({"Kiran", 3, 3});
    er.push({"Meera", 5, 4});
    std::cout << "emergency room order:";
    while (!er.empty()) { std::cout << ' ' << er.top().name; er.pop(); }
    std::cout << '\n';

    auto by_len = [](const std::string& a, const std::string& b) { return a.size() > b.size(); };
    std::priority_queue<std::string, std::vector<std::string>, decltype(by_len)> shortest(by_len);
    for (const char* w : {"banana", "fig", "cherry", "kiwi"}) shortest.push(w);
    std::cout << "shortest word first: " << shortest.top() << '\n';
    return 0;
}
```

OUTPUT

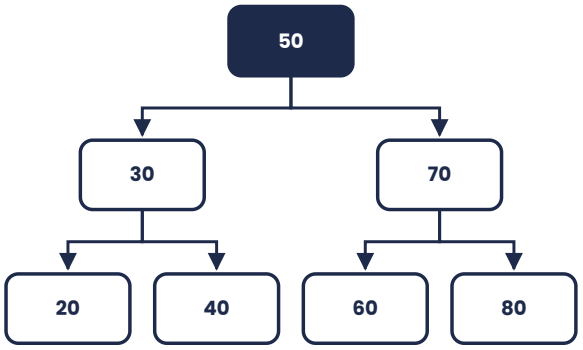
```
max-heap pops: 90 70 50 30 10
min-heap pops: 10 30 50 70 90
emergency room order: Asha Meera Kiran Ravi
shortest word first: fig
```

WATCH OUT

The comparator works "backwards" compared with sorting: `std::less` (the default) gives a **max**-heap, and `std::greater` a **min**-heap. Think of the comparator as "is a less important than b?".

19.5 Associative containers

Associative containers keep their elements **sorted by key** in a balanced binary search tree (a red-black tree in all major implementations). Search, insert and erase are O(log n), iteration visits keys in ascending order, and range queries ("all keys between 10 and 20") are efficient. Keys need `operator<` or a custom comparator.



A BALANCED SEARCH TREE: IN-ORDER TRAVERSAL GIVES 20 30 40 50 60 70 80

19.5.1 set and multiset

`std::set<T>` stores **unique** keys in sorted order: inserting a duplicate does nothing (`insert` returns a pair whose `.second` is false). `std::multiset` allows duplicates. Elements are const: to change one, erase it and insert the new value.

SET AND MULTISET

main.cpp

```cpp
#include <iostream>
#include <set>
#include <string>

int main() {
    std::set<int> s = {50, 20, 70, 20, 40};            // duplicate 20 ignored
    auto [it, inserted] = s.insert(30);
    auto [it2, inserted2] = s.insert(50);
    std::cout << std::boolalpha << "insert 30: " << inserted << ", insert 50 again: " << inserted2 <<
'\n';
    std::cout << "set: ";
    for (int x : s) std::cout << x << ' ';               // always sorted
    std::cout << "| size " << s.size() << '\n';
    s.erase(40);
    std::cout << "smallest " << *s.begin() << ", largest " << *s.rbegin() << '\n';

    std::string text = "the quick brown fox jumps over the lazy dog";
    std::set<char> letters(text.begin(), text.end());
    letters.erase(' ');
    std::cout << "distinct letters: " << letters.size() << " (a pangram uses all 26)\n";

    std::multiset<int> grades = {7, 9, 7, 8, 9, 7};
    std::cout << "multiset: ";
    for (int g : grades) std::cout << g << ' ';
    std::cout << "| count(7) = " << grades.count(7) << '\n';
    grades.erase(grades.find(7));                        // erase ONE 7
    std::cout << "after erasing one 7: count(7) = " << grades.count(7) << '\n';
    grades.erase(9);                                     // erase ALL 9s
    std::cout << "after erase(9): size " << grades.size() << '\n';
    return 0;
}
```

OUTPUT

```
insert 30: true, insert 50 again: false
set: 20 30 40 50 70 | size 5
smallest 20, largest 70
distinct letters: 26 (a pangram uses all 26)
multiset: 7 7 7 8 9 9 | count(7) = 3
after erasing one 7: count(7) = 2
after erase(9): size 3
```

19.5.2 map and multimap

`std::map<Key, Value>` stores key-value pairs (`std::pair<const Key, Value>`) sorted by unique key. `m[key]` returns a reference to the value, **inserting a default-constructed value if the key is missing**; `at(key)` throws if missing; `find` and `contains` look without inserting. `insert` and `try_emplace` do not overwrite an existing value; `insert_or_assign` does. `std::multimap` allows several values per key.

MAP AND MULTIMAP

main.cpp

```cpp
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> stock = {{"pen", 40}, {"notebook", 15}};
    stock["eraser"] = 25;                                   // insert
    stock["pen"] -= 5;                                      // update
    stock.insert({"pen", 999});                             // key exists: NOT overwritten
    stock.insert_or_assign("ruler", 12);
    for (const auto& [item, qty] : stock) std::cout << item << ": " << qty << '\n';   // sorted by key

    std::cout << "stapler via []: " << stock["stapler"] << " (oops, size is now " << stock.size() <<
")\n";
    if (auto it = stock.find("glue"); it == stock.end()) std::cout << "glue not stocked (find does not
insert)\n";
    try {
        stock.at("scissors");
    } catch (const std::out_of_range&) {
        std::cout << "at(\"scissors\") throws out_of_range\n";
    }
    stock.erase("stapler");

    std::multimap<std::string, std::string> phone = {{"Asha", "98450-11111"}, {"Ravi", "99000-22222"},
{"Asha", "080-2233-4455"}};
    auto [first, last] = phone.equal_range("Asha");
    std::cout << "Asha's numbers:";
    for (auto it = first; it != last; ++it) std::cout << ' ' << it->second;
    std::cout << '\n';
    return 0;
}
```

OUTPUT

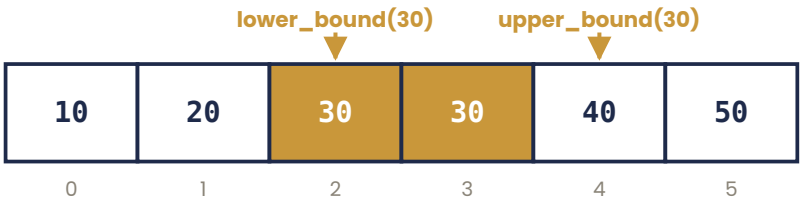
```
eraser: 25
notebook: 15
pen: 35
ruler: 12
stapler via []: 0 (oops, size is now 5)
glue not stocked (find does not insert)
at("scissors") throws out_of_range
Asha's numbers: 98450-11111 080-2233-4455
```

WATCH OUT

`operator[]` on a map silently inserts missing keys. Do not use it just to *check* whether a key exists, and it cannot be used on a `const` map. Use `find`, `contains` or `count` for lookups.

19.5.3 lower_bound(), upper_bound(), find(), count(), contains() (C++20)

Because the keys are sorted, ordered containers answer range questions in O(log n). `lower_bound(k)` returns the first element **not less than** k (≥ k); `upper_bound(k)` the first element **greater than** k. Together they bound the elements equal to k (`equal_range` returns both). `find` returns an iterator or `end()`; `count` returns 0 or 1 (any number for multi-containers); `contains` (C++20) returns a bool.



FOR THE KEY 30: LOWER_BOUND IS THE FIRST 30, UPPER_BOUND THE FIRST ELEMENT AFTER THE 30S

RANGE QUERIES ON ORDERED CONTAINERS

main.cpp

```
#include <iostream>
#include <map>
#include <set>
#include <string>

int main() {
    std::multiset<int> s = {10, 20, 30, 30, 40, 50};
    auto lo = s.lower_bound(30), hi = s.upper_bound(30);
    std::cout << "lower_bound(30) -> " << *lo << ", upper_bound(30) -> " << *hi << ", count(30) = " <<
s.count(30) << '\n';
    std::cout << "lower_bound(35) -> " << *s.lower_bound(35) << ", upper_bound(50) is end? " <<
std::boolalpha << (s.upper_bound(50) == s.end()) << '\n';
    std::cout << "contains(40): " << s.contains(40) << ", contains(45): " << s.contains(45) << '\n';

    std::set<int> ids = {3, 8, 15, 21, 34, 42};
    std::cout << "ids in [10, 35]:";
    for (auto it = ids.lower_bound(10); it != ids.upper_bound(35); ++it) std::cout << ' ' << *it;
    std::cout << '\n';

    std::map<int, std::string> grade_from = {{0, "F"}, {40, "D"}, {55, "C"}, {70, "B"}, {85, "A"}};
// lower limits
    for (int mark : {92, 70, 69, 12}) {
        auto it = grade_from.upper_bound(mark);   // first limit above the mark ...
        --it;                                     // ... the one before it is the grade
        std::cout << mark << " -> " << it->second << '\n';
    }
    return 0;
}
```

OUTPUT

```
lower_bound(30) -> 30, upper_bound(30) -> 40, count(30) = 2
lower_bound(35) -> 40, upper_bound(50) is end? true
contains(40): true, contains(45): false
ids in [10, 35]: 15 21 34
92 -> A
70 -> B
69 -> C
12 -> F
```

19.6 Unordered containers

19.6.1 unordered_set and unordered_map

`std::unordered_set` and `std::unordered_map` (and their multi- versions) have almost the same interface as set and map, but store elements in a **hash table**. Lookup, insertion and erasure take O(1) time on average, which makes them the first choice for counting, caching and fast membership tests. The price: there is no ordering (iteration order is unspecified and can change after insertions), no `lower_bound`, and the key type needs a hash function and `==`.

UNORDERED_SET AND UNORDERED_MAP

main.cpp

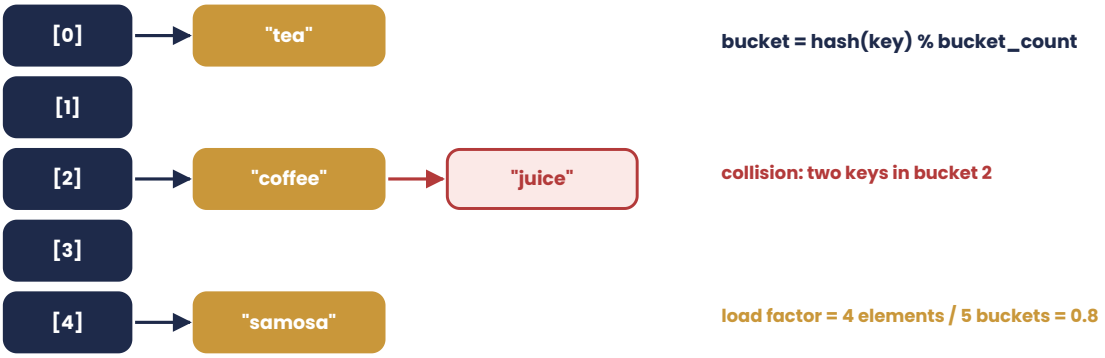
```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>

int main() {
    std::unordered_set<std::string> seen;
    for (const char* email : {"a@x.in", "b@y.in", "a@x.in", "c@z.in", "b@y.in"}) {
        if (seen.contains(email)) std::cout << "duplicate: " << email << '\n';
        else seen.insert(email);
    }
    std::cout << seen.size() << " unique e-mails\n";

    std::unordered_map<std::string, double> price = {{"tea", 15}, {"coffee", 25}, {"samosa", 18}};
    price["juice"] = 40;
    price["tea"] += 2;
    std::cout << "coffee costs " << price.at("coffee") << ", has cake? " << std::boolalpha <<
price.contains("cake") << '\n';

    // Iteration order is unspecified: copy and sort when you need a stable order
    std::vector<std::pair<std::string, double>> menu(price.begin(), price.end());
    std::sort(menu.begin(), menu.end());
    for (const auto& [item, p] : menu) std::cout << "  " << item << ' ' << p << '\n';
    return 0;
}
```

OUTPUT

```
duplicate: a@x.in
duplicate: b@y.in
3 unique e-mails
coffee costs 25, has cake? false
  coffee 25
  juice 40
  samosa 18
  tea 17
```

19.6.2 Hashing, buckets and load factor

A hash table is an array of **buckets**. To store a key, the container computes `std::hash<Key>{}(key)`, a `size_t`, and takes it modulo the bucket count to pick a bucket. Different keys can land in the same bucket (a **collision**); libstdc++ keeps the elements of a bucket in a linked list. The **load factor** is `size() / bucket_count()`, the average number of elements per bucket. When it would exceed `max_load_factor()` (1.0 by default), the table **rehashes**: it allocates more buckets and redistributes every element. `reserve(n)` creates enough buckets for n elements in advance.



A HASH TABLE WITH 5 BUCKETS AND SEPARATE CHAINING

INSPECTING BUCKETS AND REHASHING

main.cpp

```cpp
#include <functional>
#include <iostream>
#include <string>
#include <unordered_map>

int main() {
    std::hash<std::string> h;
    std::cout << "hash(\"tea\") % 13 = " << h("tea") % 13 << ", hash(42) = " << std::hash<int>{}(42)
<< '\n';

    std::unordered_map<int, int> m;
    std::cout << "empty: buckets " << m.bucket_count() << ", max_load_factor " << m.max_load_factor()
<< '\n';
    std::size_t last = m.bucket_count();
    for (int i = 0; i < 100; ++i) {
        m[i] = i * i;
        if (m.bucket_count() != last) {
            last = m.bucket_count();
            std::cout << "size " << m.size() << " -> rehash to " << last << " buckets, load factor "
<< m.load_factor() << '\n';
        }
    }
    std::cout << "key 57 is in bucket " << m.bucket(57) << ", which holds " <<
m.bucket_size(m.bucket(57)) << " element(s)\n";

    std::unordered_map<int, int> r;
    r.reserve(1000);                                    // buckets for 1000 elements: no rehash later
    std::cout << "after reserve(1000): " << r.bucket_count() << " buckets\n";
    return 0;
}
```

OUTPUT

```
hash("tea") % 13 = 9, hash(42) = 42
empty: buckets 1, max_load_factor 1
size 1 -> rehash to 13 buckets, load factor 0.0769231
size 14 -> rehash to 29 buckets, load factor 0.482759
size 30 -> rehash to 59 buckets, load factor 0.508475
size 60 -> rehash to 127 buckets, load factor 0.472441
key 57 is in bucket 57, which holds 1 element(s)
after reserve(1000): 1031 buckets
```

NOTE

The bucket counts are primes chosen by libstdc++ (MSVC uses powers of two), and `std::hash<int>` is simply the identity in libstdc++. These details are implementation-specific: never rely on them.

WATCH OUT

A rehash invalidates all iterators of an unordered container (references and pointers to elements stay valid). In the worst case (many keys hashing to the same bucket) operations degrade to O(n); a poor hash function or keys crafted by an attacker can cause this. That is why competitive programmers sometimes add a randomised hash.

19.6.3 Custom hash functions

Standard types (integers, strings, pointers, `string_view`…) already have `std::hash`. For your own key type you must provide two things: equality (`operator==`) and a hash. Either write a hash functor and pass it as a template argument, or specialise `std::hash` for your type. A good hash combines the hashes of all the fields that equality compares.

HASHING YOUR OWN KEY TYPES main.cpp

```
#include <functional>
#include <iostream>
#include <string>
#include <unordered_map>
#include <unordered_set>

struct Point {
    int x, y;
    bool operator==(const Point&) const = default;          // C++20: compares x and y
};
struct PointHash {                                           // option 1: a functor
    std::size_t operator()(const Point& p) const {
        std::size_t h1 = std::hash<int>{}(p.x);
        std::size_t h2 = std::hash<int>{}(p.y);
        return h1 ^ (h2 + 0x9e3779b9 + (h1 << 6) + (h1 >> 2));   // the boost::hash_combine formula
    }
};

struct Employee { std::string dept; int id; bool operator==(const Employee&) const = default; };
template <>
struct std::hash<Employee> {                                 // option 2: specialise std::hash
    std::size_t operator()(const Employee& e) const {
        return std::hash<std::string>{}(e.dept) * 31 + std::hash<int>{}(e.id);
    }
};

int main() {
    std::unordered_set<Point, PointHash> visited;
    for (Point p : {Point{0, 0}, Point{1, 2}, Point{0, 0}, Point{2, 1}}) visited.insert(p);
    std::cout << "distinct points visited: " << visited.size() << '\n';
    std::cout << std::boolalpha << "visited (1,2)? " << visited.contains({1, 2}) << ", (5,5)? " <<
visited.contains({5, 5}) << '\n';

    std::unordered_map<Employee, std::string> desk;          // uses std::hash<Employee>
    desk[{"IT", 7}] = "B-204";
    desk[{"HR", 7}] = "A-101";
    std::cout << "IT #7 sits at " << desk.at({"IT", 7}) << ", HR #7 at " << desk.at({"HR", 7}) <<
'\n';
    return 0;
}
```

OUTPUT

```
distinct points visited: 3
visited (1,2)? true, (5,5)? false
IT #7 sits at B-204, HR #7 at A-101
```

19.6.4 Ordered vs unordered comparison

	`map` / `set`	`unordered_map` / `unordered_set`
Structure	red-black tree	hash table
Find, insert, erase	O(log n) guaranteed	O(1) average, O(n) worst case
Iteration order	sorted by key	unspecified
Range queries, min/max, lower_bound	yes	no
Key requirements	`operator<` (or comparator)	`std::hash` and `operator==`
Memory	one node per element (3 pointers + colour)	nodes plus the bucket array
Iterator invalidation on insert	never	all, on rehash

A ROUGH SPEED COMPARISON

main.cpp

```cpp
#include <chrono>
#include <iostream>
#include <map>
#include <unordered_map>

template <typename Map>
long long time_ms(int n) {
    auto start = std::chrono::steady_clock::now();
    Map m;
    for (int i = 0; i < n; ++i) m[static_cast<int>(1LL * i * 7919 % n)] = i;   // scrambled order
(1LL: no overflow)
    long long sum = 0;
    for (int i = 0; i < n; ++i) sum += m.find(i)->second;   // n lookups
    auto end = std::chrono::steady_clock::now();
    return sum >= 0 ? std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count() : -1;
}

int main() {
    const int n = 1'000'000;
    std::cout << "1,000,000 inserts + lookups (compiled with -O2)\n";
    std::cout << "  std::map           : " << time_ms<std::map<int, int>>(n) << " ms\n";
    std::cout << "  std::unordered_map : " << time_ms<std::unordered_map<int, int>>(n) << " ms\n";
    return 0;
}
```

OUTPUT

```
1,000,000 inserts + lookups (compiled with -O2)
  std::map           : 296 ms
  std::unordered_map : 155 ms
```

The exact times depend on the machine, but the hash table is typically faster for large numbers of point lookups (about twice as fast in this run; the gap grows with string keys and random access patterns). Choose `map` when you need order or range queries, or guaranteed worst-case times; choose `unordered_map` for pure key lookups.

19.7 Utility types

19.7.1 pair and make_pair

`std::pair<A, B>` (header `<utility>`) groups two values, accessed as `.first` and `.second`. Map elements are pairs; functions use pairs to return two results. `std::make_pair(a, b)` creates one with deduced types (with CTAD, `std::pair p(a, b)` or just `{a, b}` works too). Pairs compare **lexicographically**: first by `first`, then by `second`, which makes sorting pairs very convenient.

WORKING WITH PAIRS

main.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <utility>
#include <vector>

std::pair<int, int> min_max(const std::vector<int>& v) {
    auto [lo, hi] = std::minmax_element(v.begin(), v.end());
    return {*lo, *hi};
}

int main() {
    std::pair<std::string, int> a("Asha", 91);
    auto b = std::make_pair(std::string("Ravi"), 78);
    std::pair c{"Meera", 91};                          // CTAD: pair<const char*, int>
    std::cout << a.first << ' ' << a.second << " | " << b.first << ' ' << b.second << " | " << c.first
<< '\n';

    auto [lo, hi] = min_max({4, 9, 1, 7});
    std::cout << "min " << lo << ", max " << hi << '\n';

    std::vector<std::pair<int, std::string>> ranking = {{78, "Ravi"}, {91, "Meera"}, {91, "Asha"},
{66, "Kiran"}};
    std::sort(ranking.begin(), ranking.end());       // by score, then by name
    for (const auto& [score, name] : ranking) std::cout << score << ' ' << name << '\n';
    std::cout << std::boolalpha << "(1,9) < (2,0)? " << (std::pair(1, 9) < std::pair(2, 0)) << '\n';
    return 0;
}
```

OUTPUT

```
Asha 91 | Ravi 78 | Meera
min 1, max 9
66 Kiran
78 Ravi
91 Asha
91 Meera
(1,9) < (2,0)? true
```

19.7.2 tuple, tie and get

`std::tuple<Ts...>` (header `<tuple>`) generalises pair to any number of values. Elements are accessed by index with `std::get<0>(t)` (the index must be a compile-time constant) or by type with `std::get<double>(t)` when that type is unique. `std::tie(a, b, c)` makes a tuple of **references**: assigning a tuple to it unpacks the values into existing variables, and comparing two `tie`s is a neat way to write multi-field comparisons. Structured bindings (`auto [a, b, c] = t;`) are the modern way to unpack into new variables.

TUPLE, GET AND TIE

main.cpp

```cpp
#include <iostream>
#include <string>
#include <tuple>

std::tuple<std::string, int, double> student() { return {"Asha", 20, 8.7}; }

struct Version {
    int major, minor, patch;
    bool operator<(const Version& o) const {
        return std::tie(major, minor, patch) < std::tie(o.major, o.minor, o.patch);   // lexicographic
    }
};

int main() {
    auto t = student();
    std::cout << std::get<0>(t) << ", age " << std::get<1>(t) << ", gpa " << std::get<double>(t) <<
'\n';
    std::cout << "tuple size: " << std::tuple_size_v<decltype(t)> << '\n';

    std::string name;
    int age;
    std::tie(name, age, std::ignore) = student();      // unpack into existing variables, skip one
    std::cout << "tie: " << name << ' ' << age << '\n';

    auto [n, a, g] = student();                          // structured binding: new variables
    std::cout << "binding: " << n << ' ' << a << ' ' << g << '\n';

    auto combined = std::tuple_cat(t, std::make_tuple('A', true));
    std::cout << "tuple_cat has " << std::tuple_size_v<decltype(combined)> << " elements, last = " <<
std::boolalpha << std::get<4>(combined) << '\n';

    Version v1{1, 4, 2}, v2{1, 10, 0};
    std::cout << "1.4.2 < 1.10.0? " << (v1 < v2) << '\n';
    return 0;
}
```

OUTPUT

```
Asha, age 20, gpa 8.7
tuple size: 3
tie: Asha 20
binding: Asha 20 8.7
tuple_cat has 5 elements, last = true
1.4.2 < 1.10.0? true
```

19.8 Algorithms (<algorithm> and <numeric>)

The algorithms take an iterator range [first, last) and often a function object. They never change the size of a container (they cannot: they only see iterators), so "removing" or "adding" algorithms return iterators or write through an output iterator such as `std::back_inserter`. C++20 adds **range versions** in namespace `std::ranges` that accept a whole container: `std::ranges::sort(v)` instead of `std::sort(v.begin(), v.end())` (Module 21). The examples below use the classic form, which works in every standard.

19.8.1 Sorting: sort, stable_sort, partial_sort, nth_element

Algorithm	What it does	Cost
`sort`	sorts the range (introsort: not stable)	O(n log n)
`stable_sort`	sorts, keeping equal elements in their original order	O(n log n)
`partial_sort(first, middle, last)`	puts the smallest (middle - first) elements, sorted, at the front	O(n log k)
`nth_element(first, nth, last)`	puts the element that would be at nth after sorting there; smaller ones before, larger after (unsorted)	O(n) average
`is_sorted`	checks whether a range is sorted	O(n)

FOUR WAYS TO SORT

main.cpp

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <string>
#include <vector>

struct Student { std::string name; int marks; };

void show(const std::vector<int>& v) { for (int x : v) std::cout << x << ' '; std::cout << '\n'; }

int main() {
    std::vector<int> v = {42, 7, 19, 88, 3, 56, 21, 64};
    auto a = v;
    std::sort(a.begin(), a.end());
    std::cout << "sort:            "; show(a);
    std::sort(a.begin(), a.end(), std::greater<int>());
    std::cout << "sort descending: "; show(a);

    auto b = v;
    std::partial_sort(b.begin(), b.begin() + 3, b.end());       // only the 3 smallest, in order
    std::cout << "partial_sort 3:  "; show(b);

    auto c = v;
    std::nth_element(c.begin(), c.begin() + c.size() / 2, c.end());   // median position
    std::cout << "nth_element:     "; show(c);
    std::cout << "median (upper) = " << c[c.size() / 2] << '\n';

    std::vector<Student> cls = {{"Asha", 85}, {"Ravi", 72}, {"Meera", 85}, {"Kiran", 72}, {"Dev",
91}};
    std::stable_sort(cls.begin(), cls.end(), [](const Student& x, const Student& y) { return x.marks >
y.marks; });
    std::cout << "stable_sort by marks (ties keep input order):";
    for (const auto& s : cls) std::cout << ' ' << s.name << '(' << s.marks << ')';
    std::cout << '\n';
    std::cout << std::boolalpha << "is_sorted(a)? " << std::is_sorted(a.begin(), a.end()) << '\n';
    return 0;
}
```

OUTPUT

```
sort:            3 7 19 21 42 56 64 88
sort descending: 88 64 56 42 21 19 7 3
partial_sort 3:  3 7 19 88 42 56 21 64
nth_element:     7 3 21 19 42 56 64 88
median (upper) = 42
stable_sort by marks (ties keep input order): Dev(91) Asha(85) Meera(85) Ravi(72) Kiran(72)
is_sorted(a)? false
```

NOTE

The arrangement of the other elements after `partial_sort` and `nth_element` is unspecified; what is shown is what GCC produced. Only the guaranteed part (the sorted prefix, the element at nth with smaller ones before it) may be relied on.

19.8.2 Searching: find, find_if, binary_search, lower_bound, upper_bound

`find` and `find_if` scan linearly and return an iterator to the first match (or `last`). On a **sorted** range the binary-search family runs in O(log n): `binary_search` answers yes/no; `lower_bound` returns the first element ≥ value (the insertion point that keeps the range sorted); `upper_bound` the first element > value. Other useful searches: `search` (a subsequence), `adjacent_find` (two equal neighbours), `find_first_of`.

LINEAR AND BINARY SEARCHES

main.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> names = {"Ravi", "Asha", "Meera", "Kiran"};
    auto it = std::find(names.begin(), names.end(), "Meera");
    if (it != names.end()) std::cout << "Meera at index " << it - names.begin() << '\n';
    auto k = std::find_if(names.begin(), names.end(), [](const std::string& s) { return
s.starts_with('K'); });
    std::cout << "first name starting with K: " << *k << '\n';

    std::vector<int> sorted = {3, 8, 8, 8, 15, 23, 42};
    std::cout << std::boolalpha << "binary_search 15: " << std::binary_search(sorted.begin(),
sorted.end(), 15)
              << ", 16: " << std::binary_search(sorted.begin(), sorted.end(), 16) << '\n';
    auto lo = std::lower_bound(sorted.begin(), sorted.end(), 8);
    auto hi = std::upper_bound(sorted.begin(), sorted.end(), 8);
    std::cout << "8 occupies indexes [" << lo - sorted.begin() << ", " << hi - sorted.begin() << "),
count " << hi - lo << '\n';
    auto pos = std::lower_bound(sorted.begin(), sorted.end(), 20);
    sorted.insert(pos, 20);                                  // insert keeping the order
    for (int x : sorted) std::cout << x << ' ';
    std::cout << '\n';

    std::vector<int> signal = {1, 2, 3, 2, 3, 4, 2, 3, 4};
    std::vector<int> pattern = {2, 3, 4};
    auto s = std::search(signal.begin(), signal.end(), pattern.begin(), pattern.end());
    std::cout << "pattern 2 3 4 first starts at index " << s - signal.begin() << '\n';
    auto adj = std::adjacent_find(sorted.begin(), sorted.end());
    std::cout << "first repeated neighbour: " << *adj << '\n';
    return 0;
}
```

OUTPUT

```
Meera at index 2
first name starting with K: Kiran
binary_search 15: true, 16: false
8 occupies indexes [1, 4), count 3
3 8 8 8 15 20 23 42
pattern 2 3 4 first starts at index 3
first repeated neighbour: 8
```

19.8.3 Counting and checking: count, count_if, all_of, any_of, none_of

COUNTING AND CHECKING CONDITIONS

main.cpp

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<int> marks = {78, 35, 91, 62, 35, 88, 40};
    std::cout << "count(35)      = " << std::count(marks.begin(), marks.end(), 35) << '\n';
    std::cout << "count_if(>=60) = " << std::count_if(marks.begin(), marks.end(), [](int m) { return m
>= 60; }) << '\n';
    std::cout << std::boolalpha;
    std::cout << "all passed (>=35)?  " << std::all_of(marks.begin(), marks.end(), [](int m) { return
m >= 35; }) << '\n';
    std::cout << "any distinction?    " << std::any_of(marks.begin(), marks.end(), [](int m) { return
m >= 90; }) << '\n';
    std::cout << "none above 95?      " << std::none_of(marks.begin(), marks.end(), [](int m) { return
m > 95; }) << '\n';

    std::string pwd = "Learn2Code!";
    bool ok = std::any_of(pwd.begin(), pwd.end(), ::isdigit) && std::any_of(pwd.begin(), pwd.end(),
::isupper)
              && std::any_of(pwd.begin(), pwd.end(), ::ispunct);
    std::cout << "password strong? " << ok << '\n';
    std::vector<int> empty;
    std::cout << "all_of on empty range: " << std::all_of(empty.begin(), empty.end(), [](int) { return
false; }) << '\n';
    return 0;
}
```

OUTPUT

```
count(35)      = 2
count_if(>=60) = 4
all passed (>=35)?  true
any distinction?    true
none above 95?      true
password strong? true
all_of on empty range: true
```

NOTE

`all_of` on an empty range is `true` and `any_of` is `false`, just like "every element of the empty set is red" in logic.

19.8.4 Modifying: copy, transform, fill, replace, remove, unique, reverse, rotate

Algorithm	Effect
`copy`, `copy_if`, `copy_n`	copy elements (all, matching, first n) to an output iterator
`transform`	apply a function to each element (or pairs of elements from two ranges), writing the results
`fill`, `fill_n`, `generate`	assign a value, or the results of a function, to each element
`replace`, `replace_if`	replace matching values in place

Algorithm	Effect
`remove`, `remove_if`	compact the kept elements to the front; return the new end (then erase)
`unique`	remove **consecutive** duplicates (sort first to remove all); returns the new end
`reverse`	reverse the order
`rotate(first, middle, last)`	rotate left so that middle becomes the first element

MODIFYING ALGORITHMS

main.cpp

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <iterator>
#include <string>
#include <vector>

void show(const char* label, const std::vector<int>& v) {
    std::cout << label;
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';
}

int main() {
    std::vector<int> src = {1, 2, 3, 4, 5, 6};
    std::vector<int> dst;
    std::copy_if(src.begin(), src.end(), std::back_inserter(dst), [](int x) { return x % 2 == 0; });
    show("copy_if even:  ", dst);

    std::vector<int> sq(src.size());
    std::transform(src.begin(), src.end(), sq.begin(), [](int x) { return x * x; });
    show("transform x*x: ", sq);
    std::vector<int> sum(src.size());
    std::transform(src.begin(), src.end(), sq.begin(), sum.begin(), [](int a, int b) { return a + b;
});
    show("x + x*x:       ", sum);

    std::vector<int> f(5);
    std::fill(f.begin(), f.end(), 7);
    show("fill 7:        ", f);
    int n = 0;
    std::generate(f.begin(), f.end(), [&n] { return n += 10; });
    show("generate:      ", f);

    std::vector<int> r = {1, 0, 2, 0, 3};
    std::replace(r.begin(), r.end(), 0, -1);
    show("replace 0->-1: ", r);

    std::vector<int> u = {4, 1, 4, 2, 1, 4, 3};
    std::sort(u.begin(), u.end());
    u.erase(std::unique(u.begin(), u.end()), u.end());
    show("sort + unique: ", u);

    std::vector<int> rot = {1, 2, 3, 4, 5};
    std::rotate(rot.begin(), rot.begin() + 2, rot.end());
    show("rotate by 2:   ", rot);
    std::reverse(rot.begin(), rot.end());
    show("reverse:       ", rot);

    std::string s = "Hello, STL!";
    std::transform(s.begin(), s.end(), s.begin(), [](unsigned char c) { return std::toupper(c); });
```

```
    std::cout << "uppercase:     " << s << '\n';
    return 0;
}
```

OUTPUT

```
copy_if even:  2 4 6
transform x*x: 1 4 9 16 25 36
x + x*x:       2 6 12 20 30 42
fill 7:        7 7 7 7 7
generate:      10 20 30 40 50
replace 0->-1: 1 -1 2 -1 3
sort + unique: 1 2 3 4
rotate by 2:   3 4 5 1 2
reverse:       2 1 5 4 3
uppercase:     HELLO, STL!
```

WATCH OUT

An algorithm that writes to a destination assumes there is room. `std::copy(a.begin(), a.end(), b.begin())` on an empty `b` writes past its end: undefined behaviour. Either size the destination first (`std::vector<int> b(a.size())`) or use `std::back_inserter(b)` .

19.8.5 Min/max: min_element, max_element, minmax, clamp

FINDING EXTREMES

main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

struct City { std::string name; int population; };

int main() {
    std::vector<int> temps = {31, 27, 35, 24, 35, 29};
    auto hot = std::max_element(temps.begin(), temps.end());     // first maximum
    auto cold = std::min_element(temps.begin(), temps.end());
    std::cout << "max " << *hot << " on day " << hot - temps.begin() << ", min " << *cold << " on day
" << cold - temps.begin() << '\n';
    auto [lo, hi] = std::minmax_element(temps.begin(), temps.end());   // last maximum
    std::cout << "minmax_element: " << *lo << " .. " << *hi << " (day " << hi - temps.begin() <<
")\n";

    std::cout << "max(3, 8) = " << std::max(3, 8) << ", min({5, 2, 9}) = " << std::min({5, 2, 9}) <<
'\n';
    auto [a, b] = std::minmax({7, 3, 11, 5});
    std::cout << "minmax({7, 3, 11, 5}) = " << a << ", " << b << '\n';

    for (int vol : {-5, 40, 130}) std::cout << "clamp(" << vol << ", 0, 100) = " << std::clamp(vol, 0,
100) << '\n';

    std::vector<City> cities = {{"Mysuru", 1'200'000}, {"Bengaluru", 13'600'000}, {"Hubballi",
1'100'000}};
    auto big = std::max_element(cities.begin(), cities.end(), [](const City& x, const City& y) {
return x.population < y.population; });
    std::cout << "largest city: " << big->name << '\n';
    return 0;
}
```

OUTPUT

```
max 35 on day 2, min 24 on day 3
minmax_element: 24 .. 35 (day 4)
max(3, 8) = 8, min({5, 2, 9}) = 2
minmax({7, 3, 11, 5}) = 3, 11
clamp(-5, 0, 100) = 0
clamp(40, 0, 100) = 40
clamp(130, 0, 100) = 100
largest city: Bengaluru
```

19.8.6 Numeric: accumulate, reduce, iota, partial_sum, gcd, lcm

The header `<numeric>` holds numeric algorithms. `accumulate(first, last, init, op)` folds left to right; the **type of init** decides the result type, so `accumulate(v.begin(), v.end(), 0)` on doubles truncates to int. `reduce` (C++17) is similar but may combine in any order (so it can run in parallel); `transform_reduce` maps then reduces (a dot product). `iota` fills with increasing values, `partial_sum` computes running totals (prefix sums), `adjacent_difference` the opposite, and `std::gcd` / `std::lcm` (C++17) the greatest common divisor and least common multiple.

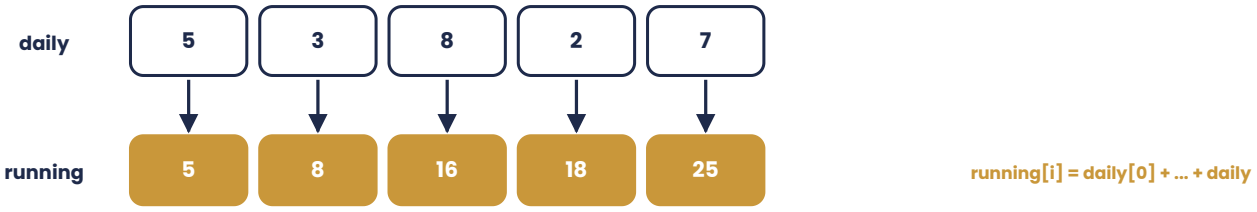
NUMERIC ALGORITHMS

main.cpp

```cpp
#include <functional>
#include <iostream>
#include <numeric>
#include <string>
#include <vector>

void show(const char* label, const std::vector<int>& v) {
    std::cout << label;
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';
}

int main() {
    std::vector<int> v(6);
    std::iota(v.begin(), v.end(), 1);                                   // 1 2 3 4 5 6
    show("iota:          ", v);
    std::cout << "sum            " << std::accumulate(v.begin(), v.end(), 0) << '\n';
    std::cout << "product        " << std::accumulate(v.begin(), v.end(), 1, std::multiplies<int>())
<< '\n';
    std::cout << "reduce         " << std::reduce(v.begin(), v.end()) << '\n';

    std::vector<double> prices = {9.5, 20.25, 3.75};
    std::cout << "accumulate with 0   -> " << std::accumulate(prices.begin(), prices.end(), 0) << "
(int init: truncated!)\n";
    std::cout << "accumulate with 0.0 -> " << std::accumulate(prices.begin(), prices.end(), 0.0) <<
'\n';

    std::vector<std::string> words = {"learn", "being", "forward"};
    std::string joined = std::accumulate(std::next(words.begin()), words.end(), words[0],
                                         [](std::string acc, const std::string& w) { return acc + "-"
+ w; });
    std::cout << "joined         " << joined << '\n';

    std::vector<int> qty = {2, 1, 3}, rate = {50, 120, 20};
    std::cout << "bill (dot)     " << std::transform_reduce(qty.begin(), qty.end(), rate.begin(), 0)
<< '\n';

    std::vector<int> daily = {5, 3, 8, 2, 7}, running(5), diff(5);
    std::partial_sum(daily.begin(), daily.end(), running.begin());
    show("partial_sum:   ", running);
    std::adjacent_difference(running.begin(), running.end(), diff.begin());
    show("adjacent_diff: ", diff);
    std::cout << "gcd(84, 36) = " << std::gcd(84, 36) << ", lcm(4, 6) = " << std::lcm(4, 6) << '\n';
    return 0;
}
```

OUTPUT

```
iota:          1 2 3 4 5 6
sum            21
product        720
reduce         21
accumulate with 0   -> 32  (int init: truncated!)
accumulate with 0.0 -> 33.5
joined         learn-being-forward
bill (dot)     280
partial_sum:   5 8 16 18 25
adjacent_diff: 5 3 8 2 7
gcd(84, 36) = 12, lcm(4, 6) = 12
```



PARTIAL_SUM BUILDS PREFIX SUMS: ANY RANGE SUM IS THEN ONE SUBTRACTION

19.8.7 Permutations: next_permutation, prev_permutation

`next_permutation` rearranges a range into the next arrangement in lexicographic order and returns `false` (after wrapping around to the smallest) when there is none. Starting from a sorted range and looping with `do { ... } while (next_permutation(...))` visits every permutation exactly once, including those with repeated elements. `prev_permutation` goes the other way.

GENERATING PERMUTATIONS

main.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::string s = "abc";
    int count = 0;
    do {
        std::cout << s << ' ';
        ++count;
    } while (std::next_permutation(s.begin(), s.end()));
    std::cout << "(" << count << " permutations)\n";

    std::string word = "noon";                          // repeated letters: only distinct arrangements
    std::sort(word.begin(), word.end());
    count = 0;
    do ++count; while (std::next_permutation(word.begin(), word.end()));
    std::cout << "distinct arrangements of noon: " << count << '\n';

    std::vector<int> v = {3, 2, 1};
    std::prev_permutation(v.begin(), v.end());
    std::cout << "permutation before 3 2 1: " << v[0] << v[1] << v[2] << '\n';

    // brute force: shortest route visiting 3 cities from city 0 and back
    int dist[4][4] = {{0, 10, 15, 20}, {10, 0, 35, 25}, {15, 35, 0, 30}, {20, 25, 30, 0}};
    std::vector<int> order = {1, 2, 3};
    int best = 1'000'000;
    std::vector<int> best_order;
    do {
        int d = dist[0][order[0]] + dist[order[0]][order[1]] + dist[order[1]][order[2]] +
dist[order[2]][0];
        if (d < best) { best = d; best_order = order; }
    } while (std::next_permutation(order.begin(), order.end()));
    std::cout << "best tour 0-" << best_order[0] << '-' << best_order[1] << '-' << best_order[2] <<
"-0, length " << best << '\n';
    return 0;
}
```

OUTPUT

```
abc acb bac bca cab cba (6 permutations)
distinct arrangements of noon: 6
permutation before 3 2 1: 312
best tour 0-1-3-2-0, length 80
```

19.8.8 Set algorithms: set_union, set_intersection, set_difference

These algorithms work on two **sorted** ranges (any sorted container, not only std::set) and write the result in sorted order to an output iterator, in linear time. `set_symmetric_difference` gives the elements in exactly one of the two ranges, and `includes` tests whether one sorted range contains all elements of another.

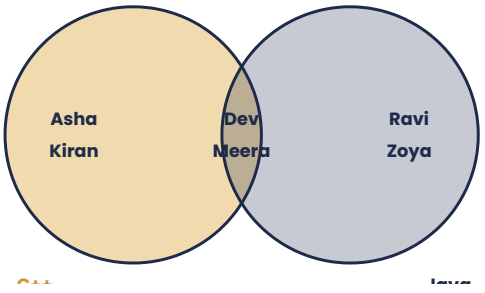
SET OPERATIONS ON SORTED RANGES

main.cpp

```
#include <algorithm>
#include <iostream>
#include <iterator>
#include <string>
#include <vector>

void show(const char* label, const std::vector<std::string>& v) {
    std::cout << label;
    for (const auto& s : v) std::cout << s << ' ';
    std::cout << '\n';
}

int main() {
    std::vector<std::string> cpp = {"Asha", "Dev", "Kiran", "Meera"};        // sorted
    std::vector<std::string> java = {"Dev", "Meera", "Ravi", "Zoya"};        // sorted
    std::vector<std::string> any, both, only_cpp, one;
    std::set_union(cpp.begin(), cpp.end(), java.begin(), java.end(), std::back_inserter(any));
    std::set_intersection(cpp.begin(), cpp.end(), java.begin(), java.end(), std::back_inserter(both));
    std::set_difference(cpp.begin(), cpp.end(), java.begin(), java.end(),
std::back_inserter(only_cpp));
    std::set_symmetric_difference(cpp.begin(), cpp.end(), java.begin(), java.end(),
std::back_inserter(one));
    show("union (any course):        ", any);
    show("intersection (both):       ", both);
    show("difference (only C++):     ", only_cpp);
    show("symmetric (exactly one):   ", one);
    std::vector<std::string> pair = {"Dev", "Meera"};
    std::cout << std::boolalpha << "C++ includes Dev and Meera? " << std::includes(cpp.begin(),
cpp.end(), pair.begin(), pair.end()) << '\n';
    return 0;
}
```

OUTPUT

```
union (any course):        Asha Dev Kiran Meera Ravi Zoya
intersection (both):       Dev Meera
difference (only C++):     Asha Kiran
symmetric (exactly one):   Asha Kiran Ravi Zoya
C++ includes Dev and Meera? true
```



union: all six

intersection: middle

difference: left part

symmetric: both sides

THE FOUR SET OPERATIONS

19.8.9 Heap algorithms: make_heap, push_heap, pop_heap

The heap algorithms turn a random-access range into a binary max-heap and maintain it; `priority_queue` is built on them. `make_heap` arranges a range into a heap in O(n). After `push_back`, `push_heap` moves the new last element into place in O(log n). `pop_heap` swaps the maximum to the end and repairs the rest in O(log n); then `pop_back` removes it. `sort_heap` turns a heap into a sorted range (heapsort). Using these directly lets you look at all the elements, which a `priority_queue` does not.

MAINTAINING A HEAP BY HAND main.cpp

```
#include <algorithm>
#include <iostream>
#include <vector>

void show(const char* label, const std::vector<int>& v) {
    std::cout << label;
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';
}

int main() {
    std::vector<int> h = {20, 50, 10, 90, 70, 30};
    std::make_heap(h.begin(), h.end());
    show("make_heap:           ", h);
    std::cout << "max = front = " << h.front() << ", is_heap: " << std::boolalpha <<
std::is_heap(h.begin(), h.end()) << '\n';

    h.push_back(80);
    std::push_heap(h.begin(), h.end());
    show("push 80:             ", h);

    std::pop_heap(h.begin(), h.end());             // max moves to the back
    std::cout << "pop_heap moved " << h.back() << " to the back\n";
    h.pop_back();
    show("after pop_back:      ", h);

    std::sort_heap(h.begin(), h.end());
    show("sort_heap:           ", h);
    return 0;
}
```

OUTPUT

```
make_heap:           90 70 30 50 20 10
max = front = 90, is_heap: true
push 80:             90 70 80 50 20 10 30
pop_heap moved 90 to the back
after pop_back:      80 70 30 50 20 10
sort_heap:           10 20 30 50 70 80
```

19.9 Function objects and comparators

19.9.1 Functors

A **function object** (functor) is an object of a class with `operator()`, so it can be called like a function. Unlike a plain function it can carry **state** (fields set in the constructor or updated on each call), and because its type is known at compile time, algorithms can inline the call, which often makes functors faster than function pointers.

WRITING FUNCTORS

main.cpp

```
#include <algorithm>
#include <iostream>
#include <vector>

class MultipleOf {                                  // a predicate with a parameter
public:
    explicit MultipleOf(int n) : n_(n) {}
    bool operator()(int x) const { return x % n_ == 0; }
private:
    int n_;
};

class Counter {                                     // a functor with state that changes
public:
    int operator()() { return ++count_; }
    int count() const { return count_; }
private:
    int count_ = 0;
};

int main() {
    std::vector<int> v = {3, 10, 15, 22, 30, 41};
    MultipleOf by5(5);
    std::cout << "by5(25) = " << std::boolalpha << by5(25) << '\n';     // called like a function
    std::cout << "multiples of 5: " << std::count_if(v.begin(), v.end(), by5) << '\n';
    std::cout << "multiples of 3: " << std::count_if(v.begin(), v.end(), MultipleOf(3)) << '\n';

    std::vector<int> ids(4);
    Counter c;
    std::generate(ids.begin(), ids.end(), std::ref(c));                 // std::ref: use c itself,
not a copy
    for (int id : ids) std::cout << id << ' ';
    std::cout << "| counter called " << c.count() << " times\n";
    return 0;
}
```

OUTPUT

```
by5(25) = true
multiples of 5: 3
multiples of 3: 3
1 2 3 4 | counter called 4 times
```

NOTE

Algorithms take function objects **by value**, so a stateful functor is copied and the original does not see the updates. Wrapping it in `std::ref` (header `<functional>`) passes a reference instead.

19.9.2 Built-in functors: less, greater, plus

The header `<functional>` defines functor templates for all the operators: comparisons (`less`, `greater`, `less_equal`, `greater_equal`, `equal_to`, `not_equal_to`), arithmetic (`plus`, `minus`, `multiplies`, `divides`, `modulus`, `negate`) and logic (`logical_and`, `logical_or`, `logical_not`). Since C++14 you can omit the type: `std::greater<>` deduces it from the arguments ("transparent" functors).

STANDARD FUNCTION OBJECTS

main.cpp

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <map>
#include <numeric>
#include <set>
#include <string>
#include <vector>

int main() {
    std::vector<int> v = {4, 1, 3, 5, 2};
    std::sort(v.begin(), v.end(), std::greater<>());               // descending
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';

    std::set<std::string, std::greater<>> words = {"apple", "mango", "kiwi"};   // reverse-sorted set
    for (const auto& w : words) std::cout << w << ' ';
    std::cout << '\n';

    std::cout << "plus: " << std::accumulate(v.begin(), v.end(), 0, std::plus<>())
              << ", multiplies: " << std::accumulate(v.begin(), v.end(), 1, std::multiplies<>()) <<
'\n';

    std::vector<int> a = {1, 2, 3}, b = {10, 20, 30}, c(3);
    std::transform(a.begin(), a.end(), b.begin(), c.begin(), std::plus<>());
    std::cout << "a + b: " << c[0] << ' ' << c[1] << ' ' << c[2] << '\n';
    std::transform(a.begin(), a.end(), c.begin(), std::negate<>());
    std::cout << "negated: " << c[0] << ' ' << c[1] << ' ' << c[2] << '\n';
    std::cout << std::boolalpha << "less<>{}(2, 3) = " << std::less<>{}(2, 3) << '\n';
    return 0;
}
```

OUTPUT

```
5 4 3 2 1
mango kiwi apple
plus: 15, multiplies: 120
a + b: 11 22 33
negated: -1 -2 -3
less<>{}(2, 3) = true
```

19.9.3 Custom comparators with lambdas

A **comparator** is a function object taking two elements and returning `true` if the first should come **before** the second. It must be a **strict weak ordering**: `comp(a, a)` is false, and the relation is consistent and transitive. Using `<=` instead of `<` breaks this rule and can crash `std::sort`. Lambdas are the most convenient comparators for algorithms; for containers such as `set` or `map` the comparator is part of the type, so we pass `decltype(lambda)` (since C++20 a capture-less lambda type is default-constructible, so the object need not be passed).

LAMBDAS AS COMPARATORS main.cpp

```
#include <algorithm>
#include <cstdlib>
#include <iostream>
#include <set>
#include <string>
#include <tuple>
#include <vector>

struct Employee { std::string name; std::string dept; int salary; };

int main() {
    std::vector<Employee> staff = {{"Asha", "IT", 90000}, {"Ravi", "HR", 60000}, {"Meera", "IT",
120000}, {"Kiran", "HR", 60000}, {"Dev", "Ops", 75000}};

    // dept ascending, then salary descending, then name ascending
    std::sort(staff.begin(), staff.end(), [](const Employee& a, const Employee& b) {
        return std::tie(a.dept, b.salary, a.name) < std::tie(b.dept, a.salary, b.name);
    });
    for (const auto& e : staff) std::cout << e.dept << ' ' << e.salary << ' ' << e.name << '\n';

    auto by_length = [](const std::string& a, const std::string& b) {
        return a.size() != b.size() ? a.size() < b.size() : a < b;      // length, then alphabetical
    };
    std::set<std::string, decltype(by_length)> words = {"banana", "fig", "apple", "kiwi", "date"};
    for (const auto& w : words) std::cout << w << ' ';
    std::cout << '\n';

    std::vector<int> v = {-7, 3, -1, 8, -4};
    std::sort(v.begin(), v.end(), [](int a, int b) { return std::abs(a) < std::abs(b); });   // by
absolute value
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
HR 60000 Kiran
HR 60000 Ravi
IT 120000 Meera
IT 90000 Asha
Ops 75000 Dev
fig date kiwi apple banana
-1 3 -4 -7 8
```

TIP

`std::tie(a.x, b.y) < std::tie(b.x, a.y)` is a compact way to express "x ascending, then y descending": swapping `a` and `b` for one field reverses that field.

19.10 Time complexity of STL operations

The standard specifies the complexity of every container operation and algorithm, so you can predict performance before writing code. "Amortised" means the average over a long sequence of operations; "average" for hash tables assumes a good hash function.

Operation	vector	deque	list	set / map	unordered_set / _map
access `[i]`	O(1)	O(1)	-	-	-
find a value / key	O(n)	O(n)	O(n)	O(log n)	O(1) avg, O(n) worst
insert / erase at back	O(1) amortised	O(1)	O(1)	-	-
insert / erase at front	O(n)	O(1)	O(1)	-	-
insert / erase in middle	O(n)	O(n)	O(1)*	O(log n)	O(1) avg
iterate over all	O(n), fastest	O(n)	O(n), slowest	O(n)	O(n + buckets)
sorted order	sort: O(n log n)	sort: O(n log n)	list::sort O(n log n)	always	never

* given an iterator to the position. Adaptors: `stack` and `queue` push/pop are O(1); `priority_queue` top is O(1), push/pop O(log n).

Algorithm	Complexity
`sort`, `stable_sort`	O(n log n)
`partial_sort` (k elements)	O(n log k)
`nth_element`	O(n) average
`binary_search`, `lower_bound`, `upper_bound`	O(log n) comparisons (on random-access iterators)
`find`, `count`, `accumulate`, `transform`, `copy`, `reverse`, `unique`, `remove`	O(n)
`set_union` and friends, `merge`	O(n + m)
`make_heap`	O(n)
`push_heap`, `pop_heap`	O(log n)
`next_permutation`	O(n) worst, O(1) amortised

STEPS FOR N = 1,000,000: THE DIFFERENCE BETWEEN THE CLASSES IS ENORMOUS (O(1) AND O(LOG N) ARE TOO SMALL TO SEE)

Containers hold data, iterators walk it, algorithms process any iterator range, and function objects customise the algorithms. Default to vector; use deque for both ends, list for stable iterators and splicing, set/map for sorted keys and range queries, unordered_set/map for O(1) average lookups, and the adaptors for LIFO, FIFO and priorities. Know the invalidation rules, use erase-remove or std::erase_if, and prefer a standard algorithm to a hand-written loop.

PRACTICE CHECKPOINT

Solve **word frequency** (unordered_map), **top-K elements** (priority_queue), and an **ordered leaderboard** (map or set with a custom comparator).

SAMPLE SOLUTION

main.cpp

```
#include <algorithm>
#include <cctype>
#include <functional>
#include <iostream>
#include <queue>
#include <set>
#include <sstream>
#include <string>
#include <unordered_map>
#include <vector>

// 1. Word frequency with unordered_map: O(n) average
std::unordered_map<std::string, int> word_frequency(const std::string& text) {
    std::unordered_map<std::string, int> freq;
    std::istringstream in(text);
    for (std::string w; in >> w; ) {
        std::string clean;
        for (unsigned char c : w)
            if (std::isalpha(c)) clean += static_cast<char>(std::tolower(c));
        if (!clean.empty()) ++freq[clean];
    }
    return freq;
}

// 2. Top-K with a min-heap of size K: O(n log K)
std::vector<std::pair<int, std::string>> top_k(const std::unordered_map<std::string, int>& freq,
std::size_t k) {
    using Entry = std::pair<int, std::string>;                // (count, word)
    auto worse = [](const Entry& a, const Entry& b) {          // min-heap: the "worst" entry on top
        return a.first != b.first ? a.first > b.first : a.second < b.second;
    };
    std::priority_queue<Entry, std::vector<Entry>, decltype(worse)> heap(worse);
    for (const auto& [word, count] : freq) {
        heap.push({count, word});
        if (heap.size() > k) heap.pop();                       // drop the weakest
    }
    std::vector<Entry> result;
    while (!heap.empty()) { result.push_back(heap.top()); heap.pop(); }
    std::reverse(result.begin(), result.end());                // best first
    return result;
}

// 3. Leaderboard: set with a custom comparator, always sorted
struct Player { std::string name; int score; long time_ms; };
struct Ranking {
    bool operator()(const Player& a, const Player& b) const {
        if (a.score != b.score) return a.score > b.score;      // higher score first
```

```
        if (a.time_ms != b.time_ms) return a.time_ms < b.time_ms;   // then faster
        return a.name < b.name;                                     // then by name (keeps entries unique)
    }
};

int main() {
    std::string text = "The quick brown fox jumps over the lazy dog. The dog sleeps; the fox runs! "
                       "A quick fox, a lazy dog.";
    auto freq = word_frequency(text);
    std::cout << "distinct words: " << freq.size() << ", 'the' = " << freq["the"] << ", 'fox' = " <<
freq["fox"] << '\n';

    std::cout << "top 3 words:\n";
    for (const auto& [count, word] : top_k(freq, 3)) std::cout << "  " << word << " x" << count <<
'\n';

    std::set<Player, Ranking> board;
    board.insert({"Asha", 920, 51200});
    board.insert({"Ravi", 870, 48800});
    board.insert({"Meera", 920, 49750});
    board.insert({"Kiran", 990, 60100});
    board.insert({"Dev", 870, 48800});

    auto update = [&board](const std::string& name, int new_score, long new_time) {
        auto it = std::find_if(board.begin(), board.end(), [&](const Player& p) { return p.name ==
name; });
        if (it != board.end()) board.erase(it);                 // keys are const: erase and re-insert
        board.insert({name, new_score, new_time});
    };
    update("Ravi", 1005, 47000);

    std::cout << "leaderboard:\n";
    int rank = 0;
    for (const auto& p : board)
        std::cout << "  " << ++rank << ". " << p.name << "  " << p.score << " pts  " << p.time_ms /
1000.0 << " s\n";
    return 0;
}
```

OUTPUT

```
distinct words: 11, 'the' = 4, 'fox' = 3
top 3 words:
  the x4
  dog x3
  fox x3
leaderboard:
  1. Ravi  1005 pts  47 s
  2. Kiran  990 pts  60.1 s
  3. Meera  920 pts  49.75 s
  4. Asha  920 pts  51.2 s
  5. Dev  870 pts  48.8 s
```

MODULE 20 · ADVANCED

Modern C++ (C++11, C++14, C++17)

Minimum duration **6 days** | Level **Advanced**

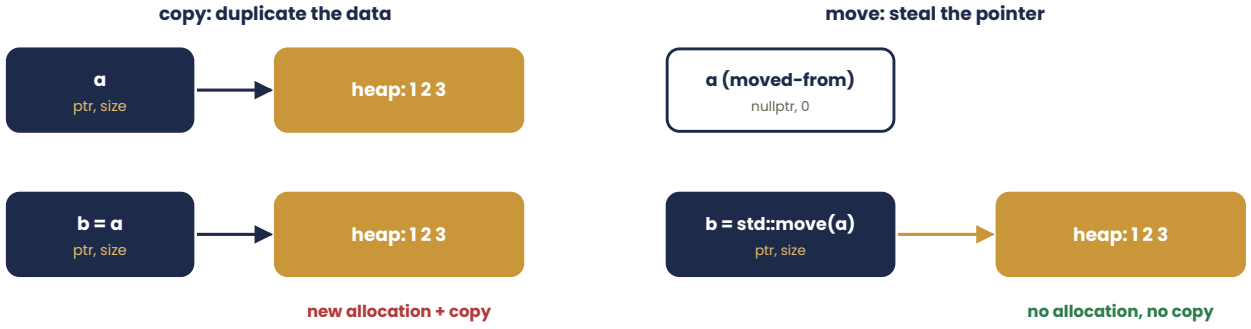
C++11 changed how C++ is written, so much that Bjarne Stroustrup said it "feels like a new language". Move semantics removed needless copies, smart pointers made manual delete almost obsolete, lambdas made algorithms pleasant to use, and C++14 and C++17 added auto everywhere, structured bindings, optional, variant and string_view. This module covers the features that appear in every modern codebase.

IN THIS MODULE

20.1 Move semantics
20.2 Smart pointers
20.3 Lambda expressions
20.4 Language features
20.5 Vocabulary types (C++17)
20.6 Other library additions

20.1 Move semantics

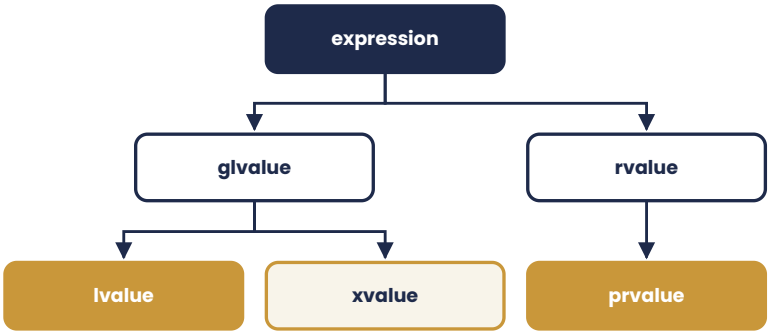
Before C++11, returning a big vector from a function or inserting a string into a container could copy all of its data, even when the source was a temporary about to be destroyed. **Move semantics** lets an object *steal* the resources of an object that is no longer needed: the heap buffer pointer is copied and the source is left empty. Moving a vector of a million elements costs the same as moving an empty one: three pointers.



COPYING DUPLICATES THE HEAP DATA; MOVING TRANSFERS OWNERSHIP OF IT

20.1.1 Lvalues, rvalues and value categories

Every expression has a type and a **value category**. An **lvalue** has an identity and a lasting location: a named variable, `*p`, `v[0]`, a function returning a reference. You can take its address. An **rvalue** is a temporary: a literal like `42`, the result of `a + b`, a function returning by value, `std::string("hi")`. Nobody else can refer to it after the full expression, so it is safe to steal from. C++11 splits rvalues further into **prvalues** (pure temporaries) and **xvalues** ("expiring" values, such as the result of `std::move(x)`); lvalues and xvalues together are **glvalues**.



VALUE CATEGORIES (XVALUE BELONGS TO BOTH GLVALUE AND RVALUE)

Expression	Category	Why
`x`, `v[2]`, `*ptr`, `"text"`	lvalue	has a name or a lasting location (string literals are arrays in static storage)
`42`, `a + b`, `f()` returning by value, `T{}`	prvalue	a temporary value
`std::move(x)`, `f()` returning `T&&`	xvalue	an object marked as expiring

20.1.2 Rvalue references &&

An **rvalue reference** `T&&` binds only to rvalues; an lvalue reference `T&` binds only to lvalues; `const T&` binds to both. Overloading on these lets a function know whether its argument is a temporary that it may steal from. Note that a **named** rvalue reference is itself an lvalue: inside the function, the parameter has a name, so it is an lvalue expression.

OVERLOADING ON VALUE CATEGORY main.cpp

```
#include <iostream>
#include <string>

void take(std::string& s)        { std::cout << "lvalue ref:  " << s << '\n'; }
void take(const std::string& s)  { std::cout << "const ref:   " << s << '\n'; }
void take(std::string&& s)       { std::cout << "rvalue ref:  " << s << " (may steal it)\n"; }

std::string make() { return "temporary"; }

int main() {
    std::string name = "Asha";
    const std::string city = "Mysuru";
    take(name);                 // lvalue
    take(city);                 // const lvalue
    take(make());               // prvalue
    take(name + "!");           // prvalue: result of operator+
    take(std::move(name));      // xvalue: we promise not to need name's value
    std::string&& r = make();   // an rvalue reference extends the temporary's lifetime
    take(r);                    // but r has a name: it is an lvalue!
    return 0;
}
```

OUTPUT

```
lvalue ref:  Asha
const ref:   Mysuru
rvalue ref:  temporary (may steal it)
rvalue ref:  Asha! (may steal it)
rvalue ref:  Asha (may steal it)
lvalue ref:  temporary
```

20.1.3 Move constructor and move assignment operator

A class becomes movable by defining a **move constructor** `T(T&& other)` and a **move assignment operator** `T& operator=(T&& other)`. They take over `other`'s resources and leave it in a **valid but unspecified** state (typically empty) that can still be destroyed or assigned to. Mark them `noexcept`: `std::vector` only moves elements during reallocation if the move constructor cannot throw (Module 16).

A CLASS WITH COPY AND MOVE OPERATIONS main.cpp

```
#include <algorithm>
#include <cstddef>
#include <iostream>
#include <utility>

class Buffer {
public:
    explicit Buffer(std::size_t n) : size_(n), data_(new int[n]{}) { std::cout << "  allocate " << n
<< '\n'; }
    ~Buffer() { delete[] data_; }

    Buffer(const Buffer& o) : size_(o.size_), data_(new int[o.size_]) {          // copy: deep copy
        std::copy(o.data_, o.data_ + size_, data_);
        std::cout << "  COPY " << size_ << " ints\n";
    }
    Buffer(Buffer&& o) noexcept : size_(o.size_), data_(o.data_) {               // move: steal
        o.size_ = 0;
        o.data_ = nullptr;                                                        // leave o empty
but valid
        std::cout << "  MOVE (no allocation)\n";
    }
```

```
    Buffer& operator=(const Buffer& o) {
        if (this != &o) { Buffer tmp(o); swap(tmp); }                          // copy-and-swap
        return *this;
    }
    Buffer& operator=(Buffer&& o) noexcept {
        if (this != &o) {
            delete[] data_;
            data_ = std::exchange(o.data_, nullptr);                           // take o's
pointer, null it
            size_ = std::exchange(o.size_, 0);
            std::cout << "  MOVE-ASSIGN\n";
        }
        return *this;
    }
    void swap(Buffer& o) noexcept { std::swap(size_, o.size_); std::swap(data_, o.data_); }
    std::size_t size() const { return size_; }
private:
    std::size_t size_;
    int* data_;
};

Buffer make_buffer(std::size_t n) { Buffer b(n); return b; }

int main() {
    std::cout << "Buffer a(1000):\n";
    Buffer a(1000);
    std::cout << "Buffer b = a:\n";
    Buffer b = a;                          // lvalue: copy
    std::cout << "Buffer c = std::move(a):\n";
    Buffer c = std::move(a);               // xvalue: move
    std::cout << "a.size() after move = " << a.size() << '\n';
    std::cout << "b = make_buffer(50):\n";
    b = make_buffer(50);                   // temporary: move assignment
    std::cout << "sizes: b " << b.size() << ", c " << c.size() << '\n';
    return 0;
}
```

OUTPUT

```
Buffer a(1000):
  allocate 1000
Buffer b = a:
  COPY 1000 ints
Buffer c = std::move(a):
  MOVE (no allocation)
a.size() after move = 0
b = make_buffer(50):
  allocate 50
  MOVE-ASSIGN
sizes: b 50, c 1000
```

NOTE

`std::exchange(obj, new_value)` (C++14, header `<utility>`) sets `obj` to `new_value` and returns the old value: exactly what a move operation needs.

20.1.4 std::move

`std::move(x)` does not move anything. It is just a cast to `T&&` that says "I no longer need the value of x; you may steal from it". The actual moving happens in the move constructor or move assignment that this cast selects. After moving from an object, only assign a new value to it or destroy it; do not rely on its

contents. Do not `std::move` a `const` object (the cast yields `const T&&`, which silently selects the copy constructor) and do not write `return std::move(local);` (it disables the return value optimisation, 20.1.6).

WHAT STD::MOVE DOES AND DOES NOT DO

main.cpp

```
#include <iostream>
#include <string>
#include <utility>
#include <vector>

int main() {
    std::string line = "a fairly long string that does not fit in the small-string buffer";
    std::vector<std::string> lines;
    lines.push_back(line);                      // copy: line still needed
    lines.push_back(std::move(line));           // move: line's buffer is taken over
    std::cout << "line after move: \"" << line << "\" (size " << line.size() << ")\n";
    line = "reused";                            // assigning a new value is fine
    std::cout << "line reassigned: " << line << '\n';

    std::vector<int> big(1'000'000, 7);
    const int* before = big.data();
    std::vector<int> other = std::move(big);
    std::cout << "same heap block after move? " << std::boolalpha << (other.data() == before)
              << ", big.size() = " << big.size() << '\n';

    const std::string fixed = "constant";
    std::string copy = std::move(fixed);        // const: silently copies!
    std::cout << "moved-from const still holds: " << fixed << '\n';

    std::string x = "left", y = "right";
    std::swap(x, y);                            // std::swap is implemented with three moves
    std::cout << x << ' ' << y << '\n';
    return 0;
}
```

OUTPUT

```
line after move: "" (size 0)
line reassigned: reused
same heap block after move? true, big.size() = 0
moved-from const still holds: constant
right left
```

20.1.5 Rule of Five and Rule of Zero

The **Rule of Five**: if a class needs a user-defined destructor, copy constructor, copy assignment, move constructor or move assignment (because it manages a resource directly), it probably needs all five. Declaring any of the copy operations or the destructor suppresses the implicit move operations, so the class silently falls back to copying. The **Rule of Zero** is the better goal: let members that already manage their resources (`std::string`, `std::vector`, `std::unique_ptr`) do the work, and declare **none** of the five. The compiler-generated versions then copy and move member by member, correctly.

You declare	Copy ctor / assign	Move ctor / assign
nothing	generated	generated
destructor	generated (deprecated)	**not generated**: moves become copies
copy constructor or copy assignment	the one you wrote; other generated (deprecated)	**not generated**

You declare	Copy ctor / assign	Move ctor / assign
move constructor or move assignment	**deleted**	the one you wrote

RULE OF ZERO, AND HOW A DESTRUCTOR DISABLES MOVES

main.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Student {                          // Rule of Zero: no special members declared
    std::string name;
    std::vector<int> marks;
    std::unique_ptr<int> locker;          // move-only member makes Student move-only
};

struct Logged {                           // declares a destructor only ...
    std::vector<int> data = std::vector<int>(1000);
    ~Logged() {}                          // ... so the implicit move is NOT generated
};

int main() {
    Student a{"Asha", {90, 85}, std::make_unique<int>(42)};
    Student b = std::move(a);             // generated move: moves every member
    std::cout << b.name << ' ' << b.marks.size() << ' ' << *b.locker
              << " | a.name empty? " << std::boolalpha << a.name.empty() << '\n';
    // Student c = b;                     error: unique_ptr cannot be copied

    Logged x;
    const int* before = x.data.data();
    Logged y = std::move(x);              // falls back to the COPY constructor
    std::cout << "Logged was moved? " << (y.data.data() == before) << " (it was copied)\n";
    return 0;
}
```

OUTPUT

```
Asha 2 42 | a.name empty? true
Logged was moved? false (it was copied)
```

20.1.6 Copy elision and return value optimisation (RVO)

When a function returns a local object or a temporary by value, the compiler may construct it **directly in the caller's variable**, with no copy and no move at all. Returning a prvalue (`return Widget(args);`) is **guaranteed** elision since C++17. Returning a named local (`Widget w; ...; return w;`) is **named RVO**: allowed, and done by all major compilers in simple cases; when it is not possible the local is automatically treated as an rvalue and moved. So returning large objects by value is cheap; there is no need for output parameters.

COUNTING COPIES AND MOVES ON RETURN main.cpp

```cpp
#include <iostream>

struct Tracer {
    Tracer() { std::cout << "  construct\n"; }
    Tracer(const Tracer&) { std::cout << "  copy\n"; }
    Tracer(Tracer&&) noexcept { std::cout << "  move\n"; }
    ~Tracer() { std::cout << "  destroy\n"; }
};

Tracer make_prvalue() { return Tracer(); }                // guaranteed elision (C++17)
Tracer make_named() { Tracer t; return t; }               // NRVO
Tracer make_choice(bool b) {                              // two different locals: NRVO impossible
    Tracer x, y;
    if (b) return x;
    return y;                                             // implicitly moved
}

int main() {
    std::cout << "prvalue:\n";
    Tracer a = make_prvalue();
    std::cout << "named:\n";
    Tracer b = make_named();
    std::cout << "choice:\n";
    Tracer c = make_choice(true);
    std::cout << "end of main\n";
    return 0;
}
```

OUTPUT

```
prvalue:
  construct
named:
  construct
choice:
  construct
  construct
  move
  destroy
  destroy
end of main
  destroy
  destroy
  destroy
```

20.1.7 Perfect forwarding with forwarding references and std::forward

In a template, `T&&` where `T` is deduced is not an rvalue reference but a **forwarding reference** (also called universal reference): it binds to anything. If the argument is an lvalue, T becomes `X&` (and `X& &&` collapses to `X&`); if it is an rvalue, T becomes `X`. `std::forward<T>(arg)` then passes the argument on with its original category: lvalues stay lvalues (copied), rvalues stay rvalues (moved). This is how `make_unique`, `emplace_back` and wrapper functions pass arguments through without extra copies.

FORWARDING REFERENCES AND STD::FORWARD

main.cpp

```
#include <iostream>
#include <string>
#include <utility>

void sink(const std::string& s) { std::cout << "  copied: " << s << '\n'; }
void sink(std::string&& s)      { std::cout << "  moved:  " << s << '\n'; }

template <typename T>
void bad_wrapper(T&& arg) { sink(arg); }                    // arg has a name: always an lvalue

template <typename T>
void good_wrapper(T&& arg) { sink(std::forward<T>(arg)); }  // keeps the original category

template <typename F, typename... Args>
void log_call(const char* name, F&& f, Args&&... args) {    // a typical forwarding wrapper
    std::cout << "calling " << name << '\n';
    std::forward<F>(f)(std::forward<Args>(args)...);
}

int main() {
    std::string s = "lvalue string";
    std::cout << "bad_wrapper:\n";
    bad_wrapper(s);
    bad_wrapper(std::string("temporary"));      // should move, but copies
    std::cout << "good_wrapper:\n";
    good_wrapper(s);
    good_wrapper(std::string("temporary"));
    log_call("sink", [](std::string&& x) { std::cout << "  lambda got " << x << '\n'; },
std::string("data"));
    return 0;
}
```

OUTPUT

```
bad_wrapper:
  copied: lvalue string
  copied: temporary
good_wrapper:
  copied: lvalue string
  moved:  temporary
calling sink
  lambda got data
```

Argument	T deduced as	Parameter type `T&&` becomes	`std::forward<T>(arg)` is
lvalue `std::string`	`std::string&`	`std::string&` (collapse)	lvalue
rvalue `std::string`	`std::string`	`std::string&&`	rvalue

20.2 Smart pointers

A **smart pointer** is an object that owns a heap object and deletes it automatically in its destructor (RAII). With smart pointers, raw `new` and `delete` disappear from ordinary code, and with them memory leaks, double deletes and most dangling pointers. The header is `<memory>`.

20.2.1 std::unique_ptr and make_unique

`std::unique_ptr<T>` is the **sole owner** of an object. It cannot be copied (two owners would delete twice), only moved, which transfers ownership. It costs nothing compared with a raw pointer: same size, no counting. Create it with `std::make_unique<T>(args...)` (C++14). Use `*` and `->` as with a raw pointer, `get()` for a non-owning raw pointer, `reset()` to delete early, `release()` to give up ownership. `unique_ptr<T[]>` manages arrays.

UNIQUE_PTR: SINGLE OWNERSHIP main.cpp

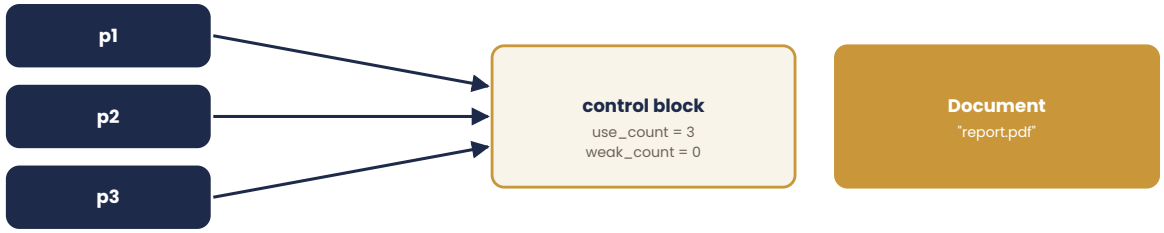
```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Engine {
    std::string model;
    explicit Engine(std::string m) : model(std::move(m)) { std::cout << "  Engine " << model << "
created\n"; }
    ~Engine() { std::cout << "  Engine " << model << " destroyed\n"; }
};

std::unique_ptr<Engine> build(const std::string& model) {     // factory returns ownership
    return std::make_unique<Engine>(model);
}
void inspect(const Engine& e) { std::cout << "  inspecting " << e.model << '\n'; }   // just uses it
void scrap(std::unique_ptr<Engine> e) { std::cout << "  scrapping " << e->model << '\n'; }   // takes
ownership

int main() {
    auto e1 = build("V8");
    inspect(*e1);
    // auto e2 = e1;                      error: unique_ptr is not copyable
    auto e2 = std::move(e1);              // ownership moves to e2
    std::cout << "e1 is " << (e1 ? "set" : "empty") << ", e2 holds " << e2->model << '\n';
    scrap(std::move(e2));                 // destroyed at the end of scrap()
    std::cout << "after scrap\n";

    std::vector<std::unique_ptr<Engine>> garage;
    garage.push_back(std::make_unique<Engine>("EV-1"));
    garage.push_back(build("Diesel"));
    garage[0].reset();                    // destroy one early
    std::cout << "sizeof(unique_ptr) = " << sizeof(std::unique_ptr<Engine>) << " = sizeof(Engine*)\n";
    auto arr = std::make_unique<int[]>(5);
    arr[2] = 7;
    std::cout << "end of main\n";
    return 0;                             // remaining Engines destroyed automatically
}
```

OUTPUT

```
  Engine V8 created
  inspecting V8
e1 is empty, e2 holds V8
  scrapping V8
  Engine V8 destroyed
after scrap
  Engine EV-1 created
  Engine Diesel created
  Engine EV-1 destroyed
sizeof(unique_ptr) = 8 = sizeof(Engine*)
end of main
  Engine Diesel destroyed
```

20.2.2 std::shared_ptr, make_shared and reference counting

`std::shared_ptr<T>` allows **shared ownership**: several shared_ptrs point to the same object, and a **reference count** in a separate **control block** tracks how many there are. Copying increments the count, destroying decrements it, and the object is deleted when the count reaches zero. `std::make_shared` allocates the object and the control block in one allocation. The count is updated atomically (thread-safe), which makes copying a shared_ptr noticeably more expensive than copying a raw pointer, so pass it by `const&` or pass the object by reference when no ownership is needed.



MAKE_SHARED PUTS THE CONTROL BLOCK AND THE OBJECT IN ONE ALLOCATION

SHARED_PTR REFERENCE COUNTING main.cpp

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Document {
    std::string name;
    explicit Document(std::string n) : name(std::move(n)) { std::cout << "open " << name << '\n'; }
    ~Document() { std::cout << "close " << name << '\n'; }
};

struct Window {
    std::shared_ptr<Document> doc;         // several windows may show the same document
};

int main() {
    auto doc = std::make_shared<Document>("report.pdf");
    std::cout << "count = " << doc.use_count() << '\n';
    {
        Window w1{doc}, w2{doc};           // two more owners
        std::cout << "count with 2 windows = " << doc.use_count() << '\n';
        std::vector<std::shared_ptr<Document>> recent = {doc};
        std::cout << "count with history = " << doc.use_count() << '\n';
    }                                      // windows and history gone
    std::cout << "count after scope = " << doc.use_count() << '\n';
    std::shared_ptr<Document> last = doc;
    doc.reset();                           // one owner gives up
    std::cout << "doc released, last.use_count() = " << last.use_count() << '\n';
    last = nullptr;                        // the last owner gives up: the document closes now
    std::cout << "end of main\n";
    return 0;
}
```

OUTPUT

```
open report.pdf
count = 1
count with 2 windows = 3
count with history = 4
count after scope = 1
doc released, last.use_count() = 1
close report.pdf
end of main
```

20.2.3 std::weak_ptr and breaking reference cycles

If two objects hold `shared_ptr`s to each other, their counts never reach zero and both leak: a **reference cycle**. `std::weak_ptr` is a non-owning observer of a shared object: it does not increase the use count. To use the object, call `lock()`, which returns a `shared_ptr` (empty if the object is already gone); `expired()` checks without locking. Typical uses: back-pointers (child to parent), observers, and caches.

WEAK_PTR BREAKS THE CYCLE main.cpp

```
#include <iostream>
#include <memory>
#include <string>

struct Employee;
struct Team {
    std::string name;
    std::shared_ptr<Employee> lead;
```

```
    ~Team() { std::cout << "  ~Team " << name << '\n'; }
};
struct EmployeeBad {
    std::shared_ptr<Team> team;            // shared back-pointer: creates a cycle
};
struct Employee {
    std::string name;
    std::weak_ptr<Team> team;              // weak back-pointer: no cycle
    ~Employee() { std::cout << "  ~Employee " << name << '\n'; }
};

int main() {
    std::cout << "with weak_ptr back-pointer:\n";
    {
        auto t = std::make_shared<Team>();
        t->name = "Platform";
        auto e = std::make_shared<Employee>();
        e->name = "Asha";
        t->lead = e;
        e->team = t;
        if (auto team = e->team.lock()) std::cout << "  " << e->name << " leads " << team->name <<
'\n';
        std::cout << "  team use_count = " << t.use_count() << '\n';
    }                                      // both destroyed
    std::weak_ptr<Team> observer;
    {
        auto t = std::make_shared<Team>();
        t->name = "Temp";
        observer = t;
        std::cout << "expired while t alive? " << std::boolalpha << observer.expired() << '\n';
    }
    std::cout << "expired after t is gone? " << observer.expired() << ", lock() gives " <<
(observer.lock() ? "object" : "nullptr") << '\n';
    return 0;
}
```

OUTPUT

```
with weak_ptr back-pointer:
  Asha leads Platform
  team use_count = 1
  ~Team Platform
  ~Employee Asha
expired while t alive? false
  ~Team Temp
expired after t is gone? true, lock() gives nullptr
```

Had `Employee::team` been a `shared_ptr`, the team would own the employee and the employee the team; at the end of the block each count would drop from 2 to 1 and neither destructor would ever run. LeakSanitizer (`-fsanitize=address`) reports exactly this kind of leak.

20.2.4 Custom deleters

By default a smart pointer calls `delete`. A **custom deleter** lets it manage any resource that has a "release" function: a `FILE*` from `fopen` (close with `fclose`), a socket, a database handle, memory from a C library. For `unique_ptr` the deleter type is a template argument; for `shared_ptr` it is just a constructor argument.

SMART POINTERS FOR C RESOURCES

main.cpp

```cpp
#include <cstdio>
#include <iostream>
#include <memory>

struct FileCloser {
    void operator()(std::FILE* f) const {
        std::cout << "  fclose called\n";
        std::fclose(f);
    }
};
using FilePtr = std::unique_ptr<std::FILE, FileCloser>;

int main() {
    {
        FilePtr f(std::fopen("log.txt", "w"));
        if (f) std::fputs("written through a smart pointer\n", f.get());
        std::cout << "leaving scope\n";
    }                                                   // fclose runs automatically

    auto closer = [](std::FILE* f) { std::cout << "  lambda deleter\n"; std::fclose(f); };
    std::unique_ptr<std::FILE, decltype(closer)> in(std::fopen("log.txt", "r"), closer);
    char line[64];
    if (std::fgets(line, sizeof line, in.get())) std::cout << "read: " << line;

    std::shared_ptr<int> counted(new int[4]{1, 2, 3, 4}, [](int* p) { std::cout << "  delete[] by
shared_ptr\n"; delete[] p; });
    std::cout << "end of main\n";
    return 0;
}
```

OUTPUT

```
leaving scope
  fclose called
read: written through a smart pointer
end of main
  delete[] by shared_ptr
  lambda deleter
```

20.2.5 Choosing the right smart pointer

Situation	Use
One clear owner (the common case)	`std::unique_ptr` created with `make_unique`
Ownership genuinely shared, lifetime decided by the last user	`std::shared_ptr` created with `make_shared`
Observe a shared object without keeping it alive; back-pointers; caches	`std::weak_ptr`
A function only uses the object, does not own it	a reference `T&`, or a raw pointer `T*` if it may be null
Polymorphic objects in a container	`std::vector<std::unique_ptr<Base>>`
Non-memory resource (file, socket)	`unique_ptr` with a custom deleter, or a small RAII class

Prefer unique_ptr: it is free and expresses ownership clearly, and it converts to a shared_ptr later if needed. Use make_unique and make_shared instead of new. Pass smart pointers to functions only when the function takes part in ownership; otherwise pass a reference.

20.3 Lambda expressions

20.3.1 Syntax [capture](params) -> ret { body }

A **lambda expression** creates an unnamed function object on the spot. Its parts: the **capture list** `[ ]` (which local variables it can use), the **parameter list** `( )`, an optional **return type** `-> type` (usually deduced), and the **body**. The compiler turns each lambda into a unique class with an `operator()`, with the captured variables as members.

```
[threshold] (int x) -> bool { return x > threshold; }
```

capture | parameters | return type | body

THE PARTS OF A LAMBDA

LAMBDA BASICS

main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    auto square = [](int x) { return x * x; };               // stored in a variable
    std::cout << "square(7) = " << square(7) << '\n';

    auto divide = [](double a, double b) -> double {          // explicit return type
        if (b == 0) return 0;                                 // int 0 converted to double
        return a / b;
    };
    std::cout << "divide(7, 2) = " << divide(7, 2) << '\n';

    std::cout << "called immediately: " << [](int a, int b) { return a + b; }(3, 4) << '\n';

    std::vector<int> v = {5, 12, 3, 18, 7};
    int threshold = 6;
    auto big = std::count_if(v.begin(), v.end(), [threshold](int x) { return x > threshold; });
    std::cout << big << " values above " << threshold << '\n';

    // What the compiler generates for [threshold](int x) { return x > threshold; } is roughly:
    struct Lambda { int threshold; bool operator()(int x) const { return x > threshold; } };
    std::cout << "hand-written equivalent: " << std::count_if(v.begin(), v.end(), Lambda{threshold})
<< '\n';
    return 0;
}
```

OUTPUT

```
square(7) = 49
divide(7, 2) = 3.5
called immediately: 7
3 values above 6
hand-written equivalent: 3
```

20.3.2 Capture by value, by reference, this, init-capture

Capture	Meaning
`[]`	capture nothing
`[x]`	copy of x, made when the lambda is created
`[&x]`	reference to x: sees later changes, may modify it
`[=]` / `[&]`	capture every used local by value / by reference
`[=, &total]`	everything by value except total
`[this]` / `[*this]`	the current object by pointer / by copy (C++17)
`[n = expr]`, `[p = std::move(ptr)]`	init-capture (C++14): a new member initialised from any expression, including move-only values

DIFFERENT KINDS OF CAPTURE

main.cpp

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Counter {
public:
    auto incrementer() { return [this] { ++count_; }; }      // captures the object pointer
    int count() const { return count_; }
private:
    int count_ = 0;
};

int main() {
    int x = 10;
    auto by_value = [x] { return x; };
    auto by_ref = [&x] { return x; };
    x = 99;
    std::cout << "by value sees " << by_value() << ", by reference sees " << by_ref() << '\n';

    int total = 0;
    std::vector<int> v = {1, 2, 3, 4};
    for (int n : v) [&total](int k) { total += k; }(n);       // modifies total through the reference
    std::cout << "total = " << total << '\n';

    Counter c;
    auto inc = c.incrementer();
    inc(); inc(); inc();
    std::cout << "counter = " << c.count() << '\n';

    auto data = std::make_unique<std::string>("owned by the lambda");
    auto keeper = [p = std::move(data), len = 0]() mutable { len = static_cast<int>(p->size()); return
*p + " (" + std::to_string(len) + " chars)"; };
    std::cout << keeper() << "; data is now " << (data ? "set" : "null") << '\n';
    return 0;
}
```

OUTPUT

```
by value sees 10, by reference sees 99
total = 10
counter = 3
owned by the lambda (19 chars); data is now null
```

WATCH OUT

A lambda that captures by reference must not outlive the variables it refers to. Returning `[&local] { ... }` from a function, or storing it for later, leaves a dangling reference. When the lambda will be called later (callbacks, threads), capture by value.

20.3.3 mutable lambdas

The `operator()` of a lambda is `const` by default, so variables captured by value cannot be modified inside it. Adding `mutable` lifts that restriction: the lambda then modifies **its own copy**, which persists between calls to the same lambda object, while the original variable is unaffected.

A LAMBDA WITH ITS OWN STATE main.cpp

```
#include <iostream>

int main() {
    int start = 100;
    auto next_id = [id = start]() mutable { return id++; };   // its own counter
    std::cout << next_id() << ' ' << next_id() << ' ' << next_id() << '\n';
    std::cout << "start is still " << start << '\n';

    // auto bad = [start]() { start++; };   error: increment of read-only variable 'start'
    auto copy = next_id;                     // copying the lambda copies its state
    std::cout << "original " << next_id() << ", copy " << copy() << '\n';
    return 0;
}
```

OUTPUT

```
100 101 102
start is still 100
original 103, copy 103
```

20.3.4 Generic lambdas with auto parameters (C++14)

A lambda whose parameters are declared `auto` is a **generic lambda**: its `operator()` is a template, so one lambda works with any argument types. C++20 also allows an explicit template parameter list: `[]<typename T>(const std::vector<T>& v) { ... }`.

GENERIC LAMBDAS main.cpp

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    auto add = [](auto a, auto b) { return a + b; };
    std::cout << add(2, 3) << ' ' << add(1.5, 2.25) << ' ' << add(std::string("C"), "++") << '\n';

    auto print_all = [](const auto& container) {
        for (const auto& x : container) std::cout << x << ' ';
        std::cout << '\n';
    };
    print_all(std::vector<int>{1, 2, 3});
    print_all(std::vector<std::string>{"generic", "lambda"});
    print_all(std::string("abc"));

    auto first = []<typename T>(const std::vector<T>& v) { return v.empty() ? T{} : v.front(); };   // C++20
    std::cout << first(std::vector<double>{2.5, 1.0}) << ' ' << first(std::vector<int>{}) << '\n';
    return 0;
}
```

OUTPUT

```
5 3.75 C++
1 2 3
generic lambda
a b c
2.5 0
```

20.3.5 Lambdas with STL algorithms

Lambdas are what makes the STL algorithms pleasant: the predicate or operation is written right where it is used.

ALGORITHMS WITH LAMBDAS main.cpp

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <string>
#include <vector>

struct Order { std::string customer; double amount; bool paid; };

int main() {
    std::vector<Order> orders = {{"Asha", 1200, true}, {"Ravi", 450, false}, {"Meera", 3100, true},
{"Kiran", 800, false}, {"Dev", 2500, true}};

    double min_amount = 1000;
    auto large = std::count_if(orders.begin(), orders.end(), [min_amount](const Order& o) { return
o.amount >= min_amount; });
    double unpaid = std::accumulate(orders.begin(), orders.end(), 0.0,
                                    [](double sum, const Order& o) { return o.paid ? sum : sum +
o.amount; });
    std::sort(orders.begin(), orders.end(), [](const Order& a, const Order& b) { return a.amount >
b.amount; });
    auto first_unpaid = std::find_if(orders.begin(), orders.end(), [](const Order& o) { return
!o.paid; });

    std::cout << large << " large orders, unpaid total " << unpaid << ", biggest unpaid: " <<
first_unpaid->customer << '\n';
    std::vector<std::string> names;
    std::transform(orders.begin(), orders.end(), std::back_inserter(names), [](const Order& o) {
return o.customer; });
    for (const auto& n : names) std::cout << n << ' ';
    std::cout << '\n';
    std::erase_if(orders, [](const Order& o) { return o.paid; });
    std::cout << orders.size() << " unpaid orders remain\n";
    return 0;
}
```

OUTPUT

```
3 large orders, unpaid total 1250, biggest unpaid: Kiran
Meera Dev Asha Kiran Ravi
2 unpaid orders remain
```

20.3.6 std::function and std::bind

Every lambda has its own unique type, so lambdas of different kinds cannot be stored in one vector or passed to a non-template function directly. `std::function<R(Args...)>` (header `<functional>`) is a type-erased wrapper that can hold **any** callable with that signature: a function, a lambda (with any captures), a functor or a bound member function. The flexibility costs a possible heap allocation and an indirect call. `std::bind` creates a callable with some arguments fixed; in modern code a lambda is usually clearer.

STD::FUNCTION HOLDS ANY CALLABLE main.cpp

```
#include <functional>
#include <iostream>
#include <map>
```

```cpp
#include <string>
#include <vector>

int add(int a, int b) { return a + b; }
struct Multiply { int operator()(int a, int b) const { return a * b; } };

class Button {
public:
    void on_click(std::function<void()> handler) { handlers_.push_back(std::move(handler)); }
    void click() const { for (const auto& h : handlers_) h(); }
private:
    std::vector<std::function<void()>> handlers_;
};

int main() {
    std::map<std::string, std::function<int(int, int)>> ops = {
        {"add", add},                                   // a plain function
        {"mul", Multiply{}},                            // a functor
        {"pow", [](int b, int e) { int r = 1; while (e--) r *= b; return r; }},   // a lambda
    };
    for (const auto& [name, op] : ops) std::cout << name << "(3, 4) = " << op(3, 4) << '\n';

    int clicks = 0;
    Button b;
    b.on_click([&clicks] { ++clicks; });
    b.on_click([] { std::cout << "button pressed\n"; });
    b.click();
    b.click();
    std::cout << "clicks = " << clicks << '\n';

    using namespace std::placeholders;
    auto add10 = std::bind(add, _1, 10);                // fix the second argument
    auto add10_lambda = [](int x) { return add(x, 10); };  // the same, more readable
    std::cout << add10(5) << ' ' << add10_lambda(5) << '\n';

    std::function<void()> empty;
    try { empty(); } catch (const std::bad_function_call& e) { std::cout << "empty std::function: " <<
e.what() << '\n'; }
    return 0;
}
```

OUTPUT

```
add(3, 4) = 7
mul(3, 4) = 12
pow(3, 4) = 81
button pressed
button pressed
clicks = 2
15 15
empty std::function: bad_function_call
```

20.4 Language features

20.4.1 auto and decltype(auto)

`auto` lets the compiler deduce a variable's type from its initializer, using the template deduction rules: top-level `const` and references are dropped, so write `const auto&` or `auto&` when you want a reference. `decltype(expr)` gives the exact declared type of an expression. `decltype(auto)` deduces with the decltype rules and keeps references: used mostly for return types of forwarding functions.

AUTO AND DECLTYPE(AUTO)

main.cpp

```cpp
#include <iostream>
#include <map>
#include <string>
#include <type_traits>
#include <vector>

std::vector<int> data = {1, 2, 3};
auto get_copy(std::size_t i) { return data[i]; }                // returns int
decltype(auto) get_ref(std::size_t i) { return data[i]; }      // returns int& (keeps the reference)

int main() {
    auto i = 42;                        // int
    auto d = 3.5;                       // double
    auto s = std::string("text");       // std::string
    auto list = {1, 2, 3};              // std::initializer_list<int>
    const int ci = 7;
    auto a = ci;                        // int: top-level const dropped
    const auto& r = ci;                 // const int&
    std::map<std::string, std::vector<int>> scores = {{"Asha", {90, 85}}};
    for (const auto& [name, marks] : scores)     // instead of std::pair<const std::string,
std::vector<int>>
        std::cout << name << " has " << marks.size() << " marks\n";

    std::cout << std::boolalpha;
    std::cout << "a is const? " << std::is_const_v<decltype(a)> << ", r is a reference? " <<
std::is_reference_v<decltype(r)> << '\n';
    std::cout << "list size " << list.size() << ", i + d = " << i + d << ", s = " << s << '\n';
    get_ref(0) = 100;                   // assigns through the returned reference
    std::cout << "data[0] = " << data[0] << ", get_copy returns a reference? " <<
std::is_reference_v<decltype(get_copy(0))> << '\n';
    return 0;
}
```

OUTPUT

```
Asha has 2 marks
a is const? false, r is a reference? true
list size 3, i + d = 45.5, s = text
data[0] = 100, get_copy returns a reference? false
```

TIP

Use `auto` when the type is obvious from the right-hand side (`auto p = std::make_unique<Shape>()`) or very long (iterators), not to hide important types. Watch out for `auto x = {5};` , which is an initializer_list, not an int.

20.4.2 Range-based for loops

`for (declaration : range)` iterates over anything with `begin()` / `end()` : containers, arrays, initializer lists, strings, views. Choose the declaration deliberately: `auto x` copies each element, `auto& x` modifies in place, `const auto& x` reads without copying. C++20 adds an init-statement: `for (auto v = make(); auto x : v)` .

RANGE-BASED FOR LOOPS

main.cpp

```cpp
#include <iostream>
#include <map>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> names = {"asha", "ravi"};
    for (auto& n : names) n[0] = static_cast<char>(n[0] - 'a' + 'A');   // modify in place
    for (const auto& n : names) std::cout << n << ' ';                    // read, no copies
    std::cout << '\n';

    int arr[] = {3, 1, 4};
    for (int x : arr) std::cout << x * 10 << ' ';
    for (char c : std::string("ok")) std::cout << c << '!';
    std::cout << '\n';

    std::map<std::string, int> stock = {{"pen", 5}, {"ink", 2}};
    for (auto& [item, qty] : stock) qty *= 2;                                  // structured binding
    for (const auto& [item, qty] : stock) std::cout << item << '=' << qty << ' ';
    std::cout << '\n';

    for (std::size_t i = 0; const auto& n : names) std::cout << i++ << ':' << n << ' ';   // C++20 init-statement
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
Asha Ravi
30 10 40 o!k!
ink=4 pen=10
0:Asha 1:Ravi
```

20.4.3 nullptr

`nullptr` (type `std::nullptr_t`) is the null pointer literal. It replaces `NULL` and `0`, which are integers and pick the wrong overload: with `f(int)` and `f(char*)`, the call `f(NULL)` calls `f(int)` (or is ambiguous), while `f(nullptr)` calls the pointer version.

NULLPTR PICKS THE POINTER OVERLOAD main.cpp

```cpp
#include <cstddef>
#include <iostream>

void f(int)   { std::cout << "f(int)\n"; }
void f(char*) { std::cout << "f(char*)\n"; }

int main() {
    f(0);              // int
    f(nullptr);        // pointer: what we meant
    char* p = nullptr;
    if (!p) std::cout << "p is null\n";
    std::cout << "sizeof(nullptr) = " << sizeof(nullptr) << '\n';
    return 0;
}
```

OUTPUT

```
f(int)
f(char*)
p is null
sizeof(nullptr) = 8
```

20.4.4 constexpr and consteval / constinit (C++20)

`constexpr` on a variable means it is a compile-time constant. On a function it means the function *can* be evaluated at compile time when its arguments are constants (and at run time otherwise). C++14 and later allow loops and local variables in constexpr functions; C++20 even allows `std::vector` and `std::string` inside them (as long as the memory is freed before the end). `consteval` (C++20) makes an **immediate function** that *must* run at compile time. `constinit` (C++20) requires a static or global variable to be initialised at compile time, but unlike constexpr it stays modifiable.

COMPUTING AT COMPILE TIME

main.cpp

```
#include <array>
#include <iostream>

constexpr long long factorial(int n) {          // compile time or run time
    long long r = 1;
    for (int i = 2; i <= n; ++i) r *= i;
    return r;
}
consteval int square(int x) { return x * x; }   // compile time only

constexpr std::array<int, 10> make_squares() {
    std::array<int, 10> a{};
    for (int i = 0; i < 10; ++i) a[i] = i * i;
    return a;
}

constinit int counter = 10;                     // initialised at compile time, modifiable later

int main() {
    constexpr long long f10 = factorial(10);     // computed by the compiler
    static_assert(f10 == 3628800, "factorial is wrong");
    int n = 0;
    std::cout << "enter n: ";
    std::cin >> n;
    std::cout << "factorial(" << n << ") = " << factorial(n) << " (computed at run time)\n";

    constexpr int s = square(12);
    // int bad = square(n);                       error: n is not a constant expression
    constexpr auto table = make_squares();        // a whole lookup table built at compile time
    int arr[square(3)];                           // usable as an array size
    counter += 5;
    std::cout << "f10 = " << f10 << ", s = " << s << ", table[7] = " << table[7] << ", arr has " <<
sizeof(arr) / sizeof(int) << " ints, counter = " << counter << '\n';
    return 0;
}
```

CONSOLE (user input in gold)

```
enter n: 15
factorial(15) = 1307674368000 (computed at run time)
f10 = 3628800, s = 144, table[7] = 49, arr has 9 ints, counter = 15
```

20.4.5 Uniform initialization and std::initializer_list

C++11 **brace initialization** `T x{args}` works for every kind of type: built-ins, aggregates, classes, containers. It also forbids **narrowing** conversions (`int x{2.5}` is an error). A constructor taking `std::initializer_list<T>` lets your class be initialized from a braced list like a container, and is preferred whenever braces are used, which is why `std::vector<int> v{3, 1}` holds 3 and 1 while `std::vector<int> v(3, 1)` holds three 1s.

BRACE INITIALIZATION AND INITIALIZER_LIST

main.cpp

```
#include <initializer_list>
#include <iostream>
#include <string>
#include <vector>

class Polynomial {
public:
    Polynomial(std::initializer_list<int> coeffs) : c_(coeffs) {}     // Polynomial p{1, 0, 2};
    int at(int x) const {
        int result = 0;
        for (auto it = c_.rbegin(); it != c_.rend(); ++it) result = result * x + *it;   // Horner's
method
        return result;
    }
    std::size_t terms() const { return c_.size(); }
private:
    std::vector<int> c_;                                              // c_[i] is the coefficient of
x^i
};

struct Point { int x, y; };

int main() {
    int a{5};
    double d{2.5};
    std::string s{"brace"};
    Point p{3, 4};
    int zero{};                                                       // value-initialised: 0
    std::vector<int> v1{3, 1};                                        // initializer_list: {3, 1}
    std::vector<int> v2(3, 1);                                        // constructor (count, value):
{1, 1, 1}
    Polynomial poly{1, 0, 2};                                         // 1 + 0x + 2x^2
    std::cout << a << ' ' << d << ' ' << s << ' ' << p.x + p.y << ' ' << zero << '\n';
    std::cout << "v1 size " << v1.size() << ", v2 size " << v2.size() << '\n';
    std::cout << "poly has " << poly.terms() << " terms, poly(3) = " << poly.at(3) << '\n';
    for (int x : {2, 4, 8}) std::cout << x << ' ';                    // initializer_list in a loop
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
5 2.5 brace 7 0
v1 size 2, v2 size 3
poly has 3 terms, poly(3) = 19
2 4 8
```

20.4.6 Structured bindings

C++17 **structured bindings** unpack an aggregate, a pair, a tuple or an array into named variables: `auto [a, b] = expr;`. Add `&` to bind to the original elements instead of a copy. They make map loops and multi-value returns very readable.

STRUCTURED BINDINGS

main.cpp

```cpp
#include <iostream>
#include <map>
#include <string>
#include <tuple>

struct Result { bool ok; int value; std::string message; };
Result parse(const std::string& s) {
    try { return {true, std::stoi(s), "ok"}; }
    catch (...) { return {false, 0, "not a number: " + s}; }
}

int main() {
    auto [ok, value, msg] = parse("123");
    std::cout << ok << ' ' << value << ' ' << msg << '\n';
    auto [ok2, value2, msg2] = parse("12x");          // stoi stops at x: still ok
    std::cout << ok2 << ' ' << value2 << ' ' << msg2 << '\n';
    auto [ok3, value3, msg3] = parse("abc");
    std::cout << ok3 << ' ' << value3 << ' ' << msg3 << '\n';

    std::map<std::string, int> ages = {{"Asha", 20}};
    auto [it, inserted] = ages.insert({"Ravi", 22});
    std::cout << it->first << " inserted? " << std::boolalpha << inserted << '\n';
    for (auto& [name, age] : ages) ++age;              // reference binding modifies the map
    for (const auto& [name, age] : ages) std::cout << name << ' ' << age << '\n';

    int arr[3] = {7, 8, 9};
    auto& [x, y, z] = arr;
    z = 90;
    std::cout << "arr[2] = " << arr[2] << '\n';
    return 0;
}
```

OUTPUT

```
1 123 ok
1 12 ok
0 0 not a number: abc
Ravi inserted? true
Asha 21
Ravi 23
arr[2] = 90
```

20.4.7 if and switch with initializers

C++17 lets `if` and `switch` start with an initializer statement: `if (init; condition)`. The variable exists only inside the if/else (or switch), which keeps scopes tight and avoids leaking temporary names into the surrounding code.

IF AND SWITCH WITH INITIALIZERS

main.cpp

```
#include <iostream>
#include <map>
#include <string>

enum class Status { Active, Suspended, Closed };
Status lookup(int id) { return id % 3 == 0 ? Status::Closed : id % 2 == 0 ? Status::Suspended :
Status::Active; }

int main() {
    std::map<std::string, int> stock = {{"pen", 12}, {"ink", 0}};
    for (const char* item : {"pen", "ink", "glue"}) {
        if (auto it = stock.find(item); it == stock.end())          // it is only visible in this
if/else
            std::cout << item << ": not sold here\n";
        else if (it->second == 0)
            std::cout << item << ": out of stock\n";
        else
            std::cout << item << ": " << it->second << " left\n";
    }
    for (int id : {7, 8, 9}) {
        switch (Status s = lookup(id); s) {
        case Status::Active:    std::cout << id << " active\n"; break;
        case Status::Suspended: std::cout << id << " suspended\n"; break;
        case Status::Closed:    std::cout << id << " closed\n"; break;
        }
    }
    return 0;
}
```

OUTPUT

```
pen: 12 left
ink: out of stock
glue: not sold here
7 active
8 suspended
9 closed
```

20.4.8 Attributes: [[nodiscard]], [[maybe_unused]], [[deprecated]], [[likely]]

Attributes in double square brackets give the compiler extra information. They do not change what correct code does, but they enable useful warnings and optimisations.

Attribute	Since	Effect
`[[nodiscard]]`, `[[nodiscard("reason")]]`	C++17 / 20	warn if the return value of the function (or of any function returning this type) is ignored
`[[maybe_unused]]`	C++17	suppress "unused variable/parameter" warnings
`[[deprecated("use g()")]]`	C++14	warn whenever the entity is used
`[[fallthrough]]`	C++17	mark an intentional fall-through in a switch
`[[likely]]`, `[[unlikely]]`	C++20	hint which branch is taken more often
`[[noreturn]]`	C++11	the function never returns (it throws or exits)

ATTRIBUTES PRODUCE HELPFUL WARNINGS

main.cpp

```
#include <iostream>

[[nodiscard("check the error code")]] int save_file() { return -1; }

[[deprecated("use area_v2() instead")]] int area(int w, int h) { return w * h; }

int classify(int x) {
    switch (x) {
    case 0:  std::cout << "zero, ";
             [[fallthrough]];                // intentional: no warning
    case 1:  return 1;
    default: return 2;
    }
}

int main([[maybe_unused]] int argc, [[maybe_unused]] char** argv) {
    save_file();                             // warning: ignoring return value
    int a = area(3, 4);                      // warning: deprecated
    [[maybe_unused]] int debug_only = 42;    // no "unused variable" warning
    for (int i = 0; i < 3; ++i) {
        if (i < 2) [[likely]] std::cout << "common ";
        else [[unlikely]] std::cout << "rare ";
    }
    std::cout << a << ' ' << classify(0) << '\n';
    return 0;
}
```

OUTPUT

```
main.cpp: In function ‘int main(int, char**)’:
main.cpp:17:14: warning: ignoring return value of ‘int save_file()’, declared with attribute
‘nodiscard’: ‘check the error code’ [-Wunused-result]
   17 |     save_file();                             // warning: ignoring return value
      |     ~~~~~~~~~^~
main.cpp:3:43: note: declared here
    3 | [[nodiscard("check the error code")]] int save_file() { return -1; }
      |                                           ^~~~~~~~
main.cpp:18:17: warning: ‘int area(int, int)’ is deprecated: use area_v2() instead [-Wdeprecated-
declarations]
   18 |     int a = area(3, 4);                      // warning: deprecated
      |             ~~~~^~~~~
main.cpp:5:45: note: declared here
    5 | [[deprecated("use area_v2() instead")]] int area(int w, int h) { return w * h; }
      |                                             ^~~~
$ ./prog
common common rare 12 zero, 1
```

20.5 Vocabulary types (C++17)

C++17 added a set of small "vocabulary" types that say precisely what a value may contain. Using them in function signatures documents intent and removes whole classes of bugs.

20.5.1 std::optional

`std::optional<T>` (header `<optional>`) holds either a `T` or nothing (`std::nullopt`). It replaces sentinel values (-1, empty string, null pointer) for "maybe a value". Test with `if (opt)` or `has_value()`; read with `*opt`, `opt->member`, `value()` (throws `std::bad_optional_access` if empty) or `value_or(default)`. C++23

adds monadic operations: `and_then`, `transform`, `or_else`. Module 16 compared it with exceptions and std::expected.

STD::OPTIONAL

main.cpp

```
#include <iostream>
#include <map>
#include <optional>
#include <string>

struct Config {
    std::string host = "localhost";
    std::optional<int> port;                // a setting that may be absent
    std::optional<std::string> proxy;
};

std::optional<std::string> find_user(int id) {
    static const std::map<int, std::string> users = {{1, "asha"}, {2, "ravi"}};
    if (auto it = users.find(id); it != users.end()) return it->second;
    return std::nullopt;
}

int main() {
    for (int id : {1, 5}) {
        auto user = find_user(id);
        if (user) std::cout << "user " << id << ": " << *user << " (" << user->size() << "
letters)\n";
        else std::cout << "user " << id << ": not found\n";
    }
    Config c;
    c.proxy = "proxy.lbf.in";
    std::cout << "port: " << c.port.value_or(8080) << ", proxy set? " << std::boolalpha <<
c.proxy.has_value() << '\n';
    c.port = 443;
    std::cout << "port now " << *c.port << '\n';
    c.proxy.reset();
    try {
        std::cout << c.proxy.value();
    } catch (const std::bad_optional_access& e) {
        std::cout << "value() on empty optional: " << e.what() << '\n';
    }
    std::cout << "sizeof(optional<int>) = " << sizeof(std::optional<int>) << " (int + a flag,
padded)\n";
    return 0;
}
```

OUTPUT

```
user 1: asha (4 letters)
user 5: not found
port: 8080, proxy set? true
port now 443
value() on empty optional: bad optional access
sizeof(optional<int>) = 8 (int + a flag, padded)
```

20.5.2 std::variant and std::visit

`std::variant<A, B, C>` (header `<variant>`) is a **type-safe union**: it holds exactly one value of one of the listed types at a time and always knows which. `index()` tells which alternative is active, `std::holds_alternative<T>` tests it, `std::get<T>` reads it (throwing `std::bad_variant_access` on a mismatch) and `std::get_if<T>` returns a pointer or null. `std::visit` calls a function with whatever value

is held; with the "overloaded" lambda trick, the compiler checks that every alternative is handled. Variants model closed sets of alternatives without inheritance or heap allocation.

STD::VARIANT AND STD::VISIT

main.cpp

```
#include <cmath>
#include <iostream>
#include <string>
#include <variant>
#include <vector>

struct Circle { double r; };
struct Rect   { double w, h; };
struct Tri    { double a, b, c; };
using Shape = std::variant<Circle, Rect, Tri>;

template <typename... Fs> struct overloaded : Fs... { using Fs::operator()...; };   // the
"overloaded" helper

double area(const Shape& s) {
    return std::visit(overloaded{
        [](const Circle& c) { return 3.14159265 * c.r * c.r; },
        [](const Rect& r)   { return r.w * r.h; },
        [](const Tri& t)    { double p = (t.a + t.b + t.c) / 2; return std::sqrt(p * (p - t.a) * (p -
t.b) * (p - t.c)); },
    }, s);
}

int main() {
    std::vector<Shape> shapes = {Circle{1}, Rect{3, 4}, Tri{3, 4, 5}};
    for (const auto& s : shapes) std::cout << "alternative " << s.index() << ": area " << area(s) <<
'\n';

    std::variant<int, std::string> v = 42;
    std::cout << std::boolalpha << "holds int? " << std::holds_alternative<int>(v) << ", value " <<
std::get<int>(v) << '\n';
    v = "now a string";
    if (auto* p = std::get_if<std::string>(&v)) std::cout << "string: " << *p << '\n';
    try {
        std::get<int>(v);
    } catch (const std::bad_variant_access& e) {
        std::cout << "wrong get: " << e.what() << '\n';
    }
    std::visit([](const auto& x) { std::cout << "visit generic: " << x << '\n'; }, v);
    return 0;
}
```

OUTPUT

```
alternative 0: area 3.14159
alternative 1: area 12
alternative 2: area 6
holds int? true, value 42
string: now a string
wrong get: std::get: wrong index for variant
visit generic: now a string
```

20.5.3 std::any

`std::any` (header `<any>`) can hold a value of **any** copyable type, chosen at run time. Retrieve it with `std::any_cast<T>`, which throws `std::bad_any_cast` if the type is wrong (the pointer form returns null

instead). It is rarely needed: prefer `variant` when the set of types is known, and templates when the type is known at compile time. Typical uses are plugin systems and property bags.

STD::ANY

main.cpp

```cpp
#include <any>
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, std::any> props;
    props["width"] = 1920;
    props["title"] = std::string("Dashboard");
    props["ratio"] = 16.0 / 9.0;

    std::cout << "width " << std::any_cast<int>(props["width"]) << ", title " <<
std::any_cast<std::string>(props["title"]) << '\n';
    std::cout << "ratio type: " << props["ratio"].type().name() << " (d = double)\n";
    if (auto* w = std::any_cast<int>(&props["width"])) *w = 1280;            // pointer form: no
exception
    std::cout << "width now " << std::any_cast<int>(props["width"]) << '\n';
    try {
        std::any_cast<double>(props["width"]);                               // holds an int, not a
double
    } catch (const std::bad_any_cast& e) {
        std::cout << "error: " << e.what() << '\n';
    }
    std::any empty;
    std::cout << std::boolalpha << "empty.has_value() = " << empty.has_value() << '\n';
    return 0;
}
```

OUTPUT

```
width 1920, title Dashboard
ratio type: d (d = double)
width now 1280
error: bad any_cast
empty.has_value() = false
```

20.5.4 std::string_view

`std::string_view` (header `<string_view>`) is a non-owning, read-only **view** of characters: just a pointer and a length. A function taking `std::string_view` accepts string literals, `std::string`s and parts of strings without copying or allocating, and `substr` on a view is O(1). The price is that the view does not own the characters: it must not outlive the string it looks at (Module 8 showed the dangling-view bug).

STRING_VIEW FOR CHEAP, READ-ONLY TEXT

main.cpp

```
#include <iostream>
#include <string>
#include <string_view>
#include <vector>

std::vector<std::string_view> split(std::string_view text, char sep) {    // no copies of the pieces
    std::vector<std::string_view> parts;
    while (true) {
        auto pos = text.find(sep);
        parts.push_back(text.substr(0, pos));
        if (pos == std::string_view::npos) break;
        text.remove_prefix(pos + 1);
    }
    return parts;
}

bool is_csv(std::string_view filename) { return filename.ends_with(".csv"); }   // C++20 member

int main() {
    std::string line = "2026-10-06,Bengaluru,28.5,humid";
    for (auto field : split(line, ',')) std::cout << '[' << field << "] ";
    std::cout << '\n';
    std::cout << std::boolalpha << is_csv("data.csv") << ' ' << is_csv(std::string("photo.png")) <<
'\n';

    std::string_view sv = line;
    std::string_view date = sv.substr(0, 10);
    std::cout << "date view: " << date << ", size " << date.size() << ", sizeof(string_view) = " <<
sizeof(sv) << '\n';
    std::string owned(date);                    // make an owning copy when you need to keep it
    std::cout << "owned copy: " << owned << '\n';
    return 0;
}
```

OUTPUT

```
[2026-10-06] [Bengaluru] [28.5] [humid]
true false
date view: 2026-10-06, size 10, sizeof(string_view) = 16
owned copy: 2026-10-06
```

20.6 Other library additions

20.6.1 <chrono> for time and durations

`<chrono>` separates three ideas: a **duration** (an amount of time with a unit, such as `std::chrono::milliseconds`), a **clock** (`system_clock` for wall-clock time, `steady_clock` for measuring intervals: it never jumps backwards), and a **time point** (a clock's moment). The types make unit mistakes impossible: adding seconds to milliseconds converts automatically, and lossy conversions require `duration_cast`. Literals such as `500ms` and `2h` come from `std::chrono_literals`. C++20 adds calendar dates and time zones.

DURATIONS, CLOCKS AND DATES

main.cpp

```
#include <chrono>
#include <iostream>
#include <thread>
using namespace std::chrono_literals;

long long slow_sum(int n) {
    long long s = 0;
    for (int i = 0; i < n; ++i) s += i % 7;
    return s;
}

int main() {
    std::chrono::seconds lecture = 1h + 30min;                    // automatic conversion to seconds
    std::cout << "lecture: " << lecture.count() << " s = " <<
std::chrono::duration_cast<std::chrono::minutes>(lecture).count() << " min\n";
    auto t = 2500ms;
    std::cout << "2500ms in whole seconds: " << std::chrono::duration_cast<std::chrono::seconds>
(t).count()
              << ", as double seconds: " << std::chrono::duration<double>(t).count() << '\n';

    auto start = std::chrono::steady_clock::now();
    std::this_thread::sleep_for(120ms);
    auto slept = std::chrono::steady_clock::now() - start;
    std::cout << "slept about " << std::chrono::duration_cast<std::chrono::milliseconds>
(slept).count() / 10 * 10 << " ms\n";

    using namespace std::chrono;
    year_month_day exam{year{2026}, month{12}, day{15}};          // C++20 calendar
    sys_days today = year{2026} / October / 6;
    std::cout << "days until the exam: " << (sys_days{exam} - today).count() << '\n';
    std::cout << "weekday of the exam: " << weekday{sys_days{exam}} << '\n';
    std::cout << "result " << slow_sum(1000) << '\n';
    return 0;
}
```

OUTPUT

```
lecture: 5400 s = 90 min
2500ms in whole seconds: 2, as double seconds: 2.5
slept about 120 ms
days until the exam: 70
weekday of the exam: Tue
result 2997
```

NOTE

The "slept about" line rounds the measured time down to tens of milliseconds because sleeping is never exact: the real value is 120 ms plus a little scheduling delay.

20.6.2 <random> random number engines and distributions

`rand()` is low quality and awkward to scale. `<random>` separates the **engine**, which produces raw random bits (`std::mt19937`, the Mersenne Twister, is the usual choice), from the **distribution**, which shapes them into the values you need: `uniform_int_distribution`, `uniform_real_distribution`, `normal_distribution`, `bernoulli_distribution`, `discrete_distribution`… Seed the engine once, from `std::random_device` for unpredictable runs or from a fixed number for reproducible tests and simulations.

ENGINES AND DISTRIBUTIONS

main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <map>
#include <random>
#include <string>
#include <vector>

int main() {
    std::mt19937 rng(2026);                                  // fixed seed: same output every run
    std::uniform_int_distribution<int> dice(1, 6);
    std::cout << "dice:";
    for (int i = 0; i < 10; ++i) std::cout << ' ' << dice(rng);
    std::cout << '\n';

    std::map<int, int> counts;
    for (int i = 0; i < 60000; ++i) ++counts[dice(rng)];
    std::cout << "60000 rolls:";
    for (auto [face, n] : counts) std::cout << ' ' << face << ':' << n;
    std::cout << '\n';

    std::normal_distribution<double> height(165.0, 7.0);       // mean 165 cm, std dev 7
    double sum = 0;
    for (int i = 0; i < 10000; ++i) sum += height(rng);
    std::cout << "average of 10000 normal samples: " << sum / 10000 << '\n';

    std::vector<std::string> quiz = {"Q1", "Q2", "Q3", "Q4", "Q5"};
    std::shuffle(quiz.begin(), quiz.end(), rng);               // random order
    for (const auto& q : quiz) std::cout << q << ' ';
    std::cout << '\n';

    std::discrete_distribution<int> loot({70, 25, 5});         // common, rare, epic with weights
    int epic = 0;
    for (int i = 0; i < 1000; ++i) epic += loot(rng) == 2;
    std::cout << "epic drops in 1000: " << epic << '\n';

    std::random_device rd;                                     // non-deterministic seed for real use
    std::mt19937 real_rng(rd());
    int roll = dice(real_rng);                                 // differs from run to run
    std::cout << std::boolalpha << "unpredictable roll in 1..6? " << (roll >= 1 && roll <= 6) << '\n';
    return 0;
}
```

OUTPUT

```
dice: 2 5 3 6 6 6 1 5 3 1
60000 rolls: 1:10006 2:9866 3:10022 4:9911 5:10111 6:10084
average of 10000 normal samples: 164.945
Q4 Q2 Q1 Q5 Q3
epic drops in 1000: 58
unpredictable roll in 1..6? true
```

20.6.3 Parallel algorithms with execution policies (C++17)

Most standard algorithms accept an **execution policy** as an extra first argument (header `<execution>`): `std::execution::seq` (sequential), `par` (may run on several threads), `par_unseq` (several threads and vectorised), and `unseq` (C++20, vectorised on one thread). The library decides how to split the work. Your callbacks must then be safe to run concurrently: no data races, no locks in `par_unseq`. With GCC, the parallel versions use Intel TBB; build with `-ltbb`. Without TBB installed, as on the machine that built these notes, they compile but run sequentially.

EXECUTION POLICIES

main.cpp

```cpp
#include <algorithm>
#include <chrono>
#include <execution>
#include <iostream>
#include <numeric>
#include <random>
#include <vector>

template <typename Policy>
long long time_sort(Policy policy, std::vector<double> v) {
    auto t0 = std::chrono::steady_clock::now();
    std::sort(policy, v.begin(), v.end());
    auto t1 = std::chrono::steady_clock::now();
    return std::chrono::duration_cast<std::chrono::milliseconds>(t1 - t0).count();
}

int main() {
    std::vector<double> data(5'000'000);
    std::mt19937 rng(1);
    std::uniform_real_distribution<double> dist(0, 1);
    for (double& x : data) x = dist(rng);

    double total = std::reduce(std::execution::par, data.begin(), data.end());    // parallel sum
    std::cout << "mean = " << total / data.size() << '\n';
    std::cout << "seq sort: " << time_sort(std::execution::seq, data) << " ms\n";
    std::cout << "par sort: " << time_sort(std::execution::par, data) << " ms\n";
    std::transform(std::execution::par_unseq, data.begin(), data.end(), data.begin(), [](double x) {
return x * 2; });
    std::cout << "max after doubling < 2? " << std::boolalpha << (*std::max_element(data.begin(),
data.end()) < 2) << '\n';
    return 0;
}
```

OUTPUT

```
mean = 0.500045
seq sort: 537 ms
par sort: 517 ms
max after doubling < 2? true
```

On a machine with TBB and 8 cores, the parallel sort of 5 million doubles is typically 4 to 6 times faster than the sequential one. Here both take about the same time because the sequential fallback was used. Parallelism pays off only for large inputs; for a few thousand elements the cost of starting threads exceeds the gain.

Move semantics transfers resources instead of copying (std::move only casts; noexcept moves matter). Follow the Rule of Zero, use unique_ptr by default, shared_ptr for real shared ownership and weak_ptr to break cycles. Lambdas with careful captures make algorithms readable; std::function stores any callable. Use auto, range-for, nullptr, constexpr, brace init, structured bindings and if-initializers freely, and express intent with optional, variant and string_view.

PRACTICE CHECKPOINT

Rewrite the **Shape hierarchy with unique_ptr**, add a **move constructor to the DynamicArray** class and measure the speed difference, and model a **result type with std::variant**.

SAMPLE SOLUTION

main.cpp

```
#include <algorithm>
#include <chrono>
#include <cmath>
#include <iostream>
#include <memory>
#include <string>
#include <utility>
#include <variant>
#include <vector>

// ---------- 1. Shape hierarchy owned by unique_ptr ----------
class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
    virtual std::string name() const = 0;
};
class Circle : public Shape {
public:
    explicit Circle(double r) : r_(r) {}
    double area() const override { return 3.14159265 * r_ * r_; }
    std::string name() const override { return "Circle"; }
private:
    double r_;
};
class Rectangle : public Shape {
public:
    Rectangle(double w, double h) : w_(w), h_(h) {}
    double area() const override { return w_ * h_; }
    std::string name() const override { return "Rectangle"; }
private:
    double w_, h_;
};

// ---------- 2. DynamicArray with copy and move ----------
class DynamicArray {
public:
    explicit DynamicArray(std::size_t n) : n_(n), data_(new double[n]) { std::fill(data_, data_ + n_,
1.0); }
    ~DynamicArray() { delete[] data_; }
    DynamicArray(const DynamicArray& o) : n_(o.n_), data_(new double[o.n_]) { std::copy(o.data_,
o.data_ + n_, data_); }
    DynamicArray(DynamicArray&& o) noexcept : n_(std::exchange(o.n_, 0)), data_(std::exchange(o.data_,
nullptr)) {}
    DynamicArray& operator=(DynamicArray o) noexcept { std::swap(n_, o.n_); std::swap(data_, o.data_);
return *this; }  // copy-and-swap
    std::size_t size() const { return n_; }
private:
    std::size_t n_;
    double* data_;
};

template <typename F>
double time_us(F f) {
    auto t0 = std::chrono::steady_clock::now();
    f();
    return std::chrono::duration<double, std::micro>(std::chrono::steady_clock::now() - t0).count();
}

// ---------- 3. A result type with std::variant ----------
struct Error { int code; std::string message; };
template <typename T>
```

```
using Result = std::variant<T, Error>;

Result<double> safe_sqrt(double x) {
    if (x < 0) return Error{1, "negative input " + std::to_string(static_cast<int>(x))};
    return std::sqrt(x);
}

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(1.5));
    shapes.push_back(std::make_unique<Rectangle>(4, 2.5));
    shapes.push_back(std::make_unique<Circle>(0.5));
    std::sort(shapes.begin(), shapes.end(), [](const auto& a, const auto& b) { return a->area() < b-
>area(); });
    for (const auto& s : shapes) std::cout << s->name() << " area " << s->area() << '\n';
    // no delete anywhere: the unique_ptrs free the shapes

    DynamicArray big(20'000'000);                                    // 160 MB of doubles
    DynamicArray copy(1), moved(1);
    double copy_time = time_us([&] { copy = big; });                 // allocate 160 MB and copy it
    double move_time = time_us([&] { moved = std::move(big); });     // swap two pointers
    std::cout << "copy has " << copy.size() << " elements, moved has " << moved.size() << ", big now
has " << big.size() << '\n';
    std::cout << "copy took " << (copy_time > 1000 ? "milliseconds" : "?") << ", move took " <<
(move_time < 1000 ? "microseconds" : "?")
              << "; copy was over " << (copy_time / move_time > 100 ? "100x" : "?") << " slower\n";

    for (double x : {16.0, -4.0}) {
        Result<double> r = safe_sqrt(x);
        if (auto* v = std::get_if<double>(&r)) std::cout << "sqrt(" << x << ") = " << *v << '\n';
        else {
            const Error& e = std::get<Error>(r);
            std::cout << "error " << e.code << ": " << e.message << '\n';
        }
    }
    return 0;
}
```

OUTPUT

```
Circle area 0.785398
Circle area 7.06858
Rectangle area 10
copy has 20000000 elements, moved has 20000000, big now has 0
copy took milliseconds, move took microseconds; copy was over 100x slower
sqrt(16) = 4
error 1: negative input -4
```

The exact copy and move times depend on the machine, so the program prints only their order of magnitude: copying 20 million doubles takes tens of milliseconds, while the move just swaps two pointers and takes well under a microsecond plus the printing time.

MODULE 21 · ADVANCED

C++20, C++23 and C++26 features

Minimum duration **4 days** Level **Advanced**

C++20 is the biggest update since C++11. Concepts make templates readable and their errors short, ranges turn loops into pipelines, modules replace header files, and coroutines make lazy sequences and asynchronous code straightforward. C++23 polishes these with std::print, std::expected, deducing this and new views, and C++26 brings static reflection and contracts. This module shows what each feature is for and how to use it today.

IN THIS MODULE

21.1 Concepts (C++20)
21.2 Ranges (C++20)
21.3 Modules (C++20)
21.4 Coroutines (C++20)
21.5 Other C++20 features
21.6 Other C++23 features
21.7 C++26 overview

NOTE

Compiler support for the newest features varies. Examples here were built with GCC 13 (`-std=c++20` or `-std=c++23`); a few C++23 features that GCC 13 lacks were built with Clang 18, and features that neither compiler in the build environment supports yet (std::print, std::generator, ranges::to, mdspan, flat_map, C++26) are marked in their boxes with the output that a current compiler (GCC 14/15) prints.

21.1 Concepts (C++20)

21.1.1 Constraining templates with requires

A **concept** is a named, compile-time predicate on types. A template can state its requirements with a `requires` clause, or by writing the concept in place of `typename`. If a call does not satisfy the requirement, the template is simply not a candidate, and the error says so at the call site in a few lines, instead of failing deep inside the template body. Constraints also take part in overloading: when several overloads with the same parameter types match, the most constrained one wins.

CONSTRAINING TEMPLATES

main.cpp

```
#include <concepts>
#include <iostream>
#include <string>

template <typename T>
    requires std::integral<T>                    // requires clause
T gcd(T a, T b) {
    while (b != 0) { T t = a % b; a = b; b = t; }
    return a;
}

template <std::floating_point T>                 // concept in place of typename
T average(T a, T b) { return (a + b) / 2; }

void describe(std::integral auto x)       { std::cout << x << " is an integer\n"; }
void describe(std::floating_point auto x) { std::cout << x << " is floating point\n"; }
void describe(auto x)                     { std::cout << "something else: " << x << '\n'; }   // least
constrained

int main() {
    std::cout << "gcd(84, 36) = " << gcd(84, 36) << ", gcd(10L, 4L) = " << gcd(10L, 4L) << '\n';
    std::cout << "average(2.0, 5.0) = " << average(2.0, 5.0) << '\n';
    describe(42);
    describe(2.5);
    describe(std::string("text"));
    return 0;
}
```

OUTPUT

```
gcd(84, 36) = 12, gcd(10L, 4L) = 2
average(2.0, 5.0) = 3.5
42 is an integer
2.5 is floating point
something else: text
```

THE ERROR MESSAGE WITH A CONCEPT

main.cpp

```cpp
#include <concepts>

template <typename T>
    requires std::integral<T>
T gcd(T a, T b) {
    while (b != 0) { T t = a % b; a = b; b = t; }
    return a;
}

int main() {
    return static_cast<int>(gcd(8.5, 2.5));        // doubles are not integral
}
```

OUTPUT

```
main.cpp: In function ‘int main()’:
main.cpp:11:32: error: no matching function for call to ‘gcd(double, double)’
   11 |     return static_cast<int>(gcd(8.5, 2.5));        // doubles are not integral
      |                             ~~~^~~~~~~~~~
main.cpp:5:3: note: candidate: ‘template<class T>  requires  integral<T> T gcd(T, T)’
    5 | T gcd(T a, T b) {
      |   ^~~
main.cpp:5:3: note:   template argument deduction/substitution failed:
main.cpp:5:3: note: constraints not satisfied
In file included from main.cpp:1:
/usr/include/c++/13/concepts: In substitution of ‘template<class T>  requires  integral<T> T gcd(T, T)
[with T = double]’:
main.cpp:11:32:   required from here
/usr/include/c++/13/concepts:100:13:   required for the satisfaction of ‘integral<T>’ [with T =
double]
/usr/include/c++/13/concepts:100:24: note: the expression ‘is_integral_v<_Tp> [with _Tp = double]’
evaluated to ‘false’
  100 |     concept integral = is_integral_v<_Tp>;
      |                        ^~~~~~~~~~~~~~~~~~
```

Compare this with the error from Module 17: the compiler states plainly that the constraint `std::integral<T>` was not satisfied for `T = double`, pointing at our call.

21.1.2 Standard concepts: integral, floating_point, same_as, sortable

The header `<concepts>` and the ranges and iterator headers define many ready-made concepts.

Concept	Satisfied when	Header
`std::integral<T>`, `std::signed_integral`, `std::floating_point`	T is an integer type / signed integer / floating-point type	`<concepts>`
`std::same_as<T, U>`	T and U are the same type	`<concepts>`
`std::convertible_to<From, To>`, `std::derived_from<D, B>`	implicit conversion exists / D publicly derives from B	`<concepts>`
`std::equality_comparable`, `std::totally_ordered`	`==` / all of `< > <= >=` work	`<concepts>`
`std::copyable`, `std::movable`, `std::regular`	can be copied / moved / behaves like an int (copyable, default-constructible, comparable)	`<concepts>`

Concept	Satisfied when	Header
`std::invocable<F, Args...>`, `std::predicate`	F can be called with Args / and returns something bool-like	`<concepts>`
`std::random_access_iterator<I>`	I is a random-access iterator	`<iterator>`
`std::sortable<I>`	the elements reachable from I can be sorted in place	`<iterator>`
`std::ranges::range`, `std::ranges::random_access_range`	has begin/end / with random-access iterators	`<ranges>`

TESTING TYPES AGAINST CONCEPTS

main.cpp

```cpp
#include <concepts>
#include <iostream>
#include <iterator>
#include <list>
#include <memory>
#include <string>
#include <vector>

struct Animal { virtual ~Animal() = default; };
struct Dog : Animal {};

int main() {
    std::cout << std::boolalpha;
    std::cout << "integral<char>:                 " << std::integral<char> << '\n';
    std::cout << "same_as<int, int32_t>:          " << std::same_as<int, std::int32_t> << '\n';
    std::cout << "convertible_to<int, double>:    " << std::convertible_to<int, double> << '\n';
    std::cout << "derived_from<Dog, Animal>:      " << std::derived_from<Dog, Animal> << '\n';
    std::cout << "totally_ordered<std::string>:   " << std::totally_ordered<std::string> << '\n';
    std::cout << "sortable<vector<int>::iterator>: " << std::sortable<std::vector<int>::iterator> <<
'\n';
    std::cout << "sortable<list<int>::iterator>:   " << std::sortable<std::list<int>::iterator> <<
'\n';
    std::cout << "regular<int>: " << std::regular<int> << ", copyable<unique_ptr<int>>: " <<
std::copyable<std::unique_ptr<int>> << '\n';
    return 0;
}
```

OUTPUT

```
integral<char>:                 true
same_as<int, int32_t>:          true
convertible_to<int, double>:    true
derived_from<Dog, Animal>:      true
totally_ordered<std::string>:   true
sortable<vector<int>::iterator>: true
sortable<list<int>::iterator>:   true
regular<int>: true, copyable<unique_ptr<int>>: false
```

Note that `std::sortable` is true even for list iterators: it only checks that the elements can be compared and swapped. `std::ranges::sort` additionally requires a `random_access_range`, which a list is not; that is the requirement that rejects `std::ranges::sort(my_list)`.

21.1.3 Writing custom concepts

A concept is defined with `template <...> concept Name = constraint-expression;`. The expression can combine other concepts with `&&` and `||`, and use a **requires expression** that lists operations which

must compile: `requires (T a) { a.area(); }`. A requirement in braces with `->` also checks the type of the result.

DEFINING YOUR OWN CONCEPTS

main.cpp

```
#include <concepts>
#include <iostream>
#include <string>
#include <vector>

template <typename T>
concept Numeric = std::integral<T> || std::floating_point<T>;

template <typename T>
concept Shape = requires(const T& s) {
    { s.area() } -> std::convertible_to<double>;      // must have area() returning something double-like
    { s.name() } -> std::convertible_to<std::string>;
};

template <typename C>
concept Container = requires(C c) {
    c.begin();
    c.end();
    c.size();
    typename C::value_type;                           // must have a nested type
};

struct Square { double side; double area() const { return side * side; } std::string name() const { return "square"; } };
struct Label  { std::string text; };                  // not a Shape

template <Numeric T>
T twice(T x) { return x * 2; }

void report(const Shape auto& s) { std::cout << s.name() << " with area " << s.area() << '\n'; }

template <Container C>
    requires Numeric<typename C::value_type>
auto sum(const C& c) {
    typename C::value_type total{};
    for (const auto& x : c) total += x;
    return total;
}

int main() {
    std::cout << twice(21) << ' ' << twice(1.25) << '\n';
    report(Square{3});
    std::cout << "sum = " << sum(std::vector<double>{1.5, 2.5, 3}) << '\n';
    std::cout << std::boolalpha << "Square is a Shape? " << Shape<Square> << ", Label? " << Shape<Label> << '\n';
    std::cout << "vector<string> is a Container? " << Container<std::vector<std::string>> << ", string is Numeric? " << Numeric<std::string> << '\n';
    // sum(std::vector<std::string>{"a"});   error: constraints not satisfied
    return 0;
}
```

OUTPUT

```
42 2.5
square with area 9
sum = 7
Square is a Shape? true, Label? false
vector<string> is a Container? true, string is Numeric? false
```

21.1.4 Abbreviated function templates void f(auto x)

In C++20 an ordinary function can use `auto` for a parameter type; it then becomes a function template, exactly like a generic lambda. A concept can be placed in front: `void f(std::integral auto x)`. Each `auto` parameter is a separate template parameter, so `f(auto a, auto b)` accepts two different types.

AUTO PARAMETERS main.cpp

```
#include <concepts>
#include <iostream>
#include <string>
#include <vector>

void show(const auto& x) { std::cout << "[" << x << "] "; }          // template <typename T> void
show(const T&)
auto larger(auto a, auto b) { return a > b ? a : b; }                // two independent types
void repeat(std::integral auto n, const std::string& s) { while (n-- > 0) std::cout << s; std::cout <<
'\n'; }

int main() {
    show(1); show(2.5); show("three"); show(std::string("four"));
    std::cout << '\n';
    std::cout << larger(3, 7.5) << '\n';
    repeat(3, "ab");
    // repeat(2.5, "x");   error: 2.5 does not satisfy std::integral
    return 0;
}
```

OUTPUT

```
[1] [2.5] [three] [four]
7.5
ababab
```

21.2 Ranges (C++20)

The ranges library (header `<ranges>` and the range versions of the algorithms) builds on two ideas. First, algorithms take a whole **range** (anything with begin and end) instead of two iterators. Second, **views** are lightweight, lazy adaptors that filter, transform, slice or combine ranges without copying, and they compose with the pipe operator `|`.

21.2.1 Range algorithms std::ranges::sort(v)

Every classic algorithm has a counterpart in namespace `std::ranges` that takes the container directly, is constrained with concepts (better errors), and accepts a **projection**: a function applied to each element before comparing, which removes the need for many custom comparators.

RANGE ALGORITHMS AND PROJECTIONS

main.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

struct Student { std::string name; int marks; };

int main() {
    std::vector<int> v = {5, 2, 9, 1, 7};
    std::ranges::sort(v);                                   // no begin()/end()
    for (int x : v) std::cout << x << ' ';
    std::cout << "| max " << std::ranges::max(v) << ", contains 9? " << std::boolalpha <<
(std::ranges::find(v, 9) != v.end()) << '\n';

    std::vector<Student> cls = {{"Ravi", 72}, {"Asha", 91}, {"Kiran", 85}};
    std::ranges::sort(cls, {}, &Student::marks);            // projection: compare by marks
    for (const auto& s : cls) std::cout << s.name << ' ';
    std::cout << '\n';
    std::ranges::sort(cls, {}, &Student::name);             // by name
    for (const auto& s : cls) std::cout << s.name << ' ';
    std::cout << '\n';
    auto best = std::ranges::max_element(cls, {}, &Student::marks);
    std::cout << "top: " << best->name << '\n';
    auto it = std::ranges::find(cls, "Kiran", &Student::name);     // find by a member
    std::cout << "Kiran has " << it->marks << '\n';
    return 0;
}
```

OUTPUT

```
1 2 5 7 9 | max 9, contains 9? true
Ravi Kiran Asha
Asha Kiran Ravi
top: Asha
Kiran has 85
```

21.2.2 Views: filter, transform, take, drop, reverse, iota

A **view** does not own elements; it refers to another range (or generates values) and computes elements on demand. Creating a view is O(1). The most used views in namespace `std::views`:

View	Produces
`views::filter(pred)`	only the elements for which pred is true
`views::transform(f)`	`f(x)` for each element
`views::take(n)` / `views::drop(n)`	the first n elements / all but the first n
`views::take_while(p)` / `drop_while(p)`	elements while p holds / after p stops holding
`views::reverse`	the elements backwards
`views::iota(a)`, `views::iota(a, b)`	a, a+1, a+2 … (infinite, or up to b − 1)
`views::keys`, `views::values`	first / second of each pair (for maps)
`views::split(delim)`, `views::join`	split into subranges / flatten nested ranges

VIEWS

main.cpp

```cpp
#include <iostream>
#include <map>
#include <ranges>
#include <string>
#include <vector>

template <typename R>
void show(const char* label, R&& r) {
    std::cout << label;
    for (const auto& x : r) std::cout << x << ' ';
    std::cout << '\n';
}

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    show("filter even:    ", std::views::filter(v, [](int x) { return x % 2 == 0; }));
    show("transform x*x:  ", std::views::transform(v, [](int x) { return x * x; }));
    show("take 3:         ", std::views::take(v, 3));
    show("drop 7:         ", std::views::drop(v, 7));
    show("reverse:        ", std::views::reverse(v));
    show("iota(5, 10):    ", std::views::iota(5, 10));
    show("take_while < 4: ", std::views::take_while(v, [](int x) { return x < 4; }));

    std::map<std::string, int> stock = {{"ink", 3}, {"pen", 12}};
    show("map keys:       ", std::views::keys(stock));
    show("map values:     ", std::views::values(stock));
    std::cout << "v is unchanged: " << v.front() << ".." << v.back() << '\n';
    return 0;
}
```

OUTPUT

```
filter even:    2 4 6 8 10
transform x*x:  1 4 9 16 25 36 49 64 81 100
take 3:         1 2 3
drop 7:         8 9 10
reverse:        10 9 8 7 6 5 4 3 2 1
iota(5, 10):    5 6 7 8 9
take_while < 4: 1 2 3
map keys:       ink pen
map values:     3 12
v is unchanged: 1..10
```

21.2.3 Pipe syntax v | views::filter(...) | views::transform(...)

Views compose with `|`: `range | view1 | view2` means "apply view1 to range, then view2 to the result". A pipeline reads left to right like a sentence, which makes data-processing code much clearer than nested loops with temporary vectors.

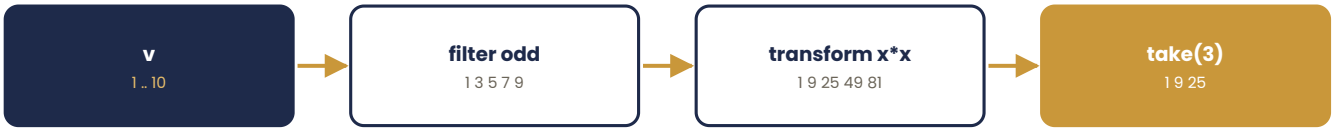
PIPELINES

main.cpp

```cpp
#include <iostream>
#include <ranges>
#include <string>
#include <vector>

struct Order { std::string customer; double amount; bool paid; };

int main() {
    namespace views = std::views;
    std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    auto pipeline = v | views::filter([](int x) { return x % 2 == 1; })    // odd numbers
                      | views::transform([](int x) { return x * x; })      // squared
                      | views::take(3);                                    // first three
    for (int x : pipeline) std::cout << x << ' ';
    std::cout << '\n';

    std::vector<Order> orders = {{"Asha", 1200, true}, {"Ravi", 450, false}, {"Meera", 3100, false},
{"Kiran", 800, false}, {"Dev", 2500, true}};
    std::cout << "unpaid customers, largest first:";
    for (const auto& name : orders | views::filter([](const Order& o) { return !o.paid; })
                                   | views::reverse
                                   | views::transform(&Order::customer))
        std::cout << ' ' << name;
    std::cout << '\n';

    for (int n : views::iota(1) | views::filter([](int x) { return x % 7 == 0; }) | views::take(4))
// infinite range!
        std::cout << n << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
1 9 25
unpaid customers, largest first: Kiran Meera Ravi
7 14 21 28
```



EACH STAGE OF THE PIPELINE, AS IF IT WERE COMPUTED EAGERLY

21.2.4 Lazy evaluation

Views are **lazy**: building the pipeline does no work. Elements are computed one at a time, only when the loop asks for the next one, and only as many as needed. That is why `views::iota(1)`, an infinite range, can be used safely when followed by `take`. The trace below shows that the filter and transform run interleaved, element by element, and stop as soon as `take(2)` is satisfied.

WATCHING LAZINESS HAPPEN

main.cpp

```cpp
#include <iostream>
#include <ranges>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8};
    auto view = v | std::views::filter([](int x) { std::cout << "  filter " << x << '\n'; return x % 2
== 0; })
                  | std::views::transform([](int x) { std::cout << "  transform " << x << '\n'; return
x * 10; })
                  | std::views::take(2);
    std::cout << "pipeline built, nothing computed yet\n";
    for (int x : view) std::cout << "got " << x << '\n';
    std::cout << "elements 5..8 were never looked at\n";
    return 0;
}
```

OUTPUT

```
pipeline built, nothing computed yet
  filter 1
  filter 2
  transform 2
got 20
  filter 3
  filter 4
  transform 4
got 40
  filter 5
  filter 6
elements 5..8 were never looked at
```

WATCH OUT

Because views refer to their source instead of copying it, a view must not outlive the container it views (the same rule as string_view). Also, some views (filter in particular) cache their begin position, so iterate them through a non-const reference.

21.2.5 C++23 additions: ranges::to, views::zip, views::enumerate, views::chunk

C++23 fills the most-requested gaps. `std::ranges::to<Container>()` materialises a pipeline into a real container. `views::zip` walks several ranges in step, yielding tuples. `views::enumerate` pairs each element with its index. `views::chunk(n)` splits into groups of n, `views::slide(n)` gives overlapping windows, `views::stride(n)` takes every n-th element, and `views::adjacent<n>` and `views::cartesian_product` also arrived.

ZIP, ENUMERATE, CHUNK, SLIDE AND STRIDE

main.cpp

```cpp
#include <iostream>
#include <ranges>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> names = {"Asha", "Ravi", "Meera"};
    std::vector<int> marks = {91, 72, 85};
    for (auto [name, mark] : std::views::zip(names, marks)) std::cout << name << '=' << mark << ' ';
    std::cout << '\n';
    for (auto [i, name] : std::views::enumerate(names)) std::cout << i << ':' << name << ' ';
    std::cout << '\n';

    std::vector<int> v = {1, 2, 3, 4, 5, 6, 7};
    std::cout << "chunk(3):";
    for (auto group : v | std::views::chunk(3)) {
        std::cout << " [";
        for (int x : group) std::cout << x;
        std::cout << ']';
    }
    std::cout << "\nslide(3) sums:";
    for (auto window : v | std::views::slide(3)) {
        int s = 0;
        for (int x : window) s += x;
        std::cout << ' ' << s;
    }
    std::cout << "\nstride(2):";
    for (int x : v | std::views::stride(2)) std::cout << ' ' << x;
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
Asha=91 Ravi=72 Meera=85
0:Asha 1:Ravi 2:Meera
chunk(3): [123] [456] [7]
slide(3) sums: 6 9 12 15 18
stride(2): 1 3 5 7
```

RANGES::TO (GCC 14 OR LATER)

main.cpp

```
#include <iostream>
#include <map>
#include <ranges>
#include <string>
#include <vector>

int main() {
    auto squares = std::views::iota(1, 6)
                 | std::views::transform([](int x) { return x * x; })
                 | std::ranges::to<std::vector>();                // materialise into a vector
    std::cout << "vector of " << squares.size() << " squares, last = " << squares.back() << '\n';

    std::vector<std::string> names = {"Asha", "Ravi"};
    auto index = std::views::zip(names, std::views::iota(100))
               | std::ranges::to<std::map<std::string, int>>();    // straight into a map
    std::cout << "Ravi -> " << index["Ravi"] << '\n';
    return 0;
}
```

OUTPUT

```
vector of 5 squares, last = 25
Ravi -> 101
```

std::ranges::to needs GCC 14, Clang 17 with libc++, or MSVC 19.34; the output shown is what such a compiler prints. With GCC 13, construct the container from the view instead: std::vector<int> squares(view.begin(), view.end()) for a common range.

21.3 Modules (C++20)

21.3.1 export module and import

Header files are textually pasted into every file that includes them: the same declarations are parsed again and again, macros leak in both directions, and include order can matter. A **module** is compiled once into a binary interface; importers see only what it explicitly **exports**. A module interface unit begins with `export module name;`, marks public entities with `export`, and other files use `import name;`. Ordinary headers that the module itself needs go in the **global module fragment** between `module;` and the module declaration.

GEOMETRY.CPP (MODULE INTERFACE)

C++

```
module;                          // global module fragment: ordinary #includes go here
#include <cmath>

export module geometry;          // this file is the module interface

export namespace geo {
    struct Point { double x, y; };
    double distance(Point a, Point b);     // exported declaration
}

double square(double v) { return v * v; }  // NOT exported: invisible to importers

double geo::distance(Point a, Point b) {
    return std::sqrt(square(a.x - b.x) + square(a.y - b.y));
}
```

MAIN.CPP C++

```cpp
#include <iostream>
import geometry;                  // no header file for geometry

int main() {
    geo::Point a{0, 0}, b{3, 4};
    std::cout << "distance = " << geo::distance(a, b) << '\n';
    return 0;
}
```

Building a module with GCC 13

```
$ g++ -std=c++20 -fmodules-ts -c geometry.cpp
$ g++ -std=c++20 -fmodules-ts main.cpp geometry.o -o app
$ ./app
distance = 5
$ ls gcm.cache
geometry.gcm
$ g++ -std=c++20 -fmodules-ts bad.cpp geometry.o
bad.cpp: In function 'int main()':
bad.cpp:2:38: error: 'square' was not declared in this scope
    2 | int main() { return static_cast<int>(square(2.0)); }
      |                                      ^~~~~~
```

The interface must be compiled **before** any file that imports it; GCC stores the compiled interface in `gcm.cache/geometry.gcm`. The last command shows that the unexported helper `square` is really hidden: a file that imports `geometry` cannot see it.

21.3.2 Modules vs header files

	Headers (#include)	Modules (import)
Mechanism	text copied into each file by the preprocessor	compiled once into a binary interface
Build speed	the same headers re-parsed in every file	much faster for large code bases
Macros	leak into and out of headers	not exported; importers cannot change the module
Visibility	everything in the header is visible	only what is explicitly exported
Order dependence	include order can change meaning	imports are independent of order
Build system	any order of compilation	must compile interfaces before their importers (dependency scanning)
Tool support (2026)	universal	good in MSVC, Clang and GCC 14+, CMake 3.28+; still maturing

21.3.3 import std; (C++23)

C++23 lets you import the entire standard library as one module: `import std;` (and `import std.compat;`, which also puts the C functions in the global namespace). It replaces dozens of `#include` lines and is faster to compile than including even a few standard headers. It is supported by MSVC 17.5+, Clang 17+ with libc++, and GCC 15, and in CMake 3.30+ with `CMAKE_CXX_MODULE_STD`.

USING IMPORT STD (C++23) C++

```cpp
import std;                          // the whole standard library

int main() {
    std::vector<std::string> v{"modules", "are", "here"};
    std::println("{} words, first = {}", v.size(), v.front());
}
```

21.3.4 Compiler and build-system support

Tool	How to build modules
GCC	`-std=c++20 -fmodules-ts` (GCC 11-14), `-fmodules` (GCC 15); compile interfaces first
Clang	`--precompile` the interface to a `.pcm`, then `-fmodule-file=name=file.pcm`; or let CMake drive it
MSVC	interfaces in `.ixx` files; Visual Studio handles the ordering automatically
CMake	3.28+: `target_sources(app PUBLIC FILE_SET CXX_MODULES FILES geometry.cppm)` with the Ninja generator scans dependencies for you

> **TIP**
> For new projects, modules are worth trying with a recent compiler and CMake. For libraries that must support many compilers, headers remain the safe default for now; a library can offer both.

21.4 Coroutines (C++20)

21.4.1 co_await, co_yield, co_return

A **coroutine** is a function that can **suspend** in the middle and be **resumed** later, keeping its local variables. Any function that uses `co_yield` (produce a value and suspend), `co_await` (suspend until something is ready) or `co_return` (finish) is a coroutine. Its state lives in a heap-allocated **coroutine frame**, and the caller controls it through a handle. The language provides only the mechanism: the return type (a "promise type") decides the behaviour, so C++20 coroutines need a small library type. Here is a minimal generator written by hand, to show the moving parts.

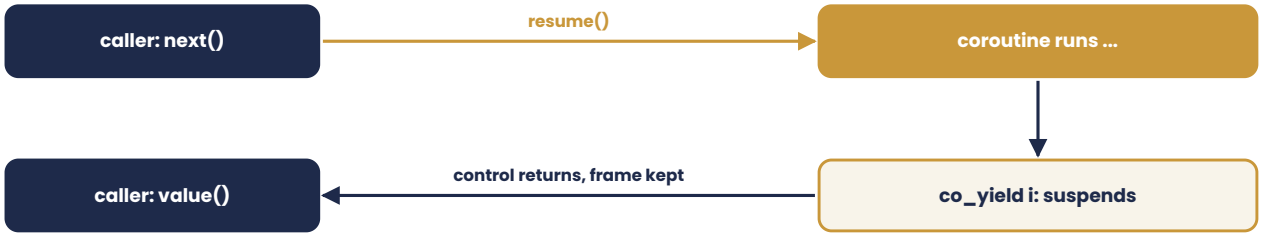
A COROUTINE AND A HAND-WRITTEN GENERATOR TYPE

main.cpp

```
#include <coroutine>
#include <iostream>

template <typename T>
class Generator {                                       // a minimal coroutine return type
public:
    struct promise_type {
        T current;
        Generator get_return_object() { return
Generator{std::coroutine_handle<promise_type>::from_promise(*this)}; }
        std::suspend_always initial_suspend() noexcept { return {}; }   // start suspended
        std::suspend_always final_suspend() noexcept { return {}; }
        std::suspend_always yield_value(T value) { current = value; return {}; }   // co_yield
        void return_void() {}
        void unhandled_exception() { throw; }
    };
    explicit Generator(std::coroutine_handle<promise_type> h) : h_(h) {}
    Generator(Generator&& o) noexcept : h_(o.h_) { o.h_ = {}; }
    ~Generator() { if (h_) h_.destroy(); }
    bool next() { h_.resume(); return !h_.done(); }      // run until the next co_yield or the end
    T value() const { return h_.promise().current; }
private:
    std::coroutine_handle<promise_type> h_;
};

Generator<int> countdown(int from) {
    std::cout << "  (coroutine starts)\n";
    for (int i = from; i > 0; --i) {
        std::cout << "  (about to yield " << i << ")\n";
        co_yield i;                                     // suspend here, hand i to the caller
    }
    std::cout << "  (coroutine ends)\n";
}

int main() {
    auto gen = countdown(3);                            // nothing runs yet
    std::cout << "generator created\n";
    while (gen.next()) std::cout << "caller got " << gen.value() << '\n';
    std::cout << "done\n";
    return 0;
}
```

OUTPUT

```
generator created
  (coroutine starts)
  (about to yield 3)
caller got 3
  (about to yield 2)
caller got 2
  (about to yield 1)
caller got 1
  (coroutine ends)
done
```



CONTROL PASSES BACK AND FORTH; THE COROUTINE'S LOCALS SURVIVE BETWEEN CALLS

Keyword	Meaning
`co_yield expr`	hand a value to the caller and suspend (generators)
`co_await expr`	suspend until the awaited operation completes, then continue with its result (asynchronous I/O, timers, tasks)
`co_return expr`	finish the coroutine, optionally with a final result

21.4.2 Generators with std::generator (C++23)

C++23 adds `std::generator<T>` (header `<generator>`), a ready-made generator that is also a range: you can loop over it with range-for and feed it into views. With it, a lazy sequence is just a function with `co_yield`.

STD::GENERATOR (GCC 14 OR LATER) main.cpp

```
#include <generator>
#include <iostream>
#include <ranges>

std::generator<long long> fibonacci() {            // an infinite, lazy sequence
    long long a = 0, b = 1;
    while (true) {
        co_yield a;
        auto next = a + b;
        a = b;
        b = next;
    }
}

int main() {
    for (long long f : fibonacci() | std::views::take(10)) std::cout << f << ' ';
    std::cout << '\n';
    for (long long f : fibonacci() | std::views::filter([](long long x) { return x % 2 == 0; }) |
std::views::take(5))
        std::cout << f << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
0 1 1 2 3 5 8 13 21 34
0 2 8 34 144
```

<generator> ships with GCC 14 and MSVC 19.37; the output is what such a compiler prints. With GCC 13, use a small generator class like the one above (the checkpoint does this).

21.4.3 Use cases: lazy sequences and asynchronous code

- **Lazy sequences**: generate values on demand (Fibonacci, primes, lines of a huge file, tree traversals) without building the whole sequence in memory, and stop whenever you like.
- **Asynchronous code**: with a task library (Boost.Asio, cppcoro, libunifex, and std::execution in C++26), `co_await socket.read(...)` looks like ordinary sequential code but does not block a thread while waiting. Thousands of connections can be served by a handful of threads.
- **State machines**: parsers and protocols written as straight-line code that suspends while waiting for the next input, instead of explicit state variables and switch statements.

WHAT CO_AWAIT LOOKS LIKE IN A NETWORKING LIBRARY C++

```
// Asynchronous style with a coroutine-aware library (Boost.Asio shown):
asio::awaitable<void> echo(tcp::socket socket) {
    char data[1024];
    for (;;) {
        std::size_t n = co_await socket.async_read_some(asio::buffer(data), asio::use_awaitable);
        co_await asio::async_write(socket, asio::buffer(data, n), asio::use_awaitable);
    }   // no callbacks, no blocked threads: the coroutine suspends at each co_await
}
```

21.5 Other C++20 features

21.5.1 Three-way comparison <=>

The "spaceship" operator `a <=> b` compares once and returns an ordering object that says less, equal or greater. Defaulting it, `auto operator<=>(const T&) const = default;`, compares members in declaration order and lets the compiler generate all of `< <= > >=`; a defaulted `==` gives `!=`. The result type is `std::strong_ordering` (integers), `std::weak_ordering` (equivalent but distinguishable values, like case-insensitive strings) or `std::partial_ordering` (some values are unordered, like NaN). Module 15 covered it with operator overloading.

THE SPACESHIP OPERATOR

main.cpp

```cpp
#include <compare>
#include <iostream>
#include <string>

struct Version {
    int major, minor, patch;
    auto operator<=>(const Version&) const = default;        // all six comparisons, member by member
};

struct Student {
    std::string name;
    double gpa;
    std::partial_ordering operator<=>(const Student& o) const { return gpa <=> o.gpa; }   // compare by gpa only
    bool operator==(const Student& o) const { return gpa == o.gpa; }
};

const char* word(std::strong_ordering o) { return o < 0 ? "less" : o > 0 ? "greater" : "equal"; }

int main() {
    Version a{1, 4, 2}, b{1, 10, 0};
    std::cout << std::boolalpha << "a < b: " << (a < b) << ", a == b: " << (a == b) << ", a >= b: " << (a >= b) << '\n';
    std::cout << "3 <=> 5 is " << word(3 <=> 5) << ", 7 <=> 7 is " << word(7 <=> 7) << '\n';
    std::cout << "\"pear\" <=> \"apple\" is " << word(std::string("pear") <=> std::string("apple")) << '\n';
    Student x{"Asha", 8.7}, y{"Ravi", 9.1};
    std::cout << "Asha > Ravi by gpa? " << (x > y) << '\n';
    double nan = 0.0 / 0.0;
    auto r = 1.0 <=> nan;
    std::cout << "1.0 <=> NaN unordered? " << (r == std::partial_ordering::unordered) << '\n';
    return 0;
}
```

OUTPUT

```
a < b: true, a == b: false, a >= b: false
3 <=> 5 is less, 7 <=> 7 is equal
"pear" <=> "apple" is greater
Asha > Ravi by gpa? false
1.0 <=> NaN unordered? true
```

21.5.2 std::span

`std::span<T>` (header `<span>`) is a non-owning view of a **contiguous** sequence: pointer plus length, like `string_view` but for any element type and writable unless `T` is const. A function taking `std::span<const int>` accepts a vector, an array, a `std::array` or part of any of them, replacing the error-prone (pointer, size) parameter pairs of C. `first(n)`, `last(n)` and `subspan(offset, count)` make sub-views.

ONE FUNCTION FOR EVERY CONTIGUOUS CONTAINER

main.cpp

```
#include <array>
#include <iostream>
#include <numeric>
#include <span>
#include <vector>

double average(std::span<const int> values) {              // works for any contiguous ints
    if (values.empty()) return 0;
    return std::accumulate(values.begin(), values.end(), 0.0) / values.size();
}
void scale(std::span<int> values, int factor) { for (int& v : values) v *= factor; }   // writable
view

int main() {
    std::vector<int> vec = {10, 20, 30, 40, 50};
    int raw[] = {1, 2, 3};
    std::array<int, 4> arr = {5, 5, 6, 8};
    std::cout << average(vec) << ' ' << average(raw) << ' ' << average(arr) << '\n';

    std::span<int> all(vec);
    std::cout << "first 2 avg " << average(all.first(2)) << ", last 2 avg " << average(all.last(2))
              << ", middle " << average(all.subspan(1, 3)) << '\n';
    scale(all.subspan(3), 10);                                    // modify part of the vector
    for (int v : vec) std::cout << v << ' ';
    std::cout << "| sizeof(span) = " << sizeof(all) << '\n';
    return 0;
}
```

OUTPUT

```
30 2 6
first 2 avg 15, last 2 avg 45, middle 30
10 20 30 400 500 | sizeof(span) = 16
```

21.5.3 std::format

`std::format` (header `<format>`) builds strings with Python-like `{}` placeholders. It is type-safe (no printf format mismatches), extensible to your own types, and fast. Inside the braces: an optional argument index, then `:` and a format spec: fill and alignment (`< > ^`), sign, `#`, width, `.precision` and type (`d x b o e f g s`). Module 5 introduced it; here are more options, and a formatter for a custom type.

STD::FORMAT IN DEPTH

main.cpp

```
#include <format>
#include <iostream>
#include <string>
#include <vector>

struct Money { long long paise; };

template <>
struct std::formatter<Money> : std::formatter<std::string> {     // custom type support
    auto format(const Money& m, std::format_context& ctx) const {
        return std::formatter<std::string>::format(std::format("Rs {}.{:02}", m.paise / 100, m.paise %
100), ctx);
    }
};

int main() {
    std::cout << std::format("{} scored {} in {}\n", "Asha", 91, "C++");
    std::cout << std::format("{1} before {0}\n", "second", "first");                // argument
indexes
    std::cout << std::format("[{:>8}] [{:<8}] [{:^8}] [{:*^8}]\n", "right", "left", "mid", "star");
    std::cout << std::format("{:.3f} {:10.2e} {:+d} {:08.2f}\n", 3.14159, 123456.0, 42, -3.5);
    std::cout << std::format("hex {:x} {:#X}  bin {:#010b}  oct {:o}\n", 255, 255, 5, 8);
    std::cout << std::format("{:>12}|\n", Money{1234505});                          // our formatter
+ width
    std::string row;
    for (int w : {5, 12, 7}) row += std::format("|{:^{}}", w, w + 2);               // dynamic
width
    std::cout << row << "|\n";
    return 0;
}
```

OUTPUT

```
Asha scored 91 in C++
first before second
[   right] [left    ] [  mid   ] [**star**]
3.142   1.23e+05 +42 -0003.50
hex ff 0XFF  bin 0b00000101  oct 10
 Rs 12345.05|
|   5   |      12      |    7    |
```

21.5.4 Designated initializers

Aggregates can be initialised by naming members: `Config c{.width = 800, .fullscreen = true};`. Members must appear in declaration order; omitted members get their default member initializers (or zero). This makes calls with many options self-documenting.

NAMING THE MEMBERS YOU SET

main.cpp

```
#include <iostream>
#include <string>

struct WindowOptions {
    std::string title = "Untitled";
    int width = 640;
    int height = 480;
    bool resizable = true;
    bool fullscreen = false;
};

void open_window(const WindowOptions& o) {
    std::cout << o.title << ": " << o.width << "x" << o.height << (o.resizable ? ", resizable" : "")
<< (o.fullscreen ? ", fullscreen" : "") << '\n';
}

int main() {
    open_window({});                                                    // all defaults
    open_window({.title = "Editor", .width = 1280, .height = 720});     // the rest default
    open_window({.title = "Game", .resizable = false, .fullscreen = true});
    // open_window({.height = 600, .width = 800});  error: designators must follow declaration order
    return 0;
}
```

OUTPUT

```
Untitled: 640x480, resizable
Editor: 1280x720, resizable
Game: 640x480, fullscreen
```

21.5.5 std::jthread and stop tokens (Module 23)

`std::jthread` is a thread that **joins automatically** in its destructor (a `std::thread` that is neither joined nor detached calls `std::terminate`), and supports cooperative cancellation: the thread function may take a `std::stop_token` and check `stop_requested()`; the destructor requests a stop before joining. Module 23 covers threads in detail.

STD::JTHREAD WITH A STOP TOKEN

main.cpp

```cpp
#include <chrono>
#include <iostream>
#include <thread>
using namespace std::chrono_literals;

int main() {
    std::jthread worker([](std::stop_token st) {
        int ticks = 0;
        while (!st.stop_requested()) {          // cooperative cancellation
            std::this_thread::sleep_for(20ms);
            ++ticks;
        }
        std::cout << "worker stopping after at least " << (ticks >= 3 ? "3" : "?") << " ticks\n";
    });
    std::this_thread::sleep_for(100ms);
    std::cout << "main: requesting stop\n";
    worker.request_stop();
    return 0;                                   // jthread joins automatically
}
```

OUTPUT

```
main: requesting stop
worker stopping after at least 3 ticks
```

21.6 Other C++23 features

21.6.1 std::print / std::println

C++23 `std::print` and `std::println` (header `<print>`) combine `std::format` with output: `std::println("{} + {} = {}", a, b, a + b);`. They are faster than iostreams, handle Unicode correctly on all platforms, and need no `<<` chains. `std::print(stderr, ...)` writes to another stream.

STD::PRINTLN (GCC 14 OR LATER)

main.cpp

```cpp
#include <print>
#include <string>
#include <vector>

int main() {
    std::string name = "Asha";
    std::println("Hello, {}!", name);
    std::println("{:<8}{:>6}", "Item", "Price");
    for (auto [item, price] : std::vector<std::pair<std::string, double>>{{"Tea", 15}, {"Samosa",
18.5}})
        std::println("{:<8}{:>6.2f}", item, price);
    std::print("no newline, ");
    std::println("then a newline");
    std::println(stderr, "warnings go to stderr");
    return 0;
}
```

OUTPUT

```
Hello, Asha!
Item     Price
Tea      15.00
Samosa   18.50
no newline, then a newline
warnings go to stderr
```

<print> needs GCC 14, Clang 18 with libc++ or MSVC 19.37. Output shown as printed by GCC 14 (the stderr line appears in the terminal too).

21.6.2 std::expected

`std::expected<T, E>` (header `<expected>`) holds either a value of type T or an error of type E, so a function can report *why* it failed without exceptions. Return `std::unexpected(error)` for failures. Module 16 introduced it; its monadic operations `and_then` (chain another step that may fail), `transform` (change the value) and `or_else` (handle the error) build error-propagating pipelines with no if statements.

CHAINING STD::EXPECTED

main.cpp

```
#include <charconv>
#include <expected>
#include <iostream>
#include <string>
#include <string_view>

enum class Err { Empty, NotNumber, Negative };
std::string to_text(Err e) {
    switch (e) { case Err::Empty: return "empty"; case Err::NotNumber: return "not a number"; case
Err::Negative: return "negative"; }
    return "?";
}

std::expected<int, Err> parse(std::string_view s) {
    if (s.empty()) return std::unexpected(Err::Empty);
    int v = 0;
    auto [p, ec] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (ec != std::errc{} || p != s.data() + s.size()) return std::unexpected(Err::NotNumber);
    return v;
}
std::expected<int, Err> non_negative(int v) {
    if (v < 0) return std::unexpected(Err::Negative);
    return v;
}

int main() {
    for (std::string_view in : {"25", "-4", "x1", ""}) {
        auto result = parse(in)
                          .and_then(non_negative)                          // another fallible
step
                          .transform([](int v) { return v * v; });          // plain
transformation
        if (result) std::cout << '"' << in << "\" -> square " << *result << '\n';
        else std::cout << '"' << in << "\" -> error: " << to_text(result.error()) << '\n';
    }
    std::cout << "value_or: " << parse("oops").value_or(0) << '\n';
    return 0;
}
```

OUTPUT

```
"25" -> square 625
"-4" -> error: negative
"x1" -> error: not a number
"" -> error: empty
value_or: 0
```

21.6.3 Deducing this

C++23 lets a member function take the object as an explicit first parameter marked `this`: `void f(this Self&& self)`. Because `Self` is deduced, one function template replaces the usual four overloads (`const&`, `&`, `&&`, `const&&`), a lambda can call itself recursively, and the Curiously Recurring Template Pattern (Module 22) becomes simpler.

EXPLICIT OBJECT PARAMETERS (BUILT WITH CLANG 18)

main.cpp

```cpp
#include <iostream>
#include <string>
#include <utility>

class Name {
public:
    explicit Name(std::string s) : value_(std::move(s)) {}
    template <typename Self>
    auto&& get(this Self&& self) {                      // one function for const, non-const and
rvalue objects
        return std::forward<Self>(self).value_;
    }
private:
    std::string value_;
};

struct Base {
    void hello(this const auto& self) { std::cout << "hello from " << self.name() << '\n'; }   //
static polymorphism
};
struct English : Base { std::string name() const { return "English"; } };
struct Kannada : Base { std::string name() const { return "Kannada"; } };

int main() {
    Name n("Asha");
    n.get() += " Rao";                                  // non-const: returns std::string&
    const Name& cn = n;
    std::cout << cn.get() << '\n';                      // const: returns const std::string&
    std::string stolen = Name("Temp").get();            // rvalue: returns std::string&& (moved)
    std::cout << stolen << '\n';

    English{}.hello();
    Kannada{}.hello();

    auto fact = [](this auto self, int k) -> long long { return k <= 1 ? 1 : k * self(k - 1); };   //
recursive lambda
    std::cout << "10! = " << fact(10) << '\n';
    return 0;
}
```

OUTPUT

```
Asha Rao
Temp
hello from English
hello from Kannada
10! = 3628800
```

21.6.4 std::mdspan, std::flat_map

`std::mdspan` (C++23, header `<mdspan>`) is a multi-dimensional, non-owning view over contiguous data: a matrix or image indexed as `m[i, j]` (C++23 allows several arguments in `operator[]`), with customisable layout (row-major, column-major, strided). `std::flat_map` and `std::flat_set` (header `<flat_map>`) have the interface of map and set but store keys and values in sorted vectors: lookups use binary search and iteration is cache-friendly, much faster for read-mostly data, while insertion is O(n).

MDSPAN AND FLAT_MAP (GCC 15)

main.cpp

```
#include <flat_map>
#include <iostream>
#include <mdspan>
#include <string>
#include <vector>

int main() {
    std::vector<int> pixels(3 * 4);
    std::mdspan img(pixels.data(), 3, 4);                    // 3 rows x 4 columns over the vector
    for (std::size_t r = 0; r < img.extent(0); ++r)
        for (std::size_t c = 0; c < img.extent(1); ++c)
            img[r, c] = static_cast<int>(r * 10 + c);        // multidimensional subscript
    std::cout << "img[2, 3] = " << img[2, 3] << ", pixels[11] = " << pixels[11] << '\n';

    std::flat_map<std::string, int> prices = {{"tea", 15}, {"coffee", 25}, {"juice", 40}};
    prices["samosa"] = 18;
    for (const auto& [item, p] : prices) std::cout << item << '=' << p << ' ';
    std::cout << "| keys stored contiguously: " << prices.keys().size() << '\n';
    return 0;
}
```

OUTPUT

```
img[2, 3] = 23, pixels[11] = 23
coffee=25 juice=40 samosa=18 tea=15 | keys stored contiguously: 4
```

std::mdspan arrived in GCC 15 (libstdc++) and Clang 17 (libc++); std::flat_map in GCC 15 and MSVC 19.42. Output as printed by GCC 15.

21.7 C++26 overview

C++26 was finalised in 2025-2026. Its headline features change how libraries are written. Compiler support is arriving gradually; the examples show the adopted syntax.

21.7.1 Static reflection

Reflection lets a program inspect its own declarations at compile time. The operator `^^` produces a reflection (a value of type `std::meta::info`) of a type, member or namespace; functions in `<meta>` query it (`members_of`, `identifier_of`, `enumerators_of`…); and a **splice** `[: r :]` turns a reflection back into code. Tasks that need macros or code generators today, such as converting an enum to a string, serialising a struct to JSON or generating bindings, become ordinary library code.

ENUM TO STRING WITH C++26 REFLECTION

C++

```
#include <meta>
#include <string>

template <typename E>
    requires std::is_enum_v<E>
constexpr std::string enum_to_string(E value) {
    template for (constexpr auto e : std::define_static_array(std::meta::enumerators_of(^^E)))  //
expansion statement
        if (value == [:e:]) return std::string(std::meta::identifier_of(e));
    return "<unnamed>";
}

enum class Color { red, green, blue };
static_assert(enum_to_string(Color::green) == "green");   // computed by the compiler
```

21.7.2 Contracts

Contracts state the conditions a function relies on and guarantees, as part of its declaration: `pre(condition)` preconditions, `post(r: condition)` postconditions on the result `r`, and `contract_assert(condition)` inside function bodies. The build chooses how they are checked: ignore, observe (report and continue), enforce (report and terminate) or quick-enforce. Contracts document interfaces in a way tools and compilers can check, replacing ad-hoc assert macros.

C++26 CONTRACTS C++

```
double safe_sqrt(double x)
    pre(x >= 0)                          // the caller must guarantee this
    post(r: r >= 0)                      // the function guarantees this
{
    return std::sqrt(x);
}

void transfer(Account& from, Account& to, Money amount)
    pre(amount > Money{0})
    pre(&from != &to)
{
    contract_assert(from.balance() >= amount);   // checked inside the body
    from.withdraw(amount);
    to.deposit(amount);
}
```

21.7.3 std::execution (senders and receivers)

`std::execution` (from the P2300 proposal) is a standard model for asynchronous and parallel work. A **scheduler** represents an execution resource (a thread pool, a GPU, an event loop); a **sender** describes work that will produce a value; algorithms such as `then`, `when_all` and `let_value` compose senders into graphs, and `sync_wait` runs them. Senders are also awaitable, so they combine naturally with coroutines. It gives C++ a common foundation that networking, GPU and task libraries can share.

COMPOSING WORK WITH SENDERS C++

```
#include <execution>
namespace ex = std::execution;

auto pool = get_thread_pool_scheduler();              // e.g. std::execution::parallel_scheduler
auto work = ex::when_all(
                ex::schedule(pool) | ex::then([] { return load_prices(); }),
                ex::schedule(pool) | ex::then([] { return load_orders(); }))
          | ex::then([](auto prices, auto orders) { return compute_bill(prices, orders); });
auto [bill] = std::this_thread::sync_wait(std::move(work)).value();   // run and wait
```

Other C++26 additions	In one line
`std::inplace_vector<T, N>`	a vector with fixed capacity stored inside the object, no heap
Pack indexing `Ts...[0]`	pick an element of a parameter pack directly
`= delete("reason")`	deleted functions with a message
Placeholder `_`	unnamed variables: `auto [_, value] = pair;`
`#embed`	include a binary file as data at compile time

Other C++26 additions	In one line
Saturation arithmetic, `std::linalg`, `std::hive`	add_sat/mul_sat, BLAS-style linear algebra, a stable-pointer bucket container
Erroneous behaviour for uninitialised reads	reading an uninitialised variable is no longer undefined behaviour

21.7.4 Checking compiler support for new features

Before using a new feature, check that your compiler supports it. The reference is the compiler support table at cppreference.com. In code, **feature-test macros** (header `<version>`) report support at compile time, so a program can fall back gracefully: `__cpp_concepts`, `__cpp_lib_ranges`, `__cpp_lib_print` and hundreds more, each defined as the year and month of the feature's version. `__has_include(<header>)` tests whether a header exists.

FEATURE-TEST MACROS (GCC 13, -STD=C++23)

main.cpp

```cpp
#include <iostream>
#include <version>

int main() {
    std::cout << "__cplusplus           = " << __cplusplus << '\n';
#ifdef __cpp_concepts
    std::cout << "concepts              : yes (" << __cpp_concepts << ")\n";
#endif
#ifdef __cpp_lib_ranges
    std::cout << "ranges library        : yes (" << __cpp_lib_ranges << ")\n";
#endif
#ifdef __cpp_lib_format
    std::cout << "std::format           : yes (" << __cpp_lib_format << ")\n";
#endif
#ifdef __cpp_lib_expected
    std::cout << "std::expected         : yes (" << __cpp_lib_expected << ")\n";
#else
    std::cout << "std::expected         : no\n";
#endif
#ifdef __cpp_lib_print
    std::cout << "std::print            : yes\n";
#else
    std::cout << "std::print            : no, fall back to std::format + cout\n";
#endif
#ifdef __cpp_explicit_this_parameter
    std::cout << "deducing this         : yes\n";
#else
    std::cout << "deducing this         : no\n";
#endif
#if __has_include(<generator>)
    std::cout << "<generator> header    : found\n";
#else
    std::cout << "<generator> header    : missing\n";
#endif
    return 0;
}
```

OUTPUT

```
__cplusplus           = 202100
concepts              : yes (202002)
ranges library        : yes (202202)
std::format           : yes (202110)
std::expected         : yes (202211)
std::print            : no, fall back to std::format + cout
deducing this         : no
<generator> header    : missing
```

Concepts constrain templates and make errors readable; ranges and views turn loops into lazy pipelines; modules replace textual inclusion; coroutines make lazy generators and asynchronous code read like ordinary code. Use <=>, span, format, designated initializers and jthread from C++20, print, expected and deducing this from C++23, and check support with feature-test macros before relying on the newest features.

PRACTICE CHECKPOINT

Constrain the **Stack<T>** template with a concept, rewrite a data-processing loop as a **ranges pipeline**, and write a **Fibonacci generator with co_yield**.

SAMPLE SOLUTION

main.cpp

```
#include <concepts>
#include <coroutine>
#include <iostream>
#include <ranges>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

// ---------- 1. Stack<T> constrained with a concept ----------
template <typename T>
concept Storable = std::movable<T> && std::default_initializable<T>;

template <Storable T>
class Stack {
public:
    void push(T v) { items_.push_back(std::move(v)); }
    T pop() {
        if (items_.empty()) throw std::out_of_range("empty stack");
        T v = std::move(items_.back());
        items_.pop_back();
        return v;
    }
    bool empty() const { return items_.empty(); }
private:
    std::vector<T> items_;
};
struct NoDefault { explicit NoDefault(int) {} };
static_assert(!Storable<NoDefault>);                // Stack<NoDefault> would be rejected at the
declaration

// ---------- 3. A minimal generator and a Fibonacci coroutine ----------
template <typename T>
class Generator {
public:
    struct promise_type {
        T current{};
        Generator get_return_object() { return
Generator{std::coroutine_handle<promise_type>::from_promise(*this)}; }
        std::suspend_always initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        std::suspend_always yield_value(T v) { current = v; return {}; }
        void return_void() {}
        void unhandled_exception() { throw; }
    };
    explicit Generator(std::coroutine_handle<promise_type> h) : h_(h) {}
    Generator(Generator&& o) noexcept : h_(std::exchange(o.h_, {})) {}
    ~Generator() { if (h_) h_.destroy(); }
    bool next() { h_.resume(); return !h_.done(); }
    T value() const { return h_.promise().current; }
private:
    std::coroutine_handle<promise_type> h_;
};

Generator<long long> fibonacci(int count) {
    long long a = 0, b = 1;
    for (int i = 0; i < count; ++i) {
        co_yield a;
        long long next = a + b;
        a = b;
        b = next;
    }
```

```
}

struct Sale { std::string city; double amount; };

int main() {
    Stack<std::string> s;
    s.push("concepts");
    s.push("ranges");
    std::cout << "popped " << s.pop() << ", then " << s.pop() << '\n';

    // ---------- 2. Loop vs ranges pipeline ----------
    std::vector<Sale> sales = {{"Mysuru", 1200}, {"Pune", 300}, {"Mysuru", 4500}, {"Delhi", 800},
{"Mysuru", 2600}, {"Mysuru", 90}};
    std::vector<double> loop_result;                          // the old way
    for (const auto& s2 : sales)
        if (s2.city == "Mysuru" && s2.amount >= 1000) {
            loop_result.push_back(s2.amount * 1.18);          // add 18% GST
            if (loop_result.size() == 2) break;
        }
    auto pipeline = sales
                  | std::views::filter([](const Sale& x) { return x.city == "Mysuru"; })
                  | std::views::filter([](const Sale& x) { return x.amount >= 1000; })
                  | std::views::transform([](const Sale& x) { return x.amount * 1.18; })
                  | std::views::take(2);
    std::cout << "loop:    ";
    for (double v : loop_result) std::cout << v << ' ';
    std::cout << "\npipeline:";
    for (double v : pipeline) std::cout << ' ' << v;
    std::cout << '\n';

    auto fib = fibonacci(12);
    std::cout << "fibonacci:";
    while (fib.next()) std::cout << ' ' << fib.value();
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
popped ranges, then concepts
loop:    1416 5310
pipeline: 1416 5310
fibonacci: 0 1 1 2 3 5 8 13 21 34 55 89
```

MODULE 22 · ADVANCED

Advanced templates and metaprogramming

Minimum duration **3 days** | Level **Advanced**

Templates are not only for containers: the compiler can run code, make decisions about types and generate specialised implementations entirely at compile time. This is template metaprogramming, the technique behind high-performance libraries such as the STL, Eigen and fmt. This module goes deeper into variadic templates, type traits, compile-time selection (SFINAE, if constexpr, concepts), compile-time computation and the classic template idioms.

IN THIS MODULE

22.1 Variadic templates in depth
22.2 Type traits (<type_traits>)
22.3 Compile-time selection
22.4 Compile-time computation
22.5 Template idioms
22.6 Templates and code organisation

22.1 Variadic templates in depth

Module 17 introduced parameter packs. Here we look at how packs are expanded, how to process them recursively, how folds work in detail, and what `sizeof...` offers.

22.1.1 Recursive expansion

Before C++17, the only way to "loop" over a pack was recursion: a function handles the first element and calls itself with the rest, and a separate overload or specialization ends the recursion. The same technique works for class templates, which lets us compute properties of a list of types. Any pattern followed by `...` is expanded once per element: `f(args)...`, `std::forward<Ts>(args)...`, `sizeof(Ts)...`.

RECURSIVE PROCESSING OF PACKS

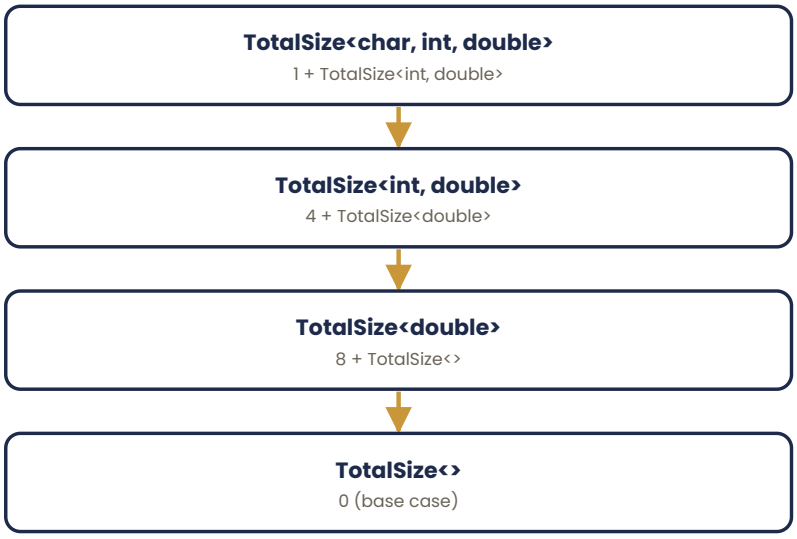
main.cpp

```
#include <iostream>
#include <string>

// Function recursion: the base case ends it
void print_line() { std::cout << '\n'; }
template <typename T, typename... Rest>
void print_line(const T& first, const Rest&... rest) {
    std::cout << first;
    if constexpr (sizeof...(rest) > 0) std::cout << " | ";
    print_line(rest...);
}

// Class recursion: total size of a list of types
template <typename... Ts> struct TotalSize;                          // primary (declared only)
template <> struct TotalSize<> { static constexpr std::size_t value = 0; };   // base case
template <typename T, typename... Rest>
struct TotalSize<T, Rest...> {
    static constexpr std::size_t value = sizeof(T) + TotalSize<Rest...>::value;
};
```

```cpp
// Does a pack contain a type?
template <typename T, typename... Ts> struct Contains { static constexpr bool value = false; };
template <typename T, typename First, typename... Rest>
struct Contains<T, First, Rest...> {
    static constexpr bool value = std::is_same_v<T, First> || Contains<T, Rest...>::value;
};

// Pattern expansion: apply an operation to every argument
template <typename... Ts>
void print_sizes(const Ts&... args) {
    std::size_t sizes[] = {sizeof(args)...};                        // expands to {sizeof(a1),
sizeof(a2), ...}
    for (auto s : sizes) std::cout << s << ' ';
    std::cout << '\n';
}

int main() {
    print_line(1, "two", 3.5, std::string("four"));
    std::cout << "TotalSize<char, int, double> = " << TotalSize<char, int, double>::value << '\n';
    std::cout << std::boolalpha << "Contains<double, int, double> = " << Contains<double, int,
double>::value
              << ", Contains<float, int, double> = " << Contains<float, int, double>::value << '\n';
    print_sizes('a', 1, 2.0, 3LL);
    return 0;
}
```

OUTPUT

```
1 | two | 3.5 | four
TotalSize<char, int, double> = 13
Contains<double, int, double> = true, Contains<float, int, double> = false
1 4 8 8
```



THE COMPILER INSTANTIATES ONE SPECIALIZATION PER STEP: 1 + 4 + 8 + 0 = 13

22.1.2 Fold expressions

Folds (C++17) replace most recursion. A fold expands a pack with a binary operator: `(args + ...)` is a right fold `a1 + (a2 + (a3))`, `(... + args)` a left fold `((a1 + a2) + a3)`, and `(init op ... op args)` adds a starting value. The direction matters for non-associative operators such as `-`. The comma operator fold `(f(args), ...)` calls a function for each element in order, and the pattern inside the fold can be any expression involving the pack.

FOLDS IN DETAIL

main.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

template <typename... Ts> auto left_minus(Ts... xs)  { return (... - xs); }   // ((a - b) - c) - d
template <typename... Ts> auto right_minus(Ts... xs) { return (xs - ...); }   // a - (b - (c - d))

template <typename... Ts>
bool all_even(Ts... xs) { return ((xs % 2 == 0) && ...); }                    // any expression as the pattern

template <typename T, typename... Ts>
void push_all(std::vector<T>& v, Ts&&... xs) { (v.push_back(std::forward<Ts>(xs)), ...); }   // comma fold

template <typename... Ts>
std::string join(const Ts&... parts) {
    std::string out;
    ((out += std::string(parts) + ","), ...);
    if (!out.empty()) out.pop_back();
    return out;
}

template <typename... Ts>
auto max_of(Ts... xs) {
    auto m = (xs, ...);                    // start with the last value (comma fold)
    ((m = xs > m ? xs : m), ...);          // then keep the largest
    return m;
}

int main() {
    std::cout << "left fold 10-3-2-1  = " << left_minus(10, 3, 2, 1) << '\n';
    std::cout << "right fold 10-3-2-1 = " << right_minus(10, 3, 2, 1) << '\n';
    std::cout << std::boolalpha << "all_even(2, 4, 6) = " << all_even(2, 4, 6) << ", all_even(2, 5) = " << all_even(2, 5) << '\n';
    std::vector<int> v;
    push_all(v, 1, 2, 3, 4);
    std::cout << "pushed " << v.size() << " values\n";
    std::cout << "join: " << join("a", "bc", std::string("def")) << '\n';
    std::cout << "max_of(3, 9, 4) = " << max_of(3, 9, 4) << '\n';
    return 0;
}
```

OUTPUT

```
left fold 10-3-2-1  = 4
right fold 10-3-2-1 = 8
all_even(2, 4, 6) = true, all_even(2, 5) = false
pushed 4 values
join: a,bc,def
max_of(3, 9, 4) = 9
```

22.1.3 sizeof...

`sizeof...(pack)` is a compile-time constant giving the number of elements in a type or value pack. It is often used in `static_assert` to check argument counts, in `if constexpr` to detect the last element, and to size arrays.

COUNTING PACK ELEMENTS main.cpp

```
#include <array>
#include <iostream>
#include <string>

template <typename... Ts>
constexpr std::size_t count(const Ts&...) { return sizeof...(Ts); }

template <typename... Ts>
auto to_array(Ts... xs) {
    static_assert(sizeof...(Ts) > 0, "need at least one value");
    return std::array<int, sizeof...(Ts)>{static_cast<int>(xs)...};   // size from the pack
}

template <typename... Ts>
void rgb(Ts... values) {
    static_assert(sizeof...(values) == 3, "rgb() needs exactly 3 values");
    ((std::cout << values << ' '), ...);
    std::cout << '\n';
}

int main() {
    static_assert(count(1, 'a', "b") == 3);
    auto arr = to_array(5, 6.9, 'A');
    std::cout << "array of " << arr.size() << ": " << arr[0] << ' ' << arr[1] << ' ' << arr[2] <<
'\n';
    rgb(201, 151, 58);
    // rgb(1, 2);   error: static assertion failed: rgb() needs exactly 3 values
    return 0;
}
```

OUTPUT

```
array of 3: 5 6 65
201 151 58
```

22.2 Type traits (<type_traits>)

A **type trait** is a template that answers a question about a type, or transforms one type into another, at compile time. They are the building blocks of generic code: `std::is_integral<T>::value` (or the shortcut `std::is_integral_v<T>`) is a `bool` constant; `std::remove_reference<T>::type` (or `std::remove_reference_t<T>`) is a type.

22.2.1 is_integral, is_same, is_pointer

Category	Examples
Primary categories	`is_integral`, `is_floating_point`, `is_pointer`, `is_array`, `is_enum`, `is_class`, `is_function`, `is_lvalue_reference`
Composite	`is_arithmetic`, `is_fundamental`, `is_object`, `is_reference`
Properties	`is_const`, `is_signed`, `is_trivially_copyable`, `is_abstract`, `is_polymorphic`, `is_empty`
Supported operations	`is_default_constructible`, `is_copy_constructible`, `is_nothrow_move_constructible`, `is_invocable`
Relations	`is_same`, `is_base_of`, `is_convertible`

ASKING QUESTIONS ABOUT TYPES

main.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <type_traits>
#include <vector>

struct Shape { virtual double area() const = 0; virtual ~Shape() = default; };
struct Circle : Shape { double r; double area() const override { return 3.14 * r * r; } };
enum class Color { Red };

int main() {
    std::cout << std::boolalpha;
    std::cout << "is_integral<char>            " << std::is_integral_v<char> << '\n';
    std::cout << "is_integral<float>           " << std::is_integral_v<float> << '\n';
    std::cout << "is_same<int, signed int>     " << std::is_same_v<int, signed int> << '\n';
    std::cout << "is_same<char, signed char>   " << std::is_same_v<char, signed char> << " (char is a
distinct type)\n";
    std::cout << "is_pointer<int*>             " << std::is_pointer_v<int*> << '\n';
    std::cout << "is_pointer<unique_ptr<int>>  " << std::is_pointer_v<std::unique_ptr<int>> << '\n';
    std::cout << "is_enum<Color>               " << std::is_enum_v<Color> << '\n';
    std::cout << "is_abstract<Shape>           " << std::is_abstract_v<Shape> << '\n';
    std::cout << "is_base_of<Shape, Circle>    " << std::is_base_of_v<Shape, Circle> << '\n';
    std::cout << "is_trivially_copyable<std::string> " << std::is_trivially_copyable_v<std::string> <<
'\n';
    std::cout << "is_nothrow_move_constructible<std::vector<int>> " <<
std::is_nothrow_move_constructible_v<std::vector<int>> << '\n';
    std::cout << "is_convertible<int, std::string> " << std::is_convertible_v<int, std::string> <<
'\n';
    return 0;
}
```

OUTPUT

```
is_integral<char>            true
is_integral<float>           false
is_same<int, signed int>     true
is_same<char, signed char>   false (char is a distinct type)
is_pointer<int*>             true
is_pointer<unique_ptr<int>>  false
is_enum<Color>               true
is_abstract<Shape>           true
is_base_of<Shape, Circle>    true
is_trivially_copyable<std::string> false
is_nothrow_move_constructible<std::vector<int>> true
is_convertible<int, std::string> false
```

22.2.2 remove_reference, decay, conditional

Transformation traits produce a new type. `remove_reference_t<int&>` is `int`; `remove_cv_t` strips const and volatile; `add_pointer_t`, `add_const_t`, `make_unsigned_t` add things. `decay_t<T>` applies what happens to a function argument passed by value: references and cv-qualifiers are removed, arrays become pointers, functions become function pointers. `conditional_t<cond, A, B>` is a compile-time if for types, and `common_type_t` finds a type that several types convert to.

TRANSFORMING TYPES

main.cpp

```cpp
#include <cstdint>
#include <iostream>
#include <string>
#include <type_traits>
#include <typeinfo>

template <typename T>
void show(const char* label) {                  // print a readable name for a few types
    std::string n = std::is_same_v<T, int> ? "int" : std::is_same_v<T, const int> ? "const int" :
                    std::is_same_v<T, int*> ? "int*" : std::is_same_v<T, const char*> ? "const char*"
:
                    std::is_same_v<T, unsigned int> ? "unsigned int" : std::is_same_v<T, double> ?
"double" :
                    std::is_same_v<T, std::int64_t> ? "int64_t" : std::is_same_v<T, std::int32_t> ?
"int32_t" :
                    std::is_same_v<T, void (*)(int)> ? "void(*)(int)" : "other";
    std::cout << label << n << '\n';
}

template <std::size_t Bytes>
using Int = std::conditional_t<(Bytes > 4), std::int64_t, std::int32_t>;   // choose a type by a
constant

int main() {
    show<std::remove_reference_t<int&>>("remove_reference_t<int&>       = ");
    show<std::remove_const_t<const int>>("remove_const_t<const int>      = ");
    show<std::remove_cvref_t<const int&>>("remove_cvref_t<const int&>     = ");
    show<std::add_pointer_t<int>>("add_pointer_t<int>             = ");
    show<std::make_unsigned_t<int>>("make_unsigned_t<int>           = ");
    show<std::decay_t<const int&>>("decay_t<const int&>            = ");
    show<std::decay_t<int[5]>>("decay_t<int[5]>                = ");
    show<std::decay_t<const char[6]>>("decay_t<const char[6]>         = ");
    show<std::decay_t<void(int)>>("decay_t<void(int)>             = ");
    show<std::common_type_t<int, double>>("common_type_t<int, double>     = ");
    show<Int<8>>("Int<8> (conditional)           = ");
    show<Int<2>>("Int<2> (conditional)           = ");
    return 0;
}
```

OUTPUT

```
remove_reference_t<int&>       = int
remove_const_t<const int>      = int
remove_cvref_t<const int&>     = int
add_pointer_t<int>             = int*
make_unsigned_t<int>           = unsigned int
decay_t<const int&>            = int
decay_t<int[5]>                = int*
decay_t<const char[6]>         = const char*
decay_t<void(int)>             = void(*)(int)
common_type_t<int, double>     = double
Int<8> (conditional)           = int64_t
Int<2> (conditional)           = int
```

22.2.3 Writing your own type traits

A trait is just a class template, usually deriving from `std::true_type` or `std::false_type` (which provide `value`) and specialised for the cases that should be true. By convention a `_v` variable template and a `_t` alias make it convenient. Detection of "does this type have member X?" is done with `std::void_t` (C++17) or, more simply, with a concept (C++20).

CUSTOM TRAITS

main.cpp

```cpp
#include <iostream>
#include <list>
#include <map>
#include <string>
#include <type_traits>
#include <vector>

// 1. A classification trait by specialization
template <typename T> struct is_vector : std::false_type {};
template <typename T, typename A> struct is_vector<std::vector<T, A>> : std::true_type {};
template <typename T> constexpr bool is_vector_v = is_vector<T>::value;

// 2. A transformation trait: the element type of a container, or T itself
template <typename T> struct element { using type = T; };
template <typename T> struct element<std::vector<T>> { using type = T; };
template <typename T> struct element<std::list<T>> { using type = T; };
template <typename T> using element_t = typename element<T>::type;

// 3. Detection with void_t: does T have a size() member?
template <typename T, typename = void>
struct has_size : std::false_type {};
template <typename T>
struct has_size<T, std::void_t<decltype(std::declval<T>().size())>> : std::true_type {};

// 4. The same detection with a C++20 concept
template <typename T>
concept HasSize = requires(const T& t) { t.size(); };

int main() {
    std::cout << std::boolalpha;
    std::cout << "is_vector<vector<int>> " << is_vector_v<std::vector<int>> << ", is_vector<list<int>>
" << is_vector_v<std::list<int>> << '\n';
    std::cout << "element_t<vector<double>> is double? " <<
std::is_same_v<element_t<std::vector<double>>, double> << '\n';
    std::cout << "element_t<int> is int? " << std::is_same_v<element_t<int>, int> << '\n';
    std::cout << "has_size<std::string> " << has_size<std::string>::value << ", has_size<int> " <<
has_size<int>::value << '\n';
    std::cout << "HasSize<std::map<int,int>> " << HasSize<std::map<int, int>> << ", HasSize<double> "
<< HasSize<double> << '\n';
    return 0;
}
```

OUTPUT

```
is_vector<vector<int>> true, is_vector<list<int>> false
element_t<vector<double>> is double? true
element_t<int> is int? true
has_size<std::string> true, has_size<int> false
HasSize<std::map<int,int>> true, HasSize<double> false
```

NOTE

`std::declval<T>()` pretends to produce a `T` inside an unevaluated context such as `decltype`, even for types that cannot be constructed. If the expression `declval<T>().size()` is invalid, the specialization is silently discarded (SFINAE, next section) and the primary template (false) is used.

22.3 Compile-time selection

22.3.1 SFINAE and std::enable_if

SFINAE, "Substitution Failure Is Not An Error", is the rule that when substituting template arguments into a function template's *signature* produces an invalid type or expression, that template is quietly removed from the overload set instead of causing an error. `std::enable_if_t<condition, T>` exploits it: it is `T` when the condition is true and does not exist otherwise. Before C++20, this was the standard way to enable a function only for certain types.

ENABLE_IF IN A RETURN TYPE AND A TEMPLATE PARAMETER main.cpp

```cpp
#include <iostream>
#include <string>
#include <type_traits>

// enabled only for integral types (via the return type)
template <typename T>
std::enable_if_t<std::is_integral_v<T>, std::string> describe(T x) {
    return std::to_string(x) + " is an integer" + (x % 2 == 0 ? " (even)" : " (odd)");
}
// enabled only for floating-point types (via a defaulted template parameter)
template <typename T, std::enable_if_t<std::is_floating_point_v<T>, int> = 0>
std::string describe(T x) {
    return std::to_string(x) + " is floating point";
}

// SFINAE on an expression: only for types with a .size() member
template <typename T>
auto length(const T& c) -> decltype(c.size()) { return c.size(); }
std::size_t length(const char* s) { std::size_t n = 0; while (s[n]) ++n; return n; }

int main() {
    std::cout << describe(7) << '\n' << describe(2.5) << '\n' << describe('A') << '\n';
    // describe(std::string("x"));   error: no matching function (both templates removed)
    std::cout << length(std::string("hello")) << ' ' << length("hi") << '\n';
    return 0;
}
```

OUTPUT

```
7 is an integer (odd)
2.500000 is floating point
65 is an integer (odd)
5 2
```

WATCH OUT

SFINAE works but is hard to read, and its error messages ("no matching function… template argument deduction/substitution failed… enable_if…") are poor. In new code prefer `if constexpr` for choosing between implementations and concepts for restricting which types are accepted. You will still meet enable_if often in existing libraries.

22.3.2 if constexpr (C++17)

`if constexpr (condition)` evaluates its condition at compile time and **discards** the branch that is not taken: inside a template, the discarded branch is not instantiated, so it may contain code that would not compile for the current type. One function can then have different implementations for different kinds of types, without overloads or SFINAE.

ONE FUNCTION, SEVERAL IMPLEMENTATIONS

main.cpp

```
#include <iostream>
#include <string>
#include <type_traits>
#include <vector>

template <typename T>
std::string to_text(const T& value) {
    if constexpr (std::is_same_v<T, std::string>)
        return '"' + value + '"';
    else if constexpr (std::is_same_v<T, bool>)
        return value ? "true" : "false";
    else if constexpr (std::is_arithmetic_v<T>)
        return std::to_string(value);                  // would not compile for std::string
    else if constexpr (std::is_pointer_v<T>)
        return value ? "pointer to " + to_text(*value) : "null";
    else {                                             // assume a container
        std::string out = "[";
        for (const auto& x : value) out += to_text(x) + ", ";
        if (out.size() > 1) out.resize(out.size() - 2);
        return out + "]";
    }
}

int main() {
    int n = 5;
    int* none = nullptr;
    std::cout << to_text(42) << ' ' << to_text(2.5) << ' ' << to_text(true) << ' ' <<
to_text(std::string("hi")) << '\n';
    std::cout << to_text(&n) << ", " << to_text(none) << '\n';
    std::cout << to_text(std::vector<std::vector<int>>{{1, 2}, {3}}) << '\n';
    std::cout << to_text(std::vector<std::string>{"a", "b"}) << '\n';
    return 0;
}
```

OUTPUT

```
42 2.500000 true "hi"
pointer to 5, null
[[1, 2], [3]]
["a", "b"]
```

WATCH OUT

`if constexpr` only discards code that depends on the template parameter. Outside a template, or for code that does not depend on `T`, both branches must still be valid. And a plain `if` would instantiate every branch, so `std::to_string(value)` with `T = std::string` would fail to compile.

22.3.3 Concepts as the modern replacement

Concepts (Module 21) express the same constraints as enable_if, but directly and readably, with clear error messages, and overloads are ordered automatically by how constrained they are. The table compares the three techniques for the same task.

Technique	Restrict a template to integers
SFINAE (C++11)	`template <typename T, std::enable_if_t<std::is_integral_v<T>, int> = 0> void f(T);`
if constexpr (C++17)	one function with `if constexpr (std::is_integral_v<T>) { ... } else { static_assert(...); }`

Technique	Restrict a template to integers
Concepts (C++20)	`void f(std::integral auto x);` or `template <std::integral T> void f(T);`

THE SAME OVERLOADS WITH CONCEPTS

main.cpp

```
#include <concepts>
#include <iostream>
#include <string>

std::string describe(std::integral auto x)       { return std::to_string(x) + " is an integer"; }
std::string describe(std::floating_point auto x) { return std::to_string(x) + " is floating point"; }

template <typename T>
    requires requires(const T& t) { t.size(); }          // ad-hoc constraint: "has size()"
std::size_t length(const T& c) { return c.size(); }

int main() {
    std::cout << describe(7) << '\n' << describe(2.5f) << '\n' << length(std::string("concepts")) <<
'\n';
    return 0;
}
```

OUTPUT

```
7 is an integer
2.500000 is floating point
8
```

22.4 Compile-time computation

22.4.1 Template metaprogramming (factorial at compile time)

Templates are Turing-complete: recursive instantiation with specializations as base cases can compute any value at compile time. This "classic" template metaprogramming (TMP) predates constexpr and is still found in older libraries. The value is computed by the compiler; at run time it is just a constant.

CLASSIC TMP

main.cpp

```cpp
#include <iostream>

template <unsigned N>
struct Factorial {
    static constexpr unsigned long long value = N * Factorial<N - 1>::value;   // recursive
instantiation
};
template <>
struct Factorial<0> {                       // base case by specialization
    static constexpr unsigned long long value = 1;
};

template <unsigned N>
struct Fib { static constexpr unsigned long long value = Fib<N - 1>::value + Fib<N - 2>::value; };
template <> struct Fib<0> { static constexpr unsigned long long value = 0; };
template <> struct Fib<1> { static constexpr unsigned long long value = 1; };

template <unsigned Base, unsigned Exp>
struct Power { static constexpr unsigned long long value = Base * Power<Base, Exp - 1>::value; };
template <unsigned Base>
struct Power<Base, 0> { static constexpr unsigned long long value = 1; };   // partial specialization

int main() {
    std::cout << "5! = " << Factorial<5>::value << ", 20! = " << Factorial<20>::value << '\n';
    std::cout << "Fib<50> = " << Fib<50>::value << '\n';         // each Fib<N> is instantiated only
once
    std::cout << "2^40 = " << Power<2, 40>::value << '\n';
    static_assert(Factorial<10>::value == 3628800);
    return 0;
}
```

OUTPUT

```
5! = 120, 20! = 2432902008176640000
Fib<50> = 12586269025
2^40 = 1099511627776
```

22.4.2 constexpr and consteval functions

Since C++14, ordinary-looking `constexpr` functions with loops and local variables do the same job far more readably, and they work at run time too. `consteval` (C++20) forces compile-time evaluation. `std::is_constant_evaluated()` (C++20), or `if consteval` (C++23), lets a function behave differently at compile time and at run time.

CONSTEXPR INSTEAD OF TMP

main.cpp

```cpp
#include <array>
#include <iostream>
#include <string_view>
#include <type_traits>

constexpr unsigned long long factorial(unsigned n) {
    unsigned long long r = 1;
    for (unsigned i = 2; i <= n; ++i) r *= i;
    return r;
}

constexpr bool is_prime(unsigned n) {
    if (n < 2) return false;
    for (unsigned d = 2; d * d <= n; ++d)
        if (n % d == 0) return false;
```

```
    return true;
}

template <std::size_t N>
constexpr std::array<unsigned, N> first_primes() {        // a lookup table built by the compiler
    std::array<unsigned, N> out{};
    for (unsigned n = 2, i = 0; i < N; ++n)
        if (is_prime(n)) out[i++] = n;
    return out;
}

consteval std::size_t length(std::string_view s) { return s.size(); }   // must run at compile time

constexpr int where() { return std::is_constant_evaluated() ? 1 : 2; }

int main() {
    constexpr auto primes = first_primes<10>();
    for (unsigned p : primes) std::cout << p << ' ';
    std::cout << '\n';
    constexpr auto f15 = factorial(15);
    unsigned n = 6;
    std::cout << "15! = " << f15 << " (compile time), " << n << "! = " << factorial(n) << " (run
time)\n";
    constexpr std::size_t len = length("metaprogramming");
    std::cout << "length = " << len << '\n';
    constexpr int at_compile = where();
    int at_run = where();
    std::cout << "is_constant_evaluated: compile time -> " << at_compile << ", run time -> " << at_run
<< '\n';
    return 0;
}
```

```
OUTPUT
2 3 5 7 11 13 17 19 23 29
15! = 1307674368000 (compile time), 6! = 720 (run time)
length = 15
is_constant_evaluated: compile time -> 1, run time -> 2
```

22.4.3 static_assert

`static_assert(condition, "message")` checks a condition at compile time and stops compilation with the message if it is false (the message is optional since C++17). Use it to document and enforce assumptions: type sizes, trait requirements in templates, results of constexpr computations, and layout of structures shared with hardware or files.

A FAILING STATIC_ASSERT main.cpp

```cpp
#include <cstdint>
#include <type_traits>

struct Header {                      // a file format header: layout must be exact
    std::uint32_t magic;
    std::uint16_t version;
    std::uint16_t flags;
};
static_assert(sizeof(Header) == 8, "Header must be exactly 8 bytes");
static_assert(std::is_trivially_copyable_v<Header>, "Header is written with memcpy");

template <typename T>
T average(T a, T b) {
    static_assert(std::is_floating_point_v<T>, "average() needs a floating-point type");
    return (a + b) / 2;
}

int main() {
    double ok = average(2.0, 3.0);
    int bad = average(2, 3);         // fails the static_assert
    return static_cast<int>(ok) + bad;
}
```

OUTPUT

```
main.cpp: In instantiation of ‘T average(T, T) [with T = int]’:
main.cpp:20:22:   required from here
main.cpp:14:24: error: static assertion failed: average() needs a floating-point type
   14 |     static_assert(std::is_floating_point_v<T>, "average() needs a floating-point type");
      |                   ~~~~~^~~~~~~~~~~~~~~~~~~~
main.cpp:14:24: note: ‘std::is_floating_point_v<int>’ evaluates to false
```

22.5 Template idioms

22.5.1 CRTP (Curiously Recurring Template Pattern)

In **CRTP** a class derives from a template instantiated with itself: `class Circle : public Shape<Circle>`. The base class knows the derived type at compile time, so it can call derived members with `static_cast<Derived&>(*this)`: **static polymorphism**, with no virtual functions and no run-time cost. It is also used to inject common functionality (comparison operators, counters, cloning) into many classes. C++23 deducing this (Module 21) often simplifies it.

STATIC POLYMORPHISM AND MIXINS WITH CRTP main.cpp

```cpp
#include <iostream>
#include <string>

template <typename Derived>
class Shape {
public:
    void describe() const {                                  // shared code, calls into Derived
        const auto& self = static_cast<const Derived&>(*this);
        std::cout << self.name() << " with area " << self.area() << '\n';
    }
};
class Circle : public Shape<Circle> {
public:
    explicit Circle(double r) : r_(r) {}
    double area() const { return 3.14159 * r_ * r_; }
```

```
    std::string name() const { return "Circle"; }
private:
    double r_;
};
class Square : public Shape<Square> {
public:
    explicit Square(double s) : s_(s) {}
    double area() const { return s_ * s_; }
    std::string name() const { return "Square"; }
private:
    double s_;
};

template <typename Derived>
struct Comparable {                                          // mixin: derive > <= >= != from <
and ==
    friend bool operator>(const Derived& a, const Derived& b)  { return b < a; }
    friend bool operator<=(const Derived& a, const Derived& b) { return !(b < a); }
    friend bool operator>=(const Derived& a, const Derived& b) { return !(a < b); }
};
struct Money : Comparable<Money> {
    long paise;
    friend bool operator<(const Money& a, const Money& b) { return a.paise < b.paise; }
};

template <typename Shape>
void show(const Shape& s) { s.describe(); }                  // works with any CRTP shape

int main() {
    Circle c(1);
    Square s(2);
    show(c);
    show(s);
    Money a{{}, 500}, b{{}, 750};                            // {} initialises the empty base
    std::cout << std::boolalpha << "a > b: " << (a > b) << ", a <= b: " << (a <= b) << '\n';
    std::cout << "sizeof(Circle) = " << sizeof(Circle) << " (no vptr)\n";
    return 0;
}
```

OUTPUT

```
Circle with area 3.14159
Square with area 4
a > b: false, a <= b: true
sizeof(Circle) = 8 (no vptr)
```

	Virtual functions	CRTP
Dispatch	run time, through the vtable	compile time, can be inlined
Heterogeneous container	`vector<unique_ptr<Base>>` works	not possible: `Shape<Circle>` and `Shape<Square>` are unrelated types
Object size	+ one vptr	no overhead
Use when	the type is chosen at run time	the type is known at compile time and speed matters

22.5.2 Tag dispatch

Tag dispatch selects an implementation by passing an empty "tag" object whose type encodes a property. The standard library uses iterator category tags: `std::advance` calls one helper for random-access iterators (O(1) jump) and another for the rest (step by step). Since C++17, `if constexpr` is usually

simpler, but tag dispatch remains common and is useful when the implementations are large separate functions.

CHOOSING AN ALGORITHM BY ITERATOR CATEGORY main.cpp

```
#include <iostream>
#include <iterator>
#include <list>
#include <vector>

namespace detail {
template <typename It>
void my_advance(It& it, int n, std::random_access_iterator_tag) {     // O(1)
    std::cout << "  (random access: jump)\n";
    it += n;
}
template <typename It>
void my_advance(It& it, int n, std::bidirectional_iterator_tag) {     // O(n)
    std::cout << "  (bidirectional: step by step)\n";
    if (n >= 0) while (n--) ++it;
    else while (n++) --it;
}
}

template <typename It>
void my_advance(It& it, int n) {
    detail::my_advance(it, n, typename std::iterator_traits<It>::iterator_category{});   // the tag
object
}

int main() {
    std::vector<int> v = {10, 20, 30, 40, 50};
    std::list<int> l = {10, 20, 30, 40, 50};
    auto vi = v.begin();
    auto li = l.begin();
    my_advance(vi, 3);
    my_advance(li, 3);
    std::cout << *vi << ' ' << *li << '\n';
    return 0;
}
```

OUTPUT

```
  (random access: jump)
  (bidirectional: step by step)
40 40
```

22.5.3 Policy-based design

Policy-based design builds a class from small, interchangeable pieces given as template parameters: a logging policy, a locking policy, a checking policy, an allocation policy. Each combination is a different class, generated at compile time with no virtual calls; unused features cost nothing. The standard library does this with allocators, comparators, hashers and char traits.

A STORE ASSEMBLED FROM POLICIES main.cpp

```
#include <iostream>
#include <mutex>
#include <stdexcept>
#include <string>
#include <vector>

struct NoLock      { void lock() {} void unlock() {} };          // locking policies
```

```
struct MutexLock   { std::mutex m; void lock() { m.lock(); } void unlock() { m.unlock(); } };
struct NoCheck     { static void check(std::size_t, std::size_t) {} };   // checking policies
struct ThrowCheck  {
    static void check(std::size_t i, std::size_t n) { if (i >= n) throw std::out_of_range("index " +
std::to_string(i)); }
};
struct SilentLog   { static void log(const std::string&) {} };          // logging policies
struct ConsoleLog  { static void log(const std::string& msg) { std::cout << "  [log] " << msg << '\n';
} };

template <typename T, typename Lock = NoLock, typename Check = NoCheck, typename Log = SilentLog>
class Store {
public:
    void add(const T& v) {
        lock_.lock();
        data_.push_back(v);
        Log::log("added, size " + std::to_string(data_.size()));
        lock_.unlock();
    }
    T get(std::size_t i) {
        Check::check(i, data_.size());
        return data_[i];
    }
private:
    std::vector<T> data_;
    Lock lock_;
};

int main() {
    Store<int> fast;                                          // no locking, no checks, no log
    fast.add(1);
    std::cout << "fast.get(0) = " << fast.get(0) << '\n';

    Store<int, MutexLock, ThrowCheck, ConsoleLog> safe;           // thread-safe, checked, logged
    safe.add(10);
    safe.add(20);
    try {
        safe.get(5);
    } catch (const std::out_of_range& e) {
        std::cout << "safe.get(5) threw: " << e.what() << '\n';
    }
    std::cout << "sizeof(fast) = " << sizeof(fast) << ", sizeof(safe) = " << sizeof(safe) << '\n';
    return 0;
}
```

OUTPUT

```
fast.get(0) = 1
  [log] added, size 1
  [log] added, size 2
safe.get(5) threw: index 5
sizeof(fast) = 32, sizeof(safe) = 64
```

22.5.4 Type erasure

Type erasure gives unrelated types a common interface **without** a common base class. A non-template wrapper class stores any object that supports the required operations, hiding its real type behind an internal abstract "concept" class and a templated "model" that forwards to the object. `std::function`, `std::any` and `std::shared_ptr`'s deleter all work this way. Users get value semantics and freedom from inheritance; the cost is a heap allocation and a virtual call inside.

A HAND-MADE TYPE-ERASED DRAWABLE

main.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Drawable {                                  // the type-erased wrapper
public:
    template <typename T>
    Drawable(T obj) : self_(std::make_unique<Model<T>>(std::move(obj))) {}    // accepts ANY type with
draw()
    Drawable(const Drawable& o) : self_(o.self_->clone()) {}
    Drawable& operator=(const Drawable& o) { self_ = o.self_->clone(); return *this; }
    void draw() const { self_->draw(); }
private:
    struct Concept {                              // internal interface
        virtual ~Concept() = default;
        virtual void draw() const = 0;
        virtual std::unique_ptr<Concept> clone() const = 0;
    };
    template <typename T>
    struct Model : Concept {                      // adapts any T to the interface
        T obj;
        explicit Model(T o) : obj(std::move(o)) {}
        void draw() const override { obj.draw(); }
        std::unique_ptr<Concept> clone() const override { return std::make_unique<Model>(obj); }
    };
    std::unique_ptr<Concept> self_;
};

// Unrelated types: no common base class, no virtual functions
struct Circle { double r; void draw() const { std::cout << "circle r=" << r << '\n'; } };
struct Text   { std::string s; void draw() const { std::cout << "text \"" << s << "\"\n"; } };
struct Logo   { void draw() const { std::cout << "LBF logo\n"; } };

int main() {
    std::vector<Drawable> scene = {Circle{2.5}, Text{"Learn Being Forward"}, Logo{}};
    scene.push_back(Circle{1});
    for (const auto& d : scene) d.draw();
    Drawable copy = scene[1];                      // value semantics: a real copy
    copy.draw();
    return 0;
}
```

OUTPUT

```
circle r=2.5
text "Learn Being Forward"
LBF logo
circle r=1
text "Learn Being Forward"
```

INHERITANCE IS HIDDEN INSIDE THE WRAPPER; USER TYPES STAY INDEPENDENT

22.6 Templates and code organisation

22.6.1 Explicit instantiation

Module 17 showed **explicit instantiation definitions** (`template class Stack<int>;` in a .cpp file), which let template bodies stay out of headers for a known set of types. The matching **explicit instantiation declaration**, `extern template class Stack<int>;` in the header, tells every other translation unit "do not instantiate this here; it exists elsewhere". This avoids compiling the same instantiation in hundreds of files, and is a common technique to cut build times for heavily used templates.

EXTERN TEMPLATE TO AVOID REPEATED INSTANTIATION C++

```
// matrix.h
template <typename T> class Matrix { /* full definition */ };
extern template class Matrix<double>;      // declaration: instantiated in matrix.cpp only
extern template class Matrix<float>;

// matrix.cpp
#include "matrix.h"
template class Matrix<double>;             // definition: generate all members here, once
template class Matrix<float>;
```

22.6.2 Compile-time cost and code bloat

Every distinct set of template arguments produces separate code. `Stack<int>`, `Stack<long>` and `Stack<double>` are three copies of every used member function. Heavy metaprogramming also makes the compiler work hard: deep recursive instantiations and large headers slow builds and increase memory use. Ways to keep templates under control:

- **Factor out type-independent code** into a non-template base class or helper (for example, a `Stack<T*>` implemented on top of a single `Stack<void*>`).
- Use `extern template` / explicit instantiation for heavily used instantiations.
- Prefer `constexpr` functions and fold expressions over recursive template instantiation.
- Prefer `if constexpr` and concepts over large SFINAE overload sets.
- Keep templates in small headers, include only what you use, and consider modules (Module 21).
- Measure: `-ftime-trace` (Clang) or `-ftime-report` (GCC) show where compile time goes; `nm -C` or `size` show the instantiations in a binary.

THREE INSTANTIATIONS OF ONE TEMPLATE

main.cpp

```cpp
#include <iostream>
#include <vector>

template <typename T>
T sum(const std::vector<T>& v) {
    T total{};
    for (const T& x : v) total += x;
    return total;
}

int main() {
    std::cout << sum(std::vector<int>{1, 2, 3}) << ' '
              << sum(std::vector<long>{4, 5}) << ' '
              << sum(std::vector<double>{0.5, 0.25}) << '\n';
    return 0;
}
```

OUTPUT

```
6 9 0.75
```

Three separate functions in the binary (unoptimised)

```
$ g++ -std=c++20 -O0 main.cpp -o prog && nm -C prog | grep ' sum<'
0000000000001a93 W double sum<double>(std::vector<double, std::allocator<double> > const&)
00000000000016d3 W int sum<int>(std::vector<int, std::allocator<int> > const&)
00000000000018b1 W long sum<long>(std::vector<long, std::allocator<long> > const&)
$ g++ -std=c++20 -O2 main.cpp -o prog && nm -C prog | grep ' sum<'
$
```

Without optimisation the binary contains three separate functions, one per type. The `W` marks a "weak" symbol: when several object files contain the same instantiation, the linker keeps one copy. With `-O2` these small functions were inlined into `main`, so no symbols remain; larger templates are not inlined and every instantiation adds machine code to the program.

Process packs with folds (or recursion), sizeof... and pattern expansion. Use type traits to inspect and transform types and write your own with specialization and void_t. Choose implementations with if constexpr and restrict templates with concepts rather than enable_if. Compute at compile time with constexpr and consteval, guard assumptions with static_assert, and know the idioms: CRTP for static polymorphism and mixins, tag dispatch, policies and type erasure. Watch compile time and code size with extern template.

PRACTICE CHECKPOINT

A compile-time **factorial with constexpr verified by static_assert**, a **print function that behaves differently for containers using if constexpr**, and a **CRTP-based counter**.

SAMPLE SOLUTION

main.cpp

```cpp
#include <iostream>
#include <list>
#include <map>
#include <string>
#include <type_traits>
#include <vector>
```

```cpp
// ---------- 1. constexpr factorial checked at compile time ----------
constexpr unsigned long long factorial(unsigned n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}
static_assert(factorial(0) == 1);
static_assert(factorial(5) == 120);
static_assert(factorial(20) == 2432902008176640000ULL, "20! must fit in 64 bits");

// ---------- 2. print() with if constexpr ----------
template <typename T>
concept Container = requires(const T& t) { t.begin(); t.end(); } && !std::is_same_v<T, std::string>;

template <typename T>
concept PairLike = requires(const T& t) { t.first; t.second; };

template <typename T>
void print(const T& value) {
    if constexpr (Container<T>) {
        std::cout << '{';
        bool first = true;
        for (const auto& x : value) {
            if (!first) std::cout << ", ";
            print(x);                                   // recursive: nested containers work
            first = false;
        }
        std::cout << '}';
    } else if constexpr (PairLike<T>) {
        print(value.first);
        std::cout << ": ";
        print(value.second);
    } else if constexpr (std::is_same_v<T, std::string>) {
        std::cout << '"' << value << '"';
    } else {
        std::cout << value;
    }
}

// ---------- 3. CRTP counter: each class counts its own live objects ----------
template <typename Derived>
class Counted {
public:
    Counted() { ++created_; ++alive_; }
    Counted(const Counted&) { ++created_; ++alive_; }
    ~Counted() { --alive_; }
    static int created() { return created_; }
    static int alive() { return alive_; }
private:
    inline static int created_ = 0;                    // one pair of counters PER Derived type
    inline static int alive_ = 0;
};
class Student : public Counted<Student> { public: std::string name; };
class Course  : public Counted<Course>  { public: std::string title; };

int main() {
    constexpr auto f10 = factorial(10);
    std::cout << "10! = " << f10 << " (computed by the compiler)\n";

    print(42); std::cout << '\n';
    print(std::string("text")); std::cout << '\n';
    print(std::vector<int>{1, 2, 3}); std::cout << '\n';
    print(std::list<std::string>{"a", "b"}); std::cout << '\n';
    print(std::vector<std::vector<int>>{{1, 2}, {}, {3}}); std::cout << '\n';
    print(std::map<std::string, std::vector<int>>{{"Asha", {90, 85}}, {"Ravi", {70}}}); std::cout <<
```

```
'\n';

    {
        Student a, b, c;
        Course x;
        Student d = a;
        std::cout << "inside scope: students alive " << Student::alive() << ", courses alive " <<
Course::alive() << '\n';
    }
    Student e;
    std::cout << "after scope: students created " << Student::created() << ", alive " <<
Student::alive()
              << "; courses created " << Course::created() << ", alive " << Course::alive() << '\n';
    return 0;
}
```

OUTPUT

```
10! = 3628800 (computed by the compiler)
42
"text"
{1, 2, 3}
{"a", "b"}
{{1, 2}, {}, {3}}
{"Asha": {90, 85}, "Ravi": {70}}
inside scope: students alive 4, courses alive 1
after scope: students created 5, alive 1; courses created 1, alive 0
```

MODULE 23 · ADVANCED

Concurrency and multithreading

Minimum duration **4 days** Level **Advanced**

Modern processors have many cores, and a program that uses only one leaves most of the machine idle. Since C++11 the language has a memory model and the standard library has threads, mutexes, condition variables, atomics and futures; C++20 added jthread, latches, barriers and semaphores. This module shows how to start threads, share data safely, avoid races and deadlocks, run asynchronous tasks, and build a simple thread pool.

IN THIS MODULE

23.1 Basics
23.2 std::thread
23.3 Synchronisation
23.4 Atomics and the memory model
23.5 Asynchronous programming
23.6 C++20 synchronisation tools
23.7 Thread pools

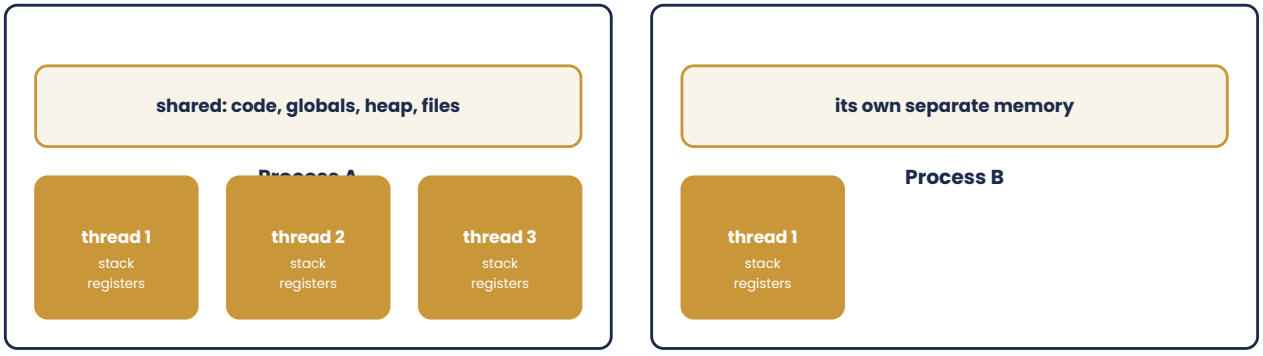
NOTE

Programs in this module that start threads are built with `-pthread` (on Linux it is implied by recent glibc, but adding it is the portable habit). Thread scheduling is not deterministic: where the order of output could vary from run to run, the programs are written to print in a fixed order, or the text says that the output is one possible run. The machine that produced these outputs had 2 cores.

23.1 Basics

23.1.1 Processes vs threads

A **process** is a running program with its own private address space: its memory, open files and resources are isolated from other processes, and communicating between processes needs explicit mechanisms (pipes, sockets, shared memory). A **thread** is a path of execution inside a process. All threads of a process share the same memory (globals, heap, open files) but each has its own **stack** and registers. Threads are cheaper to create and switch between, and communicate simply by sharing variables, which is exactly what makes them dangerous: two threads touching the same data at the same time must be coordinated.



THREADS SHARE THEIR PROCESS'S MEMORY; PROCESSES ARE ISOLATED FROM EACH OTHER

	Process	Thread
Memory	separate address space	shared with other threads of the process
Creation and switching cost	high	low
Communication	pipes, sockets, shared memory, files	shared variables (with synchronisation)
A crash	affects only that process	kills the whole process
C++ support	`std::system`, OS APIs (`fork`, `CreateProcess`)	`std::thread`, `std::jthread`

23.1.2 Concurrency vs parallelism

Concurrency means dealing with several tasks whose lifetimes overlap: a server handling many clients, a GUI staying responsive while a file downloads. It is about *structure*, and works even on a single core by switching between tasks. **Parallelism** means doing several computations *at the same instant* on several cores, to finish a big job faster. Threads give both; async I/O and coroutines give concurrency without extra threads.

CONCURRENCY INTERLEAVES TASKS; PARALLELISM RUNS THEM SIMULTANEOUSLY

`std::thread::hardware_concurrency()` returns the number of hardware threads (cores times hyper-threads), a sensible upper bound for CPU-bound worker threads.

23.2 std::thread

23.2.1 Creating threads with functions, lambdas and functors

`std::thread` (header `<thread>`) starts running a callable on a new thread as soon as it is constructed: a function pointer, a lambda, a functor or a member function. Extra constructor arguments are passed to the callable. Before a `std::thread` object is destroyed it must be **joined** (wait for it to finish) or **detached**; otherwise `std::terminate` is called.

FOUR WAYS TO GIVE A THREAD ITS WORK

main.cpp

```
#include <iostream>
#include <thread>

void download(int file_id) {                          // a plain function
    std::cout << "downloading file " << file_id << '\n';
}

struct Compressor {                                   // a functor
    int level;
    void operator()() const { std::cout << "compressing at level " << level << '\n'; }
};

class Logger {
public:
    void flush(int lines) { std::cout << "flushing " << lines << " log lines\n"; }
};

int main() {
    std::cout << "hardware threads: " << (std::thread::hardware_concurrency() > 0 ? "known" :
"unknown") << '\n';
    std::thread t1(download, 7);
    t1.join();                                        // wait before starting the next, so output stays
ordered

    std::thread t2(Compressor{9});
    t2.join();

    int total = 0;
    std::thread t3([&total] { for (int i = 1; i <= 100; ++i) total += i; });   // a lambda
    t3.join();
    std::cout << "lambda computed " << total << '\n';

    Logger log;
    std::thread t4(&Logger::flush, &log, 42);         // member function: pass the object pointer
    t4.join();
    return 0;
}
```

OUTPUT

```
hardware threads: known
downloading file 7
compressing at level 9
lambda computed 5050
flushing 42 log lines
```

23.2.2 join() and detach()

`join()` blocks the calling thread until the thread finishes. `detach()` lets the thread run independently in the background; the `std::thread` object no longer refers to it and it can never be joined. A detached

thread must not use local variables of a function that may have returned, and if `main` returns while it is still running, it is killed abruptly. Prefer join (or `jthread`, which joins automatically). `joinable()` tells whether a thread object still owns a running or finished-but-unjoined thread.

JOINING SEVERAL THREADS

main.cpp

```
#include <chrono>
#include <iostream>
#include <thread>
#include <vector>
using namespace std::chrono_literals;

int main() {
    std::vector<std::thread> workers;
    std::vector<int> results(4);
    for (int i = 0; i < 4; ++i)
        workers.emplace_back([i, &results] {                 // each thread writes its own slot: no
conflict
            std::this_thread::sleep_for(std::chrono::milliseconds(40 - i * 10));   // finish in
reverse order
            results[i] = i * i;
        });
    std::cout << std::boolalpha << "joinable before join: " << workers[0].joinable() << '\n';
    for (auto& t : workers) t.join();                        // wait for all
    std::cout << "joinable after join: " << workers[0].joinable() << '\n';
    for (int r : results) std::cout << r << ' ';
    std::cout << '\n';

    std::thread background([] { std::this_thread::sleep_for(10ms); });
    background.detach();                                     // runs on its own; cannot be joined
now
    std::cout << "detached, joinable: " << background.joinable() << '\n';
    std::this_thread::sleep_for(50ms);                       // give it time to finish before main
exits
    return 0;
}
```

OUTPUT

```
joinable before join: true
joinable after join: false
0 1 4 9
detached, joinable: false
```

WATCH OUT

If an exception is thrown between creating a `std::thread` and joining it, the destructor of the still-joinable thread calls `std::terminate`. This is one more reason to prefer `std::jthread` (23.2.4), which joins in its destructor.

23.2.3 Passing arguments by value and by reference (std::ref)

The `std::thread` constructor **copies** (or moves) every argument into the new thread's storage, even if the function parameter is a reference. This prevents accidental dangling references, but it means a function taking `int&` would modify a copy (in fact it does not compile). To pass a real reference, wrap it in `std::ref` (or `std::cref`), and make sure the referenced object outlives the thread. Move-only arguments such as `unique_ptr` must be passed with `std::move`.

ARGUMENTS ARE COPIED UNLESS WRAPPED IN STD::REF

main.cpp

```
#include <functional>
#include <iostream>
#include <memory>
#include <string>
#include <thread>
#include <vector>

void fill(std::vector<int>& out, int n) { for (int i = 0; i < n; ++i) out.push_back(i); }
void greet(std::string name) { std::cout << "hello " << name << '\n'; }
void consume(std::unique_ptr<int> p) { std::cout << "consumed " << *p << '\n'; }

int main() {
    std::vector<int> data;
    // std::thread bad(fill, data, 5);     error: cannot bind a copy to std::vector<int>&
    std::thread t1(fill, std::ref(data), 5);          // real reference
    t1.join();
    std::cout << "data has " << data.size() << " elements\n";

    std::string who = "Asha";
    std::thread t2(greet, who);                       // copied into the thread
    t2.join();

    auto p = std::make_unique<int>(42);
    std::thread t3(consume, std::move(p));            // ownership moves into the thread
    t3.join();
    std::cout << "p is " << (p ? "set" : "empty") << " in main\n";
    return 0;
}
```

OUTPUT

```
data has 5 elements
hello Asha
consumed 42
p is empty in main
```

23.2.4 std::jthread (C++20)

`std::jthread` ("joining thread") behaves like `std::thread` but its destructor **requests a stop and joins**. It owns a `std::stop_source`; if the thread function's first parameter is a `std::stop_token`, the jthread passes one in, and the function can check `stop_requested()` to finish early. This makes threads exception-safe and cancellable with no extra code.

A CANCELLABLE JTHREAD

main.cpp

```
#include <chrono>
#include <iostream>
#include <thread>
using namespace std::chrono_literals;

int main() {
    {
        std::jthread worker([](std::stop_token token, int step) {
            int done = 0;
            while (!token.stop_requested()) {
                std::this_thread::sleep_for(10ms);
                done += step;
            }
            std::cout << "worker: stop requested, finishing cleanly (did some work: " <<
std::boolalpha << (done > 0) << ")\n";
        }, 5);
        std::this_thread::sleep_for(60ms);
        std::cout << "main: leaving scope\n";
    }                                       // destructor: request_stop() + join()
    std::cout << "main: worker has been joined automatically\n";
    return 0;
}
```

OUTPUT

```
main: leaving scope
worker: stop requested, finishing cleanly (did some work: true)
main: worker has been joined automatically
```

23.2.5 std::this_thread::sleep_for, get_id

The namespace `std::this_thread` acts on the current thread: `sleep_for(duration)` and `sleep_until(time_point)` pause it, `yield()` hints that other threads may run, and `get_id()` returns a `std::thread::id` that identifies it (also available as `t.get_id()` on a thread object). Ids can be compared, hashed and printed, but their printed form is implementation-specific.

THREAD IDS AND SLEEPING

main.cpp

```
#include <chrono>
#include <iostream>
#include <thread>
using namespace std::chrono_literals;

int main() {
    std::thread::id main_id = std::this_thread::get_id();
    std::thread::id worker_id;
    std::thread t([&worker_id] {
        worker_id = std::this_thread::get_id();
        std::this_thread::sleep_for(20ms);
    });
    std::thread::id from_object = t.get_id();
    t.join();
    std::cout << std::boolalpha;
    std::cout << "worker id == t.get_id(): " << (worker_id == from_object) << '\n';
    std::cout << "worker id == main id:    " << (worker_id == main_id) << '\n';
    std::cout << "after join, t.get_id() is the empty id: " << (t.get_id() == std::thread::id{}) <<
'\n';

    auto start = std::chrono::steady_clock::now();
    std::this_thread::sleep_until(start + 30ms);
    std::cout << "slept at least 30 ms: " << (std::chrono::steady_clock::now() - start >= 30ms) <<
'\n';
    return 0;
}
```

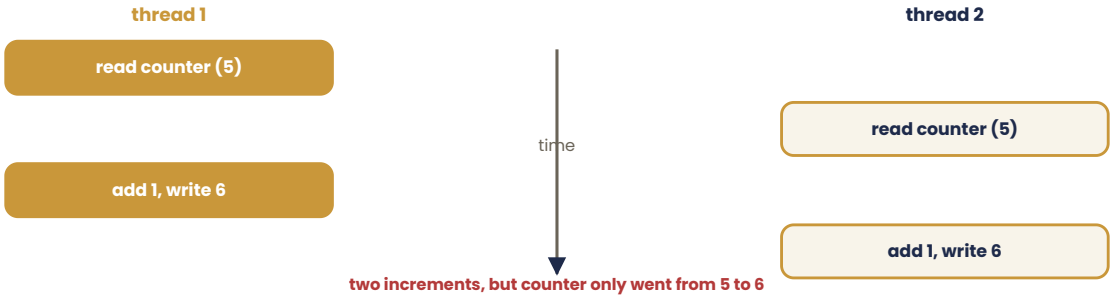
OUTPUT

```
worker id == t.get_id(): true
worker id == main id:    false
after join, t.get_id() is the empty id: true
slept at least 30 ms: true
```

23.3 Synchronisation

23.3.1 Race conditions and data races

A **race condition** is a bug where the result depends on the timing of threads. A **data race** is the specific, low-level case: two threads access the same memory location concurrently, at least one of them writes, and nothing orders the accesses. In C++ a data race is **undefined behaviour**. Even `counter++` is not one operation: it is read, add, write, and two threads can both read the same old value, so one increment is lost.



A LOST UPDATE: BOTH THREADS READ 5 BEFORE EITHER WRITES

LOST UPDATES (ONE POSSIBLE RUN) main.cpp

```cpp
#include <iostream>
#include <thread>
#include <vector>

int counter = 0;                                    // shared, unprotected

void work() {
    for (int i = 0; i < 1000000; ++i) ++counter;    // data race!
}

int main() {
    for (int run = 1; run <= 3; ++run) {
        counter = 0;
        std::vector<std::thread> ts;
        for (int i = 0; i < 4; ++i) ts.emplace_back(work);
        for (auto& t : ts) t.join();
        std::cout << "run " << run << ": counter = " << counter << " (expected 4000000)\n";
    }
    return 0;
}
```

OUTPUT

```
run 1: counter = 2936820 (expected 4000000)
run 2: counter = 1557421 (expected 4000000)
run 3: counter = 4000000 (expected 4000000)
```

The totals are wrong and different on every run (and on a different machine they may even happen to be right, which is worse: the bug hides). ThreadSanitizer finds data races reliably: build with `-fsanitize=thread -g` and it reports both conflicting accesses with their stack traces.

ThreadSanitizer report (trimmed: build ids and library frames removed)

```
$ g++ -std=c++20 -fsanitize=thread -g race.cpp -o race && ./race
==================
WARNING: ThreadSanitizer: data race (pid=11127)
  Read of size 4 at 0x55f9b45f1154 by thread T2:
    #0 work() race.cpp:8 (race+0x24b9)
    ... (library frames of std::thread)

  Previous write of size 4 at 0x55f9b45f1154 by thread T1:
    #0 work() race.cpp:8 (race+0x24d1)
    ...

  Location is global 'counter' of size 4 at 0x55f9b45f1154 (race+0x8154)

  Thread T2 (tid=11131, running) created by main thread at:
    ...
    #6 main race.cpp:13 (race+0x2555)
```

23.3.2 std::mutex

A **mutex** (mutual exclusion, header `<mutex>`) protects a **critical section**: `lock()` blocks until no other thread holds the mutex, `unlock()` releases it, and `try_lock()` returns immediately with true or false. Only one thread at a time can be between lock and unlock, so the read-modify-write sequence becomes indivisible. Every access to the shared data, reads included, must hold the same mutex.

A MUTEX MAKES THE INCREMENT SAFE main.cpp

```
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

int counter = 0;
std::mutex counter_mutex;                          // protects counter

void work() {
    for (int i = 0; i < 100000; ++i) {
        counter_mutex.lock();
        ++counter;                                 // critical section
        counter_mutex.unlock();
    }
}

int main() {
    std::vector<std::thread> ts;
    for (int i = 0; i < 4; ++i) ts.emplace_back(work);
    for (auto& t : ts) t.join();
    std::cout << "counter = " << counter << " (expected 400000)\n";
    return 0;
}
```

OUTPUT

```
counter = 400000 (expected 400000)
```

WATCH OUT

Calling `lock()` and `unlock()` by hand has the same problem as `new` and `delete`: an early return or an exception between them leaves the mutex locked forever, and every other thread waiting for it hangs. Always use an RAII lock (next section).

23.3.3 lock_guard, unique_lock, scoped_lock

Lock	Behaviour	Use for
`std::lock_guard<M>`	locks in the constructor, unlocks in the destructor; nothing else	simple critical sections
`std::unique_lock<M>`	can unlock and relock, lock later (`std::defer_lock`), try or time out, and be moved	condition variables; finer control
`std::scoped_lock<Ms...>` (C++17)	locks one or **several** mutexes at once using a deadlock-avoidance algorithm	the default choice; anything needing two mutexes

RAII LOCKS main.cpp

```
#include <iostream>
#include <map>
#include <mutex>
#include <string>
#include <thread>
#include <vector>

class BankAccount {
```

```
public:
    explicit BankAccount(int balance) : balance_(balance) {}
    void deposit(int amount) {
        std::lock_guard<std::mutex> lock(m_);              // unlocked automatically at the end of
scope
        balance_ += amount;
    }
    bool withdraw(int amount) {
        std::scoped_lock lock(m_);                         // C++17: same effect, CTAD deduces the
type
        if (amount > balance_) return false;               // early return: still unlocked correctly
        balance_ -= amount;
        return true;
    }
    int balance() const {
        std::lock_guard lock(m_);
        return balance_;
    }
private:
    mutable std::mutex m_;                                 // mutable: lockable in const functions
    int balance_;
};

int main() {
    BankAccount acc(1000);
    std::vector<std::thread> ts;
    for (int i = 0; i < 4; ++i)
        ts.emplace_back([&acc] {
            for (int k = 0; k < 1000; ++k) { acc.deposit(2); acc.withdraw(1); }
        });
    for (auto& t : ts) t.join();
    std::cout << "balance = " << acc.balance() << " (expected 1000 + 4 * 1000 = 5000)\n";

    std::mutex m;
    std::unique_lock<std::mutex> ul(m, std::defer_lock);  // not locked yet
    std::cout << std::boolalpha << "owns lock: " << ul.owns_lock();
    ul.lock();
    std::cout << " -> " << ul.owns_lock();
    ul.unlock();                                           // unlock early, before the end of scope
    std::cout << " -> " << ul.owns_lock() << '\n';
    return 0;
}
```

OUTPUT

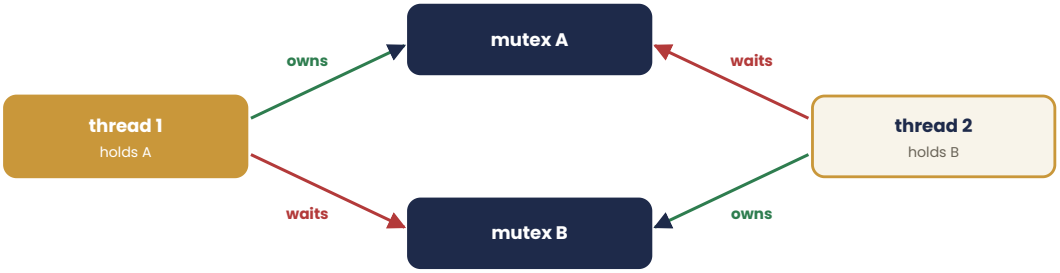
```
balance = 5000 (expected 1000 + 4 * 1000 = 5000)
owns lock: false -> true -> false
```

TIP

Keep critical sections short: lock, touch the shared data, unlock. Do slow work (I/O, computation, calling unknown callbacks) outside the lock, otherwise threads spend their time waiting for each other.

23.3.4 Deadlock and how to avoid it

A **deadlock** occurs when threads wait for each other in a cycle: thread 1 holds mutex A and waits for B, while thread 2 holds B and waits for A. Neither can continue, ever. The classic case is a bank transfer that locks the source account and then the destination: two opposite transfers at the same time deadlock.



A CYCLE OF WAITING: DEADLOCK

PROVOKING AND AVOIDING A DEADLOCK

main.cpp

```
#include <atomic>
#include <chrono>
#include <iostream>
#include <mutex>
#include <string>
#include <thread>
using namespace std::chrono_literals;

struct Account { std::mutex m; int balance = 1000; std::string name; };
std::atomic<int> stuck{0}, holding{0}, tried{0};

void transfer_bad(Account& from, Account& to, int amount) {          // can deadlock
    std::lock_guard a(from.m);
    ++holding;
    while (holding < 2) std::this_thread::yield();                   // for the demo: wait until both
threads hold one mutex
    std::unique_lock b(to.m, std::try_to_lock);                      // try instead of blocking
forever, for the demo
    bool got = b.owns_lock();
    ++tried;
    while (tried < 2) std::this_thread::yield();                     // both have tried before either
gives up
    if (!got) { ++stuck; return; }                                   // a blocking lock would wait
forever here
    from.balance -= amount; to.balance += amount;
}

void transfer_good(Account& from, Account& to, int amount) {
    std::scoped_lock both(from.m, to.m);                             // locks both, deadlock-free
    from.balance -= amount;
    to.balance += amount;
}

int main() {
    Account x{{}, 1000, "X"}, y{{}, 1000, "Y"};
    std::cout << "opposite transfers, naive locking:\n";
    std::thread t1(transfer_bad, std::ref(x), std::ref(y), 100);
    std::thread t2(transfer_bad, std::ref(y), std::ref(x), 200);
    t1.join(); t2.join();
    std::cout << "  " << stuck << " of 2 transfers found the second mutex taken: a real lock would
deadlock\n";

    std::cout << "opposite transfers with scoped_lock:\n";
    std::thread t3(transfer_good, std::ref(x), std::ref(y), 100);
    std::thread t4(transfer_good, std::ref(y), std::ref(x), 200);
    t3.join(); t4.join();
    std::cout << "  X = " << x.balance << ", Y = " << y.balance << ", total = " << x.balance +
y.balance << '\n';
```

```
    return 0;
}
```

OUTPUT

```
opposite transfers, naive locking:
  2 of 2 transfers found the second mutex taken: a real lock would deadlock
opposite transfers with scoped_lock:
  X = 1100, Y = 900, total = 2000
```

With plain blocking locks the first two threads would hang forever; the demo uses `try_to_lock` only so that it can report the situation and continue. Rules for avoiding deadlock:

1. Lock several mutexes together with `std::scoped_lock` (or `std::lock`), which acquires them without risk of a cycle.
2. Otherwise, always acquire mutexes in the same global order (for example by account id or address).
3. Never call unknown code (callbacks, virtual functions supplied by users) while holding a lock.
4. Avoid holding a lock while waiting for another thread (joining it, or waiting on a future).
5. Hold one lock at a time whenever possible.

23.3.5 shared_mutex and reader-writer locks

When data is read often and written rarely (configuration, caches, lookup tables), letting readers proceed in parallel improves throughput. `std::shared_mutex` (C++17, header `<shared_mutex>`) has two modes: **shared** (many readers at once, via `std::shared_lock`) and **exclusive** (one writer, no readers, via `std::unique_lock` or `lock_guard`).

A READER-WRITER LOCK

main.cpp

```
#include <iostream>
#include <map>
#include <mutex>
#include <shared_mutex>
#include <string>
#include <thread>
#include <vector>

class PriceList {
public:
    int get(const std::string& item) const {
        std::shared_lock lock(m_);                      // many readers at the same time
        auto it = prices_.find(item);
        return it == prices_.end() ? -1 : it->second;
    }
    void set(const std::string& item, int price) {
        std::unique_lock lock(m_);                      // exclusive: waits for readers to leave
        prices_[item] = price;
    }
private:
    mutable std::shared_mutex m_;
    std::map<std::string, int> prices_ = {{"tea", 15}, {"coffee", 25}};
};

int main() {
    PriceList list;
    std::vector<std::thread> readers;
    std::vector<long> sums(4, 0);
    for (int r = 0; r < 4; ++r)
        readers.emplace_back([&list, &sums, r] {
            for (int i = 0; i < 10000; ++i) sums[r] += list.get("tea") + list.get("coffee");
        });
    std::thread writer([&list] { list.set("samosa", 18); });
    for (auto& t : readers) t.join();
    writer.join();
    std::cout << "each reader saw consistent prices: " << std::boolalpha << (sums[0] == 400000 &&
sums[3] == 400000) << '\n';
    std::cout << "samosa = " << list.get("samosa") << '\n';
    return 0;
}
```

OUTPUT

```
each reader saw consistent prices: true
samosa = 18
```

23.3.6 Condition variables

A **condition variable** (header `<condition_variable>`) lets a thread **sleep until** some condition on shared data becomes true, instead of repeatedly locking and checking (busy-waiting). The waiting thread holds a `unique_lock` and calls `cv.wait(lock, predicate)`: it atomically releases the mutex and sleeps; when woken it reacquires the mutex and re-checks the predicate. The thread that changes the data does so under the mutex, then calls `notify_one()` or `notify_all()`. Always wait with a predicate: wake-ups can be **spurious** (happen without a notify) and a notify sent before the wait would otherwise be lost.

WAITING FOR A CONDITION

main.cpp

```
#include <chrono>
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <string>
#include <thread>

std::mutex m;
std::condition_variable cv;
bool data_ready = false;
std::string data;

void worker() {
    std::unique_lock lock(m);
    std::cout << "worker: waiting for data\n";
    cv.wait(lock, [] { return data_ready; });          // sleeps; mutex released while waiting
    std::cout << "worker: got \"" << data << "\", processing\n";
}

int main() {
    std::thread t(worker);
    std::this_thread::sleep_for(std::chrono::milliseconds(20));   // usually the worker is waiting by
now
    {
        std::lock_guard lock(m);
        data = "sensor batch #7";
        data_ready = true;                            // change the condition under the mutex
    }
    cv.notify_one();                                  // then wake the waiter
    t.join();
    std::cout << "main: done\n";
    return 0;
}
```

OUTPUT

```
worker: waiting for data
worker: got "sensor batch #7", processing
main: done
```

NOTE

If the main thread sets `data_ready` before the worker reaches `wait`, the predicate is already true and `wait` returns immediately without sleeping, so the program is correct either way. (In that case "waiting for data" is still printed, because the message comes before the wait.)

23.3.7 Producer-consumer problem

In the **producer-consumer** pattern, producer threads create work items and put them in a shared queue; consumer threads take items out and process them. The queue needs a mutex, and condition variables so that consumers sleep while the queue is empty (and, for a bounded queue, producers sleep while it is full). A "closed" flag tells consumers when no more items will come.

A BOUNDED BLOCKING QUEUE

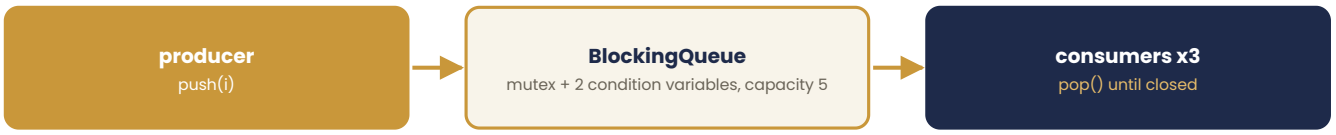
main.cpp

```
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <optional>
#include <queue>
```

```cpp
#include <thread>
#include <vector>

template <typename T>
class BlockingQueue {
public:
    explicit BlockingQueue(std::size_t capacity) : cap_(capacity) {}
    void push(T item) {
        std::unique_lock lock(m_);
        not_full_.wait(lock, [this] { return q_.size() < cap_; });       // bounded: wait while full
        q_.push(std::move(item));
        not_empty_.notify_one();
    }
    std::optional<T> pop() {                                             // empty optional = closed
and drained
        std::unique_lock lock(m_);
        not_empty_.wait(lock, [this] { return !q_.empty() || closed_; });
        if (q_.empty()) return std::nullopt;
        T item = std::move(q_.front());
        q_.pop();
        not_full_.notify_one();
        return item;
    }
    void close() {
        { std::lock_guard lock(m_); closed_ = true; }
        not_empty_.notify_all();                                         // wake every consumer
    }
private:
    std::mutex m_;
    std::condition_variable not_empty_, not_full_;
    std::queue<T> q_;
    std::size_t cap_;
    bool closed_ = false;
};

int main() {
    BlockingQueue<int> queue(5);
    std::vector<long> handled(3, 0), count(3, 0);
    std::vector<std::thread> consumers;
    for (int c = 0; c < 3; ++c)
        consumers.emplace_back([&, c] {
            while (auto item = queue.pop()) { handled[c] += *item; ++count[c]; }
        });
    std::thread producer([&queue] {
        for (int i = 1; i <= 1000; ++i) queue.push(i);
        queue.close();
    });
    producer.join();
    for (auto& t : consumers) t.join();
    long total = handled[0] + handled[1] + handled[2];
    std::cout << "items processed: " << count[0] + count[1] + count[2] << ", sum " << total << "
(expected 500500)\n";
    std::cout << "every consumer took part: " << std::boolalpha << (count[0] > 0 && count[1] > 0 &&
count[2] > 0) << '\n';
    return 0;
}
```

OUTPUT

```
items processed: 1000, sum 500500 (expected 500500)
every consumer took part: true
```



PRODUCER-CONSUMER WITH A BOUNDED QUEUE

23.4 Atomics and the memory model

23.4.1 std::atomic

`std::atomic<T>` (header `<atomic>`) makes individual operations on a variable indivisible: `++`, `fetch_add`, `load`, `store`, `exchange` and `compare_exchange` cannot be interrupted halfway, and using them from several threads is not a data race. For a single counter or flag an atomic is simpler and usually faster than a mutex, because the hardware does it in one instruction. Atomics protect only one variable at a time; an invariant involving several variables still needs a mutex.

ATOMIC OPERATIONS main.cpp

```
#include <atomic>
#include <iostream>
#include <thread>
#include <vector>

std::atomic<int> counter{0};
std::atomic<bool> ready{false};

int main() {
    std::vector<std::thread> ts;
    for (int i = 0; i < 4; ++i)
        ts.emplace_back([] { for (int k = 0; k < 100000; ++k) ++counter; });   // atomic increment
    for (auto& t : ts) t.join();
    std::cout << "counter = " << counter << " (expected 400000)\n";

    int old = counter.fetch_add(10);                      // returns the previous value
    std::cout << "fetch_add returned " << old << ", now " << counter.load() << '\n';
    int prev = counter.exchange(0);
    std::cout << "exchange returned " << prev << ", now " << counter << '\n';

    int expected = 0;
    bool swapped = counter.compare_exchange_strong(expected, 99);   // if counter == 0, set it to 99
    std::cout << std::boolalpha << "CAS(0 -> 99) succeeded: " << swapped << ", counter = " << counter
<< '\n';
    expected = 5;
    swapped = counter.compare_exchange_strong(expected, 1);
    std::cout << "CAS(5 -> 1) succeeded: " << swapped << ", expected now holds the real value " <<
expected << '\n';
    std::cout << "atomic<int> lock-free: " << counter.is_lock_free() << '\n';
    return 0;
}
```

OUTPUT

```
counter = 400000 (expected 400000)
fetch_add returned 400000, now 400010
exchange returned 400010, now 0
CAS(0 -> 99) succeeded: true, counter = 99
CAS(5 -> 1) succeeded: false, expected now holds the real value 99
atomic<int> lock-free: true
```

23.4.2 Memory ordering (overview)

Compilers and CPUs reorder memory operations for speed, as long as a single thread cannot tell. Across threads, the **memory model** defines which reorderings other threads may observe. Each atomic operation takes an optional **memory order**. The default, `memory_order_seq_cst` (sequentially consistent), behaves as if all atomic operations happened in one global order: the easiest to reason about. Weaker orders allow more optimisation.

Order	Guarantee	Typical use
`seq_cst` (default)	one total order of all seq_cst operations	everything, unless measured to be too slow
`acquire` / `release`	writes before a release-store are visible to a thread whose acquire-load sees that store	publishing data with a flag; implementing locks
`relaxed`	atomicity only, no ordering of other memory	statistics counters, reference counts (increments)

RELEASE-ACQUIRE PUBLICATION AND A RELAXED COUNTER main.cpp

```
#include <atomic>
#include <iostream>
#include <string>
#include <thread>

std::string message;                                  // ordinary data
std::atomic<bool> published{false};
std::atomic<long> hits{0};

int main() {
    std::thread producer([] {
        message = "config loaded";                    // 1. write the data
        published.store(true, std::memory_order_release);   // 2. publish: everything above is visible
...
    });
    std::thread consumer([] {
        while (!published.load(std::memory_order_acquire)) {}   // ... to whoever acquires the flag
        std::cout << "consumer sees: " << message << '\n';     // guaranteed to see the full string
    });
    producer.join();
    consumer.join();

    std::thread a([] { for (int i = 0; i < 50000; ++i) hits.fetch_add(1, std::memory_order_relaxed);
});
    std::thread b([] { for (int i = 0; i < 50000; ++i) hits.fetch_add(1, std::memory_order_relaxed);
});
    a.join(); b.join();
    std::cout << "relaxed counter: " << hits << '\n';   // the count is still exact
    return 0;
}
```

OUTPUT

```
consumer sees: config loaded
relaxed counter: 100000
```

23.4.3 Lock-free programming (overview)

Lock-free data structures use atomics, especially **compare-and-swap** (CAS) loops, instead of mutexes: at least one thread always makes progress, threads never block each other, and there is no risk of

deadlock. A CAS loop reads the current value, computes a new one, and installs it only if nobody changed the value in between; otherwise it retries. Lock-free code is extremely hard to get right (memory reclamation, the ABA problem, memory ordering), so use well-tested libraries and measure before writing your own.

A LOCK-FREE MAXIMUM WITH A CAS LOOP main.cpp

```
#include <atomic>
#include <iostream>
#include <thread>
#include <vector>

std::atomic<int> maximum{0};

void update_max(int value) {
    int current = maximum.load();
    while (value > current && !maximum.compare_exchange_weak(current, value)) {
        // CAS failed: another thread changed maximum; current now holds the new value, retry
    }
}

int main() {
    std::vector<std::thread> ts;
    for (int t = 0; t < 4; ++t)
        ts.emplace_back([t] { for (int i = 0; i < 10000; ++i) update_max((i * 7919 + t * 104729) %
100003); });
    for (auto& th : ts) th.join();
    std::cout << "maximum seen: " << maximum << '\n';
    return 0;
}
```

OUTPUT

```
maximum seen: 100001
```

23.5 Asynchronous programming

23.5.1 std::async and launch policies

`std::async(f, args...)` (header `<future>`) runs a function asynchronously and returns a `std::future` that will hold its result. It is the easiest way to run a computation in the background and collect its value: no thread objects, no shared variables. The **launch policy** says how: `std::launch::async` runs it on a new thread; `std::launch::deferred` runs it lazily, on the calling thread, when `get()` is called; the default lets the implementation choose. Pass `std::launch::async` explicitly when you need real parallelism.

STD::ASYNC

main.cpp

```cpp
#include <chrono>
#include <future>
#include <iostream>
#include <thread>
using namespace std::chrono_literals;

long long slow_square(int x) {
    std::this_thread::sleep_for(100ms);
    return 1LL * x * x;
}

int main() {
    auto start = std::chrono::steady_clock::now();
    std::future<long long> a = std::async(std::launch::async, slow_square, 12);   // both start now
    std::future<long long> b = std::async(std::launch::async, slow_square, 30);
    long long total = a.get() + b.get();                                          // wait for both
results
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::steady_clock::now() -
start).count();
    std::cout << "total = " << total << ", in parallel: about " << (ms < 180 ? "100" : "200") << " ms,
not 200\n";

    auto lazy = std::async(std::launch::deferred, [] { std::cout << "  deferred task runs now, on the
caller's thread\n"; return 7; });
    std::cout << "before get()\n";
    int r = lazy.get();                                                           // the task runs
here
    std::cout << "result " << r << '\n';

    auto failing = std::async(std::launch::async, [] () -> int { throw std::runtime_error("sensor
offline"); });
    try {
        failing.get();                                                            // exceptions travel
through the future
    } catch (const std::exception& e) {
        std::cout << "caught from the task: " << e.what() << '\n';
    }
    return 0;
}
```

OUTPUT

```
total = 1044, in parallel: about 100 ms, not 200
before get()
  deferred task runs now, on the caller's thread
result 7
caught from the task: sensor offline
```

WATCH OUT

The future returned by `std::async(std::launch::async, ...)` blocks in its destructor until the task finishes. Writing `std::async(std::launch::async, f);` without keeping the future therefore runs f synchronously: the temporary future is destroyed at the end of the statement and waits for it.

23.5.2 std::future and std::promise

A **future** is the receiving end of a one-shot channel; a **promise** is the sending end. The producer thread calls `promise.set_value(v)` (or `set_exception`), and the consumer calls `future.get()`, which blocks until the value arrives and can be called only once. `wait_for(duration)` checks with a timeout. A `std::shared_future` can be read by several threads; a `promise<void>` is a simple one-time signal.

PROMISE AND FUTURE

main.cpp

```cpp
#include <chrono>
#include <future>
#include <iostream>
#include <stdexcept>
#include <string>
#include <thread>
using namespace std::chrono_literals;

void fetch_weather(std::promise<std::string> result, bool fail) {
    std::this_thread::sleep_for(50ms);
    if (fail) result.set_exception(std::make_exception_ptr(std::runtime_error("network timeout")));
    else result.set_value("Bengaluru: 26 C, light rain");
}

int main() {
    std::promise<std::string> p;
    std::future<std::string> f = p.get_future();
    std::thread t(fetch_weather, std::move(p), false);
    if (f.wait_for(10ms) == std::future_status::timeout) std::cout << "not ready after 10 ms, still
waiting...\n";
    std::cout << f.get() << '\n';
    t.join();

    std::promise<std::string> p2;
    auto f2 = p2.get_future();
    std::thread t2(fetch_weather, std::move(p2), true);
    try { f2.get(); } catch (const std::exception& e) { std::cout << "error: " << e.what() << '\n'; }
    t2.join();

    std::promise<void> go;                                  // a start signal for several threads
    std::shared_future<void> start = go.get_future().share();
    std::thread r1([start] { start.wait(); }), r2([start] { start.wait(); });
    std::cout << "releasing the runners\n";
    go.set_value();
    r1.join(); r2.join();
    std::cout << "both runners started and finished\n";
    return 0;
}
```

OUTPUT

```
not ready after 10 ms, still waiting...
Bengaluru: 26 C, light rain
error: network timeout
releasing the runners
both runners started and finished
```

23.5.3 std::packaged_task

`std::packaged_task<R(Args...)>` wraps a callable so that calling it stores the result (or exception) in an associated future. It separates *creating* a task from *running* it: you can create tasks in one place, put them in a queue, and let worker threads execute them later. This is exactly what a thread pool needs.

TASKS CREATED NOW, RUN LATER

main.cpp

```cpp
#include <future>
#include <iostream>
#include <queue>
#include <thread>
#include <vector>

int main() {
    std::queue<std::packaged_task<int()>> tasks;
    std::vector<std::future<int>> results;
    for (int i = 1; i <= 4; ++i) {
        std::packaged_task<int()> task([i] { return i * 100; });
        results.push_back(task.get_future());          // keep the future before handing the task away
        tasks.push(std::move(task));
    }
    std::thread worker([&tasks] {                      // runs all tasks later, on another thread
        while (!tasks.empty()) {
            auto task = std::move(tasks.front());
            tasks.pop();
            task();                                    // runs the lambda, fills its future
        }
    });
    for (auto& f : results) std::cout << f.get() << ' ';
    std::cout << '\n';
    worker.join();
    return 0;
}
```

OUTPUT

```
100 200 300 400
```

Tool	Who runs the work	Best for
`std::async`	the library (new thread or deferred)	quick one-off background computations
`std::promise` + `future`	your own thread, which sets the value explicitly	results produced in the middle of a longer thread, signals
`std::packaged_task`	whoever calls the task (a queue, a pool)	task queues and thread pools

23.6 C++20 synchronisation tools

23.6.1 std::latch, std::barrier

A `std::latch` (header `<latch>`) is a single-use countdown: threads call `count_down()`, and `wait()` blocks until the counter reaches zero. Use it to wait until N workers are ready or done. A `std::barrier` (header `<barrier>`) is reusable: a fixed group of threads repeatedly meets at the barrier with `arrive_and_wait()`; when the last one arrives, an optional completion function runs and all continue into the next **phase**. Use it for algorithms that proceed in rounds.

LATCH AND BARRIER

main.cpp

```cpp
#include <barrier>
#include <iostream>
#include <latch>
#include <string>
#include <thread>
#include <vector>

int main() {
    const int n = 3;
    std::vector<int> loaded(n, 0);
    std::latch all_loaded(n);
    std::vector<std::jthread> loaders;
    for (int i = 0; i < n; ++i)
        loaders.emplace_back([&, i] { loaded[i] = (i + 1) * 10; all_loaded.count_down(); });
    all_loaded.wait();                                  // continue only when every loader is
done
    std::cout << "all data loaded: " << loaded[0] + loaded[1] + loaded[2] << " records\n";

    int phase = 0;
    std::vector<int> progress(n, 0);
    std::barrier sync(n, [&]() noexcept {               // runs once per phase, by the last
arriving thread
        ++phase;
        std::cout << "phase " << phase << " complete: " << progress[0] << ' ' << progress[1] << ' ' <<
progress[2] << '\n';
    });
    {
        std::vector<std::jthread> workers;
        for (int i = 0; i < n; ++i)
            workers.emplace_back([&, i] {
                for (int round = 0; round < 3; ++round) {
                    progress[i] += i + 1;               // this round's work
                    sync.arrive_and_wait();             // wait for the others
                }
            });
    }
    return 0;
}
```

OUTPUT

```
all data loaded: 60 records
phase 1 complete: 1 2 3
phase 2 complete: 2 4 6
phase 3 complete: 3 6 9
```

23.6.2 std::counting_semaphore

A **semaphore** (header `<semaphore>`) holds a count of available permits. `acquire()` takes a permit, blocking while there are none; `release()` returns one. A `std::counting_semaphore<N>` limits how many threads can use a resource at once (for example at most 2 simultaneous downloads); `std::binary_semaphore` (a maximum of 1) is a lightweight signal between threads.

LIMITING CONCURRENCY WITH A SEMAPHORE

main.cpp

```
#include <atomic>
#include <chrono>
#include <iostream>
#include <semaphore>
#include <thread>
#include <vector>
using namespace std::chrono_literals;

std::counting_semaphore<2> slots(2);            // at most 2 downloads at a time
std::atomic<int> active{0}, peak{0};

void download(int id) {
    slots.acquire();                            // wait for a free slot
    int now = ++active;
    int p = peak.load();
    while (now > p && !peak.compare_exchange_weak(p, now)) {}
    std::this_thread::sleep_for(30ms);          // "downloading"
    --active;
    slots.release();                            // free the slot
    (void)id;
}

int main() {
    std::vector<std::jthread> ts;
    for (int i = 0; i < 6; ++i) ts.emplace_back(download, i);
    ts.clear();                                 // jthreads join here
    std::cout << "6 downloads finished, never more than " << peak << " at once\n";

    std::binary_semaphore signal(0);            // starts "taken"
    std::jthread waiter([&signal] { signal.acquire(); std::cout << "waiter: signalled\n"; });
    std::this_thread::sleep_for(10ms);
    std::cout << "main: sending signal\n";
    signal.release();
    return 0;
}
```

OUTPUT

```
6 downloads finished, never more than 2 at once
main: sending signal
waiter: signalled
```

23.6.3 Stop tokens

Stop tokens give **cooperative cancellation**: a `std::stop_source` issues requests, `std::stop_token`s observe them, and a `std::stop_callback` registers a function to run when a stop is requested (useful for waking a blocked operation). `std::jthread` creates the source for you; `std::condition_variable_any::wait` also accepts a stop token, so a waiting thread wakes up when cancelled.

CANCELLING A WAITING THREAD main.cpp

```
#include <chrono>
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
using namespace std::chrono_literals;

int main() {
    std::mutex m;
    std::condition_variable_any cv;
    std::queue<int> jobs;

    std::jthread worker([&](std::stop_token st) {
        std::stop_callback on_stop(st, [] { std::cout << "  (stop callback: cleaning up)\n"; });
        int handled = 0;
        while (true) {
            std::unique_lock lock(m);
            bool got = cv.wait(lock, st, [&] { return !jobs.empty(); });   // wakes on a job OR on a
stop request
            if (!got) break;                                               // stopped while waiting
            jobs.pop();
            ++handled;
        }
        std::cout << "worker exits after " << handled << " jobs\n";
    });

    for (int i = 0; i < 3; ++i) {
        { std::lock_guard lock(m); jobs.push(i); }
        cv.notify_one();
        std::this_thread::sleep_for(10ms);
    }
    std::cout << "main: requesting stop\n";
    worker.request_stop();                      // the waiting worker wakes up and leaves
    worker.join();
    return 0;
}
```

OUTPUT

```
main: requesting stop
  (stop callback: cleaning up)
worker exits after 3 jobs
```

23.7 Thread pools

23.7.1 Building a simple thread pool

Creating a thread for every small task is expensive (tens of microseconds each) and creating too many oversubscribes the CPU. A **thread pool** starts a fixed number of worker threads once; tasks are put in a queue, and idle workers take them out and run them. Combining the pieces of this module gives a compact, complete pool: a task queue protected by a mutex, a condition variable to wake workers, `packaged_task` to return results through futures, and jthreads with stop tokens for shutdown.

A COMPLETE THREAD POOL main.cpp

```
#include <condition_variable>
#include <functional>
#include <future>
```

```cpp
#include <iostream>
#include <mutex>
#include <queue>
#include <string>
#include <thread>
#include <vector>

class ThreadPool {
public:
    explicit ThreadPool(std::size_t n) {
        for (std::size_t i = 0; i < n; ++i)
            workers_.emplace_back([this](std::stop_token st) { run(st); });
    }
    ~ThreadPool() {
        for (auto& w : workers_) w.request_stop();
        cv_.notify_all();                                   // wake everyone so they can exit
    }                                                       // jthreads join here

    template <typename F, typename... Args>
    auto submit(F&& f, Args&&... args) {
        using R = std::invoke_result_t<F, Args...>;
        auto task = std::make_shared<std::packaged_task<R()>>(std::bind(std::forward<F>(f),
std::forward<Args>(args)...));
        std::future<R> result = task->get_future();
        {
            std::lock_guard lock(m_);
            tasks_.push([task] { (*task)(); });             // type-erase into std::function<void()>
        }
        cv_.notify_one();
        return result;
    }

private:
    void run(std::stop_token st) {
        while (true) {
            std::function<void()> job;
            {
                std::unique_lock lock(m_);
                cv_.wait(lock, st, [this] { return !tasks_.empty(); });
                if (tasks_.empty()) return;                 // stop requested and nothing left
                job = std::move(tasks_.front());
                tasks_.pop();
            }
            job();                                          // run outside the lock
        }
    }
    std::mutex m_;
    std::condition_variable_any cv_;
    std::queue<std::function<void()>> tasks_;
    std::vector<std::jthread> workers_;                     // declared last: destroyed (joined) first
};

bool is_prime(long n) {
    if (n < 2) return false;
    for (long d = 2; d * d <= n; ++d) if (n % d == 0) return false;
    return true;
}
long count_primes(long from, long to) {
    long c = 0;
    for (long n = from; n < to; ++n) c += is_prime(n);
    return c;
}

int main() {
```

```
    ThreadPool pool(4);
    std::vector<std::future<long>> parts;
    for (long start = 0; start < 1'000'000; start += 100'000)
        parts.push_back(pool.submit(count_primes, start, start + 100'000));     // 10 tasks on 4
workers
    long total = 0;
    for (std::size_t i = 0; i < parts.size(); ++i) {
        long c = parts[i].get();
        total += c;
        if (i < 3) std::cout << "primes in [" << i * 100000 << ", " << (i + 1) * 100000 << "): " << c
<< '\n';
    }
    std::cout << "primes below 1,000,000: " << total << '\n';
    auto greeting = pool.submit([](std::string name) { return "hello, " + name; },
std::string("pool"));
    std::cout << greeting.get() << '\n';
    return 0;
}
```

OUTPUT

```
primes in [0, 100000): 9592
primes in [100000, 200000): 8392
primes in [200000, 300000): 8013
primes below 1,000,000: 78498
hello, pool
```

Real projects usually use a ready-made pool or task system (Intel TBB, Boost.Asio's thread_pool, or the C++26 `std::execution` schedulers), which add work stealing, priorities and better shutdown handling. Writing one yourself, as above, is the best way to understand how they work.

Threads share memory, so every shared variable needs protection: a mutex with an RAII lock (scoped_lock by default) for compound data, an atomic for single counters and flags. Avoid deadlock with scoped_lock or a fixed lock order. Use condition variables (always with a predicate) to wait for conditions, async/future/promise/packaged_task to return results, the C++20 latch, barrier, semaphore and stop tokens for coordination, and a thread pool instead of a thread per task. Prefer jthread, and check with ThreadSanitizer.

PRACTICE CHECKPOINT

A **shared counter updated by 10 threads** (wrong without a mutex, correct with one), a **producer-consumer queue** with a condition variable, and a **parallel sum with std::async**.

SAMPLE SOLUTION

main.cpp

```
#include <condition_variable>
#include <future>
#include <iostream>
#include <mutex>
#include <numeric>
#include <queue>
#include <thread>
#include <vector>

// ---------- 1. Shared counter ----------
long unsafe_counter = 0;
long safe_counter = 0;
std::mutex counter_mutex;
```

```
void count_unsafe() { for (int i = 0; i < 1000000; ++i) ++unsafe_counter; }       // data race
void count_safe() {
    for (int i = 0; i < 1000000; ++i) {
        std::lock_guard lock(counter_mutex);
        ++safe_counter;
    }
}

// ---------- 2. Producer-consumer ----------
std::queue<int> orders;
std::mutex q_mutex;
std::condition_variable q_cv;
bool done = false;

void producer(int count) {
    for (int i = 1; i <= count; ++i) {
        { std::lock_guard lock(q_mutex); orders.push(i); }
        q_cv.notify_one();
    }
    { std::lock_guard lock(q_mutex); done = true; }
    q_cv.notify_all();
}
void consumer(long& total, int& handled) {
    while (true) {
        std::unique_lock lock(q_mutex);
        q_cv.wait(lock, [] { return !orders.empty() || done; });
        if (orders.empty()) return;                          // done and drained
        int order = orders.front();
        orders.pop();
        lock.unlock();                                       // process outside the lock
        total += order;
        ++handled;
    }
}

// ---------- 3. Parallel sum with std::async ----------
long long parallel_sum(const std::vector<int>& v, std::size_t parts) {
    std::vector<std::future<long long>> futures;
    std::size_t chunk = v.size() / parts;
    for (std::size_t p = 0; p < parts; ++p) {
        auto first = v.begin() + p * chunk;
        auto last = (p == parts - 1) ? v.end() : first + chunk;
        futures.push_back(std::async(std::launch::async, [first, last] { return std::accumulate(first,
last, 0LL); }));
    }
    long long total = 0;
    for (auto& f : futures) total += f.get();
    return total;
}

int main() {
    std::vector<std::thread> ts;
    for (int i = 0; i < 10; ++i) ts.emplace_back(count_unsafe);
    for (auto& t : ts) t.join();
    ts.clear();
    for (int i = 0; i < 10; ++i) ts.emplace_back(count_safe);
    for (auto& t : ts) t.join();
    std::cout << "without mutex: " << unsafe_counter << (unsafe_counter == 10000000 ? " (lucky this
time)" : " (WRONG, expected 10000000)") << '\n';
    std::cout << "with mutex:    " << safe_counter << " (expected 10000000)\n";

    long total1 = 0, total2 = 0;
    int handled1 = 0, handled2 = 0;
    std::thread c1(consumer, std::ref(total1), std::ref(handled1));
```

```
    std::thread c2(consumer, std::ref(total2), std::ref(handled2));
    std::thread p(producer, 500);
    p.join(); c1.join(); c2.join();
    std::cout << "orders handled: " << handled1 + handled2 << ", order-number sum " << total1 + total2
<< " (expected 125250)\n";

    std::vector<int> data(10'000'000);
    std::iota(data.begin(), data.end(), 1);
    long long expected = 10'000'000LL * 10'000'001 / 2;
    long long result = parallel_sum(data, 4);
    std::cout << "parallel sum: " << result << (result == expected ? " (correct)" : " (wrong)") <<
'\n';
    return 0;
}
```

OUTPUT

```
without mutex: 3740445 (WRONG, expected 10000000)
with mutex:    10000000 (expected 10000000)
orders handled: 500, order-number sum 125250 (expected 125250)
parallel sum: 50000005000000 (correct)
```

MODULE 24 · ADVANCED

Building, debugging, testing and best practices

Minimum duration **4 days** Level **Advanced**

Knowing the language is half the job. Real C++ projects are split into many files and libraries, built with CMake, debugged with GDB, checked with sanitizers and static analysers, covered by unit tests, and written to shared guidelines. This module walks through that professional toolset on a small but real project, a bank account library with an application, and then covers best practices, design patterns and performance.

IN THIS MODULE

24.1 Multi-file projects
24.2 Build systems
24.3 Debugging
24.4 Memory and correctness tools
24.5 Testing
24.6 Best practices
24.7 Design patterns in C++
24.8 Performance

24.1 Multi-file projects

24.1.1 Header and source files, include guards and #pragma once

A project is split into **headers** (`.h` / `.hpp`), which declare the interface (classes, function prototypes, inline functions and templates), and **source files** (`.cpp`), which define the implementation. Each source file is compiled separately into an object file; the linker joins them. A header may be included several times in one translation unit (directly and through other headers), so it needs an **include guard**: either the classic `#ifndef/#define/#endif` trio, or `#pragma once`, which every major compiler supports.

Our running project, a small banking library with an application, is laid out the way most C++ projects are:

Project layout

```
bank/
├── CMakeLists.txt
├── Makefile
├── include/bank/account.h      public header of the library
├── src/account.cpp             library implementation
├── src/format.cpp
├── app/main.cpp                the application that uses the library
└── tests/test_account.cpp      unit tests
```

INCLUDE/BANK/ACCOUNT.H C++

```cpp
#pragma once
#include <string>
#include <vector>

namespace bank {

class Account {
public:
    Account(std::string owner, long long opening_paise);
    void deposit(long long paise);
    bool withdraw(long long paise);          // false if funds are insufficient
    long long balance() const { return balance_; }
    const std::string& owner() const { return owner_; }
    const std::vector<long long>& history() const { return history_; }
private:
    std::string owner_;
    long long balance_;
    std::vector<long long> history_;
};

std::string format_paise(long long paise);   // 123456 -> "1234.56"

}  // namespace bank
```

SRC/ACCOUNT.CPP C++

```cpp
#include "bank/account.h"

#include <stdexcept>

namespace bank {

Account::Account(std::string owner, long long opening_paise)
    : owner_(std::move(owner)), balance_(opening_paise) {
    if (opening_paise < 0) throw std::invalid_argument("negative opening balance");
    history_.push_back(opening_paise);
}

void Account::deposit(long long paise) {
    if (paise <= 0) throw std::invalid_argument("deposit must be positive");
    balance_ += paise;
    history_.push_back(paise);
}

bool Account::withdraw(long long paise) {
    if (paise <= 0) throw std::invalid_argument("withdrawal must be positive");
    if (paise > balance_) return false;
    balance_ -= paise;
    history_.push_back(-paise);
    return true;
}

}  // namespace bank
```

SRC/FORMAT.CPP C++

```cpp
#include "bank/account.h"

#include <cstdio>

namespace bank {

std::string format_paise(long long paise) {
    char buf[32];
    const char* sign = paise < 0 ? "-" : "";
    if (paise < 0) paise = -paise;
    std::snprintf(buf, sizeof buf, "%s%lld.%02lld", sign, paise / 100, paise % 100);
    return buf;
}

}  // namespace bank
```

APP/MAIN.CPP C++

```cpp
#include <iostream>

#include "bank/account.h"

int main() {
    bank::Account acc("Asha", 500000);          // Rs 5000.00
    acc.deposit(125050);
    if (!acc.withdraw(10000000)) std::cout << "withdrawal of Rs 100000.00 declined\n";
    acc.withdraw(20000);
    std::cout << acc.owner() << " has Rs " << bank::format_paise(acc.balance())
              << " after " << acc.history().size() << " transactions\n";
    return 0;
}
```

THE TRADITIONAL ALTERNATIVE TO #PRAGMA ONCE C++

```cpp
#ifndef BANK_ACCOUNT_H        // classic include guard: same effect as #pragma once
#define BANK_ACCOUNT_H
// ... declarations ...
#endif  // BANK_ACCOUNT_H
```

- Put the public headers of a library in `include/<libname>/` and include them as `"bank/account.h"`, so names cannot clash with other libraries.
- A header should compile on its own: include what it uses (`<string>`, `<vector>`) rather than relying on the includer.
- Never write `using namespace std;` in a header: it would leak into every file that includes it.
- Include your own header first in its .cpp file, which proves that the header is self-contained.

24.1.2 The One Definition Rule (ODR)

The **One Definition Rule** says: in the whole program, every non-inline function and variable must be defined **exactly once**; classes, inline functions, templates and inline variables may be defined in several translation units, but all definitions must be **identical**. Breaking the first part gives a link error; breaking the second is undefined behaviour that the toolchain usually cannot detect. The usual mistake is putting a non-inline definition in a header that several .cpp files include.

A DEFINITION IN A HEADER INCLUDED TWICE

main.cpp

```
#include "config.h"
#include <iostream>

void print_from_other();

int main() {
    std::cout << "main sees " << app_name << ", version " << version() << '\n';
    print_from_other();
    return 0;
}
```

OUTPUT

```
/usr/bin/ld: /tmp/ccOSWYeU.o:config.h:3: multiple definition of `app_name[abi:cxx11]';
/tmp/ccHnEapq.o:config.h:3: first defined here
/usr/bin/ld: /tmp/ccOSWYeU.o: in function `std::allocator<char>::~allocator()':
/usr/include/c++/13/bits/basic_string.h:288: multiple definition of `version()';
/tmp/ccHnEapq.o:config.h:4: first defined here
collect2: error: ld returned 1 exit status
```

Both main.cpp and other.cpp contain a definition of `app_name` and `version`, so the linker finds two of each. The fixes: mark them `inline` (C++17 inline variables), or declare them in the header (`extern std::string app_name;`, `int version();`) and define them in one .cpp file. Constants can be `constexpr` or `inline constexpr`.

FIXED WITH INLINE

main.cpp

```
#include "config.h"
#include <iostream>

void print_from_other();

int main() {
    std::cout << "main sees " << app_name << ", version " << version() << '\n';
    print_from_other();
    return 0;
}
```

OUTPUT

```
main sees LBF Bank, version 3
other sees LBF Bank
```

24.1.3 Static and shared libraries

A **library** packages compiled code for reuse. A **static library** (`libbank.a` on Linux, `bank.lib` on Windows) is an archive of object files; the linker copies the needed code into the executable, which then runs on its own. A **shared library** (`libbank.so`, `bank.dll`, `libbank.dylib`) is loaded at run time; many programs share one copy and it can be updated without relinking, but it must be found when the program starts.

Building a static library with make (see 24.2.1)

```
$ make
mkdir -p build
g++ -std=c++20 -Wall -Wextra -Iinclude -O2 -c src/account.cpp -o build/account.o
g++ -std=c++20 -Wall -Wextra -Iinclude -O2 -c src/format.cpp -o build/format.o
ar rcs libbank.a build/account.o build/format.o
g++ -std=c++20 -Wall -Wextra -Iinclude -O2 app/main.cpp -L. -lbank -o bank_app
$ ./bank_app
withdrawal of Rs 100000.00 declined
Asha has Rs 6050.50 after 3 transactions
```

Building and using a shared library

```
$ g++ -std=c++20 -fPIC -shared -Iinclude src/account.cpp src/format.cpp -o libbank.so
$ g++ -std=c++20 -Iinclude app/main.cpp -L. -lbank -o bank_app
$ ./bank_app
./bank_app: error while loading shared libraries: libbank.so: cannot open shared object file: No such
file or directory
$ LD_LIBRARY_PATH=. ./bank_app
withdrawal of Rs 100000.00 declined
Asha has Rs 6050.50 after 3 transactions
$ ldd bank_app | grep bank
        libbank.so => not found
$ nm -C libbank.so | grep " T "
0000000000006b86 T bank::format_paise[abi:cxx11](long long)
00000000000050b4 T bank::Account::deposit(long long)
0000000000005162 T bank::Account::withdraw(long long)
0000000000004fba T bank::Account::Account(std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> >, long long)
0000000000004fba T bank::Account::Account(std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> >, long long)
```

The program linked against `libbank.so` fails to start until the loader knows where to find it: install the library in a system directory (and run `ldconfig`), set `LD_LIBRARY_PATH` as shown, or embed a search path with `-Wl,-rpath,'$ORIGIN'`. `nm -C` lists the functions the library exports.

	Static library	Shared library
File	`.a` / `.lib`	`.so` / `.dll` / `.dylib`
Linked	at build time, copied into the executable	at load time by the dynamic loader
Executable size	larger	smaller
Updating the library	relink every program	replace the file (if the ABI is unchanged)
Deployment	one self-contained file	the library must be shipped and found
Build flag	`ar rcs`	`-fPIC -shared`

24.2 Build systems

24.2.1 Make basics

Typing compiler commands by hand does not scale. **Make** reads a `Makefile` of **rules**: *target: prerequisites* followed by tab-indented commands. It rebuilds a target only if it is missing or older than one of its prerequisites, so after a change only the affected files are recompiled. Variables (`CXX`,

`CXXFLAGS`), pattern rules (`build/%.o: src/%.cpp`) and automatic variables (`$@` the target, `$<` the first prerequisite, `$^` all prerequisites) keep it short.

MAKEFILE

text

```
CXX      := g++
CXXFLAGS := -std=c++20 -Wall -Wextra -Iinclude -O2
OBJS     := build/account.o build/format.o

bank_app: app/main.cpp libbank.a
	$(CXX) $(CXXFLAGS) app/main.cpp -L. -lbank -o $@

libbank.a: $(OBJS)
	ar rcs $@ $^

build/%.o: src/%.cpp include/bank/account.h | build
	$(CXX) $(CXXFLAGS) -c $< -o $@

build:
	mkdir -p build

clean:
	rm -rf build libbank.a bank_app

.PHONY: clean
```

make rebuilds only what changed

```
$ make
mkdir -p build
g++ -std=c++20 -Wall -Wextra -Iinclude -O2 -c src/account.cpp -o build/account.o
g++ -std=c++20 -Wall -Wextra -Iinclude -O2 -c src/format.cpp -o build/format.o
ar rcs libbank.a build/account.o build/format.o
g++ -std=c++20 -Wall -Wextra -Iinclude -O2 app/main.cpp -L. -lbank -o bank_app
$ ./bank_app
withdrawal of Rs 100000.00 declined
Asha has Rs 6050.50 after 3 transactions
$ touch src/format.cpp && make
g++ -std=c++20 -Wall -Wextra -Iinclude -O2 -c src/format.cpp -o build/format.o
ar rcs libbank.a build/account.o build/format.o
g++ -std=c++20 -Wall -Wextra -Iinclude -O2 app/main.cpp -L. -lbank -o bank_app
$ make
make: 'bank_app' is up to date.
```

24.2.2 CMake: CMakeLists.txt, targets, linking libraries

CMake is the de-facto standard build system generator for C++. A `CMakeLists.txt` describes **targets** (libraries and executables) and their properties; CMake then generates the real build files for Make, Ninja, Visual Studio or Xcode, so one description works on every platform and in every IDE. Modern CMake is target-based: `target_include_directories`, `target_compile_options` and `target_link_libraries` attach settings to targets, and `PUBLIC` settings propagate to whoever links the target. Building happens in a separate build directory, so several configurations (Debug, Release, sanitizers) can coexist.

CMAKELISTS.TXT text

```
cmake_minimum_required(VERSION 3.20)
project(BankApp VERSION 1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)          # for clang-tidy and editors

# --- optional sanitizer build: cmake -DSANITIZE=ON --- (before the targets it applies to)
option(SANITIZE "Build with AddressSanitizer and UBSan" OFF)
if(SANITIZE)
    add_compile_options(-fsanitize=address,undefined -fno-omit-frame-pointer)
    add_link_options(-fsanitize=address,undefined)
endif()

# --- the library ---
add_library(bank src/account.cpp src/format.cpp)
target_include_directories(bank PUBLIC include)
target_compile_options(bank PRIVATE -Wall -Wextra -Wpedantic)

# --- the application ---
add_executable(bank_app app/main.cpp)
target_link_libraries(bank_app PRIVATE bank)

# --- unit tests: cmake -DBUILD_TESTS=ON ---
option(BUILD_TESTS "Build the GoogleTest unit tests" OFF)
if(BUILD_TESTS)
    include(FetchContent)
    FetchContent_Declare(googletest
        URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip)
    FetchContent_MakeAvailable(googletest)
    enable_testing()
    add_executable(bank_tests tests/test_account.cpp)
    target_link_libraries(bank_tests PRIVATE bank GTest::gtest_main)
    include(GoogleTest)
    gtest_discover_tests(bank_tests)
endif()
```

Configuring and building with CMake and Ninja

```
$ cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug
-- The CXX compiler identification is GNU 13.3.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done (0.2s)
-- Generating done (0.0s)
-- Build files have been written to: /home/learner/bank/build
$ cmake --build build
[1/5] Building CXX object CMakeFiles/bank.dir/src/account.cpp.o
[2/5] Building CXX object CMakeFiles/bank.dir/src/format.cpp.o
[3/5] Building CXX object CMakeFiles/bank_app.dir/app/main.cpp.o
[4/5] Linking CXX static library libbank.a
[5/5] Linking CXX executable bank_app
$ ./build/bank_app
withdrawal of Rs 100000.00 declined
Asha has Rs 6050.50 after 3 transactions
$ cmake -S . -B build-asan -G Ninja -DSANITIZE=ON
-- Build files have been written to: /home/learner/bank/build-asan
$ cmake --build build-asan && ./build-asan/bank_app
[5/5] Linking CXX executable bank_app
withdrawal of Rs 100000.00 declined
Asha has Rs 6050.50 after 3 transactions
$ ldd build-asan/bank_app | grep -E "asan|ubsan"
        libasan.so.8 => /lib/x86_64-linux-gnu/libasan.so.8
        libubsan.so.1 => /lib/x86_64-linux-gnu/libubsan.so.1
```

Command	Meaning
`add_library(name sources...)`	a library target (static by default; `SHARED` for a shared one)
`add_executable(name sources...)`	an executable target
`target_include_directories(t PUBLIC dir)`	include path for t and for everyone linking t
`target_link_libraries(app PRIVATE lib)`	link lib into app (and inherit its PUBLIC settings)
`find_package(Pkg REQUIRED)`	locate an installed library (Threads, OpenSSL, fmt, GTest ...)
`FetchContent`	download and build a dependency as part of the project
`option()`, `if()`	user-selectable switches such as SANITIZE and BUILD_TESTS
`cmake --build build --config Release`, `ctest`	build, and run the registered tests

24.2.3 Package managers: vcpkg and Conan

C++ has no single official package manager, but two are widely used. **vcpkg** (Microsoft) builds libraries from source for your platform and integrates with CMake through a toolchain file; a `vcpkg.json` manifest in the project lists dependencies. **Conan** (JFrog) is a Python-based manager with pre-built binaries and profiles for compilers and settings; a `conanfile.txt` lists requirements and generators. Both make `find_package(fmt REQUIRED)` just work.

VCPKG

text

```
// vcpkg.json (manifest mode)
{
  "name": "bank",
  "version": "1.0.0",
  "dependencies": [ "fmt", "nlohmann-json", "gtest" ]
}

$ cmake -S . -B build -DCMAKE_TOOLCHAIN_FILE=$VCPKG_ROOT/scripts/buildsystems/vcpkg.cmake
# CMakeLists.txt then uses:  find_package(fmt CONFIG REQUIRED)
#                            target_link_libraries(bank_app PRIVATE fmt::fmt)
```

CONAN

text

```
# conanfile.txt
[requires]
fmt/10.2.1
nlohmann_json/3.11.3

[generators]
CMakeDeps
CMakeToolchain

$ conan install . --output-folder=build --build=missing
$ cmake -S . -B build -DCMAKE_TOOLCHAIN_FILE=build/conan_toolchain.cmake
```

24.3 Debugging

24.3.1 GDB / LLDB basics: breakpoints, stepping, backtrace, watch

A **debugger** runs your program under control: it stops at **breakpoints**, executes one line at a time, shows variables, the call stack, and stops when a variable changes (**watchpoints**) or the program crashes. Compile with `-g` (debug information) and preferably `-O0`, otherwise the optimiser reorders code and removes variables. GDB is the GNU debugger; LLDB (from LLVM, the default on macOS) has almost the same commands.

STATS.CPP: A PROGRAM WITH A BUG

C++

```
#include <iostream>
#include <vector>

struct Student { const char* name; int marks; };

const Student* find_topper(const std::vector<Student>& v) {
    const Student* best = nullptr;
    for (const auto& s : v)
        if (s.marks > 90) best = &s;          // bug: should compare with best->marks
    return best;
}

double average(const std::vector<Student>& v) {
    int total = 0;
    for (std::size_t i = 0; i < v.size(); ++i)
        total += v[i].marks;
    return static_cast<double>(total) / v.size();
}

int main() {
    std::vector<Student> cls = {{"Asha", 78}, {"Ravi", 85}, {"Meera", 88}};
    std::cout << "average " << average(cls) << '\n';
    const Student* top = find_topper(cls);
    std::cout << "topper: " << top->name << '\n';   // crashes: top is nullptr
    return 0;
}
```

A GDB session that finds the bug

```
$ g++ -std=c++20 -g -O0 stats.cpp -o stats
$ ./stats
average 83.6667
Segmentation fault (core dumped)
$ gdb -q ./stats
(gdb) break find_topper
Breakpoint 1 at 0x12b9: file stats.cpp, line 6.
(gdb) run
Breakpoint 1, find_topper (v=std::vector of length 3, capacity 3 = {...}) at stats.cpp:6
6       const Student* find_topper(const std::vector<Student>& v) {
(gdb) next
7           const Student* best = nullptr;
(gdb) next
8           for (const auto& s : v)
(gdb) next
9               if (s.marks > 90) best = &s;          // bug: should compare with best->marks
(gdb) print s
$1 = (const Student &) @0x55555556c2b0: {name = 0x555555557039 "Asha", marks = 78}
(gdb) print best
$2 = (const Student *) 0x0
(gdb) next
8           for (const auto& s : v)
(gdb) next
9               if (s.marks > 90) best = &s;          // bug: should compare with best->marks
(gdb) print s.marks
$3 = 85
(gdb) finish
0x00005555555554e1 in main () at stats.cpp:23
23          const Student* top = find_topper(cls);
Value returned is $4 = (const Student *) 0x0
(gdb) continue
Program received signal SIGSEGV, Segmentation fault.
0x0000555555555505 in main () at stats.cpp:24
24          std::cout << "topper: " << top->name << '\n';   // crashes: top is nullptr
(gdb) backtrace
#0  0x0000555555555505 in main () at stats.cpp:24
(gdb) print top
$5 = (const Student *) 0x0
```

The session shows the whole story: no mark is above 90, so `best` stays null, `find_topper` returns null, and `main` dereferences it. The fix is to compare against the best so far (`if (!best || s.marks > best->marks)`) and to check the result before using it.

GDB command	Short	Effect
`break file.cpp:24`, `break func`	`b`	set a breakpoint; `break 24 if i == 5` is conditional
`run [args]`	`r`	start the program
`next` / `step`	`n` / `s`	execute one line, stepping over / into calls
`finish`		run until the current function returns, print its value
`continue`	`c`	run to the next breakpoint
`print expr`, `display expr`	`p`	show a value once / after every step
`backtrace`, `frame N`, `up` / `down`	`bt`	show the call stack, move between frames
`watch var`		stop whenever var changes

GDB command	Short	Effect
`info locals`, `info breakpoints`		list local variables / breakpoints
`list`	`l`	show source around the current line

TIP

When a program crashes outside the debugger, enable core dumps (`ulimit -c unlimited`) and open the dump afterwards with `gdb ./stats core` : `bt` then shows where it crashed, without reproducing the problem.

24.3.2 Debugging in Visual Studio, CLion or VS Code

IDEs put a graphical front end on the same debuggers: click in the margin to set a breakpoint, press F5 to start, F10/F11 to step over/into (Visual Studio and VS Code), hover over a variable to see its value, and use the Watch, Locals and Call Stack panels. Visual Studio uses its own debugger; CLion bundles GDB and LLDB; VS Code uses the C/C++ extension with a `launch.json` that names the program and the debugger.

A VS CODE LAUNCH CONFIGURATION text

```
// .vscode/launch.json
{
  "version": "0.2.0",
  "configurations": [{
      "name": "Debug bank_app",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/build/bank_app",
      "cwd": "${workspaceFolder}",
      "MIMode": "gdb",
      "preLaunchTask": "cmake build"
  }]
}
```

Action	Visual Studio	VS Code	CLion
Start debugging	F5	F5	Shift+F9
Toggle breakpoint	F9	F9	Ctrl+F8
Step over / into / out	F10 / F11 / Shift+F11	F10 / F11 / Shift+F11	F8 / F7 / Shift+F8
Continue	F5	F5	F9

24.4 Memory and correctness tools

24.4.1 AddressSanitizer and UndefinedBehaviorSanitizer

Sanitizers are compiler options that insert run-time checks. **AddressSanitizer** (`-fsanitize=address`) detects out-of-bounds accesses on heap, stack and globals, use-after-free, double free and leaks, at about 2x slowdown, and reports the exact line plus where the memory was allocated. **UndefinedBehaviorSanitizer** (`-fsanitize=undefined`) detects signed overflow, invalid shifts, out-of-bounds array indexing, null dereferences, misaligned accesses and more. Run your tests under both regularly: they find bugs that "work by accident" in normal builds.

BUGS.CPP C++

```cpp
#include <climits>
#include <iostream>
#include <vector>

int sum_first(const int* data, int n) {
    int s = 0;
    for (int i = 0; i <= n; ++i) s += data[i];      // off by one: reads data[n]
    return s;
}

int main(int argc, char**) {
    int* marks = new int[3]{70, 80, 90};
    if (argc == 1) std::cout << sum_first(marks, 3) << '\n';
    delete[] marks;
    int big = INT_MAX - argc + 1;
    std::cout << big + 1 << '\n';                   // signed overflow
    return 0;
}
```

AddressSanitizer catches the off-by-one read

```
$ g++ -std=c++20 -g -fsanitize=address,undefined -fno-omit-frame-pointer bugs.cpp -o bugs && ./bugs
=================================================================
==12902==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x50200000001c at pc 0x55ec2bc663f4
bp 0x7ffd4bc69530 sp 0x7ffd4bc69520
READ of size 4 at 0x50200000001c thread T0
    #0 0x55ec2bc663f3 in sum_first(int const*, int) /home/learner/dbg/bugs.cpp:7
    #1 0x55ec2bc6677b in main /home/learner/dbg/bugs.cpp:13
    #2 0x7f8df182a1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
    #3 0x7f8df182a28a in __libc_start_main_impl ../csu/libc-start.c:360
    #4 0x55ec2bc66264 in _start (/home/learner/dbg/bugs+0x2264) (BuildId:
274f66d886b1d195bc169741711c5e5a0851f8a1)

0x50200000001c is located 0 bytes after 12-byte region [0x502000000010,0x50200000001c)
allocated by thread T0 here:
    #0 0x7f8df28fe6c8 in operator new[](unsigned long)
../../../../src/libsanitizer/asan/asan_new_delete.cpp:98
    #1 0x55ec2bc6647d in main /home/learner/dbg/bugs.cpp:12
    #2 0x7f8df182a1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
    #3 0x7f8df182a28a in __libc_start_main_impl ../csu/libc-start.c:360
    #4 0x55ec2bc66264 in _start (/home/learner/dbg/bugs+0x2264) (BuildId:
274f66d886b1d195bc169741711c5e5a0851f8a1)

SUMMARY: AddressSanitizer: heap-buffer-overflow /home/learner/dbg/bugs.cpp:7 in sum_first(int const*,
int)
```

UB.CPP C++

```cpp
#include <climits>
#include <iostream>

int factorial(int n) {
    int r = 1;
    for (int i = 2; i <= n; ++i) r *= i;          // overflows for n > 12
    return r;
}

int main() {
    std::cout << "13! = " << factorial(13) << '\n';
    int bits = 33;
    std::cout << "1 << 33 = " << (1 << bits) << '\n';   // shift past the width of int
    int values[4] = {1, 2, 3, 4};
    int idx = 4;
    std::cout << "values[4] = " << values[idx] << '\n'; // out of bounds
    return 0;
}
```

UBSan reports each undefined operation

```
$ g++ -std=c++20 -g -fsanitize=undefined ub.cpp -o ub && ./ub
ub.cpp:6:36: runtime error: signed integer overflow: 479001600 * 13 cannot be represented in type
'int'
13! = 1932053504
ub.cpp:13:37: runtime error: shift exponent 33 is too large for 32-bit type 'int'
1 << 33 = 2
ub.cpp:16:46: runtime error: index 4 out of bounds for type 'int [4]'
ub.cpp:16:51: runtime error: load of address 0x7ffc47be8db0 with insufficient space for an object of
type 'int'
0x7ffc47be8db0: note: pointer points here
 04 00 00 00  50 8e be 47 fc 7f 00 00  00 19 6a df b0 e0 c4 59  f8 8e be 47 fc 7f 00 00  01 00 00 00
              ^
values[4] = 1203670608
```

Without the sanitizer this program prints the same wrong numbers silently (or, with optimisation, possibly something else entirely). Notice that UBSan continues after each report by default; add `-fno-sanitize-recover=all` to stop at the first error, which is what you want in tests. ThreadSanitizer (`-fsanitize=thread`, Module 23) cannot be combined with ASan; use a separate build.

24.4.2 Valgrind

Valgrind's Memcheck tool runs an unmodified program on a virtual CPU and checks every memory access: invalid reads and writes, use of uninitialised values, mismatched new/delete[] and leaks. No recompilation is needed (but `-g` gives line numbers), at the cost of a 20-50x slowdown. Other Valgrind tools profile (Callgrind, 24.8.1) and detect threading errors (Helgrind).

LEAK.CPP

C++

```cpp
#include <cstring>
#include <iostream>

char* make_greeting(const char* name) {
    char* buf = new char[64];
    std::strcpy(buf, "Hello, ");
    std::strcat(buf, name);
    return buf;                                  // caller must delete[] ... and forgets
}

int main() {
    for (int i = 0; i < 3; ++i) {
        char* g = make_greeting("Asha");
        if (i == 0) std::cout << g << '\n';
    }
    int* p = new int(5);
    delete p;
    return 0;
}
```

Valgrind finds the leaked greetings

```
$ g++ -std=c++20 -g -O0 leak.cpp -o leak
$ valgrind --leak-check=full ./leak
==12904== Memcheck, a memory error detector
==12904== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==12904== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==12904== Command: ./leak
==12904==
Hello, Asha
==12904==
==12904== HEAP SUMMARY:
==12904==     in use at exit: 192 bytes in 3 blocks
==12904==   total heap usage: 6 allocs, 3 frees, 78,020 bytes allocated
==12904==
==12904== 192 bytes in 3 blocks are definitely lost in loss record 1 of 1
==12904==    at 0x48485C3: operator new[](unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-
amd64-linux.so)
==12904==    by 0x109202: make_greeting(char const*) (leak.cpp:5)
==12904==    by 0x109254: main (leak.cpp:13)
==12904==
==12904== LEAK SUMMARY:
==12904==    definitely lost: 192 bytes in 3 blocks
==12904==    indirectly lost: 0 bytes in 0 blocks
==12904==      possibly lost: 0 bytes in 0 blocks
==12904==    still reachable: 0 bytes in 0 blocks
==12904==         suppressed: 0 bytes in 0 blocks
==12904==
==12904== For lists of detected and suppressed errors, rerun with: -s
==12904== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

24.4.3 Static analysis: clang-tidy, cppcheck

Static analysers examine source code without running it. **clang-tidy** applies hundreds of configurable checks (groups such as `bugprone-*`, `performance-*`, `modernize-*`, `cppcoreguidelines-*`, `readability-*`), many with automatic fixes (`-fix`). It needs the compile flags, which CMake provides through `compile_commands.json` when `CMAKE_EXPORT_COMPILE_COMMANDS` is on. **cppcheck** is an independent analyser focused on real bugs with few false positives (`cppcheck --enable=warning,performance src/`). Both are easy to run in CI.

TIDY.CPP: COMPILES CLEANLY, BUT ...

C++

```
#include <iostream>
#include <string>
#include <vector>

class Inventory {
public:
    Inventory(std::vector<std::string> items) : items_(items) {}
    int count() { return items_.size(); }
    void print() {
        for (int i = 0; i < items_.size(); i++) std::cout << items_[i] << std::endl;
    }
private:
    std::vector<std::string> items_;
};

int main() {
    int* raw = new int(42);
    Inventory inv({"pen", "ink"});
    inv.print();
    if (raw != NULL) std::cout << *raw + inv.count() << '\n';
    delete raw;
}
```

clang-tidy suggestions

```
$ clang-tidy tidy.cpp -checks='-*,modernize-*,-modernize-use-trailing-return-type,performance-*,\
    cppcoreguidelines-owning-memory,google-explicit-constructor,bugprone-narrowing-conversions' -- -
std=c++20
tidy.cpp:7:5: warning: single-argument constructors must be marked explicit to avoid unintentional
implicit conversions [google-explicit-constructor]
    7 |     Inventory(std::vector<std::string> items) : items_(items) {}
      |     ^
      |     explicit
tidy.cpp:7:15: warning: pass by value and use std::move [modernize-pass-by-value]
    3 | #include <vector>
    4 |
    5 | class Inventory {
    6 | public:
    7 |     Inventory(std::vector<std::string> items) : items_(items) {}
      |               ^
      |                                                 std::move( )
tidy.cpp:7:56: warning: parameter 'items' is passed by value and only copied once; consider moving it
to avoid unnecessary copies [performance-unnecessary-value-param]
    3 | #include <vector>
    4 |
    5 | class Inventory {
    6 | public:
    7 |     Inventory(std::vector<std::string> items) : items_(items) {}
      |                                                        ^
      |                                                 std::move( )
tidy.cpp:8:26: warning: narrowing conversion from 'size_type' (aka 'unsigned long') to signed type
'int' is implementation-defined [bugprone-narrowing-conversions]
    8 |     int count() { return items_.size(); }
      |                          ^
tidy.cpp:10:9: warning: use range-based for loop instead [modernize-loop-convert]
   10 |         for (int i = 0; i < items_.size(); i++) std::cout << items_[i] << std::endl;
      |         ^   ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~             ~~~~~~~~~
      |             (const auto & item : items_)                   item
tidy.cpp:10:75: warning: do not use 'std::endl' with streams; use '\n' instead [performance-avoid-
endl]
   10 |         for (int i = 0; i < items_.size(); i++) std::cout << items_[i] << std::endl;
      |                                                                           ^~~~~~~~
      |                                                                           '\n'
tidy.cpp:17:5: warning: initializing non-owner 'int *' with a newly created 'gsl::owner<>'
[cppcoreguidelines-owning-memory]
   17 |     int* raw = new int(42);
      |     ^~~~~~~~~~~~~~~~~~~~~~
tidy.cpp:20:16: warning: use nullptr [modernize-use-nullptr]
   20 |     if (raw != NULL) std::cout << *raw + inv.count() << '\n';
      |                ^~~~
      |                nullptr
tidy.cpp:21:5: warning: deleting a pointer through a type that is not marked 'gsl::owner<>'; consider
using a smart pointer instead [cppcoreguidelines-owning-memory]
   21 |     delete raw;
      |     ^      ~~~
tidy.cpp:17:5: note: variable declared here
   17 |     int* raw = new int(42);
      |     ^~~~~~~~~~~~~~~~~~~~~~
```

Every warning points to a real improvement: an implicit converting constructor, a needless copy, a narrowing size conversion, an old-style index loop, a needless flush with `std::endl`, `NULL` instead of `nullptr`, and a raw owning pointer. In a project, put the chosen checks in a `.clang-tidy` file at the root.

24.4.4 Formatting with clang-format

clang-format rewrites code to a consistent style, so code reviews discuss logic instead of spaces and braces. The style is set in a `.clang-format` file (based on LLVM, Google, Chromium, Mozilla, WebKit or Microsoft style, with overrides). Editors can format on save, and `--dry-run --Werror` makes CI fail on unformatted code.

.CLANG-FORMAT text

```
BasedOnStyle: Google
IndentWidth: 4
ColumnLimit: 100
```

MESSY.CPP, BEFORE C++

```cpp
#include <vector>
#include <iostream>
int  total( const std::vector<int>&v ){int s=0;
for(int x:v){ if(x>0)s+=x; }
    return s;}
int main(){std::vector<int> v{3,-1,4};std::cout<<total(v)<<"\n";}
```

Formatting, and checking formatting in CI

```
$ clang-format messy.cpp
#include <iostream>
#include <vector>
int total(const std::vector<int>& v) {
    int s = 0;
    for (int x : v) {
        if (x > 0) s += x;
    }
    return s;
}
int main() {
    std::vector<int> v{3, -1, 4};
    std::cout << total(v) << "\n";
}
$ clang-format --dry-run --Werror messy.cpp
messy.cpp:1:1: error: code should be clang-formatted [-Wclang-format-violations]
#include <vector>
^
messy.cpp:3:4: error: code should be clang-formatted [-Wclang-format-violations]
int  total( const std::vector<int>&v ){int s=0;
   ^
```

24.5 Testing

24.5.1 Unit testing with GoogleTest or Catch2

A **unit test** checks one small piece of code (a function or class) in isolation, automatically and repeatably. A test framework provides the structure: test cases, assertions, a runner that reports results, and integration with CMake's `ctest`. **GoogleTest** is the most widely used; `EXPECT_*` assertions record a failure and continue, `ASSERT_*` stop the test. **Catch2** is a popular alternative with natural expressions (`REQUIRE(acc.balance() == 3500)`) and BDD-style sections.

TESTS/TEST_ACCOUNT.CPP

C++

```cpp
#include <gtest/gtest.h>

#include <stdexcept>

#include "bank/account.h"

TEST(AccountTest, StartsWithOpeningBalance) {
    bank::Account acc("Asha", 50000);
    EXPECT_EQ(acc.balance(), 50000);
    EXPECT_EQ(acc.owner(), "Asha");
    EXPECT_EQ(acc.history().size(), 1u);
}

TEST(AccountTest, DepositIncreasesBalance) {
    bank::Account acc("Ravi", 1000);
    acc.deposit(2500);
    EXPECT_EQ(acc.balance(), 3500);
}

TEST(AccountTest, WithdrawMoreThanBalanceIsDeclined) {
    bank::Account acc("Meera", 1000);
    EXPECT_FALSE(acc.withdraw(5000));
    EXPECT_EQ(acc.balance(), 1000);              // unchanged
}

TEST(AccountTest, InvalidAmountsThrow) {
    bank::Account acc("Kiran", 1000);
    EXPECT_THROW(acc.deposit(0), std::invalid_argument);
    EXPECT_THROW(acc.withdraw(-5), std::invalid_argument);
    EXPECT_THROW(bank::Account("Dev", -1), std::invalid_argument);
}

TEST(FormatTest, FormatsPaise) {
    EXPECT_EQ(bank::format_paise(123456), "1234.56");
    EXPECT_EQ(bank::format_paise(5), "0.05");
    EXPECT_EQ(bank::format_paise(-250), "-2.50");
}
```

Running the tests (GoogleTest 1.14)

```
$ cmake -S . -B build -DBUILD_TESTS=ON && cmake --build build
$ ./build/bank_tests
[==========] Running 5 tests from 2 test suites.
[----------] Global test environment set-up.
[----------] 4 tests from AccountTest
[ RUN      ] AccountTest.StartsWithOpeningBalance
[       OK ] AccountTest.StartsWithOpeningBalance (0 ms)
[ RUN      ] AccountTest.DepositIncreasesBalance
[       OK ] AccountTest.DepositIncreasesBalance (0 ms)
[ RUN      ] AccountTest.WithdrawMoreThanBalanceIsDeclined
[       OK ] AccountTest.WithdrawMoreThanBalanceIsDeclined (0 ms)
[ RUN      ] AccountTest.InvalidAmountsThrow
[       OK ] AccountTest.InvalidAmountsThrow (0 ms)
[----------] 4 tests from AccountTest (0 ms total)

[----------] 1 test from FormatTest
[ RUN      ] FormatTest.FormatsPaise
[       OK ] FormatTest.FormatsPaise (0 ms)
[----------] 1 test from FormatTest (0 ms total)

[----------] Global test environment tear-down
[==========] 5 tests from 2 test suites ran. (0 ms total)
[  PASSED  ] 5 tests.
```

NOTE

GoogleTest could not be downloaded in the environment that built these notes, so this run shows GoogleTest's standard output format. The five test bodies themselves were compiled and executed against the real library (with a minimal stand-in for the GoogleTest macros) and all assertions passed.

GoogleTest	Checks
`EXPECT_EQ(a, b)`, `EXPECT_NE`, `EXPECT_LT`, `EXPECT_GE` …	comparisons, printing both values on failure
`EXPECT_TRUE(c)`, `EXPECT_FALSE(c)`	conditions
`EXPECT_DOUBLE_EQ`, `EXPECT_NEAR(a, b, eps)`	floating point with tolerance
`EXPECT_THROW(stmt, Type)`, `EXPECT_NO_THROW`	exceptions
`EXPECT_STREQ`	C strings by content
`TEST_F(Fixture, Name)`	tests sharing set-up code in a fixture class
`ASSERT_*`	same checks, but stop the test at the first failure

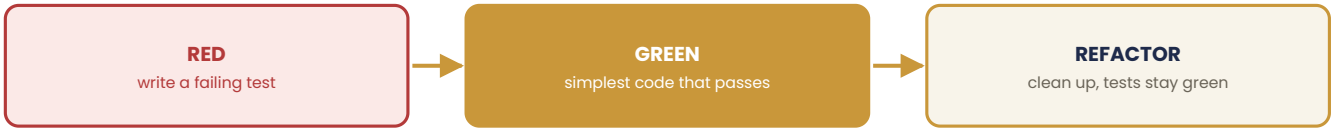
THE SAME IDEA IN CATCH2

C++

```
#include <catch2/catch_test_macros.hpp>
#include "bank/account.h"

TEST_CASE("withdrawing", "[account]") {
    bank::Account acc("Asha", 1000);
    SECTION("within the balance") {
        REQUIRE(acc.withdraw(400));
        CHECK(acc.balance() == 600);
    }
    SECTION("more than the balance") {         // each SECTION starts from a fresh acc
        REQUIRE_FALSE(acc.withdraw(5000));
        CHECK(acc.balance() == 1000);
    }
}
```

24.5.2 Test-driven development

Test-driven development (TDD) reverses the usual order: write a failing test for the next small piece of behaviour (**red**), write just enough code to make it pass (**green**), then clean up the code while the tests keep passing (**refactor**). The tests become a precise specification and a safety net for later changes. Here is how the negative-amount case of `format_paise` came about.



THE TDD CYCLE, REPEATED IN SMALL STEPS

BEFORE: FORMAT.CPP HANDLES ONLY POSITIVE AMOUNTS

C++

```
std::string format_paise(long long paise) {          // first version: only positive amounts
    char buf[32];
    std::snprintf(buf, sizeof buf, "%lld.%02lld", paise / 100, paise % 100);
    return buf;
}
```

RED: the new test for -250 paise fails

```
$ ./build/bank_tests --gtest_filter=FormatTest.*
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from FormatTest
[ RUN      ] FormatTest.FormatsPaise
tests/test_account.cpp:36: Failure
Expected equality of these values:
  bank::format_paise(-250)
    Which is: "-2.-50"
  "-2.50"

[  FAILED  ] FormatTest.FormatsPaise (0 ms)
[----------] 1 test from FormatTest (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 0 tests.
[  FAILED  ] 1 test, listed below:
[  FAILED  ] FormatTest.FormatsPaise

 1 FAILED TEST
```

CHECKING THE FAULTY VALUE THE TEST REPORTED

main.cpp

```cpp
#include <cstdio>
#include <iostream>
#include <string>

std::string old_format(long long paise) {
    char buf[32];
    std::snprintf(buf, sizeof buf, "%lld.%02lld", paise / 100, paise % 100);
    return buf;
}

int main() {
    std::cout << old_format(123456) << ' ' << old_format(5) << ' ' << old_format(-250) << '\n';
    return 0;
}
```

OUTPUT

```
1234.56 0.05 -2.-50
```

The failure message tells us exactly what happened: -250 / 100 is -2 and -250 % 100 is -50 in C++ (the remainder takes the sign of the dividend). The GREEN step is the version of `src/format.cpp` shown in 24.1.1, which prints the sign separately and formats the absolute value; with it, all five tests pass. The REFACTOR step could, for example, switch to `std::format`, with the tests guaranteeing that the behaviour does not change.

24.5.3 Benchmarking with Google Benchmark

Guessing about performance is unreliable; measure. **Google Benchmark** runs a function many times, chooses the iteration count automatically, prevents the optimiser from deleting the measured code (`benchmark::DoNotOptimize`), and reports time per iteration, optionally over a range of input sizes with complexity estimates. Always benchmark optimised builds (`-O2`, Release).

A GOOGLE BENCHMARK COMPARISON C++

```
#include <benchmark/benchmark.h>
#include <map>
#include <unordered_map>

static void BM_MapInsert(benchmark::State& state) {
    for (auto _ : state) {                              // the timed loop
        std::map<int, int> m;
        for (int i = 0; i < state.range(0); ++i) m[i] = i;
        benchmark::DoNotOptimize(m);
    }
    state.SetComplexityN(state.range(0));
}
BENCHMARK(BM_MapInsert)->Range(1 << 10, 1 << 16)->Complexity();

static void BM_UnorderedInsert(benchmark::State& state) {
    for (auto _ : state) {
        std::unordered_map<int, int> m;
        for (int i = 0; i < state.range(0); ++i) m[i] = i;
        benchmark::DoNotOptimize(m);
    }
}
BENCHMARK(BM_UnorderedInsert)->Range(1 << 10, 1 << 16);

BENCHMARK_MAIN();
```

Its report is a table with one row per benchmark and input size, giving the wall-clock **Time** and **CPU** time per iteration and the number of **Iterations** it chose, plus a fitted complexity line (such as *NlgN*) when `Complexity()` is requested. For quick experiments without a library, time a loop with `std::chrono::steady_clock` as in 24.8.3, run it several times and look at the minimum.

24.6 Best practices

24.6.1 C++ Core Guidelines

The **C++ Core Guidelines** (isocpp.github.io/CppCoreGuidelines), edited by Bjarne Stroustrup and Herb Sutter, are the most widely referenced set of rules for modern C++. They are organised in sections (P: philosophy, I: interfaces, F: functions, C: classes, R: resources, ES: expressions, CP: concurrency, T: templates…), each rule with a reason, examples and enforcement notes; clang-tidy implements many of them as `cppcoreguidelines-*` checks. A few of the most important:

Rule	Says
P.1	Express ideas directly in code
I.11	Never transfer ownership by a raw pointer (T*) or reference (T&)
F.16	For "in" parameters, pass cheaply-copied types by value and others by reference to const
C.20	If you can avoid defining default operations, do (Rule of Zero)
C.35	A base class destructor should be either public and virtual, or protected and non-virtual
R.1	Manage resources automatically using resource handles and RAII
R.11	Avoid calling new and delete explicitly

Rule	Says
ES.20	Always initialize an object
ES.46	Avoid lossy (narrowing, truncating) arithmetic conversions
Con.1	By default, make objects immutable
CP.20	Use RAII, never plain lock()/unlock()

24.6.2 RAII everywhere, no raw owning pointers

Every resource (memory, files, locks, sockets, handles) should be owned by an object whose destructor releases it. Then cleanup happens automatically on every path, including early returns and exceptions. In practice: containers instead of `new[]`, `unique_ptr` / `shared_ptr` instead of `new`, `fstream` instead of `FILE*`, `lock_guard` / `scoped_lock` instead of `lock()` / `unlock()`. A raw pointer or reference is fine as a **non-owning** view.

EXCEPTION-SAFE BY CONSTRUCTION

main.cpp

```
#include <fstream>
#include <iostream>
#include <memory>
#include <mutex>
#include <stdexcept>
#include <string>
#include <vector>

struct Report { std::string title; std::vector<double> values; };
std::mutex report_mutex;

// Every resource below is released automatically, even when the exception is thrown.
void save_report(const std::string& path, bool fail) {
    std::scoped_lock lock(report_mutex);                          // lock: RAII
    auto report = std::make_unique<Report>(Report{"Q3", {1.5, 2.5, 4.0}});   // heap object: RAII
    std::ofstream out(path);                                      // file: RAII
    out << report->title << '\n';
    if (fail) throw std::runtime_error("disk quota exceeded");  // no leaks, no stuck mutex
    for (double v : report->values) out << v << '\n';
}

int main() {
    try {
        save_report("q3.txt", true);
    } catch (const std::exception& e) {
        std::cout << "failed: " << e.what() << '\n';
    }
    std::cout << std::boolalpha << "mutex free again: " << report_mutex.try_lock() << '\n';
    report_mutex.unlock();
    save_report("q3.txt", false);
    std::ifstream in("q3.txt");
    std::cout << "file: ";
    for (std::string line; std::getline(in, line); ) std::cout << line << ' ';
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
failed: disk quota exceeded
mutex free again: true
file: Q3 1.5 2.5 4
```

24.6.3 Const correctness

Const correctness means marking everything that should not change as `const`: variables that are never reassigned, reference parameters that are only read, and member functions that do not modify the object. The compiler then enforces it, the code documents itself, and accidental modification becomes a compile error. Use `constexpr` for compile-time constants. Prefer `const` local variables by default (Con.1).

CONST ON PARAMETERS, MEMBERS AND LOCALS

main.cpp

```
#include <iostream>
#include <string>
#include <vector>

class Playlist {
public:
    void add(const std::string& song) { songs_.push_back(song); }      // reads the argument only
    std::size_t size() const { return songs_.size(); }                 // const: does not modify
    const std::string& at(std::size_t i) const { return songs_.at(i); }
    std::string& at(std::size_t i) { return songs_.at(i); }            // non-const overload for
editing
private:
    std::vector<std::string> songs_;
};

void print(const Playlist& p) {                                        // promises not to modify p
    for (std::size_t i = 0; i < p.size(); ++i) std::cout << i + 1 << ". " << p.at(i) << '\n';
}

int main() {
    constexpr int max_songs = 100;                                     // compile-time constant
    Playlist p;
    p.add("Vande Mataram");
    p.add("Jana Gana Mana");
    p.at(1) += " (instrumental)";                                      // non-const object: may edit
    print(p);
    const auto count = p.size();                                       // never reassigned: const
    std::cout << count << " of " << max_songs << " songs\n";
    return 0;
}
```

OUTPUT

```
1. Vande Mataram
2. Jana Gana Mana (instrumental)
2 of 100 songs
```

24.6.4 Prefer the standard library to hand-written code

Standard algorithms and containers are tested, optimised and instantly recognisable to every C++ programmer. A named algorithm states *what* is done (`std::any_of`, `std::partition`, `std::accumulate`) where a raw loop must be read carefully to discover it, and raw loops are where off-by-one errors live. Sean Parent's advice "no raw loops" is a good default.

LOOPS VS NAMED ALGORITHMS

main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <numeric>
#include <string>
#include <vector>

int main() {
    std::vector<int> marks = {72, 35, 91, 58, 84, 40, 67};

    // Hand-written: what does it do? Read every line to find out.
    int count = 0;
    for (std::size_t i = 0; i < marks.size(); ++i)
        if (marks[i] >= 60) count++;

    // Standard library: the names say it.
    auto passed = std::count_if(marks.begin(), marks.end(), [](int m) { return m >= 60; });
    double average = std::accumulate(marks.begin(), marks.end(), 0.0) / marks.size();
    auto [lo, hi] = std::ranges::minmax(marks);                    // values, not iterators
    bool anyone_failed = std::any_of(marks.begin(), marks.end(), [](int m) { return m < 40; });
    std::ranges::sort(marks, std::greater{});

    std::cout << "loop: " << count << ", count_if: " << passed << " passed, average " << average << ",
range " << lo << "-" << hi
              << ", anyone failed: " << std::boolalpha << anyone_failed << ", top three: " << marks[0]
<< ' ' << marks[1] << ' ' << marks[2] << '\n';
    return 0;
}
```

OUTPUT

```
loop: 4, count_if: 4 passed, average 63.8571, range 35-91, anyone failed: true, top three: 91 84 72
```

24.6.5 Avoiding undefined behaviour

Undefined behaviour (UB) means the standard places no requirements on what happens: the program may crash, give wrong results, or appear to work until the compiler or input changes. Optimisers assume UB never happens and transform code accordingly, which can make the consequences surprising. The most common sources and their defences:

Undefined behaviour	Defence
Out-of-bounds access	`at()`, range-for, algorithms, `std::span`; ASan
Use after free, dangling references/views	RAII, smart pointers, value semantics; do not return references to locals; ASan
Null pointer dereference	references or `optional` where null is not meaningful; check pointers
Uninitialised variables	always initialise (`int x{};`); compiler warnings; Valgrind / MSan
Signed integer overflow	wider types, checked arithmetic, `std::numeric_limits`; UBSan
Invalid shifts, division by zero	validate operands; UBSan
Data races	mutexes and atomics; ThreadSanitizer
Modifying a container while iterating	know the invalidation rules; erase-remove; `std::erase_if`
Strict aliasing violations, wrong casts	`std::memcpy` or `std::bit_cast` instead of `reinterpret_cast` tricks

TIP

A practical defence-in-depth routine: compile with `-Wall -Wextra -Wpedantic -Wconversion -Werror`, run the tests under ASan+UBSan and (for threaded code) TSan, run clang-tidy in CI, and use `-D_GLIBCXX_ASSERTIONS` in debug builds to make `operator[]` on standard containers bounds-checked.

24.7 Design patterns in C++

Design patterns are named, reusable solutions to recurring design problems, catalogued in the "Gang of Four" book (1994). In modern C++ some become simpler thanks to lambdas, templates and smart pointers. The examples below show the patterns most often asked about.

24.7.1 Creational: Singleton, Factory, Builder

Singleton ensures a class has exactly one instance with a global access point. The Meyers singleton (a function-local static) is lazily created and thread-safe since C++11. Use it sparingly: it is global state in disguise and makes testing harder. **Factory** creates objects without the caller naming the concrete class, typically returning `unique_ptr<Base>`. **Builder** constructs a complex object step by step with readable, chainable calls.

SINGLETON, FACTORY AND BUILDER — main.cpp

```
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>

// ---- Singleton ----
class Config {
public:
    static Config& instance() { static Config c; return c; }   // created on first use, thread-safe
    std::string currency = "INR";
    Config(const Config&) = delete;
    Config& operator=(const Config&) = delete;
private:
    Config() { std::cout << "  (config loaded once)\n"; }
};

// ---- Factory ----
struct Notifier { virtual ~Notifier() = default; virtual void send(const std::string& msg) const = 0;
};
struct EmailNotifier : Notifier { void send(const std::string& m) const override { std::cout << "
email: " << m << '\n'; } };
struct SmsNotifier   : Notifier { void send(const std::string& m) const override { std::cout << "
sms:   " << m << '\n'; } };
std::unique_ptr<Notifier> make_notifier(const std::string& kind) {
    if (kind == "email") return std::make_unique<EmailNotifier>();
    if (kind == "sms") return std::make_unique<SmsNotifier>();
    throw std::invalid_argument("unknown notifier: " + kind);
}

// ---- Builder ----
class HttpRequest {
public:
    void describe() const { std::cout << "  " << method_ << ' ' << url_ << " [" << headers_ <<
"timeout " << timeout_ << "s]\n"; }
private:
    friend class HttpRequestBuilder;                // only the builder sets the fields
    std::string url_, method_ = "GET", headers_;
    int timeout_ = 30;
```

```
};
class HttpRequestBuilder {
public:
    HttpRequestBuilder& url(std::string u) { r_.url_ = std::move(u); return *this; }
    HttpRequestBuilder& method(std::string m) { r_.method_ = std::move(m); return *this; }
    HttpRequestBuilder& header(const std::string& k, const std::string& v) { r_.headers_ += k + ": " +
v + "; "; return *this; }
    HttpRequestBuilder& timeout(int s) { r_.timeout_ = s; return *this; }
    HttpRequest build() const {
        if (r_.url_.empty()) throw std::logic_error("url is required");
        return r_;
    }
private:
    HttpRequest r_;
};

int main() {
    Config& cfg = Config::instance();                          // first use creates it
    std::cout << "currency: " << cfg.currency << '\n';
    Config::instance().currency = "USD";
    std::cout << "same object everywhere: " << Config::instance().currency << '\n';

    for (const char* kind : {"email", "sms"}) make_notifier(kind)->send("your order has shipped");

    auto req = HttpRequestBuilder().url("https://api.lbf.in/courses").method("POST")
                   .header("Content-Type", "application/json").timeout(10).build();
    req.describe();
    return 0;
}
```

OUTPUT

```
  (config loaded once)
currency: INR
same object everywhere: USD
  email: your order has shipped
  sms:   your order has shipped
  POST https://api.lbf.in/courses [Content-Type: application/json; timeout 10s]
```

24.7.2 Structural: Adapter, Decorator, Facade, PIMPL idiom

Adapter wraps an existing class to give it the interface a client expects. **Decorator** wraps an object to add behaviour while keeping the same interface, and decorators can be stacked. **Facade** provides one simple interface to a complicated subsystem. The **PIMPL** idiom ("pointer to implementation") hides a class's private members behind a pointer to an incomplete type: the header no longer changes when the implementation does, which cuts rebuild times and keeps a library's ABI stable.

ADAPTER, DECORATOR AND FACADE — main.cpp

```
#include <iostream>
#include <memory>
#include <string>

// ---- Adapter: a legacy class with the "wrong" interface ----
class LegacyThermometer { public: double read_fahrenheit() const { return 98.6; } };
struct Thermometer { virtual ~Thermometer() = default; virtual double celsius() const = 0; };
class ThermometerAdapter : public Thermometer {
public:
    explicit ThermometerAdapter(const LegacyThermometer& t) : t_(t) {}
    double celsius() const override { return (t_.read_fahrenheit() - 32) * 5 / 9; }
private:
    const LegacyThermometer& t_;
};
```

```
// ---- Decorator: add behaviour, keep the interface ----
struct DataSource { virtual ~DataSource() = default; virtual std::string read() const = 0; };
struct FileSource : DataSource { std::string read() const override { return "secret data"; } };
class Uppercase : public DataSource {
public:
    explicit Uppercase(std::unique_ptr<DataSource> inner) : inner_(std::move(inner)) {}
    std::string read() const override { std::string s = inner_->read(); for (char& c : s) c =
static_cast<char>(std::toupper(c)); return s; }
private:
    std::unique_ptr<DataSource> inner_;
};
class Bracketed : public DataSource {
public:
    explicit Bracketed(std::unique_ptr<DataSource> inner) : inner_(std::move(inner)) {}
    std::string read() const override { return "[" + inner_->read() + "]"; }
private:
    std::unique_ptr<DataSource> inner_;
};

// ---- Facade: one call hides several subsystems ----
struct Inventory { bool reserve(const std::string& item) { std::cout << "  inventory: reserved " <<
item << '\n'; return true; } };
struct Payment   { bool charge(int amount) { std::cout << "  payment: charged " << amount << '\n';
return true; } };
struct Shipping  { void ship(const std::string& item) { std::cout << "  shipping: dispatched " << item
<< '\n'; } };
class OrderFacade {
public:
    bool place_order(const std::string& item, int amount) {
        if (!inv_.reserve(item) || !pay_.charge(amount)) return false;
        ship_.ship(item);
        return true;
    }
private:
    Inventory inv_; Payment pay_; Shipping ship_;
};

int main() {
    LegacyThermometer old;
    ThermometerAdapter t(old);
    std::cout << "body temperature: " << t.celsius() << " C\n";

    std::unique_ptr<DataSource> src = std::make_unique<Bracketed>(std::make_unique<Uppercase>
(std::make_unique<FileSource>()));
    std::cout << "decorated read: " << src->read() << '\n';

    OrderFacade shop;
    bool ok = shop.place_order("C++ book", 599);
    std::cout << "order placed: " << std::boolalpha << ok << '\n';
    return 0;
}
```

OUTPUT

```
body temperature: 37 C
decorated read: [SECRET DATA]
  inventory: reserved C++ book
  payment: charged 599
  shipping: dispatched C++ book
order placed: true
```

PIMPL: THE HEADER EXPOSES NO PRIVATE DETAILS main.cpp

```
#include "widget.h"
#include <iostream>

int main() {
    Widget w("dashboard");
    w.click();
    w.click();
    std::cout << "sizeof(Widget) = " << sizeof(Widget) << " (just one pointer)\n";
    return 0;
}
```

OUTPUT

```
dashboard clicked 1 time(s)
dashboard clicked 2 time(s)
sizeof(Widget) = 8 (just one pointer)
```

24.7.3 Behavioural: Strategy, Observer, Command, Iterator

Strategy makes an algorithm interchangeable at run time; in modern C++ a `std::function` or a template parameter often replaces the class hierarchy. **Observer** lets objects subscribe to events from a subject. **Command** turns a request into an object, which makes undo/redo, queuing and logging easy. **Iterator** gives sequential access without exposing the structure; in C++ it means providing `begin()` / `end()` so your class works with range-for and the algorithms.

STRATEGY, OBSERVER, COMMAND AND ITERATOR main.cpp

```
#include <functional>
#include <iostream>
#include <memory>
#include <stack>
#include <string>
#include <vector>

// ---- Strategy with std::function ----
class Checkout {
public:
    using Discount = std::function<double(double)>;
    explicit Checkout(Discount d) : discount_(std::move(d)) {}
    double total(double amount) const { return discount_(amount); }
private:
    Discount discount_;
};

// ---- Observer ----
class StockTicker {
public:
    using Listener = std::function<void(const std::string&, double)>;
    void subscribe(Listener l) { listeners_.push_back(std::move(l)); }
    void update(const std::string& sym, double price) { for (auto& l : listeners_) l(sym, price); }
private:
    std::vector<Listener> listeners_;
};

// ---- Command with undo ----
struct Command { virtual ~Command() = default; virtual void execute() = 0; virtual void undo() = 0; };
class AppendText : public Command {
public:
    AppendText(std::string& doc, std::string text) : doc_(doc), text_(std::move(text)) {}
    void execute() override { doc_ += text_; }
```

```
    void undo() override { doc_.erase(doc_.size() - text_.size()); }
private:
    std::string& doc_;
    std::string text_;
};

// ---- Iterator: make a class usable with range-for ----
class Countdown {
public:
    explicit Countdown(int from) : from_(from) {}
    struct It {
        int v;
        int operator*() const { return v; }
        It& operator++() { --v; return *this; }
        bool operator!=(const It& o) const { return v != o.v; }
    };
    It begin() const { return {from_}; }
    It end() const { return {0}; }
private:
    int from_;
};

int main() {
    Checkout regular([](double a) { return a; });
    Checkout festival([](double a) { return a * 0.8; });
    Checkout student([](double a) { return a > 500 ? a - 100 : a; });
    std::cout << "1000 -> " << regular.total(1000) << " / " << festival.total(1000) << " / " <<
student.total(1000) << '\n';

    StockTicker ticker;
    ticker.subscribe([](const std::string& s, double p) { std::cout << "  display: " << s << " = " <<
p << '\n'; });
    ticker.subscribe([](const std::string& s, double p) { if (p > 1500) std::cout << "  alert: " << s
<< " above 1500!\n"; });
    ticker.update("INFY", 1480.5);
    ticker.update("INFY", 1512.0);

    std::string doc;
    std::stack<std::unique_ptr<Command>> history;
    for (const char* t : {"Hello", ", world", "!!!"}) {
        auto cmd = std::make_unique<AppendText>(doc, t);
        cmd->execute();
        history.push(std::move(cmd));
    }
    std::cout << "doc: " << doc << '\n';
    history.top()->undo(); history.pop();
    std::cout << "after undo: " << doc << '\n';

    for (int n : Countdown(5)) std::cout << n << ' ';
    std::cout << "liftoff\n";
    return 0;
}
```

OUTPUT

```
1000 -> 1000 / 800 / 900
  display: INFY = 1480.5
  display: INFY = 1512
  alert: INFY above 1500!
doc: Hello, world!!!
after undo: Hello, world
5 4 3 2 1 liftoff
```

24.7.4 SOLID principles

Principle	Meaning	In C++
Single responsibility	a class should have one reason to change	separate `Account` (rules) from `AccountRepository` (file storage) and `ReportPrinter`
Open/closed	open for extension, closed for modification	add a new `Notifier` subclass or strategy lambda instead of editing a switch
Liskov substitution	a derived class must work wherever its base is expected	do not make `Square` derive from a mutable `Rectangle`; overrides must keep the base contract
Interface segregation	many small interfaces beat one large one	separate `Printable`, `Scannable` instead of one `Machine` with all methods
Dependency inversion	depend on abstractions, not concretions	take `Notifier&` or a template parameter, not `EmailNotifier`; inject it (great for testing with fakes)

SINGLE RESPONSIBILITY AND DEPENDENCY INVERSION

main.cpp

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Storage {                                        // abstraction (D)
    virtual ~Storage() = default;
    virtual void save(const std::string& record) = 0;
};
struct FileStorage : Storage { void save(const std::string& r) override { std::cout << "  file <- " <<
r << '\n'; } };
struct MemoryStorage : Storage {                        // a fake for tests
    std::vector<std::string> saved;
    void save(const std::string& r) override { saved.push_back(r); }
};

class EnrollmentService {                               // one responsibility (S): enrolment rules
public:
    explicit EnrollmentService(Storage& s) : storage_(s) {}   // dependency injected (D)
    bool enroll(const std::string& student, const std::string& course) {
        if (student.empty() || course.empty()) return false;
        storage_.save(student + " -> " + course);
        return true;
    }
private:
    Storage& storage_;
};

int main() {
    FileStorage file;
    EnrollmentService real(file);
    real.enroll("Asha", "C++");

    MemoryStorage fake;                                 // the same service, tested without files
    EnrollmentService tested(fake);
    tested.enroll("Ravi", "Java");
    tested.enroll("", "Python");
    std::cout << "fake storage recorded " << fake.saved.size() << " enrolment: " << fake.saved[0] <<
'\n';
    return 0;
}
```

OUTPUT

```
  file <- Asha -> C++
fake storage recorded 1 enrolment: Ravi -> Java
```

24.8 Performance

First make it correct, then measure, then optimise the part that matters. Most programs spend most of their time in a small fraction of the code, and intuition about where that is is often wrong. Build with optimisation (`-O2` or `-O3`, Release in CMake) before measuring anything.

24.8.1 Profiling (perf, Visual Studio profiler)

A **profiler** shows where a program spends its time. On Linux, **perf** samples the CPU many times per second with very low overhead: `perf record -g ./prog` then `perf report` shows the hottest functions and their call chains (flame graphs visualise the same data). **Valgrind's Callgrind** counts every

instruction exactly (slower but deterministic), and KCachegrind or `callgrind_annotate` display the result. On Windows the **Visual Studio profiler** (Debug > Performance Profiler > CPU Usage) does the same in the IDE; Intel VTune and AMD uProf go deeper into cache and pipeline behaviour.

PROF.CPP C++

```cpp
#include <algorithm>
#include <iostream>
#include <random>
#include <vector>

long fib(int n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }   // exponential: the hot spot

std::vector<int> make_data(int n) {
    std::mt19937 rng(1);
    std::vector<int> v(n);
    for (int& x : v) x = static_cast<int>(rng() % 1000000);
    return v;
}

long checksum(std::vector<int> v) {          // takes a copy on purpose
    std::sort(v.begin(), v.end());
    long s = 0;
    for (std::size_t i = 0; i < v.size(); i += 1000) s += v[i];
    return s;
}

int main() {
    auto data = make_data(200'000);
    std::cout << "checksum " << checksum(data) << '\n';
    std::cout << "fib(30) = " << fib(30) << '\n';
    return 0;
}
```

Finding the hot spots with Callgrind (template names shortened)

```
$ g++ -std=c++20 -O2 -g prof.cpp -o prof
$ ./prof
checksum 99345720
fib(30) = 832040
$ valgrind --tool=callgrind --callgrind-out-file=cg.out ./prof
$ callgrind_annotate --inclusive=no cg.out
--------------------------------------------------------------------------------
Ir                  file:function
--------------------------------------------------------------------------------
20,135,723 (32.16%)  prof.cpp:fib(int)'2
11,118,262 (17.76%)  stl_algo.h:void std::__introsort_loop<...>          (std::sort)
 5,576,157 ( 8.91%)  predefined_ops.h:void std::__introsort_loop<...>     (std::sort)
 4,206,273 ( 6.72%)  random.tcc:make_data(int)
 3,622,940 ( 5.79%)  prof.cpp:fib(int)
 2,982,564 ( 4.76%)  stl_algo.h:checksum(std::vector<int>)
```

Almost 38% of all instructions are in the exponential `fib` (the `'2` marks its recursive calls), and about a quarter in sorting. The profile tells us where an improvement pays off: replacing the recursive fib by a loop removes over a third of the work, while micro-optimising `make_data` could save at most 7%.

24.8.2 Cache-friendly data layout

Main memory is about 100 times slower than the CPU's cache. Data is fetched in **cache lines** of 64 bytes, and hardware prefetchers predict sequential access. So contiguous storage (`vector`, arrays) traversed

in order is dramatically faster than pointer-chasing structures (`list`, maps of nodes) or strided access. For large collections of objects where a loop uses only one field, a **structure of arrays** (SoA) layout loads only the needed data, instead of an array of structures (AoS) that drags the unused fields through the cache.

MEMORY LAYOUT MATTERS (TIMES VARY BY MACHINE)

main.cpp

```
#include <chrono>
#include <iostream>
#include <list>
#include <numeric>
#include <vector>

template <typename F>
long long ms(F f) {
    auto t0 = std::chrono::steady_clock::now();
    f();
    return std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::steady_clock::now() -
t0).count();
}

struct ParticleAoS { double x, y, z, vx, vy, vz, mass, charge; };   // 64 bytes each
struct ParticlesSoA { std::vector<double> x, y, z, vx, vy, vz, mass, charge; };

int main() {
    const int n = 2'000'000;
    std::vector<int> vec(n, 1);
    std::list<int> lst(vec.begin(), vec.end());
    long long s1 = 0, s2 = 0;
    auto t_vec = ms([&] { s1 = std::accumulate(vec.begin(), vec.end(), 0LL); });
    auto t_list = ms([&] { s2 = std::accumulate(lst.begin(), lst.end(), 0LL); });
    std::cout << "sum vector: " << t_vec << " ms, sum list: " << t_list << " ms (sums " << s1 << ", "
<< s2 << ")\n";

    const int rows = 5000, cols = 5000;
    std::vector<int> grid(rows * cols, 1);
    long long r = 0, c = 0;
    auto t_rows = ms([&] { for (int i = 0; i < rows; ++i) for (int j = 0; j < cols; ++j) r += grid[i *
cols + j] * (j & 3); });
    auto t_cols = ms([&] { for (int j = 0; j < cols; ++j) for (int i = 0; i < rows; ++i) c += grid[i *
cols + j] * (j & 3); });
    std::cout << "row-by-row: " << t_rows << " ms, column-by-column: " << t_cols << " ms (sums " << r
<< ", " << c << ")\n";

    std::vector<ParticleAoS> aos(n, ParticleAoS{1, 1, 1, 1, 1, 1, 2, 0});
    ParticlesSoA soa;
    soa.mass.assign(n, 2);
    double m1 = 0, m2 = 0;
    auto t_aos = ms([&] { for (const auto& p : aos) m1 += p.mass; });     // touches 64 bytes per
particle
    auto t_soa = ms([&] { for (double m : soa.mass) m2 += m; });          // touches 8 bytes per
particle
    std::cout << "total mass AoS: " << t_aos << " ms, SoA: " << t_soa << " ms (both " << m1 << ", " <<
m2 << ")\n";
    std::cout << "faster layouts won every time: " << std::boolalpha << (t_vec <= t_list && t_rows <=
t_cols && t_soa <= t_aos) << '\n';
    return 0;
}
```

```
OUTPUT
sum vector: 1 ms, sum list: 10 ms (sums 2000000, 2000000)
row-by-row: 22 ms, column-by-column: 160 ms (sums 37500000, 37500000)
total mass AoS: 8 ms, SoA: 2 ms (both 4e+06, 4e+06)
faster layouts won every time: true
```

24.8.3 Avoiding unnecessary copies

Copies of strings, vectors and other heap-owning objects allocate and copy memory. Common sources and fixes: pass large read-only arguments by `const&` (or `string_view` / `span`); iterate with `const auto&`; `reserve()` before filling a vector; use `emplace_back` and `std::move` when handing over objects you no longer need; return by value and rely on RVO; and avoid `std::endl` in loops (it flushes).

COPIES COST TIME main.cpp

```cpp
#include <chrono>
#include <iostream>
#include <string>
#include <vector>

long long count_by_value(std::vector<std::string> words) { long long n = 0; for (std::string w :
words) n += w.size(); return n; }
long long count_by_ref(const std::vector<std::string>& words) { long long n = 0; for (const auto& w :
words) n += w.size(); return n; }

template <typename F>
double us(F f) {
    auto t0 = std::chrono::steady_clock::now();
    f();
    return std::chrono::duration<double, std::micro>(std::chrono::steady_clock::now() - t0).count();
}

int main() {
    std::vector<std::string> words(200'000, std::string(40, 'x'));   // long strings: heap allocated
    long long a = 0, b = 0;
    double t_copy = us([&] { a = count_by_value(words); });
    double t_ref = us([&] { b = count_by_ref(words); });
    std::cout << "same answer: " << std::boolalpha << (a == b) << ", by-value version slower: " <<
(t_copy > t_ref)
              << ", ratio about " << static_cast<int>(t_copy / t_ref + 0.5) << "x\n";

    std::vector<std::string> no_reserve, with_reserve;
    double t1 = us([&] { for (int i = 0; i < 200'000; ++i) no_reserve.push_back("item"); });
    double t2 = us([&] { with_reserve.reserve(200'000); for (int i = 0; i < 200'000; ++i)
with_reserve.emplace_back("item"); });
    std::cout << "reserve + emplace_back faster: " << (t2 < t1) << '\n';
    return 0;
}
```

```
OUTPUT
same answer: true, by-value version slower: true, ratio about 37x
reserve + emplace_back faster: true
```

Split code into headers and sources with include guards, respect the ODR, and package code as static or shared libraries. Build with CMake (Make underneath or Ninja), manage dependencies with vcpkg or Conan, debug with GDB or an IDE, and keep code clean with sanitizers, Valgrind, clang-tidy and clang-format. Write unit tests (GoogleTest, Catch2), consider TDD, and benchmark before optimising. Follow the Core Guidelines: RAII, no owning raw pointers, const by default, standard algorithms, no undefined behaviour. Know the classic patterns and SOLID, profile to find hot spots, and design data for the cache.

PRACTICE CHECKPOINT

Organise a project into a **library and an app with CMake**, add **GoogleTest unit tests**, and run it **clean under AddressSanitizer**.

This checkpoint is the project built throughout the module. Steps and the commands that verify each one:

1. **Library and app.** Put the classes in `include/bank/` and `src/`, the program in `app/main.cpp`, and describe them in `CMakeLists.txt` with `add_library(bank ...)`, `target_include_directories(bank PUBLIC include)`, `add_executable(bank_app ...)` and `target_link_libraries(bank_app PRIVATE bank)` (24.2.2).
2. **Unit tests.** Add `tests/test_account.cpp` (24.5.1) and the `BUILD_TESTS` block that fetches GoogleTest and registers the tests with `gtest_discover_tests`. Build with `-DBUILD_TESTS=ON` and run `ctest --test-dir build --output-on-failure`.
3. **Sanitizers.** Configure a second build directory with `-DSANITIZE=ON` and run the application (and the tests) there; any memory error or undefined behaviour stops the run with a report.

The real project: normal build, then a clean AddressSanitizer + UBSan run

```
$ cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug
-- The CXX compiler identification is GNU 13.3.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done (0.2s)
-- Generating done (0.0s)
-- Build files have been written to: /home/learner/bank/build
$ cmake --build build
[1/5] Building CXX object CMakeFiles/bank.dir/src/account.cpp.o
[2/5] Building CXX object CMakeFiles/bank.dir/src/format.cpp.o
[3/5] Building CXX object CMakeFiles/bank_app.dir/app/main.cpp.o
[4/5] Linking CXX static library libbank.a
[5/5] Linking CXX executable bank_app
$ ./build/bank_app
withdrawal of Rs 100000.00 declined
Asha has Rs 6050.50 after 3 transactions
$ cmake -S . -B build-asan -G Ninja -DSANITIZE=ON
-- Build files have been written to: /home/learner/bank/build-asan
$ cmake --build build-asan && ./build-asan/bank_app
[5/5] Linking CXX executable bank_app
withdrawal of Rs 100000.00 declined
Asha has Rs 6050.50 after 3 transactions
$ ldd build-asan/bank_app | grep -E "asan|ubsan"
        libasan.so.8 => /lib/x86_64-linux-gnu/libasan.so.8
        libubsan.so.1 => /lib/x86_64-linux-gnu/libubsan.so.1
```

Running the test suite through ctest

```
$ cmake -S . -B build -DBUILD_TESTS=ON && cmake --build build
$ ctest --test-dir build --output-on-failure
Test project /home/learner/bank/build
    Start 1: AccountTest.StartsWithOpeningBalance
1/5 Test #1: AccountTest.StartsWithOpeningBalance .....   Passed    0.00 sec
    Start 2: AccountTest.DepositIncreasesBalance
2/5 Test #2: AccountTest.DepositIncreasesBalance .....   Passed    0.00 sec
    Start 3: AccountTest.WithdrawMoreThanBalanceIsDeclined
3/5 Test #3: AccountTest.WithdrawMoreThanBalanceIsDeclined ...   Passed    0.00 sec
    Start 4: AccountTest.InvalidAmountsThrow
4/5 Test #4: AccountTest.InvalidAmountsThrow .........   Passed    0.00 sec
    Start 5: FormatTest.FormatsPaise
5/5 Test #5: FormatTest.FormatsPaise .................   Passed    0.00 sec

100% tests passed, 0 tests failed out of 5

Total Test time (real) =   0.02 sec
```

(This ctest run used the same five tests, compiled against the stand-in macros mentioned in 24.5.1 and registered under the names that `gtest_discover_tests` gives them.) The application ran under AddressSanitizer and UndefinedBehaviorSanitizer with no reports, as the `ldd` line confirms the sanitizer runtimes were linked in. Commit the project to Git with a `.gitignore` containing `build*/`, so build directories are never committed.

MODULE 25 · ADVANCED

Data structures, competitive programming, projects and interviews

Minimum duration **8 days** | Level **Advanced**

The final module applies everything to the problems that matter for placements and real work. We implement the classic data structures from scratch (so the STL holds no mysteries), study the core algorithmic techniques with complete C++ solutions, set up a competitive-programming workflow, plan portfolio-quality capstone projects, and prepare for C++ interviews.

IN THIS MODULE

25.1 Data structures from scratch
25.2 Algorithms
25.3 Competitive programming with C++
25.4 Capstone projects
25.5 Interview preparation
25.6 Next steps

25.1 Data structures from scratch

In real code you use the STL. Implementing the structures yourself once teaches how they work, why their operations cost what they cost, and how to handle pointers, ownership and edge cases: exactly what interviewers probe. The implementations below use templates, RAII and smart pointers where they fit.

25.1.1 Linked lists (singly, doubly, circular)

A **singly linked list** is a chain of nodes, each holding a value and a pointer to the next node. Insertion at the front is O(1); finding or reaching position k is O(k). Here ownership is expressed with `unique_ptr`: each node owns the next, so the whole list is freed automatically. The classic interview operations (reverse, find the middle, detect a cycle) follow.

A SINGLY LINKED LIST: HEAD POINTS TO THE FIRST NODE, THE LAST POINTS TO NULL

A SINGLY LINKED LIST WITH UNIQUE_PTR

main.cpp

```
#include <iostream>
#include <memory>

template <typename T>
class SinglyList {
    struct Node {
        T value;
        std::unique_ptr<Node> next;                    // each node owns the next one
    };
    std::unique_ptr<Node> head_;
    std::size_t size_ = 0;
public:
```

```
    void push_front(T v) {
        head_ = std::make_unique<Node>(Node{std::move(v), std::move(head_)});
        ++size_;
    }
    void push_back(T v) {                                   // O(n): walk to the end
        auto* slot = &head_;
        while (*slot) slot = &(*slot)->next;
        *slot = std::make_unique<Node>(Node{std::move(v), nullptr});
        ++size_;
    }
    bool remove(const T& v) {                               // unlink the first match
        for (auto* slot = &head_; *slot; slot = &(*slot)->next)
            if ((*slot)->value == v) { *slot = std::move((*slot)->next); --size_; return true; }
        return false;
    }
    void reverse() {                                        // classic: re-point every link
        std::unique_ptr<Node> prev;
        while (head_) {
            auto next = std::move(head_->next);
            head_->next = std::move(prev);
            prev = std::move(head_);
            head_ = std::move(next);
        }
        head_ = std::move(prev);
    }
    const T& middle() const {                               // slow/fast pointers
        const Node* slow = head_.get();
        const Node* fast = head_.get();
        while (fast && fast->next) { slow = slow->next.get(); fast = fast->next->next.get(); }
        return slow->value;
    }
    void print() const {
        for (const Node* n = head_.get(); n; n = n->next.get()) std::cout << n->value << " -> ";
        std::cout << "null (size " << size_ << ")\n";
    }
};

int main() {
    SinglyList<int> list;
    for (int v : {30, 20, 10}) list.push_front(v);
    list.push_back(40);
    list.push_back(50);
    list.print();
    std::cout << "middle: " << list.middle() << '\n';
    list.remove(30);
    list.print();
    list.reverse();
    list.print();
    return 0;
}
```

OUTPUT

```
10 -> 20 -> 30 -> 40 -> 50 -> null (size 5)
middle: 30
10 -> 20 -> 40 -> 50 -> null (size 4)
50 -> 40 -> 20 -> 10 -> null (size 4)
```

NOTE

A very long list built from `unique_ptr`s is destroyed recursively (each node's destructor destroys the next), which can overflow the stack for millions of nodes. Production code adds a destructor that unlinks nodes in a loop: `while (head_) head_ = std::move(head_->next);`.

A **doubly linked list** adds a `prev` pointer, so a node can be removed in O(1) given only a pointer to it, and the list can be walked backwards (this is `std::list`). Because two pointers point to each node, ownership is usually expressed with raw pointers managed by the list class. In a **circular** list the last node points back to the first: useful for round-robin scheduling and ring buffers. Floyd's "tortoise and hare" detects whether a list has a cycle.

DOUBLY LINKED LIST WITH A SENTINEL, AND CYCLE DETECTION main.cpp

```
#include <iostream>
#include <string>

template <typename T>
class DoublyList {                                  // with a sentinel node: no special cases at
the ends
    struct Node { T value{}; Node* prev = this; Node* next = this; };
    Node sentinel_;                                 // sentinel_.next = first, sentinel_.prev =
last
    std::size_t size_ = 0;
    Node* insert_before(Node* pos, T v) {
        Node* n = new Node{std::move(v), pos->prev, pos};
        pos->prev->next = n;
        pos->prev = n;
        ++size_;
        return n;
    }
public:
    DoublyList() = default;
    DoublyList(const DoublyList&) = delete;
    DoublyList& operator=(const DoublyList&) = delete;
    ~DoublyList() { while (size_) erase(sentinel_.next); }
    Node* push_back(T v)  { return insert_before(&sentinel_, std::move(v)); }
    Node* push_front(T v) { return insert_before(sentinel_.next, std::move(v)); }
    void erase(Node* n) {                           // O(1): just relink the neighbours
        n->prev->next = n->next;
        n->next->prev = n->prev;
        delete n;
        --size_;
    }
    void print_forward() const {
        for (Node* n = sentinel_.next; n != &sentinel_; n = n->next) std::cout << n->value << ' ';
        std::cout << '\n';
    }
    void print_backward() const {
        for (Node* n = sentinel_.prev; n != &sentinel_; n = n->prev) std::cout << n->value << ' ';
        std::cout << '\n';
    }
};

struct CNode { int v; CNode* next; };
bool has_cycle(CNode* head) {                       // Floyd: fast pointer catches up inside a
cycle
    CNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

int main() {
    DoublyList<std::string> tabs;
    tabs.push_back("mail");
```

```
    auto* docs = tabs.push_back("docs");
    tabs.push_back("music");
    tabs.push_front("home");
    tabs.print_forward();
    tabs.erase(docs);                                   // close a tab in O(1)
    tabs.print_backward();

    CNode d{4, nullptr}, c{3, &d}, b{2, &c}, a{1, &b};
    std::cout << std::boolalpha << "cycle? " << has_cycle(&a);
    d.next = &b;                                        // make it circular from 4 back to 2
    std::cout << " -> after linking 4 back to 2: " << has_cycle(&a) << '\n';
    CNode* p = &a;
    std::cout << "walking the circular part: ";
    for (int i = 0; i < 7; ++i, p = p->next) std::cout << p->v << ' ';
    std::cout << "...\n";
    return 0;
}
```

OUTPUT

```
home mail docs music
music mail home
cycle? false -> after linking 4 back to 2: true
walking the circular part: 1 2 3 4 2 3 4 ...
```

25.1.2 Stack and queue

A **stack** (LIFO) is easy on top of a dynamic array: push and pop at the end. A **queue** (FIFO) on an array needs a **circular buffer**: head and tail indexes wrap around with `% capacity`, so neither push nor pop moves elements, and the buffer doubles when full.

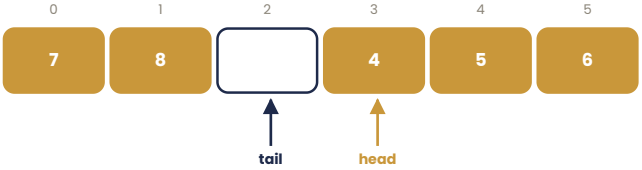
AN ARRAY STACK AND A CIRCULAR-BUFFER QUEUE — main.cpp

```
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>

template <typename T>
class ArrayStack {
    std::unique_ptr<T[]> data_ = std::make_unique<T[]>(2);
    std::size_t size_ = 0, cap_ = 2;
public:
    void push(T v) {
        if (size_ == cap_) {                            // grow by doubling: amortised O(1)
            auto bigger = std::make_unique<T[]>(cap_ * 2);
            for (std::size_t i = 0; i < size_; ++i) bigger[i] = std::move(data_[i]);
            data_ = std::move(bigger);
            cap_ *= 2;
        }
        data_[size_++] = std::move(v);
    }
    T pop() {
        if (size_ == 0) throw std::underflow_error("stack empty");
        return std::move(data_[--size_]);
    }
    bool empty() const { return size_ == 0; }
};

template <typename T>
class RingQueue {
    std::unique_ptr<T[]> data_;
    std::size_t head_ = 0, count_ = 0, cap_;
```

```
public:
    explicit RingQueue(std::size_t cap = 4) : data_(std::make_unique<T[]>(cap)), cap_(cap) {}
    void push(T v) {
        if (count_ == cap_) {                                   // full: copy into a buffer twice as
big, in order
            auto bigger = std::make_unique<T[]>(cap_ * 2);
            for (std::size_t i = 0; i < count_; ++i) bigger[i] = std::move(data_[(head_ + i) % cap_]);
            data_ = std::move(bigger);
            head_ = 0;
            cap_ *= 2;
        }
        data_[(head_ + count_) % cap_] = std::move(v);        // tail position wraps around
        ++count_;
    }
    T pop() {
        if (count_ == 0) throw std::underflow_error("queue empty");
        T v = std::move(data_[head_]);
        head_ = (head_ + 1) % cap_;
        --count_;
        return v;
    }
    std::size_t size() const { return count_; }
    std::size_t capacity() const { return cap_; }
};

int evaluate_postfix(const std::string& expr) {                 // a stack application: "3 4 + 2 *"
    ArrayStack<int> st;
    for (char c : expr) {
        if (c >= '0' && c <= '9') st.push(c - '0');
        else if (c == '+' || c == '-' || c == '*') {
            int b = st.pop(), a = st.pop();
            st.push(c == '+' ? a + b : c == '-' ? a - b : a * b);
        }
    }
    return st.pop();
}

int main() {
    std::cout << "postfix 3 4 + 2 * = " << evaluate_postfix("3 4 + 2 *") << '\n';
    std::cout << "postfix 5 1 2 + 4 * + 3 - = " << evaluate_postfix("5 1 2 + 4 * + 3 -") << '\n';
    RingQueue<int> q(4);
    for (int i = 1; i <= 3; ++i) q.push(i);
    std::cout << "pop " << q.pop() << ", pop " << q.pop() << '\n';
    for (int i = 4; i <= 8; ++i) q.push(i);                    // wraps around, then grows
    std::cout << "size " << q.size() << ", capacity " << q.capacity() << ", contents:";
    while (q.size()) std::cout << ' ' << q.pop();
    std::cout << '\n';
    return 0;
}
```

OUTPUT

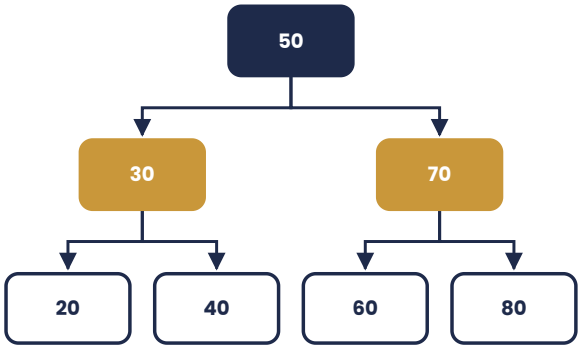
```
postfix 3 4 + 2 * = 14
postfix 5 1 2 + 4 * + 3 - = 14
pop 1, pop 2
size 6, capacity 8, contents: 3 4 5 6 7 8
```



A RING BUFFER: THE QUEUE RUNS FROM HEAD (4) AROUND TO TAIL, WRAPPING PAST THE END

25.1.3 Binary tree and binary search tree

A **binary tree** node has up to two children. Traversals visit every node: **pre-order** (node, left, right), **in-order** (left, node, right), **post-order** (left, right, node), and **level-order** (breadth-first with a queue). A **binary search tree** (BST) keeps the invariant: everything in the left subtree is smaller than the node, everything in the right subtree larger. Search, insert and delete then take O(h), where h is the height: O(log n) when balanced, O(n) in the worst case (sorted insertions), which is why `std::map` uses a self-balancing red-black tree.



THE BST BUILT BY THE PROGRAM BELOW

A BINARY SEARCH TREE WITH ALL FOUR TRAVERSALS

main.cpp

```
#include <algorithm>
#include <iostream>
#include <memory>
#include <queue>

class BST {
    struct Node {
        int key;
        std::unique_ptr<Node> left, right;
    };
    std::unique_ptr<Node> root_;

    static void insert(std::unique_ptr<Node>& n, int key) {
        if (!n) n = std::make_unique<Node>(Node{key, nullptr, nullptr});
        else if (key < n->key) insert(n->left, key);
        else if (key > n->key) insert(n->right, key);          // duplicates ignored
    }
    static void erase(std::unique_ptr<Node>& n, int key) {
        if (!n) return;
        if (key < n->key) erase(n->left, key);
        else if (key > n->key) erase(n->right, key);
        else if (!n->left) n = std::move(n->right);            // 0 or 1 child: splice it out
        else if (!n->right) n = std::move(n->left);
        else {                                                  // 2 children: copy the in-order
successor
            Node* succ = n->right.get();
            while (succ->left) succ = succ->left.get();
            n->key = succ->key;
            erase(n->right, succ->key);
        }
    }
    static void inorder(const Node* n) { if (!n) return; inorder(n->left.get()); std::cout << n->key
<< ' '; inorder(n->right.get()); }
    static void preorder(const Node* n) { if (!n) return; std::cout << n->key << ' '; preorder(n-
>left.get()); preorder(n->right.get()); }
    static void postorder(const Node* n) { if (!n) return; postorder(n->left.get()); postorder(n-
>right.get()); std::cout << n->key << ' '; }
```

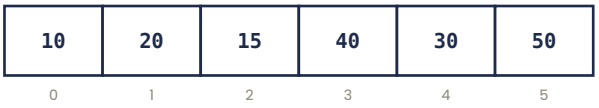
```
    static int height(const Node* n) { return n ? 1 + std::max(height(n->left.get()), height(n-
>right.get())) : 0; }
public:
    void insert(int key) { insert(root_, key); }
    void erase(int key) { erase(root_, key); }
    bool contains(int key) const {
        for (const Node* n = root_.get(); n; n = key < n->key ? n->left.get() : n->right.get())
            if (n->key == key) return true;
        return false;
    }
    void print() const {
        std::cout << "  in-order:    "; inorder(root_.get());
        std::cout << "\n  pre-order:   "; preorder(root_.get());
        std::cout << "\n  post-order:  "; postorder(root_.get());
        std::cout << "\n  level-order: ";
        std::queue<const Node*> q;
        if (root_) q.push(root_.get());
        while (!q.empty()) {
            const Node* n = q.front(); q.pop();
            std::cout << n->key << ' ';
            if (n->left) q.push(n->left.get());
            if (n->right) q.push(n->right.get());
        }
        std::cout << "\n  height: " << height(root_.get()) << '\n';
    }
};

int main() {
    BST t;
    for (int k : {50, 30, 70, 20, 40, 60, 80}) t.insert(k);
    t.print();
    std::cout << std::boolalpha << "contains 60? " << t.contains(60) << ", 65? " << t.contains(65) <<
'\n';
    t.erase(50);                                    // root with two children: replaced by 60
    std::cout << "after erasing 50:\n";
    t.print();
    BST skewed;
    for (int k = 1; k <= 7; ++k) skewed.insert(k);   // sorted input: degenerates into a list
    std::cout << "sorted insertions:\n";
    skewed.print();
    return 0;
}
```

OUTPUT

```
  in-order:    20 30 40 50 60 70 80
  pre-order:   50 30 20 40 70 60 80
  post-order:  20 40 30 60 80 70 50
  level-order: 50 30 70 20 40 60 80
  height: 3
contains 60? true, 65? false
after erasing 50:
  in-order:    20 30 40 60 70 80
  pre-order:   60 30 20 40 70 80
  post-order:  20 40 30 80 70 60
  level-order: 60 30 70 20 40 80
  height: 3
sorted insertions:
  in-order:    1 2 3 4 5 6 7
  pre-order:   1 2 3 4 5 6 7
  post-order:  7 6 5 4 3 2 1
  level-order: 1 2 3 4 5 6 7
  height: 7
```

25.1.4 Heap

A binary **heap** is a complete binary tree stored in an array: the children of index i are at 2i+1 and 2i+2, the parent at (i-1)/2. In a min-heap every parent is ≤ its children, so the minimum is at index 0. **Push** appends and "sifts up" (swaps with the parent while smaller); **pop** moves the last element to the root and "sifts down". Both are O(log n), building a heap from n elements is O(n), and repeated pops give heapsort.



THE HEAP BUILT BELOW (10 20 15 40 30 50) STORED IN AN ARRAY: CHILDREN OF I ARE 2I+1 AND 2I+2

A MIN-HEAP FROM SCRATCH

main.cpp

```
#include <iostream>
#include <stdexcept>
#include <utility>
#include <vector>

class MinHeap {
    std::vector<int> a_;
    void sift_up(std::size_t i) {
        while (i > 0) {
            std::size_t parent = (i - 1) / 2;
            if (a_[parent] <= a_[i]) break;
            std::swap(a_[parent], a_[i]);
            i = parent;
        }
    }
    void sift_down(std::size_t i) {
        while (true) {
            std::size_t l = 2 * i + 1, r = l + 1, smallest = i;
            if (l < a_.size() && a_[l] < a_[smallest]) smallest = l;
            if (r < a_.size() && a_[r] < a_[smallest]) smallest = r;
            if (smallest == i) return;
            std::swap(a_[i], a_[smallest]);
            i = smallest;
        }
    }
public:
    MinHeap() = default;
    explicit MinHeap(std::vector<int> v) : a_(std::move(v)) {        // heapify in O(n)
        for (std::size_t i = a_.size() / 2; i-- > 0; ) sift_down(i);
    }
    void push(int v) { a_.push_back(v); sift_up(a_.size() - 1); }
    int pop() {
        if (a_.empty()) throw std::underflow_error("heap empty");
        int top = a_[0];
        a_[0] = a_.back();
        a_.pop_back();
        if (!a_.empty()) sift_down(0);
        return top;
    }
    bool empty() const { return a_.empty(); }
    void show() const { for (int x : a_) std::cout << x << ' '; std::cout << '\n'; }
};

int main() {
    MinHeap h;
    for (int x : {40, 20, 50, 10, 30, 15}) h.push(x);
    std::cout << "heap array: "; h.show();
    std::cout << "pops in order:";
    while (!h.empty()) std::cout << ' ' << h.pop();
```

```
    std::cout << '\n';

    MinHeap built({9, 4, 7, 1, 8, 2, 6});                          // heapify
    std::cout << "heapified:  "; built.show();
    std::cout << "heapsort:  ";
    while (!built.empty()) std::cout << ' ' << built.pop();
    std::cout << '\n';
    return 0;
}
```

OUTPUT

```
heap array: 10 20 15 40 30 50
pops in order: 10 15 20 30 40 50
heapified:  1 4 2 9 8 7 6
heapsort:   1 2 4 6 7 8 9
```

25.1.5 Hash table

A **hash table** stores keys in an array of buckets chosen by `hash(key) % buckets`. Collisions can be handled by **separate chaining** (each bucket holds a list, as in `std::unordered_map`) or **open addressing** (probe the next slots: linear, quadratic or double hashing). When the load factor passes a threshold the table **rehashes** into a larger array. Average O(1) operations depend on a good hash function that spreads keys evenly.

A CHAINED HASH MAP WITH REHASHING — main.cpp

```
#include <functional>
#include <iostream>
#include <list>
#include <optional>
#include <string>
#include <utility>
#include <vector>

template <typename K, typename V, typename Hash = std::hash<K>>
class HashMap {
    std::vector<std::list<std::pair<K, V>>> buckets_;
    std::size_t size_ = 0;
    Hash hash_;
    std::size_t index(const K& k) const { return hash_(k) % buckets_.size(); }
    void rehash(std::size_t n) {
        std::vector<std::list<std::pair<K, V>>> old(n);
        old.swap(buckets_);
        for (auto& bucket : old)
            for (auto& kv : bucket) buckets_[index(kv.first)].push_back(std::move(kv));
    }
public:
    HashMap() : buckets_(4) {}
    void put(const K& k, V v) {
        for (auto& [key, val] : buckets_[index(k)])
            if (key == k) { val = std::move(v); return; }              // update existing key
        if (size_ + 1 > buckets_.size()) rehash(buckets_.size() * 2);  // keep load factor <= 1
        buckets_[index(k)].emplace_back(k, std::move(v));
        ++size_;
    }
    std::optional<V> get(const K& k) const {
        for (const auto& [key, val] : buckets_[index(k)])
            if (key == k) return val;
        return std::nullopt;
    }
    bool erase(const K& k) {
        auto& b = buckets_[index(k)];
```

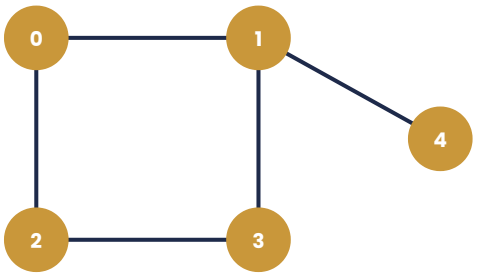
```
        for (auto it = b.begin(); it != b.end(); ++it)
            if (it->first == k) { b.erase(it); --size_; return true; }
        return false;
    }
    std::size_t size() const { return size_; }
    std::size_t bucket_count() const { return buckets_.size(); }
    std::size_t longest_chain() const {
        std::size_t m = 0;
        for (const auto& b : buckets_) m = std::max(m, b.size());
        return m;
    }
};

int main() {
    HashMap<std::string, int> pin;
    const char* cities[] = {"Mysuru", "Bengaluru", "Mangaluru", "Hubballi", "Belagavi", "Udupi",
"Tumakuru"};
    int codes[] = {570001, 560001, 575001, 580020, 590001, 576101, 572101};
    for (int i = 0; i < 7; ++i) pin.put(cities[i], codes[i]);
    std::cout << "size " << pin.size() << ", buckets " << pin.bucket_count() << ", longest chain " <<
pin.longest_chain() << '\n';
    std::cout << "Udupi -> " << pin.get("Udupi").value_or(-1) << ", Delhi -> " <<
pin.get("Delhi").value_or(-1) << '\n';
    pin.put("Udupi", 576102);                                  // update
    pin.erase("Hubballi");
    std::cout << "Udupi now " << *pin.get("Udupi") << ", Hubballi erased: " << std::boolalpha <<
!pin.get("Hubballi") << ", size " << pin.size() << '\n';
    return 0;
}
```

OUTPUT

```
size 7, buckets 8, longest chain 3
Udupi -> 576101, Delhi -> -1
Udupi now 576102, Hubballi erased: true, size 6
```

25.1.6 Graph (adjacency list and matrix)

A **graph** has vertices and edges, directed or undirected, weighted or not. An **adjacency matrix** ($V \times V$) answers "is there an edge u-v?" in $O(1)$ but uses $O(V^2)$ memory: good for small dense graphs. An **adjacency list** stores, for each vertex, the list of its neighbours: $O(V + E)$ memory and fast iteration over neighbours, the right choice for most real (sparse) graphs. In C++ it is typically `std::vector<std::vector<int>>`, or pairs (neighbour, weight).



adjacency list

```
0: 1 2
1: 0 3 4
2: 0 3
3: 1 2
4: 1
```

AN UNDIRECTED GRAPH WITH 5 VERTICES AND ITS ADJACENCY LIST

BOTH REPRESENTATIONS

main.cpp

```cpp
#include <iostream>
#include <utility>
#include <vector>

int main() {
    const int V = 5;
    std::vector<std::pair<int, int>> edges = {{0, 1}, {0, 2}, {1, 3}, {2, 3}, {1, 4}};

    std::vector<std::vector<int>> matrix(V, std::vector<int>(V, 0));    // adjacency matrix
    std::vector<std::vector<int>> adj(V);                               // adjacency list
    for (auto [u, v] : edges) {
        matrix[u][v] = matrix[v][u] = 1;                                // undirected: both directions
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    std::cout << "matrix:\n";
    for (const auto& row : matrix) {
        std::cout << "  ";
        for (int x : row) std::cout << x << ' ';
        std::cout << '\n';
    }
    std::cout << "list:\n";
    for (int u = 0; u < V; ++u) {
        std::cout << "  " << u << ':';
        for (int v : adj[u]) std::cout << ' ' << v;
        std::cout << "  (degree " << adj[u].size() << ")\n";
    }
    std::cout << std::boolalpha << "edge 1-4? " << (matrix[1][4] == 1) << ", edge 0-3? " << (matrix[0]
[3] == 1) << '\n';

    std::vector<std::vector<std::pair<int, int>>> weighted(4);          // directed, weighted: (to,
km)
    weighted[0] = {{1, 145}, {2, 350}};
    weighted[1] = {{2, 190}};
    weighted[2] = {{3, 60}};
    std::cout << "roads from city 0:";
    for (auto [to, km] : weighted[0]) std::cout << " ->" << to << " (" << km << " km)";
    std::cout << '\n';
    return 0;
}
```

OUTPUT

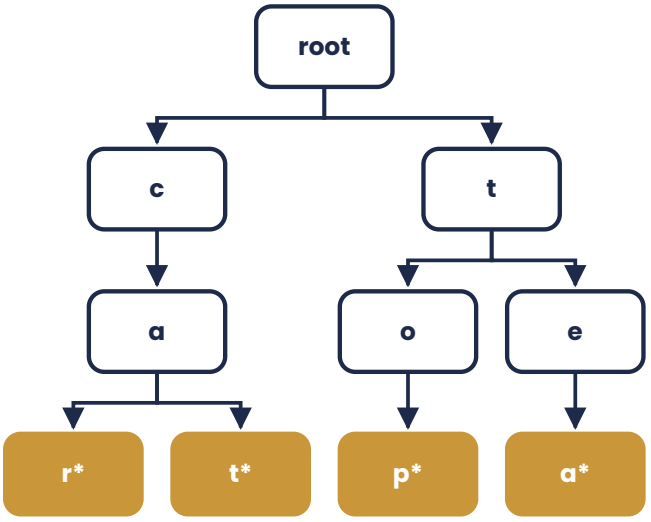
```
matrix:
  0 1 1 0 0
  1 0 0 1 1
  1 0 0 1 0
  0 1 1 0 0
  0 1 0 0 0
list:
  0: 1 2  (degree 2)
  1: 0 3 4  (degree 3)
  2: 0 3  (degree 2)
  3: 1 2  (degree 2)
  4: 1  (degree 1)
edge 1-4? true, edge 0-3? false
roads from city 0: ->1 (145 km) ->2 (350 km)
```

	Adjacency matrix	Adjacency list
Memory	$O(V^2)$	$O(V + E)$

	Adjacency matrix	Adjacency list
Is u-v an edge?	O(1)	O(degree(u))
Iterate neighbours of u	O(V)	O(degree(u))
Add an edge	O(1)	O(1)
Best for	small, dense graphs; Floyd-Warshall	sparse graphs; BFS, DFS, Dijkstra

25.1.7 Trie

A **trie** (prefix tree) stores strings character by character: each node has up to 26 children (for lowercase letters) and a flag marking the end of a word. Insert and search take O(length of the word), independent of how many words are stored, and all words with a given prefix lie in one subtree. Tries power autocomplete, spell-checkers and IP routing tables.



A TRIE HOLDING CAR, CAT, TOP AND TEA (* MARKS THE END OF A WORD)

A TRIE WITH PREFIX COUNTING AND AUTOCOMPLETE

main.cpp

```
#include <array>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Trie {
    struct Node {
        std::array<std::unique_ptr<Node>, 26> next{};
        bool end = false;
        int pass = 0;                                   // how many words go through this node
    };
    Node root_;
    const Node* find(const std::string& s) const {
        const Node* n = &root_;
        for (char c : s) {
            n = n->next[c - 'a'].get();
            if (!n) return nullptr;
        }
        return n;
    }
    static void collect(const Node* n, std::string& prefix, std::vector<std::string>& out) {
        if (n->end) out.push_back(prefix);
```

```
        for (int i = 0; i < 26; ++i)
            if (n->next[i]) {
                prefix.push_back(static_cast<char>('a' + i));
                collect(n->next[i].get(), prefix, out);
                prefix.pop_back();
            }
    }
public:
    void insert(const std::string& word) {
        Node* n = &root_;
        for (char c : word) {
            auto& child = n->next[c - 'a'];
            if (!child) child = std::make_unique<Node>();
            n = child.get();
            ++n->pass;
        }
        n->end = true;
    }
    bool contains(const std::string& w) const { const Node* n = find(w); return n && n->end; }
    int count_prefix(const std::string& p) const { const Node* n = find(p); return n ? n->pass : 0; }
    std::vector<std::string> complete(std::string prefix) const {
        std::vector<std::string> out;
        if (const Node* n = find(prefix)) collect(n, prefix, out);
        return out;
    }
};

int main() {
    Trie t;
    for (const char* w : {"car", "cat", "cart", "care", "top", "tea", "team", "ten"}) t.insert(w);
    std::cout << std::boolalpha << "contains car: " << t.contains("car") << ", ca: " <<
t.contains("ca") << '\n';
    std::cout << "words starting with 'car': " << t.count_prefix("car") << '\n';
    std::cout << "autocomplete 'te':";
    for (const auto& w : t.complete("te")) std::cout << ' ' << w;
    std::cout << "\nautocomplete 'ca':";
    for (const auto& w : t.complete("ca")) std::cout << ' ' << w;
    std::cout << '\n';
    return 0;
}
```

```
OUTPUT
contains car: true, ca: false
words starting with 'car': 3
autocomplete 'te': tea team ten
autocomplete 'ca': car care cart cat
```

25.2 Algorithms

25.2.1 Complexity analysis (Big-O)

Big-O describes how the running time (or memory) of an algorithm grows with the input size n, ignoring constant factors and smaller terms: an algorithm doing $3n^2 + 5n + 2$ steps is $O(n^2)$. To analyse code: a simple statement is $O(1)$; a loop multiplies its body's cost by the number of iterations; nested loops multiply; consecutive blocks add (keep the largest); halving the problem each step gives log n; recursion is analysed with a recurrence (merge sort: $T(n) = 2T(n/2) + n = O(n \log n)$). In competitive programming, a judge does roughly 10^8 simple operations per second, which tells you which complexity your solution needs.

Complexity	Name	Example	Largest n in ~1 second
O(1)	constant	array access, hash lookup	any
O(log n)	logarithmic	binary search, balanced tree operations	any
O(n)	linear	one pass, counting, two pointers	10^8
O(n log n)	linearithmic	sorting, heap of n items	10^6 to 10^7
O(n²)	quadratic	all pairs, simple DP on two indices	10^4
O(n³)	cubic	Floyd-Warshall, matrix multiplication	500
O(2ⁿ)	exponential	all subsets	25
O(n!)	factorial	all permutations	11

COUNTING THE STEPS: DOUBLING N QUADRUPLES O(N&SUP2;) WORK

main.cpp

```
#include <chrono>
#include <iostream>
#include <vector>

long long ops = 0;
long long pairs_quadratic(const std::vector<int>& v, int target) {        // O(n^2)
    long long c = 0;
    for (std::size_t i = 0; i < v.size(); ++i)
        for (std::size_t j = i + 1; j < v.size(); ++j) { ++ops; if (v[i] + v[j] == target) ++c; }
    return c;
}
long long pairs_linear(const std::vector<int>& v, int target) {           // O(n): counting array
    std::vector<long long> seen(2 * target + 1, 0);
    long long c = 0;
    for (int x : v) {
        ++ops;
        if (x <= target) c += seen[target - x];
        if (x <= 2 * target) ++seen[x];
    }
    return c;
}

int main() {
    for (int n : {1000, 2000, 4000}) {
        std::vector<int> v(n);
        for (int i = 0; i < n; ++i) v[i] = (i * 37) % 1000;
        ops = 0;
        long long a = pairs_quadratic(v, 1000);
        long long q_ops = ops;
        ops = 0;
        long long b = pairs_linear(v, 1000);
        std::cout << "n = " << n << ": pairs " << a << " = " << b << ", steps O(n^2) " << q_ops << "
vs O(n) " << ops << '\n';
    }
    return 0;
}
```

OUTPUT

```
n = 1000: pairs 499 = 499, steps O(n^2) 499500 vs O(n) 1000
n = 2000: pairs 1997 = 1997, steps O(n^2) 1999000 vs O(n) 2000
n = 4000: pairs 7990 = 7990, steps O(n^2) 7998000 vs O(n) 4000
```

25.2.2 Sorting: merge sort, quick sort, heap sort, counting sort

Algorithm	Idea	Time (avg / worst)	Extra space	Stable
Merge sort	split in halves, sort each, merge	O(n log n) / O(n log n)	O(n)	yes
Quick sort	partition around a pivot, recurse on both sides	O(n log n) / O(n²)	O(log n)	no
Heap sort	build a max-heap, repeatedly move the max to the end	O(n log n) / O(n log n)	O(1)	no
Counting sort	count occurrences of each key in a small range	O(n + k) / O(n + k)	O(k)	yes
`std::sort`	introsort: quick sort, switching to heap sort and insertion sort	O(n log n) guaranteed	O(log n)	no

FOUR SORTING ALGORITHMS, CHECKED AGAINST STD::SORT

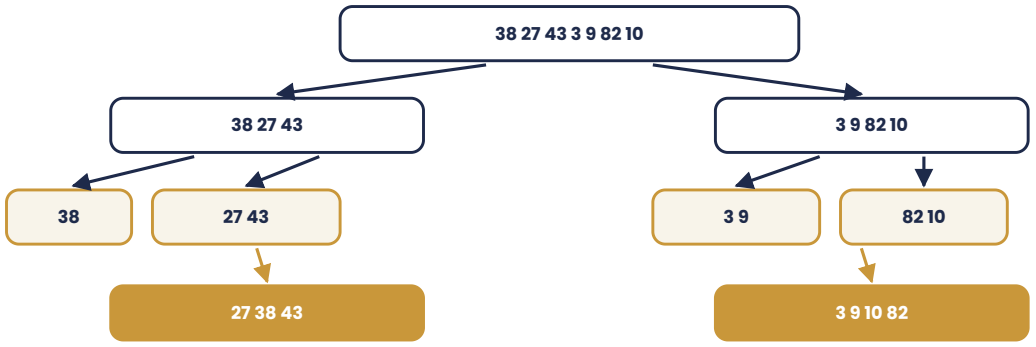
main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <random>
#include <vector>

void merge_sort(std::vector<int>& a, std::vector<int>& tmp, int lo, int hi) {   // sorts a[lo, hi)
    if (hi - lo < 2) return;
    int mid = lo + (hi - lo) / 2;
    merge_sort(a, tmp, lo, mid);
    merge_sort(a, tmp, mid, hi);
    int i = lo, j = mid, k = lo;
    while (i < mid && j < hi) tmp[k++] = (a[j] < a[i]) ? a[j++] : a[i++];       // ties take the left: stable
    while (i < mid) tmp[k++] = a[i++];
    while (j < hi) tmp[k++] = a[j++];
    std::copy(tmp.begin() + lo, tmp.begin() + hi, a.begin() + lo);
}

int partition(std::vector<int>& a, int lo, int hi) {          // Lomuto partition, pivot = a[hi]
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; ++j)
        if (a[j] < pivot) std::swap(a[i++], a[j]);
    std::swap(a[i], a[hi]);
    return i;
}
void quick_sort(std::vector<int>& a, int lo, int hi, std::mt19937& rng) {   // sorts a[lo, hi]
    if (lo >= hi) return;
    std::swap(a[std::uniform_int_distribution<int>(lo, hi)(rng)], a[hi]);   // random pivot: avoids O(n^2) on sorted input
    int p = partition(a, lo, hi);
    quick_sort(a, lo, p - 1, rng);
    quick_sort(a, p + 1, hi, rng);
}

void sift_down(std::vector<int>& a, int i, int n) {
    while (2 * i + 1 < n) {
        int c = 2 * i + 1;
        if (c + 1 < n && a[c + 1] > a[c]) ++c;
        if (a[i] >= a[c]) return;
        std::swap(a[i], a[c]);
        i = c;
    }
```

```
}
void heap_sort(std::vector<int>& a) {
    int n = static_cast<int>(a.size());
    for (int i = n / 2 - 1; i >= 0; --i) sift_down(a, i, n);   // build a max-heap
    for (int end = n - 1; end > 0; --end) {
        std::swap(a[0], a[end]);                                // largest to its final place
        sift_down(a, 0, end);
    }
}

void counting_sort(std::vector<int>& a, int max_value) {           // keys in [0, max_value]
    std::vector<int> count(max_value + 1, 0);
    for (int x : a) ++count[x];
    int k = 0;
    for (int v = 0; v <= max_value; ++v)
        while (count[v]-- > 0) a[k++] = v;
}

void show(const char* name, const std::vector<int>& v) {
    std::cout << name;
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';
}

int main() {
    const std::vector<int> data = {38, 27, 43, 3, 9, 82, 10, 27, 1, 64};
    auto a = data, tmp = data;
    merge_sort(a, tmp, 0, static_cast<int>(a.size()));
    show("merge sort:    ", a);
    auto b = data;
    std::mt19937 rng(42);
    quick_sort(b, 0, static_cast<int>(b.size()) - 1, rng);
    show("quick sort:    ", b);
    auto c = data;
    heap_sort(c);
    show("heap sort:     ", c);
    auto d = data;
    counting_sort(d, 100);
    show("counting sort: ", d);

    std::vector<int> big(200000);
    std::mt19937 r2(7);
    for (int& x : big) x = static_cast<int>(r2() % 1000000);
    auto e = big, f = big, g = big, t2 = big;
    merge_sort(e, t2, 0, static_cast<int>(e.size()));
    quick_sort(f, 0, static_cast<int>(f.size()) - 1, rng);
    heap_sort(g);
    std::sort(big.begin(), big.end());
    std::cout << "200000 random values: all four agree with std::sort: " << std::boolalpha << (e ==
big && f == big && g == big) << '\n';
    return 0;
}
```

OUTPUT

```
merge sort:    1 3 9 10 27 27 38 43 64 82
quick sort:    1 3 9 10 27 27 38 43 64 82
heap sort:     1 3 9 10 27 27 38 43 64 82
counting sort: 1 3 9 10 27 27 38 43 64 82
200000 random values: all four agree with std::sort: true
```



MERGE SORT: SPLIT DOWN TO SINGLE ELEMENTS, THEN MERGE SORTED HALVES

25.2.3 Searching and binary search on answers

Binary search finds a value in a sorted array in O(log n) by halving the search range. Write it carefully: use `lo + (hi - lo) / 2` to avoid overflow and keep a clear invariant. A powerful generalisation is **binary search on the answer**: when a question has the form "what is the smallest x such that condition(x) holds?" and the condition is **monotonic** (false, false, ..., true, true), binary-search over x and test the condition at the midpoint. Typical problems: minimum capacity to ship packages in D days, minimum time to finish tasks, maximum minimum distance (placing routers or cows), integer square roots.

BINARY SEARCH AND BINARY SEARCH ON THE ANSWER main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

int binary_search_index(const std::vector<int>& a, int target) {          // -1 if absent
    int lo = 0, hi = static_cast<int>(a.size()) - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}

// Smallest ship capacity that delivers all packages (in order) within `days` days.
bool can_ship(const std::vector<int>& w, int days, long long cap) {
    int used = 1;
    long long load = 0;
    for (int x : w) {
        if (load + x > cap) { ++used; load = 0; }
        load += x;
    }
    return used <= days;
}
long long min_capacity(const std::vector<int>& w, int days) {
    long long lo = *std::max_element(w.begin(), w.end());                 // must fit the heaviest
package
    long long hi = std::accumulate(w.begin(), w.end(), 0LL);              // one day carries
everything
    while (lo < hi) {                                                     // invariant: answer in [lo,
hi]
        long long mid = lo + (hi - lo) / 2;
        if (can_ship(w, days, mid)) hi = mid;                             // mid works: try smaller
        else lo = mid + 1;                                                // mid too small
    }
```

```
    return lo;
}

long long isqrt(long long n) {                                  // largest r with r*r <= n
    long long lo = 0, hi = 3'037'000'499;                       // sqrt(LLONG_MAX)
    while (lo < hi) {
        long long mid = lo + (hi - lo + 1) / 2;
        if (mid <= n / mid) lo = mid; else hi = mid - 1;        // mid*mid <= n, without
overflow
    }
    return lo;
}

int main() {
    std::vector<int> a = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    std::cout << "index of 23: " << binary_search_index(a, 23) << ", of 7: " << binary_search_index(a,
7) << '\n';
    std::vector<int> packages = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    for (int d : {5, 3, 1}) std::cout << "min capacity for " << d << " days: " <<
min_capacity(packages, d) << '\n';
    std::cout << "isqrt(1e18) = " << isqrt(1'000'000'000'000'000'000LL) << ", isqrt(99) = " <<
isqrt(99) << '\n';
    return 0;
}
```

OUTPUT

```
index of 23: 5, of 7: -1
min capacity for 5 days: 15
min capacity for 3 days: 21
min capacity for 1 days: 55
isqrt(1e18) = 1000000000, isqrt(99) = 9
```

25.2.4 Two pointers and sliding window

Two pointers move two indexes through an array, usually from both ends of a sorted array or both from the left, turning many O(n²) pair searches into O(n). A **sliding window** maintains a contiguous range [left, right]: extend the right end, and shrink from the left while the window violates the condition, keeping a running sum or a frequency map. Typical problems: pair with a given sum, remove duplicates in place, longest substring without repeating characters, smallest subarray with sum ≥ S, maximum sum of k consecutive elements.

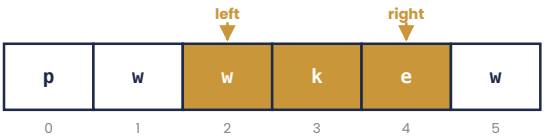
TWO POINTERS AND SLIDING WINDOWS — main.cpp

```
#include <algorithm>
#include <climits>
#include <iostream>
#include <string>
#include <unordered_map>
#include <vector>

std::pair<int, int> pair_with_sum(const std::vector<int>& sorted, int target) {   // two pointers from
both ends
    int i = 0, j = static_cast<int>(sorted.size()) - 1;
    while (i < j) {
        int s = sorted[i] + sorted[j];
        if (s == target) return {sorted[i], sorted[j]};
        if (s < target) ++i; else --j;
    }
    return {-1, -1};
}
```

```
int longest_unique_substring(const std::string& s) {                    // sliding window with last
positions
    std::unordered_map<char, int> last;
    int best = 0, left = 0;
    for (int right = 0; right < static_cast<int>(s.size()); ++right) {
        if (auto it = last.find(s[right]); it != last.end() && it->second >= left)
            left = it->second + 1;                                       // shrink past the repeated
character
        last[s[right]] = right;
        best = std::max(best, right - left + 1);
    }
    return best;
}

int min_subarray_len(const std::vector<int>& a, int target) {           // shortest window with sum >=
target
    int best = INT_MAX, left = 0, sum = 0;
    for (int right = 0; right < static_cast<int>(a.size()); ++right) {
        sum += a[right];
        while (sum >= target) {
            best = std::min(best, right - left + 1);
            sum -= a[left++];
        }
    }
    return best == INT_MAX ? 0 : best;
}

int max_sum_k(const std::vector<int>& a, int k) {                       // fixed-size window
    int sum = 0;
    for (int i = 0; i < k; ++i) sum += a[i];
    int best = sum;
    for (int i = k; i < static_cast<int>(a.size()); ++i) {
        sum += a[i] - a[i - k];                                          // slide: add new, drop old
        best = std::max(best, sum);
    }
    return best;
}

int main() {
    auto [x, y] = pair_with_sum({1, 3, 4, 6, 8, 11, 14}, 17);
    std::cout << "pair summing to 17: " << x << " + " << y << '\n';
    for (const char* s : {"abcabcbb", "bbbbb", "pwwkew", "learnbeingforward"})
        std::cout << "longest unique substring in " << s << ": " << longest_unique_substring(s) <<
'\n';
    std::cout << "shortest subarray with sum >= 7 in {2,3,1,2,4,3}: " << min_subarray_len({2, 3, 1, 2,
4, 3}, 7) << '\n';
    std::cout << "max sum of 3 consecutive in {4,2,10,1,3,8,2}: " << max_sum_k({4, 2, 10, 1, 3, 8, 2},
3) << '\n';
    return 0;
}
```

OUTPUT

```
pair summing to 17: 3 + 14
longest unique substring in abcabcbb: 3
longest unique substring in bbbbb: 1
longest unique substring in pwwkew: 3
longest unique substring in learnbeingforward: 10
shortest subarray with sum >= 7 in {2,3,1,2,4,3}: 2
max sum of 3 consecutive in {4,2,10,1,3,8,2}: 16
```



SLIDING WINDOW OVER "PWWKEW": THE WINDOW "WKE" IS THE LONGEST WITHOUT REPEATS

25.2.5 Recursion and backtracking

Backtracking builds a solution step by step and abandons (backtracks from) a partial solution as soon as it cannot lead to a valid answer. The pattern is always: choose, explore recursively, un-choose. It solves generation problems (all subsets, permutations, combinations) and constraint problems (N-Queens, Sudoku, word search, rat in a maze). Pruning invalid branches early is what makes it practical.

SUBSETS, COMBINATION SUM AND N-QUEENS main.cpp

```
#include <iostream>
#include <string>
#include <vector>

void subsets(const std::vector<int>& items, std::size_t i, std::vector<int>& cur,
std::vector<std::vector<int>>& out) {
    if (i == items.size()) { out.push_back(cur); return; }
    subsets(items, i + 1, cur, out);              // choice 1: skip items[i]
    cur.push_back(items[i]);                      // choice 2: take it
    subsets(items, i + 1, cur, out);
    cur.pop_back();                               // un-choose
}

void combination_sum(const std::vector<int>& c, int start, int remain, std::vector<int>& cur, int&
count) {
    if (remain == 0) {
        std::cout << "  ";
        for (int x : cur) std::cout << x << ' ';
        std::cout << '\n';
        ++count;
        return;
    }
    for (int i = start; i < static_cast<int>(c.size()) && c[i] <= remain; ++i) {   // sorted: prune
early
        cur.push_back(c[i]);
        combination_sum(c, i, remain - c[i], cur, count);     // i, not i + 1: reuse allowed
        cur.pop_back();
    }
}

bool safe(const std::vector<int>& queens, int row, int col) {
    for (int r = 0; r < row; ++r)
        if (queens[r] == col || queens[r] - col == r - row || queens[r] - col == row - r) return
false;
    return true;
}
int solve_queens(std::vector<int>& queens, int row, int n) {
    if (row == n) return 1;
    int total = 0;
    for (int col = 0; col < n; ++col)
        if (safe(queens, row, col)) {             // prune: never place an attacked queen
            queens[row] = col;
            total += solve_queens(queens, row + 1, n);
        }
    return total;
}
```

```
int main() {
    std::vector<std::vector<int>> all;
    std::vector<int> cur;
    subsets({1, 2, 3}, 0, cur, all);
    std::cout << all.size() << " subsets of {1,2,3}:";
    for (const auto& s : all) {
        std::cout << " {";
        for (std::size_t i = 0; i < s.size(); ++i) std::cout << (i ? "," : "") << s[i];
        std::cout << '}';
    }
    std::cout << "\ncoins {2,3,5} making 8:\n";
    int count = 0;
    combination_sum({2, 3, 5}, 0, 8, cur, count);
    std::cout << count << " ways\n";
    for (int n : {4, 6, 8}) {
        std::vector<int> q(n);
        std::cout << n << "-queens solutions: " << solve_queens(q, 0, n) << '\n';
    }
    return 0;
}
```

OUTPUT

```
8 subsets of {1,2,3}: {} {3} {2} {2,3} {1} {1,3} {1,2} {1,2,3}
coins {2,3,5} making 8:
  2 2 2 2
  2 3 3
  3 5
3 ways
4-queens solutions: 2
6-queens solutions: 4
8-queens solutions: 92
```

25.2.6 Greedy algorithms

A **greedy** algorithm makes the locally best choice at each step and never reconsiders. It is fast and simple, but correct only for problems with the **greedy-choice property**, which must be argued (usually by an exchange argument). Classic correct greedies: activity selection (pick the meeting that ends first), fractional knapsack (best value per kg first), Huffman coding, minimum spanning trees (Kruskal, Prim), and coin change for canonical coin systems such as Indian currency. Greedy fails for 0/1 knapsack and for arbitrary coin systems; those need dynamic programming.

WHEN GREEDY WORKS, AND WHEN IT DOES NOT

main.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

struct Meeting { std::string name; int start, end; };

int main() {
    // Activity selection: maximum number of non-overlapping meetings in one room
    std::vector<Meeting> m = {{"standup", 9, 10}, {"design", 9, 12}, {"lunch talk", 10, 11},
{"review", 11, 13}, {"1:1", 12, 13}, {"demo", 13, 15}, {"retro", 14, 16}};
    std::sort(m.begin(), m.end(), [](const Meeting& a, const Meeting& b) { return a.end < b.end; });
// earliest end first
    int free_at = 0;
    std::cout << "scheduled:";
    for (const auto& x : m)
        if (x.start >= free_at) { std::cout << ' ' << x.name << '(' << x.start << '-' << x.end << ')';
free_at = x.end; }
    std::cout << '\n';

    // Coin change with Indian denominations: greedy is optimal for this coin system
    std::vector<int> notes = {2000, 500, 200, 100, 50, 20, 10, 5, 2, 1};
    int amount = 3788, pieces = 0;
    std::cout << "3788 = ";
    for (int n : notes) {
        int k = amount / n;
        if (k) { std::cout << k << "x" << n << ' '; pieces += k; amount -= k * n; }
    }
    std::cout << "(" << pieces << " pieces)\n";

    // A coin system where greedy fails: coins {1, 3, 4}, amount 6
    std::vector<int> odd = {4, 3, 1};
    int amt = 6, greedy = 0;
    for (int c : odd) { greedy += amt / c; amt %= c; }
    std::cout << "coins {1,3,4}, amount 6: greedy uses " << greedy << " coins (4+1+1), optimum is 2
(3+3)\n";
    return 0;
}
```

OUTPUT

```
scheduled: standup(9-10) lunch talk(10-11) review(11-13) demo(13-15)
3788 = 1x2000 3x500 1x200 1x50 1x20 1x10 1x5 1x2 1x1 (11 pieces)
coins {1,3,4}, amount 6: greedy uses 3 coins (4+1+1), optimum is 2 (3+3)
```

25.2.7 Dynamic programming

Dynamic programming (DP) solves problems whose optimal solution is built from optimal solutions of overlapping subproblems. Define the **state** (what a subproblem is), the **transition** (how a state is computed from smaller ones), the **base cases** and the order of evaluation. Implement it **top-down** (recursion plus memoisation) or **bottom-up** (filling a table). Naive recursive Fibonacci is $O(2^n)$; with DP it is $O(n)$.

SIX CLASSIC DP PROBLEMS

main.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>
```

```
long long fib_memo(int n, std::vector<long long>& memo) {              // top-down
    if (n < 2) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo);
}

int coin_change(const std::vector<int>& coins, int amount) {           // fewest coins, bottom-up
    const int INF = 1'000'000;
    std::vector<int> dp(amount + 1, INF);                               // dp[a] = fewest coins for
amount a
    dp[0] = 0;
    for (int a = 1; a <= amount; ++a)
        for (int c : coins)
            if (c <= a) dp[a] = std::min(dp[a], dp[a - c] + 1);
    return dp[amount] >= INF ? -1 : dp[amount];
}

int knapsack(const std::vector<int>& w, const std::vector<int>& v, int cap) {   // 0/1 knapsack, 1-D
table
    std::vector<int> dp(cap + 1, 0);                                    // dp[c] = best value with
capacity c
    for (std::size_t i = 0; i < w.size(); ++i)
        for (int c = cap; c >= w[i]; --c)                               // backwards: each item used
once
            dp[c] = std::max(dp[c], dp[c - w[i]] + v[i]);
    return dp[cap];
}

int lcs(const std::string& a, const std::string& b) {                  // longest common subsequence
    std::vector<std::vector<int>> dp(a.size() + 1, std::vector<int>(b.size() + 1, 0));
    for (std::size_t i = 1; i <= a.size(); ++i)
        for (std::size_t j = 1; j <= b.size(); ++j)
            dp[i][j] = a[i - 1] == b[j - 1] ? dp[i - 1][j - 1] + 1 : std::max(dp[i - 1][j], dp[i][j -
1]);
    return dp[a.size()][b.size()];
}

int lis(const std::vector<int>& a) {                                   // longest increasing
subsequence, O(n log n)
    std::vector<int> tails;                                             // tails[k] = smallest tail of
an increasing run of length k+1
    for (int x : a) {
        auto it = std::lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) tails.push_back(x); else *it = x;
    }
    return static_cast<int>(tails.size());
}

int edit_distance(const std::string& a, const std::string& b) {
    std::vector<std::vector<int>> dp(a.size() + 1, std::vector<int>(b.size() + 1));
    for (std::size_t i = 0; i <= a.size(); ++i) dp[i][0] = static_cast<int>(i);
    for (std::size_t j = 0; j <= b.size(); ++j) dp[0][j] = static_cast<int>(j);
    for (std::size_t i = 1; i <= a.size(); ++i)
        for (std::size_t j = 1; j <= b.size(); ++j)
            dp[i][j] = std::min({dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] + (a[i - 1] !=
b[j - 1])});
    return dp[a.size()][b.size()];
}

int main() {
    std::vector<long long> memo(91, -1);
    std::cout << "fib(90) = " << fib_memo(90, memo) << '\n';
    std::cout << "coins {1,3,4} for 6: " << coin_change({1, 3, 4}, 6) << ", coins {2} for 3: " <<
coin_change({2}, 3) << '\n';
```

```
    std::cout << "knapsack (w 1,3,4,5 / v 1,4,5,7, cap 7): " << knapsack({1, 3, 4, 5}, {1, 4, 5, 7},
7) << '\n';
    std::cout << "LCS(\"ABCBDAB\", \"BDCABA\") = " << lcs("ABCBDAB", "BDCABA") << '\n';
    std::cout << "LIS of {10,9,2,5,3,7,101,18} = " << lis({10, 9, 2, 5, 3, 7, 101, 18}) << '\n';
    std::cout << "edit distance kitten -> sitting = " << edit_distance("kitten", "sitting") << '\n';
    return 0;
}
```

OUTPUT

```
fib(90) = 2880067194370816120
coins {1,3,4} for 6: 2, coins {2} for 3: -1
knapsack (w 1,3,4,5 / v 1,4,5,7, cap 7): 9
LCS("ABCBDAB", "BDCABA") = 4
LIS of {10,9,2,5,3,7,101,18} = 4
edit distance kitten -> sitting = 3
```

Problem	State	Transition
Coin change (fewest coins)	dp[a]: fewest coins for amount a	dp[a] = min over coins c of dp[a-c] + 1
0/1 knapsack	dp[c]: best value with capacity c (after i items)	dp[c] = max(dp[c], dp[c-w] + v), c descending
LCS	dp[i][j]: LCS of a[0..i), b[0..j)	match: dp[i-1][j-1] + 1, else max of left/up
Edit distance	dp[i][j]: edits to turn a[0..i) into b[0..j)	min(delete, insert, replace/keep)
LIS	tails[k]: smallest tail of an increasing subsequence of length k+1	binary search position, replace or append

25.2.8 Graph algorithms: BFS, DFS, Dijkstra, topological sort, union-find

BFS (breadth-first search, with a queue) visits vertices in order of distance and finds shortest paths in unweighted graphs. **DFS** (depth-first search, recursion or a stack) explores as deep as possible first; it detects cycles, finds connected components and underlies topological sort. **Dijkstra** finds shortest paths with non-negative weights using a min-priority queue, in O((V + E) log V). **Topological sort** orders the vertices of a DAG so every edge goes forwards (task scheduling, build order); Kahn's algorithm repeatedly removes vertices with no incoming edges. **Union-find** (disjoint set union) tracks which elements are connected, with near-O(1) operations thanks to path compression and union by size; it powers Kruskal's minimum spanning tree.

THE FIVE ESSENTIAL GRAPH ALGORITHMS

main.cpp

```
#include <algorithm>
#include <climits>
#include <functional>
#include <iostream>
#include <numeric>
#include <queue>
#include <string>
#include <vector>

std::vector<int> bfs_distances(const std::vector<std::vector<int>>& g, int src) {
    std::vector<int> dist(g.size(), -1);
    std::queue<int> q;
    dist[src] = 0;
    q.push(src);
    while (!q.empty()) {
```

```
        int u = q.front(); q.pop();
        for (int v : g[u])
            if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }
    }
    return dist;
}

void dfs(const std::vector<std::vector<int>>& g, int u, std::vector<bool>& seen, std::vector<int>&
order) {
    seen[u] = true;
    order.push_back(u);
    for (int v : g[u]) if (!seen[v]) dfs(g, v, seen, order);
}

std::vector<long long> dijkstra(const std::vector<std::vector<std::pair<int, int>>>& g, int src) {
    std::vector<long long> dist(g.size(), LLONG_MAX);
    using Item = std::pair<long long, int>;                                  // (distance, vertex)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;    // min-heap
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;                                           // stale entry
        for (auto [v, w] : g[u])
            if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }
    }
    return dist;
}

std::vector<int> topo_sort(int n, const std::vector<std::pair<int, int>>& edges) {   // Kahn's
algorithm
    std::vector<std::vector<int>> g(n);
    std::vector<int> indeg(n, 0), order;
    for (auto [u, v] : edges) { g[u].push_back(v); ++indeg[v]; }
    std::queue<int> q;
    for (int i = 0; i < n; ++i) if (indeg[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order;                                                            // shorter than n =>
there is a cycle
}

struct DSU {
    std::vector<int> parent, size;
    explicit DSU(int n) : parent(n), size(n, 1) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int x) { return parent[x] == x ? x : parent[x] = find(parent[x]); }   // path compression
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) std::swap(a, b);                                  // union by size
        parent[b] = a;
        size[a] += size[b];
        return true;
    }
};

int main() {
    std::vector<std::vector<int>> g = {{1, 2}, {0, 3, 4}, {0, 3}, {1, 2, 5}, {1}, {3}};
    auto d = bfs_distances(g, 0);
    std::cout << "BFS hops from 0:";
    for (std::size_t i = 0; i < d.size(); ++i) std::cout << ' ' << i << ':' << d[i];
```

```
    std::vector<bool> seen(g.size());
    std::vector<int> order;
    dfs(g, 0, seen, order);
    std::cout << "\nDFS order from 0:";
    for (int u : order) std::cout << ' ' << u;

    std::vector<std::string> city = {"Mysuru", "Mandya", "Bengaluru", "Tumakuru", "Hassan"};
    std::vector<std::vector<std::pair<int, int>>> roads(5);
    auto road = [&](int a, int b, int km) { roads[a].push_back({b, km}); roads[b].push_back({a, km});
};
    road(0, 1, 45); road(1, 2, 100); road(0, 4, 120); road(4, 3, 115); road(3, 2, 70); road(0, 2,
150);
    auto km = dijkstra(roads, 0);
    std::cout << "\nDijkstra from Mysuru:";
    for (int i = 1; i < 5; ++i) std::cout << ' ' << city[i] << '=' << km[i];

    std::vector<std::string> course = {"basics", "pointers", "OOP", "templates", "STL", "projects"};
    auto topo = topo_sort(6, {{0, 1}, {0, 2}, {1, 2}, {2, 3}, {3, 4}, {2, 4}, {4, 5}});
    std::cout << "\nstudy order:";
    for (int c : topo) std::cout << ' ' << course[c];

    DSU dsu(6);
    for (auto [a, b] : std::vector<std::pair<int, int>>{{0, 1}, {1, 2}, {3, 4}}) dsu.unite(a, b);
    std::cout << std::boolalpha << "\nunion-find: 0~2 " << (dsu.find(0) == dsu.find(2)) << ", 0~3 " <<
(dsu.find(0) == dsu.find(3))
              << ", uniting 2 and 0 again creates a cycle: " << !dsu.unite(2, 0) << '\n';
    return 0;
}
```

OUTPUT

```
BFS hops from 0: 0:0 1:1 2:1 3:2 4:2 5:3
DFS order from 0: 0 1 3 2 5 4
Dijkstra from Mysuru: Mandya=45 Bengaluru=145 Tumakuru=215 Hassan=120
study order: basics pointers OOP templates STL projects
union-find: 0~2 true, 0~3 false, uniting 2 and 0 again creates a cycle: true
```

Algorithm	Data structure	Complexity	Use for
BFS	queue	O(V + E)	shortest paths in unweighted graphs, levels, grids
DFS	recursion / stack	O(V + E)	components, cycles, paths, topological order
Dijkstra	min-priority queue	O((V + E) log V)	shortest paths with non-negative weights
Bellman-Ford	edge list	O(V E)	negative weights, negative-cycle detection
Topological sort (Kahn)	queue + in-degrees	O(V + E)	ordering tasks with dependencies
Union-find	parent array	about O(1) per operation	dynamic connectivity, Kruskal MST
Floyd-Warshall	V × V matrix	$O(V^3)$	all-pairs shortest paths, small V

25.3 Competitive programming with C++

25.3.1 Template setup and fast I/O

C++ is the most popular language in competitive programming: it is fast, and the STL provides almost every data structure needed. Contestants keep a small template. The two lines that matter most are for I/O speed: `std::ios::sync_with_stdio(false)` stops C++ streams from synchronising with C's stdio, and `std::cin.tie(nullptr)` stops `cin` from flushing `cout` before every read. Together they make `cin` / `cout` as fast as `scanf` / `printf`. Use `'\n'` instead of `std::endl`, which flushes every line.

A TYPICAL CONTEST TEMPLATE main.cpp

```cpp
#include <bits/stdc++.h>          // GCC-only: includes the whole standard library (contest use only)
using namespace std;
using ll = long long;
using vi = vector<int>;
#define all(x) (x).begin(), (x).end()

void solve() {
    int n;
    cin >> n;
    vi a(n);
    for (auto& x : a) cin >> x;
    sort(all(a));
    ll sum = accumulate(all(a), 0LL);
    cout << "min " << a.front() << " max " << a.back() << " sum " << sum << '\n';
}

int main() {
    ios::sync_with_stdio(false);      // fast I/O
    cin.tie(nullptr);
    int t = 1;
    cin >> t;                          // number of test cases
    while (t--) solve();
    return 0;
}
```

CONSOLE (user input in gold)

```
2
5
4 1 9 7 3
3
1000000000 1000000000 1000000000
min 1 max 9 sum 24
min 1000000000 max 1000000000 sum 3000000000
```

WATCH OUT

`#include <bits/stdc++.h>`, `using namespace std;` and macros are acceptable in throw-away contest code where typing speed matters, but they are bad practice in real projects: non-portable, slow to compile and namespace-polluting. Also note the `0LL` in `accumulate`: three values of 10^9 overflow an int.

25.3.2 Using the STL effectively under time limits

Need	Use	Watch out for
Sorting with custom order	`sort` with a lambda, `pair` / `tuple` comparison	comparator must be strict (<, not <=)

Need	Use	Watch out for
Frequency counting	`unordered_map` or a `vector` when keys are small	anti-hash tests: use `map` or a custom hash on Codeforces
Ordered set with lower_bound	`set`, `multiset`, `map`	`multiset::erase(value)` removes all copies: erase an iterator
Min/max of a changing set	`priority_queue` or `multiset`	priority_queue is a max-heap by default
Prefix sums	`partial_sum` into a `vector<long long>`	overflow: use long long
Large answers	`long long`, modulo 1e9+7	apply % after every multiplication
Binary search	`lower_bound` / `upper_bound` on sorted vectors	on a `set` use the member `s.lower_bound(x)`: the free function is O(n)

STL TECHNIQUES THAT SAVE CONTESTS

main.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    // Problem: for each query value, how many array elements lie in [L, R]?
    vector<int> a = {5, 1, 9, 3, 7, 3, 8};
    sort(a.begin(), a.end());                                   // O(n log n) once
    vector<pair<int, int>> queries = {{3, 7}, {0, 2}, {8, 100}};
    for (auto [l, r] : queries)
        cout << "[" << l << ", " << r << "]: " << upper_bound(a.begin(), a.end(), r) -
lower_bound(a.begin(), a.end(), l) << '\n';  // O(log n) each

    // Prefix sums: O(1) range-sum queries after O(n) preprocessing
    vector<long long> pre(a.size() + 1, 0);
    for (size_t i = 0; i < a.size(); ++i) pre[i + 1] = pre[i] + a[i];
    cout << "sum of sorted a[2..5] = " << pre[6] - pre[2] << '\n';

    // Modular arithmetic: 2^(10^18) mod 1e9+7 with fast exponentiation, O(log e)
    const long long MOD = 1'000'000'007;
    auto power = [&](long long b, long long e) {
        long long r = 1;
        b %= MOD;
        for (; e > 0; e >>= 1, b = b * b % MOD) if (e & 1) r = r * b % MOD;
        return r;
    };
    cout << "2^(10^18) mod 1e9+7 = " << power(2, 1'000'000'000'000'000'000LL) << '\n';

    multiset<int> ms = {4, 4, 4, 9};
    ms.erase(ms.find(4));                                       // removes ONE 4
    cout << "multiset after erasing one 4: size " << ms.size() << '\n';
    return 0;
}
```

OUTPUT

```
[3, 7]: 4
[0, 2]: 1
[8, 100]: 2
sum of sorted a[2..5] = 23
2^(10^18) mod 1e9+7 = 719476260
multiset after erasing one 4: size 3
```

25.3.3 Practice platforms: LeetCode, Codeforces, CodeChef, HackerRank

Platform	Best for	Format
LeetCode	interview preparation; problems tagged by company and topic	implement a function in a given class; easy / medium / hard
Codeforces	competitive programming, rating, the strongest community	timed contests (Div. 1-4) several times a week; full programs with stdin/stdout
CodeChef	contests popular in India (Starters, long challenges), beginner-friendly	full programs; star rating
HackerRank	language tracks and skill certificates; used by many companies for online tests	function or full program

Platform	Best for	Format
AtCoder, **CSES**	very well-designed problems / a classic problem set covering all standard techniques	full programs

A sustainable routine: solve a few problems daily at the edge of your ability, study editorials for problems you could not solve, re-solve them a week later without help, keep a notebook of techniques, and take part in live contests to practise under time pressure.

25.4 Capstone projects

A portfolio project shows what you can build. Choose one that interests you, scope it so you can finish it, and develop it like a professional: Git from day one, CMake, unit tests, sanitizers, a README with build instructions and screenshots. Each project below lists the C++ skills it demonstrates and the features to aim for.

25.4.1 Library or Bank Management System (OOP + file handling)

- **Skills:** classes and inheritance (Member, Student, Staff; Book, Journal), polymorphism, exceptions, file persistence (CSV or JSON), input validation.
- **Features:** add, search, issue and return books; due dates and fines with `<chrono>`; reports; save on exit and load on start; role-based menus (admin / member).
- **Stretch:** SQLite storage, a Qt GUI, unit tests for fine calculation.

THE CORE OF A LIBRARY SYSTEM

main.cpp

```
#include <chrono>
#include <iostream>
#include <map>
#include <stdexcept>
#include <string>

class Library {
public:
    void add_book(int id, std::string title) { books_[id] = {std::move(title), 0}; }
    void issue(int id, int member, int day) {
        auto& b = find(id);
        if (b.member) throw std::runtime_error("\"" + b.title + "\" is already issued");
        b.member = member;
        b.due_day = day + 14;                                  // 14-day loan
    }
    int return_book(int id, int day) {                         // returns the fine in rupees
        auto& b = find(id);
        if (!b.member) throw std::runtime_error("\"" + b.title + "\" was not issued");
        b.member = 0;
        return day > b.due_day ? (day - b.due_day) * 5 : 0;    // Rs 5 per late day
    }
private:
    struct Book { std::string title; int member; int due_day = 0; };
    Book& find(int id) {
        auto it = books_.find(id);
        if (it == books_.end()) throw std::out_of_range("no book with id " + std::to_string(id));
        return it->second;
    }
    std::map<int, Book> books_;
};

int main() {
```

```
    Library lib;
    lib.add_book(1, "The C++ Programming Language");
    lib.add_book(2, "Clean Code");
    lib.issue(1, 1001, 0);
    lib.issue(2, 1002, 3);
    for (auto [action, id, day] : {std::tuple{'i', 1, 5}, {'r', 1, 20}, {'r', 2, 10}, {'r', 2, 11},
{'i', 9, 12}}) {
        try {
            if (action == 'i') lib.issue(id, 1003, day);
            else {
                int fine = lib.return_book(id, day);                  // may throw: call it before
printing
                std::cout << "returned book " << id << " on day " << day << ", fine Rs " << fine <<
'\n';
            }
        } catch (const std::exception& e) {
            std::cout << "error: " << e.what() << '\n';
        }
    }
    return 0;
}
```

OUTPUT

```
error: "The C++ Programming Language" is already issued
returned book 1 on day 20, fine Rs 30
returned book 2 on day 10, fine Rs 0
error: "Clean Code" was not issued
error: no book with id 9
```

25.4.2 Inventory system with STL containers

- **Skills:** choosing containers (`unordered_map` by SKU, `map` or `multimap` by expiry date, `priority_queue` for reorder alerts), algorithms, lambdas, `std::format`.
- **Features:** stock in/out, low-stock alerts, search by name prefix, reports sorted by value, CSV import/export, an audit log.
- **Stretch:** undo with the Command pattern, multi-warehouse transfers, concurrency-safe updates.

25.4.3 Console game (Snake, Tic-Tac-Toe, Chess) or a game with SFML / Raylib

- **Skills:** game loop (input, update, render), 2-D arrays and state machines, `<random>`, `<chrono>` timing; for Chess: move generation, validation and a minimax AI.
- **Libraries: SFML** or **raylib** for windows, graphics, sound and input in a few lines of code; **ncurses** for terminal games.
- **Stretch:** unbeatable Tic-Tac-Toe with minimax, high-score persistence, levels.

THE HEART OF AN UNBEATABLE TIC-TAC-TOE: MINIMAX

main.cpp

```
#include <algorithm>
#include <array>
#include <iostream>

using Board = std::array<char, 9>;

char winner(const Board& b) {
    static const int lines[8][3] = {{0,1,2},{3,4,5},{6,7,8},{0,3,6},{1,4,7},{2,5,8},{0,4,8},{2,4,6}};
    for (auto& l : lines)
        if (b[l[0]] != ' ' && b[l[0]] == b[l[1]] && b[l[1]] == b[l[2]]) return b[l[0]];
    return std::count(b.begin(), b.end(), ' ') == 0 ? 'D' : 0;       // D = draw, 0 = still playing
}
```

```
int minimax(Board& b, bool x_turn, int& nodes) {                    // +1: X wins, -1: O wins, 0:
draw
    ++nodes;
    if (char w = winner(b)) return w == 'X' ? 1 : w == 'O' ? -1 : 0;
    int best = x_turn ? -2 : 2;
    for (int i = 0; i < 9; ++i)
        if (b[i] == ' ') {
            b[i] = x_turn ? 'X' : 'O';
            int score = minimax(b, !x_turn, nodes);
            b[i] = ' ';
            best = x_turn ? std::max(best, score) : std::min(best, score);
        }
    return best;
}

int main() {
    Board empty;
    empty.fill(' ');
    int nodes = 0;
    int v = minimax(empty, true, nodes);
    std::cout << "perfect play from the empty board: " << (v == 0 ? "draw" : v > 0 ? "X wins" : "O
wins")
              << " (" << nodes << " positions searched)\n";
    Board b = {'X','O','X',
               ' ','O',' ',
               ' ',' ',' '};                                        // X to move: must block the
column
    int best_move = -1, best_score = -2;
    for (int i = 0; i < 9; ++i)
        if (b[i] == ' ') {
            b[i] = 'X';
            int n = 0, s = minimax(b, false, n);
            b[i] = ' ';
            if (s > best_score) { best_score = s; best_move = i; }
        }
    std::cout << "X's best move: square " << best_move << " (blocks O's middle column)\n";
    return 0;
}
```

```
OUTPUT
perfect play from the empty board: draw (549946 positions searched)
X's best move: square 7 (blocks O's middle column)
```

25.4.4 Multithreaded file downloader or chat server

- **Skills:** threads and a thread pool, mutexes and condition variables, atomics for progress counters, sockets or an HTTP library (libcurl, cpr, Boost.Beast), error handling and timeouts.
- **Downloader features:** split a file into byte ranges downloaded in parallel, a progress bar, resume after interruption, checksum verification.
- **Chat server features:** many clients, broadcast and private messages, nicknames, a message history; with Boost.Asio (and coroutines) for asynchronous I/O.

25.4.5 Simple database engine or text editor (advanced)

- **Database:** a key-value store or a tiny SQL subset: a parser for `INSERT` / `SELECT ... WHERE`, fixed-size pages on disk with random access, a B-tree or hash index, a write-ahead log for crash safety, and benchmarks.

- **Text editor:** a terminal editor (in the spirit of kilo): raw terminal mode, a gap buffer or piece table, cursor movement, search, undo/redo, syntax highlighting.
- **Why it impresses:** data structures, file I/O, performance and careful design all in one project, and plenty to talk about in interviews.

Project	Difficulty	Main skills shown
Library / Bank system	beginner-intermediate	OOP, exceptions, files
Inventory system	intermediate	STL containers and algorithms
Console or SFML game	intermediate	game loop, state, AI (minimax)
Downloader / chat server	advanced	concurrency, networking
Database / text editor	advanced	data structures, systems programming, performance

25.5 Interview preparation

25.5.1 Core C++ topics checklist: OOP, virtual functions, memory, smart pointers, STL, move semantics

Topic	Be able to explain	Module
OOP basics	encapsulation, abstraction, inheritance, polymorphism; struct vs class; access specifiers	13, 14
Constructors	default, parameterised, copy, move; initialiser lists; explicit; delegating	13, 20
Rule of 3/5/0	when to write special members and why; copy-and-swap	13, 20
Virtual functions	vtable and vptr, dynamic dispatch, pure virtual, abstract classes, virtual destructor, override/final, object slicing	14
Memory	stack vs heap, new/delete, leaks, dangling pointers, RAII	10, 11
Smart pointers	unique_ptr vs shared_ptr vs weak_ptr, make_unique, reference cycles, custom deleters	20
References vs pointers	nullability, reseating, const references, lvalue/rvalue references	10, 20
Move semantics	std::move is a cast, noexcept moves, RVO, perfect forwarding	20
STL	container complexities, iterator invalidation, map vs unordered_map, vector growth	19
Templates	function/class templates, specialisation, SFINAE vs concepts	17, 22
const	const variables, member functions, pointers to const vs const pointers, mutable	3, 13
Exceptions	stack unwinding, exception safety guarantees, noexcept	16
Concurrency	data race, mutex, deadlock, condition variable, atomic	23
Static	static members, static locals, internal linkage	9, 13

25.5.2 Output-prediction and tricky questions

Interviewers like short programs whose output reveals whether you really understand a rule. Predict the output of each, then compare with the real output below it.

Q1-2: VIRTUAL CALLS DURING CONSTRUCTION main.cpp

```cpp
#include <iostream>

struct Base {
    Base() { std::cout << "Base() "; hello(); }
    virtual ~Base() { std::cout << "~Base "; }
    virtual void hello() { std::cout << "[Base::hello] "; }
};
struct Derived : Base {
    Derived() { std::cout << "Derived() "; hello(); }
    ~Derived() override { std::cout << "~Derived "; }
    void hello() override { std::cout << "[Derived::hello] "; }
};

int main() {
    Base* p = new Derived;              // Q1: which hello() runs inside Base's constructor?
    std::cout << '\n';
    delete p;                           // Q2: which destructors run?
    std::cout << '\n';
    return 0;
}
```

```
OUTPUT
Base() [Base::hello] Derived() [Derived::hello]
~Derived ~Base
```

Answer: while `Base`'s constructor runs, the object is still only a `Base`, so the virtual call goes to `Base::hello`. Destruction runs in reverse order, and because the destructor is virtual, deleting through a `Base*` correctly runs `~Derived` first. Without `virtual` on `~Base`, only `~Base` would run: undefined behaviour.

Q3-10: QUICK-FIRE OUTPUT PREDICTION

main.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

void f(int)    { std::cout << "f(int) "; }
void f(double) { std::cout << "f(double) "; }
void f(long)   { std::cout << "f(long) "; }

int main() {
    int a = 5;
    int b = a++ + 10;                          // Q3
    std::cout << "Q3: a=" << a << " b=" << b << '\n';

    unsigned u = 1;
    int n = -1;
    std::cout << "Q4: " << (n < u ? "-1 < 1" : "-1 >= 1 (!)") << '\n';   // signed/unsigned comparison

    std::cout << "Q5: ";
    f(3); f(3.0); f(3L); f('a');               // which overloads?
    std::cout << '\n';

    std::vector<int> v = {1, 2, 3};
    std::cout << "Q6: sizeof(v) = " << sizeof(v) << ", v.size() = " << v.size() << '\n';

    int arr[5] = {1, 2};
    std::cout << "Q7: arr[4] = " << arr[4] << ", sizeof(arr) = " << sizeof(arr) << '\n';

    std::string s = "abc";
    std::cout << "Q8: " << s + 'd' << ' ' << ('a' + 1) << ' ' << char('a' + 1) << '\n';

    std::cout << "Q9: " << 7 / 2 << ' ' << -7 / 2 << ' ' << -7 % 3 << ' ' << 7.0 / 2 << '\n';

    int x = 10;
    auto byval = [x] { return x; };
    auto byref = [&x] { return x; };
    x = 20;
    std::cout << "Q10: " << byval() << ' ' << byref() << '\n';
    return 0;
}
```

OUTPUT

```
Q3: a=6 b=15
Q4: -1 >= 1 (!)
Q5: f(int) f(double) f(long) f(int)
Q6: sizeof(v) = 24, v.size() = 3
Q7: arr[4] = 0, sizeof(arr) = 20
Q8: abcd 98 b
Q9: 3 -3 -1 3.5
Q10: 10 20
```

Q	Rule behind the answer
3	post-increment yields the old value (5), then a becomes 6
4	in `n < u`, -1 is converted to unsigned: 4294967295, which is not less than 1
5	`'a'` is a char, promoted to int (an exact promotion beats conversion to long or double)
6	`sizeof` a vector is its handle (3 pointers = 24 bytes), not its elements

Q	Rule behind the answer
7	a partially initialised array has its remaining elements zeroed
8	`'a' + 1` is an int (98); `string + char` appends
9	integer division truncates towards zero; the remainder takes the sign of the dividend
10	by-value captures copy at creation; by-reference captures see later changes

25.5.3 Coding rounds on DSA

Coding rounds test problem solving, not memorisation. Interviewers watch the process as much as the result. A structure that works:

1. **Clarify**: restate the problem, ask about input size, edge cases (empty, duplicates, negatives), and the expected output.
2. **Examples**: work through one normal and one edge case by hand.
3. **Brute force first**: state it and its complexity, even if you will not code it.
4. **Optimise**: look for the pattern (sorting, hashing, two pointers, sliding window, binary search, BFS/DFS, DP, heap) that removes repeated work. Say the target complexity.
5. **Code** cleanly, with meaningful names, talking as you go.
6. **Test**: trace your examples through the code, check edge cases, and state time and space complexity.

Pattern	Signals in the problem	Classic problems
Hashing	"find a pair/duplicate", counting	two sum, group anagrams, first unique character
Two pointers / sliding window	sorted array, contiguous subarray or substring	3-sum, longest substring without repeats, container with most water
Binary search	sorted input, "minimum maximum", monotonic condition	search in rotated array, koko eating bananas, median of two arrays
Stack	matching, "next greater", undo	valid parentheses, daily temperatures, largest rectangle in histogram
Heap	"top k", "k-th", merging sorted lists	kth largest, merge k lists, median of a stream
BFS / DFS	grids, networks, levels, connectivity	number of islands, rotting oranges, course schedule
DP	"number of ways", "minimum cost", choices with overlapping subproblems	climbing stairs, coin change, LIS, edit distance, house robber
Backtracking	"all combinations / permutations", constraints	subsets, permutations, N-Queens, word search

THREE INTERVIEW CLASSICS, SOLVED OPTIMALLY

main.cpp

```cpp
#include <iostream>
#include <stack>
#include <unordered_map>
#include <vector>
```

```cpp
// "Two Sum": indices of two numbers adding up to target. Brute force O(n^2); hashing O(n).
std::pair<int, int> two_sum(const std::vector<int>& a, int target) {
    std::unordered_map<int, int> index_of;                      // value -> index seen so far
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        if (auto it = index_of.find(target - a[i]); it != index_of.end()) return {it->second, i};
        index_of[a[i]] = i;
    }
    return {-1, -1};
}

// "Daily temperatures": days until a warmer day. Monotonic stack, O(n).
std::vector<int> days_until_warmer(const std::vector<int>& t) {
    std::vector<int> ans(t.size(), 0);
    std::stack<int> st;                                          // indexes with no warmer day yet
    for (int i = 0; i < static_cast<int>(t.size()); ++i) {
        while (!st.empty() && t[i] > t[st.top()]) { ans[st.top()] = i - st.top(); st.pop(); }
        st.push(i);
    }
    return ans;
}

// "Number of islands": count connected groups of '1' in a grid. DFS flood fill, O(rows * cols).
int islands(std::vector<std::vector<char>> g) {
    int rows = static_cast<int>(g.size()), cols = static_cast<int>(g[0].size()), count = 0;
    auto sink = [&](auto&& self, int r, int c) -> void {
        if (r < 0 || c < 0 || r >= rows || c >= cols || g[r][c] != '1') return;
        g[r][c] = '0';
        self(self, r + 1, c); self(self, r - 1, c); self(self, r, c + 1); self(self, r, c - 1);
    };
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c)
            if (g[r][c] == '1') { ++count; sink(sink, r, c); }
    return count;
}

int main() {
    auto [i, j] = two_sum({2, 7, 11, 15}, 26);
    std::cout << "two sum 26: indices " << i << " and " << j << '\n';
    std::cout << "days until warmer:";
    for (int d : days_until_warmer({73, 74, 75, 71, 69, 72, 76, 73})) std::cout << ' ' << d;
    std::cout << "\nislands: " << islands({{'1','1','0','0','0'}, {'1','1','0','0','1'},
{'0','0','1','0','1'}, {'0','0','0','1','1'}}) << '\n';
    return 0;
}
```

```
OUTPUT
two sum 26: indices 2 and 3
days until warmer: 1 1 4 2 1 1 0 0
islands: 3
```

25.5.4 Explaining your project

"Tell me about your project" is asked in nearly every interview. Prepare a two-minute version and be ready to go deep on any part. A reliable structure is **STAR**: Situation, Task, Action, Result. Interviewers want to hear about *your* decisions and what you learned, not a list of features.

- **What and why:** the problem it solves and who would use it, in one or two sentences.
- **Architecture:** the main components and how data flows; be ready to draw it.
- **Key decisions:** why these containers, why a thread pool, why CMake and GoogleTest; what alternatives you rejected.

- **The hardest bug or challenge:** how you found it (debugger, sanitizer, profiler) and fixed it.
- **Numbers:** lines of code, test coverage, performance before and after an optimisation.
- **What you would improve** with more time: this shows judgement and honesty.

A SAMPLE TWO-MINUTE ANSWER

text

```
"I built a multithreaded file downloader in C++20. Downloads of large files were slow over a
single connection, so I split each file into byte ranges fetched in parallel by a thread pool
I wrote with std::jthread, a mutex-protected task queue and a condition variable. Progress is
tracked with std::atomic counters, and an interrupted download resumes from a JSON state file.
The trickiest bug was a race between the progress bar and the workers that ThreadSanitizer
found; I fixed it by publishing progress through atomics instead of a shared struct. On a
100 MB file, 8 connections cut the time from 41 s to 9 s. It is built with CMake and has 34
GoogleTest tests. Next I would add bandwidth limiting and HTTP/2 support."
```

25.6 Next steps

C++ opens doors to some of the most demanding and rewarding areas of software. Each path builds on the foundations of this course.

25.6.1 Game development (Unreal Engine)

Unreal Engine is written in C++ and uses it for gameplay programming alongside the Blueprint visual scripting system. Its C++ has its own conventions: `UCLASS` / `UPROPERTY` reflection macros, garbage-collected `UObject`s, and its own containers (`TArray`, `TMap`). Learn the engine's gameplay framework (Actors, Components, GameMode), and the maths of 3-D graphics (vectors, matrices, quaternions). Custom engines and tools in the industry also use C++, with SFML, raylib, SDL and OpenGL/Vulkan as stepping stones.

25.6.2 Systems and embedded programming

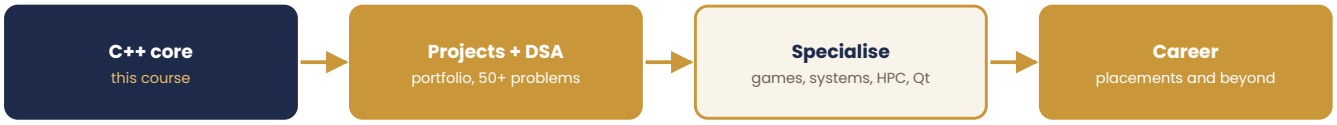
Operating systems, browsers, databases, compilers and embedded firmware are written in C and C++. For embedded work (Arduino, STM32, ESP32, automotive ECUs): limited memory, no exceptions or heap in many projects, `constexpr` and templates for zero-cost abstractions, registers, interrupts and real-time constraints; standards such as MISRA C++ and AUTOSAR. For systems work: Linux system calls, networking, memory management and concurrency in depth.

25.6.3 High-performance and financial computing

Scientific computing, machine-learning frameworks (the cores of PyTorch and TensorFlow), game physics and high-frequency trading rely on C++ for speed. Topics to study: cache-aware data structures, SIMD (`<experimental/simd>`, intrinsics), multithreading and lock-free structures, GPU programming (CUDA, SYCL), profiling at the hardware level, and low-latency techniques (avoiding allocations and system calls on hot paths). Quantitative finance roles add numerical methods and probability.

25.6.4 Qt for desktop applications

Qt is a cross-platform C++ framework for desktop, embedded and mobile GUIs, used in products from automotive dashboards to KDE and Autodesk tools. It provides widgets and the declarative QML language, the signals-and-slots mechanism for events, networking, databases and threads, with Qt Creator as the IDE and CMake support. A small Qt front end on one of your capstone projects is an excellent way to start.



YOUR PATH FROM HERE

Implement linked lists, stacks, queues, trees, heaps, hash tables, graphs and tries once to understand them, then use the STL. Master the core techniques: complexity analysis, sorting, binary search (including on the answer), two pointers and sliding windows, backtracking, greedy, dynamic programming and graph algorithms. Practise regularly on contest and interview platforms with fast I/O and the right containers. Build a finished, tested capstone project you can explain clearly, and revise the core C++ checklist before interviews.

PRACTICE CHECKPOINT

Build **one capstone project with CMake, unit tests and a Git repository**, and solve **50 DSA problems in C++** across the topics above.

Use this plan to complete the final checkpoint:

1. **Week 1, set-up:** pick a project from 25.4; create the Git repository with a `.gitignore`, the CMake layout of Module 24 (library + app + tests), a README, and a first passing test.
2. **Weeks 2-3, features:** implement the core features in small commits; write tests alongside each feature; run the tests under ASan/UBSan (and TSan for threaded projects) before every merge.
3. **Week 4, polish:** clang-format and clang-tidy clean, error handling and input validation, a demo GIF or screenshots in the README, and a short "design decisions" section you can use in interviews.
4. **Throughout, DSA:** about 50 problems spread across the topics, for example: arrays and hashing 8, two pointers and sliding window 6, binary search 5, stack and queue 5, linked list 4, trees and BST 6, heap 3, graphs 5, backtracking 3, DP 5. Record each problem's pattern and complexity in a log.

Week	Project milestone	DSA target
1	repository, CMake, first test	12 problems: arrays, hashing, two pointers
2	core features + tests	13 problems: binary search, stack, queue, linked list
3	remaining features, sanitizer-clean	13 problems: trees, heaps, graphs
4	polish, README, demo, interview notes	12 problems: backtracking, DP, mixed revision